

Team Androids - Grand Finale: Concepts and Q&A Study Guide

Amazon ML Challenge 2026 - Grand Finale, 7 October 2026

Presenter: Pardheev

The full companion to the Essentials PDF (QA_ESSENTIALS.pdf): 99 concepts explained from zero, a 20-stop walk through our pipeline, 258 judge questions with spoken answers, and a cheat sheet. Final public F0.5: 0.991300 (say "0.9913").

Contents

How to use this guide	3
The 2-day plan: Monday 5 and Tuesday 6 October	4
Before the finale: to do	6
The 30-second and 2-minute pitch	7
The 40 most likely questions	8
Concept map: which concept is used where	10
Part A - Learn the concepts	11
A0 - Machine learning from zero (the prerequisites for everything else)	12
A1 - Concepts: foundations	47
A2 - Concepts: the models	91
A3 - Concepts: training, validation and selection	134
A4 - Our pipeline, step by step, following one record	184
Part B - Q&A bank	221
1. Problem and data (15)	222
2. Metric and the precision-first choice (13)	228
3. Normalisation and multilingual text (15)	233
4. Blocking and candidate generation (13)	239
5. Scale, efficiency and cost (11)	244
6. The CatBoost ranker and the shortlist (12)	248
7. The readers (cross-encoders) (15)	253
8. Features and the "what changed" idea (14)	259
9. Decision models and thresholds (9)	264
10. Second-chance searches and the consistency check (10)	267
11. France with zero labels (11)	271
12. Training strategy (10)	275
13. Validation, model selection and the leaderboard (12)	280
14. Edge cases (11)	284
15. Results and error analysis (9)	288
16. Limitations and what we would do differently (13)	292
17. Why not X? Alternatives we did not use (14)	297
18. Taking it to production at Amazon scale (13)	302
19. Reproducibility, engineering and rules (12)	307
20. Rapid-fire ML fundamentals a scientist may probe (15)	312
21. Curveballs and reflection (11)	317
Part C - Cheat sheet	321
(a) Numbers table	322
(b) A-Z glossary	327
(c) Internal code names and their plain names	340
(d) The honest-limitations answers in one place	342
(e) If you don't know: how to answer gracefully	343

How to use this guide

Start with the Essentials. Before you open this guide, read the separate **Essentials PDF (QA_ESSENTIALS.pdf)**: the pitch, the numbers you must know by heart and the short answers to the questions you are most likely to get. Then use this full guide to go deeper: it explains every idea from zero, and it gives every answer with its follow-up and the numbers behind it. This guide is written for you, Pardheev, presenting Team Androids' solution at the Amazon ML Challenge 2026 Grand Finale on Wednesday 7 October 2026 (a 10-minute talk, then 5 minutes of questions from senior Amazon scientists). It assumes you start as a beginner and takes you to the level where you can answer a senior applied scientist calmly and exactly. Our final public F0.5 is 0.991300: say "0.9913".

What is in it.

- **Part A - Learn the concepts** (99 concepts in four chapters, plus a 20-stop walk through our pipeline). Every concept is taught in the same order: **In one line**, then a **Simple explanation** (an everyday picture), a **Worked example** you redo with a pen, the **Technical explanation** (the level a senior scientist expects), then **How we used it**, **Expert nuance**, **Remember** and **Judge might ask**. A0 starts from zero; A4 follows one record through the whole pipeline. The [Concept map](#) shows which concept is used at which pipeline stage.
- **Part B - The Q&A bank** (258 questions in 21 categories, numbered Q1 to Q258). Each question has *Why they ask* (what the judge is really testing), **Say** (the spoken answer, 20-35 seconds), **If they push** (the follow-up, only if asked again) and **Numbers** (with sources). Start with [the 40 most likely questions](#).
- **Part C - Cheat sheet.** The numbers table, an A-Z glossary of every technical term with a link to where it is taught, the internal code names and their plain names, the honest-limitations answers in one place, and what to say when you do not know.

How to read the badges and labels.

Badge or label	Meaning
★ / ★★ / ★★★	difficulty of a question: basic / probing / hard or adversarial
LIKELY	a question we expect the judges to ask
Doc 3, Doc B.4 ...	a section or appendix of our submitted Documentation: the primary truth, safe to quote
CodeREADME, Evidence, Code: ..., (code)	the code package: true, but not in the Documentation; use only if asked
background	correct general knowledge that is not in our Documentation; use only if asked
derived	plain arithmetic on documented numbers, shown so you can redo it
toy / illustrative	invented numbers that only teach the arithmetic: never quote them as results
QA n	answer n of the talk's vetted Q&A prep sheet (the short version of Part B)
Qn	question n of Part B of this guide (click it)
A1.5, A4.17 ...	concept 5 of chapter A1, section 17 of the pipeline walk-through (click it)

The answer formula. Direct answer first, then one number, then stop (2-4 sentences). If a question touches a limitation, say the limitation plainly first and then the mitigation. Use the plain names of the talk: business, incoming record, shortlist, quick rule-based filter, CatBoost ranker, the four readers (MiniLM reader, E5 reader, French MiniLM reader, French E5 reader), "what changed" features, LightGBM decision model per country, confident + clear-winner rule, second-chance searches, per-business consistency check, France rules computed by code.

Six things never to do on stage.

1. Never use internal code names or rule letters; they are printed only in the [code-name table](#).
2. If a judge asks how the decision models reject near-identical look-alike businesses, use exactly the wording of [Q115](#) and add nothing; never raise the topic yourself.
3. Never invent a number, experiment, ablation or timing. If a number is missing, answer qualitatively and say what you would measure.
4. Talk only about the final submitted approach; "what didn't work" uses only the items our Documentation reports ([Q189](#)).
5. Never call the held-out scores an end-to-end test, never call agreement with French pseudo-labels evidence of French accuracy, and never say the 0.4 lead "was never tuned" (say: one fixed lead for every country; the threshold was tuned on fold 8).
6. Never present our AI-assisted development as a rule problem; if asked, answer briefly and factually ([Q229](#)).

The 2-day plan: Monday 5 and Tuesday 6 October

Day 1 builds understanding; Day 2 turns it into fast, exact spoken answers. Each block ends with a **self-test checkpoint**: close the guide, answer aloud or on paper, then check. If you miss more than one item of a checkpoint, re-read the linked concepts before moving on. Times are a guide; take a 10-minute break every hour.

Reading rule for a beginner. On the first pass of a concept, read only **In one line**, **Simple explanation**, **Worked example** and **How we used it**, and redo every calculation. Read **Technical explanation** and **Expert nuance** on a second pass, for the concepts marked *must-know* below. "Skim" means: read the *In one line* and *Remember* blocks only.

Day 1 - Monday 5 October: learn

Time	Block	What to read	Self-test checkpoint
09:00-10:15	0. The Essentials	The Essentials PDF (QA_ESSENTIALS.pdf), all of it; then the 30-second and 2-minute pitch of this guide	CP0. Say the 30-second pitch aloud in at most 35 seconds. Then say: the final public score (0.991300, "0.9913"), the number of businesses and incoming records (1,732,544 and 9,969,589) and the threshold and lead (0.75, or 0.82 in France, and 0.4).
10:30-12:45	1. Machine learning from zero	Part A0 , all 18 concepts. Must-know (second pass too): A0.4 , A0.5 , A0.7 , A0.8 , A0.10 , A0.12 , A0.13 , A0.17	CP1. (a) A business has 4 true records; you link 3 of them and nothing wrong. Compute its F0.5. (b) Same business, you link all 4 plus 1 wrong record. (c) Why is a probability score not yet a decision? (d) Why do we need a validation fold that the model never trained on? <i>Answers: 0.9375; about 0.833; a decision needs a threshold and a rule; to measure what the model does on unseen data.</i>
13:30-16:00	2. Our pipeline, end to end	Part A4 , all of it, slowly; the concepts it uses are linked and taught in A1-A3, so for now read only their <i>In one line</i> . Then retell the three real examples (A4.17) aloud	CP2. (a) Draw the pipeline from memory with its numbers: about 130 candidates per business (blocking study), 21.71 after the filter, 6.75 in the shortlist, 7.10 in the candidate file; 5,856,936 links. (b) Retell the French example: why was a 0.999837 link vetoed? (c) Read A4.19 , the traps, and say which three you would most likely fall into.
16:15-18:45	3. Foundations	Part A1 . Must-know: A1.1 , A1.4 , A1.5 , A1.6 , A1.7 , A1.8 , A1.10 , A1.16 , A1.19 , A1.20 , A1.21 , A1.22 , A1.23 , A1.25 , A1.26 . Skim the rest	CP3. (a) What is blocking, and what are our numbers (6.7 trillion same-country pairs, about 7 candidates per business in the candidate file)? (b) Why does a business with no true records make us precision first? (c) What does the quick rule-based filter keep, and what did it cost (0.990117 to 0.990086)? (d) Why is our index score not BM25?

Time	Block	What to read	Self-test checkpoint
Evening, 20 min	Review	Re-read only the Remember blocks of today's must-know concepts	Explain F0.5, blocking, the reader and the decision rule to someone at home in plain words.

Day 2 - Tuesday 6 October: deepen and drill

Time	Block	What to read	Self-test checkpoint
09:00-11:00	4. The models	Part A2. Must-know: A2.2, A2.4, A2.5, A2.7, A2.8, A2.9, A2.11, A2.13, A2.14, A2.17, A2.19, A2.21, A2.22, A2.23, A2.24, A2.25, A2.26. Skim the rest	CP4. (a) Cross-encoder versus bi-encoder in two sentences. (b) A logit of 0 is what probability? A logit of 7.5? (c) Why is our CatBoost ranker a pointwise classifier and not a ranking loss? (d) What is stacking, and why do the ranker and readers train on folds 0-5 but the decision models on 6-9? <i>Answers: (b) 0.5; about 0.9994.</i>
11:15-13:15	5. Training, validation and selection	Part A3. Must-know: A3.1 to A3.12, A3.15, A3.16, A3.20, A3.21, A3.23, A3.24, A3.25, A3.26. Skim the rest	CP5. (a) Why are folds assigned by business, not by record? (b) What did fold 8 choose, and why is it therefore not untouched? (c) Name our four main limitations and one mitigation each (Part C (d)). (d) Why is 0.999206 agreement with French pseudo-labels not evidence of French accuracy?
14:00-16:00	6. The 40 most likely questions	The 40 most likely questions. For each: read only the question, answer aloud in 20-30 seconds, then compare with Say . Tick the ones you got right; mark the others	CP6. A second round on the marked ones only. Then read If they push for every ★★★ question in the list and say it aloud once.
16:15-18:30	7. The categories, then the cheat sheet	Part B by priority: Limitations, Validation and the leaderboard, France, Alternatives, Production, Metric, Blocking, Decision models; then the rest, Say lines only. Finish with Part C: the numbers, the honest-limitations answers and the "if you don't know" phrases	CP7. Mock Q&A for 5 minutes: a teammate picks ten random Q numbers and you answer without looking. Score yourself: direct answer first? one exact number? stopped in time? limitation first? Then say from memory: 0.991300; 1,732,544 and 9,969,589; 6,724,569,566,212; 99.3694 % and 98.5068 %; 0.75, 0.82 and 0.4; 0.991212 / 0.990046; 887,630 to 867,559; 11.5 hours on one 16 GB GPU.
Evening, 30 min	Final rehearsal	The talk once, then five LIKELY questions at random	Stop when it feels routine, not when it feels perfect.

Morning of 7 October (30 minutes). Only the Essentials PDF, Part C and the 30-second pitch. Do not start new material.

Before the finale: to do

Three things to settle before you walk on stage. Tick them off on Tuesday evening at the latest.

- ☐ **Agree one accurate sentence on the team's split of work.** Agree it with Monika and Hemachand and write it into the answer to [Q228](#) ("Who did what in your team?"), which shows **TO FILL IN** where the sentence goes. Our documents do not record roles, so do not improvise one on stage.
- ☐ **Know what is "background" and what is Documentation.** Items marked "background" or "(code)", and facts whose source is Code, CodeREADME or Evidence, are true but are not in our submitted Documentation. Use them only if a judge asks directly; on any conflict the Documentation wins.
- ☐ **Never say a record id aloud.** The worked examples print ids such as "S1-..." (a business) or "S2-..." (an incoming record) only so that the records can be found in our files. On stage say "a US business", "an incoming record from Nantes" and so on.

The 30-second and 2-minute pitch

Taken word for word from the end of the pipeline walk-through ([A4.20](#)). Learn the 30-second version by heart; it is the answer to "so what did you build?". The 2-minute version is for a judge who asks you to walk through the whole pipeline.

One sentence (about 20 seconds). "Per country, a cheap index and two pruning steps cut 6.7 trillion possible pairs to about seven candidates per business, small language-model readers and a per-country decision model score them, and we link a record only to a confident, clear winner, with second-chance searches and French rules on top."

30 seconds (about 80 words). "Per country, we clean the text and a C++ inverted index pulls up businesses that share rare words, letter pieces or sounds, about 130 per business. A quick rule with no model cuts that to 21.7, and a CatBoost ranker keeps at most four per record. Cross-encoder readers score each pair, 'what changed' features describe the differences, and a LightGBM decision model links only a confident, clear winner. Second-chance searches, a consistency check and France rules finish it: public F0.5 0.9913."

2 minutes (about 300 words). "We link about ten million incoming records from the US, India and France to 1.7 million businesses, each record to one business or to none. The metric is F0.5 per business, and a business with no true records scores a point only if we link nothing, so we are precision first. The pipeline runs per country and goes from cheap to expensive.

First we clean the text: lower case, abbreviations unified, Indian-script words transliterated with a word list learned from training matches, and French address and legal-form rules. A C++ inverted index then finds businesses that share rare words, letter pieces or sounds, weighted by IDF, with separate lists for name and address so one-sided matches survive. That gives about 130 candidates per business. A quick rule-based filter with no model cuts that to 21.7, at a held-out cost of 0.00003 F0.5, and a CatBoost ranker keeps at most four per record: 6.75 per business.

Cross-encoder readers, fine-tuned multilingual MiniLM and E5, read each record and candidate together. We add 'what changed' features: which numbers changed, which words were added, dropped or replaced, how rare and how close in meaning, and whether the legal form changed. A LightGBM decision model per country turns about two hundred features into a probability, and we link only a confident, clear winner: at least 0.75, or 0.82 in France, and 0.4 ahead of the runner-up.

Then targeted fixes: two second-chance searches for unlinked records, by transliterated name in India and by rare address keys, adding 9,759 and 3,137 links; a per-business consistency check; and for France, which has no labels, self-trained French readers and rules computed by code.

The final file scores 0.9913 on the public leaderboard, and close versions of our decision models score between 0.990 and 0.992 on held-out folds."

If a judge then asks for limitations, add: "We had no untouched end-to-end validation of the final file, France has no labels and its rules were designed after looking at unlabelled test records, and we kept the French E5 reader although it failed two of its four written checks."

The 40 most likely questions

Part B tags 83 of its 258 questions LIKELY. These are the 40 most central ones: the questions about the task, the metric and the decision rule, blocking, the readers, the "what changed" features, France, how we trained and validated, the leaderboard, the limitations, the alternatives and production. Drill them first (Day 2, block 6 of the plan): read the question, answer aloud in 20-30 seconds, then click through and compare with **Say**. All are tagged LIKELY in Part B.

#	Question	Level	Category
1	Q1 What exactly is the task, in one sentence?	★	1. Problem and data
2	Q3 What makes this problem hard?	★	1. Problem and data
3	Q16 How is the competition metric computed?	★	2. Metric and the precision-first choice
4	Q18 Why does the empty-row rule matter so much?	★★	2. Metric and the precision-first choice
5	Q21 How did you choose the decision threshold, and why not tune for AUC?	★★	2. Metric and the precision-first choice
6	Q31 How do you handle names written in Indian scripts?	★	3. Normalisation and multilingual text
7	Q45 Walk me through your candidate funnel.	★	4. Blocking and candidate generation
8	Q53 How do you know blocking does not lose true matches?	★	4. Blocking and candidate generation
9	Q72 Why at most four candidates per incoming record?	★★	6. The CatBoost ranker and the shortlist
10	Q88 If the readers are that good, why do you need a decision model?	★★	7. The readers (cross-encoders)
11	Q95 What are the "what changed" features, in plain words?	★	8. Features and the "what changed" idea
12	Q96 Why describe differences instead of just measuring similarity?	★★	8. Features and the "what changed" idea
13	Q102 Did the "what changed" features actually help? Do you have an ablation?	★★★	8. Features and the "what changed" idea
14	Q110 Explain the "confident and clear winner" rule. Why do you need both conditions?	★★	9. Decision models and thresholds
15	Q111 How did you choose the 0.4 lead?	★★★	9. Decision models and thresholds
16	Q113 Why are the decision models trained on folds 6-9, and how do fit, early stopping and refit work?	★★	9. Decision models and thresholds
17	Q118 What are the second-chance searches, and why do you need them?	★	10. Second-chance searches and the consistency check
18	Q125 What is the per-business consistency check?	★★	10. Second-chance searches and the consistency check
19	Q128 France has no labels. How did you build it at all?	★	11. France with zero labels
20	Q131 How did you set the French threshold of 0.82 without labels?	★★	11. France with zero labels
21	Q139 Which data trained which model?	★	12. Training strategy
22	Q140 Why do the ranker and readers train on different folds from the decision models?	★★	12. Training strategy
23	Q141 Why assign folds by business rather than by record?	★	12. Training strategy
24	Q149 How did you validate your models?	★	13. Validation, model selection and the leaderboard
25	Q152 Which decisions used held-out folds, and which used the public leaderboard?	★★	13. Validation, model selection and the leaderboard

#	Question	Level	Category
26	Q153 How did you avoid overfitting the public leaderboard?	★★★	13. Validation, model selection and the leaderboard
27	Q154 Will 0.9913 hold on the private leaderboard?	★★★	13. Validation, model selection and the leaderboard
28	Q162 What do you do with a country you have never seen?	★	14. Edge cases
29	Q172 What is your final result?	★	15. Results and error analysis
30	Q174 What are your most common wrong links?	★★	15. Results and error analysis
31	Q181 What is the biggest limitation of your work?	★★	16. Limitations and what we would do differently
32	Q182 The France rules were designed by looking at the test set. Isn't that overfitting?	★★★	16. Limitations and what we would do differently
33	Q183 Why did you keep the French E5 reader when it failed two of its checks?	★★★	16. Limitations and what we would do differently
34	Q192 What would you do differently?	★★	16. Limitations and what we would do differently
35	Q194 Why not simply ask a large language model whether two records match?	★★	17. Why not X? Alternatives we did not use
36	Q195 Why an inverted index for blocking, and not embedding (vector) search with an approximate-nearest-neighbour index?	★★	17. Why not X? Alternatives we did not use
37	Q200 Why separate models per country rather than one global model?	★	17. Why not X? Alternatives we did not use
38	Q208 Amazon has billions of records. What would the deployed system look like?	★★	18. Taking it to production at Amazon scale
39	Q210 What is the latency for a single incoming record?	★★	18. Taking it to production at Amazon scale
40	Q226 Did you use the test data in any way?	★★★	19. Reproducibility, engineering and rules

Concept map: which concept is used where

Read across a row: the pipeline stage (taught step by step in Part A4), what happens there in plain words, and the concepts it uses (click to jump to where each is taught). The last two rows are not stages but run through the whole pipeline.

Pipeline stage	What happens, in plain words	Concepts it uses (where taught)
Stop 0: before the record arrives	The task, the score, and which folds trained which model	A1.1 entity resolution; A1.2 many-to-one links; A0.4 precision, recall, F0.5; A1.4 F-beta; A1.5 per-business macro F0.5; A3.4 group k-fold; A3.5 hashing ids; A3.6 layer-wise folds
Stop 1: normalisation	Clean names and addresses so that the same thing is written the same way	A1.8 text normalisation; A1.9 Unicode and accents; A1.10 transliteration; A1.11 address parsing; A0.15 tokens
Stop 2: index lookup	Find businesses that share rare words, letter pieces or sounds	A1.19 blocking; A1.20 inverted index; A1.21 multi-channel retrieval; A1.16 IDF; A1.12 character n-grams; A1.18 sound key; A1.14 LCS ratio; A1.17 set overlaps; A1.26 bounded work
Stop 3: quick rule-based filter	Drop clearly weaker candidates with fixed rules, no model	A1.22 the quick filter; A1.23 recall ceiling and funnel
Stop 4: CatBoost ranker and shortlist	Order the candidates and keep at most four per record	A0.12 trees; A2.5 gradient boosting; A2.6 boosting knobs; A2.7 CatBoost; A2.9 pointwise ranking; A2.2 log loss; A3.9 early stopping
Stop 5: readers	Small multilingual transformers read record and candidate together and give one score	A0.14 neural networks; A2.10 tokens and embeddings; A2.11 attention; A2.12 fine-tuning; A2.13 cross- vs bi-encoder; A2.14 inside the reader; A2.15 MiniLM; A2.16 E5; A2.17 training recipe; A2.18 bf16; A2.19 hard examples
Stop 6: "what changed" features	Describe exactly how the candidate differs: numbers, words, legal form	A0.2 features; A1.13 edit distances; A1.14 RapidFuzz; A1.16 IDF; A1.17 IDF-weighted Jaccard; A0.16 embeddings and cosine; A2.20 frozen embeddings
Stop 7: LightGBM decision model	One model per country turns about 200 features into a probability per candidate	A2.8 LightGBM; A2.23 stacking; A2.24 early stopping and refit; A2.25 seed averaging; A0.11 regularisation; A3.10 scaled refit; A3.18 hard examples
Stop 8: confident + clear-winner rule	Link the best candidate only if it is likely enough and clearly ahead	A0.5 scores vs decisions; A1.6 thresholds and AUC; A1.7 abstention; A3.12 tuning on the exact metric; A3.15 the rule; A3.14 calibration
Stop 9: native-script second chance	Search again for unlinked Indian records by transliterated name in the same state	A1.10 transliteration; A2.27 small second-chance models; A3.7 cross-fitting; A3.13 cost-sensitive thresholds
Stop 10: per-business consistency check	Re-decide with a bar that depends on how many links the business already has	A1.5 per-business F0.5; A3.16 count-dependent thresholds
Stop 11: address second chance	Search again for unlinked records by rare numbers and rare address words	A2.27 retrieve, filter, score; A3.13 cost-sensitive thresholds; A2.25 seed averaging
Stop 12: France rules and France	No French labels: self-trained readers, a stricter bar and rules computed by code	A2.21 self-training; A3.20 pseudo-labels; A3.21 replay; A2.22 logit blending; A3.23 rules computed by code; A3.24 domain shift; A3.26 written checks
Stop 13: output and validation	Write the two files; check them with assertions and the organisers' validator	A3.27 validation hygiene; A3.28 reproducibility; A1.19 reduction ratio
Training (all learned stages)	How every model learned without leaking	A0.7 train/validation/test; A0.8 loss; A0.9 gradient descent; A0.10 overfitting; A3.2 cross-validation; A3.3 leakage; A0.18 hyper-parameters
Evaluation and selection	How we measured and chose, and where the limits are	A3.8 held-out protocol; A3.17 paired bootstrap; A3.25 leaderboard overfitting; A3.26 written checks; A0.6 class imbalance

Part A - Learn the concepts

Five chapters, in the order of the 2-day plan's reading list: **A0** builds the machine-learning basics from zero; **A1** covers entity resolution, the metric, text cleaning and blocking; **A2** the models; **A3** training, validation and selection; **A4** follows one record through our whole pipeline and links back to the concepts it uses. The 99 concepts of A0-A3 are numbered A0.1, A0.2 ... and all have the same parts: In one line, Simple explanation, Worked example, Technical explanation, How we used it, Expert nuance, Remember and Judge might ask. Where A2 and A3 teach the same idea from two sides (early stopping, stacking, refitting, seed averaging, calibration, self-training, blending, hard examples), a grey *See also* line under the concept title links the other one.

A0 - Machine learning from zero (the prerequisites for everything else)

How to read this part

This part assumes no background at all. It teaches the eighteen machine-learning ideas that every later part of the study guide uses, in an order where each idea builds on the earlier ones, and every idea is explained with an example from our own problem: linking messy incoming records to the right business in the US, India and France. Read it three times. First pass: only the **In one line**, **Simple explanation** and **Worked example** blocks of all eighteen concepts, with a pen, redoing the arithmetic. Second pass: the **Technical explanation**, **How we used it** and **Expert nuance** blocks; this is the level a senior applied scientist expects. Third pass: say the **Judge might ask** answers aloud until they come out in your own words. Bold words are new terms, defined where they first appear; later concepts point back to earlier ones by name ("see Decision trees").

THREE KINDS OF NUMBERS

Read this before anything else.

- **Real numbers** about our pipeline come from the submitted Documentation (tagged "Doc 3", "Doc D.1" and so on) and the shared fact sheet. They are exact and safe to say on stage. A few are marked **derived**: plain arithmetic on documented numbers, shown so you can redo it.
- **Toy numbers** appear in worked examples only to teach the arithmetic. They are always marked "toy". **Never quote them to the judges as results.**
- A few facts are marked **background**: correct general knowledge or facts read from the public models' configuration files, but not in our Documentation. Use them only if asked.

Spoken answers use the plain names of the v3 talk (business, incoming record, shortlist, quick rule-based filter, CatBoost ranker, the four readers, "what changed" features, LightGBM decision model per country, confident + clear-winner rule, second-chance searches, per-business consistency check, France rules computed by code). Say the final public score as "0.9913".

The problem in one paragraph (you will meet these words everywhere)

A **business** is a record in the reference list: 1,732,544 of them in the test files across the US, India and France. An **incoming record** is a record from one of the two other sources (9,969,589 of them) that must be linked to the one business it belongs to, or to none. A business can own several incoming records. The score is **F0.5 per business, averaged over all businesses**: a number between 0 and 1 that rewards correct links and punishes a wrong link more than a missed one (see Confusion matrix). The US and India have training **labels** (the known right answers); France has none.

The pipeline in one line (plain names)

clean the text → **index lookup** (find businesses that share rare words, letter pieces or sounds) → **quick rule-based filter** (no model) → **CatBoost ranker** → **shortlist** (up to 4 candidates per incoming record) → **readers** (MiniLM reader and E5 reader; for France the MiniLM reader plus the French MiniLM reader and the French E5 reader) → **"what changed" features** → **LightGBM decision model per country** → **confident + clear-winner rule** → refinements (**second-chance searches**, **per-business consistency check**, **France rules computed by code**) → two output files → the organisers' validator.

If most of these names mean nothing to you yet, that is expected; each is explained below. A short decoder for now: **CatBoost** and **LightGBM** are two widely used software libraries that build many small decision trees and add up their answers (see Decision trees and Ensembles); the **readers** are small neural language models that read an incoming record and a candidate business side by side and give one match

score (see Neural networks to Pre-trained and fine-tuned models); a **threshold** is a cut-off score above which we act (see Probability scores).

Concepts in this part (learning order)

Foundations

1. A0.1 What a model is
2. A0.2 Features and labels
3. A0.3 Classification, regression and ranking

Measuring and deciding

4. A0.4 Confusion matrix, precision, recall and F0.5
5. A0.5 Probability scores: why a score is not yet a decision
6. A0.6 Class imbalance

Learning and generalising

7. A0.7 Training, validation and test data
8. A0.8 Loss functions and log loss
9. A0.9 Gradient descent
10. A0.10 Overfitting, underfitting, bias and variance
11. A0.11 Regularisation

Tree models

12. A0.12 Decision trees
13. A0.13 Ensembles: bagging and boosting

Neural text models

14. A0.14 Neural networks
15. A0.15 Tokens and tokenisation
16. A0.16 Embeddings and cosine similarity
17. A0.17 Pre-trained and fine-tuned models

Putting it together

18. A0.18 Parameters and hyper-parameters

(The confusion matrix and class imbalance come early, not last, because "a score is not yet a decision" and everything about thresholds needs them.) A one-page recap table closes the part.

A0.1 What a model is

IN ONE LINE

A **model** is a function, learned from examples, that takes an input (for us, an incoming record paired with a candidate business) and returns an output (a match score).

SIMPLE EXPLANATION

Think of a new clerk at a business registry whose job is to decide whether a scribbled form ("Joes Pizza, 410 Main St") refers to a registered business ("Joe's Pizza, 410 Main Street"). You could hand the clerk a rulebook you wrote yourself, but real forms are messy in a thousand ways no rulebook anticipates. Instead you give the clerk a big pile of old forms where the right answer is already known, and the clerk gradually

works out which signs point to a match. What the clerk ends up carrying in their head is the model: not a copy of every old form, but a compact recipe that turns a new form into a judgement. In a computer the recipe is a set of numbers (for example the split points of decision trees or the weights of a neural network), and **training** is the process of finding good numbers from the examples. Using the trained model on new pairs it has never seen is called **inference** or prediction. Not everything in a system is a model: a hand-written rule such as "keep the candidate if its score is at least 60 % of the best" is also a function, but nobody learned it from data.

WORKED EXAMPLE (TOY)

The simplest learned model is a lookup table of match rates. Suppose we have 20 labelled training pairs, each described by two clues: is the name similarity high (at least 0.85), and is the house number the same?

Name similarity	Same house number	Training pairs	Of which true matches	Learned score
high	yes	6	6	$6 / 6 = 1.00$
high	no or missing	5	3	$3 / 5 = 0.60$
low	yes	4	1	$1 / 4 = 0.25$
low	no or missing	5	0	$0 / 5 = 0.00$

1. Group the training pairs by their clues.
2. Count the share of true matches in each group.
3. Store the four shares. These four numbers are the model's **parameters** (see Parameters and hyper-parameters).
4. Apply it to new pairs. "Joes Pizza, 410 Main St" against "Joe's Pizza, 410 Main Street": high, same number → 1.00. "Kumar Electricals, 3 Ring Road, Delhi" against "Kumar Textiles, 41 Chandni Chowk, Delhi": low, different → 0.00. "Patel Medical Store" with no address against "Patel Medical Store, 5 Station Road, Ahmedabad": high, missing → 0.60.

The examples have become a function: clues in, score out.

TECHNICAL EXPLANATION

In **supervised learning** (learning from examples whose right answers are known) we assume labelled examples (x, y) drawn from some distribution and choose a function f_θ from a **hypothesis class** (all lookup tables, all sums of trees of a given size, all transformer networks of a given shape) by minimising the average **loss** over the training examples (see Loss functions), hoping that it **generalises** to new draws from the same distribution. Three design choices define a model: the **representation** of the input (hand-made features, or raw text cut into tokens), the hypothesis class with its parameters θ , and the training objective with its optimiser. For pair matching, x is an (incoming record, candidate business) pair, $y \in \{0, 1\}$ says whether the candidate is the record's true owner, and $f_\theta(x)$ estimates $P(y = 1 | x)$. A full system wraps models in deterministic decision logic: models produce scores, rules turn scores into actions (see Probability scores). Training cost is paid once; inference cost is paid for every pair scored, and in entity resolution that dominates the design, because the number of possible pairs is enormous (6,724,569,566,212 same-country pairs in our test files, Doc 3). The assumption "new data looks like the training data" is the one that breaks most often; in our data it is weakest for France, whose address formats and legal vocabulary differ from the labelled US and Indian data and which has no labels to check against.

HOW WE USED IT

Our system is a chain of functions, some learned and some hand-written.

- Learned from scratch on the provided labelled US/India data: the **CatBoost ranker** (76 index features → probability that the candidate is the owner); one **LightGBM decision model** per country plus a default one for unseen countries (about two hundred features → probability); and the **second-chance**

models (the native-script rescue model for India; the address rescue filter and scorer for the US and India).

- Learned by fine-tuning pre-trained networks (see Pre-trained and fine-tuned models): the four **readers** (raw text of a pair → one score).
- Learned by simple counting: the Indic-to-Latin word list used in cleaning, built from labelled train matches of folds 0-5 by counting which Latin word each native-script word co-occurs with. It is a small statistical model, not a neural transliterator.
- Hand-written: text cleaning, the quick rule-based filter, the confident + clear-winner rule, the per-business consistency check, and the France rules computed by code (designed from analysis of the unlabelled French records, which our documentation discloses).

EXPERT NUANCE

The score that matters (public F0.5 0.991300) belongs to the whole system, not to one model, and several important choices sit in the hand-written decision layer. Learned parts adapt to data but need labels; hand-written rules are auditable but do not adapt, which is why we used rules mainly where labels were missing (France) and stated that they came from test-set analysis. A common misconception is that a model "understands" a pair: it can only use what its input shows it. A reader sees one pair at a time and cannot know that a better candidate exists, so we give that information to the decision model as competition features (see Features and labels). The toy table also shows a classic danger: "6 of 6 = 1.00" is a confident claim from only six examples (see Overfitting).

REMEMBER

- Model = a function whose numbers were learned from labelled examples; training finds the numbers, inference applies them.
- Our system mixes learned models (ranker, readers, decision models, second-chance models) with hand-written rules (filter, decision rule, consistency check, France rules).
- A model can only use what its inputs show it.

JUDGE MIGHT ASK

- *Which parts of your system are learned, and which are hand-written?* The CatBoost ranker, the four readers, one LightGBM decision model per country and the second-chance models are learned; the cleaning, the quick rule-based filter, the confident + clear-winner rule, the per-business consistency check and the France rules are code. For the US and India, the decision threshold and the consistency-check bars were tuned on held-out folds for the exact competition metric. France has no labels, so its rules came from analysing the unlabelled French records, and some of their parameters were chosen after seeing public scores, as we disclose.

A0.2 Features and labels

IN ONE LINE

Features are the numbers we compute to describe an example; the **label** is the correct answer we want the model to learn to predict.

SIMPLE EXPLANATION

A detective comparing two descriptions of a shop writes down clues: Do the names share rare words? Is the house number the same? Did "Inc" become "LLC"? Each clue, written as a number, is a feature, and the list of all clues for one pair is its **feature vector**. The label is the verdict from the closed case files: "same business" (1) or "not the same" (0). Training means showing the model many feature vectors together with their labels so it learns which patterns of clues go with which verdict. Two golden rules: a feature must be

computable when the answer is unknown (otherwise the model cheats during training and fails in use), and better clues usually beat a cleverer detective.

WORKED EXAMPLE

From the Documentation's US example (Doc D.1). Incoming record: "Legacy Marketing Enterprises (Inc)", no address. Candidate business: "Legacy Marketing Industries Inc", 2360 Airbase Road, Louisville, TN.

1. Lower-case and drop punctuation: "legacy marketing enterprises inc" against "legacy marketing industries inc".
2. Our word features ignore legal-form words, so compare {legacy, marketing, enterprises} with {legacy, marketing, industries}.
3. Words replaced: 1 (enterprises → industries); added: 0; dropped: 0; the replacement sits in the same position ("in place"): yes.
4. Plain **Jaccard overlap** = shared words / all distinct words = $|\{\text{legacy, marketing}\}| / |\{\text{legacy, marketing, enterprises, industries}\}| = 2 / 4 = 0.50$. Our feature weights each word by its rarity (IDF, below); the Documentation reports 0.52 for this pair.
5. Meaning similarity of the replaced word, from a pre-trained language model (see Embeddings): 0.77 (Doc D.1).
6. Legal-form relation: same (Inc and Inc). Number features: left empty (missing), because the record has no address and so no number to compare.
7. So part of this pair's feature vector reads [replaced 1, added 0, dropped 0, in place 1, IDF-weighted overlap 0.52, meaning similarity 0.77, legal form "same", number features missing, ...]: 217 numbers in all for a US pair.
8. Label: in training data it comes from the ground truth (1 if this business owns the record, else 0). Test labels are private; the decision model gave this pair 0.000088, while "Legacy Marketing Enterprises Inc" in Windsor, CT got 0.965022.

TECHNICAL EXPLANATION

A feature vector $x \in \mathbb{R}^d$ is the model's entire view of an example; for tree models (see Decision trees) features can be on any scale, and categorical relations can be coded as integers. Our feature families are: (a) **similarity** features (string similarities, word and letter-piece overlaps, IDF-weighted overlaps); (b) **difference** features that describe the edit turning the business into the record (number deltas; words added, dropped or replaced, with position, rarity and meaning similarity; the legal-form relation); (c) **model-output** features (ranker probability, reader scores), which make the decision model a **stacked** model (see Ensembles); (d) **context** features over a record's whole shortlist (rank, best other score, margin over the best other); (e) **collective** features over the other incoming records competing for the same business (US and India only). **IDF** (inverse document frequency) measures rarity, $\text{IDF} = \log((N + 1) / (df + 1))$ with N the number of records and df the number containing the word, so common words count less than rare ones (the index lookup uses the smoothed variant $\log(1 + (N + 1) / (df + 1))$, Doc 3); it needs no labels. In the US example the IDF-weighted overlap (0.52) differs from the plain overlap (0.50) only because the four words have different rarities. The label is defined per pair ($y = 1$ if candidate = owner) but the decision is per record, so the labels of one record's candidates are mutually exclusive: at most one can be 1. Unowned incoming records contribute pairs whose labels are all 0, which is how the model learns to say "none". **Leakage**, a feature that encodes the label or uses information unavailable at prediction time, is the cardinal sin: it inflates validation scores and fails in use. Missing information must be coded deliberately: our generic number differences are left missing (NaN) when there is no substituted number, and LightGBM learns which branch missing values should take; other generic features use a **sentinel** value outside their normal range (-1 for most "not applicable" cases, such as a digit edit distance

or a word position when there is nothing to compare; -2 for the meaning similarities, because a real cosine can itself be as low as -1), so a single tree split can separate "not applicable" from every real value.

HOW WE USED IT

The CatBoost ranker uses 76 features from the index lookup. The decision models use 217 features (US), 211 (India) and 158 (France and the default route); in the talk, "about two hundred features per country". Labels exist for the US and India only; France has none, which is why the French readers learned from **pseudo-labels** (see Pre-trained and fine-tuned models). Training rows for the decision models are the shortlist pairs of training records that have a candidate in business folds 6-9, labelled "candidate is the owner". We removed 21 data-specific inputs and replaced them with 25 generic "what changed" features; a code check asserts that none of the removed inputs is used.

EXPERT NUANCE

Similarity features alone treat two edits as equal when they cost the same number of characters, yet dropping "Inc" and swapping "Inc" for "LLC" mean very different things for identity; difference features let the model learn which edits are harmless. Our word features deliberately ignore legal-form tokens, so a legal-form change shows up only in the legal-form features and in the readers, which keeps the two kinds of edit separate. The IDF and index statistics are computed on the unlabelled test files themselves (disclosed, Doc C.1): that uses no labels, but it does adapt the pipeline to this test set. A long feature list is not a virtue in itself, and we have no feature-importance ranking prepared, so do not claim which feature matters most; show the mechanism with the US example instead.

REMEMBER

- Feature = a clue written as a number; label = the right answer from training data; a feature must never peek at the label.
- Our key idea: describe "what changed" (numbers, words added, dropped or replaced, legal form), not only "how similar".
- Labels are per pair, decisions are per record; France has no labels.

JUDGE MIGHT ASK

- *Your readers already read both texts. Why add hand-made "what changed" features?* A reader returns one number for one pair; the decision model also needs to know which edit happened, how rare the changed word is and how the candidate compares with its rivals. In our US example the legal-form features and the readers put the LLC business last although the ranker had placed it second, and the word-replacement features mark the "Industries" business, which scores 0.000088.
- *Where do your labels come from, and what exactly is one training example?* The organisers' training files give each US and Indian incoming record its owning business, or none; one example is one (incoming record, candidate) pair from the shortlist, labelled 1 if the candidate is the owner. France has no labels at all, so its readers learned from confident pseudo-labels and its decision model from US and Indian rows.

A0.3 Classification, regression and ranking

IN ONE LINE

Classification predicts a category, **regression** predicts a number, and **ranking** predicts an order; our task needs ranking and classification together, with "none" as an allowed answer.

SIMPLE EXPLANATION

Three questions a registry clerk might ask. "Is this form about this business, yes or no?" is classification. "How many forms will this business receive next month?" is regression, because the answer is a number on a scale. "Of these twenty possible businesses, which is most likely, which second, and so on?" is ranking. Our job is: for each incoming record, find the most likely business (ranking), then decide whether it is good enough to link (a yes/no classification), and otherwise answer "none". We do not treat each of the 1.7 million businesses as its own category, because that list is huge and changes; instead we ask the same yes/no question about one pair at a time.

WORKED EXAMPLE

Incoming record "Legacy Marketing Enterprises (Inc)", no address (Doc D.1).

Candidate business	Classification view: P(match) from the decision model	Ranking view
Legacy Marketing Enterprises Inc, Windsor, CT	0.965022	1st
Legacy Marketing Enterprises PC, Joshua Tree, CA	0.088664	2nd
Legacy Marketing Industries Inc, Louisville, TN	0.000088	3rd
Legacy Marketing Enterprises LLC, Irondequoit, NY	0.000007	4th

1. Score each pair on its own (classification).
2. Sort the scores (ranking).
3. Apply the decision rule to the top one: $0.965022 \geq 0.75$, and its lead is $0.965022 - 0.088664 = 0.876 \geq 0.4$, so the record is linked to the Windsor business only.

A regression question about the same data, which we did not model, would be "how many incoming records will this business own?"; our final file links on average 3.3805 records per business (Doc B.1).

TECHNICAL EXPLANATION

Binary classification models $P(y = 1 | x)$ and is trained with log loss (see Loss functions). Multi-class classification puts a **softmax** (a function that turns K scores into K probabilities summing to 1) over K fixed classes, which is unusable when the "classes" are 1,732,544 businesses, a list that is huge, changes over time and has only a handful of examples per business, so entity resolution is normally cast as binary classification over **candidate pairs**. Regression predicts $y \in \mathbb{R}$ with losses such as squared or absolute error. **Learning to rank** has three families: **pointwise** (score each item alone with a classification or regression loss, then sort), **pairwise** (a loss on ordered pairs of items, as in RankNet) and **listwise** (a loss targeting a ranking metric over the whole list, as in LambdaMART for **NDCG**, a ranking-quality measure that rewards placing the right items near the top). Our CatBoost ranker is pointwise: a binary classifier with log loss whose probabilities are sorted. The final step is **selection with abstention**: take the **argmax** (the candidate with the highest score) over a record's candidates, then apply a **reject option** (a threshold and a margin; "reject" here means "decline to link"). The structure is many-to-one, a record links to at most one business while a business may own many records, so it is not one-to-one assignment (no Hungarian matching, the classic algorithm for pairing two lists one to one). Pointwise models never see the competitors, so list context must be supplied as features: rank, best other score and margin within the record's shortlist.

HOW WE USED IT

The CatBoost ranker is a pointwise classifier used to rank and then cut: the top candidate always enters the shortlist, and ranks 2-4 only if their ranker probability is at least 0.001 (so at most 4 per record). The readers and the decision models are binary classifiers on pairs. The confident + clear-winner rule turns pair scores into one decision per record. The decision model receives the ranking context as features: rank, best other score, margin and group maximum of the ranker and reader scores.

EXPERT NUANCE

We did not use a ranking loss (LambdaMART, YetiRank or a pairwise loss), and the Documentation reports no comparison, so do not claim one would be worse. A practical point for a classification loss here: we need a probability, not just an order, because both the 0.001 shortlist floor and the decision model's input use the ranker's probability, while a listwise score is only comparable within one record's list. Injecting rank and margin features is a cheap way to make a pointwise second stage list-aware. The "none" answer is essential: about 41 % of incoming records are linked to nothing in our final file (derived from 0.5875 links per incoming record).

REMEMBER

- Classification = category, regression = number, ranking = order.
- Our ranker is a pointwise classifier (log loss) used for ranking; it is not trained with a ranking loss.
- Final decision = pick the best candidate or abstain.

JUDGE MIGHT ASK

- *Your "ranker" is a classifier. Why not a learning-to-rank objective such as LambdaMART?* Yes, it is a pointwise CatBoost classifier with log loss whose probabilities we sort. We needed a probability, not just an order: ranks 2-4 enter the shortlist only above 0.001, and the probability is an input of the decision model. We did not benchmark a ranking loss, so we do not claim it would be better or worse; what we did measure is that on held-out fold 9 the shortlist still holds the true business for at least 98.87 % of US records, against 99.37 % at the index.

A0.4 Confusion matrix, precision, recall and F0.5

IN ONE LINE

A **confusion matrix** counts the four ways a yes/no prediction can be right or wrong; **precision**, **recall** and **F0.5** summarise it, with F0.5 weighting precision more.

SIMPLE EXPLANATION

Imagine auditing a clerk's links. Each link they made is either right (a **true positive**, TP) or wrong (a **false positive**, FP). Each true link they failed to make is a **false negative** (FN). Each wrong pair they correctly left alone is a **true negative** (TN); in matching there are trillions of those, so they tell us nothing. **Precision** asks "of the links you made, how many were right?"; **recall** asks "of the links you should have made, how many did you make?". F0.5 blends the two but cares more about precision, because a wrong link hurts more than a missed one. Our competition computes it separately for every business and then takes the plain average.

WORKED EXAMPLE (TOY)

Business A, "Joe's Pizza, 410 Main St, Boston", truly owns 3 incoming records: R1, R2 and R3.

	Truly A's record	Not A's record
We linked it to A	TP	FP
We did not link it to A	FN	TN (millions; ignored)

1. Option 1: link R1, R2 and R7 (R7 belongs elsewhere). TP = 2, FP = 1, FN = 1. Precision = $2/3 = 0.667$; recall = $2/3 = 0.667$. F0.5 = $1.25 \times P \times R / (0.25 \times P + R) = 1.25 \times 0.444 / (0.167 + 0.667) = 0.667$. The count form is easier by hand: F0.5 = $1.25 \times TP / (\text{predicted} + 0.25 \times \text{true}) = 1.25 \times 2 / (3 + 0.75) = 0.667$.
2. Option 2: link only R1 and R2: $1.25 \times 2 / (2 + 0.75) = 0.909$. (F1 would give 0.800 here; F0.5 rewards caution more.)

3. Option 3: link R1, R2, R3 and R7: $1.25 \times 3 / (4 + 0.75) = 0.789$.
4. Option 4: link exactly R1, R2 and R3: $1.25 \times 3 / (3 + 0.75) = 1.000$.
5. Macro average over three businesses: A with option 2 (0.909); B, "Kumar Textiles", which has no true records and gets nothing linked (1.000 by the competition's rule); C, "Sharma Traders", which has no true records but gets one wrong link (TP 0, predicted 1, true 0 $\rightarrow$ 0.000). Macro F0.5 = $(0.909 + 1.000 + 0.000) / 3 = 0.636$. One wrong link on an empty business cost a whole point.

TECHNICAL EXPLANATION

Precision = $TP / (TP + FP)$; recall = $TP / (TP + FN)$. $F_b = (1 + b^2) \times P \times R / (b^2 \times P + R) = (1 + b^2) \times TP / ((1 + b^2) \times TP + b^2 \times FN + FP)$. In van Rijsbergen's definition b is the relative importance of recall to precision, so with $b = 0.5$ ($b^2 = 0.25$) recall counts half as much as precision; equivalently F0.5 is a weighted **harmonic mean** (the reciprocal of the average of reciprocals, which is pulled towards the smaller of the two values) with weight $1 / (1 + b^2) = 0.8$ on precision: $1 / F = 0.8 / P + 0.2 / R$. Our per-business form is $F0.5 = 1.25 \times TP / (\text{predicted} + 0.25 \times \text{true})$, defined as 1 when both predicted and true are 0. **Macro-averaging** gives each of the 1,732,544 businesses equal weight regardless of size, unlike **micro-averaging**, which pools TP, FP and FN over everything first. TN never enters an F-score, which is why accuracy (dominated by TN) is useless here. Because each incoming record has at most one owner, a wrong link creates an FP at the wrong business and leaves an FN at the true one, while abstaining creates only the FN, so a wrong link is never better than no link and usually worse (it lowers the wrong business's score unless that score is already 0). Threshold-free metrics such as **AUC** (the area under the ROC curve, which plots the true-positive rate against the false-positive rate as the threshold moves; equivalently, the probability that a random positive scores above a random negative) summarise ranking quality over all thresholds, but the competition scores one **operating point** (one chosen threshold).

HOW WE USED IT

This exact metric is implemented in our evaluation code: per business, macro-averaged, with the empty-row rule, and with true records that never reached our candidates still counted as misses. The labelled operating points were tuned on it: the 0.75 threshold on fold 8, the consistency-check bars on folds 8 and 9, and the second-chance thresholds on fold 8 with false positives counted twice (France's 0.82 was set without labels; see Probability scores). Results: public 0.991300; on held-out fold 9 / fold 8, close versions of our decision models (the India recipe with 3 of its 5 seeds) score US 0.991212 / 0.991545 and India 0.990046 / 0.990298 (Doc 5). 100,087 businesses (5.8 %, derived) end with an empty row.

EXPERT NUANCE

Under macro F0.5 the most fragile businesses are those with no true records: one wrong link takes them from 1 to 0, while missing one record of a business that owns 6 only takes it from 1 to 0.9615 (toy arithmetic: $1.25 \times 5 / (5 + 1.5)$). AUC can be nearly perfect while the F-score at a threshold is mediocre, and two models with almost equal AUC can produce different link files; for the French E5 reader against its base model on labelled US and Indian pairs, AUC was 0.998544 against 0.998570, a difference that says little about the final file. Recall of candidate generation (is the owner among the candidates at all?) caps the recall of everything after it, which is why we measured it at every step: 99.3694 % (US) and 98.5068 % (India) of owned fold-9 records at the index (Doc B.2).

REMEMBER

- Precision = right links / links made; recall = right links / true links; TN is irrelevant.
- Per-business F0.5 = $1.25 \times TP / (\text{predicted} + 0.25 \times \text{true})$, and 1 for a correctly empty row; then average over businesses.
- A wrong link costs a false positive plus a miss; abstaining costs only the miss.

JUDGE MIGHT ASK

- *Why does this metric push you towards abstaining?* A wrong link costs twice: the wrong business gets a false positive and the true business still misses the record, while abstaining costs only the miss; and a business with no true records scores 1 only if its row stays empty. So we link only the best candidate when it is confident and clearly ahead, and 100,087 businesses end with an empty row.
- *Why not report accuracy or AUC?* Accuracy is dominated by trillions of correct non-links, and AUC ignores the operating point. We tuned and reported the exact per-business macro F0.5; for example, the French E5 reader's AUC on US and Indian pairs was within 0.00003 of its base model's, which says little about the linked file.

A0.5 Probability scores: why a score is not yet a decision

IN ONE LINE

A model gives a **score** such as 0.97 ("how likely is a match"); a separate **decision rule** turns scores into actions (link or not), and the right rule depends on what each kind of mistake costs.

SIMPLE EXPLANATION

A forecast of "70 % chance of rain" does not tell you whether to carry an umbrella; that depends on how much you mind getting wet. In the same way, a model can say "97 % sure this is the same business", but whether we should link depends on the cost of a wrong link (high under F0.5) and on whether another candidate is almost as good (if two candidates both look great, the model is confused). Models also speak in different units: some output a **probability** between 0 and 1, others a **logit**, an unbounded number that the **sigmoid** function squeezes into the range 0 to 1. Our final decision is a small rule on top of the scores: link the best candidate only if it is confident enough and clearly ahead of the runner-up.

WORKED EXAMPLE

1. Logit to probability, $\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$. In the France example (Doc D.3), for the record "TQ COMITE SARL", "2 R SUZANNE LENGLEN, NANTES", the MiniLM reader gave the business "TQ Comite SARL", "2 RUE du Lycee, Pessac" a logit of 7.125, so $\text{sigmoid}(7.125) = 0.9992$ (derived). Same name, but another city. The French reader blend gave -3.2008 ($\text{sigmoid } 0.039$) and the decision model 0.000098. One high score was not a decision.
2. The decision rule on the US example: best $0.965022 \geq 0.75$ (confident) and $0.965022 - 0.088664 = 0.876 \geq 0.4$ (clear winner), so link. Toy counter-example: best 0.91, runner-up 0.70: confident, but the lead is $0.21 < 0.4$, so no link.
3. Why the best threshold depends on context (toy). Business A owns 3 records and we already linked 2 of them correctly, so its F0.5 is $1.25 \times 2 / (2 + 0.75) = 0.909$. A third candidate link has match probability p . If right, F0.5 becomes $1.25 \times 3 / (3 + 0.75) = 1.000$, a gain of 0.091. If wrong, it becomes $1.25 \times 2 / (3 + 0.75) = 0.667$, a loss of 0.242. Linking pays off only if $p \times 0.091 > (1 - p) \times 0.242$, that is $p > 0.727$. From a different starting point the break-even moves, which is the intuition behind our per-business consistency check using different bars depending on how many other records were accepted for the same business.

TECHNICAL EXPLANATION

A classifier trained with log loss estimates $P(y = 1 | x)$ on its training distribution, but tree ensembles and neural networks can be **miscalibrated** (among pairs scored 0.9, the true share of matches is not 90 %), and calibration does not survive a change of input distribution. The **logit** is $z = \ln(p / (1 - p))$ and sigmoid is its inverse; averaging logits (as in our French blend) lets confident votes count more than averaging probabilities would. Decision theory: for one yes/no decision with costs c_{FP} and c_{FN} , act when $p > c_{FP}$

/ (c_FP + c_FN). F-scores are not additive costs, but a short calculation (background, not from our documents) gives a clean rule for a pooled (micro) F_b. Write $F = (1 + b^2) \times TP / D$ with $D = \text{predicted} + b^2 \times \text{true}$. Adding one link with match probability p leaves $D + 1$ in the denominator either way, so the expected score becomes $(1 + b^2) \times (TP + p) / (D + 1)$, which is higher than F only if $p > TP / D = F / (1 + b^2)$. For F1 this is the known "best threshold is half the best F1" result; for F0.5 it is $0.8 \times F$, about 0.79 when $F \approx 0.99$. Our metric is macro per business, so the break-even depends on each business's state (step 3 above); we therefore tuned thresholds empirically on held-out folds rather than deriving them. The **margin** (lead over the runner-up) is an ambiguity reject option: a record's candidates are mutually exclusive, so calibrated probabilities for one record cannot sum to more than 1, and scores of 0.9 and 0.8 signal that the model cannot separate the two. In code, a record with a single candidate treats the runner-up as minus infinity, so the lead condition passes automatically (say this only if asked); 90.59 % of incoming records have exactly one candidate in the final candidate file (Doc B.1).

HOW WE USED IT

- Scores in our pipeline: the ranker's probability (for the 0.001 shortlist floor, and as a decision-model input); the reader logits (for France blended in logit space, 0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5); the decision model's probability (the mean of 5 seed models; a single fit on the default route).
- The decision: confident (≥ 0.75 for the US and India, tuned on fold 8; ≥ 0.82 for France and the default route) and clear winner (lead ≥ 0.4 , one fixed value for every country).
- The per-business consistency check re-decides each US and Indian record with the bar 0.75 / 0.68 / 0.72 / 0.77 for 0 / 1 / 2 / 3 or more other accepted records (chosen on folds 8 and 9).
- Second chances: native-script rescue links if its score is ≥ 0.88 with a lead ≥ 0.3 (the 0.88 chosen on fold 8 with false positives counted twice); address rescue US 0.86 with margin 0.0, India 0.66 with margin 0.1 (threshold and margin chosen on fold 8 with false positives counted twice).
- No **Platt scaling** (fitting a small logistic curve from scores to observed match rates) or **isotonic calibration** (fitting a step-wise increasing curve for the same purpose): thresholds are tuned directly on the exact metric.

EXPERT NUANCE

Tuning the threshold on held-out data for the exact metric makes calibration unnecessary for the decision, as long as the held-out fold resembles the test; when an input changes meaning, the threshold's meaning changes too. France is the case in point: its decision model was trained with the MiniLM reader's logits in its reader slot but at test receives the French blend, and France has no labels, so 0.82 was set label-free from the French link volume per business together with a fold-8 threshold sweep on US/India, more conservative than 0.75. Counting false positives twice when tuning the second-chance thresholds is an explicit cost weighting that reflects F0.5. Do not say the 0.4 lead was "never tuned": the Documentation does not record how it was picked; say it is one fixed lead used in every country.

REMEMBER

- Score $\neq$ decision; the rule (threshold + lead) encodes the cost of mistakes.
- Logit = log-odds; sigmoid turns it into a probability; France blends its readers in logit space.
- Our rule: best candidate only, ≥ 0.75 (US, India) or ≥ 0.82 (France, default), and a lead ≥ 0.4 .

JUDGE MIGHT ASK

- *Are your decision-model probabilities calibrated?* We did not apply Platt or isotonic calibration; instead the thresholds were tuned directly on the exact per-business macro F0.5 on held-out fold 8, which is what the decision needs. For France, with no labels, the 0.82 bar was set label-free from the number of French links per business together with a fold-8 threshold sweep on US and Indian data.

- *Why require a lead over the runner-up instead of just a threshold?* A record's candidates compete for one slot, so two high scores mean the model cannot tell them apart, and under F0.5 a wrong link costs more than a miss. The 0.4 lead is one fixed value used in every country; the threshold itself was tuned on fold 8.

A0.6 Class imbalance

IN ONE LINE

Class imbalance means one answer ("not a match") is vastly more common than the other, which misleads naive training and naive metrics.

SIMPLE EXPLANATION

Matching is a needle-in-a-haystack problem: an incoming US record could in principle belong to any of 663,106 US businesses, and at most one is right. A lazy model that always says "no match" would be right on about 99.99985 % of pairs and completely useless. When nearly all examples are "no", a naively trained model learns mostly about easy "no" cases and too little about the rare "yes" ones. There are three standard remedies: shrink the haystack before learning, choose metrics that ignore the easy "no" answers, and choose training examples carefully. We used all three.

WORKED EXAMPLE (DERIVED FROM DOCUMENTED COUNTS)

1. All same-country pairs: 6,724,569,566,212. Each incoming record has at most one owner, so at most 9,969,589 pairs are true: at most 1 pair in 674,508, about 1.5 per million.
2. "Always say no" accuracy for one US record: $1 - 1 / 663,106 = 99.99985 \%$.
3. After the index lookup and the quick rule-based filter: 37,614,782 pairs, of which true pairs can be at most 26.5 %.
4. The shortlist: 11,687,317 pairs, 1.17 per incoming record. In the final candidate file (12,293,019 pairs) our 5,856,936 links make up 47.6 % of the pairs.

So the cascade turns a one-in-a-million problem into a nearly balanced one before the expensive readers and the decision models see it.

TECHNICAL EXPLANATION

Imbalance hurts in three places. In learning, the average loss and its gradient are dominated by the many easy negatives, and the model's outputs inherit the training **base rate** (prior), so a model trained on a sample re-balanced towards positives over-predicts positives on the natural distribution unless the prior is corrected: subtract [log-odds of the training base rate – log-odds of the true base rate] from the logit. In evaluation, accuracy and even ROC curves look excellent when negatives dominate, because the false-positive rate divides by the huge number of negatives, so a rate that looks negligible can still mean more wrong links than right ones; precision-based metrics (precision, F-scores, precision-recall curves) stay informative because TN never enter them. In decisions, the best threshold depends on the base rate and on the costs (see Probability scores). Standard remedies: candidate generation (**blocking**) that removes obvious negatives; **informative sampling** (keep hard negatives, subsample easy ones); class weights or **focal loss** (a variant of log loss that down-weights examples the model already gets right); and tuning the threshold on a held-out set drawn like the deployment data. Hard negatives are worth far more than many easy ones because they sit next to the decision boundary.

HOW WE USED IT

- Blocking: index lookup → quick rule-based filter (21.7107 candidates per business) → shortlist (at most 4 per incoming record).

- Reader training sample: every "hard" training record (top ranker probability between 0.003 and 0.9995, a non-owner ranked first, or a second candidate above 0.003) plus a random 25 % of the others; candidates 1-4 if ranked first, at probability ≥ 0.001 , or the owner; candidates 5-8 only if they are the owner, so the readers also see true owners that the ranker placed low.
- French E5 reader pseudo-labels: at most one positive and one negative per record, balanced classes, at most 250,000 per class.
- No class weights in any model (the training code passes no class or sample weights). The decision models train on the natural shortlist pairs of training records (folds 6-9), not on a sample re-balanced towards matches (for the US and India with the extra hard examples described in the next bullet), and their thresholds are tuned on held-out folds, so the base rate is handled at decision time.
- Only if a judge asks about hard negatives, use the QA 15 wording: "for the US and India only, the decision models were trained with extra hard examples: copies of unlinked training records that sit right next to a candidate business, with small name edits; how many to add was set from label-free counts on the test records, never from labels; the France decision model uses no such examples."

EXPERT NUANCE

Because the readers are trained on a deliberately hard-heavy sample, their raw scores need not be calibrated to the test base rate; that is acceptable because they are inputs to a decision model trained on other folds, which learns its own mapping from reader score to probability. Imbalance also exists at the business level: a business with no true records scores 1 only if its row stays empty, so a single false positive there costs a whole point; our final file leaves 100,087 rows empty. A common misconception is that imbalance must be "fixed" by oversampling; often the better fix is a better candidate set plus a threshold tuned for the real metric.

REMEMBER

- Raw pairs: at most about 1.5 true per million; after the shortlist: about 1.17 pairs per incoming record.
- Handle imbalance by shrinking the haystack, sampling hard examples and tuning thresholds on the true metric.
- No class weights anywhere in our pipeline.

JUDGE MIGHT ASK

- *Matching is extremely imbalanced. Did you use class weights or oversampling?* No class weights; the cascade removes most of the imbalance: of 6.7 trillion same-country pairs at most one per incoming record is true, but after the shortlist each record has on average 1.17 pairs. For the readers we sampled training records where the ranker was unsure or wrong plus a random quarter of the easy ones, and the decision thresholds were tuned on held-out folds, so the base rate is handled at decision time.

A0.7 Training, validation and test data

IN ONE LINE

We learn on one part of the data (**training**), make choices on a second part (**validation**), and measure honestly on a third part that influenced nothing (**test**).

SIMPLE EXPLANATION

A student preparing for an exam does textbook exercises (training), sits mock exams to decide what to revise and when to stop (validation), and then takes the real exam (test). If the student has seen the real exam paper beforehand, a high mark proves nothing. Models are the same: any data used to make a choice stops being a fair test. Matching has an extra trap: the natural unit is the business. If some records of "Sharma Traders, Pune" are in training and others in the test, the model may simply recognise Sharma

Traders instead of learning how to match, and the test score looks better than reality. So we split by business: all records of a business go to the same side.

WORKED EXAMPLE (TOY HASH VALUES)

Each labelled training business gets a **fold** number = $\text{hash}(\text{business id}) \bmod 10$, and every incoming record goes with its owner. A **hash** is a fixed scrambling function that turns an id into a large, random-looking but repeatable number; "mod 10" keeps its last digit, so the businesses spread evenly over 10 folds. (Folds split the labelled training data; the business names below are only illustrations.)

Business	Toy hash	Fold = hash mod 10	Its incoming records
Sharma Traders, Pune	4,213	3	4 records, all in fold 3
Joe's Pizza, Boston	7,846	6	2 records, fold 6
Legacy Marketing Enterprises Inc, Windsor	1,239	9	3 records, fold 9
Patel Medical Store, Ahmedabad	5,518	8	5 records, fold 8
Green Leaf Salon, Austin	2,990	0	none

With a record-level split, two of Sharma's four records could land in training and two in the test; the model would have seen "Sharma Traders, 12 MG Road" as a positive and would find the test copies easy. With the business-level split, fold 9 contains whole businesses the models never saw. Then each group of folds gets a job (our real plan):

1. Folds 0-5: train the Indic-to-Latin word list, the CatBoost ranker (with fold 6 for **early stopping**, i.e. deciding when to stop adding trees; see Overfitting) and the readers.
2. Folds 6-7: train the decision models; fold 8: early stopping and the 0.75 threshold; for the US and India the decision models are then refitted on folds 6-9.
3. Folds 8 and 9: choose the consistency-check bars; fold 9 (and fold 8 with a mirrored set-up): report held-out scores.

TECHNICAL EXPLANATION

Training data fits parameters; validation data selects hyper-parameters, early-stopping points, thresholds and model variants; test data estimates performance once. Every decision taken by looking at a dataset makes it partly training data (**adaptive overfitting**). With grouped data, splits must respect the groups (**group k-fold**); ours is $\text{fold} = \text{hash}(\text{business id}, \text{seed } 20260927) \bmod 10$, where owned records inherit their owner's fold and unowned records hash their own id. Our **stacked** design needs a second layer of hygiene: the decision model takes ranker and reader scores as inputs, and a model's scores on its own training records are over-confident, so the base models (ranker, readers) train on folds 0-5 and the decision models on folds 6-9. The usual alternative is **out-of-fold predictions** (k-fold cross-fitting of every base model); our layer-wise split costs only one training run per base model. Our held-out protocol scores fold 9 with fits on folds 6-8 that exclude every record touching fold 9, and fold 8 with fits on folds 6, 7 and 9 that exclude records touching fold 8, with each seed's iteration count fixed from its early-stopping point and scaled to the fit size. The public leaderboard is a test set to which we made 18 uploads (17 returned a score), so it was partly used as validation.

HOW WE USED IT

As above, plus: the US and India decision models fit on folds 6-7 with early stopping on fold 8 (at most 3,000 rounds), then refit on folds 6-9 for best iteration $\times \text{rows}(6-9) / \text{rows}(6-7)$ rounds; the France and default decision models fit on 6-7 with early stopping on 8 (at most 1,500 rounds) and no refit. The native-script rescue model trains on folds 0-7 with word lists that leave out the record's own fold (**cross-fitting**, so no record benefits from its own label); the address rescue models train on folds 6-7, with thresholds on fold 8. Held-out scores of close versions of our decision models (the India recipe with 3 of its 5 seeds), fold 9 / fold 8: US 0.991212 / 0.991545, India 0.990046 / 0.990298; public 0.991300.

EXPERT NUANCE

Limitation first: there is no untouched end-to-end validation of the final file. The held-out scores exclude the second-chance searches and the France rules, the final US/India decision models are refitted on folds 6-9, folds 8 and 9 also chose thresholds and the consistency check, and France has no labels; the mitigation is that held-out scores from fits that leave the scored fold out sit in the same range as the public score. The pipeline also computes unsupervised statistics on the unlabelled test files (IDF, the indexes, for India a word alignment on very confident test matches, for France the geography) and self-trains the French readers on French test pseudo-labels; no labels are involved and it is disclosed (Doc C.1), but it means the system was adapted to this test set. The refit rule (scale the number of trees by the row ratio) is a heuristic, needed because no fold is left for early stopping.

REMEMBER

- Train fits, validation chooses, test judges; anything you looked at to choose is no longer a clean test.
- Split by business (hash of the business id, 10 folds); ranker and readers on folds 0-5, decision models on folds 6-9.
- Disclosed limitation: no untouched end-to-end validation of the final file.

JUDGE MIGHT ASK

- *Why split folds by business rather than by record?* The business is the unit of the metric and of several features: one business owns several records, the score is computed per business, and the consistency check looks at all records competing for the same business. A record-level split would put records of the same business on both sides and inflate the held-out score; when we score fold 9, the fits also leave out every record that touches fold 9.
- *Do you have a clean test set for the final file?* No, and our documentation says so: there is no untouched end-to-end validation; the held-out scores exclude the second-chance searches and the France rules, folds 8 and 9 also chose thresholds, and France has no labels. What we do have: close versions of our decision models score between 0.990 and 0.992 on held-out folds from fits that never saw the scored fold, the same range as the public 0.9913.

A0.8 Loss functions and log loss

IN ONE LINE

A **loss function** is one number that measures how wrong a model's predictions are on the training examples; training means adjusting the model to make that number small.

SIMPLE EXPLANATION

Imagine a quiz where you must also say how sure you are. Saying "99 % sure" and being wrong costs many penalty points; saying "60 % sure" and being wrong costs a few; being right and confident costs almost nothing. Over many questions, the best strategy is to be honest about your uncertainty. **Log loss** (also called **binary cross-entropy**) is exactly this penalty system, and every learned model in our pipeline is trained to collect as few penalty points as possible across its training pairs. The models never see F0.5 during training; they see log loss. We turn good scores into a good F0.5 afterwards, with the decision rule.

WORKED EXAMPLE (TOY)

For one pair, log loss = $-[y \times \ln(p) + (1 - y) \times \ln(1 - p)]$, where y is the label and p the predicted match probability.

Pair	Truth y	Predicted p	Loss
"Joes Pizza, 410 Main St" vs "Joe's Pizza, 410 Main Street"	1	0.90	$-\ln 0.90 = 0.105$

Pair	Truth y	Predicted p	Loss
"Patel Medical Store" (no address) vs "Patel Medical Store, 5 Station Road"	1	0.60	$-\ln 0.60 = 0.511$
"Starbucks, 200 Pine St" vs "Starbucks, 1500 Broadway"	0	0.30	$-\ln 0.70 = 0.357$
"Sunrise Pharmacy, 5 Park Ave" vs "Sunrise Bakery, 5 Park Ave"	0	0.90	$-\ln 0.10 = 2.303$
"Kumar Electricals" vs "Kumar Textiles"	0	0.999	$-\ln 0.001 = 6.908$

1. Add the five losses: 10.183. Average: $10.183 / 5 = 2.037$.
2. Without the last row the average is 0.819.
3. Lesson: one confidently wrong prediction dominates the total, so training pushes hardest on exactly those mistakes.

TECHNICAL EXPLANATION

Log loss is the negative log-likelihood of a Bernoulli model (a coin-flip model in which each pair is a match with probability p), so minimising the average log loss is **maximum-likelihood estimation**: choosing the parameters under which the observed labels are most probable. It is a **strictly proper scoring rule**: the expected loss is minimised only by predicting the true conditional probability, which is why models trained with it tend to produce probability-like scores on their training distribution. With $p = \text{sigmoid}(z)$, the gradient with respect to the logit is simply $\partial L / \partial z = p - y$, and the second derivative is $p \times (1 - p)$; gradient-boosting libraries build trees from exactly these two quantities (see Ensembles), and neural networks back-propagate $p - y$ (see Gradient descent). Implementations compute the loss from the logit directly ("with logits") for numerical stability. Other losses exist for other tasks: squared error for regression, hinge loss for support-vector machines, focal loss for heavy imbalance, pairwise or listwise losses for ranking. Our evaluation metric (per-business macro F0.5) is not differentiable and does not decompose over pairs, so we optimise a smooth **surrogate** (log loss) and then optimise the metric itself at decision time by choosing thresholds. **Validation loss** on a held-out fold is also the usual signal for early stopping.

HOW WE USED IT

Every learned model minimises log loss: the CatBoost ranker (loss "Logloss", monitored on fold 6 for early stopping), the four readers (binary cross-entropy on their single logit), the LightGBM decision models (binary objective; early stopping on fold-8 log loss with a patience of 100 rounds) and the LightGBM second-chance models (binary objective). F0.5 is used only to choose thresholds and to evaluate.

EXPERT NUANCE

Log loss is unbounded, so a few confidently wrong labels (label noise) can dominate training; that matters for pseudo-labels, which is one reason we kept only very confident French pseudo-labels (≥ 0.97 for matches; ≤ 0.03 for the French E5 reader's non-matches). Lower log loss does not guarantee a higher F0.5 at the chosen threshold, because log loss rewards good probabilities everywhere, including far from the threshold; that is why selection used the exact metric. Do not confuse the **loss** (what training minimises) with the **metric** (what we report).

REMEMBER

- Log loss = $-[y \ln p + (1 - y) \ln(1 - p)]$; confident mistakes are punished hardest.
- All our learned models minimise log loss; F0.5 chooses thresholds, it does not train anything.
- The gradient of log loss with respect to the logit is $p - y$.

JUDGE MIGHT ASK

- *Your models minimise log loss but the competition scores F0.5. How do you bridge the two?* Log loss is a smooth, proper objective that gives good probability-like scores; F0.5 per business is not differentiable and depends on whole businesses, so we do not train on it. We bridge the gap at decision time: on top

of one fixed 0.4 lead over the runner-up, the threshold and the consistency-check bars were chosen on held-out folds for the exact per-business macro F0.5.

A0.9 Gradient descent

IN ONE LINE

Gradient descent improves a model step by step by nudging every adjustable number in the direction that lowers the loss fastest.

SIMPLE EXPLANATION

You are on a foggy hillside and want to reach the valley floor. You cannot see far, but you can feel which way the ground slopes under your feet, so you take a small step downhill, feel again, and repeat. The slope is the **gradient**, the size of each step is the **learning rate**, and the valley floor is low loss. Steps that are too big overshoot and bounce around; steps that are too small take forever. A neural network such as our MiniLM reader has about 118 million adjustable numbers, so its hillside has 118 million directions, but calculus (a method called **backpropagation**) gives the slope in all of them at once.

WORKED EXAMPLE (TOY)

A one-feature model $p = \text{sigmoid}(w \times x + b)$, where x is name similarity. Two training pairs: A = "Joe's Pizza" vs "Joes Pizza" ($x = 0.9, y = 1$) and B = "Kumar Electricals" vs "Kumar Textiles" ($x = 0.2, y = 0$). Start with $w = 0$ and $b = 0$; learning rate 2.

1. Predictions: $p_A = p_B = \text{sigmoid}(0) = 0.5$. Average log loss = $\ln 2 = 0.693$.
2. Gradients, averaged over the two pairs: for w , the average of $(p - y) \times x = ((0.5 - 1) \times 0.9 + (0.5 - 0) \times 0.2) / 2 = (-0.45 + 0.10) / 2 = -0.175$; for b , the average of $(p - y) = (-0.5 + 0.5) / 2 = 0$.
3. Update (step against the gradient): $w = 0 - 2 \times (-0.175) = 0.35$; $b = 0 - 2 \times 0 = 0$.
4. New predictions: $p_A = \text{sigmoid}(0.315) = 0.578$; $p_B = \text{sigmoid}(0.07) = 0.517$. Loss = 0.638, down from 0.693.
5. Repeat: after the second step $w = 0.626, b = -0.096$ and the loss is 0.597; after the third $w = 0.871, b = -0.218$ and the loss is 0.561. The weight w keeps growing because high name similarity goes with the match, and the bias b turns negative so that low-similarity pairs are pushed towards "no match".

TECHNICAL EXPLANATION

The update is $\theta \leftarrow \theta - \eta \times \nabla L(\theta)$, with η the learning rate. **Mini-batch stochastic gradient descent** estimates ∇L on a small batch (ours: 128 pairs) instead of the whole dataset; one **epoch** is one pass over all training pairs. **Backpropagation** applies the chain rule of calculus layer by layer, from the output back to the input, and obtains every gradient in one backward pass that costs roughly twice the forward pass.

AdamW adapts the step for each parameter using running averages of the gradient (momentum) and of its square, and applies **weight decay** directly to the weights, decoupled from the gradient (see Regularisation). A **learning-rate schedule** with a linear **warm-up** (ours: the first 3 % of steps) followed by linear decay protects pre-trained weights from large early updates while Adam's statistics are still noisy.

Gradient clipping rescales the whole gradient when its norm exceeds a cap (ours: 1.0), preventing rare explosive steps. **bf16 autocast** runs most arithmetic (the large matrix multiplications) in 16-bit "brain floating point", which keeps the range of 32-bit numbers with less precision, for speed and memory; the stored weights stay 32-bit and our loss is computed in 32-bit from the logit. Gradient boosting is gradient descent in function space: each new tree is fitted to the negative gradient of the loss with respect to the current scores, and the boosting learning rate (0.05 for LightGBM, 0.075 for CatBoost) is the step size (see Ensembles).

HOW WE USED IT

All four readers: AdamW with weight decay 0.01, batches of 128, one epoch, 3 % linear warm-up then linear decay, gradient clipping at 1.0, bf16 autocast, length-bucketed batches. Learning rates: 5e-5 for the encoders of the MiniLM and E5 readers and five times that for the newly added head (2.5e-4, derived); 2e-5 for the encoders of the French MiniLM and French E5 readers, again with the head at five times that rate. The French E5 reader's fine-tune took 7,419 steps (949,616 pairs / 128 per batch). Tree models: learning rate 0.05 (LightGBM) and 0.075 (CatBoost).

EXPERT NUANCE

The head gets the higher learning rate because it starts from random weights while the encoder is already good; the low encoder rate and the warm-up limit damage to pre-trained knowledge. One epoch is our fixed recipe, and the Documentation reports no comparison with more epochs, so do not claim it is optimal. GPU training in bf16 is not bit-deterministic (Doc A), which is one reason a clean re-run from raw data reproduced 99.82 % of the submitted links rather than all of them. The smaller learning rate for the French fine-tunes (2e-5) is a standard guard against drifting away from what the reader already knew.

REMEMBER

- Step downhill: $\theta \leftarrow \theta - \text{learning rate} \times \text{gradient}$; backpropagation computes all the gradients.
- Readers: AdamW, learning rate 5e-5 (head $\times$ 5), 3 % warm-up, clipping 1.0, bf16, batch 128, one epoch.
- Boosting is gradient descent in which each step is a tree.

JUDGE MIGHT ASK

- *Why a higher learning rate for the head than for the encoder?* The encoder is pre-trained and needs only gentle adjustment, while the one-layer head starts from random weights, so the head learns five times faster: 2.5e-4 against 5e-5 for the main readers. Warm-up over the first 3 % of steps and clipping at norm 1.0 keep early updates from damaging the pre-trained weights.
- *Why only one epoch?* It is our fixed recipe: one pass over a large set of training pairs focused on hard records, starting from pre-trained encoders, which kept the training of both readers to about 2.1 hours in the clean run. We did not report a comparison with more epochs, so we do not claim that one epoch is optimal.

A0.10 Overfitting, underfitting, bias and variance

IN ONE LINE

Underfitting is a model too simple to capture the pattern; **overfitting** is a model that memorises its training examples, noise included, and does worse on new data.

SIMPLE EXPLANATION

A clerk who has learned only "same name means same business" underfits: they will link every Starbucks to every other Starbucks. A clerk who has memorised the answer sheet ("form 4471 goes with business 902") overfits: perfect on old forms, lost on new ones. **Bias** is the systematic error of an overly simple model (wrong in the same way every time); **variance** is how much the model would change if trained on a slightly different sample (wrong in a different way every time). A good model balances the two, and the only way to see the balance is to check on data the model did not train on. People can overfit too: if you try many variants and keep whichever scored best on the same test, part of the "improvement" may be luck.

WORKED EXAMPLE (TOY LEARNING CURVE)

A boosted model adds trees one by one; we watch the log loss on the training folds and on held-out fold 8.

Trees	Training loss	Fold-8 loss	Diagnosis
50	0.080	0.085	underfitting: both high
200	0.040	0.048	improving
600	0.020	0.041	best on fold 8
1,200	0.008	0.044	overfitting begins
2,000	0.003	0.052	training keeps improving, held-out gets worse

1. Training loss always falls as trees are added: it cannot tell you when to stop.
2. Fold-8 loss falls, bottoms out (600 trees here), then rises: the model has started to fit noise.
3. **Early stopping** keeps the model at the best held-out round and stops once there has been no improvement for a set number of rounds (ours: 100).

A second toy: in the lookup table of "What a model is", the cell "high similarity + same number" learned $6 / 6 = 1.00$ from six examples; with more data that cell might turn out to be 0.97. Trusting tiny groups is overfitting.

TECHNICAL EXPLANATION

For squared loss, expected test error decomposes exactly into $\text{bias}^2 + \text{variance} + \text{irreducible noise}$; the intuition carries over to classification. **Capacity** controls (tree depth or number of leaves, number of trees, minimum examples per leaf, network size, number of epochs) move a model along the bias-variance curve, and the diagnostic is the gap between training and validation performance. Variance can be reduced without adding bias by averaging models trained with different randomness (seed averaging, bagging; see Ensembles). Boosting lowers bias with every tree and therefore needs early stopping and regularisation to keep variance in check (see Regularisation). **Adaptive overfitting**, overfitting a validation set or a leaderboard through repeated selection, is a property of the research process rather than of one model: each comparison on the same data spends some of its credibility. Small score differences on a leaderboard can lie within noise, and the best of many uploads is biased upwards as an estimate of its own future score.

HOW WE USED IT

Decision models: early stopping on fold 8 (patience 100 rounds; at most 3,000 rounds for the US and India, 1,500 for France), at least 200 rows per leaf, L2 1.0, 0.9 feature and 0.8 row sampling, mean of five seeds. Ranker: early stopping after 180 rounds without improvement on fold 6. Readers: one epoch, dropout 0.1, weight decay 0.01. Process: held-out folds as the main evidence; 18 public uploads, with the final file chosen by public score and the late public gains small ($0.991114 \rightarrow 0.991300$). The decisions taken on public scores are listed in our Documentation: the US/India refit kept ($0.990621 \rightarrow 0.990676$); the France refit dropped (0.990726 against 0.990748 without it); the second French self-training round dropped (0.991277 against 0.991300).

EXPERT NUANCE

The largest overfitting risks in our system are process-level, not model-level: the France rules came from test-set analysis with some parameters chosen after seeing public scores, and the final file was picked among 18 uploads; we disclose both. Refitting the US/India decision models on folds 6-9 trades lower variance (more data) against an iteration count that can no longer be validated (the best iteration scaled by the row ratio). Replacing data-specific inputs with generic "what changed" features guards against fitting regularities of this particular training set; on the public board the generic version cost nothing and helped slightly ($0.990881 \rightarrow 0.990934$). Underfitting has a concrete face too: one similarity score with a

threshold cannot separate businesses whose names differ by a single word, which is why the system has several stages.

REMEMBER

- Underfit = too simple (high bias); overfit = memorised noise (high variance); check on held-out data.
- Our guards: early stopping, ≥ 200 rows per leaf, subsampling, L2, five-seed averaging, one epoch.
- The biggest risk is process-level: 18 uploads and France rules tuned with test-set analysis (both disclosed).

JUDGE MIGHT ASK

- *How do you know the decision model is not overfitting?* The number of trees is set by early stopping on a fold the model does not train on, and the fits use at least 200 rows per leaf, row and feature sampling, L2 and an average of five seeds. The held-out check scores fold 9 with fits that never saw records touching fold 9, and close versions of our decision models (the India recipe with 3 of its 5 seeds) score US 0.991212 and India 0.990046 there, in line with the public 0.9913.
- *You dropped the France refit because it lost 0.00002 on the public board. Isn't that noise?* It may well be, and we say so: 0.990726 against 0.990748 is a tiny difference, and there are no French labels to confirm it. We kept the simpler France fit with the better public score; it is one of the leaderboard-driven decisions our documentation lists.

A0.11 Regularisation

IN ONE LINE

Regularisation is any deliberate constraint that stops a model from fitting noise, trading a little training accuracy for better behaviour on new data.

SIMPLE EXPLANATION

Think of the rules you would give a junior detective to stop them jumping to conclusions. "Do not form a rule from fewer than 200 cases" (a minimum number of examples per leaf). "Prefer modest conclusions unless the evidence is strong" (a penalty on large values). "Five detectives each see a random 80 % of the files, then average their verdicts" (subsampling and averaging). "Stop studying when your mock-exam marks stop improving" (early stopping). Each of these makes the detective slightly worse at reciting old cases and better at solving new ones.

WORKED EXAMPLE (TOY)

How the L2 penalty shrinks a leaf's value in gradient boosting. With log loss, every training pair in a leaf contributes a gradient $g = p - y$ (the slope of its loss) and a **curvature** $h = p \times (1 - p)$ (the second derivative: how quickly that slope changes, a measure of how much evidence the pair carries); the leaf's value, added to the logit of every pair that lands in it, is $w = -G / (H + \lambda)$, where G and H are the sums over the leaf and λ is the L2 penalty.

1. A leaf with 4 pairs, all true matches, current $p = 0.5$: $G = 4 \times (0.5 - 1) = -2.0$ and $H = 4 \times 0.25 = 1.0$.
2. $\lambda = 0$: $w = 2.0 / 1.0 = 2.0$. $\lambda = 1$ (our LightGBM setting): $w = 2.0 / 2.0 = 1.0$. $\lambda = 5$: $w = 2.0 / 6.0 = 0.33$.
3. A leaf with 400 such pairs: $G = -200$ and $H = 100$. With $\lambda = 1$: $w = 200 / 101 = 1.98$, almost unchanged.
4. Lesson: the penalty mostly shrinks leaves built on little evidence. Our decision models also forbid leaves with fewer than 200 rows, so the 4-pair leaf could not exist at all.

TECHNICAL EXPLANATION

Explicit penalties: L2 (ridge) adds $\lambda \times \sum w^2$ to the objective and shrinks values; L1 adds $\lambda \times \sum |w|$ and pushes values to exactly zero. **Weight decay** in AdamW subtracts $\eta \times \lambda \times w$ from every weight at each step, decoupled from Adam's adaptive scaling, which is not the same as adding an L2 term to the loss under Adam. **Dropout** randomly zeroes activations with probability p during training (ours: 0.1 before each reader's output layer), so the network cannot lean on any single unit. **Structural limits:** maximum leaves or depth, minimum rows per leaf. **Stochastic** regularisation: row subsampling at each boosting iteration (**stochastic gradient boosting**) and feature subsampling per tree. **Shrinkage:** the boosting learning rate scales each tree's contribution, and many small steps generalise better than a few large ones. **Early stopping** limits the effective number of trees or epochs. **Data-level** regularisation: in self-training, **replay** of labelled source-domain pairs keeps the model close to the task it already solves.

HOW WE USED IT

- LightGBM decision models: L2 1.0, at least 200 rows per leaf, at most 127 leaves, feature fraction 0.9, row fraction 0.8 at every iteration, learning rate 0.05, early stopping on fold 8, mean of 5 seeds.
- CatBoost ranker: L2 5, depth 7, learning rate 0.075, early stopping after 180 rounds on fold 6.
- Readers: weight decay 0.01, dropout 0.1, one epoch, warm-up, gradient clipping 1.0.
- French readers: a smaller learning rate (2e-5), replay of labelled US/India pairs (500,000 for the French MiniLM reader; 474,808, one to one, for the French E5 reader) and only very confident pseudo-labels.

EXPERT NUANCE

A floor of 200 rows per leaf with up to 127 leaves means the decision model cannot carve out tiny groups; the trade-off is that a genuinely rare pattern may be smoothed away. Gradient clipping is often listed as regularisation, but its main job is optimisation stability. Regularisation strengths interact (learning rate, number of trees and L2 together set the effective complexity), so they are best judged as a recipe; the Documentation reports ours as fixed values and records no sweep, so do not claim they are optimal. Replay is regularisation in function space: it penalises forgetting rather than large weights.

REMEMBER

- Regularisation = constraints against fitting noise: penalties, limits, randomness, early stopping, averaging, replay.
- Decision models: L2 1.0, ≥ 200 rows per leaf, 0.9 / 0.8 sampling, learning rate 0.05, early stopping, five seeds.
- Readers: weight decay 0.01, dropout 0.1, one epoch; the French readers add replay and learning rate 2e-5.

JUDGE MIGHT ASK

- *Which regularisers does the decision model use, and why at least 200 rows per leaf?* L2 1.0, at least 200 rows per leaf, 90 % of features and 80 % of rows sampled, learning rate 0.05, early stopping on fold 8 and the mean of five seeds. The 200-row floor stops a 127-leaf tree from building rules on a handful of pairs that could be noise; it is part of a fixed recipe, and we did not report a sweep of that value.
- *What stops the French readers from drifting while they learn from their own guesses?* We kept only very confident French pseudo-labels (for the French E5 reader, at least 0.97 for a match and at most 0.03 for a non-match), mixed them about one to one with labelled US and Indian pairs, and used a smaller learning rate, 2e-5 instead of 5e-5. A written check before adoption confirmed that the French E5 reader's US and Indian quality stayed within tolerance, although, as we disclose, it failed two of its other checks.

A0.12 Decision trees

IN ONE LINE

A **decision tree** is a flowchart of yes/no questions about the features, learned from data, that ends in a score.

SIMPLE EXPLANATION

Think of a triage nurse, or the game "twenty questions": "Is the name nearly identical? If yes, is the house number the same?" Each question splits the examples into two groups, and the tree chooses which question to ask at each point by trying them all and keeping the one that best separates matches from non-matches in the training data. At the end of each path (a **leaf**) sits a score, for example "most training pairs that ended here were matches". Trees are good at **interactions**, rules of the form "high name similarity is a good sign, unless the address is different". A single tree is simple but unstable, so in practice we use many trees together (our caps are 1,600 trees for the ranker and 3,000 rounds per US or Indian decision-model fit; see Ensembles).

WORKED EXAMPLE (TOY)

Eight labelled pairs, two features.

#	Incoming record	Business	Name similarity	Same house number	Match?
1	Sharma Traders, 12 MG Road, Pune	Sharma Traders Pvt Ltd, 12 MG Road, Pune	0.95	yes	1
2	Joes Pizza, 410 Main St, Boston	Joe's Pizza, 410 Main Street, Boston	0.90	yes	1
3	Cafe Lumiere, 8 Rue Victor Hugo, Lyon	Café Lumière SARL, 8 Rue Victor Hugo, Lyon	0.88	yes	1
4	Sunrise Pharmacy, 5 Park Ave, Austin	Sunrise Bakery, 5 Park Ave, Austin	0.55	yes	0
5	Starbucks, 200 Pine St, Seattle	Starbucks, 1500 Broadway, Seattle	0.97	no	0
6	Patel Medical Store (no address)	Patel Medical Store, 5 Station Road, Ahmedabad	0.92	no (missing)	1
7	Green Leaf Salon, 77 Elm St, Austin	Blue Ocean Travels, 12 Oak St, Austin	0.30	no	0
8	Kumar Electricals, 3 Ring Road, Delhi	Kumar Textiles, 41 Chandni Chowk, Delhi	0.45	no	0

We measure how mixed a group is with **Gini impurity** = $1 - (\text{share of matches})^2 - (\text{share of non-matches})^2$: 0 means pure, 0.5 means a 50/50 mix.

1. Root: 4 matches and 4 non-matches, $\text{Gini} = 1 - 0.25 - 0.25 = 0.5$.
2. Candidate split A, "same house number?": yes = {1, 2, 3, 4} (3 matches, 1 not), $\text{Gini} = 1 - 0.5625 - 0.0625 = 0.375$; no = {5, 6, 7, 8} (1 match, 3 not), $\text{Gini} = 0.375$. Weighted average = 0.375, so the impurity drops by 0.125.
3. Candidate split B, "name similarity ≥ 0.85 ?" (any cut between 0.55 and 0.88 gives the same groups; every other cut does worse): yes = {1, 2, 3, 5, 6} (4 matches, 1 not), $\text{Gini} = 1 - 0.64 - 0.04 = 0.32$; no = {4, 7, 8} (all non-matches), $\text{Gini} = 0$. Weighted = $5/8 \times 0.32 = 0.20$, a drop of 0.30. Split B wins.
4. Inside the "yes" branch, "same house number?" separates {1, 2, 3} (all matches) from {5, 6} (one each). A third question, "does the record have an address at all?", separates the Starbucks branch (a different address) from Patel Medical Store (no address).
5. The learned tree: similar names and the same number $\rightarrow$ match; similar names and no address $\rightarrow$ match; similar names and a different address $\rightarrow$ no match; dissimilar names $\rightarrow$ no match.

TECHNICAL EXPLANATION

Trees are grown greedily (the CART algorithm): at each node, for every feature and candidate threshold, compute the impurity decrease, take the best, and recurse until a stopping rule. Classification trees use Gini or entropy; LightGBM's gradient-boosted trees choose splits by the second-order gain $\text{Gain} = \frac{1}{2} \times [G_L^2 / (H_L + \lambda) + G_R^2 / (H_R + \lambda) - (G_L + G_R)^2 / (H_L + H_R + \lambda)]$, where G and H are the sums of the loss's gradients and curvatures in each child (see Regularisation for the matching leaf value $-G / (H + \lambda)$).

Histogram methods (LightGBM) bucket each feature into a limited number of bins once, before training; at each node one pass over the rows fills the bins with gradient and curvature sums, and the split search then scans only the bins (about 255 per feature by default) instead of sorting every value. Growth policies differ: LightGBM grows **leaf-wise** (always split the leaf with the largest gain, up to 127 leaves for us), while CatBoost by default grows **oblivious (symmetric) trees**, in which every node at a given depth uses the same split (depth 7 gives $2^7 = 128$ leaves); these are fast at inference and act as a regulariser. Missing values (NaN) get a learned default direction in LightGBM, and sentinel values (our -1 or -2 , placed below every real value of the feature) can be cut off by one split. Splits depend only on the order of a feature's values, so trees are invariant to monotone transforms (probability or log-odds, raw count or its logarithm give the same tree) and need no feature scaling. Weaknesses: a single tree has high variance, splits are axis-aligned, outputs are piecewise constant, and trees cannot extrapolate beyond the range seen in training.

HOW WE USED IT

Never a single tree; always ensembles: the CatBoost ranker (up to 1,600 symmetric trees of depth 7), the LightGBM decision models (127 leaves, at least 200 rows per leaf) and the LightGBM second-chance models (address rescue: a filter with 15 leaves and 200 rounds, and a scorer with 31 leaves and 300 rounds, averaged over seeds 7, 8 and 9).

EXPERT NUANCE

Gradient-boosted trees suit our decision problem because the inputs are about two hundred heterogeneous tabular features on different scales, with sentinel values and strong interactions ("same house number but one word swapped"); this is the setting where boosted trees are the standard strong choice. Because trees cannot extrapolate, an input larger than anything seen in training lands in the same leaf as the largest training values; this can matter for France, whose decision model was trained with MiniLM reader logits and receives a blend of three readers at test. Tree feature importances exist, but we did not prepare a ranking for this talk: say so rather than guess.

REMEMBER

- A split is chosen by trying every feature and threshold and keeping the biggest impurity (or loss) reduction.
- Trees need no feature scaling, handle missing values and learn interactions; they cannot extrapolate.
- LightGBM grows leaf-wise (127 leaves for us); CatBoost grows symmetric trees (depth 7 for us).

JUDGE MIGHT ASK

- *Why gradient-boosted trees for the decision model rather than a neural network?* The inputs are about two hundred tabular features on very different scales, with special values for "missing", and the signal lies in interactions such as "same house number but a word swapped"; boosted trees handle that natively, are fast enough for five seed models per pair, and are easy to inspect. The neural part of our system is the readers, whose scores enter the trees as features.
- *Why both CatBoost and LightGBM?* They do different jobs: CatBoost is the first-stage ranker on the 76 index features, trained on GPU with symmetric depth-7 trees, and LightGBM is the per-country decision model and the second-chance models. We have no head-to-head comparison of the two libraries to quote, so we do not claim either would be better in the other's role.

A0.13 Ensembles: bagging and boosting

IN ONE LINE

An **ensemble** combines many models; **bagging** averages models trained independently to reduce variance, and **boosting** adds models one after another, each correcting the errors of those before it.

SIMPLE EXPLANATION

Bagging is a panel of judges who each read a random part of the evidence and then vote; their individual quirks cancel out. Boosting is a relay of specialists: the first makes a rough guess, the second concentrates on the cases the first got wrong, the third on what is still wrong, and their corrections add up. **Gradient boosting** (CatBoost, LightGBM) is the relay. We also use the panel idea on top of it: five boosted decision models trained with different random seeds, averaged. A third kind, **stacking**, trains a final model to combine other models' outputs; our decision model does this with the ranker and reader scores.

WORKED EXAMPLE (TOY)

Three small trees add up their scores, in log-odds, for two candidates of "Legacy Marketing Enterprises (Inc)", which has no address. The leaf values already include the learning rate; the sum starts at 0.

Tree	Question and leaf values	Inc, Windsor	LLC, Irondequoit
1	name similarity ≥ 0.85 ? yes +1.2 / no -1.5	+1.2	+1.2
2	record address against business address: same number +0.8 / record has no address -0.4 / different -1.0	-0.4	-0.4
3	legal form changed? yes -1.6 / no +0.2	+0.2	-1.6
Sum (log-odds)		1.0	-0.8
sigmoid(sum)		0.731	0.310

1. Each tree adds a correction; the sum is a logit; sigmoid turns it into a probability.
2. Tree 3 is what separates the two: the only difference is the legal form.
3. Panel step (toy): five seed models give 0.73, 0.70, 0.76, 0.74 and 0.72 for the Inc pair; their mean, 0.73, is the final score.

The real decision models add up many more trees (up to 3,000 rounds per seed for the US and India, chosen by early stopping) over about two hundred features, and gave 0.965022 (Inc) and 0.000007 (LLC).

TECHNICAL EXPLANATION

Bagging (bootstrap aggregating) trains B models on **bootstrap resamples** (samples of the same size as the data, drawn with replacement) and averages them; for models with variance σ^2 and pairwise correlation ρ , the average has variance $\rho\sigma^2 + (1 - \rho)\sigma^2/B$, so the gain depends on decorrelation (random forests add feature subsampling for that reason). **Gradient boosting** builds $F_m(x) = F_{(m-1)}(x) + v \times h_m(x)$, where each new tree h_m is fitted to the negative gradient of the loss at the current predictions (for log loss, $y - p$), with v the learning rate; LightGBM also uses the curvature $p(1 - p)$ to set leaf values (a Newton step). Boosting mainly reduces bias and needs early stopping. LightGBM's settings "bagging_fraction 0.8, bagging_freq 1" mean sampling 80 % of rows without replacement at every iteration, which is **stochastic gradient boosting**, not bagging in Breiman's sense. **Seed averaging** of boosted models is a bagging-like reduction of the variance that comes from row and feature sampling. **Stacking** trains a meta-model on base-model outputs, and it requires that the base-model scores used for training do not come from the base models' own training data (see Training, validation and test data). **Blending** is a fixed weighted average; our French reader blend is a blend of logits.

HOW WE USED IT

Boosting: the CatBoost ranker and every LightGBM model. Seed averaging: each per-country decision model (US, India, France) is the mean of five seed models' probabilities (seeds 1-5; the default route uses a single fit); the address-rescue scorer is the mean of seeds 7, 8 and 9. Stacking: the decision model takes the ranker probability and the reader scores (plus features), with the ranker and readers trained on folds 0-5 and the decision models on folds 6-9. Blending: France's reader input = 0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5 logits; in the France example, $0.35 \times 7.5 + 0.15 \times 8.9375 + 0.5 \times 10.8125 = 9.3719$ (Doc D.3).

EXPERT NUANCE

Averaging five seeds costs five small tree-model calls per pair, which is cheap next to the readers. Its effect is not isolated: the held-out table uses 3 of the 5 seeds, and the 5-seed models entered the final file together with the French E5 reader (public 0.991114 → 0.991300), so we claim no separate number for it. Boosting reduces bias and averaging reduces variance, so a seed-averaged boosted model gets some of both. Blending in logit space keeps a strong "no" from one reader strong: in the France example, for the same-name business in another city, the French E5 reader's -10.8125 outweighs the MiniLM reader's +7.125 and the blend is -3.2008 (sigmoid 0.039). Averaging the same three readers' probabilities with the same weights would give 0.369 (derived: $0.35 \times 0.9992 + 0.15 \times 0.128 + 0.5 \times 0.00002$), because probabilities are capped at 0 and 1, so in probability space the French E5 reader's extreme confidence counts for no more than a plain "no", whereas in logit space it does.

REMEMBER

- Bagging = independent models averaged (variance down); boosting = sequential corrections (bias down).
- LightGBM's "bagging" setting is row subsampling inside boosting; our real averaging is the five seeds.
- Stacking = the decision model learns from the ranker's and readers' scores, trained on different folds.

JUDGE MIGHT ASK

- *What does averaging five seeds buy you, and did you measure it?* Our LightGBM settings sample 80 % of rows at every iteration and 90 % of features, so single fits vary; the mean of five seeds reduces that variance for five small model calls per pair. Honestly, its effect is not isolated: the 5-seed models entered the final file together with the French E5 reader (public 0.991114 → 0.991300), so we do not claim a separate number.
- *Your decision model is stacked on other models' scores. How did you avoid leakage?* The ranker and readers were trained on folds 0-5 and the decision models on folds 6-9, so the scores the decision model learns from come from models that never saw those records. Otherwise the reader scores would be over-confident on their own training records and the decision model would learn to over-trust them.

A0.14 Neural networks

IN ONE LINE

A **neural network** is a stack of simple layers; each multiplies its inputs by learned **weights**, adds them up, and passes the result through a small non-linear **activation** function, and together the layers can learn very complex patterns.

SIMPLE EXPLANATION

Picture rows of clerks. The first row looks at the raw inputs and each clerk calls out a number meaning "how strongly do I see my pattern?"; the second row listens to the first and combines what it hears; and so on, until a final clerk gives the answer. Each clerk has a volume knob for every voice they listen to (the weights) plus a personal starting level (the **bias**), and learning means turning millions of knobs so that the final answer is right (see Gradient descent). The activation is a rule such as "stay silent unless the combined signal is positive"; without it, any stack of layers would collapse into one simple weighted sum. Our readers are networks of a special kind, called **transformers**, with 12 layers.

WORKED EXAMPLE (TOY)

Two inputs, x_1 = name similarity and x_2 = address similarity; two hidden units with **ReLU** activation ($\text{ReLU}(z) = \max(0, z)$); one output with sigmoid.

- $h_1 = \text{ReLU}(2.0 \times x_1 + 1.0 \times x_2 - 1.0)$; $h_2 = \text{ReLU}(-1.0 \times x_1 + 3.0 \times x_2 - 0.5)$; $z = 1.5 \times h_1 + 2.0 \times h_2 - 2.0$; $p = \text{sigmoid}(z)$.
1. Pair 1, a chain branch, "Starbucks, 200 Pine St" vs "Starbucks, 1500 Broadway": $x_1 = 0.9$, $x_2 = 0.2$. $h_1 = \text{ReLU}(1.8 + 0.2 - 1.0) = 1.0$; $h_2 = \text{ReLU}(-0.9 + 0.6 - 0.5) = \text{ReLU}(-0.8) = 0$; $z = 1.5 \times 1.0 + 0 - 2.0 = -0.5$; $p = 0.378$.
 2. Pair 2, a true match, "Joes Pizza, 410 Main St" vs "Joe's Pizza, 410 Main Street": $x_1 = 0.9$, $x_2 = 0.95$. $h_1 = \text{ReLU}(1.8 + 0.95 - 1.0) = 1.75$; $h_2 = \text{ReLU}(-0.9 + 2.85 - 0.5) = 1.45$; $z = 2.625 + 2.9 - 2.0 = 3.525$; $p = 0.971$.
 3. Unit h_2 has, in effect, learned to fire for address agreement.
 4. Parameter count: 4 hidden weights + 2 hidden biases + 2 output weights + 1 output bias = 9. Our MiniLM reader's output layer alone has 385 parameters (384 weights + 1 bias, so its pooled vector has 384 numbers, derived), on top of a 117,653,760-parameter encoder.

TECHNICAL EXPLANATION

A dense layer computes $h = \varphi(Wx + b)$; stacking layers with a non-linear φ (ReLU, **GELU**, a smooth version of ReLU, or tanh) gives universal function approximators (with enough hidden units, any continuous function on a bounded input range can be approximated as closely as wanted), and depth lets later layers build on features computed by earlier ones. A **transformer encoder** first maps each token to a vector (token embedding plus position embedding) and then applies L identical blocks, each made of **multi-head self-attention** and a position-wise feed-forward network (a small two-layer network applied to every token separately), with **residual connections** (each block adds its output to its input, which keeps gradients flowing through many layers) and **layer normalisation** (rescaling each token vector to a standard mean and spread). In self-attention each token forms a query q , a key k and a value v , and token i reads from token j with weight softmax over j of $(q_i \cdot k_j / \sqrt{d})$, so every token can gather information from every other token; the cost grows with the square of the sequence length. A **cross-encoder** feeds both texts as one sequence, so the tokens of the incoming record attend to the tokens of the business in every layer; a **bi-encoder** encodes each text separately and compares the two vectors only at the end. Our readers pool the last layer's token vectors by **masked mean pooling** (the average over real tokens, ignoring padding), apply dropout 0.1 and a linear layer, and output one logit. Background, from the models' configuration files rather than our Documentation: both encoders have 12 layers and 12 attention heads with GELU activations, with hidden size 384 (MiniLM) and 768 (E5).

HOW WE USED IT

Four readers, all cross-encoders: the MiniLM reader (117,653,760 + 385 parameters, Apache-2.0), the E5 reader (278,043,648 + 769, MIT) and their two French fine-tunes. Input: the raw "name | address" text of both sides, longest-first truncation to 128 tokens; then masked mean pooling, dropout 0.1, a linear head, one logit. They score only the shortlist (11,687,317 pairs). The pre-trained MiniLM, not fine-tuned, is also

used to embed single words for three meaning-similarity features (see Embeddings). The decision models are not neural.

EXPERT NUANCE

The cross-encoder's joint attention is why it can notice that exactly one word or the legal form changed; the price is one full network pass per pair with nothing pre-computable, hence the shortlist. Background, derived from the model configuration: about 96 million of the MiniLM's 117,653,760 parameters (about 82 %) are its multilingual vocabulary table, so its computation per token is that of a small model. A larger reader, XLM-R large (about 560 million parameters, MIT), was tried during development and discarded; no number was reported, so do not quote one. Mean pooling rather than a single special-token vector is a design choice; we use mean pooling.

REMEMBER

- Layer = weights $\times$ inputs + bias, then a non-linear activation; learning tunes the weights.
- Transformer = token vectors + 12 layers of self-attention; a cross-encoder reads both records together.
- Our readers: 117.7 million (MiniLM) and 278.0 million (E5) parameters, 128 tokens, one logit per pair.

JUDGE MIGHT ASK

- *What is a cross-encoder, and why is it better here than comparing two embeddings?* A cross-encoder reads the incoming record and the business as one input, so every word of one side can attend to every word of the other in all 12 layers, and it outputs one match score. A bi-encoder squeezes each side into one vector first and can miss that exactly one word or the legal form changed; the price is one model pass per pair, which is why the readers see only the shortlist, about 1.17 pairs per incoming record.
- *You were allowed up to 8 billion parameters. Why such small readers?* Cost: about 11.7 million shortlist pairs must be read, and scoring them with two readers per pair took about 1.4 hours on one 16 GB GPU in our clean run. Small fine-tuned readers plus the decision model were accurate enough; we tried a larger XLM-R reader of about 560 million parameters during development and dropped it, with no reported number.

A0.15 Tokens and tokenisation

IN ONE LINE

Tokenisation cuts text into pieces (**tokens**) from a fixed vocabulary so that a network can turn each piece into numbers; modern tokenisers use **subwords**, so rare words are spelled out from common fragments.

SIMPLE EXPLANATION

A neural network cannot read letters; it needs a dictionary of known pieces, each with an ID number. A dictionary of whole words would be enormous and would still miss new names such as "Lenglen" or "Ghatkopar"; a dictionary of single letters would make every text very long. Subword tokenisers are the compromise: frequent words are one piece, and rare words are split into a few known fragments. A multilingual tokeniser keeps pieces for Latin, Devanagari and many other scripts in one vocabulary, which is why one reader can read US, Indian and French records without any transliteration. Note that "token" has a second meaning in our pipeline: in the index lookup, a token is simply a whole cleaned word.

WORKED EXAMPLE (THE REAL MINILM TOKENIZER, RUN ON OUR EXAMPLE NAMES FOR THIS GUIDE; "_" MARKS THE START OF A WORD)

Text	Tokens	Count
Legacy Marketing Enterprises (Inc)	__Lega · cy · __Marketing · __Enterprise · s · __ (· In · c ·)	9
Legacy Marketing Industries Inc	__Lega · cy · __Marketing · __Industrie · s · __Inc	6
TQ COMITE SARL	__ · TQ · __COM · ITE · __SAR · L	6
विजय एंटरप्राइजेज प्राइवेट लिमिटेड	__विजय · __ · एं · टर · प्र · ाइज · ेज · __प्र · ाइ · वे · ट · __लिमिटेड	12

1. Frequent words such as "Marketing" are single tokens.
2. "Enterprises" becomes "Enterprise" + "s", so the singular and the plural share a piece.
3. "Inc" in brackets becomes "In" + "c", while a free-standing "Inc" is one token: punctuation changes what the model sees.
4. "विजय" (vijay) is one token, while "एंटरप्राइजेज" (enterprises) needs six.
5. For a pair, the two texts are joined with special tokens: `<s> record text </s><s> business text </s>`. An illustrative US pair, "Legacy Marketing Enterprises (Inc) | " with "Legacy Marketing Enterprises Inc | 46 Lincoln Way, Windsor, CT", comes to 31 tokens, well under our 128-token limit.

TECHNICAL EXPLANATION

Both of our base encoders use the XLM-RoBERTa **SentencePiece** vocabulary (background: a **unigram language-model** tokenizer with a vocabulary of about 250,000 pieces). A unigram tokenizer keeps the vocabulary that maximises the likelihood of a training corpus and segments a new string into its most probable sequence of pieces (a **Viterbi search**, dynamic programming that finds the best split without listing every possible one); BPE and WordPiece build their vocabularies by greedy merges instead. Each token ID indexes a row of the embedding table (see Embeddings). Pairs are encoded in the RoBERTa format shown above, and **longest-first truncation** to 128 tokens removes tokens from the end of whichever side is currently longer. Batches are padded to their longest sequence and an **attention mask** marks the real tokens; grouping sequences of similar length into the same batch (our training sorts by length within chunks of 100 batches, then shuffles the batch order) reduces wasted padding. Attention cost grows with the square of the sequence length, so the 128-token cap bounds the cost per pair. The tokenizer is cased: "COMITE" and "comite" become different pieces, so the readers see case information that our cleaning would have removed.

HOW WE USED IT

All four readers use the MiniLM tokenizer (the E5 readers too), on the raw text "name | address" of each side, not on cleaned text; longest-first truncation to 128 tokens; length-bucketed batches of 128. Cleaning (lower case, "&" to "and", French street types, "S.A.R.L." to "sarl") and the Indic-to-Latin word list serve the index lookup and the features, which work on whole words, letter 3- and 4-grams and a sound key, not on subword tokens (the three meaning-similarity features pass whole words through the frozen MiniLM, which splits them into subwords internally).

EXPERT NUANCE

Scripts and spellings with less vocabulary coverage split into more pieces (one Devanagari word above took six), which lengthens sequences and spreads meaning over fragments. Feeding raw text to the readers is deliberate: it keeps information that cleaning throws away (case, punctuation, the spelling of the legal form), while the cleaned view drives recall in the index. Native-script names are a retrieval problem first: the Latin index does not retrieve them, so the native-script second-chance search finds candidates by transliterated name within the same state. No E5 "query:" or "passage:" prefixes are used, because the E5 model is fine-tuned as a cross-encoder, not used as an embedding model.

REMEMBER

- Subword tokens: common words whole, rare words in fragments; one multilingual vocabulary covers all our scripts.
- Readers read raw text, at most 128 tokens per pair; the index uses cleaned whole words and letter pieces.
- Punctuation and case change the tokens ("(Inc)" vs "Inc"; "COMITE" vs "comite").

JUDGE MIGHT ASK

- *How do your readers handle Devanagari names and French accents?* They read the raw text through a multilingual subword tokenizer whose vocabulary covers Latin and Indian scripts, so the readers need no transliteration; the cleaning rules and the transliteration word list are for the index lookup and the features. When the Latin index misses a native-script name altogether, the native-script second-chance search finds candidates by transliterated name within the same state; it added 9,759 links in India.

A0.16 Embeddings and cosine similarity

IN ONE LINE

An **embedding** turns a piece of text into a list of numbers (a **vector**) such that texts with similar meaning end up close together; **cosine similarity** measures how closely two vectors point in the same direction.

SIMPLE EXPLANATION

Imagine a map on which every word has a position, with similar words close together: "enterprises" near "industries", both far from "pharmacy". An embedding is a word's coordinates on such a map, except that the map has hundreds of dimensions instead of two. Cosine similarity asks whether two arrows from the centre point the same way: 1 means the same direction, 0 means unrelated, negative means opposite. This lets a computer see that two different words mean nearly the same thing even when they share no letters. But "similar meaning" is not "same business": "Enterprises" and "Industries" are close in meaning, yet they can name different companies.

WORKED EXAMPLE

Toy three-number embeddings: enterprises = (0.8, 0.5, 0.1); industries = (0.6, 0.7, 0.2); pharmacy = (0.1, 0.2, 0.9). $\text{Cosine}(a, b) = (a \cdot b) / (|a| \times |b|)$, where $a \cdot b$ multiplies matching positions and adds them up, and $|a| = \sqrt{a \cdot a}$ is the length.

1. enterprises · industries = $0.8 \times 0.6 + 0.5 \times 0.7 + 0.1 \times 0.2 = 0.48 + 0.35 + 0.02 = 0.85$.
2. $|enterprises| = \sqrt{0.64 + 0.25 + 0.01} = 0.949$; $|industries| = \sqrt{0.36 + 0.49 + 0.04} = 0.943$.
3. $\text{Cosine} = 0.85 / (0.949 \times 0.943) = 0.950$: very similar.
4. enterprises · pharmacy = $0.08 + 0.10 + 0.09 = 0.27$; $|pharmacy| = 0.927$; cosine = $0.27 / (0.949 \times 0.927) = 0.307$: not similar.
5. The real value in our US example: the pre-trained MiniLM gives 0.77 for the replaced pair Enterprises → Industries (Doc D.1).

TECHNICAL EXPLANATION

At the bottom of every transformer is a lookup table that maps each token ID to a learned vector (the token embedding); after the layers, each token has a **contextual** vector, and pooling (for example a masked mean) gives one vector per text. Sentence-embedding models are further trained, typically with **contrastive** objectives on text pairs (pull the vectors of related texts together and push unrelated texts in the same batch apart), so that paraphrases land close together (background: the paraphrase-multilingual MiniLM was trained for multilingual paraphrase similarity, and multilingual-e5 with contrastive training on

text pairs). Cosine similarity is the dot product of unit-length vectors; for normalised vectors, ranking by cosine is the same as ranking by Euclidean distance, because $|a - b|^2 = 2 - 2 \times \text{cosine}$. A **bi-encoder** system embeds every business once and retrieves nearest neighbours with an approximate nearest-neighbour index (for example FAISS or HNSW); this is the usual "dense blocking". A cross-encoder reader does not compare two embeddings at all. Embedding spaces are often anisotropic (even unrelated words have positive cosines), so raw cosines are best read relative to each other, and trees learn their own thresholds on such features. Embedding similarity measures relatedness of meaning, not identity of the thing named.

HOW WE USED IT

Three "what changed" features use the pre-trained MiniLM, not fine-tuned, to embed single words as unit vectors: the cosine of the first new word against the first dropped word; the best cosine of the first new word to any business word; and the best cosine of the first dropped word to any record word (-2 when not applicable). Inside the readers there are token embeddings and contextual vectors, but the readers are cross-encoders. We do not use vector search for blocking.

EXPERT NUANCE

Embedding single words out of context with a sentence encoder is a cheap, multilingual signal that tells the decision model whether a swapped word is a near-synonym or a change of business type; it is one input among about two hundred, never a decision on its own. Dense blocking was not benchmarked, so we cannot claim it would be worse; our index keeps the true business for 99.37 % (US) and 98.51 % (India) of held-out fold-9 records, and vector search would be a natural extra channel for brand or "doing business as" names that share few words. Keeping the word encoder frozen gives a stable, general notion of meaning that cannot overfit our labels.

REMEMBER

- Embedding = text as a vector; cosine = $a \cdot b / (|a| \times |b|)$: 1 same direction, 0 unrelated.
- We use frozen MiniLM word embeddings in three meaning-similarity features (Enterprises vs Industries = 0.77).
- No vector search for blocking: not benchmarked; a natural extra channel for brand names.

JUDGE MIGHT ASK

- *Where do embeddings appear in your system, and why no vector search for blocking?* Inside the readers, and in three "what changed" features where a frozen pre-trained MiniLM embeds the replaced, added and dropped words. We did not benchmark a dense retriever, so we cannot claim it would be worse; our index keeps the true business for 99.37 % of US and 98.51 % of Indian held-out records, and vector search would be a natural extra channel for brand or "doing business as" names.
- *Enterprises and Industries have a cosine of 0.77. Doesn't that push the model to link them?* No; meaning similarity is one input among about two hundred. The same pair also shows one word replaced in place and an IDF-weighted word overlap of 0.52, and the decision model gave that business 0.000088 while the identical-name Inc business scored 0.965022.

A0.17 Pre-trained and fine-tuned models

IN ONE LINE

A **pre-trained** model has already learned general language skills from a large amount of text; **fine-tuning** continues its training briefly on our labelled examples so that it specialises in our task.

SIMPLE EXPLANATION

Hiring an experienced multilingual clerk who already reads English, Hindi and French is pre-training; giving them a short on-the-job course on matching business records is fine-tuning. That is far faster and better than teaching someone to read from scratch using only matching examples. Fine-tuning changes the clerk's habits a little, so you must not overdo it, or they may forget general skills. We even fine-tuned twice: first on US and Indian examples, then on French examples, for which there were no answers, using the models' own most confident guesses (**pseudo-labels**); learning from your own confident guesses is called **self-training**.

WORKED EXAMPLE (THE LIFE STORY OF OUR FRENCH E5 READER)

1. Start: intfloat/multilingual-e5-base, MIT licence, 278,043,648 parameters, pre-trained by its authors and downloaded once at a pinned revision.
2. Fine-tune 1, giving the E5 reader: add a 769-parameter head and train on labelled US/India pairs of folds 0-5 for one epoch at learning rate 5e-5.
3. Make French pseudo-labels: on the France shortlist, confidence $p = \text{mean}(\text{sigmoid}(\text{MiniLM reader logit}), \text{sigmoid}(\text{French MiniLM reader logit}))$. Positive: a record's best pair with $p \geq 0.97$ while every other pair of that record is below 0.5. Negative: the record's highest- p pair among those with $p \leq 0.03$. At most 250,000 per class; 5 % of records held out for checking.
4. Fine-tune 2, giving the French E5 reader: 474,808 French pseudo-labelled pairs + 474,808 labelled US/India pairs, learning rate 2e-5, 7,419 steps (949,616 pairs / 128).
5. Checks before adoption: agreement with the held-out French pseudo-labels 0.999206 (pass); US/India quality within tolerance (pass); two other written checks failed (see the nuance below).

By contrast, the CatBoost ranker, the decision models and the second-chance models start from nothing (trained from scratch on the provided data), and the plain pre-trained MiniLM is also used frozen for the word-meaning features.

TECHNICAL EXPLANATION

Pre-training learns general representations with **self-supervised** objectives on large corpora: tasks whose answers come from the text itself, such as predicting masked words, so no human labels are needed.

(Background: both of our starting checkpoints come from models pre-trained on large multilingual text and were then trained so that similar sentences get similar vectors.) **Transfer learning** works because lower layers encode general lexical and syntactic features and upper layers adapt quickly to a new task.

Full fine-tuning, updating all weights with a new task head and a small learning rate, is what we did; parameter-efficient alternatives (adapters, LoRA) update only a small subset. The risks are **catastrophic forgetting** (losing general skills) and over-specialisation; the mitigations are a small learning rate, short training and **replay** of data from the original task. **Self-training** uses a teacher's confident predictions on unlabelled target data as labels for a student; it is a form of semi-supervised domain adaptation whose main risk is **confirmation bias** (the student learns the teacher's mistakes), mitigated by strict confidence thresholds, replay of labelled source data and held-out agreement checks. Pre-trained weights count as models, not data: the competition allowed Apache- or MIT-licensed models of at most 8 billion parameters and banned external data and lookup.

HOW WE USED IT

Reader	Starts from	Fine-tuned on	Learning rate	Used for
MiniLM reader	paraphrase-multilingual-MiniLM-L12-v2 (Apache-2.0, 117,653,760 parameters)	labelled US/India pairs, folds 0-5	5e-5	all countries
E5 reader	multilingual-e5-base (MIT, 278,043,648 parameters)	the same pairs	5e-5	US, India

Reader	Starts from	Fine-tuned on	Learning rate	Used for
French MiniLM reader	the MiniLM reader	very confident French pseudo-labels from an earlier run (matches: best candidate, score ≥ 0.97 , clear lead; plus low-scoring non-matches) + 500,000 labelled US/India pairs	2e-5	France
French E5 reader	the E5 reader	474,808 French pseudo-labelled + 474,808 labelled US/India pairs	2e-5	France

Network access is needed only to download the two pre-trained encoders at pinned revisions; every later step runs offline.

EXPERT NUANCE

Limitation first: the French E5 reader failed two of its four written checks (a generated labelled French test set, best F0.5 0.910516 against 0.914964 for its base model; and our "expected gain must be safely positive" rule, pessimistic estimate -0.0001806), and we kept it because the file containing it had our best public score; that is a leaderboard-driven decision, disclosed as such. It changes French links only ($-2,228$ / $+3,396$). Self-training learns only from what its teacher was already sure about, so its main gain is adapting the reader to French formats and vocabulary, and it cannot correct mistakes the teacher makes confidently. The France decision model was trained with the MiniLM reader's logits in its reader slot and at test receives the French blend; that changes the meaning of one input and is part of why France uses the more conservative 0.82.

REMEMBER

- Pre-trained = general skills; fine-tuned = specialised on our pairs; from scratch = ranker, decision and second-chance models.
- French readers: self-trained on confident pseudo-labels plus replay of labelled US/India pairs, learning rate 2e-5.
- French E5 reader: passed 2 checks, failed 2, kept for the best public score (disclosed).

JUDGE MIGHT ASK

- *Isn't a pre-trained language model a form of external data?* The rules allowed pre-trained models under Apache or MIT licences up to 8 billion parameters and banned external data and lookups. Our two base encoders are Apache-2.0 and MIT, with 117.7 and 278.0 million parameters, downloaded once at pinned revisions; every later step runs offline with no lookup.
- *How do you know French fine-tuning did not damage the reader on US and Indian data?* A written check before adoption compared the French E5 reader with its base model on 509,260 labelled US and Indian pairs: AUC 0.998544 against 0.998570 and top-1 accuracy 0.993161 against 0.993004, within tolerance. Honestly, it failed two other checks, and we kept it because the file with it had our best public score, which we disclose.

A0.18 Parameters and hyper-parameters

IN ONE LINE

Parameters are the numbers a model learns from data by itself; **hyper-parameters** are the settings we choose around training, and decision thresholds are settings of that kind too.

SIMPLE EXPLANATION

In cooking, tasting and adjusting the seasoning as you go is like learning parameters; the oven temperature and the timer you set beforehand are hyper-parameters. A model learns its millions of weights or its tree splits; we choose the learning rate, the number of leaves, the maximum number of trees, the threshold 0.75 and so on. Because we choose hyper-parameters by looking at results, we must

look at validation data, not at the final test, or we fool ourselves (see Training, validation and test data). Some settings are chosen automatically from validation data, such as the number of trees by early stopping.

WORKED EXAMPLE

Component	Learned parameters	Hyper-parameters we set	How the setting was chosen
MiniLM reader	117,653,760 encoder weights + 385 head weights (384 + 1 bias)	learning rate 5e-5 (head × 5), weight decay 0.01, warm-up 3 %, batch 128, one epoch, 128 tokens, dropout 0.1	fixed recipe
CatBoost ranker	splits and leaf values of up to 1,600 trees	depth 7, learning rate 0.075, L2 5, early stopping 180	number of trees by early stopping on fold 6
LightGBM decision model (one seed)	splits and leaf values; a tree with 127 leaves has 126 split rules (a feature and a threshold each) and 127 leaf values	learning rate 0.05, 127 leaves, ≥ 200 rows per leaf, 0.9 feature and 0.8 row sampling, L2 1.0, at most 3,000 rounds (US, India)	number of trees by early stopping on fold 8, then scaled for the refit
Decision rule	none	threshold 0.75 / 0.82, lead 0.4, consistency-check bars 0.75 / 0.68 / 0.72 / 0.77	0.75 on fold 8; bars on folds 8 and 9; 0.82 label-free; 0.4 one fixed lead for every country
Toy lookup table (A0.1)	the 4 learned match rates	the 0.85 cut for "high similarity"	chosen by hand

1. Read the second column: everything there was found by training.
2. Read the third column: everything there was chosen by us, before or after training.
3. Read the fourth column: this is what a judge is really asking when they say "why this value?".

TECHNICAL EXPLANATION

Parameters θ minimise the training loss for fixed hyper-parameters λ ; hyper-parameters are chosen to maximise a validation metric, which makes model selection a two-level (bilevel) optimisation. Search strategies include grid search, random search, Bayesian optimisation, or simply a sensible fixed recipe. Some hyper-parameters are resolved by the data during training, such as the number of boosting rounds through early stopping; others are post-hoc, such as decision thresholds, which are hyper-parameters of the decision rule and should be tuned for the exact deployment metric. Every additional hyper-parameter tuned on the same validation fold spends some of that fold's credibility, which is why reporting on a fold not used for tuning matters. Random seeds are hyper-parameters as well: results vary with them, and seed averaging reduces that variance. Parameter count is a poor measure of capacity across model families: a tree's few hundred numbers and a transformer's hundred million are not comparable, and most of a multilingual transformer's parameters sit in its vocabulary table.

HOW WE USED IT

As in the table, plus: the US/India refit rule, number of trees = best iteration × rows(6-9) / rows(6-7); seeds 1-5 for the decision models, 7, 8 and 9 for the address-rescue scorer, and 20260927 for the fold hash. The 0.75 threshold was tuned on fold 8 for the exact per-business macro F0.5; the consistency-check bars on folds 8 and 9; the second-chance thresholds on fold 8 with false positives counted twice; the France threshold 0.82 and the French MiniLM reader's blend weight label-free.

EXPERT NUANCE

Be precise about which settings were tuned and which are a fixed recipe: the tree and reader training settings are reported as fixed values and no sweep is recorded, so do not claim they are optimal; the decision thresholds were tuned on held-out folds; and the Documentation does not record how the 0.4 lead was picked (say "one fixed lead used in every country"). Some settings were chosen with public-leaderboard feedback (for example some France rule parameters and the choice of the final file), which

our Documentation lists. When asked "why this value?", the honest shape of the answer is: how it was chosen, on which data, and what we did not test.

REMEMBER

- Parameters are learned (weights, splits, leaf values); hyper-parameters are chosen (learning rate, leaves, thresholds).
- Choose hyper-parameters on validation data; our numbers of trees came from early stopping.
- Know which of ours were tuned (thresholds, consistency check) and which are a fixed recipe (tree and reader settings).

JUDGE MIGHT ASK

- How did you tune the LightGBM hyper-parameters?* Honestly, the tree settings (learning rate 0.05, 127 leaves, at least 200 rows per leaf, 0.9 feature and 0.8 row sampling, L2 1.0) are one fixed recipe, and we did not report a sweep. What we did tune on held-out data is the number of trees, by early stopping on fold 8, and the decision thresholds, for the exact per-business F0.5; for the US and India the refit on folds 6-9 then scales the number of trees by the row count.

Recap: the eighteen concepts on one page

Concept	The idea in a few words	Where it lives in our pipeline
A0.1 What a model is	a function learned from examples	ranker, readers, decision models, second-chance models are learned; filter, decision rule, consistency check, France rules are code
A0.2 Features and labels	clues as numbers; the right answer	76 index features; 217 / 211 / 158 decision features; label = "candidate is the owner"
A0.3 Classification, regression, ranking	category, number, order	pointwise ranker, yes/no decision per pair, pick the best or abstain
A0.4 Confusion matrix and F0.5	TP, FP, FN; precision first	per-business F0.5, macro-averaged; public 0.991300
A0.5 Scores vs decisions	a rule turns scores into actions	≥ 0.75 (US, India) or ≥ 0.82 (France), lead ≥ 0.4 ; consistency-check bars
A0.6 Class imbalance	rare positives	at most about 1.5 true per million raw pairs $\rightarrow$ 1.17 pairs per record in the shortlist
A0.7 Train, validation, test	learn, choose, judge	business-level folds; base models on 0-5, decision models on 6-9; no untouched end-to-end validation (disclosed)
A0.8 Log loss	penalty for confident mistakes	every learned model minimises it; F0.5 only chooses thresholds
A0.9 Gradient descent	step downhill	AdamW, learning rate $5e-5$ (head $\times 5$), 3 % warm-up, one epoch; boosting learning rates 0.05 / 0.075
A0.10 Over- and underfitting	memorising vs too simple	early stopping on held-out folds; 18 uploads and France rules disclosed as process risks
A0.11 Regularisation	constraints against noise	L2, ≥ 200 rows per leaf, sampling, dropout, weight decay, replay
A0.12 Decision trees	a learned flowchart	CatBoost symmetric trees of depth 7; LightGBM 127 leaves
A0.13 Ensembles	many models combined	boosting, five-seed mean, stacking on ranker and reader scores, French logit blend
A0.14 Neural networks	layers of weights and activations	12-layer cross-encoder readers, one logit per pair
A0.15 Tokenisation	text cut into subword IDs	MiniLM tokenizer on raw text, at most 128 tokens per pair

Concept	The idea in a few words	Where it lives in our pipeline
A0.16 Embeddings	text as vectors; cosine	three frozen-MiniLM word-meaning features (Enterprises vs Industries 0.77)
A0.17 Pre-trained and fine-tuned	general skills, then specialise	four readers; French self-training with replay; French E5 reader kept despite two failed checks
A0.18 Parameters and hyper-parameters	learned vs chosen	thresholds tuned on folds 8 and 9; tree and reader settings a fixed recipe

A1 - Concepts: foundations

How to read this part

This part teaches, from zero, the ideas that sit underneath our pipeline: what entity resolution is, how the competition scores it, how text is cleaned, how two strings are compared, and how candidates are found among trillions of possible pairs. It stops where candidates reach the shortlist; later parts cover the CatBoost ranker, the readers, the "what changed" features, the LightGBM decision models, training and validation, and France in depth. Read the concepts in order, because each one builds on the earlier ones and refers back to them by name (for example "see Inverted index"). Every concept has the same eight blocks: **In one line**, **Simple explanation**, **Worked example**, **Technical explanation**, **How we used it**, **Expert nuance**, **Remember** and **Judge might ask**. On a first pass read only the first three blocks of every concept; on a second pass read the technical blocks; before the finale, say the Remember bullets and the judge answers aloud. Words in **bold** are defined where they first appear.

About numbers: every number about our pipeline comes from the submitted Documentation (or the shared fact sheet, which quotes it). Numbers marked "(derived)" are simple arithmetic on those numbers. Numbers marked "(illustrative)" are invented for teaching and must never be quoted as our results. Outputs marked "(computed)" were produced by running our own normalisation code or the RapidFuzz / text-unidecode libraries on the example strings shown; they are teaching examples, not pipeline results.

Spoken answers use the plain names of the talk. Here they are in pipeline order, each in one line, so that you can follow the "How we used it" blocks before the later parts explain them in depth:

- **business**: one row of the clean reference list we link to (the organisers call it a Source 1 record).
- **incoming record**: one row of Source 2 or Source 3 that has to be linked to a business, or to nothing.
- **index lookup**: a fast word-and-letter-piece search (an inverted index, written in C++) that finds businesses sharing rare words with the record.
- **quick rule-based filter**: a fixed checklist, with no model, that drops candidates clearly weaker than the record's best one.
- **CatBoost ranker**: a gradient-boosted tree model (CatBoost is the library) that orders the remaining candidates.
- **shortlist**: the top candidates the ranker keeps, at most 4 per incoming record.
- **readers**: small multilingual neural language models that read the raw name and address of both records together and output one match score (the MiniLM reader and E5 reader for the US and India; for France the MiniLM reader plus the French MiniLM reader and French E5 reader).
- **"what changed" features**: numbers that describe exactly how a candidate differs from the record (digits changed, words added, dropped or replaced, legal form changed).
- **LightGBM decision model per country**: a gradient-boosted tree classifier (LightGBM is the library) that turns about two hundred features into one match probability per pair.
- **confident + clear-winner rule**: link the best candidate only if its probability clears a bar and it leads the runner-up by at least 0.4.
- **second-chance searches**: two extra searches for records left unlinked (native-script rescue in India, address rescue in the US and India).
- **per-business consistency check**: a re-decision that looks at how many other records were accepted for the same business.
- **France rules computed by code**: fixed rules on scores, raw text and addresses that veto or add French links.

Concepts in this part

The problem and its vocabulary

1. Entity resolution and record linkage: the vocabulary
2. Link structures: many-to-one, one-to-one and clustering

Measuring success

3. Confusion matrix, precision and recall
4. F-beta and why F0.5
5. Macro-averaged per-business F0.5 and the empty-row rule
6. Scores, thresholds, precision-recall curves and AUC
7. Abstention: the reject option and empty rows

Cleaning text

8. Text normalisation
9. Unicode, scripts and accent stripping
10. Transliteration of Indian native scripts
11. Address normalisation and parsing (US, India, France)

Comparing strings

12. Tokens, sets and character n-grams
13. Edit distances: Levenshtein, Damerau-Levenshtein and OSA
14. LCS ratio and the RapidFuzz family
15. Jaro-Winkler similarity (not used by us)
16. IDF, TF-IDF, BM25 and our IDF-weighted overlap
17. Set-overlap measures: Jaccard, overlap coefficient and IDF-weighted versions
18. Phonetic keys: Soundex and Metaphone (not used) and our consonant skeleton

Finding candidates

19. Blocking and candidate generation: reduction ratio, pair completeness, pairs quality
20. The inverted index
21. Multi-channel retrieval and the union of top lists
22. The quick rule-based filter
23. Recall ceiling and the candidate funnel
24. Other blocking families: sorted neighbourhood, canopy clustering, LSH / MinHash (not used)
25. Dense vector retrieval with approximate nearest neighbours (not used)

Scale

26. Scaling: sharding, bounded work per record and MapReduce-style blocking

A table of the numbers used in this part closes the file.

A1.1 Entity resolution and record linkage: the vocabulary

IN ONE LINE

Entity resolution is deciding which records, written in different ways, describe the same real-world thing; in our task it means attaching each incoming record to the one business it belongs to, or to

nothing.

SIMPLE EXPLANATION

Imagine a post office in Pune with a master register of every shop in the city. Thousands of letters arrive every day, addressed by hand: "Shri Ganesh Traders, MG Rd", "SHREE GANESH TRADERS", or the same name written in Devanagari (the script used for Hindi and Marathi). The clerk must put each letter into the pigeonhole of the right shop, and if no shop in the register fits, the letter goes into no pigeonhole at all. The clerk never compares letters with each other, only letters with the register. Two shops can have almost the same name, so carelessness causes harm: a letter in the wrong pigeonhole is worse than a letter put aside. That is our task, with 1.7 million shops and 10 million letters.

The words you need:

- An **entity** is the real thing, for example one actual shop.
- A **record** is one row of data describing an entity (here a name and an address), possibly with errors.
- **Entity resolution (ER)** is the general task of finding records of the same entity. It is called **record linkage** when records of different files are joined and **deduplication** when duplicates inside one file are merged.
- The **reference list** is the clean list we link to. Its rows are our **businesses** (the organisers call them Source 1 records): 1,732,544 in the test files.
- An **incoming record** is a record from Source 2 or Source 3 that has to be linked: 9,969,589 in the test files.
- A **pair** is one (incoming record, business) combination considered together. A **candidate** is a business that a search step proposes for an incoming record.
- A **match** (true link) is a pair whose two records really describe the same entity; the business is then the record's **owner**. Every other pair is a **non-match**.
- A **predicted link** is a pair our system outputs as a match.
- A **singleton** (no-match record) is an incoming record whose owner is not in the reference list; the right answer is to link nothing.
- An **empty row** is a business to which we link no incoming record.
- To **abstain** is to deliberately output no link.
- **Labels** (ground truth) are the known correct owners. We had them for the US and India training data, and none for France.

WORKED EXAMPLE (ILLUSTRATIVE NAMES)

Reference list of one country:

Business	Name	Address
B1	Shree Ganesh Traders	12 MG Road, Pune, Maharashtra
B2	Shree Ganesh Traders	45 FC Road, Pune, Maharashtra
B3	Vijay Enterprises Pvt Ltd	24/11 Ghatkopar, Mumbai, Maharashtra

Incoming records:

Record	Text	Truth	Good decision
R1	Shri Ganesh Traders, 12 M.G. Rd, Pune	B1	link to B1: spelling variant, same address
R2	SHREE GANESH TRADERS (no address)	B1	abstain: without an address B1 and B2 cannot be told apart
R3	विजय एंटरप्राइजेज, 24/11, Mumbai	B3	link to B3: same name in another script
R4	Ganesh Sweets, 9 Station Road, Nashik	none	link nothing: a singleton

1. There are $4 \times 3 = 12$ pairs: 3 matches (R1-B1, R2-B1, R3-B3) and 9 non-matches.
2. Output per business: B1 = {R1}, B2 = {} (an empty row), B3 = {R3}.
3. Abstaining on R2 loses a little recall at B1 but risks nothing; a coin-flip link to B2 would have put a wrong record into B2's row.

TECHNICAL EXPLANATION

Formally there are two sets: A, the reference list ($|A| = 1,732,544$ in test), and B, the incoming records ($|B| = 9,969,589$), and an unknown function owner: $B \rightarrow A \cup \{\text{none}\}$. The pair space $A \times B$ splits into matches M and non-matches U, and $|M| \leq |B|$ because each incoming record has at most one owner. A typical ER system has three parts: candidate generation (blocking), which picks a small subset of $A \times B$ worth examining; pair scoring, which estimates how likely each candidate pair is a match from comparison features or a model reading both records; and a linking decision that respects the structure of the task (here at most one business per record, abstention allowed). The classical formalisation is Fellegi-Sunter (1969): build a comparison vector γ for each pair (for example "names agree, house numbers disagree") and compute the likelihood ratio $m(\gamma)/u(\gamma)$, where m and u are the probabilities of seeing γ among matches and among non-matches; pairs above an upper threshold are linked, pairs below a lower threshold are not, and pairs in between go to clerical review. m and u are often estimated without labels by expectation-maximisation (EM, an iterative method that alternates between guessing each pair's hidden match status and re-estimating m and u from those guesses). With labels, the same idea becomes supervised classification: a model maps comparison features to $P(\text{match})$. What makes our instance hard is imbalance and scale: at most 9,969,589 true links among 6,724,569,566,212 same-country pairs, fewer than 1.5 per million (derived), so both the search and the decision must be extremely selective.

HOW WE USED IT

Our task is linkage to a reference list: each incoming record is linked to at most one business of its own country, or to none, while a business may own several records (Doc 2.1). The test files hold, per country, US 663,106 businesses and 3,817,031 incoming records, India 809,986 and 4,717,565, France 259,452 and 1,434,993, so 5.7543 incoming records per business overall; labels exist for US and India training data, none for France. We deliver two files: the matching file, one row per business with its linked incoming records (5,856,936 links, 100,087 empty rows), and the candidate file (12,293,019 candidate pairs). No pair joins records of different countries and no incoming record is linked to more than one business (Doc B.1). Our pipeline maps onto the three parts: index lookup, quick rule-based filter and CatBoost ranker build the shortlist (candidate generation); readers, "what changed" features and the LightGBM decision model score pairs; the confident + clear-winner rule, the per-business consistency check, the second-chance searches and the France rules make the linking decisions.

EXPERT NUANCE

The evaluation unit follows the task shape: the metric is computed per business, so a link is judged from the business's point of view (which records it owns). Deduplication would require clustering records among themselves (see Link structures), but reference linking does not. Fellegi-Sunter is the textbook baseline; with labelled US and India data, a supervised decision model is the same idea (comparison vector in, match probability out) fitted directly, which is the safe way to answer if asked. Do not confuse "incoming records per business" (5.75, all records in the files) with "true records per business": some incoming records are singletons, and our file links 3.3805 records per business on average.

REMEMBER

- Business = reference list row (1,732,544); incoming record = Source 2 or 3 row (9,969,589); each record links to at most one business of its country, or to none.
- A typical ER system = find candidates, score pairs, decide links.
- Matches are rare: at most 1.5 per million same-country pairs (derived).

JUDGE MIGHT ASK

- *Is this deduplication or record linkage?* "It is linkage to a reference list: every incoming record goes to at most one business of its country, or to none, and a business can own several records. We never merge incoming records with each other, so we do not need a clustering step."
- *Why not classic Fellegi-Sunter probabilistic linkage?* "Fellegi-Sunter scores a comparison vector by a likelihood ratio whose probabilities are usually estimated without labels. We had labels for the US and India, so our decision model is the supervised version of the same idea: about two hundred features per pair in, a match probability out."

A1.2 Link structures: many-to-one, one-to-one and clustering

IN ONE LINE

The rules about who may link to whom (one-to-one, many-to-one, or groups) decide which decision algorithm is correct.

SIMPLE EXPLANATION

Think of students and schools. Each student attends at most one school, but a school has many students: that is **many-to-one**. A marriage register is **one-to-one**: each person has at most one spouse. Friend circles are **clusters**: if Asha is a friend of Ravi and Ravi of Meera, all three may form one group. Each shape needs a different method. Our task is students-and-schools: every incoming record goes to at most one business, and a business can collect many records. So for each incoming record we only need to pick its best business and decide whether to trust it.

WORKED EXAMPLE (ILLUSTRATIVE SCORES)

Three incoming records, two businesses; truth: R1 and R2 belong to B1, R3 to B2.

	B1	B2
R1	0.95	0.10
R2	0.90	0.20
R3	0.30	0.85

1. Many-to-one (our rule): each row picks its best column. $R1 \rightarrow B1$ (0.95), $R2 \rightarrow B1$ (0.90), $R3 \rightarrow B2$ (0.85). With a bar of 0.75 all three are linked, all correct.
2. One-to-one (**Hungarian assignment**, which uses every column at most once and maximises the total score): the best plan is $R1 \rightarrow B1$ and $R3 \rightarrow B2$ (total 1.80), leaving R2 unlinked. A true link is lost only because of the wrong constraint.
3. Clustering by **transitive closure** (if A matches B and B matches C, then A, B and C are one group): one loose false edge between R2 and R3 would chain R1, R2 and R3 into a single cluster.

TECHNICAL EXPLANATION

Many-to-one with abstention means a constraint "at most one link per incoming record" and no constraint per business. With an objective that adds up per record, the optimal decision then decomposes record by record: take the argmax (the candidate with the highest score) over the record's candidates and accept it only if it passes the decision rule (see Abstention), at cost $O(k)$ for k candidates. Our metric is a per-business ratio, so it adds up only approximately per record; that gap is what the per-business consistency check addresses. One-to-one linkage is bipartite matching, solved exactly by the Hungarian algorithm in $O(n^3)$ on dense matrices or by min-cost flow on sparse candidate graphs; it is the right model when both files are already deduplicated and describe the same population. Deduplication needs clustering: build a graph of pairwise match decisions, then take connected components (transitive closure,

which suffers from chaining because one false edge merges two clusters), or solve correlation clustering (NP-hard, meaning no fast exact algorithm is known, so it is solved approximately), or use centre or star clustering. In reference linking each entity already has a fixed hub, the business row, so records never need to be linked to each other. Cross-record evidence is still useful when several incoming records compete for the same business; that is handled by the collective features (US and India) and the per-business consistency check, covered in later parts.

HOW WE USED IT

Each record takes its best candidate and is linked only if the score is confident and a clear winner, so there is at most one link per record; the export asserts that every matched pair is a candidate pair, and the final file links no record to more than one business (Doc B.1). Businesses own many records: our file links 3.3805 records per business on average, with median 3, p90 6, p99 7 and maximum 11 (Doc B.1). Not used: one-to-one assignment (Hungarian or bipartite matching) and graph clustering (transitive closure, correlation clustering). The Documentation does not discuss clustering; our reasoning is that the task links records to a reference list, so there is no record-to-record graph to close.

EXPERT NUANCE

Many-to-one does not mean "anything goes" at the business side: a business that suddenly collects many accepted records is information in itself. The per-business consistency check uses it locally, re-deciding each record with a bar that depends on how many other records the decision model accepted for the same business (0.75, 0.68, 0.72, 0.77 for 0, 1, 2, 3 or more), which is a light substitute for a global constraint, not an assignment solver. A general caveat of reference linking is that it trusts the reference list to hold one row per entity; when two rows are near-identical, the per-record argmax plus the lead requirement makes us abstain rather than guess, which is the precision-first behaviour F0.5 rewards.

REMEMBER

- Our shape is many-to-one with abstention, so the decision is per record: argmax, then accept or reject.
- One-to-one assignment would discard true links, because a business owns several records (3.38 linked on average).
- No clustering is needed: records link to a reference list, not to each other.

JUDGE MIGHT ASK

- *Why not solve a global assignment, for example with the Hungarian algorithm?* "Because the task is many-to-one: a business owns several incoming records, and our file links 3.38 per business on average, so a one-to-one assignment would throw away true links. Each record still links to at most one business: its best candidate, and only if confident and a clear winner."
- *Why no clustering or transitive-closure step?* "Records link to a fixed reference list, not to each other, so there is no record graph to close. The cross-record logic we need sits at the business: the collective features and the per-business consistency check look at all records competing for the same business."

A1.3 Confusion matrix, precision and recall

IN ONE LINE

Precision is the share of our links that are right; **recall** is the share of the true links that we found.

SIMPLE EXPLANATION

You want to catch only salmon in a river full of fish. Precision asks: of all the fish in your net, what fraction are salmon? Recall asks: of all the salmon in the river, what fraction ended up in your net? A tiny net with one salmon has perfect precision and terrible recall; a huge net catches every salmon and a lot of other fish. Entity resolution is the same: every link we output is a catch. The four possible outcomes of a decision

are summarised in a **confusion matrix**: a **true positive (TP)** is a correct link, a **false positive (FP)** is a wrong link, a **false negative (FN)** is a missed true link, and a **true negative (TN)** is a non-match we correctly left alone.

WORKED EXAMPLE

Suppose 12 true links exist and we output 10 links, 8 of them correct.

	Truly a match	Truly not a match
We linked	TP = 8	FP = 2
We did not link	FN = 4	TN = every other pair (trillions)

1. Precision = $8 / (8 + 2) = 0.80$.
2. Recall = $8 / (8 + 4) = 0.667$.
3. Accuracy = $(TP + TN) / \text{all pairs}$ is about 1.000 whatever we do, because TN is enormous; it is useless here.
4. The twist of linking: incoming record R is owned by B1 but we link it to B2. That is a false positive at B2 and a false negative at B1. Had we abstained, it would only be a false negative at B1. For a singleton, abstaining is exactly right. So a wrong link is never better than abstaining.

TECHNICAL EXPLANATION

Precision $P = TP / (TP + FP)$ and recall $R = TP / (TP + FN)$. In ER the count of true negatives is roughly the whole comparison space (about 6.7 trillion same-country pairs in our test), so accuracy, specificity and the false-positive rate of a ROC curve are dominated by TN and say almost nothing; precision, recall and precision-recall curves do not use TN at all. The unit of counting matters: pairs, incoming records, or entities (businesses). In many-to-one linking each predicted link is one pair, and per business we can count TP_b (records correctly linked to b), $pred_b$ (records linked to b) and $true_b$ (records b owns). A wrong link on an owned record creates one FP and one FN at two different businesses, which is why the decision rule is built to abstain when in doubt. Clustering-style ER has its own metrics (pairwise F, B-cubed, CEAF), but the competition defined a per-business F0.5, so we did not need them. Recall can never exceed the share of owners that the candidate search retrieves (see Recall ceiling).

HOW WE USED IT

The competition metric is built from per-business TP, predicted and true counts (see the next two concepts). Our evaluation code computes the exact per-business score on labelled held-out folds and also counts false positives and false negatives per fold, including owned records that never reached the candidates (they still count in $true_b$). For candidate generation, recall is measured as the share of owned incoming records whose owner is among the candidates: 99.3694 % (US) and 98.5068 % (India) at the index lookup on fold 9 (Doc B.2). A **fold** here is one of ten groups of businesses, split by a hash of the business id; a later part covers folds in depth.

EXPERT NUANCE

Precision and recall trade off through the threshold (see Thresholds), so quoting one without the other means little. In our setting the main precision risks are look-alike businesses: names at the business's address with one generic word replaced or added or the legal form changed, acronyms that differ by one initial at another address, and branches of a chain at nearby addresses (QA 14). The main recall risks are retrieval misses and the lead requirement when two candidates are close. Both precision and recall need labels; France has none, so its precision and recall are not measured, and we say so.

REMEMBER

- $P = TP / (TP + FP)$, $R = TP / (TP + FN)$; true negatives are meaningless in ER.

- A wrong link on an owned record is one false positive plus one false negative; abstaining is one false negative only.
- Recall can never exceed the share of owners that reach the candidates.

JUDGE MIGHT ASK

- *Why not report accuracy or ROC-AUC for the linker?* "Almost every pair is a non-match, about 6.7 trillion same-country pairs against at most 10 million true links, so accuracy is near 1 for any system and a ROC curve is dominated by true negatives. We use the competition's per-business F0.5, which ignores true negatives."
- *What are your precision and recall separately?* "We tuned on, and report, the exact per-business F0.5 rather than separate precision and recall: for example, a close version of our US decision model scores 0.991212 on held-out fold 9. Our evaluation code also counts false positives and false negatives per fold, so a split by country and error type is what we would report next."

A1.4 F-beta and why F0.5

IN ONE LINE

F-beta combines precision and recall into one number, and $\beta = 0.5$ makes precision count more than recall.

SIMPLE EXPLANATION

You cannot optimise two numbers at once, so you need one. A plain average would flatter silly systems: precision 1.0 with recall 0.01 averages to 0.505. F-beta uses a **harmonic mean** (two divided by the sum of the two reciprocals), which is dragged down by whichever of the two is worse: for the same system it is $2 \times 1.0 \times 0.01 / (1.0 + 0.01) = 0.0198$. Beta is a dial: $\beta = 1$ treats precision and recall equally (F1), $\beta = 2$ favours recall, $\beta = 0.5$ favours precision. The competition chose 0.5, which says: a wrong link hurts more than a missed one.

WORKED EXAMPLE

(a) Two systems with the same F1:

System	Precision	Recall	F1	F0.5	F2
Careful	0.9	0.6	0.720	0.818	0.643
Greedy	0.6	0.9	0.720	0.643	0.818

(b) Count form, $TP = 2$, $FP = 0$, $FN = 1$: $F0.5 = 1.25 \times 2 / (1.25 \times 2 + 0.25 \times 1 + 0) = 2.5 / 2.75 = 0.909$, while $F1 = 2 \times 2 / (2 \times 2 + 1 + 0) = 0.800$.

(c) A 1 % loss on each side: precision 1.00 with recall 0.99 gives $F0.5 = 1.25 \times 0.99 / (0.25 + 0.99) = 0.998$; precision 0.99 with recall 1.00 gives $1.25 \times 0.99 / (0.2475 + 1) = 0.992$. Losing 1 % of precision costs about four times as much as losing 1 % of recall.

TECHNICAL EXPLANATION

$F_{\beta} = (1 + \beta^2) \cdot P \cdot R / (\beta^2 \cdot P + R)$. It is a weighted harmonic mean: $1 / F_{\beta} = [1 / (1 + \beta^2)] \cdot (1 / P) + [\beta^2 / (1 + \beta^2)] \cdot (1 / R)$, so for $\beta = 0.5$ the weights are 0.8 on $1/P$ and 0.2 on $1/R$. In counts, $F_{\beta} = (1 + \beta^2) \cdot TP / ((1 + \beta^2) \cdot TP + \beta^2 \cdot FN + FP)$, and since predicted = $TP + FP$ and true = $TP + FN$ this equals $(1 + \beta^2) \cdot TP / (\text{predicted} + \beta^2 \cdot \text{true})$; for $\beta = 0.5$ that is $F0.5 = 1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$. In the denominator one false positive weighs 1 and one false negative weighs $\beta^2 = 0.25$: a wrong link costs four times a missed link. Van Rijsbergen's reading is that F_{β} suits a user who gives recall β times the importance of precision, so $\beta = 0.5$ means "precision twice as important" in his sense; both statements

describe the same formula. A useful marginal rule follows from the count form: with the true count fixed, adding one link that is right with probability p raises the expected F_{beta} exactly when $p > F_{\text{beta}} / (1 + b^2)$, because the denominator grows by 1 while TP grows by p . For F1 that is the known result $p > F/2$; for F0.5 it is $p > 0.8 \cdot F$, so near $F = 0.99$ a link is worth adding only if p exceeds about 0.79 (for a pooled metric and calibrated probabilities). F is a ratio of sums, not a sum of per-pair losses, so models are trained with log loss and the decision threshold is tuned afterwards on the exact metric.

HOW WE USED IT

Beta = 0.5 is the competition's choice, applied per business and macro-averaged (next concept). The whole pipeline is precision-first: high thresholds (0.75 for US and India, 0.82 for France), a required lead of 0.4 over the runner-up, abstention, France vetoes, and second-chance thresholds chosen with false positives counted twice. The US and India operating points (the 0.75 threshold, the consistency-check bars, the second-chance thresholds) were tuned on the exact per-business macro F0.5 on labelled folds, not on AUC or log loss (Doc 4); France's 0.82 was set without French labels (see Thresholds), and the 0.4 lead is one fixed value for every country.

EXPERT NUANCE

The " $0.8 \cdot F$ " rule is a sanity check, not how we chose 0.75: our metric is per business rather than pooled, and our scores are not guaranteed to be calibrated, so we swept thresholds on fold 8 with the exact metric. It does explain why the best threshold rises as a system gets better: a higher F raises the bar a new link must clear. A common confusion is "one false positive costs two false negatives": the factor two is van Rijsbergen's importance ratio $1/b$, while in the count form's denominator the weight ratio is $1/b^2 = 4$, and that is the number that moves thresholds. Another is that averaging F over businesses equals F of pooled counts; it does not (next concept).

REMEMBER

- $F0.5 = 1.25 \cdot P \cdot R / (0.25 \cdot P + R) = 1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$.
- In the denominator one wrong link weighs as much as four missed links.
- A link helps only if its chance of being right exceeds about $F / 1.25$ (pooled, calibrated sanity check).

JUDGE MIGHT ASK

- *How does F0.5 change your decisions compared with F1?* "Under F0.5 a wrong link weighs four times a missed link, so the best operating point moves to a higher bar. We link only when the decision model is confident, at least 0.75 (0.82 in France), and the winner leads the runner-up by 0.4; otherwise we abstain."
- *Is precision 'twice' or 'four times' as important under F0.5?* "Both, in different senses: van Rijsbergen's beta gives recall half the importance of precision, and in the count form one false positive weighs four times one false negative in the denominator. The count form is the one that drives our thresholds."

A1.5 Macro-averaged per-business F0.5 and the empty-row rule

IN ONE LINE

The competition computes F0.5 separately for every business and then takes the plain average, and a business with no true records scores 1 only if we link nothing to it.

SIMPLE EXPLANATION

A teacher can grade a class in two ways: pool all marks into one big total (**micro-averaging**), or grade each student separately and average the grades (**macro-averaging**). Our competition grades each business like a student and averages, so a tiny business with one record counts exactly as much as a big

one. A business that truly owns no records is like a student with no homework due: full marks only if you hand them nothing, zero if you hand them one wrong paper. That is why one wrong link can turn a perfect 1 into a 0.

WORKED EXAMPLE

Per-business formula: $F_b = 1.25 \cdot TP_b / (pred_b + 0.25 \cdot true_b)$, and $F_b = 1$ when $pred_b = true_b = 0$.

Business	True records	Linked	Correct	F_b
A	2	2	2	$1.25 \times 2 / (2 + 0.5) = 1.000$
B	0	0	0	1 (empty-row rule)
C	0	1	0	$0 / (1 + 0) = 0.000$

1. Macro average = $(1 + 1 + 0) / 3 = 0.667$.
2. Micro (pooled: TP 2, predicted 3, true 2) = $1.25 \times 2 / (3 + 0.5) = 0.714$.
3. The single wrong link costs a full third of the macro score here.
4. A business D with 3 true records scores 0.909 with 2 correct links, 1.000 with 3 correct links, and 0.667 with 2 correct links plus 1 wrong one.

TECHNICAL EXPLANATION

Macro F0.5 = $(1/N) \sum_b F_b$ over all N businesses of the test set; with N = 1,732,544 each business weighs about 5.8×10^{-7} of the score (derived). Consequences: small businesses count as much as large ones; a business with no true records drops from 1 to 0 with any link; a "link nothing" output scores exactly the share of businesses that truly own no record (unknown for the test set); singleton incoming records can only hurt when linked. Recall counts every owned record, including those that never reached the candidates, so retrieval misses lower the score even though no model saw them. The metric is not a sum over pairs: the value of one more link depends on the other links of the same business, which is why a count-dependent bar (the per-business consistency check) can help. To compare two systems on the same businesses, resample businesses with replacement and recompute the difference each time (a **paired bootstrap**), which respects the pairing and the business as the unit.

HOW WE USED IT

Our evaluation code implements the exact metric: per business TP, predicted and true counts, $F = 1.25 \cdot TP / (pred + 0.25 \cdot true)$, 1 when both are 0, averaged over the businesses of a held-out fold; every threshold and the consistency check were tuned on it. Folds are assigned per business (10 folds by a hash of the business id), so the unit of splitting matches the unit of the metric. Our held-out evaluation code (the protocol behind the Documentation's held-out table) reports 95 % intervals of differences with a paired bootstrap over businesses (2,000 resamples). The final file has 100,087 empty rows (5.8 %, derived), a direct consequence of linking only on a clear win. A diagnostic upload shows how much a group of rows matters: emptying every French row of an early upload moved the public score from 0.970125 to 0.839 (Doc B.6).

EXPERT NUANCE

Macro averaging changes the economics of a link: its value depends on the business it lands on (see the break-even example in Abstention). Because we cannot know which test businesses are truly empty, the safest policy is a uniformly high bar plus a lead requirement, with extra care where a business already has links. French businesses are 15.0 % of all businesses (259,452 of 1,732,544, derived), so French precision moves the score visibly; the diagnostic upload confirms French rows carry a large share of it. Per-business F values are coarse (0, 0.667, 0.909, 1, ...), so differences in the fifth decimal need paired comparisons on the same businesses before anyone believes them.

REMEMBER

- $F_b = 1.25 \cdot TP_b / (pred_b + 0.25 \cdot true_b)$; empty truth plus empty prediction = 1.

- Every business weighs the same: one wrong link into an empty business costs a whole business-point.
- Tune and compare on the exact metric, per business, with paired resampling over businesses.

JUDGE MIGHT ASK

- *Why is a correct empty row worth a full point, and what did that do to your design?* "That is the competition's definition: a business with no true records scores 1 only if we link nothing. One wrong link can turn a 1 into a 0, so we link only confident, clear winners, and 100,087 businesses end with an empty row."
- *Your variants differ in the fifth decimal. How do you know a difference is real?* "We compare variants on the same held-out businesses, and our held-out evaluation code uses a paired bootstrap over businesses, 2,000 resamples, for 95 % intervals of the difference. Where a difference is tiny we say so: the quick filter moved fold-9 F0.5 only from 0.990117 to 0.990086."

A1.6 Scores, thresholds, precision-recall curves and AUC

IN ONE LINE

A model outputs a score, a **threshold** turns scores into yes-or-no decisions, and moving the threshold trades precision for recall.

SIMPLE EXPLANATION

A smoke detector has a sensitivity knob. Turned up, every slice of toast sets it off (many false alarms, no missed fires); turned down, it stays silent even with real smoke. The knob is the threshold, and the knob position you choose is the **operating point**. A **precision-recall (PR) curve** plots precision against recall for every knob position. A **ROC curve** plots the true-positive rate (the share of real fires that set the alarm off, which is recall) against the false-positive rate (the share of non-fires, such as toast, that set it off), and **AUC** (area under the ROC curve) is one grade for how well the detector ranks real fires above toast, without choosing a knob position. In the end you still have to choose one, and the best choice depends on what mistakes cost.

WORKED EXAMPLE (ILLUSTRATIVE SCORES)

Ten candidate links, 7 true matches in total.

Score	0.97	0.93	0.90	0.84	0.78	0.70	0.62	0.55	0.41	0.30
True match?	yes	yes	yes	yes	no	yes	yes	no	yes	no

Threshold	Linked	TP	FP	Precision	Recall	F1	F0.5
0.80	4	4	0	1.000	0.571	0.727	0.870
0.70	6	5	1	0.833	0.714	0.769	0.806
0.60	7	6	1	0.857	0.857	0.857	0.857
0.50	8	6	2	0.750	0.857	0.800	0.769
0.40	9	7	2	0.778	1.000	0.875	0.814
0.25	10	7	3	0.700	1.000	0.824	0.745

Same scores, two metrics, two different best knobs: F1 prefers 0.40, F0.5 prefers 0.80.

TECHNICAL EXPLANATION

A threshold rule predicts "match" when the score $s \geq t$. The PR curve is the set of points $(R(t), P(t))$; the ROC curve is $(FPR(t), TPR(t))$; ROC-AUC equals the probability that a random positive scores above a random negative, with ties counted as one half (the normalised Mann-Whitney statistic). AUC is invariant to any monotone transformation of the scores and uses no threshold, so it says nothing about where to

operate; with extreme imbalance ROC is also dominated by true negatives, while PR is not. Our decision is not even a single threshold on pair scores: per record it is an argmax, an absolute threshold and a lead over the runner-up, followed by per-business effects. The correct procedure is therefore to simulate the exact decision rule and the exact metric on labelled held-out data for a grid of thresholds and keep the best, choosing on one fold and checking on another to limit optimism. **Calibration** maps scores to probabilities (Platt scaling fits a logistic curve to the scores; isotonic regression fits a monotone step function). For a pure threshold rule, a monotone calibration only renames the threshold, so it cannot improve the best achievable result; for a lead rule (best minus second ≥ 0.4) a non-linear calibration does change which records pass, because differences between scores are not preserved. **Cost-sensitive threshold selection** weights error types differently while choosing t ; for the second-chance models we counted each false positive twice.

HOW WE USED IT

The US and India threshold 0.75 was chosen on fold 8 for the exact per-business macro F0.5; the consistency-check bars 0.75 / 0.68 / 0.72 / 0.77 on folds 8 and 9; the native-script rescue's 0.88 (used with a lead of 0.3) on fold 8 with false positives counted twice; the address rescue's thresholds also on fold 8 with false positives counted twice (US 0.86 with lead 0.0, India 0.66 with lead 0.1), after a filter that keeps a pair if its probability reaches the largest grid value that still keeps 98 % of the fold-8 reference links (US 0.06, India 0.015) (Doc 4). France's 0.82 was set without labels, from the French link volume per business together with a fold-8 threshold sweep on US and India (Doc C.1). AUC appears only as a sanity check of readers: on 509,260 labelled US/India pairs the French E5 reader and the E5 reader have AUC 0.998544 and 0.998570, and best F0.5 0.992782 and 0.992796 (Doc B.4). We used no Platt or isotonic calibration.

EXPERT NUANCE

AUC near 0.9985 for both readers shows saturation: ranking quality is excellent, yet the result still depends on where you cut and on look-alike competitors, so AUC cannot choose between operating points. A disclosed limitation: folds 8 and 9 were also used to choose the thresholds and the consistency check, so the held-out scores on them are not untouched (Doc 5). Moving a threshold to a country without labels is the hardest case; 0.82 for France is a deliberately conservative, label-free choice whose true optimum is unknown. Whenever the model changes, its threshold must be re-tuned, because scores of different models are not on a common scale.

REMEMBER

- Same scores, different metric, different best threshold (F1 0.40 against F0.5 0.80 in the toy).
- AUC is threshold-free and cannot pick an operating point; we tune on the exact metric.
- 0.75 for US and India (fold 8); 0.82 for France (label-free); false positives counted twice for the second-chance models.

JUDGE MIGHT ASK

- *Why not use 0.5, or choose models by AUC?* "0.5 is arbitrary and AUC has no threshold at all. Our E5 reader and French E5 reader both have AUC around 0.9985 on labelled US and Indian pairs, yet what matters is per-business F0.5 at the chosen cut, so we swept thresholds on held-out fold 8 with the exact metric and got 0.75."
- *Would calibrating the scores have helped?* "For a single threshold, calibration only renames the cut-off, so we tuned directly on the metric instead. It would change the 0.4 lead rule, because a non-linear calibration changes score differences; we did not calibrate, and that is something we would test."

A1.7 Abstention: the reject option and empty rows

IN ONE LINE

Abstention means deliberately outputting no link when the evidence is not strong enough, because under F0.5 a wrong answer costs more than no answer.

SIMPLE EXPLANATION

Picture a quiz show where a wrong answer loses points but passing loses nothing: smart contestants answer only when they are sure. Our rule has two parts. First, the best candidate must be **confident**: its score must clear a bar. Second, it must be a **clear winner**: well ahead of the second-best candidate, because if two candidates look equally good you are guessing. If either part fails, the record stays unlinked. This is the classic **reject option** of decision theory.

WORKED EXAMPLE

Part 1, a toy break-even for business A with 3 true records and 2 correct links so far:

1. Abstain: $F = 1.25 \times 2 / (2 + 0.75) = 0.909$.
2. Link a third record that is correct: $1.25 \times 3 / (3 + 0.75) = 1.000$.
3. Link a third record that is wrong: $1.25 \times 2 / (3 + 0.75) = 0.667$.
4. Linking pays only if the chance p of being right satisfies $p \times 1.000 + (1 - p) \times 0.667 > 0.909$, that is $p > 0.727$ (which is $0.909 / 1.25$, the rule from F-beta). If A has no true records at all, a wrong link drops A from 1 to 0.

This is a teaching toy; our real 0.75 was found by sweeping thresholds on fold 8 with the exact metric over all businesses.

Part 2, our rule on the records of the talk (Doc D):

Record	Best score	Runner-up	Confident?	Clear winner (lead ≥ 0.4)?	Result
US: "Legacy Marketing Enterprises (Inc)"	0.965022	0.088664	yes (≥ 0.75)	yes, 0.876	linked to the Windsor, CT business
France: "TQ COMITE SARL"	0.999837	0.000098	yes (≥ 0.82)	yes, 0.999740	accepted, then the category-swap veto drops it: no link
Illustrative	0.90	0.70	yes	no, 0.20	abstain
India: Devanagari "Vijay Enterprises", main pass	7.2e-6	6.9e-6	no	(not reached)	unlinked; the native-script rescue then links it at 0.99994 with lead 0.99992

TECHNICAL EXPLANATION

Chow's reject rule (1970) says that with error cost 1 and rejection cost c , the Bayes-optimal classifier rejects when the largest posterior probability (the model's probability of the most likely class given the input) is below $1 - c$. Our version works per record: the candidates $c_1 \dots c_k$ have decision-model scores s_i , which are pointwise binary probabilities (each pair scored on its own) and need not sum to 1 across a record's candidates; we link the argmax if $s_{(1)} \geq \text{thr}$ and $s_{(1)} - s_{(2)} \geq 0.4$, and abstain otherwise. The lead condition targets ambiguity between candidates, which a per-pair probability cannot express: two near-identical businesses can both score 0.9. A softmax over a record's candidates (a function that turns a set of scores into probabilities summing to 1) would build mutual exclusivity into the model; we keep pointwise scores and enforce it in the decision rule, and the decision model also sees, per record, the rank, best-other score and margin of the ranker and reader scores as features. In code, a record with a single candidate has its runner-up treated as minus infinity, so only the threshold applies; that is the common case, since 90.59 % of incoming records have exactly one candidate in the final candidate file. Under macro F0.5 the value of a link depends on the state of its business (break-even $F_b / 1.25$ in the toy), which motivates the count-dependent bar of the per-business consistency check (later part). Abstention also

creates a clean pool for targeted recovery: the second-chance searches run only on unlinked records, so they never undo a link.

HOW WE USED IT

Threshold 0.75 for US and India, 0.82 for France and the unseen-country route; a lead of 0.4 for every country; consistency-check bars 0.75 / 0.68 / 0.72 / 0.77; native-script rescue 0.88 with lead 0.3; address rescue US 0.86 with lead 0.0 and India 0.66 with lead 0.1 (Doc 4). The France rules can veto accepted links (the category-swap veto and the word-substitution veto together removed 29,021 of 887,630 accepted French links) or rescue unlinked records inside fixed score bands (Doc B.3). Outcome: 100,087 empty rows (5.8 %, derived) and 0.5875 links per incoming record, so about 41 % of incoming records are linked to nothing (derived). The Documentation does not record how 0.4 was chosen; say "one fixed lead used in every country", never "it was never tuned".

EXPERT NUANCE

Abstention costs recall exactly where a true owner has a near-twin; QA 14 lists "correct candidates rejected because a second one is too close" as a false-negative type. Alternatives include normalising scores across a record's candidates (softmax or listwise models), selective classification with risk-coverage curves (plotting the error rate among accepted links against the share of records accepted, and picking the point the metric prefers), or learning the reject rule itself. The empty rows are not all misses: many businesses may truly own nothing, but test labels are private, so we cannot split them. A wrong link is never better than abstaining (the owner loses the record either way, and the wrong business gains a false positive), which is why every refinement after the main decision either vetoes risky links or adds links only to unlinked records.

REMEMBER

- Link the best candidate only if confident (0.75; 0.82 in France) and a clear winner (lead at least 0.4); otherwise abstain.
- Toy break-even: a link helps business A only if its chance of being right exceeds $F_A / 1.25$.
- 100,087 empty rows; about 41 % of incoming records unlinked (derived).

JUDGE MIGHT ASK

- *Why require a 0.4 lead over the runner-up, not just a threshold?* "Two near-identical candidates can both score high, and a threshold alone would then pick one almost at random. Under F0.5 a wrong link costs more than a miss, so we link only a clear winner; 0.4 is one fixed lead for every country, and the threshold itself was tuned on fold 8."
- *How many records do you leave unlinked, and is that a problem?* "Our file links 0.5875 records per incoming record, so about 41 % stay unlinked, and 100,087 businesses have an empty row. Some of those records truly have no business; test labels are private, so we cannot say how many, but abstaining is never worse than a wrong link under this metric."

A1.8 Text normalisation

IN ONE LINE

Normalisation rewrites names and addresses into one standard form, so that the same thing written differently looks the same to a computer.

SIMPLE EXPLANATION

Two people copy the same shop sign: one writes "Shree Ganesh Traders & Co.", the other "SHREE GANESH TRADERS AND CO". A computer compares characters exactly, so these look different. Normalisation means rewriting both neatly in the same style before comparing: small letters only (**case folding**), punctuation removed, "&" written as "and", "Street" written as "st", "007" written as "7". It is like a teacher insisting that everyone writes dates as DD-MM-YYYY before checking homework. It must be done with care, because rewriting too aggressively can erase real differences. Four more terms: the **legal form** is the company-type word (Inc, LLC, Pvt Ltd, SARL), and the **core name** is the name with legal-form and filler words removed. **ASCII** is the basic computer alphabet of 128 characters: English letters, digits and common punctuation, with no accents and no other scripts. To **romanise** is to rewrite text from another script in Latin letters, and **unidecode** is a widely used generic library that does this character by character (it turns "é" into "e" and विजय into "vijy").

WORKED EXAMPLE (COMPUTED WITH OUR CLEANING CODE)

Raw text	After the US/India cleaning rules
Shree Ganesh Traders & Co.	shree ganesh traders and co
B00K WORLD	book world
Apt 007, 12 North Main Street, California	apt 7 12 n main st ca
Suite 0042, 5th Avenue	ste 42 5th ave
Legacy Marketing Enterprises (Inc)	legacy marketing enterprises inc

Then, for the index, the core name drops legal and filler words:

1. "shree ganesh traders and co" → core name "shree ganesh traders" ("and" and "co" removed).
2. "legacy marketing enterprises inc" → "legacy marketing enterprises".
3. The businesses "Legacy Marketing Enterprises Inc" and "Legacy Marketing Enterprises LLC" get the same core name, so the index retrieves both; the later stages must separate them (they do: 0.965022 against 0.000007 in Doc D.1).

TECHNICAL EXPLANATION

Our US/India cleaning rules run in this order: (1) a token containing non-ASCII characters is romanised, in names through the learned Indic-to-Latin word list with unidecode as fallback, in addresses always through unidecode; (2) lower-case; (3) "&" becomes " and "; (4) every run of characters outside a-z and 0-9 becomes one space; (5) in addresses, US and Indian state names become short codes (california → ca, maharashtra → mh) and a small table maps street, direction and unit words (road → rd, street → st, avenue → ave, north → n, suite → ste, apartment → apt, floor → fl, nagar → ngr, sector → sec), and every purely numeric token of up to 11 digits is rewritten as an integer, which drops leading zeros; (6) in names, a 0 next to a letter becomes o (repairing "leet" spellings such as B00K). The index then builds the core name by removing a fixed list of legal and filler words (inc, incorporated, llc, llp, ltd, limited, pvt, private, corp, corporation, co, company, plc, sarl, sas, sa, sci, sas, eurl, and, the); a name made only of such words is kept as it is. The same function runs on both sides of every pair, deterministically, once per record, so its cost is linear in the data. The hand-crafted features use a separate tokeniser (lower-case, punctuation removed, Indic word list, accents stripped, legal-form words and and/et/& removed). The readers read the raw text, not the normalised text (Doc 2.2).

HOW WE USED IT

The US/India rules serve the US, India and any unseen country (the default route); France adds French rules on top (see Address normalisation). The index lookup, the quick rule-based filter and the 76 index-lookup features all work on normalised text; the readers see the raw "name | address" text. Hand-written

lists are limited to small word lists for normalisation, decision-model features, the native-script rescue and the France rules; there is no external dictionary or lookup (Doc C.6).

EXPERT NUANCE

Normalisation is a recall-against-precision dial: collapsing variants raises the chance of retrieving the true business but erases distinctions. We erase distinctions only where recall matters (the index keys drop legal forms) and keep them where precision matters (legal-form relation features in the decision model; the readers on raw text), which is exactly what separates the Inc from the LLC in the US example. Abbreviations depend on context: "st" is street in a US address but saint in a French name or address, and "ste" is suite in a US address but sainte in French text, which is one reason France has its own rules. Table collisions such as goa → ga and georgia → ga are harmless because every index is built per country. Dropping leading zeros is safe because both sides are transformed the same way: a ZIP code 02134 becomes 2134 everywhere and still matches.

REMEMBER

- Lower-case, & → and, punctuation → space, abbreviation and state tables, leading zeros dropped, a 0 beside a letter → o.
- Legal words are removed only for the index's core name; the legal form stays a decision feature and the readers read raw text.
- One function for both sides of every pair; one shared US/India rule set (also used for unseen countries), with French rules on top for France.

JUDGE MIGHT ASK

- *Doesn't removing Inc or LLC throw away exactly the signal that separates look-alikes?* "Only inside the index, where we want recall: the Inc and the LLC are both retrieved. The decision model has legal-form features and the readers read the raw text, so in our US example the Inc scores 0.965 and the LLC 0.000007."
- *Why do the readers get raw text instead of the cleaned text?* "Cleaning is designed for retrieval: it throws away case, punctuation and accents, and the index's core name also drops legal forms, all of which can matter for the final decision. The readers are multilingual models fine-tuned on labelled pairs, so they learn their own invariances from the original 'name | address' text."

A1.9 Unicode, scripts and accent stripping

IN ONE LINE

Unicode gives every character a number, and Unicode normalisation makes visually identical text identical inside the computer, so that comparisons work.

SIMPLE EXPLANATION

A computer stores text as numbers. The letter "é" can be stored as one number (a ready-made é) or as two (a plain "e" followed by an accent sign), and both look identical on screen while the computer sees them as different. Some characters are decorated variants of plain ones, such as the ligature "fi" or a full-width "A". **Unicode normalisation** is the rulebook that rewrites all of these into one standard form.

Accent stripping then removes the accent signs, so "Société" and "Societe" compare equal. But careful: in Hindi, Tamil or Telugu, the small signs attached to letters are vowels that belong to the word; removing them destroys it.

The words you need: a **code point** is the number of one character (written U+00C9 and so on); a **script** is a writing system (Latin, Devanagari, Tamil); a **combining mark** is a character that attaches to the previous one (an accent, or an Indic vowel sign); **NFC, NFD, NFKC and NFKD** are the four standard normal forms.

WORKED EXAMPLE (COMPUTED WITH PYTHON'S UNICODATA AND TEXT-UNICODE)

Text	Code points	After NFKD	After deleting U+0300 to U+036F
É	U+00C9	U+0045 U+0301 (E + combining acute)	E
Société Générale			Societe Generale
fi (ligature)	U+FB01	f i	fi
A (full width)	U+FF21	A	A
①	U+2460	1	1
विजय (Vijay)	4 code points: letter VA, vowel sign I, letter JA, letter YA	unchanged	unchanged

The Devanagari vowel sign is a combining mark, but it lives outside the block U+0300 to U+036F, so our accent stripping leaves it alone. Deleting every combining mark would turn विजय into वजय, a different word.

TECHNICAL EXPLANATION

Unicode assigns code points to characters grouped into scripts (Latin, Devanagari U+0900 to U+097F, Tamil, Telugu and so on). Two kinds of equivalence exist: canonical (a precomposed é against e plus a combining acute), handled by NFC (composed) and NFD (decomposed), and compatibility (ligatures, full-width forms, circled digits, superscripts), which NFKC and NFKD also fold into plain forms. Accent stripping is NFKD followed by deleting the Combining Diacritical Marks block (U+0300 to U+036F). Each character also has a general category (L letters, M marks, N numbers, P punctuation, S symbols, Z separators), which lets code clean tokens without knowing the script. Indic scripts are abugidas (writing systems in which each consonant letter carries a default vowel that attached signs change; see Transliteration), and their vowel signs (matras) and virama are combining marks (categories Mn or Mc) that carry meaning, so cleaning for those scripts must keep category M. Our code does four things: before looking a native token up in the Indic word list it keeps only characters of categories L, M and N; the feature tokeniser applies NFKD and removes only U+0300 to U+036F, after the word list; native-script names are detected by the range U+0900 to U+0DFF (Devanagari through Sinhala); and the index text is pure ASCII after cleaning (non-ASCII tokens romanised, everything outside a-z0-9 removed), so the C++ index's byte-level letter pieces are exactly character-level letter pieces.

HOW WE USED IT

As described: accent stripping for feature tokens, category filtering for the word-list lookup, romanisation to ASCII for the index text. French accents disappear in the cleaning, because a token with non-ASCII characters is romanised by unicode ("Comité" → "comite", "Frères" → "freres"). The readers see the raw Unicode text through a multilingual subword tokeniser (which splits text into frequent word pieces from a vocabulary covering many scripts), so accents and native scripts reach them intact, within the 128-token input limit.

EXPERT NUANCE

NFKD is lossy ("²" becomes "2", ligatures split), which is fine for matching and wrong for display. Accent stripping can merge distinct French words (for example "pâte" and "pate"); the index accepts that loss for recall, and the readers still see the accents. **Homoglyphs** (a Cyrillic "a" that looks like a Latin "a") differ as code points; unicode maps them to Latin approximations. Running character algorithms on UTF-8 bytes is wrong for non-ASCII text, because one character can be several bytes; this is one reason the C++ index works only on romanised ASCII.

REMEMBER

- NFKD plus deleting U+0300 to U+036F strips Latin accents; never strip all combining marks, because Indic vowel signs are marks.
- Our index text is pure ASCII after cleaning, so byte n-grams equal character n-grams.
- The readers see the raw Unicode text.

JUDGE MIGHT ASK

- *How do you handle accents and non-Latin scripts?* "For the index and features everything becomes plain ASCII: French accents are stripped, and Indian-script name words go through a word list learned from labelled matches, with a generic romaniser as fallback. The readers see the original text with a multilingual tokeniser, so accents and native scripts reach them intact."

A1.10 Transliteration of Indian native scripts

IN ONE LINE

Transliteration rewrites a word from one script into another (विजय → vijay); we learned the mapping for business-name words from labelled training matches instead of relying on a generic romaniser.

SIMPLE EXPLANATION

If someone spells a name to you over a noisy phone line, you write it in English letters as best you can, and different people produce different spellings. Generic romanisers work letter by letter and often produce strange spellings, especially for English words written in Hindi letters: "एंटरप्राइजेज" is simply the English word "enterprises". Our training data already contained many correct pairs: a Devanagari incoming record and its business written in English letters. We counted which Latin word appears again and again with each Hindi word and built a small dictionary (a **lexicon**) from those counts, like learning a language from bilingual shop signs. Transliteration changes the script, not the language; **translation** would change the language.

WORKED EXAMPLE 1 (COMPUTED; THE LEXICON OUTPUT IS DOC D.2)

Native word	Generic romaniser (unicode, lower-cased)	Learned lexicon
विजय	vijy	vijay
एंटरप्राइजेज	enttrpraijej	enterprises
प्राइवेट	praaivett	private
लिमिटेड	limitedd	limited

The generic output shares no word with "Vijay Enterprises Private Limited"; the lexicon output is identical to it.

WORKED EXAMPLE 2 (ILLUSTRATIVE COUNTS): HOW ONE LEXICON ENTRY IS LEARNED

Suppose the token विजय appears in 40 labelled training records. In the true business names of those records, the Latin word "vijay" appears 38 times, "enterprises" 20 times and "traders" 6 times. Across all native-script training examples, these Latin words appear in business names 60, 2,000 and 900 times. Score = co-occurrences / (Latin word count + 1)^{0.6}:

Latin word	Co-occurrences	(count + 1) ^{0.6}	Score
vijay	38	61 ^{0.6} = 11.78	3.225
enterprises	20	2,001 ^{0.6} = 95.66	0.209
traders	6	901 ^{0.6} = 59.27	0.101

Checks: co-occurrences at least 3 (38, yes); purity $38 / 40 = 0.95$, at least 0.35 (yes); best score at least 1.8 times the runner-up (3.225 against $1.8 \times 0.209 = 0.376$, yes). So the lexicon gets the entry विजय → vijay. Dividing by the Latin word's frequency stops very common words such as "enterprises" from winning for every Hindi word.

TECHNICAL EXPLANATION

Indic scripts are **abugidas**: each consonant letter carries an inherent vowel "a", **vowel signs (matras)** replace it, the **virama** suppresses it, and conjuncts join consonants. In Hindi the inherent vowel is often silent at the end or in the middle of a word (schwa deletion), so letter-by-letter romanisation either drops every inherent vowel ("vijy", unicode's choice) or keeps them all ("vijaya"); the usual spelling "vijay" requires knowledge of the language. English loanwords spelled phonetically in Devanagari cannot be recovered reliably by any character-level script mapping, because the English spelling ("enterprises") is not a function of the sounds, and standard schemes such as ISO 15919 or IAST are reversible but do not produce English spellings. Our lexicon is supervised word alignment from parallel data: for every labelled training pair whose incoming name contains native script (folds 0-5 only), each native token is counted once per example against each word of its owner's normalised name (one-letter Latin words are ignored), and a token receives its best Latin word only if it co-occurs at least 3 times, its purity (the share of the token's training examples whose business name contains that Latin word) is at least 0.35, and its score $n / (\text{count of the Latin word} + 1)^{0.6}$ is at least 1.8 times the runner-up. The exponent 0.6 discounts frequent Latin words, an association statistic in the spirit of IDF or pointwise mutual information (PMI, which compares how often two words occur together with how often they would by chance); it is not a neural transliterator. Tokens without an entry fall back to unicode. The lexicon is used for name tokens in the cleaning step and in the feature tokeniser; addresses are always romanised with unicode.

HOW WE USED IT

The main lexicon is learned only from labelled training pairs of folds 0-5, the same folds as the CatBoost ranker and the readers. Two other word lists exist and are disclosed. The native-script rescue uses a lexicon of training positives that keeps every alternative with at least 5 % of a native token's count, generates up to 64 transliterations per name, and is built leaving the record's own fold out when training the rescue model (Doc 3, Doc 4). For India's forensic name-comparison features, a label-free native-to-Latin alignment is computed on test records whose first-stage top candidate has probability at least 0.9 (Doc C.1; code). The native-script rescue added 9,759 links in India; in Doc D.2 the key (vijay, enterprises) plus the state Maharashtra led to a link at 0.99994.

EXPERT NUANCE

Coverage is the main weakness: a native token seen fewer than 3 times in labelled data stays with unicode, and QA 14 lists "native-script names without a known transliteration" as a miss type. The consonant-skeleton channel of the index softens this, because unicode output and Latin spellings often share a skeleton: vijy and vijay both give "vj", and enttrpraiijej ("ntrprj") and enterprises ("ntrprs") share the sound 4-grams ntrp and trpr (see Phonetic keys). The readers read the native script directly with a multilingual vocabulary and learn cross-script matches from labelled pairs. Neural transliteration models are an alternative we did not use or benchmark.

REMEMBER

- Generic romanisation: विजय → "vijy", एंटरप्राइजेज → "enttrpraiijej"; the learned lexicon gives "vijay enterprises".
- Lexicon = co-occurrence counts on labelled folds 0-5, kept if $n \geq 3$, purity ≥ 0.35 and score $\geq 1.8 \times$ runner-up; unicode otherwise.
- The rescue lexicon leaves each record's own fold out; the label-free test alignment (probability ≥ 0.9) is disclosed.

JUDGE MIGHT ASK

- *How do you match a Devanagari name to a Latin business name?* "In three ways. The cleaning step maps Indian-script name words through a word list learned from labelled training matches, so विजय एंटरप्राइजेज becomes 'vijay enterprises', and the readers read the original script directly. Records that still stay unlinked get the native-script second-chance search, by transliterated name within the same state, which added 9,759 links in India."
- *Could the transliteration word lists leak labels?* "The main list is learned only from labelled training folds 0 to 5, and the second-chance search's list leaves each record's own fold out, so a record never benefits from its own label. One alignment is computed on test records, but it uses no labels, only confident first-stage matches, and the Documentation discloses it."

A1.11 Address normalisation and parsing (US, India, France)

IN ONE LINE

Address parsing splits an address into its parts (house number, street type, street name, city, region), and address normalisation writes each part in one standard way.

SIMPLE EXPLANATION

A postal sorter knows that "Rd", "Rd." and "Road" are the same, and that "15B" and "15 bis" mean the same house. Addresses are often the strongest evidence that two records describe the same shop: two shops can share a name, but rarely a house number and a street. Yet addresses are messy: abbreviations, typos, missing parts, different orders, different languages. So we wrote rules per country that put every address into a standard shape. In French addresses, "bis", "ter" and "quater" mark a second, third or fourth dwelling at the same number, like "15A" or "15B".

WORKED EXAMPLE (COMPUTED WITH OUR CLEANING CODE)

Raw address	Normalised
Apt 007, 12 North Main Street, California (US rules)	apt 7 12 n main st ca
N° 15B R. Suzanne Lenglen (France)	15 bis rue suzanne lenglen
2 R SUZANNE LENGLEN (France; the record of Doc D.3)	2 rue suzanne lenglen
8 q Rue Victor Hugo	8 quater rue victor hugo
12 Q. Wilson	12 quai wilson
3ter Av. des Champs	3 ter av des champs

1. "N°" is removed before a number; the glued "15B" is split into "15 bis".
2. The token right after the house number and its suffix is the street-type position, so a lone "R" there means "rue".
3. A lone "q" means "quater" when a street type follows it ("8 q Rue ..."), but "quai" (a riverside street) when it sits in the street-type position itself ("12 Q. Wilson"): parsing depends on position.
4. The typo table: a word seen at the street-type position in incoming records but never at that position in the businesses (and almost never anywhere in them), whose closest street-type word is unique and within one or two edits, is mapped to it (illustrative: "boulevrad" is one transposition from "boulevard"). The word "rute" is one edit from both "rue" and "route", so it is ambiguous and is not mapped.

TECHNICAL EXPLANATION

An address has components: house number (with a suffix such as bis, ter, quater or a letter), street type (rue, avenue, boulevard; Road, Street), street name, locality or city, department, region or state, and postcode. The US/India rules map state names to codes, apply a street, direction and unit word table, and rewrite numbers as integers. The French rules (France only) remove number prefixes such as N°, No and numéro before digits; split glued suffixes (15B → 15 bis, 3ter → 3 ter) and unify bis, ter and quater; locate the street-type position as the first token after the house number(s) and suffixes; apply single-letter abbreviations only at that position (r → rue, q → quai); canonicalise multi-letter street types (avenue / ave / av → av, boulevard / blvd / bd → bd, allée → all, impasse → imp, route → rte, chemin → ch, place → pl); and apply a street-type typo table learned from the supplied test files (a word of at least 4 letters seen at the type position in incoming records, never at that position in business addresses and almost never elsewhere in them, whose closest street-type word is unique and within optimal-string-alignment distance 1 for type words of up to 6 letters, or 2 for longer ones). The French geography (regions, departments, cities) is learned without labels from how comma-separated address components co-occur in the test files, and in code the region and department components are dropped by default. French names get their own rules: S.A.R.L. → sarl, leet legal forms such as 5arl → sarl, st / ste → saint / sainte, et → and (between two words), and a few abbreviations unified (Établissements → ets, Compagnie → cie, Frs → freres). Every rule runs once per record.

HOW WE USED IT

Normalised addresses feed the address-word channel and address score of the index lookup, the address-word and address-number statistics among its 76 features, and the address coverage used by the quick rule-based filter. Other components parse addresses in their own way: the "what changed" number features extract digit runs directly from the raw address; the France rules use their own parser of the raw French address for their "same address" tests and the street veto (which allows Damerau distance 1, or 2 for words of 8 or more characters) (code); and the address rescue retrieves businesses sharing one of a record's 3 rarest digit runs and one of its 3 rarest alphabetic address words. Disclosed: the French geography, the street-type typo table and the postcode prefixes are computed from the unlabelled test files (Doc C.1). No geocoder, postal table, business register or API was used; they were banned (Doc C.6).

EXPERT NUANCE

In production one would reach for a statistical address parser or a geocoder, but external data and lookups were banned, so the rules are hand-written per country plus tables learned from the supplied files. Learning tables from unlabelled test files is **transductive** (it uses the unlabelled test inputs themselves while building the system) but label-free, like computing word frequencies, and it is disclosed. House numbers are the most discriminative address tokens because digit strings are rare, which is why the address rescue starts from the rarest digit runs. Canonicalisation is limited to street types, suffixes and abbreviations; street-name words themselves are never dropped. Missing parts are common (the US example of Doc D.1 has no address at all), so every address feature must handle "empty" explicitly.

REMEMBER

- Parse first (number, suffix, type, name, city), then canonicalise each part; position matters ("q" is quater or quai).
- France: 15B → 15 bis, R. → rue, typo table and geography learned from the unlabelled test files (disclosed).
- No external postal data, geocoder or API.

JUDGE MIGHT ASK

- *Did you use a postal database, geocoder or address-parsing library?* "No. External data and lookups were banned and we used none. The parsing rules are hand-written per country, and the French

geography and street-type typo table are learned from the supplied files without labels; the Documentation discloses that."

- *Isn't learning tables from the test files a kind of leak?* "It is transductive: the tables are learned from the text of the unlabelled test records, and the Documentation discloses that. It uses no labels, in the same way as computing word frequencies, and it only makes 'avenue', 'av' and their typos one token; it never decides a match on its own."

A1.12 Tokens, sets and character n-grams

IN ONE LINE

To compare text we cut it into pieces, whole words (**tokens**) or short overlapping letter chunks (**character n-grams**), and compare the collections of pieces.

SIMPLE EXPLANATION

Comparing two names letter by letter is fragile, because one typo pushes every later letter out of line. Instead, cut each name into pieces and count the pieces they share, like comparing two Lego models by the bricks they use. Words are big pieces: "shree ganesh" and "shri ganesh" share only one of their three different words. Letter chunks are small pieces ("shr", "hre", "ree", ...): a typo spoils only a few of them, so the two names still share most chunks. A **set** keeps each piece once and ignores order; a **bag** (multiset) also counts repeats. An n-gram of 3 letters is a 3-gram (also called a q-gram or shingle).

WORKED EXAMPLE (COMPUTED WITH OUR INDEX RULES; LETTER PIECES ARE CUT FROM THE CORE NAME WITH SPACES REMOVED, SOUND PIECES FROM ITS WORDS SORTED ALPHABETICALLY)

"shree ganesh" against "shri ganesh".

Pieces	shree ganesh	shri ganesh	Shared	Union	Jaccard
Words	shree, ganesh	shri, ganesh	1	3	0.333
3-grams	shr hre ree eeg ega gan ane nes esh (9)	shr hri rig iga gan ane nes esh (8)	5	12	0.417
4-grams	shre hree reeg eega egan gane anes nesh (8)	shri hrig riga igan gane anes nesh (7)	3	12	0.250
Sound 4-grams (see Phonetic keys)	gnsh nshs shsh hshr	gnsh nshs shsh hshr	4	4	1.000

A second case: "book world" against "bookworld" share no word, but their compacted letter pieces are identical.

TECHNICAL EXPLANATION

Tokenisation here is splitting normalised text on spaces. Sets ignore order and repetition; bags keep counts. A string of length L has $L - n + 1$ character n-grams; in our code a string shorter than n becomes one single gram, with no padding. The **q-gram lemma** (Ukkonen) makes n-grams a filter for typos: one edit changes at most n of a string's n-grams, so two strings within edit distance k still share at least $(L - n + 1) - k \cdot n$ grams, where L is the length of the longer string and grams are counted with repeats (as bags; with sets the bound is approximate). Shorter n tolerates more typos but is less selective: short grams such as "ent" are shared by many unrelated names, which makes their posting lists long. Our index keeps five kinds of pieces: name words (of the core name) and address words (unique sets), 3-grams and 4-grams of the compacted core name in its original word order, and 4-grams of the consonant skeleton; each piece is hashed (turned by a fixed formula into a 64-bit number used as its key) and tagged with its channel, so the 3-gram "gan" and a word "gan" get different keys. Each key points to a **posting list**, the list of businesses that contain that piece (see The inverted index). For name words, address words, 3-grams, 4-

grams, sound grams, name numbers and address numbers, the index lookup outputs the intersection size, Jaccard, overlap coefficient and intersection over the larger set (see Set-overlap measures).

HOW WE USED IT

Retrieval channel weights are 1 for name words and address words, 0.45 for 4-grams, 0.22 for 3-grams and 0.18 for sound grams; the posting caps and budgets are 2,500 / 15,000 for 4-grams and 1,000 / 6,000 for 3-grams (Doc 3). The set statistics above are among the 76 index-lookup features used by the CatBoost ranker and the decision models.

EXPERT NUANCE

Letter pieces of the compacted name handle words that are split or merged ("book world" and "bookworld"), which word tokens miss; the price is that word boundaries vanish, which is harmless in practice. Very common pieces ("ent", "ers", "ing") carry little evidence, hence lower channel weights and IDF weighting. Sets ignore order, so "ganesh shree" equals "shree ganesh", usually what we want for names; order-sensitive measures (LCS ratio) complement them. With 64-bit hashed keys, collisions are possible in principle but negligible in practice.

REMEMBER

- Words are precise but brittle; letter n-grams survive typos and split or merged words.
- One edit spoils at most n of a string's n-grams (the q-gram lemma).
- Our index uses words, 3-grams, 4-grams and sound 4-grams, each with its own weight and caps.

JUDGE MIGHT ASK

- *Why add character 3-grams and 4-grams on top of words?* "Words miss typos and split or merged words: 'shree ganesh' and 'shri ganesh' share one word in three but five of twelve 3-grams. 4-grams are more selective and 3-grams more forgiving, so they carry weights 0.45 and 0.22 against 1 for whole words, each with its own posting cap."

A1.13 Edit distances: Levenshtein, Damerau-Levenshtein and OSA

IN ONE LINE

An **edit distance** counts the smallest number of single-character changes needed to turn one string into another.

SIMPLE EXPLANATION

How many keystrokes would a typist need to fix one name into the other? "shri" into "shree" needs two (change i to e, then add an e). The changes allowed are **insertion**, **deletion** and **substitution**; that count is the **Levenshtein distance**. Swapping two neighbouring letters ("gaensh" for "ganesh") is one of the most common typing slips; Levenshtein counts it as two changes, while the **Damerau-Levenshtein** distance also allows a **transposition** of neighbours and counts it as one. **OSA** (optimal string alignment) is a cheaper, restricted version of Damerau-Levenshtein.

WORKED EXAMPLE 1: THE DYNAMIC-PROGRAMMING TABLE FOR "SHRI" → "SHREE"

Dynamic programming means solving a big problem by filling a table of small sub-problems, each computed from neighbours already filled in. Here each cell is the distance between the prefixes (the first letters) of the two words; it is the minimum of (cell above + 1: delete), (cell to the left + 1: insert), and (diagonal cell + 0 if the two letters are equal, else + 1: substitute).

	""	s	h	r	e	e
""	0	1	2	3	4	5
s	1	0	1	2	3	4

	"	s	h	r	e	e
h	2	1	0	1	2	3
r	3	2	1	0	1	2
i	4	3	2	1	1	2

The answer is the bottom-right cell: 2.

WORKED EXAMPLE 2 (COMPUTED WITH RAPIDFUZZ)

Pair	Levenshtein	Damerau-Levenshtein	OSA
ganesh / gaensh	2	1	1
1250 / 1205 (house numbers)	2	1	1
boulevard / boulevard	2	1	1
ca / abc	3	2	3

"1250" and "1205" are 45 apart as numbers but one transposition apart as digit strings: two views of the same change. The last row shows why OSA is not a true distance. A true distance (a **metric**) must obey the **triangle inequality**: going directly from one string to another never costs more than going via a third string. Here "ca" → "ac" costs 1 and "ac" → "abc" costs 1, yet OSA("ca", "abc") = 3, more than 1 + 1.

TECHNICAL EXPLANATION

Levenshtein distance uses unit-cost insertions, deletions and substitutions and is computed by dynamic programming over an $(|a| + 1) \times (|b| + 1)$ table in $O(|a| \cdot |b|)$ time and $O(\min(|a|, |b|))$ memory; bit-parallel algorithms (Myers 1999, Hyrö), which pack one column of the table into the bits of a machine word so that one CPU instruction updates up to 64 cells at once, reduce this to about $\lceil |a| / 64 \rceil \cdot |b|$ word operations, which is how libraries such as RapidFuzz run fast. Unrestricted Damerau-Levenshtein adds adjacent transpositions and is a true metric. OSA, the restricted version, forbids editing a substring more than once; it is cheaper but violates the triangle inequality, as the "ca / abc" row shows. A normalised similarity is $1 - d / \max(|a|, |b|)$. The **Indel distance** (insertions and deletions only) equals $|a| + |b| - 2 \cdot \text{LCS}$, which links edit distances to the LCS ratio of the next concept. All of these are quadratic per pair in the worst case, so they run on candidate pairs only, never on all 6.7 trillion pairs.

HOW WE USED IT

The Damerau-Levenshtein distance between the digit strings of the closest pair of unmatched address numbers (one only in the record, one only in the business) is one of the "what changed" number features. OSA distance builds the French street-type typo table (at most 1 for words up to 6 letters, at most 2 for longer words), and the France street veto tolerates typos up to Damerau distance 1, or 2 for words of 8 or more characters (Doc B.3). The code's forensic name comparisons include a Levenshtein distance between names. All run on shortlisted or candidate pairs only.

EXPERT NUANCE

An edit distance counts edits but not their meaning: inc → llc is 2 edits (computed, after lower-casing) and changes the legal entity; shree → shri is also 2 edits and is only a spelling variant; a transposed house number is 1 edit and may point to another building. That gap is the motivation for the "what changed" features, which describe which edit happened (legal form changed, word replaced, number changed and by how much) and let the decision model learn which edits are harmless. Weighted edit distances (keyboard-aware or learned costs) exist; we did not use them. Raw distances favour short strings, so lengths are given to the models as separate features.

REMEMBER

- Levenshtein = minimum insertions, deletions and substitutions; Damerau-Levenshtein adds adjacent swaps and is a metric; OSA is the restricted version and is not.

- Dynamic programming, $O(|a| \cdot |b|)$: fine on shortlists, impossible on 6.7 trillion pairs.
- We use Damerau-Levenshtein on digit strings and OSA / Damerau tolerance in the French typo table and street veto.

JUDGE MIGHT ASK

- *Isn't an edit distance enough to match names?* "It counts edits but not which edit: Inc to LLC is two edits and a different company, Shree to Shri is also two and the same shop. So beside similarities we describe what changed, legal form, words added, dropped or replaced, numbers changed, and the decision model learns which changes are harmless."
- *Why Damerau-Levenshtein for house numbers?* "Swapped digits are a typical copying error, and Damerau counts a swap as one edit, so 1250 against 1205 is one edit although the numbers are 45 apart. We give the model both views, the digit edit distance and the numeric difference."

A1.14 LCS ratio and the RapidFuzz family

IN ONE LINE

These measures score, from 0 to 100, how much of two strings lines up in order, with variants that forgive extra words or a different word order.

SIMPLE EXPLANATION

Write two names one above the other and draw lines between equal letters, never letting lines cross. The largest number of lines you can draw is the **longest common subsequence (LCS)**. The **ratio** is how much of both names those lines cover. Two popular variants: **partial ratio** asks whether the shorter name appears almost exactly inside the longer one, and **token-set ratio** ignores word order and gives full marks when all the words of one name appear in the other. **RapidFuzz** is the open-source library (MIT licence) that we used to compute them.

WORKED EXAMPLE (COMPUTED)

1. $\text{LCS}(\text{"shree ganesh"}, \text{"shri ganesh"}) = \text{"shr"} + \text{" ganesh"} = 10$ characters. $\text{Ratio} = 2 \times 10 / (12 + 11) = 0.870$, which RapidFuzz reports as 86.96.
2. Three pairs:

Record	Business	Ratio	Partial ratio	Token-set ratio
vijay enterprises	vijay enterprises private limited	68.0	100	100
ganesh traders	shree ganesh traders mumbai	68.3	100	100
legacy marketing enterprises	legacy marketing industries	83.6	83.3	83.6

3. In the India example of Doc D.2, the chosen business has an address token-set ratio of 100 against 66.7 for each of the other four rescue candidates.

TECHNICAL EXPLANATION

The LCS is a subsequence, not a substring, and costs $O(n \cdot m)$ by dynamic programming, or $O(\lceil m / w \rceil \cdot n)$ word operations with bit-parallel algorithms (Allison-Dix, Hyvärö). The ratio $2 \cdot \text{LCS} / (|a| + |b|)$ equals $1 - \text{Indel} / (|a| + |b|)$, where $\text{Indel} = |a| + |b| - 2 \cdot \text{LCS}$ is the number of insertions and deletions; RapidFuzz's **ratio** is exactly this times 100. Our C++ index computes the same LCS ratio bit-parallel, with 128-bit masks over the first 127 characters of the shorter string and the first 192 of the longer one. Partial ratio is the best ratio between the shorter string and equally long alignments inside the longer string. Token-sort ratio sorts the words before comparing. Token-set ratio splits both strings into the shared word set I and the leftovers, compares " I " with " $I + \text{leftovers of } a$ ", " I " with " $I + \text{leftovers of } b$ ", and the two combined strings with each other, and takes the maximum, so it returns 100 when one word set contains the other

(and they share at least one word). The index lookup's **token alignment** scores take, for each word of one name, its best LCS ratio against any word of the other name and average these best matches, in both directions (2 scores), and also keep, in each direction, the worst word's best match (2 more); the averages are the **Monge-Elkan** construction with LCS ratio as the inner similarity. The index code is parallelised with **OpenMP**, a standard way to spread a C++ loop over several CPU cores.

HOW WE USED IT

Among the 76 index-lookup features are 7 LCS-ratio similarities (5 on name forms: the normalised name, the core name, the sorted core words, the compacted core name and the sound skeleton; 2 on address forms: the full address and its sorted unique words), 4 token-alignment scores and the common-prefix share of the compacted core names. RapidFuzz ratio and token-set ratio appear in the forensic name and address comparisons; ratio, partial ratio and token-set ratio in the second-chance models' features; the native-script rescue's fixed score includes the address token-set ratio; and the category-swap veto and generic-word swap veto in France require a street token-set ratio of at least 80 for "same address" (Doc B.3).

EXPERT NUANCE

Token-set ratio's subset rule cuts both ways: it forgives truncation (a true record that drops "Private Limited" scores 100), but it also forgives additions (a different organisation at the same address whose name is the business name plus one extra word also scores 100). A similarity score cannot tell "lost a word" from "gained a word"; the "what changed" features (words added versus dropped, and how rare they are) can, and that is the central idea of our decision features. LCS ratio respects order and set measures ignore it, so together they cover reordering and truncation. The bit-parallel LCS is effectively linear for names, because a whole name fits into one 128-bit mask.

REMEMBER

- $\text{Ratio} = 2 \cdot \text{LCS} / (|a| + |b|)$; RapidFuzz ratio is the same times 100; our C++ index computes it bit-parallel.
- Partial and token-set ratios forgive extra words, both truncations and additions.
- Token alignment = Monge-Elkan: the average best word match.

JUDGE MIGHT ASK

- *Which string similarity matters most in your model?* "We did not prepare a feature-importance ranking. The design point is that no single similarity is enough: token-set ratio gives 100 both when a true record drops 'Private Limited' and when a different organisation adds a word, so the decision model gets several similarities plus the 'what changed' features that say which side gained or lost words."
- *Why compute the similarities in C++?* "Because they run on every retrieved candidate, about 130 per business in our blocking study, roughly 225 million pairs for the test set (derived), each with 76 features. With a bit-parallel LCS in C++ and OpenMP, index lookup, quick filter and ranker together take about 1.3 hours on the test set."

A1.15 Jaro-Winkler similarity (not used by us)

IN ONE LINE

Jaro-Winkler is a classic typo-tolerant similarity for short names that gives extra credit when the beginnings agree; we did not use it.

SIMPLE EXPLANATION

It was designed for matching people's names in census records, where "Martha" is typed as "Marhta". It counts letters that match at nearby positions, penalises letters that appear swapped, and adds a bonus when the first few letters agree, because people rarely mistype the start of a name. It works well for short single names and less well for long multi-word business names.

WORKED EXAMPLE

"martha" against "marhta".

1. Matching window = $L6 / 2J - 1 = 2$ positions; all 6 letters find a match, so $m = 6$.
2. In matched order the letters read m a r t h a against m a r h t a: two positions disagree, which counts as $t = 2 / 2 = 1$ transposition.
3. $Jaro = (6/6 + 6/6 + (6 - 1)/6) / 3 = (1 + 1 + 0.833) / 3 = 0.944$.
4. The common prefix "mar" has length $l = 3$; with the usual $p = 0.1$, $Jaro\text{-}Winkler = 0.944 + 3 \times 0.1 \times (1 - 0.944) = 0.961$.

Our domain (computed with RapidFuzz): shree / shri 0.848; boulevard / boulevard 0.978 (LCS ratio 88.9); "legacy marketing enterprises" / "legacy marketing industries" 0.933 (LCS ratio 83.6).

TECHNICAL EXPLANATION

Two characters match if they are equal and no further apart than $L_{\max}(|a|, |b|) / 2J - 1$ positions; m is the number of matches and t half the number of matched characters that appear in a different order. $Jaro = (m / |a| + m / |b| + (m - t) / m) / 3$, or 0 when $m = 0$. Winkler's modification adds $l \cdot p \cdot (1 - Jaro)$, with l the common prefix length capped at 4 and p usually 0.1 (in Winkler's original, and in RapidFuzz, the bonus is applied only when Jaro exceeds a boost threshold of 0.7). The cost is about $O(|a| \cdot \text{window})$. It is not a metric. It became standard in census linkage of personal names (Winkler 1990); for multi-word strings it is often used as the inner similarity of Monge-Elkan.

HOW WE USED IT

Not used. Our character-level similarities are the LCS ratio and the edit distances; agreement at the start of a name is a separate feature (the common-prefix share of the compacted core names, and common prefix and suffix of digit strings), so the decision model learns how much the beginning matters.

EXPERT NUANCE

Business names often vary at the front: honorifics such as "Shree" or "M/s", a leading "The", or the legal form written first in French names ("SARL TQ ..."), so a fixed prefix bonus can mislead. Giving prefix agreement as its own feature lets the model weight it per country. We did not benchmark Jaro-Winkler, so we do not claim it would be worse.

REMEMBER

- Jaro counts nearby matches and transpositions; Winkler adds a prefix bonus ($p = 0.1$, up to 4 letters).
- Built for short personal names; not used by us.
- We give prefix agreement as its own feature instead.

JUDGE MIGHT ASK

- *Why not Jaro-Winkler, the classic record-linkage similarity?* "It is tuned for short single names whose beginnings are reliable; our names are multi-word and often vary at the front, for example the legal form written first in French names. We use LCS ratio, token-set and word-alignment measures and give the common prefix as its own feature. We did not benchmark Jaro-Winkler, so we don't claim it would be worse."

A1.16 IDF, TF-IDF, BM25 and our IDF-weighted overlap

IN ONE LINE

IDF (inverse document frequency) gives rare words more weight than common ones, because sharing a rare word is much stronger evidence of a match.

SIMPLE EXPLANATION

In a phone book, two entries sharing the word "Traders" tells you almost nothing, because thousands of shops are traders; two entries sharing "Legacy" tells you a lot. The **document frequency (df)** of a word is the number of records that contain it; IDF turns "few records contain it" into "high weight". **TF** (term frequency) counts how often a word appears inside one record. **TF-IDF** and **BM25** are standard search-engine formulas that combine both; ours is a simpler cousin built for short names, which uses IDF only.

WORKED EXAMPLE (N = 663,106 US TEST BUSINESSES; DF VALUES ILLUSTRATIVE)

Word	df (illustrative)	IDF = $\ln(1 + (N + 1) / (df + 1))$
legacy	300	7.698
marketing	8,000	4.429
industries	6,000	4.714
enterprises	20,000	3.531
a word in every business	663,106	0.693 (= $\ln 2$, the minimum)
a word in no business	0	13.405 (formula value; see the note below)

Record "legacy marketing enterprises" (no address); three businesses, each with 3 core words, so each is divided by $(1 + 3)^{0.22} = 1.3566$:

Business	Shared words	IDF sum	Name-word score	Share of best
A: Legacy Marketing Enterprises	all three	15.658	11.542	1.000
B: Legacy Marketing Industries	legacy, marketing	12.127	8.940	0.775
C: Marketing Enterprises Group	marketing, enterprises	7.960	5.868	0.508

B beats C because they share "marketing" and differ only in the other shared word: "legacy" (7.7) is worth more than twice "enterprises" (3.5). (This shows the word channel only; real name scores also add the letter-piece and sound channels.) Note on the last IDF row: a word that no business contains has no posting list, so it retrieves nothing; when the index code totals a record's IDF for its features, it gives such an unseen term a fixed weight of 12 (code).

TECHNICAL EXPLANATION

The classic IDF is $\log(N / df)$; smoothed variants avoid division by zero and negative weights. Ours is $\log(1 + (N + 1) / (df + 1))$ with the natural logarithm, N the number of businesses of the country and df the number containing the term; its minimum is $\ln 2$ when every business contains the term, and for rare and mid-frequency terms it is close to the BM25 IDF of Lucene (the open-source search library behind Elasticsearch and Solr), $\log(1 + (N - df + 0.5) / (df + 0.5))$ (both behave like $\ln(N / df)$); they differ only for terms in almost every business, where Lucene's value falls towards 0. TF-IDF cosine builds vectors of $tf \cdot idf$ weights and compares them by dot product divided by both vector lengths. BM25 scores $\sum idf \cdot tf \cdot (k_1 + 1) / (tf + k_1 \cdot (1 - b + b \cdot dl / avgdl))$, with k_1 about 1.2 to 2 and b about 0.75: term frequency saturates and long documents are penalised. Our score has no TF, because records are sets of unique terms: it sums channel weight $\times$ IDF over shared terms and divides by $(1 + \text{the business's word or gram count})^\gamma$, with $\gamma = 0.22$ for name words, 0.38 for the letter-piece and sound evidence (normalised by the business's 4-gram count) and 0.30 for address words. That sublinear penalty for long business names, which match many query terms by chance, plays the role of BM25's b ; the query's own length does not matter, because only

candidates of the same record are compared with each other. The total is $ns + as + 1.2 \cdot \min(ns, as)$ (Doc 3). The "what changed" word features use a slightly different, also label-free IDF, $\log((N + 1) / (df + 1))$ over all records of the country, businesses and incoming records together.

HOW WE USED IT

IDF weights every index channel (computed over the country's businesses), the IDF-overlap features of the index lookup, the address coverage of the quick rule-based filter (the IDF share of the record's address words found in the business address), the rarity features of the "what changed" family, and the IDF containment that ranks address-rescue candidates. On the test set these statistics are computed from the unlabelled test files themselves, which the Documentation discloses (Doc C.1). We did not use BM25 or TF-IDF cosine.

EXPERT NUANCE

Term frequency rarely exceeds 1 in a business name, so the TF part of TF-IDF or BM25 adds little, and cosine's normalisation by query length is irrelevant when ranking the candidates of one query; a per-channel length exponent is simpler and transparent. IDF computed on the test files is transductive: a word frequent in this market gets a low weight, which is what we want, and no label is involved. A typo has df near 1 and therefore maximal IDF; that is harmless for retrieval (it matches nothing) but inflates "rarity" features, which the letter-piece channels and the decision model absorb. We did not benchmark BM25 against our scoring, so we claim nothing either way.

REMEMBER

- $IDF = \log(1 + (N + 1) / (df + 1))$: rare words weigh more; the minimum is $\ln 2$.
- Our retrieval score is an IDF-weighted set overlap with per-channel length penalties: no TF, not BM25, not cosine.
- IDF statistics come from the unlabelled records at run time (label-free, disclosed).

JUDGE MIGHT ASK

- *Why not BM25 or TF-IDF cosine for retrieval?* "Names and addresses are short sets where a word rarely repeats, so the term-frequency part adds little. We use a smoothed IDF-weighted overlap with a length penalty per channel, rarest terms first. We did not benchmark BM25 against it; ours keeps the true business for 99.37 % of US and 98.51 % of Indian held-out records."
- *Where do your IDF statistics come from, train or test?* "From the records being processed: for the index and its features, the businesses of the country; for the 'what changed' features, all records of the country. On the test set that means the unlabelled test files; no labels are involved, and the Documentation discloses these unsupervised statistics."

A1.17 Set-overlap measures: Jaccard, overlap coefficient and IDF-weighted versions

IN ONE LINE

Set-overlap measures count how many pieces two records share, relative to how many pieces they have.

SIMPLE EXPLANATION

Compare two shopping lists. **Jaccard** = items on both lists divided by items on either list. **Overlap coefficient** = items on both lists divided by the length of the shorter list, so a short list contained entirely in a long one scores 1. **Dice** = twice the shared items divided by the two lengths added together. Weighted versions count rare items (saffron) more than common ones (salt), using IDF.

WORKED EXAMPLE

Record words {legacy, marketing, enterprises} against business words {legacy, marketing, industries}.

1. Shared = 2; union = 4.
2. Jaccard = $2 / 4 = 0.500$; overlap coefficient = $2 / 3 = 0.667$; Dice = $2 \times 2 / (3 + 3) = 0.667$; shared over the larger set = $2 / 3$.
3. IDF-weighted Jaccard with the illustrative IDFs of the previous concept: $(7.698 + 4.429) / (7.698 + 4.429 + 3.531 + 4.714) = 12.127 / 20.372 = 0.595$.
4. For comparison, the real IDF-weighted Jaccard of "Legacy Marketing Industries Inc" in Doc D.1 is 0.52, computed with the real IDF values.
5. Truncation case: {ganesh, traders} against {shree, ganesh, traders} gives Jaccard $2 / 3 = 0.667$ but overlap coefficient $2 / 2 = 1.000$.

TECHNICAL EXPLANATION

Jaccard $J = |A \cap B| / |A \cup B|$ (and $1 - J$ is a metric); Dice = $2 |A \cap B| / (|A| + |B|)$; the overlap or Szymkiewicz-Simpson coefficient = $|A \cap B| / \min(|A|, |B|)$; containment of A in B = $|A \cap B| / |A|$, which is asymmetric. Weighted versions replace counts by sums of weights, for example weighted Jaccard = $\sum \text{over } A \cap B \text{ of } w / \sum \text{over } A \cup B \text{ of } w$. With both sets stored as sorted lists of hashed terms, each measure costs one merge, $O(|A| + |B|)$. The index lookup outputs intersection, Jaccard, overlap coefficient and intersection over the larger set for seven sets (name words, address words, 3-grams, 4-grams, sound grams, name numbers, address numbers), plus six IDF overlaps (shared IDF, and shared IDF as a share of the record's and of the business's total, for name and for address). The quick rule-based filter uses address coverage, the IDF share of the record's address words found in the business's address, a containment measure. The "what changed" family includes the IDF-weighted Jaccard of the two core-name word sets, with legal-form words removed.

HOW WE USED IT

As listed: in the 76 index-lookup features, in the quick filter's coverage conditions (0.95, 0.4, 0.3), and in the "what changed" features (Doc 3, Doc 4).

EXPERT NUANCE

The measures disagree exactly where it matters, on subsets: a truncated true record and a look-alike that adds a word both give an overlap coefficient of 1 with a lower Jaccard. Direction matters (record contained in business means the record lost words; business contained in record means the record gained words), so we give shares relative to each side plus the "what changed" features that say which side changed. Word-set measures are brittle to typos, hence the letter-piece sets. MinHash can estimate Jaccard very quickly for huge collections (see Other blocking families); with an inverted index we compute it exactly on candidates.

REMEMBER

- Jaccard = shared / union; overlap coefficient = shared / smaller set; Dice = $2 \cdot \text{shared} / (\text{sum of sizes})$.
- IDF-weighted versions make rare shared words count more.
- Symmetric measures cannot tell dropped words from added words; direction needs its own features.

JUDGE MIGHT ASK

- *Why give the model both Jaccard and the overlap coefficient?* "They disagree exactly on subsets: a record that drops a word from the business name has overlap coefficient 1 but a lower Jaccard, and so does a record that adds a word. The model sees both, plus shares relative to each side and the 'what changed' features that say which side gained or lost words."

A1.18 Phonetic keys: Soundex and Metaphone (not used) and our consonant skeleton

IN ONE LINE

A **phonetic key** writes a name roughly as it sounds, so spellings that sound alike get the same key.

SIMPLE EXPLANATION

People spell what they hear: Mohammed, Mohamed; Shree, Shri. Vowels vary most between spellings, while consonants form the skeleton of a name. Our key throws away the vowels (and y) and squeezes repeated consonants, keeping a **consonant skeleton**. **Soundex** and **Metaphone** are older, English-specific versions of the same idea, with rules for how English letters sound.

WORKED EXAMPLE (COMPUTED WITH OUR INDEX RULES)

Name	Key (vowels and y removed, repeats collapsed)
shree ganesh traders	gnsh shr trdrs
shri ganesh traders	gnsh shr trdrs (identical)
mohammed traders	mhmd trdrs
mohamed traders	mhmd trdrs (identical)
vijy (generic romaniser) / vijay (Latin)	vj / vj
praaivett / private	prvt / prvt
enttrpraaiej / enterprises	ntrprj / ntrprs (share the 4-grams ntrp and trpr)
mona / mina	mn / mn (a false collision)

The key is built on the sorted core words, with spaces removed, then cut into 4-grams: "gnshshrtrdrs" → gnsh, nshs, shsh, hshr, shrt, hrtr, rtrd, trdr, rdrs. The praaivett / private row shows the rule on single words only: in the index, "private" is a legal word and is removed from the core name before the key is built. For comparison, classic Soundex gives Robert → R163 and Rupert → R163.

TECHNICAL EXPLANATION

Soundex (patented in 1918) keeps the first letter, maps the following consonants to six digit classes, drops vowels and h, w, y, collapses repeated digits and pads to four characters. Metaphone and Double Metaphone apply dozens of English pronunciation rules; NYSIIS is similar. All of them encode English pronunciation. Our key is language-neutral: delete a, e, i, o, u and y, and collapse repeated consonants (vowels are skipped before the comparison, so consonants separated only by vowels also collapse); it is applied to the sorted unique core-name words, spaces are removed, and the result is cut into 4-grams. A skeleton shorter than 4 letters ("vj") becomes a single gram. The sound channel has the lowest weight (0.18) and the tightest posting cap and budget (800 / 4,000); the index lookup also computes one LCS-ratio similarity on the sound string and set statistics of the sound grams (Doc 3).

HOW WE USED IT

As one of the five retrieval channels and inside the 76 index-lookup features. Soundex, Metaphone and NYSIIS are not used.

EXPERT NUANCE

The key is lossy by design: unrelated names collide (mona and mina both give "mn"), so it is a weak channel that only proposes candidates, and the later stages decide. It works across scripts only after romanisation, but it helps exactly there: a generic romaniser's output and the Latin spelling often share a skeleton or several sound 4-grams (vijy / vijay; enttrpraaiej / enterprises), which partly covers native-script words missing from the learned lexicon. English pronunciation rules would mishandle Indian and French names; dropping vowels is neutral across languages.

REMEMBER

- Our key: remove vowels and y, collapse repeated consonants, cut into 4-grams; weight 0.18.
- It merges spelling variants (shree / shri, mohammed / mohamed) and many romanisations; it also merges unrelated names, so it only proposes candidates.
- Soundex, Metaphone and NYSIIS are not used: they are English-centric.

JUDGE MIGHT ASK

- *Why not Soundex or Metaphone?* "They encode English pronunciation rules, and our names are Indian, French and American, often romanised from other scripts. A language-neutral consonant skeleton catches Shree against Shri and Mohammed against Mohamed, and it is only one retrieval channel with a low weight, so its false collisions are filtered later."

A1.19 Blocking and candidate generation: reduction ratio, pair completeness, pairs quality

IN ONE LINE

Blocking (candidate generation) quickly picks, for each incoming record, a small set of plausible businesses, so that the expensive comparison runs only on those pairs.

SIMPLE EXPLANATION

A librarian asked for "Ganesh Traders, Pune" does not read all 1.7 million catalogue cards; she opens the catalogue at "Ganesh" and at "Pune" and pulls out a handful. Blocking is that catalogue step. It must be fast, it must be small (few cards pulled), and it must be safe (the right card is almost always among them). If the right card is not pulled, no amount of careful reading later can find it. The name comes from the oldest form of the idea: put records into "blocks" that share a **blocking key** (for example the same postcode) and compare only inside a block.

The words you need: the **comparison space** is the set of all possible pairs; the **reduction ratio (RR)** is the share of that space we do not keep; **pair completeness (PC)** is the share of true matches we keep (the recall of blocking); **pairs quality (PQ)** is the share of kept pairs that are true matches (the precision of blocking).

WORKED EXAMPLE

Toy (illustrative): 1,000 records $\times$ 1,000 businesses = 1,000,000 pairs, with 1,000 true matches. Blocking keeps 3,000 pairs, 950 of them true matches.

1. $RR = 1 - 3,000 / 1,000,000 = 0.997$.
2. $PC = 950 / 1,000 = 0.95$.
3. $PQ = 950 / 3,000 = 0.317$.

Our test set (Doc 3; per-country products derived):

Country	Businesses $\times$ incoming records	Pairs
US	663,106 $\times$ 3,817,031	2,531,096,158,286
India	809,986 $\times$ 4,717,565	3,821,161,604,090
France	259,452 $\times$ 1,434,993	372,311,803,836
All		6,724,569,566,212

The candidate file holds 12,293,019 pairs, so $RR = 1 - 12,293,019 / 6,724,569,566,212 = 0.99999817$: about 1.83 pairs kept per million (derived). PC at the index lookup on labelled fold 9: 99.3694 % (US) and 98.5068 % (India).

TECHNICAL EXPLANATION

A blocking method maps each record to keys, or runs a ranked search, so that only pairs sharing a key or retrieved by the search become candidates. The main families are standard blocking on an exact key, multi-pass blocking (the union of several keys), token and q-gram blocking, sorted neighbourhood, canopy clustering, locality-sensitive hashing, dense vector retrieval, and ranked top-k retrieval, which is ours. The standard quality measures (Christen) are $RR = 1 - |C| / |A \times B|$, $PC = |C \cap M| / |M|$ and $PQ = |C \cap M| / |C|$, for a candidate set C and true matches M ; a blocking method must be judged on RR and PC together, and the harmonic mean of the two is sometimes reported. In many-to-one linking it is natural to measure PC per owned incoming record: the share whose owner is among its candidates. With a capped top-k per record and capped posting lists, total work is proportional to the number of records times a bounded cost, instead of $|A| \cdot |B|$. Blocking sets the recall ceiling for every later model (see Recall ceiling).

HOW WE USED IT

All retrieval stays within one country. The funnel per business is about 130 candidates from the index lookup (blocking study of 25 Sep 2026), 21.71 after the quick rule-based filter, 6.75 in the shortlist (derived) and 7.10 in the candidate file, which also contains the second-chance candidates (Doc 3). Per incoming record that is 1.23 candidates; 90.59 % of records have exactly one, all but 2 have at least one, and 125 businesses have no candidate (Doc B.1). The size of the candidate file was itself judged (candidate efficiency), so a small file was a goal in its own right.

EXPERT NUANCE

RR on its own is meaningless (keeping nothing gives $RR = 1$), and at our scale it moves in the seventh decimal: each 0.0000001 of RR is about 672,457 pairs (derived). PQ was not reported; we would compute it on labelled folds as true matches per kept pair. Candidate counts are heavily skewed, with a median of 7 per business but a maximum of 3,182 (Doc B.1), so any average hides long tails. The blocking key choice is a modelling decision: every key encodes an assumption about how true matches can differ, which is why we combine several channels.

REMEMBER

- Blocking = a cheap search that bounds what the expensive models see; it sets the recall ceiling.
- RR 0.99999817 (12,293,019 of 6,724,569,566,212 pairs); read it only together with recall.
- Pair completeness at the index lookup on fold 9: 99.37 % US, 98.51 % India.

JUDGE MIGHT ASK

- *What is your reduction ratio, and is it meaningful on its own?* "0.99999817: 12.3 million candidate pairs out of 6.7 trillion same-country pairs, fewer than two per million. On its own it isn't meaningful, since keeping nothing scores 1; it matters together with recall, and the true business is still among the index results for 99.37 % of US and 98.51 % of Indian held-out records."
- *Why search only within a country?* "Every record carries a country, and all our retrieval stays inside it. That shrinks the comparison space, keeps each index in memory, and lets each country use its own cleaning rules, readers and decision model."

A1.20 The inverted index

IN ONE LINE

An **inverted index** maps every word (or letter piece) to the list of businesses that contain it, so candidates are found by looking words up instead of scanning everything.

SIMPLE EXPLANATION

It is the index at the back of a book: "Ganesh: pages 12, 57". To find businesses that share words with an incoming record, look up each of the record's words, collect the businesses listed, and give each business points for every word it shares, more points for rarer words. Very common words, like "the" in a book, have enormous lists and say little, so we skip lists that are too long and stop once we have read enough.

The words you need: a **posting list** is the list of businesses for one word; one entry of it is a **posting**; an **accumulator** is the running score of each business during a lookup; a **posting-list cap** is the longest list we are willing to read; a **budget** is the total number of postings we read per record and channel.

WORKED EXAMPLE

A tiny index of three businesses: B1 "shree ganesh traders", B2 "ganesh sweets", B3 "vijay traders".

Word	Posting list
ganesh	B1, B2
shree	B1
sweets	B2
traders	B1, B3
vijay	B3

Query "shri ganesh traders":

1. "shri" has no list (no business contains it).
2. "ganesh" adds IDF(ganesh) to B1 and B2.
3. "traders" adds IDF(traders) to B1 and B3.
4. Accumulators: B1 has both words, B2 and B3 one each, so B1 ranks first. We read 4 postings instead of comparing against every business.

Caps and budget (list lengths illustrative; the limits are the real name-channel values, cap 14,000 per list and budget 22,000 per record), with words processed rarest first:

Query word	List length	Action	Postings used so far
vijay	3,000	read	3,000
shree	9,000	read	12,000
traders	12,000	skipped: 12,000 + 12,000 would exceed the 22,000 budget	12,000
enterprises	40,000	skipped: longer than the 14,000 cap	12,000

TECHNICAL EXPLANATION

To build the index, each business's unique terms are hashed to 64-bit keys and the business id is appended to each term's posting list in a hash map (a table that finds the entry for a key in roughly constant time); IDF is computed once per term. A lookup accumulates scores term at a time into a per-thread dense array with one slot per business of the country, plus a list of "touched" businesses so that resetting costs only what was touched (code). Terms are processed rarest first; a term whose list is longer than the channel's cap is skipped (frequency-based stop-wording), and a term whose list would push the channel past its budget is skipped too; because lists come in increasing length, no later term of that channel can fit either. This bounds the work per record to at most 65,000 postings across the five channels plus up to 2,000 businesses from the exact-name key (derived from the budgets in Doc 3). After accumulation, the name score, address score and total are computed for the touched businesses, partial sorts pick the top lists, and the 76 features are computed for the kept candidates. Incoming records are processed in batches (20,000 by default), parallelised with OpenMP and dynamic scheduling, so that threads that finish early take more records (code). Cost: building is linear in the total number of business

terms, and a lookup is linear in the postings read plus the businesses touched. Search engines also use document-at-a-time strategies with dynamic pruning (WAND, MaxScore), which skip documents that provably cannot enter the top k and so return the exact top-k; our caps are a simpler, static form of pruning that does not guarantee the exact top-k of the uncapped score.

HOW WE USED IT

C++17 with OpenMP, one index per country over normalised business text, five channels plus the exact-name key, with the caps and budgets above (Doc 3); the lookup ran on 8 to 11 threads, and index lookup, quick filter and ranker together took about 1.3 hours on the test set (clean-run report). No embeddings, no vector search and no learned model in this stage. Recall at the index on fold 9: 99.3694 % (US) and 98.5068 % (India).

EXPERT NUANCE

Skipping very common terms is a recall risk for records whose words are all common ("Ganesh Traders"); the exact sorted core-name key, the letter-piece and sound channels (each with its own cap) and the address channel compensate, and the quick filter's address conditions keep address-only candidates. Unlike approximate nearest-neighbour search with randomised index construction, an inverted index with fixed caps is deterministic: it is a pruned search, not an exact top-k, but the pruning is a fixed rule, so the same input always gives the same candidates, which makes it easy to audit and reproduce (from the trained models, our inference stages rewrite both output files byte for byte). The per-thread accumulators are small compared with the posting lists, so memory is dominated by the index itself.

REMEMBER

- Posting list = term → businesses containing it; a lookup adds IDF for every shared term.
- Rarest first; skip lists longer than the cap; stop at the budget: bounded work per record.
- Deterministic and auditable (a fixed pruning rule); C++ with OpenMP; about 1.3 hours for lookup, filter and ranker on test.

JUDGE MIGHT ASK

- *What happens to a record whose words are all very common, like 'Ganesh Traders'?* "Very common words exceed the posting cap and are skipped, but the exact-name channel still returns every business with exactly that core name if at most 2,000 share it, and the letter-piece, sound and address channels have their own caps. Then the address usually separates them."
- *How do you bound the cost per record?* "Every channel has a cap per posting list and a total budget, processed rarest first, so one record reads at most about 65,000 postings plus up to 2,000 exact-name matches (derived from the budgets), and we keep a fixed number of top candidates. Cost grows linearly with the number of records."

A1.21 Multi-channel retrieval and the union of top lists

IN ONE LINE

We search five ways at once (name words, address words, two kinds of letter pieces, sounds) plus an exact-name key, and keep the union of several top lists, so that a candidate strong on only one side is not lost.

SIMPLE EXPLANATION

A detective questions several witnesses: one remembers the name, one the street, one only how the name sounded. If you trust only the combined story, you miss suspects that just one witness saw. So we keep the top suspects overall, plus the top suspects by name alone, plus the top suspects by address alone, plus

anyone with exactly the same name. Each witness is a **channel**; keeping the union of several lists is called **multi-pass blocking**.

WORKED EXAMPLE (ILLUSTRATIVE SCORES)

total = ns + as + 1.2 × min(ns, as), where ns is the name score and as the address score.

Candidate	ns	as	Total
X: right name, address does not match	9.0	0.0	9.0
Y: both match moderately	4.5	4.5	14.4
Z: address only	0.0	6.0	6.0

1. By total the order is Y, X, Z. If we kept only the top 2 by total, Z, an address-only match such as a brand name at the right address, would be lost; the top-by-address list keeps it.
2. Y gets a bonus of $1.2 \times 4.5 = 5.4$ for agreeing on both fields; X gets none.
3. Exact-name key: "Legacy Marketing Enterprises Inc" and "Legacy Marketing Enterprises LLC" both have the key "enterprises legacy marketing" (sorted core words), so the record "Legacy Marketing Enterprises (Inc)" retrieves both whatever their scores.

TECHNICAL EXPLANATION

The channels (weight; posting cap / budget) are: name words (1; 14,000 / 22,000), address words (1; 10,000 / 18,000), core-name 4-grams (0.45; 2,500 / 15,000), core-name 3-grams (0.22; 1,000 / 6,000) and sound 4-grams (0.18; 800 / 4,000). The name score is $ns = \text{name-word IDF sum} / (1 + \text{the business's name words})^{0.22} + \text{letter-piece and sound sum} / (1 + \text{the business's 4-grams})^{0.38}$; the address score is $as = \text{address IDF sum} / (1 + \text{the business's address words})^{0.30}$; total = ns + as + 1.2 · min(ns, as). Kept per incoming record: the top 12 by total, the top 6 by ns, the top 6 by as (only if as > 0), and every business with the same sorted core-name word set if at most 2,000 businesses share it (Doc 3). This union is multi-pass blocking in ranked form. For every kept candidate the index lookup computes 76 pair features, including 8 retrieval scores: ns, as, total, total divided by the best total, the best total, the second-best total, best minus second, and the raw candidate count. In the all-fold samples the index lookup returns 27.541 (US) and 23.145 (India) candidates per incoming record (Doc B.2).

HOW WE USED IT

Exactly as above; the 76 features feed the quick rule-based filter, the CatBoost ranker and the decision models. Recall at the index lookup on fold 9: 99.3694 % (US) and 98.5068 % (India).

EXPERT NUANCE

The min term acts as a soft AND: with a plain sum, a strong name-only match could outrank a moderate match on both fields, which is usually the better candidate. The separate lists protect the two recall risks that matter most: a missing or wrong address (the name list) and a brand or renamed business at the right address (the address list). The exact-key channel guarantees that identical core names are always compared, even when their words are too common to pass the caps. The weights, caps and exponents are fixed values reported in the Documentation; the evidence we report for them is the measured recall, not a tuning study.

REMEMBER

- Five channels plus the exact core-name key; total = ns + as + 1.2 · min(ns, as).
- Keep top 12 by total ∪ top 6 by name ∪ top 6 by address ∪ same core name (if at most 2,000 share it).
- The union is what makes retrieval recall-first; 76 features per kept candidate.

JUDGE MIGHT ASK

- *Why keep separate top lists by name and by address?* "A record can match its business on only one side: the right name with no address, or a brand name at the right address. A combined score would

bury those, so beside the top 12 overall we keep the top 6 by name and the top 6 by address."

- *What is the exact-name channel for?* "It returns every business whose core name, as a sorted word set, equals the record's, if at most 2,000 share it. That guarantees, for example, that 'Legacy Marketing Enterprises Inc' and the LLC are both compared even when their words are too common to pass the caps; the later models then separate them."

A1.22 The quick rule-based filter

IN ONE LINE

A nearly free, model-less rule drops candidates that are clearly weaker than the record's best one, while keeping special cases that a single score would wrongly drop.

SIMPLE EXPLANATION

Think of a first interview round where the receptionist uses a checklist: keep the top applicant, anyone close to the top, the best at one particular skill, and a few special profiles; send the rest home. The checklist costs almost nothing, and it shrinks the pile from about 130 to about 22 candidates per business before the expensive interviews begin. Because it is a fixed rule, not a trained model, it is called **deterministic pruning**.

WORKED EXAMPLE (ILLUSTRATIVE)

one record's candidates.

Candidate	Total	Share of best total	Address coverage	Kept? Why
Y	14.4	1.00	0.60	yes: best total
X	9.0	0.63	0.00	yes: at least 0.6 of the best
Z	6.0	0.42	0.97	yes: best address score, and covers at least 0.95 of the record's address
W	5.0	0.35	0.50	no
V	4.0	0.28	0.10	no

Real case (Doc D.3): for "TQ COMITE SARL" the index lookup found 16 candidates, the quick filter kept 2, and both entered the shortlist.

TECHNICAL EXPLANATION

A candidate is kept if any of six conditions holds: (1) it has the record's best total (ties kept); (2) its total is at least $0.6 \times \max(1, \text{best total})$, where the $\max(1, \cdot)$ guard stops the rule from keeping everything when all scores are tiny; (3) it is the record's best candidate by address score (with $as > 0$); (4) its address coverage is at least 0.95, coverage being the IDF share of the record's address words found in the business's address; (5) the record's name has at most 3 characters and coverage is at least 0.4 (initials and acronyms, whose name scores are unreliable); (6) it has the same sorted core-name word set and coverage at least 0.3 (Doc 3). The inputs are retrieval statistics the index lookup has already computed, so the cost is linear in the candidates and needs no model call. Conditions 3 to 6 are recall protection for address-only, short-name and same-name candidates. Result: 37,614,782 pairs, 21.7107 per business (US 15.0352, India 23.6328, France 32.7713), against 130.1 per business before pruning in the blocking study of 25 Sep 2026; in the all-fold samples 3.111 (US) and 3.878 (India) candidates per incoming record remain (Doc 3, Doc B.2). The decision models still receive the raw candidate count from before the filter (code).

HOW WE USED IT

Between the index lookup and the CatBoost ranker, on every country. Its measured cost, from the blocking study of 25 Sep 2026 with the decision model of that date: the owner survived for 99.974 % of fold-8/9 records, and fold-9 macro F0.5 moved from 0.990117 to 0.990086 (fold 8: -0.00004) (Doc 3).

EXPERT NUANCE

Why a rule and not a learned pruner: the ranker would otherwise have to score about 130 candidates per business, while a rule on already-computed statistics is nearly free and its loss was measured. The thresholds were set in that blocking study with the decision model of the time, so the 0.00003 cost estimate belongs to that model and was not re-measured for the final one. A tiny learned model on the 8 retrieval scores might prune more at equal recall, but it adds a model pass and its own training. The rule embodies the cascade principle: cheap tests first, expensive models last.

REMEMBER

- Keep a candidate if any of six simple conditions holds; no model.
- About 130 → 21.71 candidates per business; the owner survived for 99.974 %; fold-9 F0.5 0.990117 → 0.990086.
- Four of the six conditions exist only to protect recall (address-only, acronyms, same name).

JUDGE MIGHT ASK

- *Why a hand-written rule before the ranker instead of letting the ranker see everything?* "Cost against measured loss: the rule uses scores the index already computed, so it is nearly free, and it cuts about 130 candidates per business to 21.7. On held-out records the true business survived for 99.974 %, and fold-9 F0.5 moved only from 0.990117 to 0.990086."
- *How were the 0.6 and the coverage thresholds chosen?* "They come from our blocking study of 25 September, which measured on held-out folds 8 and 9, with the decision model of that date, how many true businesses survive and how exact F0.5 changes. The special conditions keep what a single score would drop: address-only matches, very short names and identical core names."

A1.23 Recall ceiling and the candidate funnel

IN ONE LINE

The **recall ceiling** is the share of true matches still present among the candidates after each step; no later model can recover what candidate generation lost.

SIMPLE EXPLANATION

Imagine panning for gold with a stack of sieves: each layer removes sand, and after every layer we count how many gold grains are left. If gold is lost at layer two, no clever work at layer five can bring it back, except a separate second search. So we counted the gold after every layer: that is the candidate **funnel** with its recall at each stage.

WORKED EXAMPLE (DOC B.2, TRAIN FOLD 9; MISSED COUNTS DERIVED)

Stage	US recall	India recall	Owned records missed (derived)
Index lookup	99.3694 %	98.5068 %	about 2,888 of 457,971 (US); about 4,536 of 303,786 (India)
Shortlist (quick filter, then top 4), lower to upper bound	98.8748 % to 99.3628 %	98.0572 % to 98.4831 %	about 2,918 to 5,153 (US); about 4,608 to 5,902 (India)

Candidates per incoming record in the all-fold samples: index 27.541 (US) / 23.145 (India) → after the filter 3.111 / 3.878 → shortlist 1.114 / 1.174.

Why recall losses hurt less than precision losses: with perfect precision, a recall of 0.99 still gives $F0.5 = 1.25 \times 0.99 / (0.25 + 0.99) = 0.998$ (pooled; see F-beta).

TECHNICAL EXPLANATION

Recall at stage s = (owned incoming records whose owner is among the stage- s candidates) / (owned incoming records), measured on labelled train fold 9 with the submitted index lookup, quick filter and ranker; the ranker was fitted on folds 0-5, so fold 9 is unseen by it. For the full fold the evidence gives the shortlist recall after the filter as a lower and an upper bound; the fold-9 samples (7,411 US and 7,444 Indian owned records) give exact values: 99.3523 % and 98.3745 % after the filter, 98.8935 % and 97.985 % in the shortlist. The largest loss is the cap of four candidates per record: the top 4 without the filter already gives 98.8814 % (US) and 98.0809 % (India). Second-chance candidates are not included in these numbers; they target the remaining miss types (native-script names, and records whose name differs while the address matches). The final recall is lower than the ceiling, because the decision rule also abstains; the ceiling is an upper bound. For France it cannot be measured, because there are no labels.

HOW WE USED IT

Measuring recall at every candidate stage on held-out labelled data is part of our validation hygiene (Doc 3, Doc B.2): it separates retrieval misses from decision misses, and the two second-chance searches target the retrieval miss types found on train (native-script names the Latin index does not retrieve, and records whose name differs while the address matches) (Doc 3).

EXPERT NUANCE

The ceiling is counted per owned incoming record (singletons excluded); under the per-business metric a missed record costs only its own business's recall. Separating "retrieval misses" (fixed by better blocking or a second search) from "decision misses" (fixed by better scoring or thresholds) is the main diagnostic value: lowering a threshold cannot help a record whose owner is not among its candidates. The shortlist cap trades recall for reader cost; finer blocking keys would make a larger cap affordable where it loses true businesses (QA 18).

REMEMBER

- Index lookup 99.37 % US / 98.51 % India; shortlist at least 98.87 % / 98.06 % (fold 9).
- The cap of four is the largest loss; the quick filter costs very little.
- Second-chance searches exist because a missing owner cannot be fixed by any threshold.

JUDGE MIGHT ASK

- *Where does your pipeline lose the most true matches?* "At the shortlist cap of four: on held-out fold 9 the true business is in the index results for 99.37 % of US records and in the shortlist for at least 98.87 %; for India 98.51 % and at least 98.06 %. The quick filter itself is almost free."
- *Why cap the shortlist at four rather than eight?* "It is a cost-recall trade-off: four protects the expensive readers, which see 1.17 pairs per record on average. We did not report a curve for other caps; to decide properly we would measure shortlist recall and F0.5 against the cap on fold 9, and finer blocking keys would make a larger cap affordable."

A1.24 Other blocking families: sorted neighbourhood, canopy clustering, LSH / MinHash (not used)

IN ONE LINE

These are classic alternative ways to find candidate pairs; we did not use them.

SIMPLE EXPLANATION

- **Sorted neighbourhood:** sort all records alphabetically by a key and compare each record only with its few neighbours in the sorted list, like checking only adjacent names in a phone book.

- **Canopy clustering**: use a cheap distance to throw records into overlapping rough groups ("canopies") and compare only within a group.
- **LSH (locality-sensitive hashing) with MinHash**: give each record several random fingerprints built so that similar records are likely to share at least one; compare only records that share a fingerprint.

WORKED EXAMPLE

Sorted neighbourhood with key = normalised name and a window of 3 (illustrative): in the alphabetically sorted list, a business written "ganesh traders shree" sits among the thousands of names starting with "ganesh", while the record "shree ganesh traders" sits among the names starting with "shree". They are thousands of rows apart, so a window of 3 never compares them: a word-order change or a first-letter typo breaks the method, which is why it is usually run several times with different keys (for example the name with its words sorted, or the postcode).

MinHash LSH with $b = 8$ bands of $r = 4$ rows: the probability that two records with Jaccard similarity J become candidates is $1 - (1 - J^4)^8$ (computed):

Jaccard J	P(candidate)
0.2	0.013
0.5	0.403
0.8	0.985

TECHNICAL EXPLANATION

Sorted neighbourhood (Hernández and Stolfo, 1995) costs $O(n \log n)$ to sort plus $O(n \cdot w)$ comparisons for window w ; it is sensitive to the key and is normally run in several passes with different keys. Canopy clustering (McCallum, Nigam and Ungar, 2000) uses a cheap distance with a loose threshold $T1$ and a tight threshold $T2 < T1$: records within $T1$ of a centre join its canopy, records within $T2$ are removed from the pool of future centres, so canopies overlap. MinHash relies on the fact that for a random permutation π , $P[\min \pi(A) = \min \pi(B)] = J(A, B)$; a signature of k min-hashes is split into b bands of r rows, and records sharing any whole band become candidates, giving the S-shaped curve $1 - (1 - J^r)^b$ with its steepest rise near $(1/b)^{1/r}$ (about 0.59 for $b = 8$, $r = 4$). Weighted MinHash variants can include IDF-like weights. In this vocabulary, our method is token, q-gram, phonetic and exact-key blocking with IDF-weighted scoring and a top-k per record, closely related to **meta-blocking**, which weights candidate pairs by the evidence of the blocks they share and keeps the best pairs per record.

HOW WE USED IT

Not used. Our candidate generation is the inverted index with five channels, the exact-name key, the union of top lists and the quick rule-based filter.

EXPERT NUANCE

The main weakness of these methods at our scale is control: block or bucket sizes are uncontrolled on skewed data (thousands of names share "enterprises"), whereas a ranked top-k per record bounds the output, and plain MinHash weighs every shingle equally, while IDF knows that "legacy" matters more than "enterprises". Their strengths are simplicity and easy distribution (they map naturally onto MapReduce) without an in-memory index. We did not benchmark them, so we do not claim they would be worse; our evidence is our own measured recall.

REMEMBER

- Sorted neighbourhood compares neighbours in a sorted list; it is fragile to word order and first-letter typos.
- MinHash LSH: $P(\text{candidate}) = 1 - (1 - J^r)^b$, a tunable S-curve, with no IDF unless weighted.

- Not used; ours is closest to weighted token and q-gram blocking with a top-k per record (meta-blocking).

JUDGE MIGHT ASK

- *Did you consider LSH or sorted-neighbourhood blocking?* "We did not use them. An inverted index with IDF weights gives each record a scored, bounded top list, which suits skewed data where some words are shared by thousands of businesses, and it keeps the true business for 99.37 % of US and 98.51 % of Indian held-out records. We did not benchmark the alternatives, so we don't claim they would be worse."

A1.25 Dense vector retrieval with approximate nearest neighbours (not used)

IN ONE LINE

Dense retrieval turns every name into a vector with a neural encoder and finds the nearest vectors with an approximate search index; we did not use it.

SIMPLE EXPLANATION

Imagine placing every business on a "meaning map" so that names meaning the same thing land close together, even with no letters in common, for example the same name written in two scripts. Finding candidates then means finding the nearest points on the map. With millions of points, special search structures, a bit like a road network with motorway shortcuts, find near neighbours quickly at the price of occasionally missing one; that is **approximate nearest-neighbour (ANN)** search.

The words you need: an **embedding** is the vector for one text; a **bi-encoder** embeds each side separately, so business vectors can be computed once in advance; **cosine similarity** compares two vectors by their angle; **HNSW** and **IVF-PQ** are two ANN index types; **FAISS** is a widely used ANN library. A **cross-encoder**, like our readers, instead reads both texts together.

WORKED EXAMPLE (ILLUSTRATIVE 3-NUMBER VECTORS)

Name	Vector
Vijay Enterprises	(0.90, 0.10, 0.40)
Vijay Ent.	(0.88, 0.12, 0.41)
Ajay Traders	(0.20, 0.90, 0.10)

$\cos(\text{Vijay Enterprises}, \text{Vijay Ent.}) = 0.9996$; $\cos(\text{Vijay Enterprises}, \text{Ajay Traders}) = 0.338$. Real encoders use hundreds of numbers per vector: 384 for MiniLM and 768 for E5, which matches the head sizes of 385 and 769 parameters in the Documentation (one weight per number plus a bias). Back-of-envelope memory (derived): $809,986 \text{ Indian businesses} \times 384 \text{ numbers} \times 4 \text{ bytes} \approx 1.24 \text{ GB}$ in 32-bit floats.

TECHNICAL EXPLANATION

A bi-encoder f maps each text independently into a d -dimensional space and scores a pair by cosine or dot product; business vectors are precomputed, so retrieval reduces to nearest-neighbour search. Exact search costs $O(N \cdot d)$ per query; ANN indexes trade a little recall for speed. HNSW builds a hierarchical navigable small-world graph (parameters M , efConstruction , efSearch) with roughly logarithmic search; IVF clusters vectors with k -means (which groups points around a chosen number of centres) and searches only the few clusters nearest the query (n_{probe}), and product quantisation compresses vectors into short codes. Bi-encoders are trained contrastively on labelled pairs (pulling the vectors of matching pairs together and pushing non-matching ones apart), using the other pairs of the same batch and deliberately chosen similar-looking non-matches as negatives. A cross-encoder attends across both texts jointly, which is more accurate, but nothing can be precomputed, so it suits re-ranking a shortlist, not retrieval over

millions. The usual production answer is hybrid retrieval: a lexical index and a dense index, with the union of their candidate lists.

HOW WE USED IT

Not used, and not benchmarked (QA 1). Retrieval is the inverted index; the readers are cross-encoders that see only the shortlist; a frozen pretrained MiniLM is used only to embed single words for the meaning-similarity "what changed" features.

EXPERT NUANCE

Dense retrieval is strongest on paraphrases, abbreviations and cross-script names, and weakest on exact digits, house numbers and rare proper nouns, where lexical matching wins; ANN results are approximate and harder to audit, and a good encoder needs fine-tuning. The natural place for it in our pipeline is one more channel in the union of top lists, aimed at brand or "doing business as" names with little word overlap, one of our documented miss types, rather than a replacement for the index. It would be allowed by the rules as long as the encoder is Apache- or MIT-licensed and at most 8B parameters.

REMEMBER

- Bi-encoder plus ANN = precomputed vectors and approximate nearest-neighbour search; cross-encoder = joint reading, shortlist only.
- Not used and not benchmarked, so no claim that it would be worse.
- Best use for us: an extra channel for brand or "doing business as" names, added to the union of lists.

JUDGE MIGHT ASK

- *Why not embedding (vector) search for blocking?* "We did not benchmark a dense retriever, so we can't claim it would be worse. Our index already keeps the true business for 99.37 % of US and 98.51 % of Indian held-out records, deterministically. Dense search would be the natural extra channel for brand or 'doing business as' names that share few words, added to our union of lists rather than replacing it."
- *Would a dense retriever help the Indian-script records?* "Plausibly, because a multilingual encoder can place विजय and Vijay close together. Today we handle that with the learned word list and the native-script second-chance search, which added 9,759 links; we have not measured a dense alternative, so it is a hypothesis we would test on labelled Indian folds."

A1.26 Scaling: sharding, bounded work per record and MapReduce-style blocking

IN ONE LINE

The pipeline scales by splitting the data into independent pieces (today one per country) and by capping the work per record at every stage, so total cost grows linearly with the number of records.

SIMPLE EXPLANATION

A grocery chain does not run one giant warehouse for the whole country: each region has its own warehouse and catalogue, every order goes only to its own region, and every order has a maximum number of items. Adding customers then adds work in proportion, not explosively. Our regions are countries: each has its own index, and an incoming record is only ever searched in its own country's index. Each such piece is a **shard**. Work that splits into independent pieces with no communication is called **embarrassingly parallel**. **MapReduce** is a classic framework for such jobs: a map step emits key-value pairs, a shuffle groups them by key, and a reduce step processes each group.

WORKED EXAMPLE

Bounded work per incoming record (from the Documentation's caps; the index bound is derived):

Stage	Bound per incoming record
Index lookup	at most 65,000 postings plus 2,000 exact-name matches; keep top 12 + 6 + 6 + the exact key
Quick rule-based filter	linear in those candidates, no model
Readers	at most 4 pairs (1.17 on average)
Decision model	a handful of small tree-model calls per shortlisted pair

A rough linear extrapolation (QA 2; not a measurement): scoring our 11.69 million shortlist pairs took about 1.4 hours on one 16 GB GPU (every reader except the French E5 reader), so a billion incoming records, about 1.17 billion pairs, would be on the order of 140 GPU-hours, spread across shards.

MapReduce-style blocking (illustrative): map each record to (key, record) for each of its keys ("ganesh", "traders", "pune"); shuffle groups records by key; reduce compares pairs inside each group. If the key "traders" has 50,000 members, one reducer receives an enormous group (**data skew**), so such keys must be capped or dropped, which is the same job our posting-list caps do.

TECHNICAL EXPLANATION

Today all processing is per country, and each country's index fits in memory: every stage needs less than 8 GB of RAM except the address rescue (about 3.2 GB per worker, 13 GB with 4 workers), and the index lookup runs on 8 to 11 threads (code README). Our pattern is "broadcast the index, partition the queries": any number of workers can hold a copy of a country's index and process disjoint slices of incoming records with no shuffle, like a broadcast join (every worker receives a full copy of the smaller table) or a map-only job. Classic key-based MapReduce blocking needs load balancing for skewed keys (splitting or capping large blocks); our caps do it at query time. Finer sharding by country plus state or postcode would shorten posting lists and make a larger shortlist cap affordable, but records with a missing or wrong state then need a fallback search across the whole country, as the native-script rescue already does when the state is unknown or has no match. IDF must be computed consistently across shards (one global value per country, or per shard by design). For streaming, one record would need one index lookup in its shard, on average 1.17 reader pairs and a few small tree models; single-record latency was not measured.

HOW WE USED IT

Everything ran on one machine: one RTX 4070 Ti SUPER 16 GB, a Ryzen 7 9800X3D, 32 GB RAM and about 70 GB of disk. A full retrain from raw data took about 11 hours 30 minutes; at inference, index lookup plus quick filter plus ranker took about 1.3 hours and reader scoring about 1.4 hours for the 9.97 million incoming records (code README and clean-run report). The index is split by country only; state or postcode sharding is proposed for scale, not tested (QA 2, QA 18).

EXPERT NUANCE

Linear scaling assumes bounded work per record and stable data; candidate counts per business are skewed (maximum 3,182), so shards can take uneven time, which dynamic scheduling of record batches absorbs within one machine. New businesses can be appended to posting lists incrementally, while IDF drifts slowly and can be refreshed periodically. Always present the 140 GPU-hours as an extrapolation, not a measurement, and say that single-record latency was not measured.

REMEMBER

- Shard by country today (state or postcode proposed); broadcast the index, partition the records.
- Every stage has a per-record cap, so cost is linear in the number of records.
- The billion-record estimate (about 140 GPU-hours for the readers) is a rough linear extrapolation.

JUDGE MIGHT ASK

- *How would this scale to billions of records?* "The work per record is bounded at every stage: posting caps and budgets in the index, a fixed number of top candidates, and at most four pairs for the

readers, so cost grows linearly and shards run independently. As a rough linear extrapolation, not a measurement, scoring our 11.69 million shortlist pairs took about 1.4 hours on one 16 GB GPU. A billion records would therefore be on the order of 140 GPU-hours, spread across shards."

- *What would you shard on, and what could break?* "Country plus state or postcode: shorter posting lists and room for a larger shortlist cap. What breaks is records with a missing or wrong state, so we would keep a fallback search across the whole country, as the native-script second-chance search already does when the state is unknown."

Numbers used in this part

Topic	Number	Source
Test businesses / incoming records	1,732,544 / 9,969,589 (US 663,106 / 3,817,031; India 809,986 / 4,717,565; France 259,452 / 1,434,993)	Doc 2.1
Same-country comparison space	6,724,569,566,212 pairs	Doc 3
Candidate file; reduction ratio	12,293,019 pairs (7.0954 per business); 0.99999817 (1.83 per million, derived)	Doc 3
Funnel per business	about 130 (index lookup, blocking study 25 Sep) → 21.7107 (quick filter) → 6.75 (shortlist, derived) → 7.0954 (candidate file)	Doc 3
Per incoming record	1.2331 candidates; 90.59 % exactly one; 1.17 shortlist pairs (derived)	Doc B.1
Recall, fold 9	index 99.3694 % US / 98.5068 % India; shortlist 98.8748-99.3628 % / 98.0572-98.4831 %	Doc B.2
Quick-filter cost	owner kept 99.974 %; fold-9 F0.5 0.990117 → 0.990086; fold 8 -0.00004	Doc 3
Index channels	weights 1 / 1 / 0.45 / 0.22 / 0.18; caps / budgets 14,000 / 22,000, 10,000 / 18,000, 2,500 / 15,000, 1,000 / 6,000, 800 / 4,000	Doc 3
Index scoring	IDF = $\log(1 + (N + 1) / (df + 1))$; exponents 0.22 / 0.38 / 0.30; total = ns + as + $1.2 \cdot \min(ns, as)$; keep top 12 / 6 / 6 + exact key ($\leq 2,000$)	Doc 3
Metric	$F_b = 1.25 \cdot TP_b / (pred_b + 0.25 \cdot true_b)$, 1 if both 0; macro over businesses	Doc 2.1
Decision rule	0.75 (US, India; fold 8), 0.82 (France, default), lead 0.4	Doc 4
Second-chance thresholds	native script 0.88 / lead 0.3; address US 0.86 / 0.0, India 0.66 / 0.1	Doc 4
Output	5,856,936 links; 100,087 empty rows (5.8 %, derived); 0.5875 links per incoming record	Doc B.1
Reader AUC sanity check	0.998544 (French E5) vs 0.998570 (E5); best F0.5 0.992782 vs 0.992796	Doc B.4
Compute	index lookup + filter + ranker about 1.3 h; reader scoring about 1.4 h; full retrain about 11.5 h	code README; clean-run report
Billion-record estimate	about 140 GPU-hours for the readers (rough linear extrapolation)	QA 2

A2 - Concepts: the models

How to read this part

This part teaches every model in our pipeline from zero, up to the depth a senior applied scientist expects. Read the concepts in order: each one uses words defined in the earlier ones, and every new term is in **bold** where it is first explained. Every concept has the same eight blocks: a one-line summary, a simple explanation with an everyday picture, a worked example you can follow with a pen, the technical explanation, exactly how we used it, expert nuances (trade-offs, traps, alternatives), three things to remember, and the questions a judge might ask with short spoken answers.

Three labels tell you where a number comes from. Numbers about our run or our recipe are exact and come from the submitted Documentation ("Doc"), the code READMEs or the shared fact sheet; **derived** means simple arithmetic on those numbers. **Toy** marks made-up numbers that exist only to teach the arithmetic; they are never results. **Background** marks general knowledge about public models and libraries (for example how many attention heads MiniLM has), checked against the shipped model files where we could; it is not a claim about our results, so you do not need to volunteer it. Spoken answers use the plain names of the talk: business, incoming record, shortlist, quick rule-based filter, CatBoost ranker, the four readers, "what changed" features, LightGBM decision model per country, confident and clear winner, second-chance searches, per-business consistency check, France rules computed by code.

Words this part borrows from Part A0 (one line each, so you can start here):

- **Precision**: of the links we made, the share that are right. **Recall**: of the true links, the share we found.
- **F0.5**: one score that combines precision and recall but counts precision more (a wrong link hurts more than a missed one). For one business, $F0.5 = 1.25 \times \text{correct links} / (\text{links made} + 0.25 \times \text{true links})$. The competition computes it for every business and averages over all businesses (**macro-average**); a business that truly owns no records scores 1 only if we link nothing to it.
- **Fold**: one of 10 roughly equal parts into which we split the labelled businesses (by a hash of the business id); a model is trained on some folds and checked on another, **held-out** fold it never saw.
- **Epoch**: one full pass over the training data. **Random seed**: the starting number of a random-number generator; a different seed gives different random choices.
- **GPU**: a graphics processor, a chip that does thousands of multiplications in parallel, which makes neural networks (and CatBoost) fast to train.
- **IDF** (inverse document frequency): a rarity score for a word, high for a rare word (a surname) and low for a common one ("services"); for example $\log((N + 1) / (df + 1))$, where N is the number of records and df the number that contain the word.
- **AUC** (area under the ROC curve): the probability that a randomly chosen true pair gets a higher score than a randomly chosen wrong pair; 1.0 means perfect ordering, 0.5 means random.

Concepts in this part (learn them in this order)

1. Models, features, labels and loss
2. Logits, the sigmoid and log-loss
3. Overfitting, held-out data and early stopping
4. Decision trees
5. Gradient boosting, step by step
6. The knobs of boosted trees
7. CatBoost: symmetric trees, ordered boosting, categorical encoding
8. LightGBM: histograms and leaf-wise growth

9. Learning to rank: pointwise, pairwise, listwise
10. Text into numbers: tokens, sub-words and embeddings
11. Transformers and attention
12. Pre-training, transfer learning and fine-tuning
13. Bi-encoders versus cross-encoders
14. Inside our reader: pair input, mean pooling and the head
15. MiniLM and knowledge distillation
16. E5 and contrastive pre-training
17. The fine-tuning recipe
18. Number formats: fp32, fp16, bf16 and mixed precision
19. Hard-example selection for reader training
20. Frozen pretrained embeddings as features
21. Self-training with pseudo-labels and replay
22. Blending readers in logit space (France)
23. Stacking: a two-level model with separate folds per layer
24. Early stopping, then refit on more data
25. Seed averaging
26. Calibration: what a model's "probability" means
27. Small models for the second-chance searches

Where each model sits in our pipeline (orientation map; every term in it is explained in the concepts below)

Step of the pipeline	Plain name	Kind of model	Library	Reads	Outputs	Learns from
Ranking the candidates that survive the quick rule-based filter	CatBoost ranker	gradient-boosted trees, binary classifier	CatBoost, on the GPU	76 numeric features computed by the index lookup for one pair	probability that this business is the record's owner	a 140,000-record pilot of labelled US/India train records, fitted on its fold 0-5 part, early stopping on fold 6 (Doc 4)
Reading the shortlist (up to 4 pairs per record)	MiniLM reader, E5 reader	cross-encoders (transformers)	PyTorch + Hugging Face	the raw "name address" of the record and of the business, together	one logit (match score)	labelled US/India pairs of folds 0-5 (Doc 4)
Reading French shortlist pairs	French MiniLM reader, French E5 reader	the same readers, fine-tuned again	same	same	one logit	confident French pseudo-labels plus labelled US/India pairs (Doc 4)
Word-meaning part of the "what changed" features	frozen MiniLM	pretrained encoder, not fine-tuned	sentence-transformers	single words	a vector of 384 numbers per word	not trained by us
Deciding	LightGBM decision model, one per country	gradient-boosted trees, mean of 5 seeds	LightGBM	158 to 217 features (217 US, 211 India, 158 France and default), including ranker probability, reader logits and "what changed"	probability; then the confident + clear-winner rule	folds 6-9 (Doc 4)
Second chances	native-script rescue model; address-rescue filter and scorer	gradient-boosted trees	LightGBM	features of the new candidates	probability	India folds 0-7 (native script); folds 6-7 (address) (Doc 4)

A2.1 Models, features, labels and loss

IN ONE LINE

A model is a formula with adjustable numbers that turns a description of a record pair into a score, and training adjusts those numbers until the scores agree with known answers.

SIMPLE EXPLANATION

Think of a new bank clerk learning to judge whether two signatures come from the same person. Each case she studies comes with clues she can check (same initials? same slant?) and, in the archive, the true answer. At first she guesses badly; each time she is wrong she adjusts how much weight she gives each clue, and slowly her guesses improve. In machine learning the clues are called **features**, the true answers are **labels**, the clerk's habits are the model's **parameters**, the penalty for a wrong guess is the **loss**, and the process of adjusting is **training**. Learning from examples that come with answers is called **supervised learning**. Using the finished model on new cases, with its habits frozen, is called **inference**.

WORKED EXAMPLE (TOY NUMBERS)

Three training pairs, each an incoming record and a candidate business, described by two features.

Pair	Incoming record	Candidate business	Name similarity (0-1)	Same legal form? (1/0)	Label: is this business the owner?
1	Sharma Traders Pvt Ltd, Pune	Sharma Traders Private Limited, Pune	0.82	1	1
2	Sharma Traders Pvt Ltd, Pune	Sharma Textiles Pvt Ltd, Nagpur	0.71	1	0
3	Boulangerie Martin SARL, Lyon	Boulangerie Martin SAS, Lyon	0.90	0	0

A very simple model is: $\text{score} = 3 \times \text{similarity} + 2 \times \text{same_legal} - 3$. Pair 1 gets $3 \times 0.82 + 2 - 3 = 1.46$; pair 2 gets 1.13; pair 3 gets -0.30. The order is right for pair 3 but pair 2 (a different business) scores almost as high as pair 1. Training would change the three numbers 3, 2 and -3 (the parameters) to push pair 1 up and pair 2 down. The numbers we choose ourselves, such as how big each adjustment step is, are **hyper-parameters**.

TECHNICAL EXPLANATION

All our learned stages solve **binary classification** on pairs: the input x is a (record, candidate business) pair, the label y is 1 if the business is the record's **owner** (the business it truly belongs to) and 0 otherwise. Training is **empirical risk minimisation**: choose parameters θ to minimise $L(\theta) = (1/N) \times \sum_i \text{loss}(y_i, f_{\theta}(x_i)) + R(\theta)$, where R is a **regulariser** that discourages overly complex models. Neural networks minimise this by **gradient descent**, $\theta \leftarrow \theta - \eta \times \text{gradient of } L$, where η is the **learning rate**; boosted trees minimise it by adding trees (see Gradient boosting). The quantity that matters is not the training loss but **generalisation**: performance on pairs the model never saw. Standard theory assumes training examples are independent; ours are not, because one business owns several records and several candidates share a record. That is why our folds are built per business (see Overfitting, held-out data and early stopping). The pair label also has structure: a record has at most one owner, so at most one candidate in its list has label 1, and some records have none. Finally, the pair-level loss is only a proxy: the competition scores F0.5 per business (defined in the word list at the top of this part), so the decision thresholds are tuned separately on that exact metric (see Calibration).

HOW WE USED IT

Every learned model in the pipeline answers the same yes/no question, "is this business the owner of this incoming record?" (Doc 4). The CatBoost ranker answers it from 76 index features, the readers from the raw text, the LightGBM decision models from 158 to 217 features (217 US, 211 India, 158 France and default), and the second-chance models from features of their own candidates. The ranker, the decision

models and the second-chance models were trained from scratch on the provided data; only the readers start from pretrained language models, and a pretrained MiniLM is also used, unchanged, to compare single words (Doc 4, B.5). Labels exist only for the US and India; France has none (Doc 2.1).

EXPERT NUANCE

The unit of the metric (the business) differs from the unit of the loss (the pair), and that gap shapes the whole design: a model can improve its average pair loss and still lose F0.5, for example if the change that lowers the loss on many easy pairs also lets one extra wrong record be linked to a business that should have an empty row. The shape of the candidate lists matters too: 90.59 % of incoming records have exactly one candidate in the final candidate file (Doc B.1), so much of the difficulty is deciding "link or abstain" rather than "which one". A common misconception is that a better classifier automatically gives a better match file; the decision rule and its thresholds matter as much as the model.

REMEMBER

- One question everywhere: is this business the owner of this record? Label 1 or 0.
- Parameters are learned; hyper-parameters are chosen by us; generalisation is judged on unseen businesses.
- Pair-level loss is a proxy; the competition metric is per business, so thresholds are tuned on it.

JUDGE MIGHT ASK

- *What exactly is the label your models learn?* For every model it is the same yes/no question about a pair: is this business the true owner of this incoming record. Labels exist only in the US and Indian training files, so the France decision model learns from US and Indian rows, and the French readers also learn from confident pseudo-labels, that is, very confident predictions of our own models on French records.

A2.2 Logits, the sigmoid and log-loss

IN ONE LINE

A logit is a model's raw score on an unlimited scale, the sigmoid squashes it into a probability between 0 and 1, and log-loss measures how surprised the model was by the true answer.

SIMPLE EXPLANATION

Imagine a weather forecaster. She can express confidence as a probability ("90 % rain") or as betting odds ("9 to 1"). A **logit** is the logarithm of those odds: 0 means 50/50, +2.2 means about 90 %, +4.6 means about 99 %, and every step of +2.3 multiplies the odds by ten. The **sigmoid** function converts a logit back into a probability. If she says "99 % rain" and it stays dry, she deserves a much bigger penalty than if she had said "60 %". **Log-loss** (also called **binary cross-entropy**, BCE) is exactly that penalty: small when a confident forecast is right, very large when a confident forecast is wrong.

WORKED EXAMPLE (REAL LOGITS FROM DOC D.3, THE FRENCH RECORD "TQ COMITE SARL")

Reader, candidate	Logit z	Probability sigmoid(z)
MiniLM reader, TQ Parents SARL (same address)	7.5000	0.99945
French MiniLM reader, TQ Parents SARL	8.9375	0.99987
French E5 reader, TQ Parents SARL	10.8125	0.99998
French MiniLM reader, TQ Comite SARL (Pessac, another city)	-1.9219	0.128
French E5 reader, TQ Comite SARL (Pessac)	-10.8125	0.00002

Steps: $\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$. For $z = 7.5$, $e^{(-7.5)} = 0.000553$, so the probability is $1 / 1.000553 = 0.99945$. Notice that 0.99945 and 0.99998 look almost identical as probabilities, yet as logits they are 7.5 and 10.8, a difference of 3.3, which means the odds differ by a factor $e^{3.3125} = 27.5$ (derived). Log-loss for a pair with label 1 is $-\ln(p)$: if a reader gave logit 7.5, the loss is $-\ln(0.99945) = 0.00055$; if it gave -1.9219, the loss is $-\ln(0.128) = 2.06$. (Test labels are private, so these labels are hypothetical; the arithmetic is the point.) The US decision-model score 0.965022 in Doc D.1 corresponds to a logit of $\ln(0.965022 / 0.034978) = 3.32$ (derived).

TECHNICAL EXPLANATION

$\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$ and its inverse $\text{logit}(p) = \ln(p / (1 - p))$. Binary cross-entropy for one example is $\text{BCE} = -[y \ln \text{sigmoid}(z) + (1 - y) \ln(1 - \text{sigmoid}(z))]$. Its derivative with respect to the logit is simply $\text{sigmoid}(z) - y$, the predicted probability minus the label; this clean gradient is what both gradient boosting and neural-network training use. Log-loss is a **proper scoring rule**: its expected value is minimised by predicting the true probability, which is why a log-loss-trained model is roughly calibrated on its own training distribution (see Calibration). Numerically, computing sigmoid and then log separately loses precision for large $|z|$; PyTorch's `BCEWithLogitsLoss` fuses them with the log-sum-exp trick. A boosted-tree model's raw output (the sum of its tree outputs) is also a logit, and LightGBM's binary objective returns sigmoid of that sum. Because logits add, averaging logits corresponds to a weighted geometric mean of odds (see Blending readers in logit space).

HOW WE USED IT

Each reader outputs one logit per shortlist pair (for example 7.5000 above) and was trained with BCE on the logit cast to fp32 (Doc 4; code). The CatBoost ranker is trained with Logloss and outputs a probability; in the code the decision model receives it converted to log-odds, together with its rank (background, code detail). The LightGBM decision models are trained on binary log-loss and output the probability to which the thresholds 0.75 (US, India) and 0.82 (France, default) apply (Doc 4). For France, the decision model's single reader input receives a weighted blend of three reader logits (Doc 4).

EXPERT NUANCE

Logits from different models are not on a common, calibrated scale, so "logit 7.5" from one reader and from another do not mean the same confidence; the decision model learns how to read each. Trees split on the order of values, so they are (up to binning details) invariant to a monotone transformation of a single input: giving a tree the ranker's probability or its logit leads to essentially the same splits. But derived inputs such as the mean of two readers or a margin between candidates do change with the scale, which is one reason we compute them on logits. Saturation is real: in floating point, probabilities very close to 1 lose resolution, while logits keep it.

REMEMBER

- Logit = log of the odds; sigmoid turns it into a probability; 0 means 50/50.
- Log-loss punishes confident mistakes heavily; its gradient is (probability - label).
- Readers produce logits; decision models produce probabilities that the thresholds apply to.

JUDGE MIGHT ASK

- *Why do you keep reader logits instead of probabilities?* Because probabilities saturate: 0.99945 and 0.99998 look the same, but as logits they are 7.5 and 10.8, a 27-fold difference in odds. Keeping logits gives the decision model that information and lets us blend the French readers on a scale where a confident "no" really counts.

A2.3 Overfitting, held-out data and early stopping

See also [A3.9 Early stopping](#): early stopping from the training-and-validation side.

IN ONE LINE

A model overfits when it memorises its training examples instead of learning patterns that carry over, so we always judge it on data it never saw and stop training when that judgement stops improving.

SIMPLE EXPLANATION

A student who memorises last year's answer key scores perfectly on last year's paper and badly on this year's. That is **overfitting**. The cure is a mock exam with questions she has never seen: a **held-out** (or **validation**) set. In our project the labelled businesses are split into 10 roughly equal parts called **folds**, so "a held-out fold" simply means about one tenth of the businesses, which the model did not train on. **Early stopping** means you keep revising only while the mock-exam score keeps improving, and stop once it has not improved for a while. The opposite problem, a model too simple to learn the pattern at all, is **underfitting**.

WORKED EXAMPLE (TOY NUMBERS)

A boosted-tree model is trained round by round; after each round we measure log-loss on the training data and on a held-out fold.

Rounds (trees)	Training log-loss	Held-out log-loss
100	0.050	0.060
200	0.040	0.052
300	0.034	0.050 (best)
400	0.030	0.051

Training loss keeps falling, but held-out loss is best at 300 trees and then rises: the extra trees are memorising. With a **patience** of 100 rounds (no improvement for 100 rounds means stop), training stops at round 400 and the model keeps its best 300 trees.

TECHNICAL EXPLANATION

The **generalisation gap** is the difference between training and held-out performance; it grows with model capacity and shrinks with more data and regularisation (the **bias-variance trade-off**). A held-out set is only honest if nothing about it leaked into training: no shared records, no shared businesses, no tuning on it. Because records of one business are strongly related, a record-level random split leaks; we assign the 10 folds per business with a hash of the business id, so all of a business's records sit in one fold (Doc 4). Early stopping is itself a regulariser: it chooses the number of boosting rounds (or training steps) on the validation fold, which makes that fold slightly optimistic afterwards, so final scores must be reported elsewhere. It optimises the pair-level log-loss, not F0.5, which is why decision thresholds are tuned separately on the exact metric.

HOW WE USED IT

The CatBoost ranker stops early with patience 180 on fold 6 (at most 1,600 trees). The LightGBM decision models stop early with patience 100 rounds on fold 8 (at most 3,000 rounds for the US and India, 1,500 for France and the default) (Doc 4; code). The readers do not use early stopping: they train for exactly one epoch (one pass over their training pairs). Our held-out evidence uses a strict protocol: fold 9 is scored by fits on folds 6-8 without the records that touch fold 9, and fold 8 by fits on folds 6, 7 and 9 without the records that touch fold 8; close versions of our decision models score US 0.991212 / 0.991545 and India 0.990046 / 0.990298 (fold 9 / fold 8) (Doc 5).

EXPERT NUANCE

Our biggest validation limitation, stated plainly: there is no untouched end-to-end validation of the final file. The held-out scores exclude the second-chance searches and the France rules, the final US/India decision models are refitted on folds 6-9, folds 8 and 9 also helped choose thresholds and the consistency check, and France has no labels (Doc 5). The mitigation is that every held-out number comes from fits that never saw the scored fold, and they sit in the same range as the public 0.9913. Overfitting can also happen to a leaderboard: we made 18 uploads and chose the final file using public scores (Doc C.4), which is a mild form of the same problem.

REMEMBER

- Judge a model only on businesses it never trained on; folds are per business.
- Early stopping picks the number of trees on a separate fold (ranker fold 6, decision models fold 8).
- Limitation: no untouched end-to-end validation; held-out scores are in the same range as public.

JUDGE MIGHT ASK

- *How do you know your models are not overfitting?* The honest limitation first: we have no fully untouched end-to-end check, because the held-out folds also helped choose the thresholds and the final file includes steps they do not cover. What we do have comes from businesses the scoring fits never trained on: close versions of our decision models score 0.9912 and 0.9915 for the US and 0.9900 and 0.9903 for India on held-out folds, the same range as the public 0.9913.

A2.4 Decision trees

IN ONE LINE

A decision tree is a flowchart of yes/no questions about the features that ends in a leaf holding a score.

SIMPLE EXPLANATION

Think of the game "twenty questions" or a hospital triage chart: "Is the name similarity above 0.8? If yes, does the address overlap a lot?" Each question splits the examples into two groups, and each final group, a **leaf**, stores the average outcome of the training examples that ended there. The tree learns which questions to ask by trying every possible question and keeping the one that best separates matches from non-matches. One question is a **split**; the number of questions on the longest path is the tree's **depth**.

WORKED EXAMPLE (TOY NUMBERS)

Eight training pairs with two features.

Pair	Name similarity	Address coverage	Owner?
1	0.95	1.00	1
2	0.92	0.40	1
3	0.88	0.97	1
4	0.85	0.10	0
5	0.60	0.98	0
6	0.55	0.20	0
7	0.91	0.00	0
8	0.40	0.05	0

We measure "mixedness" with **Gini impurity**, $G = 2p(1 - p)$, where p is the share of owners in a group (0 = pure). At the root $p = 3/8$, so $G = 2 \times 0.375 \times 0.625 = 0.469$.

- Split A, "name similarity ≥ 0.8 ": yes = pairs 1, 2, 3, 4, 7 (3 owners of 5, $G = 0.48$); no = 5, 6, 8 ($G = 0$). Weighted impurity = $5/8 \times 0.48 = 0.300$. Improvement 0.169.
- Split B, "address coverage ≥ 0.3 ": yes = 1, 2, 3, 5 (3 of 4, $G = 0.375$); no = 4, 6, 7, 8 ($G = 0$). Weighted = $4/8 \times 0.375 = 0.1875$. Improvement 0.281.

Split B wins, although a human might have asked about the name first. Inside its "yes" branch, "name similarity ≥ 0.8 " then separates pairs 1, 2, 3 (all owners) from pair 5. Two questions, three pure leaves.

TECHNICAL EXPLANATION

Classic trees (CART, "classification and regression trees") are grown greedily: at each node, for every feature and every candidate threshold, compute the impurity decrease (or, in boosting, the gradient-based gain, see Gradient boosting) and take the best split; stop at a maximum depth, a maximum number of leaves or a minimum number of examples per leaf. Searching thresholds naively costs $O(n)$ per feature per node after sorting; histogram methods bin each feature first and search only bin boundaries (see LightGBM). Trees handle features on any scale, need no normalisation, capture interactions (a split inside a split), and are invariant to monotone transformations of a feature. Libraries also handle **missing values** without imputation: LightGBM (and XGBoost) learn a default direction at each split, sending rows with a missing feature to whichever side lowers the loss, while CatBoost by default treats a missing value as smaller than every observed value (its "Min" mode; background, our shipped ranker uses it). A single deep tree has high **variance** (small data changes reshape it), which is why trees are almost always used in ensembles: **bagging** (random forests) averages independent trees, **boosting** adds trees that correct each other.

HOW WE USED IT

We never use a single tree. All our tree models are ensembles: the CatBoost ranker uses up to 1,600 symmetric trees of depth 7; each LightGBM decision model uses trees of up to 127 leaves, up to 3,000 rounds for the US and India; the address-rescue filter uses 200 trees of 15 leaves and its scorer 300 trees of 31 leaves (Doc 4). Missing values matter in our data: in the US worked example the record has no address, so the "what changed" number features are missing for every candidate (Doc D.1), and the LightGBM decision models route those rows by their learned default direction.

EXPERT NUANCE

Trees split on one feature at a time (axis-aligned), so a relationship like "the difference of two features" must be approximated by many splits unless we compute it as a feature, which is why we engineer margins and differences explicitly. Trees cannot extrapolate: beyond the largest value seen in training, the prediction stays constant, so a French reader logit more extreme than anything in the training rows is simply treated like the most extreme value the model saw (see Blending readers in logit space). Greedy growth is not globally optimal, but in an ensemble that matters little.

REMEMBER

- A tree = nested yes/no questions; each leaf holds a score.
- Splits are chosen by how much they purify the groups (Gini, or gradient gain in boosting).
- Single trees are unstable; we only use boosted ensembles of hundreds to thousands of trees.

JUDGE MIGHT ASK

- *Can you explain why the model made one particular decision?* Each decision model is an average of five boosted ensembles, so there is no single readable tree, but every input has a plain meaning, such as "legal form changed" or "one word replaced". In the US example the LLC and PC candidates differ from the record's name only in the legal form, and the decision model gives them 0.000007 and 0.088664 against 0.965022 for the exact name; we did not prepare per-feature attribution plots, so we would not claim exactly how much each input contributed.

A2.5 Gradient boosting, step by step

IN ONE LINE

Gradient boosting builds a team of small trees one after another, where each new tree is trained to correct the mistakes the team still makes.

SIMPLE EXPLANATION

Think of putting in golf. The first putt gets the ball near the hole; each following putt only has to cover the remaining distance; and you deliberately putt a little short each time so you never overshoot wildly. In gradient boosting the first "putt" is a constant guess, each new tree is fitted to the remaining error (the **residual**), and its contribution is shrunk by the **learning rate** so that many small corrections add up. To score a pair, you add the outputs of all trees and pass the total through the sigmoid to get a probability.

WORKED EXAMPLE (TOY NUMBERS, LOG-LOSS, ONE FEATURE)

Four training pairs.

Pair	Name similarity s	Owner y
A	0.95	1
B	0.90	1
C	0.60	0
D	0.40	0

1. Start: $F_0 = \log\text{-odds of the base rate} = \ln(0.5 / 0.5) = 0$, so every pair gets $p = 0.5$.
2. Errors: the **gradient** of log-loss for each pair is $g = p - y$: A and B get -0.5 , C and D get $+0.5$. The **hessian** (curvature) is $h = p(1 - p) = 0.25$ for each.
3. Choose the split. For " $s \geq 0.75$ ": left = {A, B} with $G_L = -1.0$, $H_L = 0.5$; right = {C, D} with $G_R = +1.0$, $H_R = 0.5$. Gain = $1/2 \times [G_L^2/H_L + G_R^2/H_R - (G_L+G_R)^2/(H_L+H_R)] = 1/2 \times [2 + 2 - 0] = 2.0$. For " $s \geq 0.5$ ": {A, B, C} versus {D} gives $1/2 \times [0.25/0.75 + 0.25/0.25] = 0.667$. The first split wins.
4. Leaf values (a **Newton step**): $w = -G / H$. Left leaf = $-(-1.0)/0.5 = +2.0$; right leaf = -2.0 .
5. Shrink by the learning rate 0.1: $F_1 = +0.2$ for A and B, -0.2 for C and D, so $p = \text{sigmoid}(0.2) = 0.550$ and 0.450 .
6. Tree 2 repeats on the new errors: $g = 0.550 - 1 = -0.450$ for A and B, $h = 0.550 \times 0.450 = 0.2475$; left leaf = $0.900 / 0.495 = 1.819$, shrunk to 0.182 ; now $F_2 = 0.382$ and $p = 0.594$. The predictions creep towards the labels.
7. With an L2 penalty $\lambda = 1$ in step 4, the left leaf would be $-G/(H + \lambda) = 1.0/1.5 = 0.667$ instead of 2.0 : regularisation makes each step more cautious.

Scoring a new pair is just addition: if three trees give $+1.2$, $+0.8$ and -0.3 , the total is 1.7 and the probability is $\text{sigmoid}(1.7) = 0.846$.

TECHNICAL EXPLANATION

Gradient boosting (Friedman, 2001) performs gradient descent in function space: $F_m(x) = F_{(m-1)}(x) + \eta * f_m(x)$, where f_m is a tree fitted to the negative gradient of the loss at the current predictions (the **pseudo-residuals**). For log-loss the negative gradient is $y - p$, literally the residual in probability space. Modern libraries (XGBoost, LightGBM, CatBoost) use a second-order Taylor expansion: they minimise $\sum_i [g_i f(x_i) + 1/2 h_i f(x_i)^2] + \Omega(f)$, with $\Omega(f) = \gamma * T + 1/2 * \lambda * \sum \text{squared leaf values}$, which gives the optimal leaf value $w^* = -G/(H + \lambda)$ and the split gain $1/2 [G_L^2/(H_L + \lambda) + G_R^2/(H_R + \lambda) - (G_L + G_R)^2/(H_L + H_R + \lambda)] - \gamma$. The tree structure is found greedily with that gain. Training cost per tree is roughly proportional to rows $\times$ features (to accumulate gradient statistics), and prediction cost to the number of trees times the depth. The main

hyper-parameters are the learning rate η , the tree size (depth or number of leaves), the number of trees (chosen by early stopping), the L2 penalty, the minimum data per leaf, and row and feature subsampling (see The knobs of boosted trees).

HOW WE USED IT

All our tree models are gradient-boosted binary classifiers on log-loss. The CatBoost ranker: up to 1,600 trees, depth 7, learning rate 0.075, L2 5 (Doc 4). The LightGBM decision models: learning rate 0.05, up to 127 leaves, L2 1.0, up to 3,000 rounds (US, India) or 1,500 (France, default) (Doc 4). The second-chance models are smaller LightGBM ensembles (see Small models for the second-chance searches).

EXPERT NUANCE

"Each tree fits the residuals" is exactly true only for squared loss; for log-loss the tree's structure is chosen on gradients and its leaf values are Newton steps, which is why the hessian appears. A smaller learning rate needs more trees but usually generalises better; the product of the two is what roughly matters. Boosting mainly reduces bias; shrinkage, subsampling and early stopping control its variance. Boosted trees with many rounds can become over-confident in probability terms, one reason we tune decision thresholds on the metric rather than trusting raw probabilities.

REMEMBER

- Each new tree corrects the current errors: target = label minus predicted probability.
- Leaf value = $-(\text{sum of gradients}) / (\text{sum of Hessians} + L2)$; learning rate shrinks every step.
- Final score = sum of all trees, passed through the sigmoid.

JUDGE MIGHT ASK

- *What does each new tree actually learn?* It learns the current errors: for log-loss the next tree's target is the label minus the current predicted probability, and its leaf values are Newton steps, the summed errors divided by their curvature plus the L2 penalty. The learning rate, 0.05 in our decision models and 0.075 in the ranker, shrinks each step so that many small trees add up.

A2.6 The knobs of boosted trees

IN ONE LINE

A handful of settings control how fast boosted trees learn, how complex each tree may be, and how strongly they are protected against memorising noise.

SIMPLE EXPLANATION

Think of seasoning a big pot of soup. The **learning rate** is how much salt you add each time you taste; small additions are safer but need more tastings. **Tree size** (depth or number of leaves) is how complicated each correction may be. The **number of trees** is how many rounds of tasting you do; early stopping tells you when to stop. **L2 regularisation** and **minimum rows per leaf** say "do not trust a judgement based on very few spoonfuls". **Row and feature subsampling** mean tasting from different parts of the pot each time, so no single spoonful dominates.

WORKED EXAMPLE (OUR REAL SETTINGS, DOC 4, PLUS BACKGROUND READ FROM THE SHIPPED CATBOOST MODEL FILE)

Setting	CatBoost ranker	LightGBM decision models	What it controls
Learning rate	0.075	0.05	size of each correction
Tree size	depth 7, symmetric (so $2^7 = 128$ leaves)	up to 127 leaves, grown leaf by leaf	how many feature interactions one tree can express

Setting	CatBoost ranker	LightGBM decision models	What it controls
Maximum trees	1,600	3,000 (US, India); 1,500 (France, default)	upper limit; early stopping picks the actual number
Early-stopping patience	180 rounds, on fold 6	100 rounds (code), on fold 8	when to stop
L2 penalty on leaf values	5	1.0	shrinks leaf values towards 0
Minimum rows per leaf	not used by symmetric trees (background)	200	no leaf may be based on fewer than 200 training rows
Row subsampling	Bayesian bootstrap, the library's GPU default: every row gets a random weight instead of being in or out (background, model file)	80 % of rows, redrawn every iteration	randomness that decorrelates trees
Feature subsampling	none, the library default (background, model file)	90 % of features per tree	same, across features
Random seed	2026 (code)	1 to 5, averaged	which random samples are drawn

Toy illustration of the minimum-rows rule: suppose a candidate split would isolate 3 training rows that happen to be all owners. Without a limit the tree would create a leaf with a large positive value from 3 examples, which is likely noise; with "at least 200 rows per leaf" that split is simply not allowed.

TECHNICAL EXPLANATION

The learning rate and the number of trees trade off: halving the learning rate roughly doubles the trees needed, usually with slightly better generalisation. Tree size sets the interaction order: a depth-7 tree can combine up to 7 features on one path; LightGBM's `num_leaves` caps the total leaves of an unbalanced tree. The L2 penalty λ enters every leaf value as $-G/(H + \lambda)$, so it shrinks leaves supported by little hessian mass. Minimum data per leaf (and minimum hessian per leaf) stops tiny, noisy leaves. Row subsampling (stochastic gradient boosting, Friedman 2002) and feature subsampling reduce correlation between trees and speed up training, at the price of making each fit depend on the random seed (see Seed averaging). Training time is roughly proportional to trees x sampled rows x sampled features.

HOW WE USED IT

The values in the table are our recipe (Doc 4), except those marked code or background, which come from the code and the shipped model file. The number of trees is never fixed by hand: it is chosen by early stopping, and for the US and India it is then scaled up for the refit on more folds (see Early stopping, then refit on more data). The operating thresholds that turn scores into links are tuned separately on the exact per-business F0.5 (Doc 4).

EXPERT NUANCE

These hyper-parameters interact, so tuning one at a time can mislead; 127 leaves with 200 rows per leaf is a sensible large-data setting but would overfit a small dataset. Our Documentation records the settings, not a hyper-parameter search, so we should not claim they are optimal; what we can say is that the number of trees was validated by early stopping and the final decisions by held-out F0.5. Subsampling makes single fits vary, which we then average away with five seeds.

REMEMBER

- Learning rate x number of trees is the main trade-off; early stopping picks the trees.
- Tree size sets interaction depth; L2 and minimum rows per leaf keep leaves stable.
- Subsampling adds healthy randomness, then seed averaging removes its noise.

JUDGE MIGHT ASK

- *How did you tune the boosting hyper-parameters?* We did not run a broad hyper-parameter search that we can report; the settings are standard large-data choices, the number of trees is chosen by early stopping on a held-out fold, and the thresholds that decide links are tuned on the exact per-business F0.5. The held-out scores, 0.9912 to 0.9915 for the US, are our evidence that the recipe generalises.

A2.7 CatBoost: symmetric trees, ordered boosting, categorical encoding

IN ONE LINE

CatBoost is a gradient-boosting library whose trees ask the same question at every level (symmetric trees) and which offers special machinery, ordered boosting and ordered target statistics, against a subtle kind of training leakage.

SIMPLE EXPLANATION

A **symmetric tree** (also called an **oblivious tree**) is like a fixed questionnaire: every pair answers the same 7 questions in the same order, and the 7 yes/no answers form a 7-digit binary code that points to one of 128 boxes, each holding a score. It is quick to apply and hard to overfit. **Ordered boosting** is like marking each student's homework with an answer key built only from students who handed in before them, so nobody's mark is influenced by their own answers. **Ordered target statistics** turn a text category such as "city = Nantes" into a number using only earlier rows, for the same reason.

WORKED EXAMPLE (TOY NUMBERS)

- *A symmetric tree of depth 2.* Level 1 asks "name similarity ≥ 0.8 ?" (bit b1). Level 2 asks "address coverage ≥ 0.95 ?" (bit b2) on both branches. The leaf index is $2 \times b1 + b2$, and the leaf table is $[-2.1, -0.4, 0.3, 1.9]$. A pair with similarity 0.9 ($b1 = 1$) and coverage 0.5 ($b2 = 0$) lands in leaf 2 and gets 0.3. Our ranker's trees work the same way with 7 questions and 128 leaves: 7 comparisons and one table look-up per tree.
- *An ordered target statistic.* Rows arrive in a random order with a category and a label: Nantes (1), Pessac (0), Nantes (0), Nantes (1). With a prior of 0.5 and a prior weight of 1, each row is encoded as (sum of earlier labels of its category + 0.5) / (count of earlier rows of its category + 1): row 1 Nantes = $0.5/1 = 0.5$; row 2 Pessac = 0.5; row 3 Nantes = $(1 + 0.5)/(1 + 1) = 0.75$; row 4 Nantes = $(1 + 0 + 0.5)/(2 + 1) = 0.5$. No row ever sees its own label.

TECHNICAL EXPLANATION

CatBoost (Prokhorenkova et al., 2018) has three signature ideas. (1) **Oblivious trees**: the same (feature, threshold) split is used for every node of a level, so a depth- d tree is defined by d splits and 2^d leaf values; prediction is a vector of bit comparisons plus a look-up, which is very fast and GPU friendly, and the constraint acts as a regulariser. (2) **Target leakage and ordered boosting**: in ordinary boosting, the gradient for example i is computed by a model that was itself fitted on example i , so gradients on training rows are biased compared with unseen rows (the CatBoost paper calls this **prediction shift**); ordered boosting keeps random permutations and computes example i 's gradient from a model built only on the examples before it. (3) **Categorical features** are encoded with ordered target statistics (as in the example), plus greedy combinations of categories. CatBoost also quantises every numeric feature into a fixed number of **borders** (bins) before training; the GPU default is 128. The boosting mode can be "Ordered" or "Plain"; plain (classic) boosting is the default on the GPU and for large datasets (background). Leaf values are estimated with Newton steps, and on the GPU each row gets a random **Bayesian bootstrap** weight for every tree (a continuous weight drawn at random rather than a yes/no sample; the bagging temperature 1 controls how uneven the weights are) (background).

HOW WE USED IT

The CatBoost ranker is a binary classifier with Logloss, up to 1,600 trees, depth 7, learning rate 0.075, L2 5, early stopping 180, trained on the GPU (Doc 4), random seed 2026 (code). Its inputs are the 76 numeric pair features of the index lookup (string similarities, word and n-gram overlaps, retrieval scores). It was trained on a 140,000-record pilot of labelled train records (35,000 per country and source), fitted on its fold 0-5 part, with early stopping on fold 6 (Doc 4). On the test set it scored the 37,614,782 pairs left after the quick rule-based filter (Doc 3), and its probability builds the shortlist and feeds the decision models. Background, checked in the shipped model file (not stated in the Documentation): boosting type "Plain" (not ordered), grow policy "SymmetricTree", 128 borders, Bayesian bootstrap, Newton leaf estimation, no categorical features. All 76 inputs are numbers, so the categorical machinery was not used.

EXPERT NUANCE

Ordered boosting matters most on small datasets and with target-encoded categories; with a large training set of purely numeric features, plain boosting is standard and the gradient bias it corrects is small. Symmetric trees are less expressive per tree than free-form trees, which is fine for a ranker whose job is recall; they make scoring tens of millions of pairs cheap. CatBoost training on the GPU is not bit-deterministic, which is one reason a full retrain from raw data reproduced 99.82 % of our links rather than all of them (CodeREADME 6): a retrained ranker gives slightly different scores, so some candidate lists change (the clean run's candidate file had 12,334,519 pairs against 12,293,019 submitted), and the later stages follow from that. We did not run a head-to-head comparison of CatBoost and LightGBM for either role.

REMEMBER

- Symmetric tree: same question per level; depth 7 means 7 comparisons and one look-up among 128 leaves.
- Our ranker: plain boosting, numeric features only, GPU, depth 7, up to 1,600 trees, learning rate 0.075.
- GPU training is fast but not bit-deterministic.

JUDGE MIGHT ASK

- *Why CatBoost for the ranker but LightGBM for the decision models?* We did not run a head-to-head comparison, so we do not claim one library is better. The ranker scores about 37.6 million pairs on 76 numeric features, and CatBoost's depth-7 symmetric trees train fast on the GPU and are very cheap to apply; the decision models have about two hundred features and use LightGBM's more flexible leaf-wise trees.
- *Did you use ordered boosting or CatBoost's categorical encoding?* No. All 76 ranker inputs are numbers, so no categorical encoding was needed, and the shipped model uses CatBoost's plain boosting mode with symmetric trees; ordered boosting mainly helps small datasets.

A2.8 LightGBM: histograms and leaf-wise growth

IN ONE LINE

LightGBM is a fast gradient-boosting library that sorts feature values into a small number of bins (histograms) and grows each tree leaf by leaf, always splitting the leaf where the gain is largest.

SIMPLE EXPLANATION

Instead of testing every exact value of a feature as a possible question, LightGBM first groups the values into up to 255 buckets, like grouping exam marks into ranges, and only tests the bucket boundaries; that is much faster and loses almost nothing. Then, instead of growing a tree layer by layer like a pyramid, it

behaves like a gardener who always works on the one branch that will improve the plant most: **leaf-wise growth**. The result is deep, uneven trees that capture the most useful combinations of features first.

WORKED EXAMPLE (TOY NUMBERS)

- *Histograms*. A feature such as "address coverage" over 1,000,000 training rows is binned into 255 bins. To find the best split, LightGBM adds up the gradients and Hessians of the rows in each bin once (one pass over the rows), then scans 254 bin boundaries instead of up to a million distinct values. After splitting a node, it builds the histogram of the smaller child only and gets the larger child's by subtraction: parent histogram minus smaller child = larger child.
- *Leaf-wise versus level-wise*. A tree currently has three leaves whose best possible splits would gain 5.0, 0.4 and 2.1. Level-wise growth would split all three. Leaf-wise growth splits only the leaf with gain 5.0, recomputes, and continues until the tree has 127 leaves (our setting). The budget of leaves is spent where it helps most.

TECHNICAL EXPLANATION

LightGBM (Ke et al., 2017) builds per-feature histograms of gradient and Hessian sums (default 255 bins, background), so split finding costs $O(\text{bins})$ per feature after an $O(\text{rows})$ accumulation, and **histogram subtraction** halves the work. **Leaf-wise (best-first) growth** expands the leaf with the largest gain, bounded by `num_leaves`; for the same number of leaves it reaches lower loss than level-wise growth but can overfit small data, which `min_data_in_leaf` and L2 control. Missing values get a learned default direction. The paper's two further tricks are **GOSS** (sampling rows by gradient size; we do not use it, we use plain bagging) and **exclusive feature bundling** (merging sparse features that are never non-zero together; on by default, background). The binary objective is log-loss, and the model outputs sigmoid of the summed tree outputs. `feature_fraction` samples features per tree; `bagging_fraction` with `bagging_freq` = 1 resamples rows every iteration. Training runs multithreaded on the CPU.

HOW WE USED IT

Our per-country **LightGBM decision models** are binary classifiers with learning rate 0.05, 127 leaves, at least 200 rows per leaf, feature fraction 0.9, bagging 0.8 every iteration, L2 1.0, label "candidate is the owner" (Doc 4). Training rows are the train shortlist of records that have a candidate in business folds 6-9 (Doc 4). The US model trains on US rows, the India model on US and Indian rows, and the France model on US and Indian rows only (there are no French labels); the default model for unseen countries is built like France's (Doc 4). Feature counts: 217 US, 211 India, 158 France and default (Doc 4). Each final model is the mean of five seeds, except the default (single fit). The second-chance models are also LightGBM (see Small models for the second-chance searches); retraining the India native-script rescue model and the India address-rescue models on the CPU reproduced their files byte for byte (CodeREADME 6).

EXPERT NUANCE

With about two hundred heterogeneous tabular features (similarities, counts, reader logits, rank and margin features), boosted trees are a strong default; neural networks rarely beat them on such data without much more tuning, and trees are fast and easy to check. Leaf-wise growth with 127 leaves can express interactions such as "no address, legal form changed and the readers unsure" that a shallow tree would need many rounds to approximate; 200 rows per leaf keeps each such leaf statistically grounded. LightGBM can report split-count and gain importances for every model, but we did not prepare a feature-importance ranking, so do not quote one.

REMEMBER

- Histograms: bin each feature (default 255 bins), search only bin boundaries; subtraction trick.
- Leaf-wise growth: always split the best leaf; capped at 127 leaves, at least 200 rows per leaf.
- Our decision models: learning rate 0.05, feature fraction 0.9, bagging 0.8, L2 1.0, mean of 5 seeds.

JUDGE MIGHT ASK

- *Why gradient boosting and not a neural network for the decision?* The inputs are about two hundred tabular features, and boosted trees are strong, fast and easy to check on such data; the neural networks are where they help most, in the readers that compare the raw text. We average five seeds for stability.
- *Why 127 leaves and 200 rows per leaf?* They are standard large-data settings in our recipe, and we did not report a search over them. With hundreds of thousands of training records per fold, 200 rows per leaf keeps each leaf's estimate stable while 127 leaves allow interactions between features, and early stopping on fold 8 chooses how many trees to keep.

A2.9 Learning to rank: pointwise, pairwise, listwise

IN ONE LINE

Learning to rank trains a model to order a list of candidates so the right one is near the top; it can learn from single items (pointwise), from pairs of items (pairwise) or from whole lists (listwise).

SIMPLE EXPLANATION

Picture a librarian who, for each question, must pull books off the shelf in order of usefulness. She could grade each book on its own ("useful or not": **pointwise**), compare books two at a time ("is this one better than that one?": **pairwise**), or judge the order of the whole pile at once (**listwise**). In our pipeline the ranker is like a shortlisting committee whose main job is not to leave the best applicant off a short interview list: what matters most is that the true business is among the few candidates the expensive readers will see.

WORKED EXAMPLE (TOY NUMBERS, PLUS REAL RECALL FROM DOC B.2)

- *Ranking metrics.* A record has four candidates, ranked B, A, C, D, and A is the owner. **Recall@1** (is the owner first?) = 0; **recall@4** (is it in the top 4?) = 1; **reciprocal rank** = 1/2; **NDCG** with one relevant item at rank 2 = $1 / \log_2(2 + 1) = 0.631$.
- *Pointwise versus pairwise loss.* Suppose the model scores the owner A at 1.0 and a non-owner B at 2.0 (logits). Pointwise log-loss treats each alone: A costs $-\ln(\text{sigmoid}(1.0)) = 0.313$ and B costs $-\ln(1 - \text{sigmoid}(2.0)) = 2.127$. A pairwise (RankNet) loss looks only at the order: $\ln(1 + e^{-(1.0 - 2.0)}) = 1.313$, and it would be the same for scores 11 and 12.
- *Our shortlist rule.* Record 1 has ranker probabilities 0.62, 0.30, 0.0008, 0.0005: the shortlist keeps ranks 1 and 2 (ranks 3 and 4 are below the 0.001 floor). Record 2 has 0.0002 and 0.0001: the top one is kept anyway (the top-ranked candidate is always kept), the second is dropped.
- *Real recall (derived from Doc B.2, US full fold 9).* The index finds the owner for 99.3694 % of owned records; the ranker's top 4 (without the filter) still holds it for 98.8814 %. So among records whose owner the index found, the ranker put it in its top 4 for $98.8814 / 99.3694 = 99.51$ % (India: $98.0809 / 98.5068 = 99.57$ %).

TECHNICAL EXPLANATION

Pointwise methods fit a regression or classification loss to each (query, item) independently; pairwise methods minimise a loss over ordered pairs, for example RankNet's $\log(1 + \exp(-(s_i - s_j)))$ for items i more relevant than j (CatBoost's PairLogit is such a loss); listwise methods optimise the whole list, for example LambdaMART, which weights pairwise gradients by the change in NDCG that swapping the two items would cause (LightGBM's lambdarank objective; CatBoost's YetiRank is a related list-aware method), or ListNet's softmax cross-entropy. Typical metrics are recall@k, **MRR** (mean reciprocal rank, the average of $1 / \text{rank}$ of the first relevant item) and **NDCG@k** = DCG/IDCG with DCG = sum over ranks of $(2^{\text{rel}} -$

$1/\log_2(1 + \text{rank})$ and IDCG the DCG of the perfect order. Two properties of our problem favour a pointwise classifier. First, some lists contain no relevant item at all, because an incoming record may belong to no business (Doc 2.1) or its owner may not have been retrieved; the final file leaves about 41 % of incoming records unlinked (derived from Doc B.1), and although not all of those lack an owner, it shows that "no link" is a common outcome, not an edge case. Pairwise and listwise losses get no gradient from a list without a relevant item, while a pointwise log-loss still learns to give all of its candidates low probabilities. Second, we need scores that are comparable across records: the shortlist uses one absolute floor (0.001) for every record, and the ranker's probability is an input to the decision model; scores trained with a ranking loss are usually only comparable within one list, because adding a constant to every score of a list does not change that loss.

HOW WE USED IT

The CatBoost ranker is a pointwise binary classifier (Logloss), not a learning-to-rank loss (Doc 4). The **shortlist** keeps, per incoming record, the top-ranked candidate always and ranks 2-4 only if their probability is at least 0.001, so at most 4 per record (Doc 3): 11,687,317 pairs, 6.75 per business and 1.17 per incoming record (derived). We measure its success as recall at each stage on labelled fold 9 (Doc B.2): US index 99.3694 %, shortlist between 98.8748 % and 99.3628 %; India index 98.5068 %, shortlist between 98.0572 % and 98.4831 %. The cap of four is the largest recall loss in the cascade (Doc 3). The ranker's probability and rank are also inputs of the decision model (Doc 4).

EXPERT NUANCE

Because the decision model re-scores and reorders the shortlist, the ranker does not need a perfect order, only high recall at 4; in the US example the ranker put the LLC business second, and the decision model gave it the lowest score of the four (Doc D.1). We did not benchmark a ranking loss, so we cannot claim pointwise is better in general, only that it fits our two requirements. The 0.001 floor's value has no separate tuning record in the Documentation; its effect is folded into the measured shortlist recall.

REMEMBER

- Pointwise = score items alone; pairwise = compare two; listwise = optimise the whole list (e.g. NDCG).
- Our ranker is pointwise (Logloss); its job is recall at the shortlist (top 1 always, ranks 2-4 if at least 0.001).
- The cap of 4 is the largest recall loss: US 99.37 % at the index versus at least 98.87 % in the shortlist.

JUDGE MIGHT ASK

- *Why not LambdaMART or another ranking loss for the ranker?* We did not benchmark one. The ranker's job is recall at a small cut-off with one absolute probability floor, and its probability is also an input to the decision model, so we wanted scores comparable across records; and some records own no business at all, which a pointwise log-loss still learns from while a ranking loss gets no signal from them.
- *Why a cap of four?* It is a cost-recall trade-off and our largest recall loss: on held-out fold 9 the shortlist keeps at least 98.87 % of US owners against 99.37 % at the index. Four means at most four expensive reader passes per record, 1.17 on average, and the second-chance searches target the main misses.

A2.10 Text into numbers: tokens, sub-words and embeddings

IN ONE LINE

Before a neural model can read text, the text is cut into pieces called tokens, and each token is looked up in a table that turns it into a list of numbers called an embedding.

SIMPLE EXPLANATION

A neural network only does arithmetic, so words must become numbers. First a **tokenizer** cuts the text into **tokens**. Rare or long words are broken into common pieces, like building any word from a fixed box of Lego bricks; these pieces are **sub-words**, and the full list of pieces the model knows is its **vocabulary**. Then each token is replaced by its **embedding**, a list of a few hundred numbers that acts like map coordinates: tokens used in similar ways end up near each other. "Nearness" between two such lists is usually measured with **cosine similarity**, which is 1 when they point the same way and 0 when unrelated.

WORKED EXAMPLE (REAL TOKENS FROM THE SHIPPED TOKENIZER; A TOY COSINE)

- "Legacy Marketing Enterprises (Inc)" becomes `__Lega | cy | __Marketing | __Enterprise | s | __(| | In | c | )`; "Legacy Marketing Enterprises LLC" ends in the single token `__LLC`. The symbol `__` marks the start of a word.
- The Devanagari name "विजय एंटरप्राइजेज प्राइवेट लिमिटेड" becomes 12 pieces: विजय and लिमिटेड are single tokens, while एंटरप्राइजेज is split into 6 pieces.
- The French pair of Doc D.3, record "TQ COMITE SARL | 2 R SUZANNE LENGLEN, NANTES" plus business "TQ Parents SARL | 2 Rue Suzanne Lenglen, Nantes, Pays de la Loire", is 49 tokens including the special start and separator tokens `<s> ... </s></s> ... </s>`. Note that "SUZANNE" in capitals becomes `__S | UZ | ANNE` while "Suzanne" becomes `__Suzan | ne`: the same word in a different case gives different tokens.
- Toy cosine: vectors $a = (1, 2, 2)$ and $b = (2, 1, 2)$. Dot product = $2 + 2 + 4 = 8$; lengths are both 3; cosine = $8 / 9 = 0.889$.

TECHNICAL EXPLANATION

Our encoders use a **SentencePiece** sub-word tokenizer with a vocabulary of 250,002 pieces covering about a hundred languages (the XLM-RoBERTa vocabulary; background, checked: the MiniLM and E5 tokenizers give identical ids). Sub-word segmentation gives an open vocabulary: any string, in any script, can be encoded, at the cost of longer sequences for words the vocabulary splits finely (Devanagari words here split more than English ones). A pair of texts is encoded as `<s> A </s></s> B </s>`, padded to a common length in a batch with a padding token that an **attention mask** tells the model to ignore. The **embedding matrix** has one row per token id (MiniLM's table has 250,037 rows, 35 more than the tokenizer ever produces; E5's has exactly 250,002; background, from the configurations); the token embeddings plus learned **position embeddings** form the input to the transformer layers, which then make each token's vector depend on its context (**contextual embeddings**, see Transformers and attention). In small multilingual models the embedding table dominates the parameter count: in MiniLM it holds 96.2 million of 117.7 million parameters (82 %), in E5 192.4 million of 278.0 million (69 %) (background, derived from the shipped model configurations).

HOW WE USED IT

All four readers use the MiniLM tokenizer (Doc 4), which is valid for the E5-based readers too because both share the same vocabulary (background, checked). Readers read the **raw** "name | address" text, not the normalised text (Doc 2.2): case, accents, punctuation such as "(Inc)" and native scripts reach the model as they are. Sequences are truncated to 128 tokens (Doc 4). The Indic-to-Latin word list is used by the index lookup and the features, not by the readers, which see Devanagari directly.

EXPERT NUANCE

Raw text keeps signal that normalisation would erase (a bracketed legal form, capitals that mark an acronym) but adds variation the reader must learn to ignore, such as the capitals-versus-title-case tokenisation above; fine-tuning on labelled pairs teaches that invariance. Truncation to 128 tokens rarely bites for name-plus-address pairs (our French example is 49), and attention cost grows with the square of length (see Transformers and attention). A tokenizer mismatch between pre-training and fine-tuning

would silently ruin a model; feeding the E5 readers with the MiniLM tokenizer is safe only because the two tokenizers are identical (background, checked when preparing this guide: same 250,002-piece vocabulary and the same ids on our examples).

REMEMBER

- Tokens are sub-word pieces from a fixed vocabulary (250,002 for our encoders); embeddings are their number vectors.
- Readers see raw text; a typical pair, like our French example, is about 49 tokens; the limit is 128.
- Most parameters of small multilingual models sit in the embedding table.

JUDGE MIGHT ASK

- *How do your readers cope with Devanagari names or French accents?* All four readers use one shared multilingual vocabulary of about 250,000 sub-word pieces, so Devanagari and accented French are tokenised directly from the raw text; for example विजय is a single token. Transliteration to Latin is used separately, for the index lookup and the features, where exact word matches matter.

A2.11 Transformers and attention

IN ONE LINE

A transformer builds a context-aware representation of every token by letting each token look at every other token and take a weighted mix of what it finds; the weights come from attention.

SIMPLE EXPLANATION

Imagine a library. You walk in with a question (the **query**). Every book has a label on its spine (the **key**) and contents inside (the **value**). You compare your question with every spine; the better a spine matches, the more of that book's contents you copy into your notes. In a transformer every token does this at the same time: it asks a question, every token offers a key and a value, and each token's new representation is a mix of values weighted by how well its query matched each key. This is **attention**. A **layer** repeats it several times in parallel with different "questions" (**heads**), then passes each token through a small neural network; a model stacks 12 such layers.

WORKED EXAMPLE (TOY NUMBERS; FOR SIMPLICITY EACH WORD IS ONE TOKEN HERE, ALTHOUGH THE REAL TOKENIZER SPLITS SOME OF THEM)

In a pair, the record's token "inc" looks at three tokens of the business: "legacy", "enterprises" and "llc". Use vectors of size $d = 4$.

1. Query of "inc": $q = (1, 1, 1, 1)$. Keys: legacy $(0, 0, 0, 0)$, enterprises $(1, 1, 0, 0)$, llc $(1, 1, 1, 1)$.
2. Dot products $q \cdot k$: 0, 2, 4.
3. Divide by $\sqrt{d} = 2$: 0, 1, 2.
4. Softmax (exponentiate and normalise): $e^0 = 1$, $e^1 = 2.718$, $e^2 = 7.389$, sum 11.107, so the attention weights are 0.090, 0.245, 0.665.
5. Values (size 2 for simplicity): legacy $(1, 0)$, enterprises $(0, 1)$, llc $(1, 1)$. Output = $0.090 \times (1, 0) + 0.245 \times (0, 1) + 0.665 \times (1, 1) = (0.755, 0.910)$.

The record's "inc" mostly draws information from the business's "llc": the two legal-form words find each other across the two records, so later layers can notice that they differ.

TECHNICAL EXPLANATION

For an input matrix X (n tokens $\times$ d dimensions), $Q = X W_Q$, $K = X W_K$, $V = X W_V$, and $\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$. The division by $\sqrt{d_k}$ keeps the dot products, whose variance grows with d_k , from saturating the softmax and killing its gradients. **Multi-head attention** runs h heads with

$d_k = d / h$ each, concatenates their outputs and projects them with W_O , so different heads can track different relations. Each encoder layer applies multi-head attention with a **residual connection** (the layer's input is added back to its output, so information and gradients can pass straight through) and **layer normalisation** (each token's vector is rescaled to mean 0 and variance 1, then multiplied and shifted by learned parameters), then a position-wise feed-forward network (two linear maps with a GELU non-linearity, applied to each token separately) with another residual and normalisation. Order information comes from learned absolute position embeddings in these encoders. Padding positions receive a very large negative score (effectively minus infinity) before the softmax via the attention mask, so they get zero attention weight. Cost per layer is $O(n^2 d)$ for attention plus $O(n d d_{ff})$ for the feed-forward part; at our short lengths (n at most 128) the feed-forward part is the largest share (for MiniLM at $n = 128$, about 151 million multiply-adds per layer for the feed-forward part against about 6 million for the attention scores themselves; derived from the configuration). Background, read from the shipped configurations: MiniLM has 12 layers, hidden size 384, 12 heads of 32 dimensions and a feed-forward size of 1,536; E5-base has 12 layers, hidden size 768, 12 heads of 64 and a feed-forward size of 3,072. Both are encoder-only, bidirectional models: every token sees the whole sequence.

HOW WE USED IT

Each reader is built on one of the two pretrained 12-layer transformer encoders (Doc B.5; the layer count is background, from the shipped configurations). Because a reader receives the record and the business as one sequence, every layer's attention runs across the two texts, which is what makes it a cross-encoder (see Bi-encoders versus cross-encoders). Inputs are at most 128 tokens and padding is masked (Doc 4; code).

EXPERT NUANCE

Attention weights are not reliable explanations of a decision; they show where information flowed, not why the final logit is what it is. The quadratic cost in sequence length is why transformers are expensive for long documents, but name-and-address pairs are short, so for us the expense is the number of pairs, not their length. Encoders like ours read the whole sequence at once and output a representation; most generative LLMs are decoder-only and produce text token by token, a different and much more expensive use.

REMEMBER

- Query asks, key advertises, value carries content; weights = $\text{softmax}(QK^T / \sqrt{d})$.
- 12 layers of multi-head attention plus feed-forward networks, with residuals and layer norm.
- In a cross-encoder, attention connects record tokens and business tokens directly.

JUDGE MIGHT ASK

- *What does attention give you that a string-similarity score does not?* A similarity score counts shared characters and treats every edit alike; attention lets each word of the record look at each word of the business, and fine-tuning on labelled pairs can teach which differences matter, for example that Inc versus LLC matters more than "Rue" written as "R". The decision model then adds the explicit "what changed" features on top.

A2.12 Pre-training, transfer learning and fine-tuning

IN ONE LINE

Pre-training teaches a model general language from huge amounts of text; fine-tuning then adapts all its weights to our narrow task using our labelled pairs.

SIMPLE EXPLANATION

It is like hiring a graduate who already reads many languages (pre-training, done by someone else on public text) and then giving them focused on-the-job training in comparing business records (fine-tuning, done by us). Reusing knowledge from one task for another is **transfer learning**. Fine-tuning needs far less data and time than learning a language from scratch, but it must be gentle, or the model forgets what it knew; that forgetting is called **catastrophic forgetting**.

WORKED EXAMPLE (OUR REAL CHAIN, DOC 4 AND B.5)

Step	Model	Who trained it	Data	Learning rate
1	multilingual MiniLM (117,653,760 parameters, Apache-2.0)	public model	general multilingual text (background)	-
2	MiniLM reader = step 1 + a new 385-parameter head	us	labelled US/India pairs of folds 0-5, one epoch	5e-5
3	French MiniLM reader = step 2, trained further	us	confident French pseudo-labels + 500,000 labelled US/India pairs	2e-5
1'	multilingual E5 base (278,043,648 parameters, MIT)	public model	general multilingual text pairs (background)	-
2'	E5 reader = step 1' + a new 769-parameter head	us	the same pairs as the MiniLM reader	5e-5
3'	French E5 reader = step 2', trained further, 7,419 steps	us	474,808 French pseudo-labelled + 474,808 labelled US/India pairs	2e-5

In steps 2 and 2' the head is new and starts random, while all other weights start from the pretrained values; in steps 3 and 3' every weight, head included, starts from the reader of step 2 or 2' (code). In every step all weights are updated (full fine-tuning).

TECHNICAL EXPLANATION

Encoders such as ours are pretrained mainly with self-supervised or weakly supervised objectives on public text: masked-language modelling for the XLM-R family (predict hidden words from their context), then distillation or contrastive training on text pairs for the sentence models (see MiniLM and E5) (background).

Full fine-tuning updates every weight with a small learning rate and a new task head; the alternative, **feature extraction**, freezes the encoder and trains only a head on its outputs (we use a frozen encoder only for the word-meaning features; see Frozen pretrained embeddings as features). Fine-tuning learning rates are typically several times smaller than pre-training ones, to adapt the pretrained structure without destroying it; a warm-up and a higher learning rate for the random head are standard (see The fine-tuning recipe). **Sequential fine-tuning** (labelled US/India first, then French) is a form of **domain adaptation**; mixing old-domain data back in (**replay**) limits forgetting (see Self-training with pseudo-labels and replay).

HOW WE USED IT

Exactly the two pretrained encoders above, at pinned revisions, downloaded once; every later step runs offline (Doc B.5; CodeREADME 1). Everything else, the CatBoost ranker, the LightGBM decision models and the second-chance models, was trained from scratch on the provided data (Doc B.5). No LLM runs in the pipeline, and every model is far below the 8-billion-parameter limit (Doc C.6). A larger reader, XLM-R large (about 560 million parameters, MIT), was tried during development and discarded (Doc B.5).

EXPERT NUANCE

Both public checkpoints were pretrained as sentence-embedding models (each text encoded separately), but we use them as backbones of cross-encoders that read the pair jointly; the cross-text attention patterns are learned during our fine-tuning (background reasoning). That works because the transformer itself does not care whether its input is one sentence or two. Fine-tuning on our labelled data also inherits whatever biases the pre-training text had, which is acceptable for name and address matching but worth naming.

REMEMBER

- Pre-training (public, general) then fine-tuning (ours, task-specific, small learning rate, new head).
- Our chain: MiniLM to MiniLM reader to French MiniLM reader; E5 to E5 reader to French E5 reader.
- Only the readers (and the frozen word encoder) start from pretrained weights; all tree models are trained from scratch; no LLM in the pipeline.

JUDGE MIGHT ASK

- *Which pretrained models does the pipeline use, and are they within the competition rules?* Exactly two pretrained encoders, multilingual MiniLM under Apache-2.0 with 117.7 million parameters and multilingual E5 base under MIT with 278.0 million, both far below the 8-billion-parameter limit and fine-tuned by us. The CatBoost ranker and all LightGBM models were trained from scratch on the provided data, and no external data or lookup is used.

A2.13 Bi-encoders versus cross-encoders

IN ONE LINE

A bi-encoder turns each text into a vector separately and compares the vectors; a cross-encoder reads both texts together and outputs one match score, which is more accurate but must be run once for every pair.

SIMPLE EXPLANATION

A **bi-encoder** is like comparing passports by measurements written on index cards: you measure every person once, and comparing two cards is instant, but anything not on the card is lost. A **cross-encoder** is like an examiner who holds both passports side by side and studies them together: slow, because she must sit with every pair, but she notices that exactly one letter of the name or the legal form differs. Our pipeline uses cheap steps to pick very few pairs, so the careful examiner only sees about one pair per incoming record.

WORKED EXAMPLE (DERIVED FROM DOC 2.1, DOC 3 AND DOC D)

- *Cost.* A bi-encoder would encode every text once: 1,732,544 businesses + 9,969,589 incoming records = 11,702,133 encodings (derived). Our shortlist holds 11,687,317 pairs (Doc 3), almost exactly the same number; each pair is read by two readers (US, India) or three (France), and each input is about twice as long because it holds two texts. Without the funnel, a cross-encoder over all same-country pairs would face 6,724,569,566,212 pairs (Doc 3), about 575,000 times the shortlist (derived).
- *Accuracy.* "Legacy Marketing Enterprises (Inc)" versus "Legacy Marketing Enterprises LLC": a bi-encoder squeezes each name into one vector where the legal-form difference is a small part of a 384-number summary; a cross-encoder can attend from "Inc" to "LLC" directly (Doc D.1 shows the decision model separating these names).
- *Real reader disagreement (Doc D.3).* For "TQ COMITE SARL" in Nantes, the same-name business "TQ Comite SARL" in Pessac got +7.125 from the MiniLM reader but -10.8125 from the French E5 reader: the French readers rejected this business in another city, which the MiniLM reader alone had liked, and the decision model gave it 0.000098.

TECHNICAL EXPLANATION

A bi-encoder scores $s(a, b) = \cos(E(a), E(b))$ with a shared encoder E ; embeddings can be precomputed and searched with approximate nearest-neighbour indexes (FAISS, HNSW), which makes bi-encoders the standard tool for dense retrieval. Its weakness is the single-vector bottleneck: fine token-level interactions are lost before the two texts ever meet. A cross-encoder computes $s(a, b) = \text{head}(\text{Encoder}([a; b]))$ with full attention between the two texts in every layer, so it can model exact correspondences, but nothing can be

precomputed per text and cost is proportional to the number of pairs. Late-interaction models (for example ColBERT) sit in between. The standard architecture in search is therefore **retrieve-then-rerank**: a cheap, recall-oriented retriever produces a short list and an expensive, precision-oriented cross-encoder re-scores it. Our funnel is that architecture: index lookup, quick rule-based filter and CatBoost ranker, then the readers.

HOW WE USED IT

All four readers are cross-encoders that score shortlist pairs only, at most 4 and on average 1.17 per incoming record (derived) (Doc 4). Retrieval uses an inverted index with word, letter-piece and sound lists, not embeddings (Doc 3). In the clean run, scoring the shortlist with two readers per pair (every reader except the French E5 reader) took about 1.4 hours on one 16 GB GPU (Evidence: clean-run report). The only bi-encoder-style use is the frozen MiniLM embedding of single words for the meaning-similarity features (see Frozen pretrained embeddings as features).

EXPERT NUANCE

Bi-encoders shine at recall for paraphrases, such as brand or "doing business as" names that share few words with the registered name, which is one of our documented miss types; a dense retriever would be a natural extra retrieval channel, but we did not benchmark one, so we cannot claim it would be worse (QA 1). Cross-encoders are not symmetric: the record is always the first text and the business the second, in training and at test, so the model never has to handle the reverse order. An LLM would be a far more expensive cross-encoder; under the competition's 8-billion-parameter cap and with 11.7 million shortlist pairs, small fine-tuned cross-encoders plus the decision model were the practical choice.

REMEMBER

- Bi-encoder: encode separately, compare vectors, precomputable, good for retrieval.
- Cross-encoder: read jointly, one score per pair, more accurate, cost proportional to pairs.
- Our funnel makes cross-encoders affordable: 11.7 million shortlist pairs, about one per record.

JUDGE MIGHT ASK

- *Why not use embeddings and vector search?* We did not benchmark a dense retriever, so we cannot claim it would be worse. Our index keeps the true business for 99.37 % of US and 98.51 % of Indian held-out records, and the expensive cross-encoders then see only 1.17 pairs per record; a vector index would be a natural extra channel for brand or "doing business as" names that share few words.
- *Why not a large language model as the matcher?* The rules cap models at 8 billion parameters, and at 11.7 million shortlist pairs the cost matters; small fine-tuned cross-encoders were accurate enough, and the decision model adds what they cannot see. No LLM runs in our pipeline.

A2.14 Inside our reader: pair input, mean pooling and the head

IN ONE LINE

Our reader feeds "record name | address" and "business name | address" as one sequence into the encoder, averages the final token vectors, and a single linear layer turns that average into one match logit.

SIMPLE EXPLANATION

The examiner reads both cards in one go, forms an overall impression by averaging everything she read (**pooling**), and then applies one final rule that turns the impression into a confidence score (the **head**). The rule is learned: it is a weighted sum of the impression's numbers plus a constant. Padding added to make inputs the same length is ignored when averaging.

WORKED EXAMPLE (TOY NUMBERS, PLUS OUR REAL INPUT)

1. Real input (49 tokens, Doc D.3 pair): `<s> TQ COMITE SARL | 2 R SUZANNE LENGLEN, NANTES </s></s> TQ Parents SARL | 2 Rue Suzanne Lenglen, Nantes, Pays de la Loire </s>`.
2. Toy final hidden vectors for 5 positions, of which the last 2 are padding: (1, 2), (3, 0), (2, 4), (9, 9), (9, 9). Mask = 1, 1, 1, 0, 0.
3. **Masked mean pooling**: average only the real tokens: $((1 + 3 + 2)/3, (2 + 0 + 4)/3) = (2, 2)$. A naive average including padding would give (4.8, 4.8), which is wrong.
4. Head: weights $w = (0.5, -0.25)$ and bias $b = 0.1$: $\text{logit} = 0.5 \times 2 - 0.25 \times 2 + 0.1 = 0.6$, so the probability is $\text{sigmoid}(0.6) = 0.646$.
5. Real head sizes (Doc B.5): MiniLM's hidden size is 384, so its head has 384 weights + 1 bias = 385 parameters; E5's has 768 + 1 = 769.

TECHNICAL EXPLANATION

The tokenizer encodes the pair with longest-first truncation to 128 tokens (tokens are removed one at a time from whichever text is currently longer). The encoder returns the last hidden state H ($n \times d$). Pooling is $\text{pooled} = \sum_i m_i h_i / \sum_i m_i$ with the attention mask m . During training, **dropout** with rate 0.1 is applied to the pooled vector (each of its numbers is set to zero with probability 0.1 and the rest are scaled up by $1/0.9$, so the head cannot rely on any single number), then a linear layer d to 1 gives the logit; the loss is binary cross-entropy on the logit (BCEWithLogitsLoss, computed in fp32). At inference the model is in evaluation mode (dropout off), batches are sorted by length to minimise padding, and the forward pass runs under bf16 autocast (code). No token-type ids, no E5 "query:"/"passage:" prefixes, and no pooler layer are used: the encoder's built-in pooler weights are loaded but ignored (background, code). Mean pooling matches how both base models were pretrained to summarise text (MiniLM's shipped pooling configuration is mean pooling, checked; E5 also uses average pooling, background).

HOW WE USED IT

The same architecture for all four readers (Doc 4). Each reader writes one logit per shortlist pair. The US and India decision models receive the MiniLM and E5 reader logits and their mean, each with its rank, the best other value, the margin over it and the record's top value within the shortlist (Doc 4; code). For France the decision model's single reader input receives the blend $0.35 \text{ MiniLM} + 0.15 \text{ French MiniLM} + 0.5 \text{ French E5}$ (Doc 4). The default route uses the MiniLM reader only (Doc 2.2).

EXPERT NUANCE

The alternative is to use the first token's vector (the `<s>` or CLS token); with a sentence-embedding backbone pretrained for mean pooling, mean pooling starts closer to a useful representation. A reader sees one pair at a time and cannot know whether another candidate is even better; that blind spot is why the decision model receives rank, margin and best-other features and why the final rule requires a clear winner. The head is tiny on purpose: almost all the task knowledge lives in the fine-tuned encoder weights.

REMEMBER

- Input: record text and business text in one sequence, raw "name | address", at most 128 tokens.
- Masked mean pooling over real tokens, dropout 0.1, linear head (385 or 769 parameters), one logit.
- A reader scores one pair in isolation; the decision model adds the competition between candidates.

JUDGE MIGHT ASK

- *Why mean pooling rather than the first token?* Both base models were pretrained to summarise text by averaging their token vectors, so mean pooling starts from a representation that already works; we did not compare it with first-token pooling.
- *Does a reader see the other candidates of the same record?* No, a reader scores one pair at a time; that is exactly why the decision model gets each reader's rank, its lead over the best other candidate and

the record's top score, because linking is a competition between candidates.

A2.15 MiniLM and knowledge distillation

IN ONE LINE

MiniLM is a small transformer trained to imitate a bigger one (knowledge distillation), and the multilingual MiniLM we used was further distilled so that a sentence in any of 50-plus languages lands near the vector of its English meaning.

SIMPLE EXPLANATION

Knowledge distillation is like an apprentice learning from a master craftsman: instead of learning only from right-or-wrong answers, the apprentice copies how the master works, including the master's uncertainty ("70 % this, 20 % that"). MiniLM's apprentice copies something specific: where the master's attention goes. The multilingual version then adds a translation lesson: a bilingual student learns to produce, for a French or Hindi sentence, the same summary vector that an English teacher produces for the English version. The result is a small, fast model that reads many languages.

WORKED EXAMPLE (TOY NUMBERS, PLUS REAL PARAMETER COUNTS)

- *Copying attention.* For one token, the teacher's attention over three tokens is (0.7, 0.2, 0.1) and the student's is (0.5, 0.3, 0.2). The distillation loss is the **KL divergence**, $\text{sum of teacher} \times \ln(\text{teacher} / \text{student}) = 0.7 \ln(1.4) + 0.2 \ln(0.667) + 0.1 \ln(0.5) = 0.236 - 0.081 - 0.069 = 0.085$. Training pushes it towards 0.
- *Copying sentence vectors across languages.* The English teacher gives (0.6, 0.8, 0.0) for an English sentence; the student gives (0.5, 0.7, 0.2) for its French translation. The loss is the mean squared error: $(0.1^2 + 0.1^2 + 0.2^2) / 3 = 0.02$.
- *Where MiniLM's 117,653,760 parameters sit (background, derived from the shipped configuration):* 96,212,352 in the embedding tables (82 %), 21,293,568 in the 12 transformer layers, and 147,840 in a pooler layer our reader does not use.

TECHNICAL EXPLANATION

Classic distillation (Hinton et al., 2015) trains a student on the teacher's softened output distribution (softmax with a temperature) using KL divergence. **MiniLM** (Wang et al., 2020; background) is "deep self-attention distillation": the student mimics the teacher's last-layer self-attention distributions, $\text{softmax}(QK^T / \sqrt{d})$, and the "value relation" $\text{softmax}(VV^T / \sqrt{d})$, both via KL divergence; because these are $n \times n$ matrices, the student's hidden size can differ from the teacher's. The multilingual MiniLM base (12 layers, hidden size 384) was distilled from XLM-R base and uses its 250,002-piece tokenizer (background). The checkpoint we used, paraphrase-multilingual-MiniLM-L12-v2, was then trained by **multilingual knowledge distillation** (Reimers and Gurevych, 2020; background): an English paraphrase model is the teacher, and the student is trained with mean squared error so that both an English sentence and its translation map to the teacher's English vector, using parallel sentences in 50-plus languages. Compute per token is governed by the transformer layers, not the embedding table (a look-up is cheap): MiniLM's layers have 21.3 million parameters against E5's 85.1 million, so MiniLM does roughly a quarter of the layer arithmetic per token (derived from the configurations).

HOW WE USED IT

The MiniLM reader (fine-tuned on labelled US/India pairs of folds 0-5, learning rate $5e-5$) is used in every country; the French MiniLM reader is its further fine-tune for France; the MiniLM tokenizer serves all four readers; and the frozen, not fine-tuned MiniLM embeds single words for the meaning-similarity features

(Doc 4, B.5). Licence Apache-2.0; 117,653,760 parameters plus a 385-parameter head (Doc B.5). In the France blend it has weight 0.35; the default route for unseen countries uses it alone (Doc 4).

EXPERT NUANCE

A sentence-embedding model trained for paraphrase similarity is a good starting point but not a matcher: two different businesses with near-identical names are "paraphrases" in its pre-training sense, which is exactly why fine-tuning on labelled owner/non-owner pairs, and the decision model's "what changed" features, are needed. Small models trade capacity for speed; we tried a larger XLM-R reader (about 560 million parameters) during development and discarded it, and we report no number for it (Doc B.5). Most of MiniLM's size is its multilingual vocabulary, so it is light on compute but not on memory.

REMEMBER

- Distillation: a small student imitates a big teacher (MiniLM copies attention patterns).
- Our MiniLM: 12 layers, hidden size 384, 117.7 million parameters (82 % embeddings), Apache-2.0, 50-plus languages.
- Used as the MiniLM reader (all countries), French MiniLM reader, shared tokenizer, and frozen word encoder.

JUDGE MIGHT ASK

- *Why such small models? Did you try bigger ones?* We tried a larger XLM-R reader, about 560 million parameters, during development and discarded it; we do not report a number for it. The small readers are fast enough to score 11.7 million shortlist pairs on one 16 GB GPU, and the decision model adds what they cannot see.

A2.16 E5 and contrastive pre-training

IN ONE LINE

E5 is a text-embedding model trained contrastively: it learns to place matching text pairs close together and non-matching pairs far apart.

SIMPLE EXPLANATION

Imagine a room full of question cards and answer cards. Each question card must find its own answer card among all the others; the model is rewarded when the true partner scores highest and punished when a stranger scores higher. With huge rooms (very large batches), every other card in the room is a free example of a wrong partner. This is **contrastive learning**. After training on an enormous number of such pairs, texts with the same meaning end up close together in the vector space.

WORKED EXAMPLE (TOY NUMBERS)

One query card and three answer cards in the batch, with cosine similarities 0.9 (its true partner), 0.5 and 0.3. The **temperature** tau sharpens the comparison; we use $\tau = 0.1$ here for readable numbers (E5 itself uses 0.01, background).

1. Divide by tau: 9, 5, 3.
2. Softmax: $e^9 = 8,103$, $e^5 = 148$, $e^3 = 20$; the true partner's share is $8,103 / 8,272 = 0.980$.
3. Loss = $-\ln(0.980) = 0.021$: small, because the true partner already wins.
4. If instead the true partner had 0.5 and a stranger 0.9, the true share would be 0.018 and the loss $-\ln(0.018) = 4.02$: large, so training pulls the true pair together and pushes the stranger away.

TECHNICAL EXPLANATION

The contrastive loss is **InfoNCE**: $L = -\log[\exp(s(q, p+)/\tau) / \sum_j \exp(s(q, p_j)/\tau)]$, where s is cosine similarity, $p+$ is the true partner and the other p_j are **in-batch negatives**; larger batches give more negatives and a better estimate. E5 ("Embeddings from bidirectional Encoder representations", Wang et al.; background) trains in two stages: weakly supervised contrastive pre-training on a very large set of naturally occurring text pairs (titles and passages, questions and answers; for the multilingual model, on the order of a billion multilingual pairs), then supervised fine-tuning on labelled datasets with mined hard negatives. multilingual-e5-base is initialised from XLM-R base: 12 layers, hidden size 768, 12 heads, and the same 250,002-piece vocabulary (background, checked in the shipped configuration). For its intended embedding use, inputs carry "query: " or "passage: " prefixes and are mean-pooled. Its 278,043,648 parameters split into 192,398,592 in the embedding tables (69 %), 85,054,464 in the transformer layers and 590,592 in an unused pooler (derived from the configuration).

HOW WE USED IT

The E5 reader was fine-tuned as a cross-encoder on the same labelled US/India pairs as the MiniLM reader, learning rate $5e-5$, and is used for the US and India (Doc 4). It is the starting point of the French E5 reader, fine-tuned on 474,808 French pseudo-labelled pairs plus 474,808 labelled US/India pairs for 7,419 steps at learning rate $2e-5$ (Doc 4), about 16 minutes on our RTX 4070 Ti SUPER (code). The plain E5 reader is not used for France; the French E5 reader has weight 0.5 in the France blend (Doc 4). Licence MIT; 278,043,648 parameters plus a 769-parameter head (Doc B.5). We use no "query:"/"passage:" prefixes (code).

EXPERT NUANCE

Contrastive pre-training gives an encoder whose representations already reflect "same meaning", which helps a cross-encoder converge quickly, but the joint reading of the pair is learned during our fine-tuning. Two readers with different pre-training (distillation for MiniLM, contrastive for E5) tend to make partly different mistakes, so feeding both to the decision model, plus their mean, lets it combine them; we did not isolate the gain of the second reader. E5 has about 2.4 times MiniLM's parameters and about four times its per-token layer arithmetic (derived from the configurations), so it is the more expensive reader. A written check showed that fine-tuning on French did not degrade it on US/India: **AUC** (the probability that a random true pair scores above a random wrong pair) was 0.998544 for the French E5 reader against 0.998570 for the E5 reader, within the 0.001 tolerance (Doc B.4).

REMEMBER

- Contrastive learning: pull true pairs together, push in-batch strangers apart (InfoNCE with a temperature).
- multilingual-e5-base: XLM-R base backbone, 278.0 million parameters, MIT.
- E5 reader for the US and India; French E5 reader for France (weight 0.5 in the blend); no prefixes used.

JUDGE MIGHT ASK

- *E5 is meant to be used with "query:" and "passage:" prefixes. Did you use it that way?* No. We use its pretrained encoder as the backbone of a cross-encoder: the record and the business are read together as one sequence and a new linear head gives one logit, fine-tuned on our labelled pairs, so the prefixes of its embedding use do not apply.
- *Why two different readers for the US and India?* They come from different pre-training, distillation for MiniLM and contrastive learning for E5, so they are likely to make partly different mistakes; the decision model receives both logits and their mean and learns from labels how far to trust each. We did not isolate the gain of the second reader.

A2.17 The fine-tuning recipe

IN ONE LINE

Our readers were fine-tuned with a standard, careful recipe: AdamW with small learning rates, a short warm-up, gradient clipping, batches of 128 similar-length pairs, and a single pass over the data.

SIMPLE EXPLANATION

Teaching a skilled worker a new task: start gently (**warm-up**), take small steps (a low **learning rate**), never overreact to one confusing example (**gradient clipping**), group similar jobs together so no time is wasted (**length-bucketed batches**), and go through the training material once, carefully (**one epoch**, meaning one full pass over the training data). The optimiser, **AdamW**, is the rule that decides how to adjust each weight after each **batch** (a group of examples processed together).

WORKED EXAMPLE (REAL SCHEDULE OF THE FRENCH E5 READER, DERIVED FROM DOC 4 AND THE CODE FORMULAS; TOY NUMBERS WHERE MARKED)

1. Steps: 949,616 pairs / batch 128 = 7,418.9, so 7,419 steps (one epoch).
2. Warm-up: 3 % of 7,419 = 222 steps. The encoder's learning rate rises linearly from 0 to $2e-5$ over those steps (at step 111 it is $1e-5$), then falls linearly to 0 at step 7,419 (at step 3,820 it is about $1e-5$ again). The head's learning rate is 5 times larger at every step (peak $1e-4$).
3. Gradient clipping (toy): if the gradient vector is (2.4, 3.2), its length is 4.0, above the limit 1.0, so it is rescaled to (0.6, 0.8): same direction, length 1.
4. Weight decay: at the peak learning rate $2e-5$ with weight decay 0.01, one step multiplies each encoder weight by $1 - 2e-5 \times 0.01 = 0.9999998$ before the update. Over the whole schedule the learning rates add up to $2e-5 \times (222 + 7,197) / 2 = 0.0742$, so the total shrink factor is about $e^{(-0.0742 \times 0.01)} = 0.99926$: every encoder weight is pulled towards zero by only about 0.07 % in the whole epoch (derived).
5. Length bucketing (toy): six pairs with token lengths 20, 90, 25, 88, 30, 95 and batches of 3. Random batches [20, 90, 25] and [88, 30, 95] are padded to 90 and 95: 555 slots for 348 real tokens, 37 % wasted. Sorted batches [20, 25, 30] and [88, 90, 95] need 375 slots: 7 % wasted.

TECHNICAL EXPLANATION

AdamW (Loshchilov and Hutter, 2019) keeps running averages of each weight's gradient (first moment) and squared gradient (second moment), steps by $m_hat / (\sqrt{v_hat} + \epsilon)$ scaled by the learning rate, and applies weight decay directly to the weights (decoupled from the gradient), $\theta \leftarrow \theta - lr \times \text{weight_decay} \times \theta$, which behaves better than L2 inside Adam. We use two parameter groups: the encoder at the base learning rate and the head at 5 times that, both with weight decay 0.01 (code; Adam's moment settings are the library defaults, background). The schedule is **linear warm-up for 3 % of the steps, then linear decay to zero** (code). Warm-up protects the pretrained weights while Adam's moment estimates are still noisy and, in the first fine-tuning, while the random head still produces large gradients. **Gradient clipping** rescales the global L2 norm of all gradients to at most 1.0, preventing a single bad batch from causing a huge update. Batches are formed by taking random chunks of 100 batches' worth of pairs, sorting each chunk by length, cutting it into batches of 128 and shuffling the batch order (code), and each batch is padded only to its own longest pair (**dynamic padding**). Training runs exactly one epoch under bf16 autocast (see Number formats), with seeds torch 0 and numpy 0 (code).

HOW WE USED IT

All four readers share the recipe: BCE loss, AdamW (weight decay 0.01, head learning rate x5), 3 % linear warm-up, bf16 autocast, gradient clipping 1.0, one epoch, batch 128, length-bucketed batches (Doc 4). Learning rates: $5e-5$ for the MiniLM and E5 readers (from the pretrained encoders), $2e-5$ for the two

French readers (continuing from already fine-tuned readers, so smaller steps) (Doc 4). In the clean run, training the MiniLM and E5 readers took about 2.1 hours together (Evidence: clean-run report).

EXPERT NUANCE

We did not compare one epoch with more epochs, so we cannot claim one is optimal; the argument is that pretrained encoders adapt quickly, one pass is already thousands of steps, and the readers' quality is then checked through the decision model on held-out folds. Many BERT recipes exclude biases and layer-norm weights from weight decay; ours applies 0.01 to every parameter, which at these learning rates is a very mild pull (step 4), so the difference is unlikely to matter, but it is a deviation worth knowing. Length bucketing changes which examples share a batch (similar lengths together), but shuffling the batch order keeps the training signal varied.

REMEMBER

- AdamW, weight decay 0.01; learning rate 5e-5 (MiniLM, E5 readers) or 2e-5 (French readers); head x5.
- 3 % linear warm-up then linear decay; gradient clipping at 1.0; batch 128; one epoch.
- Length-bucketed batches with dynamic padding cut wasted computation.

JUDGE MIGHT ASK

- *Why only one epoch?* Honestly, we did not compare it with more epochs. The pretrained encoders adapt quickly, one pass is already thousands of steps, 7,419 for the French E5 reader alone, and a single epoch keeps training time down; the readers are then judged through the decision model on held-out folds.
- *Why a higher learning rate for the head?* In the first fine-tuning the head is new and random while the encoder is already trained, so the head must move faster; five times the encoder's rate lets it catch up without shaking the pretrained weights. The warm-up protects the encoder in the same way during the first 3 % of the steps, and the French readers simply kept the same recipe at a lower base rate.

A2.18 Number formats: fp32, fp16, bf16 and mixed precision

IN ONE LINE

Computers store numbers with a fixed number of bits; fp32 is precise, while fp16 and bf16 use half the memory and run faster on modern GPUs, and bf16 keeps fp32's range while giving up precision.

SIMPLE EXPLANATION

Think of rulers. **fp32** (32-bit floating point) is a very long ruler with very fine marks. **fp16** (16-bit) is a short ruler with medium marks: very large numbers fall off the end (**overflow**) and very tiny numbers round to zero (**underflow**). **bf16** ("brain floating point", also 16-bit) is a long ruler with coarse marks: it can measure both tiny and huge things, but only roughly. **Mixed precision** means doing the heavy arithmetic with a 16-bit ruler while keeping the master copy of the weights on the fine 32-bit ruler.

WORKED EXAMPLE (REAL CONVERSIONS COMPUTED WITH PYTORCH, AND OUR OWN LOGITS)

Value	Stored in bf16	Stored in fp16
257	256 (marks are 2 apart at this size)	257
1.003	1.0 (too fine for bf16)	1.00293
0.00000001 (1e-8)	1.0012e-8 (kept, slightly rounded)	0 (underflow)
70,000	70,144	infinity (overflow; the fp16 maximum is 65,504)

Our stored reader logits sit exactly on the bf16 grid (derived from Doc D.3): between 8 and 16, bf16 can only store steps of $1/16 = 0.0625$, and the French readers' logits are $8.9375 = 143/16$ and $10.8125 =$

173/16; between 4 and 8 the step is 1/32, and the MiniLM logits are $7.5000 = 240/32$ and $7.1250 = 228/32$; between 1 and 2 the step is 1/128, and -1.9219 is $-1.921875 = -246/128$, rounded to four decimals in the Documentation. The head's output is computed in bf16 under autocast, and that is visible in the published numbers.

TECHNICAL EXPLANATION

A floating-point number is stored as a sign bit, an **exponent** (which sets the range, how large or small the number can be) and a **mantissa** (the significant digits, which set the precision). Bit layouts (sign / exponent / mantissa): fp32 is 1/8/23 (about 7 significant decimal digits; relative step $1.2e-7$); fp16 is 1/5/10 (maximum 65,504; relative step $9.8e-4$); bf16 is 1/8/7 (the same exponent range as fp32, maximum about $3.4e38$; relative step $7.8e-3$) (background). **Autocast** runs matrix multiplications in the 16-bit format on the GPU's **tensor cores** (units specialised in fast low-precision matrix arithmetic) while numerically sensitive operations such as softmax, layer normalisation and loss reductions stay in fp32; the weights and optimiser states remain fp32 (background). With fp16, small gradients underflow, so training needs **loss scaling** (multiply the loss before back-propagation, unscale afterwards); with bf16 the exponent range equals fp32's, so no loss scaling is needed, which makes training simpler and more robust. The cost is precision: bf16 rounding can differ between GPU kernels, between devices, and with batch composition (a different padded length can change which kernels run), so results are not bit-identical across runs.

HOW WE USED IT

All reader training runs under bf16 autocast on one RTX 4070 Ti SUPER (16 GB), with no loss scaling and the loss computed in fp32 (Doc 4; code); reader scoring also uses bf16 autocast, on that GPU and, for a small number of pairs, on the CPU (code; Evidence: scoring log). The Documentation states that bf16 reader training, like CatBoost GPU training, is not bit-deterministic (Doc A). Consequences we measured: a full retrain from raw data reproduced 99.82 % of the submitted links (validator PASS), while running the inference stages from the stored trained weights and reader logits rewrote both files byte for byte (CodeREADME 6). In the submitted run, 11,638,504 shortlist pairs carry GPU logits reused from earlier scoring runs and 48,813 were scored on the CPU (CodeREADME 6).

EXPERT NUANCE

bf16's coarse grid is harmless for a classifier whose logits feed tree models (a split at 7.5 versus 7.51 rarely changes anything), but tiny differences can flip borderline links near a threshold, which is exactly why a retrain is "very close, not byte-identical". fp16 would risk overflow in activations and require loss scaling; fp32 throughout would roughly double activation memory and lose most of the 16-bit tensor-core speed-up. Reproducibility was handled by shipping the stored logits and weights, which make the inference stages exactly repeatable.

REMEMBER

- fp32 fine and wide; fp16 fine but narrow (overflow above 65,504); bf16 coarse but wide.
- bf16 autocast: 16-bit arithmetic, fp32 master weights, no loss scaling needed.
- Not bit-deterministic: retrain reproduced 99.82 % of links; inference from stored weights and logits is byte-identical.

JUDGE MIGHT ASK

- *Why bf16 and not fp16 or full fp32?* bf16 halves the memory of the activations and uses the GPU's fast 16-bit tensor cores like fp16, but keeps fp32's exponent range, so we needed no loss scaling and had no more overflow risk than in fp32; the precision loss is small for a classifier. The cost is that training is not bit-deterministic.
- *Is your pipeline reproducible?* From the trained weights and stored reader scores, the inference stages rewrite both submitted files byte for byte. Retraining from raw data is not bit-identical, because GPU

CatBoost and bf16 reader training are not deterministic, and it reproduced 99.82 % of our links with the validator passing.

A2.19 Hard-example selection for reader training

See also [A3.18 Hard examples and hard negatives](#): hard examples and hard negatives from the training side.

IN ONE LINE

Instead of training the readers on every candidate pair, we mostly train them on records where the cheaper CatBoost ranker was unsure or wrong, plus a random quarter of the easy records.

SIMPLE EXPLANATION

A driving instructor spends most lessons on roundabouts and busy junctions, where learners actually make mistakes, but still includes some straight-road driving so the learner does not forget it. Our readers are the expensive specialists; spending their training on cases the ranker already gets right with near certainty teaches them little. So we pick the **hard examples**, the records near the decision boundary or where the ranker failed, and add a random sample of easy ones.

WORKED EXAMPLE (TOY RECORDS; THE RULES ARE THE DOCUMENTED ONES, DOC 4)

A record is **hard** if the ranker's top probability is strictly between 0.003 and 0.9995, or a non-owner is ranked first, or the second-ranked probability is above 0.003.

Record	Top probability	Top candidate is the owner?	Second probability	Selected?
R1	0.9999	yes	0.0001	easy: kept only by the 25 % random draw
R2	0.62	yes	0.30	hard: top probability inside (0.003, 0.9995)
R3	0.9997	yes	0.004	hard: second probability above 0.003
R4	0.9998	no, a non-owner is first	0.002	hard: a non-owner ranked first
R5	0.002 (record owns no business)	-	0.001	easy: kept only by the 25 % random draw

R5 does not count as "a non-owner ranked first", because in the code that condition applies only to records that have an owner (code). Then, inside a selected record, candidates at ranks 1-4 are used if they are ranked first, have probability at least 0.001, or are the owner; candidates at ranks 5-8 are used only if they are the owner. For R4, if the owner sat at rank 6, that pair would still be added as a positive example: the reader learns to recognise owners that the ranker underrated.

TECHNICAL EXPLANATION

This is **hard-example mining** combined with **random under-sampling of easy examples**. Concentrating training on the region where the cheaper model is uncertain spends the expensive model's capacity where it adds information, a standard idea in cascades and in re-ranker training. Keeping a random 25 % of easy records preserves coverage of the easy majority, so the reader still outputs confidently high and low logits there instead of drifting. Candidate selection mirrors the inference-time shortlist (ranks 1-4, floor 0.001, top always), so the training distribution resembles what the reader sees at test; adding owners from ranks 5-8 enriches positives that the shortlist would have missed. All selected records and businesses come from folds 0-5, keeping folds 6-9 clean for the decision models (see Stacking). The sampling changes the base rate of positives, so reader logits are not calibrated to the test distribution; the decision model, trained on the folds 6-9 shortlists, corrects that.

HOW WE USED IT

Both the MiniLM reader and the E5 reader were trained on exactly these pairs (Doc 4). The French readers use pseudo-labelled French pairs plus labelled US/India replay pairs instead (see Self-training with pseudo-labels and replay).

EXPERT NUANCE

Hard examples contain more label noise and more genuinely ambiguous pairs, so a model trained only on them can become erratic; the random easy fraction counters that. The thresholds 0.003, 0.9995 and the 25 % fraction are design choices recorded in our Documentation, not tuned values we can defend with an ablation. Because no link is ever made on a reader score alone, a calibration distortion from sampling does not reach the links directly: the decision model, trained on other folds, learns how to read the reader scores.

REMEMBER

- Hard = ranker unsure (top in (0.003, 0.9995)), wrong (non-owner first), or a strong runner-up (second above 0.003).
- Plus a random 25 % of easy records; candidates mirror the shortlist, plus owners from ranks 5-8.
- Sampling distorts the readers' base rate; the decision model, trained on other folds, corrects it.

JUDGE MIGHT ASK

- *Doesn't training on hard examples bias the readers?* It changes their base rate, yes, which is why we never link on a reader score directly: the decision model is trained on different folds and learns how to read the reader scores. The random quarter of easy records keeps the readers confident on the easy majority.

A2.20 Frozen pretrained embeddings as features

IN ONE LINE

For the "what changed" features we use the pretrained MiniLM, without fine-tuning, to measure how close in meaning a replaced word is to the word it replaced.

SIMPLE EXPLANATION

Imagine a thesaurus that gives a closeness score to any two words: "Enterprises" and "Industries" are fairly close, "Enterprises" and "Bakery" are far apart. When a candidate business differs from a record by one word, it matters whether that word was swapped for a near-synonym or for something unrelated. We ask the pretrained MiniLM, used as it is (**frozen**, meaning not trained further), for each word's vector and compare the vectors with cosine similarity.

WORKED EXAMPLE (REAL NUMBERS FROM DOC D.1)

Record "Legacy Marketing Enterprises (Inc)" and candidate "Legacy Marketing Industries Inc". The "what changed" features see one word replaced in place ("Enterprises" became "Industries"). Its meaning similarity, the cosine between the frozen MiniLM vectors of the two words, is 0.77, and the IDF-weighted word overlap of the two names (the rarity-weighted share of words they have in common, IDF as defined at the top of this part) is 0.52 (Doc D.1). For comparison, the toy vectors (1, 2, 2) and (2, 1, 2) have cosine 0.889 (see Text into numbers). The decision model gave this candidate 0.000088 while it gave the exact-name candidate 0.965022 (Doc D.1): a fairly close synonym still changes the name, and the model treated it as a real difference. (Test labels are private, so this shows how the model decides, not whether it was right.)

TECHNICAL EXPLANATION

Each distinct word is embedded on its own by the sentence-transformers MiniLM (mean pooling, vectors normalised to length 1), so cosine similarity is a dot product; this gives effectively static word vectors from a contextual model (code; background). Here a **new** word is a word of the incoming record's name that the business name lacks, and a **dropped** word is a word of the business name that the record lacks (legal-form words removed first). Three features use the vectors: the cosine between the first new word and the

first dropped word; the best cosine between the first new word and any word of the business name; and the best cosine between the first dropped word and any word of the record's name (FACTS A6, from the code). In the example, "Enterprises" is the new word and "Industries" the dropped one. When there is no such word the feature takes a sentinel value (-2) that the trees treat as its own case. Only the words of records that appear in pairs are embedded, and the vectors are cached in fp16 to save memory (code). The features are label-free and generic, so they work unchanged in France.

HOW WE USED IT

The three meaning-similarity features are part of the 25 generic "what changed" pair features in every decision model: US, India, France and the default route (Doc 4). The encoder is the same pretrained checkpoint the MiniLM reader starts from, but here it is never fine-tuned (Doc B.5).

EXPERT NUANCE

General-language meaning is not business identity: "Industries" and "Enterprises" are related words, yet two names that differ by them may belong to different businesses, so a high cosine must not be read as "harmless change"; the decision model learns from labels how much each kind of change matters, combined with rarity (IDF) and position features. Embedding single words out of context loses some meaning (a word such as "Martin" can be a surname or part of a place name), which is why these features complement the readers rather than replace them. Fine-tuning this encoder would make the features task-specific but would tie them to labelled countries and add training cost.

REMEMBER

- Frozen MiniLM, single words, cosine similarity: how close in meaning the replaced word is.
- Three features, part of the 25 generic "what changed" features in every country's decision model.
- Example: Enterprises vs Industries, meaning similarity 0.77; the decision model still scored that candidate 0.000088.

JUDGE MIGHT ASK

- *Why not fine-tune the encoder used for these word features?* These features are meant to be generic and label-free, so they carry over to France, where we had no labels; the fine-tuned readers already learn task-specific meaning from whole pairs, and the frozen word similarity adds a separate, simple signal the decision model can weigh.

A2.21 Self-training with pseudo-labels and replay

See also [A3.20 Self-training with pseudo-labels](#): self-training from the training side; [A3.21 Replay and catastrophic forgetting](#): replay.

IN ONE LINE

When a country has no labels, a model's own very confident predictions on that country's records are used as training labels (pseudo-labels), mixed with real labelled examples so the model does not forget what it already knew.

SIMPLE EXPLANATION

A teacher moves to a school in another country with no answer keys. She marks only the papers she is completely sure about and uses those to adjust her marking to the new style; that is **self-training**, and her confident marks are **pseudo-labels**. To avoid drifting into her own mistakes, she keeps re-marking some old papers whose answers she knows for certain; that is **replay**. She also sets aside a few of her confident marks, never uses them for practice, and later checks that she still agrees with them.

WORKED EXAMPLE (DERIVED: THE DOCUMENTED FRENCH E5 SELECTION RULE APPLIED TO THE REAL LOGITS OF DOC D.3)

The rule: confidence $p = \text{mean of sigmoid}(\text{MiniLM logit}) \text{ and } \text{sigmoid}(\text{French MiniLM logit})$. A pair becomes a **positive** if it is the record's best pair, $p \geq 0.97$, and every other pair of that record has $p < 0.5$. The record's highest- p pair with $p \leq 0.03$ becomes a **negative** (the hardest confident non-match).

1. TQ Parents SARL (same address): $p = (0.99945 + 0.99987) / 2 = 0.99966$, at least 0.97.
2. TQ Comite SARL (Pessac): $p = (0.99920 + 0.12765) / 2 = 0.5634$, which is not below 0.5.
3. So the record has no positive (its runner-up is not clearly rejected) and no negative (no pair at or below 0.03): it contributes nothing. The selection is deliberately strict.
4. Real totals (Doc 4, B.4): 474,808 French pseudo-labelled pairs plus 474,808 labelled US/India pairs, 949,616 in all; 5 % of French records held out, giving 25,192 held-out pairs (12,616 positive).

TECHNICAL EXPLANATION

Self-training (pseudo-labelling) trains a student on a teacher's confident predictions on unlabelled target-domain data; it works when the confident region is accurate and covers useful variation, and fails through **confirmation bias** (the teacher's systematic errors are reinforced) and **selection bias** (confident pairs are the easy ones). Mitigations used here: a high confidence threshold (0.97 / 0.03), a **teacher ensemble** (agreement of two readers via their mean probability), a requirement that the runner-up be clearly rejected, class balance (at most 250,000 per class, at most one positive and one negative per record), hard but confident negatives, and replay of labelled source-domain pairs about one to one, which limits **catastrophic forgetting** and anchors the decision boundary learned with real labels. This is **unsupervised domain adaptation**: adapting from labelled US/India to unlabelled France. Evaluation is the weak point: agreement with held-out pseudo-labels measures fidelity to the teachers, not accuracy, so independent checks are needed.

HOW WE USED IT

Two French readers (Doc 4). The French MiniLM reader was fine-tuned (learning rate $2e-5$) on very confident pseudo-labels from an earlier run of our pipeline (best candidate, score at least 0.97, clear lead) plus confident negatives, together with 500,000 labelled US/India pairs (QA 9). The French E5 reader was fine-tuned from the E5 reader on the 474,808 + 474,808 pairs above, 7,419 steps (Doc 4). Written checks for the French E5 reader (Doc B.4): US/India quality within tolerance (AUC 0.998544 against 0.998570 for the E5 reader): pass; agreement 0.999206 with held-out French pseudo-labels (the E5 reader: 0.981304; required 0.98): pass; a generated labelled French test set (best F0.5, the highest F0.5 over all thresholds, 0.910516 against 0.914964 for the E5 reader): fail; our integration rule that the expected gain be safely positive (pessimistic estimate -0.0001806): fail. Two variants were tried and not adopted on public score: a second French self-training round (0.991277 against 0.991300) and the French MiniLM reader retrained with different pseudo-label filters (0.991258) (Doc B.6).

EXPERT NUANCE

Limitation first: France has no labels, so we have no measured French F0.5, and the French E5 reader was kept although it failed two of its four written checks, because the file containing it had our best public score; that was a leaderboard-driven decision against our own written rule, and it is disclosed (Doc C.3). The 0.999206 agreement is partly circular (the teachers chose those labels), which is why we also wrote the generated-test-set check and the integration rule. Self-training chains teachers: the French MiniLM reader, itself self-trained, is one of the two teachers of the French E5 reader, so errors can compound; strict thresholds and replay are the guard.

REMEMBER

- Pseudo-labels: only very confident pairs (p at least 0.97 with a clearly rejected runner-up; negatives at most 0.03).

- Replay: about one to one with labelled US/India pairs (exactly 474,808 + 474,808 for the French E5 reader).
- Checks: two passed, two failed; reader kept on public score; no measured French F0.5.

JUDGE MIGHT ASK

- *Isn't self-training circular, the model just learning its own mistakes?* That is the main risk, so we kept only very confident pairs (at least 0.97 with no close rival, or at most 0.03), mixed them about one to one with real labelled US and Indian pairs, and checked the result in writing. It kept US/India quality within tolerance and agreed 0.999206 with held-out French pseudo-labels, but that agreement is partly circular and it failed on a generated French test set; we have no measured French F0.5, and we say so.
- *Why mix in labelled US and Indian pairs?* So the readers keep what they learned from real labels while adapting to French text; the check on labelled US/India pairs shows it worked, AUC 0.998544 against 0.998570 for the reader we started from.

A2.22 Blending readers in logit space (France)

See also A3.22 Blending readers in logit space: the same blend from the validation side.

IN ONE LINE

For France, the three readers' logits are combined as a weighted average, 0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5, and that single number fills the decision model's reader input.

SIMPLE EXPLANATION

Three judges give their verdicts on a "log-odds" scale, where a very sure "no" is a large negative number. A weighted vote on that scale lets one judge's strong, well-founded "no" outweigh two mild "yes" votes. If we averaged their percentages instead, each judge's influence would be capped by its weight, and a confident "no" (0.00002) would count for little against a "yes" (0.999).

WORKED EXAMPLE (REAL LOGITS, DOC D.3)

Candidate	MiniLM	French MiniLM	French E5	Blend 0.35 / 0.15 / 0.5
TQ Parents SARL, same address	7.5000	8.9375	10.8125	$0.35 \times 7.5 + 0.15 \times 8.9375 + 0.5 \times 10.8125 = 2.625 + 1.341 + 5.406 = \mathbf{9.3719}$
TQ Comite SARL, Pessac	7.1250	-1.9219	-10.8125	$2.494 - 0.288 - 5.406 = \mathbf{-3.2008}$ (probability 0.039)

Had we averaged probabilities instead for Pessac: $0.35 \times 0.99920 + 0.15 \times 0.12765 + 0.5 \times 0.00002 = 0.369$ (a logit of -0.54): the French E5 reader's confident rejection would have been almost ignored (derived). The weights sum to 1, so the blend stays on the scale of a single reader's logit. They also decompose as $0.5 \times (0.7 \text{ MiniLM} + 0.3 \text{ French MiniLM}) + 0.5 \times \text{French E5}$, half the earlier France blend plus half the new reader (FACTS A12; Doc B.4).

TECHNICAL EXPLANATION

Averaging logits is **log-linear pooling**: in probability terms it is a weighted geometric mean of the odds, which preserves strong evidence in either direction, while averaging probabilities (linear pooling) compresses it. The France decision model was trained on labelled US/India rows with the MiniLM reader's logit in its reader input (Doc 4); at test that input receives the blend. Feeding a model an input whose distribution differs from the one it was trained on is a form of **covariate shift** (the inputs change between training and use while the task stays the same); keeping the weights summing to 1 keeps the blend roughly on the trained scale, and tree models treat values beyond the training range as the nearest learned region (they do not extrapolate), so unusually confident French logits such as -10.8 behave like strong "no" values. The weights could not be learned by the decision model because there are no French

labels; the French MiniLM reader's weight was set from label-free counts on the test records (Doc C.1), and the final blend is half the earlier 0.7 / 0.3 blend plus half the French E5 reader; the Documentation does not record how the 0.5 was chosen.

HOW WE USED IT

France only (Doc 4). The US and India decision models receive the MiniLM and E5 logits separately, plus their mean, and learn the combination from labels; the default route uses the MiniLM reader alone (Doc 4). Comparing the final file with the same pipeline without the French E5 reader (France blend 0.7 MiniLM + 0.3 French MiniLM): French links -2,228 / +3,396 (5,437 business rows changed), with US, India and candidates identical (Doc B.4).

EXPERT NUANCE

Fixed blending is the right tool when the combination cannot be learned, which is exactly France's situation; where labels exist, a learned combination (stacking) is usually better, and that is what the US and India get. The blend changes the input distribution of a model trained without it, so its effect can only be judged label-free or through the public score; we disclose that the French E5 reader, and thus this blend, failed two of four written checks and was kept on public score (Doc B.4, C.3). The worked example shows the benefit in one case: the blend rejects the same-name business in another city that the MiniLM reader alone liked.

REMEMBER

- France reader input = 0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5, on logits; weights sum to 1.
- Logit averaging keeps a confident "no" strong; probability averaging would mute it.
- Fixed weights because there are no French labels; US/India learn the combination instead.

JUDGE MIGHT ASK

- *Why blend logits with fixed weights in France, when for the US and India the decision model combines the readers itself?* The France decision model is trained on US and Indian rows, where the French readers' behaviour on French text cannot be learned, and we have no French labels to fit weights; so its single reader input receives a fixed blend whose weights sum to one, the scale it was trained on. The French MiniLM weight was set from label-free counts on the test records, and the final blend is half the earlier blend plus half the French E5 reader.

A2.23 Stacking: a two-level model with separate folds per layer

See also [A3.6 Layer-wise fold separation for stacked models](#): the fold design that makes stacking safe.

IN ONE LINE

Stacking puts a second model on top of first-level models: the CatBoost ranker's probability and the readers' logits become inputs of the LightGBM decision model, which is trained on different businesses so it learns how far to trust them.

SIMPLE EXPLANATION

A hiring panel's chair (the **meta-model**, our decision model) listens to specialists (the **base models**: the ranker and the readers) and also reads the plain facts of the CV (our features). To learn whose opinion to trust, the chair must watch the specialists judge candidates they have never met; if the chair only watched them re-judge candidates they had already studied, every specialist would look perfect and the chair would learn to over-trust them. So the specialists and the chair are trained on different groups of businesses.

WORKED EXAMPLE (OUR REAL FOLD LAYOUT, DOC 4, AND A TOY ILLUSTRATION)

Folds (by business)	What learns there
0-5	CatBoost ranker (fitted on the fold 0-5 part of its 140,000-record pilot); MiniLM and E5 readers; the Indic-to-Latin word list
6	early stopping of the ranker
6-7	decision models are fitted
8	early stopping of the decision models; threshold 0.75 chosen
9	main held-out report (scored by fits that leave it out); with fold 8, sets the consistency check
6-9	US/India decision models refitted

Toy over-trust: a reader trained on pair X memorised it and gives it +9, while on unseen pairs of the same difficulty it gives about +3. If the decision model trained on X, it would learn "+9 means certain" and become over-confident at test, where such values are rarer. Real effect of the second level (Doc D.1): for "Legacy Marketing Enterprises (Inc)" the ranker ordered the candidates Inc, LLC, PC, Industries; the decision model scored them 0.965022, 0.000007, 0.088664 and 0.000088, moving the LLC from second to last. The Documentation credits the legal-form features and the readers with separating these names; we have no per-feature attribution for this record.

TECHNICAL EXPLANATION

Stacked generalisation (Wolpert, 1992) trains a meta-learner on base-model predictions, which must be out-of-sample for the meta-learner's training rows; otherwise base predictions are optimistically biased there and the meta-learner over-weights them. The usual recipe is **out-of-fold (OOF) predictions**: train K copies of each base model, each predicting the fold it did not see. We use a **layer-wise split** instead (a hold-out form of stacking): base models on folds 0-5, meta-model on folds 6-9, so the base scores on the meta-model's rows are genuinely held out: they come from models that never saw those records, exactly as at test time. The trade-off is that each layer sees only part of the data, in exchange for training each reader once instead of K times. Our meta-features include the base scores, their within-record context (rank, best other score, margin, record maximum), the index lookup's similarity features, the "what changed" features and, for the US and India, collective features over the records competing for the same business (Doc 4). The decision is then made per record: best candidate, confident and clear winner (Doc 4).

HOW WE USED IT

LightGBM decision models per country, trained on the train shortlist of records with a candidate in business folds 6-9 (Doc 4): US on US rows (217 features), India on US and Indian rows (211), France on US and Indian rows (158), default as France (Doc 4). For the US and India the training rows also include extra hard examples; say this only if asked, and only in these words: copies of unlinked training records that sit right next to a candidate business, with small name edits, where how many to add was set from label-free counts on the test records, never from labels, and the France model uses no such examples (QA 15). Held-out per-business F0.5: US 0.991212 / 0.991545, India 0.990046 / 0.990298 (fold 9 / fold 8) (Doc 5).

EXPERT NUANCE

Two small, honest caveats a sharp judge may spot. Fold 6 both stopped the ranker's training and is a decision-model training fold; only the stopping point was chosen there, a far weaker leak than fitting on it. And the decision models' training shortlist was built from the index output without the quick rule-based filter, while at test the filter runs first; the filter kept the owner for 99.974 % of held-out records and moved fold-9 F0.5 from 0.990117 to 0.990086 (Doc 3), so the mismatch is small. Out-of-fold stacking would use the data more fully but needs several copies of each reader; with fold 9 alone holding about 760,000 owned US and Indian records (457,971 + 303,786, derived from Doc B.2), each layer had ample data.

REMEMBER

- Base models (ranker, readers) on folds 0-5; meta-model (decision model) on folds 6-9.
- Base scores on the meta-model's rows must be out-of-sample, or the meta-model over-trusts them.
- The decision model adds what readers cannot see: competition between candidates and "what changed".

JUDGE MIGHT ASK

- *Why not use out-of-fold predictions so that every model sees all the data?* Out-of-fold scores would need several copies of each reader and of the ranker, which is expensive for cross-encoders; a clean split, ranker and readers on folds 0-5 and decision models on 6-9, gives the decision model scores on records the readers never trained on, at the cost of less data per layer. Fold 9 alone holds about 760,000 owned US and Indian records, so each layer still had plenty.

A2.24 Early stopping, then refit on more data

See also [A3.9 Early stopping](#): early stopping; [A3.10 Refitting on more folds with scaled iterations](#): the refit with scaled iterations.

IN ONE LINE

We first find the right number of trees on a validation fold, then retrain on more folds (6-9) with that number scaled up in proportion to the extra rows.

SIMPLE EXPLANATION

A recipe tested for four people tells you the cooking time; when you cook for eight, you adjust the time for the bigger batch instead of guessing. Early stopping on fold 8 tells us how many trees suit a model trained on folds 6-7. When we retrain on folds 6-9, roughly twice the rows, no fold is left to stop on, so we scale the number of trees by the ratio of rows.

WORKED EXAMPLE (TOY NUMBERS, THEN REAL PUBLIC SCORES)

Seed 1 is fitted on folds 6-7 with 1,000,000 rows and early stopping on fold 8 finds the best iteration at 600 trees. Folds 6-9 have 2,000,000 rows. The refit trains $600 \times 2,000,000 / 1,000,000 = 1,200$ trees with no early stopping. Because folds are assigned by a hash of the business id, folds are of similar size and the ratio is close to 2 (reasoning, not a reported number). Real: adding the US/India refit moved the public score from 0.990621 to 0.990676, and it was kept; the same refit for France scored 0.990726 against 0.990748 without it, and it was not adopted (Doc B.6).

TECHNICAL EXPLANATION

At a fixed learning rate, the optimal number of boosting rounds usually grows with the amount of training data, because overfitting sets in later; scaling the early-stopped iteration count linearly with the number of rows is a common heuristic, not a law. It is applied per seed, each seed using its own best iteration: $\text{iterations} = \text{best_iteration} \times \text{rows}(6-9) / \text{rows}(6-7)$ (Doc 4). Our held-out protocol uses the same rule (fixed iterations scaled by the ratio of fit rows), so the held-out scores evaluate the refit recipe itself, not just the early-stopped model (FACTS C3). Alternatives are to keep the early-stopped model on less data, or to average models trained on different fold subsets.

HOW WE USED IT

US and India decision models: per seed, fit on folds 6-7 with early stopping on fold 8 (at most 3,000 rounds, patience 100), then refit on folds 6-9 with the scaled number of rounds (Doc 4; code). France: fit on 6-7 with early stopping on 8 (at most 1,500 rounds), no refit; default: a single such fit (Doc 4).

EXPERT NUANCE

The refit is part of why we have no untouched end-to-end validation: the final US/India models train on folds 8 and 9, which also helped choose the thresholds and the consistency check (Doc 5). We say that plainly, then point to the mitigation: the held-out scores come from fits that left the scored fold out, using the same iteration rule. The decision to refit the US and India but not France was made on public scores, one of the leaderboard-influenced choices we disclose (Doc C.4).

REMEMBER

- Early stopping on fold 8 finds the trees; the refit on folds 6-9 scales them by $\text{rows}(6-9) / \text{rows}(6-7)$.
- US/India refit kept (public 0.990621 to 0.990676); France refit dropped (0.990726 against 0.990748).
- The refit is one reason there is no untouched end-to-end validation.

JUDGE MIGHT ASK

- *How do you choose the number of trees when you refit without a validation fold?* For each seed we take the best iteration found with early stopping on fold 8 for a model fitted on folds 6-7, and multiply it by the ratio of training rows, folds 6-9 over folds 6-7. Our held-out evaluation uses the same rule, and on the public board the US and India refit moved the score from 0.990621 to 0.990676.

A2.25 Seed averaging

See also [A3.11 Seed averaging](#): seed averaging from the training side.

IN ONE LINE

We train the same decision model five times with different random seeds and average the five probabilities, which smooths out the randomness of each individual fit.

SIMPLE EXPLANATION

Ask five equally trained judges, each of whom happened to study a slightly different random sample of the evidence, and take their average opinion: one judge's odd reading of a case matters less. In our LightGBM settings, each tree sees a random 80 % of the rows and 90 % of the features, so two fits with different **random seeds** (the starting number of the random-number sequence) give slightly different models.

WORKED EXAMPLE (TOY NUMBERS)

One candidate gets probabilities 0.78, 0.74, 0.71, 0.80 and 0.77 from five seeds. Their mean is 0.76, above the 0.75 threshold, so it would be linked if it is also a clear winner. Taken alone, two of the seeds (0.74, 0.71) would have rejected it. Variance view: if each seed's score varies with standard deviation σ and the seeds are correlated with $\rho = 0.5$, the variance of the five-seed mean is $\rho \times \sigma^2 + (1 - \rho) \times \sigma^2 / 5 = 0.5 + 0.1 = 0.6 \times \sigma^2$, a 40 % reduction.

TECHNICAL EXPLANATION

Row bagging (bagging_fraction 0.8, redrawn every iteration) and feature sub-sampling (feature_fraction 0.9 per tree) make each fit a random function of the seed; averaging M fits reduces the seed-induced variance by a factor that depends on their correlation, $\text{Var}(\text{mean}) = \rho \sigma^2 + (1 - \rho) \sigma^2 / M$, and does not reduce bias. Our code averages the five predicted probabilities (code); averaging logits is an alternative with similar effect for well-agreeing models. The cost is five times the training and scoring of the decision model, which is small compared with the readers. Seed averaging is the cheapest form of ensembling; stronger diversity would come from different features or data.

HOW WE USED IT

US, India and France decision models are each the mean of seeds 1-5; the default model is a single fit; the address-rescue scorer averages seeds 7, 8 and 9 (Doc 4). The held-out table uses 3 of the 5 seeds (Doc 5). The five-seed decision models entered the final file in the same upload as the French E5 reader (public 0.991114 to 0.991300), so the effect of the seeds alone is not isolated (QA 6).

EXPERT NUANCE

Seed averaging stabilises borderline decisions, which matters precisely because our thresholds cut through the score distribution; but it cannot fix a systematic error that all seeds share. Compared with averaging logits, averaging probabilities gives less extreme values when seeds disagree near 0 or 1 (toy: seeds at 0.999 and 0.9 average to 0.9495 as probabilities but to 0.9896 via their logits), which can interact with the fixed 0.4 margin; the threshold was chosen on fold 8, and we do not have a separate number for the seeds' effect on it. Do not claim a measured gain from the seeds.

REMEMBER

- Five seeds, same recipe, different random rows and features; average the probabilities.
- Reduces variance, not bias; the default route uses a single fit.
- Effect not isolated: it shipped together with the French E5 reader (0.991114 to 0.991300).

JUDGE MIGHT ASK

- *Why five seeds?* Our LightGBM settings sample 80 % of rows every iteration and 90 % of features, so single fits vary; the mean of five seeds reduces that variance for five small model calls per pair. Honestly, its effect is not isolated: the five-seed models entered the final file together with the French E5 reader, so we do not claim a separate number for them.

A2.26 Calibration: what a model's "probability" means

See also [A3.14 Calibration and reliability](#): calibration and reliability from the validation side.

IN ONE LINE

A model is calibrated if, among all candidates it scores about 0.8, about 80 % are true matches; our decision rule does not assume calibration, because its thresholds are tuned directly on the competition metric.

SIMPLE EXPLANATION

A weather forecaster is **calibrated** if it rains on about 70 % of the days she says "70 %". A forecaster trained in another city, or on a deliberately unusual set of days, may be systematically off. Our decision models were trained on specially chosen rows, and France's model on other countries, so instead of trusting "0.75 means 75 %", we choose the cut-off that scores best on held-out businesses with the real metric. Because the cut-off is chosen this way, any consistent over- or under-confidence is absorbed.

WORKED EXAMPLE (TOY NUMBERS)

1. *Reliability check.* Of 1,000 candidates scored between 0.80 and 0.90, if 850 are owners the model is calibrated there; if only 700 are, it is over-confident.
2. *A threshold does not care about the scale.* Linking when $p \geq 0.75$ is the same decision as linking when $\text{logit} \geq \ln(0.75 / 0.25) = 1.099$, and the same as any other monotone rescaling of the score with the threshold moved accordingly.
3. *The margin does care.* Best candidate 0.95, runner-up 0.60: lead 0.35, below 0.4, so no link. After the monotone rescaling $p \rightarrow p^2$ the scores become 0.9025 and 0.36, a lead of 0.5425: link. Same ranking,

different decision. So whenever a model changes, the threshold and the margin must be re-checked on held-out data.

TECHNICAL EXPLANATION

Calibration is measured with reliability diagrams or expected calibration error; common fixes are **Platt scaling** (a logistic regression on the score) and **isotonic regression**. A log-loss-trained model is roughly calibrated on its own training distribution, but our models do not see the test distribution in training: the readers train on selected hard examples, the decision models train on a training shortlist whose make-up differs from the test files, the France model is applied to a country it never saw, and five seeds are averaged. For a rule of the form "best candidate if score $\geq t$ ", only the ranking and t matter, so tuning t on the exact metric makes calibration unnecessary, provided the miscalibration is the same monotone distortion for every record; if it differs between groups (for example between countries), one shared threshold cannot absorb it, which is why a threshold tuned on US/India folds cannot simply be assumed right for France. The absolute lead requirement (0.4) and the count-dependent consistency-check thresholds are scale-dependent and must be validated together with the model. AUC, a common model metric, measures ranking quality over all pairs; it ignores the per-record competition, the per-business averaging and the full point for a correct empty row, so a model can gain AUC and lose F0.5. Code detail, mention only if asked: when a record has a single candidate, the runner-up is treated as minus infinity, so the lead condition passes automatically.

HOW WE USED IT

We did not calibrate scores separately (no Platt or isotonic step). Operating points (Doc 4): 0.75 for the US and India, chosen on fold 8 for the exact per-business macro F0.5; the per-business consistency check 0.75 / 0.68 / 0.72 / 0.77 for 0 / 1 / 2 / 3 or more other accepted records, chosen on folds 8 and 9; one fixed lead of 0.4 in every country (the Documentation does not record how 0.4 was picked); 0.82 for France and the default route, set without labels from the French link volume per business together with a fold-8 threshold sweep on US/India (Doc C.1).

EXPERT NUANCE

France is where calibration is both unknowable and most important: its decision model was trained on US/India rows and receives a blended French reader input, so we chose the more conservative 0.82 rather than trusting 0.75 to transfer. Because thresholds absorb monotone miscalibration but margins do not, retraining any model in the stack requires re-checking both, which our held-out protocol does for the US and India. A judge may ask "is 0.965 a 96.5 % chance?"; the honest answer is that it is a score whose cut-offs were validated on the metric, not a calibrated probability.

REMEMBER

- Calibrated = scores match observed frequencies; ours are not separately calibrated.
- Thresholds tuned on the exact per-business F0.5 absorb miscalibration; the 0.4 margin does not, so both are validated.
- France uses the conservative 0.82 because its calibration cannot be measured.

JUDGE MIGHT ASK

- *Are your decision-model scores calibrated probabilities?* We do not rely on that and did not calibrate them separately; the thresholds were tuned directly on the exact per-business F0.5 on held-out folds, 0.75 for the US and India on fold 8, which absorbs any consistent over- or under-confidence. France is the exception where we cannot measure it, so it uses the more conservative 0.82, set without labels.
- *Why tune thresholds on the metric rather than on AUC?* AUC measures ranking over all pairs, but the score is per business, a correct empty row is worth a full point and each record gets at most one link, so a model can gain AUC and lose F0.5. That is why the US and India operating points were chosen on the exact metric on held-out folds; France, without labels, uses a label-free choice.

A2.27 Small models for the second-chance searches

IN ONE LINE

The two second-chance searches each use small LightGBM models, a single scorer for the native-script rescue and a cheap filter followed by a careful scorer for the address rescue, with thresholds chosen so that every wrong link counts double.

SIMPLE EXPLANATION

Think of airport security. A quick scanner lets most bags through and flags a few (the **filter**); only flagged bags get a careful manual search (the **scorer**). Because accusing an innocent traveller is costly, the bar for acting is set high. Our second-chance searches work only on records the main pass left unlinked, so they cannot undo a link; their job is to add a few confident links without adding wrong ones.

WORKED EXAMPLE (TOY BUSINESS, THEN TOY THRESHOLD CHOICE, THEN REAL COUNTS)

1. *Why a wrong link hurts more.* A business has 2 true records and currently 1 correct link. Its $F0.5 = 1.25 \times TP / (\text{predicted} + 0.25 \times \text{true}) = 1.25 \times 1 / (1 + 0.5) = 0.833$. Adding a correct link gives $1.25 \times 2 / (2 + 0.5) = 1.0$, a gain of +0.167. Adding a wrong link gives $1.25 \times 1 / (2 + 0.5) = 0.5$, a loss of -0.333. Counting the wrong link twice (predicted = 3) gives $1.25 / 3.5 = 0.357$, a loss of -0.476.
2. *Choosing between two thresholds.* Threshold A adds 90 right and 10 wrong links; threshold B adds 110 right and 20 wrong. With the gains above, $A = 90 \times 0.167 - 10 \times 0.333 = 11.67$ and $B = 110 \times 0.167 - 20 \times 0.333 = 11.67$, a tie. With wrong links counted twice, $A = 15.0 - 4.76 = 10.24$ and $B = 18.33 - 9.52 = 8.81$, so the cautious threshold A wins.
3. *Real funnel (Doc 3, B.1).* Address rescue: US 3,029,652 retrieved pairs, 1,827 survive the filter, +192 links; India 4,766,484, 25,579 survivors, +2,945 links. Native-script rescue: 649,965 pairs scored for 171,713 records, +9,759 links. In the India example of Doc D.2, the rescue model gave the business with the same house number 24/11 a score of 0.99994 against 0.0000191 for the next.

TECHNICAL EXPLANATION

A **cascade** applies a cheap model to many candidates and an expensive one to the few survivors, the same logic as our main funnel. The address-rescue filter is a LightGBM model with 10 features, 15 leaves and 200 rounds; it keeps at most 2 pairs per record with probability at least a filter cut-off, the largest grid value that keeps 98 % of the fold-8 reference links. The scorer has 41 features, 31 leaves and 300 rounds, averaged over seeds 7, 8 and 9; it links the best kept pair if its score is at least the threshold and leads the second kept pair by at least the lead (Doc 3, 4). The native-script rescue model is a LightGBM model trained on India rescue candidates of train folds 0-7, retrieved with word lists that leave out each record's own fold, a form of **cross-fitting** so a record never benefits from its own label; it links the best of its top 5 if the score is at least 0.88 and the lead at least 0.3 (Doc 3, 4). Thresholds were chosen on fold 8 by the exact per-business F0.5 gain with each false link weighted 2, which in the code means a wrong link adds 2 to its business's predicted count (code): a **cost-sensitive** decision rule.

HOW WE USED IT

Operating points (Doc 4): native-script rescue 0.88 with lead 0.3; address rescue US filter cut-off 0.06, threshold 0.86, lead 0.0; India filter cut-off 0.015, threshold 0.66, lead 0.1. The address models are trained per country on address-retrieval pairs of train records that the main decision leaves unlinked, folds 6-7 (Doc 4). Order of refinements for the US and India: native-script rescue, then the per-business consistency check, then the address rescue (Doc 2.2).

EXPERT NUANCE

Counting false positives twice is a deliberate safety margin: these models act on the hard leftovers, where the training folds may not represent the test well, and under F0.5 a wrong link can already cost twice what a right link gains (the toy business of step 1). These searches add few links (+192 in the US), so the fold-8 cases behind each threshold are probably few as well (reasoning; the Documentation gives no fold-8 count), and a threshold estimated from few cases is another argument for caution. Only if pressed: the second-chance models' training records were selected with earlier decision models that had inputs we later removed, and we disclose it (Doc C.8).

REMEMBER

- Native-script rescue: one LightGBM scorer, 0.88 and lead 0.3, +9,759 India links.
- Address rescue: filter (10 features) then scorer (41 features, 3 seeds); +192 US, +2,945 India links.
- Thresholds on fold 8 with wrong links counted twice; they act only on unlinked records.

JUDGE MIGHT ASK

- *Why count false positives twice when choosing the second-chance thresholds?* Under F0.5 a wrong link already costs more than a right link gains, and records reaching a second-chance search are the hard leftovers, so we added a safety margin: on fold 8 we picked the threshold that maximised the exact per-business F0.5 with every wrong link counted twice. That gave 0.88 for the native-script rescue, and 0.86 and 0.66 for the US and Indian address rescue.

Numbers to memorise for this part

Item	Value	Source
CatBoost ranker	Logloss; up to 1,600 trees; depth 7; learning rate 0.075; L2 5; early stopping 180 on fold 6; GPU; 76 features; 140,000-record pilot (fold 0-5 part)	Doc 4
Ranker workload on test	37,614,782 pairs after the quick filter	Doc 3
Shortlist	top 1 always, ranks 2-4 if probability at least 0.001; 11,687,317 pairs; 1.17 per record (derived)	Doc 3
Recall, fold 9	US index 99.3694 %, shortlist 98.8748 % to 99.3628 %; India 98.5068 %, 98.0572 % to 98.4831 %	Doc B.2
MiniLM	paraphrase-multilingual-MiniLM-L12-v2; 117,653,760 parameters (+385 head); Apache-2.0	Doc B.5
E5	multilingual-e5-base; 278,043,648 parameters (+769 head); MIT	Doc B.5
Reader recipe	raw "name address"; 128 tokens; masked mean pooling; dropout 0.1; BCE; AdamW (decay 0.01, head x5); 3 % warm-up; bf16; clipping 1.0; one epoch; batch 128	Doc 4
Reader learning rates	5e-5 (MiniLM, E5 readers); 2e-5 (French readers)	Doc 4
French E5 reader data	474,808 French + 474,808 US/India pairs; 7,419 steps	Doc 4
French blend	0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5 (logits)	Doc 4
Decision models	LightGBM; learning rate 0.05; 127 leaves; 200 rows per leaf; feature fraction 0.9; bagging 0.8; L2 1.0; mean of seeds 1-5	Doc 4
Features	US 217; India 211; France and default 158	Doc 4
Thresholds	0.75 US/India (fold 8); 0.82 France/default; lead 0.4 everywhere	Doc 4
Held-out F0.5 (fold 9 / fold 8)	US 0.991212 / 0.991545; India 0.990046 / 0.990298	Doc 5

Item	Value	Source
Public F0.5	0.991300 ("0.9913")	Doc 5
Larger reader tried	XLN-R large, about 560 million parameters, discarded	Doc B.5

A3 - Concepts: training, validation and selection

HOW TO READ THIS PART

This part explains how our models were trained, how we checked them without fooling ourselves, and how we decided what to keep in the final file. The concepts run from foundational to advanced, so read them in order the first time; later concepts point back to earlier ones by name (for example "see Early stopping"). Every concept has the same eight blocks: a one-line summary, a simple explanation with an everyday analogy, a worked example you can follow with a pen, the technical explanation a senior scientist expects, exactly how we used it, the expert nuance (trade-offs, traps, alternatives), three things to remember, and one or two questions a judge might ask with a short spoken answer. In the worked examples, numbers marked **(toy)** are invented only to teach the idea and must never be quoted as results; numbers marked **(ours)** come from our Documentation, READMEs or code and can be quoted; "derived" means simple arithmetic on our numbers. A term is printed in **bold** where it is first defined. When you answer aloud: say a limitation plainly first and then the mitigation, quote numbers exactly, and use the plain names of the talk.

Words used throughout this part (defined in earlier parts, repeated here so you can start anywhere).

Word	Meaning
Business	A record of the reference list (Source 1). The test files hold 1,732,544 of them.
Incoming record	A Source 2 or Source 3 record that must be linked to one business or to none. The test files hold 9,969,589 of them.
Label	The known right answer for a training example, here "this incoming record belongs to this business". The US and India have labels in the training files; France has none.
Candidate	A business that our search proposes for an incoming record.
Shortlist	Up to 4 candidates per incoming record kept by the CatBoost ranker; the only pairs the readers and decision models score.
Readers	Cross-encoders (MiniLM reader, E5 reader, French MiniLM reader, French E5 reader): neural networks that read the record and the candidate together and output one raw score, a logit . A logit can be any number; large positive means "very likely the same business", large negative "very unlikely", 0 "no idea".
Sigmoid	The function that turns a logit x into a probability: $\text{sigmoid}(x) = 1 / (1 + e^{(-x)})$. For example $\text{sigmoid}(0) = 0.5$, $\text{sigmoid}(2) \approx 0.88$, $\text{sigmoid}(-2) \approx 0.12$.
Gradient-boosted trees (GBDT)	A model made of many small decision trees added one at a time, each new tree correcting the errors of the trees before it (technically, each tree is fitted to the gradient of the loss with respect to the current predictions, hence the name). Each added tree is one boosting round (or iteration); the learning rate is how big a step each new tree is allowed to take. Our CatBoost ranker and our LightGBM decision models are both of this kind.
Decision model	The LightGBM model, one per country, that turns reader scores and about two hundred features into a probability that the candidate is the right business.
TP, FP, FN	For links: a true positive (TP) is a correct link, a false positive (FP) a wrong link, a false negative (FN) a true link we failed to make.
Precision and recall	Precision = $TP / (TP + FP)$, the share of the links we made that are correct. Recall = $TP / (TP + FN)$, the share of the true links that we found. Raising a threshold usually raises precision and lowers recall.
Per-business F0.5	The competition metric. For each business: $F0.5 = 1.25 \times TP / (\text{predicted} + 0.25 \times \text{true})$, where TP = correct links, predicted = links we made to it, true = incoming records that really belong to it; the score is 1 if predicted and true are both 0. The final score is the plain average over all businesses (a macro average).
AUC	Area under the ROC curve: the probability that a randomly chosen true pair gets a higher score than a randomly chosen false pair. 1.0 is perfect ranking, 0.5 is random. It ignores any threshold.
Top-1 accuracy	The share of incoming records whose highest-scored candidate is the right business.

Word	Meaning
Epoch	One full pass of a neural network over its training examples.

CONCEPTS IN THIS PART, IN READING ORDER

- *Honest evaluation, the foundations*: 1. Training, validation and test data; 2. Cross-validation and k folds; 3. Data leakage.
- *Fold design*: 4. Group k-fold: folds by business; 5. Hashing ids into folds; 6. Layer-wise fold separation for stacked models; 7. Cross-fitting; 8. The held-out evaluation protocol.
- *Training procedures*: 9. Early stopping; 10. Refitting on more folds with scaled iterations; 11. Seed averaging.
- *Thresholds and decisions*: 12. Tuning thresholds on held-out folds; 13. Cost-sensitive threshold selection; 14. Calibration and reliability; 15. The confident + clear-winner rule; 16. Count-dependent thresholds: the per-business consistency check.
- *Statistical evidence*: 17. Paired bootstrap intervals.
- *Where training data comes from*: 18. Hard examples and hard negatives; 19. Synthetic training pairs; 20. Self-training with pseudo-labels; 21. Replay and catastrophic forgetting.
- *France and new countries*: 22. Blending readers in logit space; 23. Weak supervision and rules computed by code; 24. Domain shift and unseen countries.
- *Selection, checks and reproducibility*: 25. Model selection and public-leaderboard overfitting; 26. Written acceptance checks before keeping a model; 27. Validation hygiene: recall at every stage, assertions and the validator; 28. Determinism and byte-exact reproducibility.
- At the end: a quick-reference table of every number used in this part.

A3.1 Training, validation and test data

IN ONE LINE

A model must be graded on data it did not learn from, so labelled data is given three separate jobs: learning, making choices, and grading.

SIMPLE EXPLANATION

Think of a student preparing for an exam. The textbook exercises are the **training data**: the student practises on them and sees the answers. A mock exam is the **validation data**: the teacher uses it to make choices, such as how many more weeks to practise or which study method works better. The real exam is the **test data**: its score means something only if the student has never seen those questions. A student who memorises the textbook answers can score perfectly on the exercises and still fail the exam; in machine learning this is called **overfitting**, doing well on what you studied and badly on anything new. The ability to do well on new data is called **generalisation**, and it is the only thing we really care about. Almost every concept in this part is a way of keeping these three jobs apart.

WORKED EXAMPLE

(toy) Suppose we have 1,000 labelled businesses with their incoming records and split them 600 / 200 / 200.

Step	Data used	What happens	Score (toy)
1. Fit the model	600 training businesses	the model learns from their labels	F0.5 0.999 on these same businesses
2. Choose the threshold	200 validation businesses	try 0.6, 0.7, 0.8; keep the best	F0.5 0.990 at the best threshold

Step	Data used	What happens	Score (toy)
3. Grade once	200 test businesses	apply the chosen model and threshold	F0.5 0.989

Read the gaps. The drop from 0.999 to 0.990 is overfitting: the model partly memorised its training businesses. The small drop from 0.990 to 0.989 is **selection optimism**: the threshold was picked because it looked best on the validation businesses, so part of its advantage was luck on those particular businesses. Only the test score (0.989) is an honest estimate, and only if we look at it once.

(ours) How the three jobs map onto our project:

Job	In our project
Training	folds 0-5 for the CatBoost ranker and the readers; folds 6-7 (then 6-9 for US/India) for the decision models
Validation (making choices)	fold 6 to stop the ranker; fold 8 to stop the decision models and to choose the 0.75 threshold; folds 8 and 9 to choose the per-business consistency check
Test (grading)	the hidden competition labels, seen only through the public leaderboard score of each upload

TECHNICAL EXPLANATION

What we want is the **expected loss** (or expected metric) of a model f on new data drawn from the deployment distribution, E over (x, y) of $L(f(x), y)$. Any finite sample gives an estimate, and the estimate is unbiased only if the sample is independent of every choice that produced f . Training data fails this badly because the parameters were fitted to it (training error is optimistically biased, more so for high-capacity models). Validation data fails it mildly once it has been used to choose **hyperparameters** (settings chosen by the modeller, such as the threshold, the number of trees or the learning rate, as opposed to **parameters** fitted by the algorithm, such as tree splits or network weights); the optimism grows with the number of options tried and with the noise of the estimate. A test set is unbiased only for its first use; every later decision taken by looking at it turns it into a validation set (see Model selection and public-leaderboard overfitting). Independence must hold at the level of the sampling unit: if records are clustered (several records per business), the independent unit is the cluster, not the record (see Group k-fold: folds by business). The usual remedies are **regularisation** (any constraint that stops a model fitting noise, such as a penalty on large weights, a minimum number of rows per tree leaf, or random subsampling) and early stopping against overfitting, held-out data and cross-validation for selection, and a locked final test set for grading.

HOW WE USED IT

All of our labelled data are the US and Indian training files; France has no labels at all. We split the labelled businesses into 10 folds (see Cross-validation and k folds) and gave the folds fixed jobs, as in the table above. Our main internal evidence is the per-business F0.5 on folds 8 and 9 scored by models that did not train on them: US 0.991212 (fold 9) / 0.991545 (fold 8) and India 0.990046 / 0.990298, from close versions of our decision models (the India recipe with 3 of its 5 seeds). Our Documentation states the limitation plainly: there is no untouched end-to-end validation of the final file, because folds 8 and 9 also helped choose the thresholds and the consistency check, the final US/India decision models were refitted on folds 6-9, the held-out scores exclude the second-chance searches and the France rules, and France has no labels.

EXPERT NUANCE

"Test" is a role, not a dataset: a fold becomes validation the moment a decision depends on it, and the public leaderboard becomes validation when you pick files by it. The honest description of our setup is therefore "train, then several validation sets of increasing reuse", not "train, validation, test". The ideal fix, which is the first thing we say we would do differently, is to lock one fold on day one, run the whole

pipeline on it (including the second-chance searches) and score it exactly once at the end. A common misconception is that a large validation set protects you from selection optimism; size reduces noise, but repeated reuse still leaks information into your choices.

REMEMBER

- Training score shows fitting; validation score is used for choices; only an unused test set grades honestly.
- Our evidence is held-out folds 8 and 9 (US 0.991212 / 0.991545, India 0.990046 / 0.990298), in the same range as the public 0.9913.
- Limitation, said first: no untouched end-to-end validation of the final file.

JUDGE MIGHT ASK

- **Q:** "What was your untouched test set?"
A: We did not have one for the final file, and we say so in our documentation. Our main evidence is held-out folds 8 and 9: models fitted without those businesses score US 0.991212 and 0.991545 and India 0.990046 and 0.990298, the same range as the public 0.9913, but those folds also helped choose the thresholds.

A3.2 Cross-validation and k folds

IN ONE LINE

Cut the labelled data into k equal parts, called **folds**, and let the parts take turns at being the unseen part, or give each part a fixed job.

SIMPLE EXPLANATION

Imagine a cookery contest with ten judges where you want to test a recipe fairly. You let nine judges help you tune the recipe and keep the tenth blindfolded until the end; then you repeat with a different judge kept aside each time. Every judge is used for testing once and for tuning nine times, so the final verdict does not depend on one lucky or unlucky judge. That rotation is **k-fold cross-validation**. A second use of the same folds is simply bookkeeping: you can say "folds 0 to 5 are for learning, fold 8 is for choosing settings, fold 9 is for reporting", so that every piece of data has a clear job and never does two jobs at once.

WORKED EXAMPLE

(toy) Ten businesses, B0 to B9, and k = 5 folds of two businesses each: fold 0 = {B0, B5}, fold 1 = {B1, B6}, fold 2 = {B2, B7}, fold 3 = {B3, B8}, fold 4 = {B4, B9}.

Round	Train on folds	Test on fold	Score (toy)
1	1, 2, 3, 4	0	0.986
2	0, 2, 3, 4	1	0.991
3	0, 1, 3, 4	2	0.989
4	0, 1, 2, 4	3	0.990
5	0, 1, 2, 3	4	0.988

Mean = $(0.986 + 0.991 + 0.989 + 0.990 + 0.988) / 5 = 0.9888$. The sample standard deviation across folds is about 0.0019, which tells you how much a single split could have misled you.

(ours) Our 10 folds with fixed jobs:

Folds	Job
0-5	train the CatBoost ranker (a 140,000-record pilot, its fold 0-5 part), the MiniLM and E5 readers, and the Indic-to-Latin word list used in cleaning
6	early stopping of the CatBoost ranker
6-7	fit the decision models (and the address second-chance models)
8	early stopping of the decision models; choice of the 0.75 threshold and the second-chance thresholds
8 and 9	choice of the per-business consistency check; held-out scoring
6-9	final refit of the US and India decision models
0-7	the native-script second-chance model, trained once on these folds; the word lists that build its training examples leave out each record's own fold (see Cross-fitting)

TECHNICAL EXPLANATION

k-fold cross-validation estimates the expected performance of a *training procedure*, not of one fitted model: each of the k models trains on $(k - 1)/k$ of the data, so the estimate is slightly pessimistic for a model trained on all of it, and its variance comes both from the test folds and from the overlapping training sets. Because the training sets overlap, the k fold scores are correlated, and there is no unbiased estimator of the variance of the k-fold estimate (a known result of Bengio and Grandvalet, 2004), so the fold standard deviation is only a rough guide. Variants: **stratified** k-fold keeps the class ratio equal in every fold; **group** k-fold keeps every member of a group in one fold (see Group k-fold: folds by business); **time-series** splits never train on the future. When the same folds are used both to tune and to report, the report is optimistic; **nested cross-validation** fixes this with an inner loop for tuning inside an outer loop for reporting, at $k \times k'$ times the cost. Cross-validation also produces **out-of-fold (OOF) predictions**: every example scored by the model that did not see it, which is how stacked models are usually trained (see Layer-wise fold separation for stacked models). The cost is k full trainings; typical k is 5 or 10.

HOW WE USED IT

Ten folds assigned per business by a hash of the business id (see Hashing ids into folds). We did not train ten copies of every model and average them; instead the folds are a role system, as in the table above, because our pipeline has several learned layers and the readers are expensive to train. For honest scoring we used a two-way swap: fold 9 is scored by fits on folds 6-8 and fold 8 by fits on folds 6, 7 and 9 (see The held-out evaluation protocol). Fold rotation appears in one cheap place: the word lists used to build the native-script second-chance training examples are rebuilt once per fold, each leaving that fold out (see Cross-fitting); the second-chance model itself is trained once.

EXPERT NUANCE

The role system buys clean separation between layers at a price: each layer learns from only part of the labels (the readers from 6 of 10 folds, the ranker from a 140,000-record sample of those same 6 folds, the decision models from 4). Our held-out estimate rests on two folds rather than ten, so it is noisier than a full cross-validation, which is why the paired bootstrap matters (see Paired bootstrap intervals). And because fold 8 chose the threshold and folds 8 and 9 chose the consistency check, our setup is closer to "validation reused for reporting" than to nested cross-validation; we disclose that. A full nested scheme would multiply the training of every learned layer, cross-encoders included; we did not attempt it on one GPU in the competition window.

REMEMBER

- k-fold = every example is tested once by a model that did not see it; the average is steadier than one split.
- Our 10 folds are mainly a job system: 0-5 base models, 6-7 decision-model fit, 8 stopping and thresholds, 9 reporting (and, with fold 8, choosing the consistency check).

- Cross-validation estimates a procedure, and reusing folds for tuning makes the estimate optimistic.

JUDGE MIGHT ASK

- **Q:** "Why didn't you just run standard 10-fold cross-validation for every model?"
A: Because the pipeline has layers, and each layer must learn from scores produced on data the layer below never saw; giving folds fixed jobs (0-5 for the ranker and readers, 6-9 for the decision models) does that with one training per model instead of ten, which mattered with cross-encoders on a single GPU. We rotated folds only where it was cheap: the word lists behind the native-script second-chance search leave out each training record's own fold.

A3.3 Data leakage

IN ONE LINE

Leakage is any route by which information about the answers, or about the evaluation data, reaches training and makes a model look better than it really is.

SIMPLE EXPLANATION

A student who has secretly seen the exam paper gets a great score, but the score says nothing about what the student knows. Leakage is any way the "exam paper" reaches the model. Sometimes it is obvious, like testing on the training data. More often it is sneaky: two records of the same business end up on both sides of the split; a feature is computed with help from the labels; one model's over-confident score on its own training data is fed to another model; or settings are chosen by looking at test results again and again. The symptom is always the same: an excellent validation score followed by a disappointing real score. Most of the fold design in this part exists to block one leakage route or another.

WORKED EXAMPLE

Six leakage routes, each with a tiny illustration and our defence.

Route	(toy) illustration	Our defence (ours)
Group leakage	"Sharma Traders, 12 MG Road, Pune" owns three incoming records; two land in training, one in validation, so the validation pair looks familiar	folds assigned per business; every record goes with its business
Feature (target) leakage	a feature "number of records truly linked to this business" uses the answers	features use only text, model scores and model decisions; the collective features count records whose ranker top choice is that business, not true links
Stacking leakage	the reader trained on pair P scores P at 0.9999; a decision model trained on P learns "0.9999 means match" and is fooled later	ranker and readers learn on folds 0-5, decision models on folds 6-9
Learned-resource leakage	a word list learned from all labelled pairs already knows the validation record's own spelling	cleaning word list learned from folds 0-5 only; the second-chance word lists leave out the record's own fold
Selection leakage	trying 13 thresholds on fold 8 and reporting fold 8's best	fold 9 reported too; disclosure that folds 8 and 9 also chose settings
Test-feedback leakage	uploading 18 files and keeping the best public score	held-out folds as main evidence; disclosure of every public-score decision

TECHNICAL EXPLANATION

An evaluation estimate is unbiased only if the evaluation data are independent of every fitted object: parameters, preprocessing statistics, learned vocabularies, features, hyperparameters and the final model choice. The standard taxonomy: **target leakage** (a feature is a function of the label or of information that only exists after the outcome); **train-test contamination** (duplicates, near-duplicates or members of the

same group on both sides); **preprocessing leakage** (scalers, encoders or vocabularies fitted on data that include the evaluation labels); **temporal leakage** (training on the future); **stacking leakage** (in-sample base-model predictions used as meta-features); and **adaptive leakage** (repeated queries of a holdout). Symptoms include implausibly high scores, a single feature dominating importance, and a validation score that collapses when a suspect input is removed. Detection tools are ablation of suspect inputs, **adversarial validation** (train a classifier to tell training rows from evaluation rows; if it succeeds, the two differ), and re-deriving every learned resource inside the training folds only. One important distinction: computing **label-free** statistics on the test inputs (word frequencies, an index) is **transductive** learning, not label leakage; it is legitimate when allowed and disclosed, but it means the pipeline must recompute those statistics on every new batch.

HOW WE USED IT

The defences in the table are all in the code. Two more details matter. First, the CatBoost ranker's training mask requires both the incoming record and the candidate business to be in folds 0-5, and its early-stopping rows have the record in fold 6 and the candidate in folds 0-6; so no business from folds 7-9 ever appears in ranker training. Second, when we score a fold for evidence, we also drop from training every incoming record that has any candidate in that fold (see The held-out evaluation protocol). Disclosed transductive uses (Documentation, Appendix C): IDF (inverse document frequency, a word weight that is high for rare words and low for common ones) and other token statistics, the search indexes and the competition features are computed on the unlabelled test files; for India a native-script-to-Latin token list is aligned on test records whose first-stage top candidate has probability at least 0.9; for France the geography, the street-type typo table and postcode prefixes are learned from the test files; the two French readers are self-trained on French test pairs; label-free counts on the test records set the size of the extra hard training examples, the France threshold and the French MiniLM reader's blend weight. No test label is used anywhere.

EXPERT NUANCE

The most dangerous leakage in a competition is not a bug in a feature; it is adaptive leakage through many leaderboard uploads, because it is invisible in any single experiment (see Model selection and public-leaderboard overfitting). The second most common is group leakage, which inflates scores in entity resolution precisely because records of one entity share text. A subtle case in our pipeline is that fold 8 both stopped the decision models and chose the threshold, so fold-8 scores carry a little selection optimism; this is why we report fold 9 next to it and why the Documentation says the folds were reused. Transductive statistics are not leakage of labels, but a judge may still probe them, so name them precisely.

REMEMBER

- Leakage = the answers or the evaluation data reaching training by any route; the result is a score that will not hold.
- Our defences: business folds, separate folds per model layer, word lists learned without the evaluated folds, removal of records touching the scored fold.
- Unsupervised statistics on the unlabelled test files are transductive, allowed and disclosed; no test label is used.

JUDGE MIGHT ASK

- **Q**

"Where could leakage hide in your pipeline, and how did you block it?"

A: In four places, and three of them are blocked by design. Records of one business on both sides of a split are blocked by folds by business; over-confident scores passed from one model to the next are blocked by training the ranker and readers on folds 0-5 and the decision models on folds 6-9; and learned word lists are built without the evaluated folds. The fourth, reuse of folds 8 and 9 and of public scores for choosing settings, we cannot remove, so we disclose it.

- **Q**

"You compute IDF and the search index on the test files. Isn't that leakage?"

A: No labels are involved: those are unsupervised statistics on the test inputs, which makes the pipeline transductive, and our documentation lists every such use. The practical consequence is that a new batch of data needs those statistics recomputed, which our inference stages already do.

A3.4 Group k-fold: folds by business

IN ONE LINE

Every incoming record must sit in the same fold as its business, so the models are always tested on businesses they have never seen.

SIMPLE EXPLANATION

Imagine teaching a child to recognise relatives from photographs. If you test with a different photo of the same uncle the child practised on, the child may simply remember that uncle's face rather than learn a general skill. Our "uncles" are businesses: each business owns several incoming records (about 5.75 on average in the test files). If records were split at random, some records of a business would be in training and others in testing, and the model could lean on memory of that particular business, its name, its address, its competitors. **Group k-fold** keeps the whole family together: a business and all its records go into the same fold. Then a good test score really means "works on new businesses", which is what the competition asks.

WORKED EXAMPLE

(toy) A training business "Sri Balaji Textiles, 7 Station Road, Salem" owns three incoming records:

Record	Text
R1	Sri Balaji Textiles Pvt Ltd, 7 Station Rd, Salem
R2	Balaji Textiles, Station Road, Salem
R3	श्री बालाजी टेक्सटाइल्स, Salem

Record-level random split: R1 and R2 go to training, R3 to validation. During training the decision model sees (R1, Sri Balaji Textiles) = match and (R2, Sri Balaji Textiles) = match, along with the same competing look-alike businesses in Salem. At validation, the pair (R3, Sri Balaji Textiles) shares the business text, the same competitors and the same "other records competing for this business"; the model has effectively been shown the answer. The validation score goes up, but nothing about the model improved.

Business-level split: the business and R1, R2, R3 all get the same fold, say fold 4. If fold 4 is the held-out fold, none of the three records and none of the business's pairs was seen in training.

Two more reasons in our pipeline (ours): the metric is computed per business, so a business must be scored with all of its records together; and the collective features and the per-business consistency check

look at all incoming records competing for the same business, which only makes sense if they share a fold.

TECHNICAL EXPLANATION

Standard cross-validation assumes the rows are independent and identically distributed (**i.i.d.**). Clustered data break this: rows within a group are correlated (high **intra-class correlation**), so the effective sample size is closer to the number of groups than to the number of rows, and a record-level split lets the model exploit group identity, which will not be available for new groups. **Group k-fold** assigns folds to groups, never splitting a group; the estimate then targets "performance on new groups". The leak is largest when (a) records of a group are similar to each other and (b) the model has the capacity to memorise group-specific signals, both true for gradient-boosted trees over name and address features. (**Intra-class correlation** is the share of the variation in the data that comes from differences between groups rather than within them; when it is high, a second record of the same business adds little new information.) In our code, an owned incoming record inherits its business's fold; an unowned record (one that belongs to no business) gets a fold by hashing its own id. The decision-model rows are pairs (record, candidate business), assigned by the candidate business's fold, and the held-out protocol additionally removes records whose candidates span folds (see The held-out evaluation protocol).

HOW WE USED IT

Ten folds per business, $\text{fold} = \text{hash}(\text{business id}) \bmod 10$ (see Hashing ids into folds). Every training and evaluation step uses these folds. The sizes came out balanced: in held-out scoring, fold 9 had 132,365 US and 87,778 Indian businesses, fold 8 had 132,407 and 88,103 (ours).

EXPERT NUANCE

Grouping by business is necessary but may not be sufficient. Near-identical businesses (for example branches of one chain, or two businesses with the same name in one city) can fall into different folds and still share text; a stricter scheme would group by connected components of similar names, giving a more pessimistic and arguably more realistic estimate. We did not do that. Also, unowned incoming records can be candidates for businesses in several folds, which is why the held-out protocol removes every record that touches the scored fold. We did not measure how much a record-level split would have inflated our score; to measure it, we would score the same model under both splits and compare.

REMEMBER

- The business is the unit of the metric and of the collective features, so it must be the unit of the split.
- A record-level split would put records of one business on both sides and inflate the score.
- Fold sizes came out balanced: fold 9 US 132,365 vs fold 8 US 132,407.

JUDGE MIGHT ASK

- **Q:** "How much would a record-level split have inflated your score?"
A: We did not measure that, so I will not guess a number. The reason we avoided it is structural: one business owns several records, the metric is per business, and the collective features use all records competing for a business; to quantify the inflation we would score the same model under a record split and a business split and compare.

A3.5 Hashing ids into folds

IN ONE LINE

Compute $\text{fold} = \text{hash}(\text{business id}) \bmod 10$, so every machine and every run puts each business in the same fold without storing a random draw.

SIMPLE EXPLANATION

A **hash function** is like a very thorough blender for text: you put in an id such as "S1-158550741" and it outputs a huge number that looks random, but the same id always gives the same number. Taking the remainder after dividing by 10 (written "mod 10") gives a fold from 0 to 9. Because the blender mixes so well, about one tenth of the businesses land in each fold. Because the id alone decides the fold, any record that belongs to the business can simply take the business's fold, so the group stays together automatically. And because nothing random is drawn, anyone who runs the code later gets exactly the same folds.

WORKED EXAMPLE

(toy hash values; the business ids are the real ones from our US worked example, but the folds shown are invented and are not their real folds)

Business id	Hash value (toy)	Last digit = hash mod 10	Fold
S1-158550741	9,214,307,552,118,406,233	3	3
S1-734720318	4,551,092,887,310,045,679	9	9
S1-44256287	12,008,553,190,276,431,500	0	0

- An incoming record owned by S1-158550741 gets fold 3 too.
- An unowned incoming record hashes its own id instead.
- A different seed (we used 20260927) gives a completely different but equally random-looking assignment.

(ours) Balance check: fold 9 had 132,365 US businesses and fold 8 had 132,407, a difference of 42 (derived); for India 87,778 and 88,103, a difference of 325 (derived). How big a difference is normal? If each business lands in a fold with probability 0.1, a fold's size has a standard deviation of $\sqrt{(\text{fold size} \times 0.9)}$, about $\sqrt{(132,000 \times 0.9)} \approx 345$ for the US, and the difference between two folds has a standard deviation of about $\sqrt{(2 \times \text{fold size})}$: $\sqrt{(2 \times 132,000)} \approx 514$ for the US and $\sqrt{(2 \times 88,000)} \approx 420$ for India (derived). Differences of 42 and 325 are well inside that, exactly what a good hash should give.

TECHNICAL EXPLANATION

Hash partitioning assigns $\text{fold}(\text{id}) = H_{\text{seed}}(\text{id}) \bmod k$. For a well-mixed 64-bit hash the outputs are approximately uniform and independent of any structure in the ids (sequential ids, prefixes), so the fold sizes are jointly multinomial: each is approximately $\text{Binomial}(N, 1/k)$ with standard deviation $\sqrt{(N \times (1/k) \times (1 - 1/k))}$, and the difference of two folds has variance $2N/k$ (the negative covariance between folds adds to it). **Modulo bias** (some remainders being slightly more likely because 2^{64} is not a multiple of k) is negligible because 2^{64} is vastly larger than k . Advantages over shuffling: **stateless** (no stored permutation), **deterministic** across processes and machines, trivially parallel, **stable under growth** (adding new businesses never moves old ones, while a reshuffle would), and **group-consistent** if you hash the group id. Limitations: it is not stratified (class balance is equal only in expectation, which is fine at our sizes), and the result depends on the exact hash implementation. Non-cryptographic hashes are adequate; if stability across library versions matters, a cryptographic hash such as SHA-256 of the id is a portable alternative.

HOW WE USED IT

fold = hash of the business id with seed 20260927, mod 10, computed with Polars (the data-frame library our pipeline uses). An owned record inherits its business's fold; an unowned record hashes its own id. Our fold-building code writes a manifest with the seed, the Polars version and a SHA-256 digest (a fingerprint of the file: change one byte and the digest changes completely) of every fold file, because Polars documents that its hash is stable only within one library version.

EXPERT NUANCE

The seed must be fixed before any experiment; choosing the seed whose split gives the best validation score would be selection leakage in disguise. Hash stability is an easy trap: upgrading a data library can silently change every fold and make old and new experiments incomparable, which is why the manifest records the version and the file hashes. A stricter alternative is to store the fold table itself as a versioned file (we do both: the fold files and their digests).

REMEMBER

- $\text{fold} = \text{hash}(\text{business id}, \text{seed } 20260927) \bmod 10$; records inherit their business's fold.
- Hashing is deterministic, stateless, group-preserving and stable when new data arrive.
- Library hash functions can change between versions; record the version and file hashes.

JUDGE MIGHT ASK

- **Q:** "Why a hash rather than a random shuffle?"
A: A hash of the business id gives the same folds on every machine and every run without storing anything, keeps each business's records together automatically, and does not move existing businesses when new ones arrive. The folds came out balanced, for example 132,365 and 132,407 US businesses in folds 9 and 8.

A3.6 Layer-wise fold separation for stacked models

See also [A2.23 Stacking: a two-level model with separate folds per layer](#): stacking from the models side.

IN ONE LINE

When one model's scores are inputs to another model, the second model must learn from scores that the first model produced on data it never trained on.

SIMPLE EXPLANATION

Imagine a manager who decides promotions from reports written by team leads. If team leads write about their own favourite projects, every report is glowing; a manager who learns "a glowing report means promote" from such reports will be fooled by over-optimistic reports later. Our CatBoost ranker and readers are the team leads, and the LightGBM decision model is the manager. A reader that has trained on a pair has partly memorised it, so its score on that pair is over-confident. If the decision model learned from such scores, it would learn to over-trust the reader. So the ranker and readers learn on folds 0-5, and the decision models learn on folds 6-9, where every ranker and reader score is a genuine "first sight" score, just like on the test data.

WORKED EXAMPLE

(toy) A genuinely hard true match: record "Laxmi Agency, Coimbatore" and business "Sri Lakshmi Agencies, Coimbatore".

Setting	Reader logit on this kind of pair (toy)	Probability
The reader trained on this exact pair (in-sample)	+9.0	0.9999
A similar pair the reader never saw (out-of-sample)	+1.5	0.82

If the decision model trains on in-sample scores, nearly every true pair it sees has a logit of +8 or more, so it learns "below +4 means not a match". On the test data, where hard true pairs come in at +1.5, it rejects them. Trained on out-of-sample scores, it sees hard true pairs at +1.5 during training and learns to combine that moderate score with the other features.

(ours) Which folds each layer used:

Layer	Fit on	Stopping or tuning on
Indic-to-Latin word list used in cleaning	folds 0-5	none
CatBoost ranker	folds 0-5 (a 140,000-record pilot, both record and candidate in folds 0-5)	fold 6
MiniLM and E5 readers	folds 0-5	none (one epoch)
US and India decision models	folds 6-7, then refit on 6-9	fold 8
France and default decision models	folds 6-7	fold 8
Thresholds	none	0.75 on fold 8; consistency check on folds 8 and 9

TECHNICAL EXPLANATION

Stacked generalisation (Wolpert, 1992), or **stacking**, trains a **meta-model** (level 1) on the predictions of **base models** (level 0). The meta-features must be out-of-sample, because in-sample predictions have systematically smaller error and higher confidence than predictions on new data; a meta-model trained on them learns the wrong reliability for each base model. There are two standard ways. (a) **Out-of-fold stacking**: train k copies of each base model, each predicting the fold it did not see, so the meta-model can use all labelled rows; at test time a base model refitted on all data is used, which creates a small train-test mismatch, and the cost is k trainings per base model. (b) **Blending, or a layer-wise split**: give the base models and the meta-model disjoint data; one training per base model, a perfect train-test match for the meta-features, but each layer sees less data. Our design is (b). It also matters for non-model resources: the cleaning word list was learned on folds 0-5, so the decision models on folds 6-9 see cleaned text exactly as test records do. The decision model's inputs are the ranker probability and rank, the reader logits, the index lookup's similarity features, the "what changed" features and the competition and collective features; the similarity and "what changed" features are fixed formulas whose word statistics are label-free; their only label-derived ingredient is the Indic-to-Latin word list, learned on folds 0-5, so they too are out-of-sample on folds 6-9.

HOW WE USED IT

Exactly the table above. The decision-model training rows are the training shortlist of incoming records that have a candidate in business folds 6-9, built from the unpruned matcher output, with label "candidate is the owner". The India decision model uses 211 features, the US one 217, and France and the default route 158.

EXPERT NUANCE

The trade-off is data efficiency: our decision models learn from 4 of the 10 folds, and the readers from 6. Out-of-fold predictions for the readers would let the decision models use all ten folds but would need several reader trainings; the reader-training stage alone (MiniLM and E5 together) took about 2.1 hours in our clean run on one GPU, and out-of-fold reader training would multiply that by the number of folds, which we chose not to spend in the competition window. One mild overlap exists: fold 6 both stopped the ranker and is part of the decision models' training data; early stopping only chooses the number of trees, so the ranker's fold-6 scores are at most very slightly optimistic. Note also that the separation only works if the test-time inputs are produced the same way as the training inputs; this is why France needs care (the French readers were self-trained on French test pairs, see Self-training with pseudo-labels) and why the France blend is fed in logit units (see Blending readers in logit space).

REMEMBER

- A model's scores on its own training data are over-confident; never train the next layer on them.
- Ranker and readers: folds 0-5. Decision models: folds 6-9. Thresholds: folds 8 and 9.

- Layer-wise split = one training per model and identical train and test inputs, at the cost of less data per layer.

JUDGE MIGHT ASK

- **Q:** "Why not use out-of-fold predictions so the decision model could learn from all ten folds?"
A: Out-of-fold scores would need several trainings of each cross-encoder; on one 16 GB GPU the two readers already took about 2.1 hours to train once, so we chose a layer-wise split: readers and ranker on folds 0-5, decision models on folds 6-9. The decision models then see reader scores produced exactly as at test time, and they still learn from four folds of labelled data.

A3.7 Cross-fitting

IN ONE LINE

When a learned ingredient (a word list, an encoder, a model score) is used to build training examples, compute it for each record from a version that was learned without that record's fold, so training examples look exactly like test examples.

SIMPLE EXPLANATION

If students mark their own homework with an answer key they wrote after seeing the answers, every answer looks right, and the marks tell you nothing. **Cross-fitting** is the rule "you may only use an answer key written without your own homework". In our India second-chance search, a name written in an Indian script such as Devanagari (the script of Hindi and Marathi) is rewritten in Latin letters (**transliterated**) with a word list learned from labelled matches. When we built the training examples for the second-chance model, each record was transliterated with a word list learned without its own fold. At test time the word list has never seen the test record either, so the training examples and the test examples are made the same way, and the model learns how reliable the word list really is on unseen names.

WORKED EXAMPLE

(toy) Training records of the India second-chance search are spread over folds 0-7. Record R in fold 2 has the native name "बालाजी ट्रेडर्स" and owns the business "Balaji Traders". Suppose R is the only labelled record anywhere that contains the word "बालाजी".

Word list used to build R's training example	Does it know "बालाजी"?	R's transliterated key	Looks like test time?
Learned from all folds 0-7 (including R itself)	yes, learned from R's own label	(balaji, traders): a perfect key	no: at test time, rare words like this are often unknown
Learned from folds 0-7 except fold 2 (cross-fitted)	no	the word is missing or falls back to a generic romanisation	yes

With the first option the second-chance model learns "keys always match perfectly" and becomes over-confident; with cross-fitting it learns how often keys are incomplete, which is the truth on new records.

TECHNICAL EXPLANATION

Cross-fitting is sample splitting for any fitted component that feeds a later estimator: split the data into K folds; for each fold k, fit the component on the other folds and apply it to fold k; the downstream model is then trained on these out-of-fold values. It generalises out-of-fold predictions (see Layer-wise fold separation for stacked models) to anything learned from labels: target encoders, vocabularies, lexicons, propensity models. The name comes from double or debiased machine learning (Chernozhukov et al., 2018), where cross-fitting removes the over-fitting bias of **nuisance models** (auxiliary models whose outputs feed the quantity you actually care about). The classic everyday example is **target encoding** of a category (replacing a category by the mean label of its rows), which leaks badly unless computed out-of-

fold. Cost: K fits of the component. At test time the component is fitted on all training folds, so test inputs are very slightly "better" than training inputs (more data); this mismatch usually errs on the safe side, because the test keys are, if anything, slightly more complete than the keys the model learned to be cautious about.

HOW WE USED IT

The native-script second-chance model (India) was trained on India second-chance candidates of training folds 0-7, retrieved with word lists that leave out the record's own fold (Documentation, Section 4). It links the best candidate if its score is at least 0.88 and it leads the second candidate by at least 0.3; the 0.88 was chosen on fold 8 with false positives counted twice (see Cost-sensitive threshold selection). At test time the word list is learned from all labelled training records, since no test record has a label to leak. In the final run it scored 649,965 pairs for 171,713 records and added 9,759 links. The separate word list used in the main cleaning step is not cross-fitted; it is simply learned on folds 0-5 only, so every later layer (folds 6-9) sees cleaned text made without its labels.

EXPERT NUANCE

Cross-fitting removes the optimism of the learned ingredient, but not the optimism of choices made afterwards (the 0.88 threshold was still chosen on fold 8). A common misconception is that out-of-fold treatment is only needed for model predictions; any label-derived preprocessing needs it. A second subtlety is consistency: the procedure that builds training keys and the one that builds test keys must be identical apart from the data the word list was learned on; otherwise the second-chance model learns a distribution it will never see.

REMEMBER

- Cross-fitting = compute each record's learned ingredient from a version fitted without that record's fold.
- Our India second-chance search: word lists leave out the record's own fold; trained on folds 0-7; threshold 0.88 (lead 0.3) with 0.88 chosen on fold 8.
- Any label-derived preprocessing (encoders, word lists) needs it, not only model scores.

JUDGE MIGHT ASK

- **Q:** "Why go to the trouble of cross-fitting a word list?"
A: Because a word list that has learned from a record's own label makes that record's search key look perfect, so the second-chance model would learn to trust keys more than it should; at test time the list has never seen the record. With lists that leave out the record's own fold, the training keys are as imperfect as the test keys, and the model that added 9,759 Indian links learned the true reliability.

A3.8 The held-out evaluation protocol

IN ONE LINE

To grade a recipe honestly on fold 9, we retrain it without fold 9 and without any incoming record that touches fold 9, and then compute the exact competition metric on fold 9's businesses (and the same for fold 8).

SIMPLE EXPLANATION

Think of re-running a whole training camp without the players you want to test, so nobody in the camp has practised against them. There is one extra twist. An incoming record can have candidates in several folds: for example, its true business is in fold 9 and a look-alike candidate is in fold 7. If that record stayed in training through its fold-7 pair, the model would learn "this record is not the fold-7 business", which partly reveals the fold-9 answer by elimination, and the competition features that compare the two

candidates would tie them together. So we remove from training every incoming record that has any candidate in the fold being scored. That is the meaning of "held-out" in our numbers.

WORKED EXAMPLE

(toy) Four incoming records and their candidate businesses, with each business's fold in brackets. We want to score fold 9.

Record	Candidates (fold)	In the fit for scoring fold 9?
R1	B_a (6), B_b (7)	yes
R2	B_c (9)	no: touches fold 9
R3	B_d (7), B_e (9)	no: touches fold 9 (even its fold-7 pair is removed)
R4	B_f (8)	yes

1. Fit the decision model on the fold 6-8 pairs of R1 and R4 only.
2. Use a fixed number of trees: (toy) seed 1 early-stopped at 1,000 trees when fitted on folds 6-7; the new fit on folds 6-8 has 1.5 times as many rows as folds 6-7 (both counted without the records touching fold 9), so it uses $1,000 \times 1.5 = 1,500$ trees.
3. Score all pairs of R2 and R3, including R3's fold-7 pair, so that "best candidate" and "lead over the runner-up" are computed exactly as at test time.
4. Apply the decision rule (0.75 threshold, 0.4 lead, consistency check) and compute per-business F0.5 for every fold-9 business, including fold-9 businesses whose true records were never retrieved (they count as misses).
5. Average over fold-9 businesses.

(ours) The results:

Recipe, as measured	Fold 9	Fold 8
India: the final India recipe with 3 of its 5 seeds, with consistency check	0.990046	0.990298
US: the same 3-seed model, scored on US businesses, with consistency check	0.991212	0.991545
France recipe (one fit) applied to US / India at the France operating point, no consistency check: a proxy, not a French score	0.990508 / 0.989467	0.990977 / 0.989850

TECHNICAL EXPLANATION

The protocol (Documentation Section 5; code in the evidence folder) is: fold 9 is scored by fits on folds 6-8 without the records touching fold 9; fold 8 by fits on folds 6, 7 and 9 without the records touching fold 8; the number of boosting rounds is fixed at each seed's early-stopping best iteration $\times n_{\text{fit}} / n_{\text{fit}}(6, 7)$ (with the same exclusion), so the scored fold is never used for stopping; 3 of the 5 seeds are averaged. The decision uses the production rule: threshold 0.75, lead 0.4, and for the US and India the consistency check (0.68, 0.72, 0.77 for 1, 2 and 3 or more other accepted records). The metric is the exact per-business macro F0.5, where each business's true count comes from the label table, so owners lost by the search are counted as misses: the score includes retrieval losses, not only decision errors. The "touching" exclusion is stricter than group k-fold: it removes cross-fold information carried by records whose candidates straddle folds. Differences between recipes are compared with a paired bootstrap over businesses (see Paired bootstrap intervals).

HOW WE USED IT

This is our main evidence for the decision-model recipes and for the claim, said on the results slide, that "close versions of our decision models score between 0.990 and 0.992 on hidden folds". We are precise about what it does not cover: the final US and India models are refitted on folds 6-9 (so they differ from the evaluated fits); the US number comes from the India recipe applied to US businesses (the final US

model trains on US rows only and adds 6 features); the 0.75 threshold was chosen on fold 8 and the consistency check on folds 8 and 9; and the scores exclude the second-chance searches and the France rules. France has no labels, so the France line is only a proxy on US and Indian businesses.

EXPERT NUANCE

Because the same scaled-iteration rule is used to produce these held-out fits, the protocol also tests the refit rule itself (see Refitting on more folds with scaled iterations), which is a nice property. The weak point is reuse: folds 8 and 9 chose thresholds, so the held-out numbers carry some selection optimism, which is why we call them "evidence in the same range" and not "an unbiased estimate". A simpler protocol (fit on 6-7, stop on 8, score 9) gives an honest fold 9 but not an honest fold 8, since fold 8 was the stopping fold; the swap gives two scores, each from fits that never touched it.

REMEMBER

- Held-out = refit without the scored fold and without every record touching it, fixed tree count, exact per-business F0.5.
- Fold 9 / fold 8: US 0.991212 / 0.991545; India 0.990046 / 0.990298 (3 of 5 seeds).
- It excludes the second-chance searches and the France rules, and folds 8 and 9 also chose thresholds.

JUDGE MIGHT ASK

- **Q:** "What exactly does 'held-out' mean in your 0.991212?"
A: The decision model was refitted on folds 6-8 without fold 9 and without any incoming record that has a candidate in fold 9, with a fixed tree count, and then fold 9's businesses were scored with the exact per-business F0.5, our thresholds and the consistency check. It uses 3 of the 5 seeds and excludes the second-chance searches and France rules, and fold 9 also helped choose the consistency check, so it is strong evidence, not an untouched test.

A3.9 Early stopping

See also [A2.3 Overfitting, held-out data and early stopping](#): overfitting and early stopping from the models side; [A2.24 Early stopping, then refit on more data](#): early stopping, then refit.

IN ONE LINE

Keep adding trees (or training steps) only while a score on a separate validation fold keeps improving, and stop when it has not improved for a set number of rounds.

SIMPLE EXPLANATION

Baking a cake, you test with a toothpick and take it out when it is done, not when the maximum time in the recipe runs out; over-baking ruins it. A boosted-tree model adds small trees one by one. At first each tree fixes real mistakes; later, trees start fitting noise peculiar to the training data, and performance on new data gets worse. **Early stopping** watches the score on a validation fold and keeps the model from the round where that score was best. **Patience** is how many extra rounds you wait in case the score improves again, because validation curves are a little noisy.

WORKED EXAMPLE

(toy) A decision model is trained with validation on fold 8, patience 100. The validation measure is **log loss**: for one pair with predicted probability p and label y , loss = $-\ln(p)$ if $y = 1$ and $-\ln(1 - p)$ if $y = 0$. A true pair scored 0.9 costs $-\ln(0.9) = 0.105$; a false pair scored 0.9 costs $-\ln(0.1) = 2.303$, so confident mistakes are punished hard.

Trees	Training log loss (toy)	Fold-8 log loss (toy)
100	0.0280	0.0300

Trees	Training log loss (toy)	Fold-8 log loss (toy)
400	0.0150	0.0210
800	0.0090	0.0195 (best)
850	0.0086	0.0196
900	0.0083	0.0197

Training loss keeps falling, but the fold-8 loss is best at 800 and has not improved for 100 rounds by 900, so training stops at 900 and the model is saved at 800 trees. The "best iteration" is 800.

TECHNICAL EXPLANATION

Early stopping is a form of regularisation: for gradient-boosted decision trees (**GBDT**), the number of trees times the learning rate controls model capacity, and for linear models trained by gradient descent early stopping behaves much like an **L2 penalty** (a penalty on the sum of squared weights, which keeps weights small). The procedure monitors a validation metric each round and keeps the iteration with the best value (best_iteration); training ends after a patience window without improvement or at a maximum round count. The monitored metric should be smooth and **proper** (a **proper scoring rule** is one whose expected value is best when the predicted probability equals the true probability), so log loss is the usual choice; a thresholded metric such as F0.5 is piecewise constant in the scores and too noisy to stop on. Smaller learning rates need more trees but usually generalise slightly better. Two costs: you need a validation fold, and because the stopping point is chosen on that fold, the fold's own score becomes slightly optimistic. With several seeds, each seed has its own best iteration.

HOW WE USED IT

Model	Maximum	Patience	Stopping fold	Metric
CatBoost ranker	1,600 trees (depth 7, learning rate 0.075)	180 rounds	fold 6	Logloss
US and India decision models	3,000 rounds (learning rate 0.05)	100 rounds	fold 8	binary log loss
France and default decision models	1,500 rounds	100 rounds	fold 8	binary log loss
Readers	no early stopping: one fixed epoch; the learning rate rises over the first 3 % of steps (warm-up), then falls linearly to zero			
Address second-chance filter / scorer	fixed 200 / 300 rounds			

For the US and India the early-stopped tree count is then scaled up for a refit (see the next concept).

EXPERT NUANCE

We used fold 8 both to stop the decision models and to choose the 0.75 threshold, so fold-8 numbers carry a little double-use optimism; fold 9 is the cleaner report. Early stopping on log loss and then choosing a threshold for F0.5 is the right split of jobs: the trees are tuned to produce good probabilities, and the decision rule is tuned to the business metric. For the readers we chose a fixed one-epoch budget rather than early stopping; we did not run an epoch sweep, so we cannot say what a second epoch would have done, but their quality on held-out pairs is measured: the E5 reader has AUC 0.998570 and top-1 accuracy 0.993004 (both defined in the word table above) on 509,260 pairs of 437,715 labelled fold-8/9 records, which the readers never trained on.

REMEMBER

- Early stopping keeps the iteration with the best validation loss; patience absorbs noise.
- Ranker: patience 180 on fold 6. Decision models: patience 100 on fold 8, at most 3,000 rounds (US/India) or 1,500 (France).

- Stop on a smooth proper loss (log loss), tune the threshold on the business metric.

JUDGE MIGHT ASK

- **Q:** "Why don't your readers use early stopping?"
A: We gave them a fixed budget of one epoch with a short warm-up and linear decay, which is a common choice when fine-tuning a pretrained encoder on a large set of selected examples. We did not run an epoch sweep, so we cannot quantify the alternative, but their held-out quality is measured, for example the E5 reader's AUC of 0.998570 on fold-8 and fold-9 pairs.

A3.10 Refitting on more folds with scaled iterations

See also [A2.24 Early stopping, then refit on more data](#): early stopping, then refit, from the models side.

IN ONE LINE

After early stopping has told you how many trees to use, retrain on all available folds with that tree count scaled up in proportion to the extra data.

SIMPLE EXPLANATION

Suppose you tested a recipe with half the ingredients and found it needs 20 minutes. Now you cook the full amount, but you no longer have a spare cake to test with a toothpick, so you use a rule of thumb: twice the food, roughly twice the time. Early stopping needs a validation fold; once you want to use that fold for training too, nothing is left to stop on. More data supports a bigger model before it starts memorising, so the refit uses more trees, in proportion to the number of training rows.

WORKED EXAMPLE

(toy) For one seed of the US decision model:

Step	Rows (toy)	Trees
Fit on folds 6-7, early stopping on fold 8	2.0 million	best iteration 1,200
Refit on folds 6-9, no validation	4.1 million	$1,200 \times 4.1 / 2.0 = 2,460$

(ours) The public results of this step:

Change	Public score before → after	Decision
US and India decision models refitted on folds 6-9	0.990621 → 0.990676	kept
France decision model refitted on folds 6-9	0.990726 with it vs 0.990748 without it	not adopted

TECHNICAL EXPLANATION

The rule is $T_{\text{refit}} = T_{\text{best}} \times n(6-9) / n(6-7)$, applied per seed, with a fixed number of rounds and no validation set. The reasoning: at a fixed learning rate, the number of boosting rounds that a dataset can support before over-fitting tends to grow with its size, because more rows mean more real signal for each extra tree to fit; scaling by the row ratio is a common heuristic (another is multiplying by $k/(k-1)$ after k -fold cross-validation). It is a heuristic, not a law; it can over- or under-shoot. The gain is more training data for the final model (folds 6-9 hold roughly twice the rows of folds 6-7, since the folds are of similar size); the risk is that the exact refitted model is never validated. In our case the risk is reduced because the held-out protocol uses the same rule: the fold-9 fits on folds 6-8 use the best iteration scaled by $n(6-8) / n(6-7)$, and those fits are what produce our held-out scores (see The held-out evaluation protocol).

HOW WE USED IT

US and India: per seed (1 to 5), fit on folds 6-7 with early stopping on fold 8 (at most 3,000 rounds), then refit on folds 6-9 for best iteration $\times$ rows(6-9) / rows(6-7); the five refits are averaged. France and the default route: the early-stopped fit on folds 6-7 is used as it is (at most 1,500 rounds), with no refit, because the France refit scored 0.990726 against 0.990748 without it on the public leaderboard.

EXPERT NUANCE

After the refit, folds 8 and 9 are inside the training data of the final US and India models, so no labelled fold is left to score those exact models; this is one of the reasons our Documentation says there is no untouched end-to-end validation. The France decision was made on a public difference of 0.000022 (derived), which is tiny and could be noise; when a change showed no public gain, we kept the existing, simpler version. A further subtlety: the 0.75 threshold was chosen on fold 8 and then reused for the refitted models, which have fold 8 in their training data; the held-out fits, which use the same scaled refit rule and the same threshold, are the reassurance that this transfer works.

REMEMBER

- Refit rule: trees = best iteration $\times$ rows(6-9) / rows(6-7), because no fold is left for stopping.
- US/India refit kept (public 0.990621 $\rightarrow$ 0.990676); France refit dropped (0.990726 vs 0.990748).
- The held-out scores use the same scaled rule, so the rule itself is tested.

JUDGE MIGHT ASK

- **Q:** "Isn't refitting without a validation fold risky?"
A: A little, and that is why we scale the tree count from early stopping by the row ratio instead of guessing, and why our held-out fits use the same rule: fits on folds 6-8 with scaled trees score 0.991212 on US and 0.990046 on Indian fold-9 businesses. On the public board the refit moved US/India from 0.990621 to 0.990676; for France it did not help (0.990726 vs 0.990748), so France keeps the plain fit.

A3.11 Seed averaging

See also [A2.25 Seed averaging](#): seed averaging from the models side.

IN ONE LINE

Train the same model several times with different random seeds and average their predicted probabilities, so the randomness of any single training run cancels out.

SIMPLE EXPLANATION

Ask five equally good doctors for an opinion and average them, and the result is steadier than any one doctor's. Our LightGBM decision models use deliberate randomness: every boosting round uses a random 80 % of the rows, and every tree a random 90 % of the features. A **random seed** is the starting number for that randomness; a different seed gives a slightly different model. Most pairs get the same answer from every seed, but pairs near the 0.75 line can flip between "link" and "no link" from one seed to the next. Averaging five seeds makes those borderline decisions steadier.

WORKED EXAMPLE

(toy) One borderline US pair scored by five seeds:

Seed	1	2	3	4	5	Mean
Probability	0.74	0.78	0.71	0.80	0.77	0.760
Decision at 0.75	no	link	no	link	link	link

With a single seed, two runs out of five would reject this pair. The spread across seeds is 0.035 (sample standard deviation); if the seeds were independent, the mean of five would have a spread of $0.035 / \sqrt{5} = 0.016$. Seeds trained on the same data are not independent, so the real reduction is smaller (see the formula below).

TECHNICAL EXPLANATION

If M models each have prediction variance σ^2 and pairwise correlation p , the variance of their average is $p\sigma^2 + (1 - p)\sigma^2/M$: only the uncorrelated part shrinks with M , which is why **subsampling** (bagging of rows, here bagging_fraction 0.8 every iteration, and feature_fraction 0.9) matters; it decorrelates the members. Averaging reduces variance, not bias. We average predicted probabilities (an arithmetic mean); averaging logits is an alternative that weights confident members more. Averaging tends to pull extreme scores slightly inwards, which reduces over-confidence. The cost is M times the training and inference of the tree models, which is small next to the cross-encoders. Picking the single best seed on a validation fold is the wrong alternative: it selects partly on noise and is a mild form of selection leakage.

HOW WE USED IT

The US, India and France decision models are each the mean of seeds 1-5 (LightGBM, learning rate 0.05, 127 leaves, at least 200 rows per leaf, feature fraction 0.9, bagging 0.8 every iteration, L2 1.0). The default route for unseen countries uses a single fit. The address second-chance scorer averages seeds 7, 8 and 9. The held-out table uses 3 of the 5 seeds. Honestly, the effect of the seeds is not isolated: the five-seed models entered the final file in the same upload as the French E5 reader (public 0.991114 → 0.991300), so we do not claim a separate number for them.

EXPERT NUANCE

Seed averaging is a cheap, low-risk way to stabilise the decision boundary, and it matters more for a precision-oriented rule with a hard threshold, where instability near the threshold turns directly into flipped links. To measure its own effect we would compare one-seed and five-seed decisions on held-out folds 8 and 9 with a paired bootstrap over businesses. A misconception is that averaging always improves the metric; it improves stability, and the metric gain, if any, is usually small.

REMEMBER

- Five seeds, probabilities averaged; row (0.8) and feature (0.9) subsampling makes the seeds differ.
- Averaging reduces variance (only the uncorrelated part), not bias.
- Its own effect is not isolated: it arrived with the French E5 reader (0.991114 → 0.991300).

JUDGE MIGHT ASK

- **Q:** "Why average five seeds rather than keep the seed that scored best on validation?"
A: Picking the best seed on fold 8 would partly pick fold 8's noise; averaging uses all five, steadies decisions near the 0.75 line, and costs only a few small tree-model calls per pair. Its own gain is not isolated in our evidence, because the five-seed models arrived in the same upload as the French E5 reader.

A3.12 Tuning thresholds on held-out folds

IN ONE LINE

Choose the score cut-off for linking by trying several values on a held-out fold and keeping the one that gives the best exact competition score there, not the best AUC.

SIMPLE EXPLANATION

The decision model gives every candidate a probability, but you still need a rule: "link if the score is above X". If X is too low you make many wrong links (precision falls); if X is too high you miss true links (recall falls). The best X depends on how the scoring works. Our competition scores each business separately with F0.5, whose formula counts a wrong link four times as heavily as a missed one, then averages over businesses, and gives a full point to a business with no true records only if we link nothing to it. So we try candidate values of X on fold 8, compute exactly that score for each, and keep the best. A general-purpose measure such as AUC cannot do this job, because AUC does not depend on X at all.

WORKED EXAMPLE

(toy) A held-out fold with four businesses and six incoming records. Each record's best candidate is shown; second candidates are far behind, so the lead rule is always passed.

Record	Best candidate	Score	Truth
r1	B1	0.95	correct
r2	B1	0.70	correct
r3	B2	0.80	correct
r6	B2	0.85	wrong (r6 belongs to no business)
r5	B3	0.72	wrong (r5 belongs to no business)
r4	B4	0.60	correct

True counts: B1 owns 2 records, B2 owns 1, B3 owns 0, B4 owns 1. Per business, $F0.5 = 1.25 \times TP / (\text{predicted} + 0.25 \times \text{true})$, and 1 if predicted = true = 0.

Threshold	Links made	B1	B2	B3	B4	Macro F0.5
0.65	r1, r2, r3, r6, r5	$2.5 / 2.5 = 1.000$	$1.25 / 2.25 = 0.556$	0 (wrong link to an empty business)	0 (missed)	0.389
0.75	r1, r3, r6	$1.25 / 1.5 = 0.833$	0.556	1 (correctly empty)	0	0.597
0.90	r1	0.833	0	1	0	0.458

The best threshold here is 0.75. Notice what drives it: protecting the empty business B3 is worth a whole point, which a pair-level measure would barely notice. The AUC of these six scores is one fixed number whichever threshold we pick, because AUC only checks whether true links are ranked above wrong ones; it cannot tell us where to cut.

TECHNICAL EXPLANATION

For one business, $F_{\beta} = (1 + \beta^2) \times P \times R / (\beta^2 \times P + R)$ with $P = TP / \text{predicted}$ and $R = TP / \text{true}$; with $\beta = 0.5$ this simplifies to $F0.5 = 1.25 \times TP / (\text{predicted} + 0.25 \times \text{true}) = 1.25 \times TP / (1.25 \times TP + FP + 0.25 \times FN)$, which shows directly that a false positive (FP, a wrong link) weighs four times as much as a false negative (FN, a missed link). The competition averages it over businesses (macro), with the convention that an empty prediction for an empty business scores 1. This metric is **non-decomposable**: the value of a link depends on the other links of the same business, so pair-level summaries (AUC, log loss, pair-level F) can disagree with it. Threshold tuning is therefore a direct search: for each value on a grid, apply the full decision rule on the held-out fold, compute the exact macro metric, and keep the first maximum. A useful sanity rule from theory (shown for F1 by Lipton, Elkan and Narayanaswamy, 2014; the same argument extends to F_{β}): for a single global F_{β} and calibrated probabilities, the optimal threshold equals $F^* / (1 + \beta^2)$, where F^* is the best achievable F; for $\beta = 0.5$ that is $0.8 \times F^*$, so with F^* near 0.99 the optimum is near 0.79. Our 0.75 (US, India) and 0.82 (France) sit on either side of that value; the rule is only a sanity check, because our metric is a per-business average and our scores are not proven calibrated (see Calibration and reliability). Tuning on the same fold you report is optimistic; the usual remedy is to report a second fold.

HOW WE USED IT

Every US and India operating point was chosen on labelled folds, never by AUC: the threshold 0.75 on fold 8 and the per-business consistency check on folds 8 and 9, both for the exact per-business macro F0.5; the second-chance thresholds on fold 8 for the same metric with false positives counted twice; and the address second-chance filter's cut-off by a recall rule, the largest grid value that keeps 98 % of the fold-8 reference links. The lead of 0.4 over the runner-up is one fixed value shared by every country's rule. France has no labels, so its threshold 0.82 was set without labels, from the number of French links per business together with a fold-8 threshold sweep on US and Indian data.

EXPERT NUANCE

Near the optimum the metric curve is usually flat, so a slightly wrong threshold costs little on the tuning distribution; under a distribution change the optimum can move a lot, which is why France got a stricter value rather than a copied one. Threshold and model interact: if the model changes (for example, five seeds instead of one, or a refit), the score distribution moves a little and the tuned threshold may no longer be exactly optimal; our held-out fits, which use the production threshold on refitted models, are the check. A frequent misconception is "we maximised AUC, so the thresholds are fine"; AUC says nothing about where to cut.

REMEMBER

- Per-business F0.5 = $1.25 \times \text{TP} / (\text{predicted} + 0.25 \times \text{true})$; 1 for a correctly empty business.
- Thresholds were tuned on the exact metric on held-out folds: 0.75 on fold 8; France 0.82 without labels.
- AUC is threshold-free and pair-level; it cannot choose an operating point.

JUDGE MIGHT ASK

- **Q:** "Why not pick the threshold by maximising AUC or F1?"
A: AUC does not depend on the threshold at all, and F1 weighs recall as much as precision, while the competition uses F0.5 per business with a full point for a correctly empty row. So we tuned directly on that exact metric on held-out fold 8, which gave 0.75 for the US and India.

A3.13 Cost-sensitive threshold selection

IN ONE LINE

When choosing a threshold, count each wrong link as costing more than the metric says (we counted false positives twice), so the chosen threshold is deliberately more cautious.

SIMPLE EXPLANATION

Engineers add a safety factor when they are not sure the load on a bridge is exactly what was measured. Our second-chance searches work on records that the main decision model already left unlinked: a riskier group, where the training data come from a slightly different situation than the test data. To build in a safety factor, when we chose their thresholds on fold 8 we pretended that every wrong link cost twice as much as it really does. That pushes the cut-off higher: we accept fewer second-chance links, but the ones we accept are surer.

WORKED EXAMPLE

(toy) A second-chance model on fold 8. Simplified units: a true link is worth +1, a false link costs 1 (normal) or 2 (counted twice).

Threshold	True links	False links	Net, normal cost	Net, false links ×2
0.70	1,000	260	740	480

Threshold	True links	False links	Net, normal cost	Net, false links ×2
0.80	900	120	780	660
0.88	830	60	770	710
0.95	600	20	580	560

With the normal cost, 0.80 wins; with false links counted twice, 0.88 wins. You give up 70 true links (900 → 830) to avoid 60 false ones (120 → 60).

TECHNICAL EXPLANATION

In **cost-sensitive** decision theory, you accept a link if $p \times g > (1 - p) \times w \times c$, where p is the probability that the link is true, g the gain of a true link, c the cost of a false one and w a weight; the break-even probability is $p^* = w \times c / (g + w \times c)$, which rises with w (for $g = c$, p^* goes from 0.5 at $w = 1$ to 0.67 at $w = 2$). In our implementation the "gain" is the exact change in per-business macro F0.5 on fold 8, and each false link is counted twice in the predicted count of its business when the candidate thresholds are compared. Two motivations: robustness to **prior shift** (if the share of wrong candidates is higher at test time than in training, false links become more frequent than the training estimate) and a hedge against the optimism of choosing a threshold on the same fold you measure. An alternative is to estimate the test prior and re-weight explicitly; another is to require a lead over the second candidate, which we also do.

HOW WE USED IT

Native-script second-chance search (India): link the best candidate if its score is at least 0.88 and it leads the second by at least 0.3; 0.88 chosen on fold 8 with false positives counted twice; +9,759 links. Address second-chance search: the scorer's threshold and lead chosen on fold 8 with false positives counted twice: US 0.86 with lead 0.0, India 0.66 with lead 0.1; the filter keeps a pair if its probability is at least 0.06 (US) or 0.015 (India), the largest grid values that keep 98 % of the fold-8 reference links; +192 US and +2,945 Indian links. The main decision models were tuned on the plain exact metric, without the doubling.

EXPERT NUANCE

The factor 2 is a judgement call, a safety factor, not an estimated quantity; a judge may call it arbitrary, and the honest reply is that it is a deliberate hedge whose price is a little recall on small add-on modules. Doubling costs makes sense precisely where training and test differ most: the second-chance models act on the leftovers of the main decision, and their training records were themselves selected with earlier decision models. Over-caution has a cost, but for modules that add about 13,000 links out of 5,856,936 (derived), a little lost recall is cheap insurance.

REMEMBER

- Counting false positives twice raises the break-even probability and gives a more cautious threshold.
- Used only for the second-chance searches: 0.88 (native script), 0.86 US / 0.66 India (address).
- It is a safety factor against train-test differences, not a measured cost.

JUDGE MIGHT ASK

- **Q:** "Why count false positives twice for the second-chance searches? Isn't that arbitrary?"
A: It is a deliberate safety factor: those searches act on records the main model already rejected, a riskier group whose training data differ most from test, so we chose their thresholds on fold 8 as if each wrong link cost double. That gave, for example, 0.88 for the native-script search, which still added 9,759 Indian links.

A3.14 Calibration and reliability

See also [A2.26 Calibration: what a model's "probability" means](#): calibration from the models side.

IN ONE LINE

A model is **calibrated** if, among all pairs it scores 0.8, about 80 % really are matches.

SIMPLE EXPLANATION

A weather forecaster who says "70 % chance of rain" is calibrated if it rains on about 70 % of the days she says that. A model can rank pairs perfectly (every true pair above every false one) and still be badly calibrated, for example saying 0.99 when only 80 % of such pairs are true. Ranking quality and calibration are different properties. Calibration matters when you want the number itself to mean something: to reuse a threshold in a new country, to compare models, or to compute expected gains. A **reliability diagram** plots the predicted probability against the observed share of true matches; a calibrated model lies on the diagonal.

WORKED EXAMPLE

(toy) 1,000 held-out pairs grouped by the decision model's score:

Score bin	Pairs	Mean score	True matches	Observed share	Gap
0.0-0.2	600	0.02	6	0.010	0.010
0.2-0.5	100	0.35	30	0.300	0.050
0.5-0.8	100	0.66	70	0.700	0.040
0.8-1.0	200	0.95	180	0.900	0.050

Expected calibration error (ECE) = sum over bins of (share of pairs in the bin) $\times$ |observed – mean score| = $0.6 \times 0.010 + 0.1 \times 0.050 + 0.1 \times 0.040 + 0.2 \times 0.050 = 0.025$. The top bin is over-confident: the model says 0.95, reality is 0.90.

The **Brier score** is the mean of $(p - y)^2$ over pairs: a true pair scored 0.9 contributes 0.01; a false pair scored 0.9 contributes 0.81.

TECHNICAL EXPLANATION

Calibration means $P(Y = 1 \mid \text{score} = p) = p$ for every p . Measures: the reliability diagram; ECE (simple, but depends on the binning); the Brier score, a proper scoring rule that decomposes into reliability (calibration error) minus resolution (how well scores separate the classes) plus uncertainty (the base rate's own variance); and log loss, also proper. Post-hoc **calibration maps** are fitted on held-out data: **Platt scaling** (a logistic regression on the score), **isotonic regression** (a monotone step function, fitted by pool-adjacent-violators; flexible but data-hungry) and **temperature scaling** (divide neural logits by one fitted constant). All are monotone (non-decreasing), so they never reverse the order of two scores: Platt and temperature scaling leave the ranking and the AUC exactly unchanged, while isotonic regression can merge nearby scores into ties, which changes AUC only slightly. What they change is where a fixed threshold falls. Gradient-boosted trees trained with log loss are often reasonably calibrated on their training distribution, but row subsampling, class re-weighting, added synthetic examples and averaging all move them; fine-tuned cross-encoders tend to produce large, over-confident logits (our French worked example has reader logits up to ± 10.8). Calibration is distribution-specific: a model calibrated on US and Indian data need not be calibrated on French data. Finally, decision trees are invariant to monotone transformations of an input feature, so it does not matter whether the decision model receives reader logits or probabilities, provided training and test use the same transformation.

HOW WE USED IT

We did not use a separate calibration step (no Platt scaling, no isotonic regression). Instead the US and India thresholds (and the second-chance thresholds) were tuned directly on the exact metric on held-out labelled folds, which gives a good decision whether or not the scores are calibrated, as long as the held-out data resemble the test data. Two places where calibration thinking still shaped choices: the US and India decision models were trained with extra hard examples (see Hard examples and hard negatives),

which changes the class mix they see, so their raw scores need not equal real-world frequencies, and choosing the threshold on held-out labelled data absorbs that; and France, where no labels exist to check calibration, received the more conservative threshold 0.82, set without labels. Our documentation reports no calibration measurement.

EXPERT NUANCE

Ranking is not calibration: AUC 0.998 can coexist with poorly calibrated probabilities. A probability such as 0.999837 means "99.98 % sure" only if the model is calibrated on that population; our French worked example shows a link scored 0.999837 that a France rule then vetoed, a reminder that a high score is not proof. If we had wanted one threshold shared across countries, per-country calibration on a labelled sample would be the principled route. The reasoning behind the per-business consistency check (break-even probabilities, see Count-dependent thresholds) assumes calibrated probabilities, which is why its thresholds were tuned empirically rather than computed from a formula.

REMEMBER

- Calibrated = a score of p means a match about p of the time; ranking quality (AUC) is a different property.
- We did not calibrate explicitly; we tuned thresholds on the exact metric on held-out folds.
- Calibration does not transfer across domains automatically; France got the stricter 0.82.

JUDGE MIGHT ASK

- **Q:** "Are your decision-model probabilities calibrated, and does it matter for you?"
A: We did not calibrate them explicitly and our documentation reports no calibration measurement. For the US and India it matters little, because their thresholds were tuned on the exact per-business metric on held-out labelled folds, which works whether or not the scores are calibrated. Where it would matter, in France without labels, we used the stricter 0.82 instead of copying 0.75.

A3.15 The confident + clear-winner rule

IN ONE LINE

Link a record to its best candidate only if that candidate's score is high enough (at least 0.75 in the US and India, 0.82 in France and for unseen countries) and at least 0.4 ahead of the runner-up; otherwise link nothing.

SIMPLE EXPLANATION

A careful detective names a suspect only when the evidence is strong and nobody else is nearly as likely. If two suspects look almost equally guilty, naming either one risks accusing the wrong person, and saying "not enough evidence" is the safer call. Under F0.5 a wrong link hurts more than a missed one, and a business with no true records keeps its full point only if nothing is linked to it. So "abstain when unsure" is the right default. The rule has two parts: **confident** (the score is above a threshold) and **clear winner** (the lead over the second-best candidate is at least 0.4). This kind of "abstain unless sure" rule is called a **reject option**.

WORKED EXAMPLE

(ours) The US record "Legacy Marketing Enterprises (Inc)", with no address, has four shortlisted businesses:

Business	Decision-model score
Legacy Marketing Enterprises Inc, Windsor, CT	0.965022
Legacy Marketing Enterprises PC, Joshua Tree, CA	0.088664
Legacy Marketing Industries Inc, Louisville, TN	0.000088

Business	Decision-model score
Legacy Marketing Enterprises LLC, Irondequoit, NY	0.000007

1. Best = 0.965022 $\geq$ 0.75: confident.
2. Lead = 0.965022 – 0.088664 = 0.876 $\geq$ 0.4: clear winner.
3. Link to the Windsor business, and to nothing else.

(toy) Three other records:

Record	Best	Second	Confident?	Clear winner?	Decision
X	0.81	0.62	yes	no (lead 0.19)	no link
Y	0.78	none (only one candidate)	yes	yes (runner-up treated as minus infinity)	link
Z	0.70	0.05	no	yes	no link

(toy) Why abstaining is cheaper: a business owns 3 records and we linked 2 correctly: $F0.5 = 2.5 / (2 + 0.75) = 0.909$. Add one wrong link: $2.5 / (3 + 0.75) = 0.667$. A loss of 0.24 from a single wrong link; on an empty business, one wrong link turns 1 into 0.

TECHNICAL EXPLANATION

The rule is $a^* = \text{argmax}_j s_j$; accept a^* if $s_{a^*} \geq \tau$ and $s_{a^*} - s_{(2)} \geq m$, where $s_{(2)}$ is the second-highest score of the record (minus infinity if there is none); at most one link per record. It combines Chow's **reject option** (abstain when the top posterior is low) with **ambiguity rejection** (abstain when the top two are close). The margin is needed because our decision model is **pointwise**: each pair is scored independently, so the probabilities of a record's candidates do not sum to 1 and two candidates can both score 0.9; a pointwise model does not enforce "at most one owner". The margin enforces mutual exclusivity cheaply. Alternatives are a **listwise** model (for example a softmax over a record's candidates) or a joint assignment; our decision model partly sees the competition through its rank, margin, best-other and group-maximum features. Raising τ or m trades coverage (records linked) for precision, the classic risk-coverage curve.

HOW WE USED IT

The same rule in every decision model: $\tau = 0.75$ for the US and India (chosen on fold 8), $\tau = 0.82$ for France and the default route, $m = 0.4$ everywhere. The second-chance searches use their own pairs: native script 0.88 with a lead of 0.3; address search US 0.86 / 0.0 and India 0.66 / 0.1. Most records have a single candidate (90.59 % of incoming records have exactly one in the final candidate file), so for them only the threshold bites. Result: 100,087 businesses end with an empty row, and with 0.5875 links per incoming record about 41 % of incoming records are linked to nothing (derived).

EXPERT NUANCE

The margin is also a cause of misses, and our Documentation lists it among false negatives: correct candidates rejected because a second one is too close (for example two branches of one chain). How the value 0.4 was picked is not recorded in our documentation; say "0.4 is one fixed lead used in every country; the threshold was tuned on fold 8 for the exact metric", and do not claim it was never tuned. A subtle point worth knowing: for a single-candidate record the margin is satisfied automatically, so the clear-winner condition protects mainly the minority of records with several plausible candidates, which is exactly where wrong links concentrate.

REMEMBER

- Link only if score ≥ 0.75 (0.82 France) and lead ≥ 0.4 ; at most one link per record.
- The margin makes up for pointwise scores that are not exclusive across candidates.
- Abstaining is cheap under F0.5; one wrong link can turn a correct empty row from 1 into 0.

JUDGE MIGHT ASK

- **Q**

"Why both a threshold and a margin?"

A: The threshold answers "is this a match at all?", the margin answers "is it clearly this one and not its look-alike?", and our decision model scores each pair separately, so two candidates can both look high. In our US example the winner had 0.965 and the runner-up 0.089, a lead of 0.876, so we linked; with a lead below 0.4 we abstain.

- **Q**

"How was the 0.4 margin chosen?"

A: 0.4 is one fixed lead shared by every country's rule, and the threshold it works with was tuned on fold 8 for the exact per-business metric. Its job is to stop a link when two candidates both look plausible, which matters mainly for the minority of records with more than one candidate.

A3.16 Count-dependent thresholds: the per-business consistency check

IN ONE LINE

After the main decision, each record is re-decided with a bar that depends on how many other records were already linked to the same business: 0.75 if none, 0.68 if one, 0.72 if two, 0.77 if three or more, always with the 0.4 lead.

SIMPLE EXPLANATION

The value of one more link depends on what the business already has. If a business has no links yet, it might be one of the businesses with no true records, and a wrong link there wipes out a full point, so the bar must be high. Once a business already has an accepted link, it is very probably a real, active business, so a wrong extra link only costs part of a point, while a right one completes its list; the bar can come down a little. But when a business already has three or more links, one more true link adds little and a wrong one still costs a lot, so the bar goes back up. The **per-business consistency check** applies exactly this idea, with values tuned on held-out folds 8 and 9.

WORKED EXAMPLE

(toy, for intuition only) Assume the k links a business already has are all correct and no other true record is missing. A record with probability p of belonging to it is being considered.

k (other accepted links)	If the record is true: F before → after	Gain	If the record is false: F before → after	Loss	Break-even $p = \text{loss} / (\text{gain} + \text{loss})$
1	$1.25 / 1.5 = 0.833 \rightarrow 1.000$	0.167	$1.000 \rightarrow 1.25 / 2.25 = 0.556$	0.444	0.727
2	$2.5 / 2.75 = 0.909 \rightarrow 1.000$	0.091	$1.000 \rightarrow 2.5 / 3.5 = 0.714$	0.286	0.759
3	$3.75 / 4 = 0.9375 \rightarrow 1.000$	0.0625	$1.000 \rightarrow 3.75 / 4.75 = 0.789$	0.211	0.771

The break-even rises with k , as the tuned bars do from one link upwards (0.68, 0.72, 0.77). The toy does not cover $k = 0$: whether a first link helps depends on how many true records the business has, which we do not know, and the business may have none at all. So the $k = 0$ bar is simply the main threshold 0.75, tuned on fold 8 over all records. The real values for $k \geq 1$ were tuned on folds 8 and 9; this toy only shows why the bar should depend on k . (If some true records of the business are still unlinked, a true link gains more and a false one costs less, so the break-even is lower: for $k = 1$ with one more true record missing, a true link moves F from $1.25 / 1.75 = 0.714$ to $2.5 / 2.75 = 0.909$ (gain 0.195) and a false one from $1.25 / 1.5 = 0.833$ to $1.25 / 2.5 = 0.500$ (loss 0.333), so the break-even drops from 0.727 to $0.333 / (0.195 + 0.333) = 0.63$. That may be one reason the tuned bars sit below the toy values; the scores are also not guaranteed to be calibrated.)

TECHNICAL EXPLANATION

Per-business F0.5 is non-decomposable, so the marginal value of a link depends on the other links of the same business (see Tuning thresholds on held-out folds). A fully optimal decision would choose all links of a business jointly to maximise expected F0.5; the consistency check is a one-pass approximation.

Procedure: take the main decisions; for each candidate business count k = the number of *other* incoming records accepted for it; re-decide each record with threshold $T(k)$ and the same 0.4 lead. k is computed once from the initial decision (no iteration, so the result does not depend on processing order). It never adds a link for a record that already has a native-script second-chance link. The break-even reasoning assumes calibrated probabilities and independent errors, neither of which is guaranteed, which is why the thresholds were tuned on the exact metric rather than computed.

HOW WE USED IT

US and India only (France, which has no labels to tune it on, does not use it): 0.75 / 0.68 / 0.72 / 0.77 for $k = 0 / 1 / 2 / 3$ or more, margin 0.4, chosen on folds 8 and 9. Effect in the final run: US +277 links added and -284 removed; India +247 and -248, with 1 addition skipped because the record already had a native-script second-chance link. The held-out scores in Section 5 of our Documentation include it.

EXPERT NUANCE

The check changes very few links (a few hundred out of about 2.25 million US and 2.73 million Indian accepted links), so it is a fine-tuning step, not a core component. Its thresholds were chosen on folds 8 and 9, which are also our reporting folds, so its held-out benefit is not independently measured; we disclose that the folds were reused. A judge may worry about chains with branches: the lowest bar is 0.68 and the 0.4 lead still applies, so a record caught between two branches still abstains.

REMEMBER

- Bars 0.75 / 0.68 / 0.72 / 0.77 for 0 / 1 / 2 / 3+ other accepted records; lead 0.4; US and India only.
- Reason: the value of one extra link under per-business F0.5 depends on how many links the business has.
- Small effect: US +277 / -284, India +247 / -248; tuned on folds 8 and 9.

JUDGE MIGHT ASK

- **Q:** "Isn't lowering the bar to 0.68 when a business already has a link a recipe for errors with chain stores?"
A: The 0.4 lead still applies, so a record between two near-identical branches still abstains, and the bar rises again to 0.77 once a business has three or more links. The thresholds were tuned on held-out folds 8 and 9 for the exact metric, and the net change is small: US +277 / -284 and India +247 / -248 links.

A3.17 Paired bootstrap intervals

IN ONE LINE

To judge whether recipe A really beats recipe B, resample the businesses with replacement many times, recompute the average score difference each time, and read off the middle 95 % of those differences.

SIMPLE EXPLANATION

If a new recipe wins by 0.00003, is that real or luck? The **bootstrap** treats our held-out fold as a small copy of the world: draw businesses at random with replacement (some twice, some not at all), recompute the score difference, and repeat a few thousand times. If almost every redraw shows A ahead, the difference is believable; if the redraws straddle zero, it could be noise. **Paired** means A and B are always scored on

exactly the same drawn businesses, so the noise of "this draw happened to contain easy businesses" cancels out, and only the difference between the recipes remains.

WORKED EXAMPLE

(toy) Six businesses, per-business F0.5 difference A – B:

Business	b1	b2	b3	b4	b5	b6
A – B	0	+0.167	0	+0.091	0	–0.444

Mean difference = $(0.167 + 0.091 - 0.444) / 6 = -0.031$.

Resample (drawn with replacement)	Differences	Mean
b2, b2, b3, b4, b5, b1	0.167, 0.167, 0, 0.091, 0, 0	+0.071
b6, b6, b1, b3, b4, b5	–0.444, –0.444, 0, 0, 0.091, 0	–0.133
b1, b2, b4, b6, b3, b3	0, 0.167, 0.091, –0.444, 0, 0	–0.031

Repeat 2,000 times, sort the 2,000 means, and take the 2.5th and 97.5th percentiles as the 95 % interval. With six businesses the interval is very wide and straddles zero: no conclusion. Our held-out folds have about 132,000 US and 88,000 Indian businesses each, so the intervals are far narrower.

TECHNICAL EXPLANATION

The nonparametric bootstrap (Efron) approximates the sampling distribution of a statistic by resampling the observed units with replacement; the **percentile interval** takes the empirical quantiles. Pairing exploits the correlation between A and B on the same unit: $\text{Var}(A - B) = \text{Var}(A) + \text{Var}(B) - 2 \text{Cov}(A, B)$, and since most businesses score the same under both recipes, the covariance is high and the variance of the difference is small, which makes paired comparisons far more sensitive than comparing two separate intervals. The resampling unit must be the independent unit of the metric, here the business (resampling records would ignore their clustering and understate the variance). With 2,000 resamples the **Monte Carlo error** (the extra randomness from using a finite number of resamples) of the 2.5th and 97.5th percentiles is small next to the interval width. Limits: it captures evaluation-sample noise only, not training randomness (seeds) and not the optimism of having tuned on the same fold. Alternatives: a paired permutation (sign-flip) test, or McNemar's test for paired binary decisions. Cost: $B \times N$ additions, trivial.

HOW WE USED IT

Our held-out evaluation code compares recipes on the same fold-8 and fold-9 businesses and reports 95 % intervals from a paired bootstrap over businesses with 2,000 resamples. The Documentation reports the absolute held-out values (US 0.991212 / 0.991545, India 0.990046 / 0.990298). On the public leaderboard no interval is possible: each upload gives a single number.

EXPERT NUANCE

An interval that excludes zero on both fold 8 and fold 9 is much stronger evidence than one fold alone. Many comparisons inflate false discoveries, so an interval is not licence to try fifty variants and keep the one that "wins". The bootstrap also says nothing about the public leaderboard differences between our late uploads (a few hundred-thousandths): those are single numbers with unknown noise, which is why we did not build claims on them and, when a change showed no public gain, kept the existing version.

REMEMBER

- Resample businesses (the unit of the metric) with replacement, 2,000 times; read the middle 95 %.
- Pairing (same businesses for A and B) removes shared noise and makes small differences testable.
- It measures evaluation noise only, not seed noise or tuning optimism.

JUDGE MIGHT ASK

- **Q:** "How do you know a difference of a few hundred-thousandths is real?"
A: On held-out folds we compare two recipes on the same businesses and bootstrap the per-business differences 2,000 times, so we get a 95 % interval; a difference whose interval excludes zero on both folds 8 and 9 is believable. On the public leaderboard there is only one number per upload, so very small public differences could be noise, and when a change showed no public gain we kept the version we already had.

A3.18 Hard examples and hard negatives

See also [A2.19 Hard-example selection for reader training](#): hard-example selection for the readers.

IN ONE LINE

Train mainly on the cases a model finds difficult (near-misses and look-alikes), plus a random sample of easy cases, because easy cases teach very little.

SIMPLE EXPLANATION

A driving instructor does not spend lessons on empty straight roads; the learner practises roundabouts, merging and reverse parking, with some ordinary driving mixed in so normal roads still feel normal. Most candidate pairs are trivially easy: a record and a business with nothing in common. A model trained mostly on those learns little about the hard calls. A **hard negative** is a non-match that looks like a match (same name, a different branch); a **hard positive** is a true match that looks different (abbreviated, transliterated, missing its house number). Choosing training examples this way is called **hard-example mining**. Keeping a random share of easy examples stops the model from forgetting what ordinary data look like.

WORKED EXAMPLE

(toy) Record "Sharma Traders, Laxmi Road, Pune":

Candidate	Type
Sharma Traders, Laxmi Rd, Pune	positive (the owner)
Blue Ocean Logistics, Chennai	easy negative: nothing in common
Sharma Traders, MG Road, Pune	hard negative: same name, another branch

(toy) ranker outputs; the selection rules are ours) Which records went into reader training? A record is "hard" if the ranker's top probability is between 0.003 and 0.9995, or a non-owner is ranked first, or the second candidate's probability is above 0.003; other records are kept with probability 25 %.

Record	Top-1 probability	Top-1 is the owner?	Second probability	Selected because
R1	0.9999	yes	0.0001	easy: only if it falls in the random 25 %
R2	0.62	yes	0.30	hard: top probability inside (0.003, 0.9995)
R3	0.9997	no	0.0002	hard: a non-owner is ranked first (even though the ranker is very sure)
R4	0.9998	yes	0.004	hard: second probability above 0.003

For each selected record, candidates ranked 1-4 are used if ranked first, if their probability is at least 0.001, or if they are the owner; candidates ranked 5-8 are added only if they are the owner, so the reader also sees true matches the shortlist would have missed.

TECHNICAL EXPLANATION

In a typical candidate set the gradient signal is dominated by a minority of informative pairs; uniform sampling wastes compute on pairs the model already gets right with near-zero loss. Hard-example mining (for example online hard-example mining in detection, hard-negative mining in metric learning,

uncertainty sampling in active learning) concentrates training on high-loss or high-uncertainty examples. Using the *ranker's* uncertainties and errors to choose the *readers'* training pairs makes the readers specialise where the cheaper model is weak, much as boosting focuses each new learner on the previous one's mistakes (it is a one-off selection, not a curriculum, which would order examples from easy to hard over training). Risks: label noise concentrates among hard examples; an all-hard training set distorts the base rate, so the model's raw scores no longer match real frequencies (see Calibration and reliability); and in incompletely labelled data, "hard negatives" can be unlabelled positives. Mixing in a random sample of easy examples anchors the distribution. In stacked systems the next layer can correct the distortion: our decision models learn on the natural shortlist distribution of folds 6-9 how far to trust the readers' logits.

HOW WE USED IT

- Readers (MiniLM and E5): labelled US and Indian training pairs of folds 0-5 selected exactly as above: hard records plus a random 25 % of the others; candidates 1-4 under the stated conditions, candidates 5-8 only if they are the owner.
- French E5 reader: for a French record that has any pair the teacher scored at most 0.03, the negative is the highest-scored of those pairs, the hardest confident non-match (see Self-training with pseudo-labels).
- Decision models: for the US and India only, the decision models were trained with extra hard examples: copies of unlinked training records that sit right next to a candidate business, with small name edits; how many to add was set from label-free counts on the test records, never from labels, and our documentation discloses it. The France decision model uses no such extra examples.

EXPERT NUANCE

The decision models learn to reject near-identical candidates mainly through the "what changed" features and the margin rule; the extra hard examples are an addition, not the core. Hard-example selection by the ranker means the readers see a harder mix than at test time, which is fine because their logits are only features, and the decision model, trained on unselected shortlist rows, re-learns how much to trust them. A common misconception is "more hard negatives is always better"; past a point the model becomes over-cautious and recall drops, so the amount must be controlled and validated.

REMEMBER

- Easy pairs teach little; hard negatives (look-alikes) and hard positives (true but different-looking) teach the most.
- Reader training: records where the ranker was unsure or wrong, plus a random 25 % of easy ones.
- Decision models (US/India only): extra hard examples, amount set from label-free counts, disclosed; France uses none.

JUDGE MIGHT ASK

- **Q**
"How did you choose the training pairs for the readers?"
A: From labelled US and Indian pairs of folds 0-5 we took the records where the CatBoost ranker was unsure or wrong (top probability between 0.003 and 0.9995, a wrong candidate ranked first, or a runner-up above 0.003) plus a random 25 % of the easy ones, so the readers specialise where the cheaper model struggles but still see normal cases. True owners ranked 5 to 8 were added too, so the readers also learn from matches the shortlist would miss.

- Q

"How do the decision models learn to reject near-identical candidates?"

A: Mainly through the "what changed" features and the margin rule. In addition, for the US and India only, the decision models were trained with extra hard examples: copies of unlinked training records that sit right next to a candidate business, with small name edits; how many to add was set from label-free counts on the test records, never from labels, and our documentation discloses it; the France decision model uses no such examples.

A3.19 Synthetic training pairs

IN ONE LINE

Synthetic pairs are labelled examples you create yourself instead of collecting them, used to teach or test a model on situations that are rare or unlabelled in the real data.

SIMPLE EXPLANATION

Pilots practise engine failures in a flight simulator because real failures are rare and dangerous. Synthetic training pairs are simulator practice for a matching model: cheap, plentiful, and aimed at exactly the situations you care about. The catch is that a simulator is never perfectly realistic. A model can learn simulator habits that do not occur in reality, or learn the wrong proportions of easy and hard cases. So synthetic data must always be checked against real labelled data, and the amount must be controlled. In this competition synthetic pairs were explicitly allowed (external data were not).

WORKED EXAMPLE

(toy textbook illustration of the general technique; this is not how our own extra examples were made)
From one real labelled match, "Green Leaf Cafe, 5 Park Street, Kolkata" ↔ its business:

Created pair	How	Label
"Green Leaf Cafe, 5 Park St, Kolkatta"	abbreviation and a typo	match
"GREEN LEAF CAFE, PARK STREET, KOLKATA"	upper case, house number dropped	match
record ↔ "Green Valley Cafe, 12 Camac Street, Kolkata"	paired with a different real business of the same city and trade	non-match

Then the essential check: train once with and once without the created pairs, and compare on real held-out labelled data. If the real held-out score does not improve, the synthetic pairs are not helping, however good they look.

TECHNICAL EXPLANATION

In entity resolution, **data augmentation** creates new pairs by applying label-preserving transformations (typos, abbreviations, token deletion or reordering, attribute dropping), as in the Ditto matcher's augmentation operators, or creates negatives by sampling non-matching records from the same block. It raises coverage of rare patterns and balances classes. Its risks are: **simulator bias** (the generated distribution differs from reality); **shortcut learning** (the model keys on a quirk that only the generated examples have); label noise when a transformation actually changes identity; and **prior shift** (the class mix changes, so raw scores are no longer real frequencies). Standard safeguards: validate on real held-out data only, control the mix ratio, tune thresholds on real data, and keep a model trained without augmentation as a reference. Synthetic *evaluation* sets are a separate use: they are useful where no labels exist, but they measure performance on the simulator's distribution, which may disagree with reality in either direction.

HOW WE USED IT

- Training: for the US and India decision models, extra hard examples (described generically in the previous concept): copies of unlinked training records that sit right next to a candidate business, with small name edits; how many to add was set from label-free counts on the test records, never from labels. The France decision model uses none.
- Evaluation only: a generated labelled French test set, built from the French business records, was used to check the French E5 reader (195,380 pairs, 69,910 positive). On it the French E5 reader's best F0.5 was 0.910516 against 0.914964 for its base E5 reader, so that written check failed; the blend containing the French E5 reader scored 0.90843 against 0.898031 for the blend without it.
- No external data, lookup, geocoder, postal or business table or API was used anywhere; the only hand-written resources are small word lists (Documentation, Appendix C).

EXPERT NUANCE

Our French evidence shows why synthetic evaluation needs care. On the generated French set, the French E5 reader alone scored below its base E5 reader on best F0.5 (0.910516 against 0.914964), so its written check failed, yet its AUC there was slightly higher (0.98038 against 0.979415), the blend containing it scored higher than the blend without it (0.90843 against 0.898031), and the upload that added it had our best public score. One synthetic set, read three ways, gives a mixed answer, and none of these readings is a measurement on real French labels. We must not over-read the public number either, because the same upload also added the five-seed decision models, so the reader's own effect is not isolated. For training, the safest pattern is the one we followed: the synthetic examples only change the training mix of the US and India decision models, while the thresholds of those same models are tuned on real labelled held-out folds, which absorbs the change in class mix.

REMEMBER

- Synthetic pairs were allowed; external data were banned and not used.
- We used synthetic examples in two places: extra hard training examples for US/India decision models, and a generated labelled French set for evaluation only.
- Always validate synthetic-data choices on real labelled data; a synthetic test can disagree with reality.

JUDGE MIGHT ASK

- **Q:** "Did you use synthetic data, and how did you make sure it didn't mislead the model?"
A: Yes, as the rules allowed: extra hard training examples for the US and India decision models, with the amount set from label-free counts on the test records, and a generated labelled French test set used only for evaluation. The US and India thresholds were tuned on real labelled held-out folds, not on synthetic data, and when the French E5 reader failed its check on the generated French set but the file containing it had our best public score, we kept it and disclosed that as a leaderboard-driven decision.

A3.20 Self-training with pseudo-labels

See also [A2.21 Self-training with pseudo-labels and replay](#): self-training with pseudo-labels and replay, from the models side.

IN ONE LINE

Self-training uses a model's own very confident predictions on unlabelled data as if they were labels (**pseudo-labels**) and trains on them.

SIMPLE EXPLANATION

A student who knows English moves to France and reads shop signs. When she is very sure ("Boulangerie Dupont" is "Dupont Bakery"), she treats that as a solved example and learns from it; she skips the signs she is unsure about. Over time she learns French conventions and gets better. The danger is **confirmation bias**: if she is confidently wrong about something, she trains herself to be wrong more firmly. So the safeguards are: use only very confident cases, require two "teachers" to agree, skip anything ambiguous, keep practising with real English answers, and keep some pseudo-labelled examples aside to check against. This is how our French readers learned French without a single French label.

WORKED EXAMPLE

(ours, derived from the logits in our French worked example) Teacher confidence $p = \text{mean of sigmoid}(\text{MiniLM reader logit}) \text{ and } \text{sigmoid}(\text{French MiniLM reader logit})$, where $\text{sigmoid}(x) = 1 / (1 + e^{(-x)})$.

Candidate of "TQ COMITE SARL, 2 R SUZANNE LENGLEN, NANTES"	MiniLM logit → probability	French MiniLM logit → probability	Teacher p
TQ Parents SARL, 2 Rue Suzanne Lenglen, Nantes	7.5 → 0.99945	8.9375 → 0.99987	0.99966
TQ Comite SARL, 2 Rue du Lycee, Pessac	7.125 → 0.99920	-1.9219 → 0.12765	0.56342

The rule for a positive is "the best pair has $p \geq 0.97$ **and every other pair of the record is below 0.5**". The second pair has $p = 0.563$, so this ambiguous record cannot give a positive; no pair is at or below 0.03, so it gives no negative either. The record teaches nothing, by design.

(toy) Three other records:

Record	Teacher p of its candidates	Positive?	Negative?
A	0.991, 0.012	yes: $0.991 \geq 0.97$ and the other < 0.5	yes: 0.012, the highest p among pairs ≤ 0.03
B	0.85	no (below 0.97)	no
C	0.020, 0.004	no	yes: 0.020, the hardest of the confident non-matches

TECHNICAL EXPLANATION

Self-training is one of the oldest semi-supervised methods (Scudder, 1965; Yarowsky, 1995; "pseudo-labelling", Lee, 2013; "noisy student", Xie et al., 2020). The loop: a **teacher** predicts on unlabelled target data; confident predictions are selected; a **student** is trained on them (often together with the original labels); optionally repeat. It works when confident predictions are mostly right and the errors are not systematic, so that learning from easy target examples transfers to harder ones through shared structure (here, French street types, legal forms and spellings). It fails through confirmation bias: systematic teacher errors are amplified, especially over several rounds. Mitigations, all of which we used for the French E5 reader: high confidence thresholds; combining more than one teacher (our teacher p is the mean of two readers' probabilities, so $p \geq 0.97$ forces each reader to be at least 0.94; a related idea is co-training, Blum and Mitchell, 1998, where two models trained on different views of the data label examples for each other); rejecting ambiguous groups; class balance; **replay** of labelled source data (see the next concept); held-out pseudo-labels for monitoring; and few rounds. A second, subtler bias is **selection bias**: confident examples are the easy ones, so the student sees few hard French cases. In domain-adaptation language this is unsupervised adaptation from a labelled source (US and India) to an unlabelled target (France).

HOW WE USED IT

- **French MiniLM reader**: the MiniLM reader fine-tuned (learning rate $2e-5$) on very confident pseudo-labels from an earlier run of our pipeline over French test pairs (best candidate, score at least 0.97, clear lead) and confident non-matches, plus 500,000 labelled US and Indian pairs.

- **French E5 reader:** the E5 reader fine-tuned (learning rate $2e-5$, one epoch, 7,419 steps) with teacher $p = \text{mean}(\text{sigmoid}(\text{MiniLM logit}), \text{sigmoid}(\text{French MiniLM logit}))$ on the France shortlist; positive = the best pair with $p \geq 0.97$ and every other pair below 0.5; negative = the highest- p pair with $p \leq 0.03$; at most 250,000 per class; 5 % of records held out; 474,808 French pairs plus 474,808 labelled US and Indian pairs.
- **Checks of the French E5 reader:** agreement with held-out French pseudo-labels 0.999206 on 25,192 pairs (the E5 reader before French training: 0.981304), above the 0.98 bar; US and Indian quality within tolerance; it failed its two other written checks (see Written acceptance checks before keeping a model). A second self-training round for it was tried and not adopted (public 0.991277 against 0.991300).
- We have no measured French F0.5, and we say so.

EXPERT NUANCE

The agreement check measures consistency with the teacher, not truth: a student that copies the teacher's mistakes also passes it. That is why the other checks exist (US and Indian labels for forgetting, a generated labelled French set), and why we never call 0.999206 an accuracy. A fair challenge a scientist may raise: the French readers then score the same French pairs they were trained on, so for the pseudo-labelled pairs their scores are in-sample, the very effect our fold design avoids elsewhere. The effect is limited by construction: the French E5 reader was trained only on pairs its teachers were at least 97 % sure of or at most 3 % sure of, and the French MiniLM reader likewise only on confident pseudo-labels (positives at a score of at least 0.97 with a clear lead, negatives at low scores), so the uncertain middle, where decisions are actually made, was never trained on; and the MiniLM reader, which never saw a French pair, keeps 0.35 of the blend. We did not measure the size of the effect.

REMEMBER

- Pseudo-labels = the model's own very confident guesses used as labels; the risk is confirmation bias.
- French E5 reader: $p \geq 0.97$ with every rival below 0.5 for positives; hardest pair ≤ 0.03 for negatives; 474,808 + 474,808 pairs; 5 % held out.
- Agreement with held-out pseudo-labels (0.999206) is consistency, not accuracy; no French F0.5 exists.

JUDGE MIGHT ASK

- **Q**
"How do you stop self-training from amplifying its own mistakes?"
A: We cannot rule it out completely without French labels, so for the French E5 reader we limited it: only pairs both MiniLM readers together were at least 97 % sure of, with every rival below 50 %, or at most 3 % sure for negatives; at most one positive and one negative per record; an equal number of labelled US and Indian pairs mixed in; 5 % of records held out to check; and a single round, because a second round did not improve the public score (0.991277 against 0.991300).
- **Q**
"Your French readers score the same French pairs they were trained on. Isn't that the in-sample over-confidence your fold design avoids elsewhere?"
A: For the pseudo-labelled pairs, partly yes. But they were only confident pairs, for the French E5 reader those its teachers were at least 97 % or at most 3 % sure of, so the uncertain pairs where decisions are made were never trained on, and the MiniLM reader, which never saw French data, keeps 0.35 of the blend. We did not measure the size of the effect.

A3.21 Replay and catastrophic forgetting

See also [A2.21 Self-training with pseudo-labels and replay](#): self-training with pseudo-labels and replay.

IN ONE LINE

When fine-tuning a model on a new domain, keep mixing in labelled examples from the old domain (**replay**) so that it does not forget what it already knew.

SIMPLE EXPLANATION

A musician learning jazz keeps practising classical pieces so the old skills stay sharp. Neural networks trained only on new data tend to overwrite the weights that held their old skills, a failure called **catastrophic forgetting**. Our French readers started from readers trained on US and Indian labels. While learning from French pseudo-labels, they also re-read real labelled US and Indian pairs, in about equal numbers. This does two jobs: it keeps the old skill (matching names and addresses in general) and it anchors the student with true labels, which partly protects it against the teacher's mistakes.

WORKED EXAMPLE

(ours) The French E5 reader's training set and its forgetting check:

Item	Value
French pseudo-labelled pairs	474,808
Labelled US and Indian pairs (replay)	474,808
Total pairs; batch size	949,616; 128
Steps for one epoch	$949,616 / 128 = 7,418.9$, so 7,419 steps

Forgetting check on 509,260 labelled pairs of 437,715 fold-8 and fold-9 US and Indian records (the readers were trained on folds 0-5, so these are unseen):

Measure	French E5 reader	E5 reader (before French training)	Allowed drop	Result
AUC	0.998544	0.998570	0.001	pass
Top-1 accuracy	0.993161	0.993004	0.002	pass
Best F0.5	0.992782	0.992796	0.002	pass

The French training cost almost nothing on US and Indian data: AUC moved by 0.000026 (derived), far inside the allowed 0.001.

TECHNICAL EXPLANATION

Catastrophic forgetting (McCloskey and Cohen, 1989) happens because gradient updates for the new task move shared weights without any term that preserves the old task's loss. Remedies: **rehearsal or replay** (mix stored old examples into new training); regularisation towards the old solution (elastic weight consolidation, which penalises changes to important weights, or an L2 penalty towards the starting weights); a lower learning rate and fewer steps; freezing layers; or parameter-efficient adapters such as LoRA, which leave the base weights untouched. The replay ratio is a trade-off: more replay protects old skills and dilutes the new domain; about 1:1 is a common, conservative choice. Measuring forgetting requires held-out labelled data from the old domain that neither the original training nor the replay used.

HOW WE USED IT

Replay plus a lower learning rate ($2e-5$ instead of the original $5e-5$) plus a single epoch, for both French readers: 474,808 labelled pairs for the French E5 reader (exactly 1:1) and 500,000 labelled pairs for the French MiniLM reader (about one to one). The replay pairs come from folds 0-5, the readers' original training folds, so the forgetting check on folds 8 and 9 is clean.

EXPERT NUANCE

Replay also carries knowledge that transfers across countries, such as how legal forms behave (Inc against LLC in the US is analogous to SARL against SAS in France), so it is not only a defence but also a teacher. Without replay, a student trained purely on confident French pseudo-labels would learn a distribution made only of easy French cases, and its scores on hard pairs could drift. The forgetting check shows the old skill was kept; it does not show the French skill improved, which is what the other checks address.

REMEMBER

- Catastrophic forgetting = new training overwrites old skills; replay mixes old labelled examples back in.
- French E5 reader: 474,808 French + 474,808 labelled US/Indian pairs, learning rate $2e-5$, one epoch, 7,419 steps.
- Forgetting check passed: AUC 0.998544 against 0.998570, tolerance 0.001.

JUDGE MIGHT ASK

- **Q:** "Why mix US and Indian pairs into the French fine-tuning, and how do you know it worked?"
A: To stop the readers forgetting what they knew and to anchor them with real labels while they learn from French pseudo-labels; the French E5 reader was trained on 474,808 French and 474,808 labelled US and Indian pairs. On unseen labelled fold-8 and fold-9 pairs its AUC was 0.998544 against 0.998570 before French training, inside our written tolerance of 0.001.

A3.22 Blending readers in logit space

See also [A2.22 Blending readers in logit space \(France\)](#): blending readers in logit space, from the models side.

IN ONE LINE

Combine several readers into one score by a weighted average of their **logits** (raw scores before the sigmoid), which is what the French decision model receives in its single reader slot.

SIMPLE EXPLANATION

Imagine three experts who each give an opinion on a scale from "certainly not" (−10) through "no idea" (0) to "certainly yes" (+10). Averaging those raw opinions respects how strongly each expert feels: one very firm "no" can outweigh a mild "yes". Converting each opinion to a percentage first and then averaging would squash the firm opinions, because 99.9 % and 99.99 % look almost the same as percentages. For France, the decision model has one input slot for "the reader's opinion". During training (on US and Indian data) that slot always held the MiniLM reader's logit. In France it receives a blend: $0.35 \times \text{MiniLM} + 0.15 \times \text{French MiniLM} + 0.5 \times \text{French E5}$.

WORKED EXAMPLE

(ours) Our French worked example, "TQ COMITE SARL, 2 R SUZANNE LENGLEN, NANTES":

Candidate	MiniLM	French MiniLM	French E5	Blend = $0.35 \times \text{MiniLM} + 0.15 \times \text{French MiniLM} + 0.5 \times \text{French E5}$
TQ Parents SARL, same address in Nantes	7.5000	8.9375	10.8125	$2.625 + 1.341 + 5.406 = 9.3719$
TQ Comite SARL, Pessac (another city)	7.1250	−1.9219	−10.8125	$2.494 - 0.288 - 5.406 = -3.2008$

The MiniLM reader alone liked the identical name in another city (+7.1); the French readers firmly rejected it, and the blend says a clear "no" (−3.2).

(derived) Compare averaging probabilities for the Pessac candidate: $0.35 \times 0.99920 + 0.15 \times 0.12765 + 0.5 \times 0.00002 = 0.369$, whose logit is −0.54: only a weak "no", because the French E5 reader's firm −10.8 became a probability near zero and lost its strength.

TECHNICAL EXPLANATION

A weighted average of logits is **log-linear pooling**: the blended odds are a weighted geometric mean of the individual odds, so confident experts count strongly, much like a product of experts. Averaging probabilities (linear pooling) is more conservative and is dominated by moderate opinions. The units matter for a second reason: decision trees split on absolute input values, and the France decision model was trained with MiniLM *logits* in that slot, so the test-time input must be in the same units and on a similar scale; a blend of logits stays in logit units, while a blend of probabilities would not. Logit scales of different readers are comparable but not identical (in this example E5-based logits reach ± 10.8 while the MiniLM reader's stay near 7.5), so the blend changes the slot's distribution somewhat: a covariate shift for the decision model that the conservative 0.82 threshold helps absorb. With labels, the weights would be fitted (for example by logistic regression on held-out logits, i.e. stacking); without French labels they must be set by other means.

HOW WE USED IT

France only; the plain E5 reader is not used for France. The 0.35 / 0.15 / 0.5 weights equal $0.5 \times$ the previous blend (0.7 MiniLM + 0.3 French MiniLM) + $0.5 \times$ French E5. The French MiniLM reader's weight in the previous blend was set by label-free counts on the test records; the equal split between the previous blend and the French E5 reader was fixed in a written rule before any French decision result with the new reader existed, and that rule also recorded the reason for blending logits: the decision model's reader input is a logit. For the US and India no hand-set blend is needed: the decision models receive the MiniLM and E5 logits as separate features, plus their mean and, for each, its rank, best other, margin and top value within the record's shortlist, and learn the combination from labels.

EXPERT NUANCE

Feeding a blend into a slot trained on one reader's logits is a deliberate approximation: the France decision model never saw the blend during training, so keeping the slot in logit units and using the conservative 0.82 threshold are what keep the approximation safe. The French example is illustrative, not proof: test labels are private, so it shows how the blend decides, not that it is right. Without labels, weight setting is the weakest link of the French pipeline; we disclose how each weight was set.

REMEMBER

- France reader slot = 0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5, in logits.
- Logit averaging keeps firm opinions strong and keeps the units the decision model was trained on.
- Weights: French MiniLM share from label-free counts; the 50/50 split with the French E5 reader fixed in a written rule before its effect on French decisions was known.

JUDGE MIGHT ASK

- **Q:** "Why average logits rather than probabilities, and how did you set the weights without labels?"
A: The France decision model was trained with the MiniLM reader's logit in that slot, so the blend must stay in logit units, and logit averaging lets a firm "no" count fully: in our French example it turns an identical name in another city into -3.2 instead of a weak -0.5 . The French MiniLM share was set by label-free counts on the test records, and the equal split with the French E5 reader was fixed in a written rule before we saw its effect on French decisions.

A3.23 Weak supervision and rules computed by code

IN ONE LINE

Weak supervision means using programmatic knowledge (rules, heuristics, word lists) instead of hand-labelled examples when labels are missing; our France rules are a related but simpler idea:

fixed rules, written as code, that veto or rescue links after the decision model.

SIMPLE EXPLANATION

With no answer key, an expert can still write rules of thumb such as "if the record is at the same address as the business but one word of the name was swapped for a common category word like 'comité', it is probably a different organisation". In classic weak supervision (for example the Snorkel system), many such rules vote on unlabelled examples, a statistical model estimates how reliable each rule is, and the resulting noisy labels train a normal model. We did something simpler and more direct for France: each rule is a piece of code that reads the decision model's scores, the raw text and the address, and either removes a link (a **veto**) or adds one (a **rescue**). No human or AI judges any pair when the pipeline runs.

WORKED EXAMPLE

(ours) The category-swap veto on our French worked example:

Step	Check	Result
Decision model	best candidate "TQ Parents SARL, 2 Rue Suzanne Lenglen, Nantes", score $0.999837 \geq 0.82$, lead $0.999740 \geq 0.4$	accepted
Same house number?	2 and 2	yes
Same street?	street token-set ratio $100 \geq 80$ (a 0-100 similarity of the two sets of street words that ignores word order; 100 = same words)	yes
Business lost a core word?	"parents" is missing from the record	yes
Record gained a category word?	"comite"	yes
Rule fires	the link is vetoed	record left unlinked

(ours) The whole France chain, in execution order:

Step	Rule	Action	France links after
0	decision model (0.82 / 0.4)	887,630 accepted	
1-2	category-swap veto; word-substitution veto and rescue	-29,021 vetoed together; +5,272 rescued	
3	residual rescue	+5,846	869,727
4	street veto	-826	868,901
5	generic-word swap veto	-538	868,363
6	generic-word rescue	+332	868,695
7	cell veto	-1,322	867,373
8	acronym rescue	+186	867,559 (final)

(toy) Classic weak supervision for comparison: three labelling functions vote on one pair, "same name and same street → match", "different city → non-match", "acronym of the business name → match"; votes are match, abstain, match, so the label model outputs, say, 0.8 probability of match, and that soft label trains a classifier.

TECHNICAL EXPLANATION

In data-programming style weak supervision, **labelling functions** with unknown and correlated accuracies are combined by a **label model** that estimates their accuracies from their agreement and disagreement patterns, producing probabilistic labels for training a discriminative **end model** that generalises beyond the rules' coverage. Our France rules are instead **deterministic post-model decision rules**: they act on the decision model's output, they do not create training labels, and their order matters (the counts above are actions in execution order, not per-rule accuracies). Without labels, a rule can be checked with **label-**

free control groups (compare the population a rule acts on with a matched population it does not act on, using statistics that need no labels), with independence or invariance checks, and by transferring facts measured on labelled countries. Example of the last (from the code's documentation of the street veto): among 1,313,608 decidable true US top-1 matches, only 48 (0.004 %) have a street name sharing no word with the business's street, while 60,565 of 60,613 (99.92 %) US top-1 pairs with a genuinely different street are wrong. For the cell veto, our Documentation cites the independence of name noise and address noise measured on US labels. The main risk of hand-written rules is fitting the data you looked at; the standard cure is to turn rule conditions into features and learn them from labels.

HOW WE USED IT

France only, eight rules in the order above, acting on France shortlist pairs and adding no candidates. Limitation, said first: the rules were designed after analysing the unlabelled French test records, including AI-assisted review of sampled pairs (about 1,000 pairs in five review samples, plus smaller checks), and their parameters were chosen after seeing test results, including public-leaderboard scores. Mitigation: each rule is code on model scores, raw text and the address; no test label and no human or AI judgement of a pair is an input to the pipeline; the evidence is label-free control groups and facts measured on US labels; everything is disclosed. The largest action, the two word-swap vetoes together, removes 29,021 of 887,630 accepted French links (3.3 %, derived).

EXPERT NUANCE

Rules give precise, auditable control where a model is unsure, but they do not generalise beyond their written conditions and they invite overfitting to the analysed data; that is the honest cost. In production we would turn each rule condition into a decision-model feature and learn its weight from a small labelled French sample, keeping only the rules that still add value. "Rules computed by code" is also the precise answer to "did humans label the test set?": no human or AI decision about any pair enters the pipeline.

REMEMBER

- Our France rules are deterministic code on scores, raw text and address: vetoes and rescues after the decision model.
- France links: 887,630 accepted → 867,559 after eight rules; the two word-swap vetoes remove 29,021.
- Limitation first: designed after analysing unlabelled French test records, some parameters chosen after public scores; disclosed.

JUDGE MIGHT ASK

- **Q**
"How did you check a France rule without any French labels?"
A: With label-free control groups on the French records and with facts measured on US labels; for example, the street veto rests on the US finding that only 48 of 1,313,608 true top-1 matches have a street sharing no word with the business's street. We still have no French F0.5 for the rules, and we disclose that they were designed after looking at unlabelled French test records.
- **Q**
"Did an AI choose your France rules?"
A: AI coding assistants wrote and ran code and analysis scripts during development, and for the three later rule changes they also chose the cell-veto combinations and the thresholds and wrote the rule code, which the team adopted; our documentation states this. At run time no AI or human judgement of any pair is an input: every rule is code on model scores, raw text and the address.

A3.24 Domain shift and unseen countries

IN ONE LINE

Domain shift is when the data a model meets in use differ from the data it learned from; a country the model has never seen is the extreme case.

SIMPLE EXPLANATION

A doctor trained in Chennai who moves to Paris knows medicine, but the forms, the abbreviations, the names and even how common each illness is are different. Our models learned on US and Indian records. France has different address formats ("15 bis", "R" for "Rue", postcodes, department names), different legal words (SARL, SAS, EURL, SCI) and, crucially, no labels. There are three kinds of change to watch: what the inputs look like (**covariate shift**), how common matches are (**prior** or **label shift**), and what counts as a match for a given input (**concept shift**). Our French plan answers each one: clean the French format into a familiar shape, let readers adapt themselves on French text, use a stricter bar, and add careful rules. For a country we have never seen at all, there is a cautious default route.

WORKED EXAMPLE

(toy) Covariate shift in the address:

Raw French address	Problem for a US/India-trained model	After French cleaning (our rules)
"15B R DE LA PAIX, PARIS"	"15b" is an unknown token and "r" is not a recognised street type	"15 bis", and "R" and "Rue" mapped to one street-type form, so the number feature sees 15 and the street words line up

(toy) Prior shift: suppose 60 % of training records have a true business but only 40 % do in a new country. A threshold tuned where matches are common produces too many links where they are rare, because more of the borderline candidates are wrong there. A stricter threshold is the simple defence.

(ours) How much France matters: our diagnostic upload that emptied every French row dropped the public score from 0.970125 to 0.839.

TECHNICAL EXPLANATION

Under covariate shift ($P(x)$ changes, $P(y | x)$ fixed) a model can stay correct where the domains overlap, and importance weighting by $P_{\text{target}}(x) / P_{\text{source}}(x)$ is the textbook correction when the supports overlap. Under prior shift ($P(y)$ changes, $P(x | y)$ fixed), posteriors must be re-weighted and thresholds moved; the target prior can be estimated without labels by expectation-maximisation (Saerens et al., 2002). Under concept shift no label-free method is reliable; target labels are needed. Label-free detection tools include comparing feature and score distributions, **adversarial validation**, and monitoring output volumes (links per business). Practical adaptation without target labels combines input normalisation (make the target look like the source), self-training on the target (see Self-training with pseudo-labels) with replay of source labels (see Replay and catastrophic forgetting), conservative operating points, and targeted rules. The key risk is that every check is then indirect: proxies on the source domain, agreement statistics and synthetic sets can all disagree with the truth.

HOW WE USED IT

- France: French cleaning rules (region, department and city components learned from the unlabelled test files; "N°" removed; "15B" → "15 bis"; bis / ter / quater unified; street types and their typos canonicalised; "st" / "ste" → "saint" / "sainte"; dotted and leet legal forms collapsed, e.g. "S.A.R.L." → "sarl"); the same index, quick filter, ranker and shortlist (ranker trained on US and India); the MiniLM reader plus two self-trained French readers, blended; the France decision model trained on US and Indian rows of the normal training set (no French rows, no extra hard examples, 158 features, mean of

five seeds); threshold 0.82 set without labels from French link volume per business together with a fold-8 threshold sweep on US and Indian data; lead 0.4; then the France rules computed by code.

- Proxy evidence: the France recipe (one fit) applied to US and Indian held-out businesses at 0.82 scores 0.990508 / 0.989467 on fold 9 and 0.990977 / 0.989850 on fold 8; this shows the recipe works where labels exist, not that it works in France. We have no measured French F0.5.
- Unseen countries: the default route, with the US/India cleaning rules, the MiniLM reader only, the default decision model (158 features, trained on US and Indian rows, a single fit), threshold 0.82 and lead 0.4, and no second-chance searches or rules. The test set contained only the US, India and France, so this route is untested on real data.

EXPERT NUANCE

Thresholds are the first thing that breaks under shift, because the meaning of a score changes when calibration changes (see Calibration and reliability); choosing a stricter 0.82 is a hedge, not a measurement. The default route is the honest minimum: a multilingual reader that never saw the country, a decision model trained without its labels, and a cautious bar; we would not deploy it to a real new market without first adding a country normaliser, self-training readers on the new country's unlabelled records, and labelling a small sample to set the threshold. A common misconception is that a multilingual encoder solves the problem; it handles scripts and words, not address formats, legal vocabularies or match rates.

REMEMBER

- Three shifts: inputs (covariate), match rate (prior), meaning of a match (concept).
- France: French cleaning, self-trained French readers, US/India-trained decision model, stricter 0.82, rules by code; no measured French F0.5.
- Unseen country: default route (MiniLM reader only, single-fit decision model, 0.82 / 0.4), untested on real data.

JUDGE MIGHT ASK

- **Q**
"How do you know the France decision model works when it never saw a French label?"
A: We do not have a French F0.5, and we say so. What we have is indirect: the same recipe applied to US and Indian held-out businesses at the stricter 0.82 bar scores 0.990508 and 0.989467 on fold 9, the French E5 reader passed its US/India tolerance and pseudo-label agreement checks, and a diagnostic upload showed French rows carry real weight (emptying them dropped the public score from 0.970125 to 0.839).
- **Q**
"What happens with a country you have never seen?"
A: It takes the default route: the decision model built like France's (158 features, trained on labelled US and Indian rows, a single fit), the multilingual MiniLM reader only, and the conservative 0.82 bar. The test set had no such country, so it is untested; for a real new market we would add a country normaliser, self-train readers, and label a small sample to set the threshold.

A3.25 Model selection and public-leaderboard overfitting

IN ONE LINE

Every time you choose between versions by looking at the same evaluation scores, you fit those scores a little; after many uploads, the best public score is biased upwards.

SIMPLE EXPLANATION

Flip 18 ordinary coins ten times each and pick the one that showed the most heads: it looks lucky, but it is an ordinary coin, and it will not keep winning. Leaderboard scores contain noise too, because each is computed on one particular set of test records. If you upload many versions and keep the best, you reward both real improvements and luck. This is **adaptive overfitting**, also called the **winner's curse**. The defences are: decide mainly with your own held-out folds, treat tiny public differences as ties, limit the number of uploads, and be open about which decisions used the leaderboard.

WORKED EXAMPLE

(toy) Three files of exactly equal true quality, 0.9910, with random public noise:

File	True quality	Public score	Chosen?
A	0.9910	0.99112	yes, the best
B	0.9910	0.99094	no
C	0.9910	0.99105	no

You would report 0.99112, but 0.00012 of it is luck.

(ours) Our upload history, key rows:

Upload	Change	Public F0.5
2	first scored baseline	0.968212
6	first decision-model submission	0.982561
7	France adaptation, first France rules	0.990556
9	+ US/India decision-model refit on folds 6-9	0.990676
10	+ India address second-chance search	0.990748
11	France decision-model refit (not adopted)	0.990726
14	generic decision-model inputs (data-specific features removed)	0.990934
15	further France rule changes	0.991114
16	five-seed decision models + French E5 reader: final file	0.991300
17	second French self-training round (not adopted)	0.991277
18	French MiniLM reader retrained with different pseudo-label filters (not adopted)	0.991258

The late steps are small: +0.000186 from upload 15 to 16, and the dropped variants trailed by 0.000022, 0.000023 and 0.000042 (all derived).

TECHNICAL EXPLANATION

If each public score equals the true score plus independent noise with standard deviation σ , the expected maximum of N equally good versions exceeds the true score by $\sigma \times E[\max \text{ of } N \text{ standard normals}]$, about 1.8σ for $N = 18$ (a standard order-statistics value). Real versions differ in true quality and their noise is correlated (they share most predictions), so the real inflation is smaller, but it is never zero, and it grows with the number of decisions taken on the same data. Adaptive data analysis (Dwork et al., 2015, the "reusable holdout") formalises this: each query to a holdout leaks information, and safe reuse requires limiting or noising the answers. Practical mitigations: use your own held-out folds for most decisions; pre-register acceptance rules (see Written acceptance checks before keeping a model); keep the number of final candidates small; prefer the simpler or existing version when public differences are within noise; and report how the leaderboard was used. A single public number cannot be bootstrapped, so its noise level is unknown to the participant.

HOW WE USED IT

Held-out folds or written evidence decided the quick rule-based filter (fold-9 F0.5 0.990117 → 0.990086), the 0.75 threshold (fold 8), the consistency check (folds 8 and 9), the second-chance thresholds (fold 8, false positives counted twice) and the French E5 reader's checks. Public scores decided the US/India refit (0.990621 → 0.990676), dropping the France refit (0.990726 against 0.990748), dropping the second French self-training round (0.991277 against 0.991300), the French diagnostic (upload 5), the calibration of label-free estimates for the US address second-chance search and for the generic-word rescue (which then selected two further rescue families), some France rule parameters, keeping the French E5 reader against its own written rule, and the final pick among 18 uploads. All of this is disclosed.

EXPERT NUANCE

Limitation first: the final file was chosen among 18 uploads by public score, so 0.9913 carries some selection bias. What limits it: on held-out folds, close versions of our decision models score in the same range (US 0.991212 / 0.991545, India 0.990046 / 0.990298), and the late public gains were small (0.991114 → 0.991300). France carries the most risk, because its rules were tuned with public feedback and it has no labels. Several of our "not adopted" decisions rested on public differences of a few hundred-thousandths, which may well be within noise; the defensible reading is "no evidence of improvement, so we kept what we had", not "the variant was worse".

REMEMBER

- Picking the best of many noisy scores inflates it (about 1.8σ for 18 equal versions).
- Held-out folds set the thresholds; public scores chose the refit, the dropped variants, some France parameters and the final file.
- Mitigation: held-out scores in the same range; late public gains small (0.991114 → 0.991300).

JUDGE MIGHT ASK

- **Q:** "With 18 uploads, isn't 0.9913 an overfit number?"
A: Partly, and we say so: the final file was chosen among 18 uploads using public scores, and public feedback also shaped some France rule parameters. What limits the damage is that the thresholds came from held-out folds, close versions of our decision models score 0.990 to 0.992 on those folds, and the late public gains were small, from 0.991114 to 0.991300; France is where the risk is highest.

A3.26 Written acceptance checks before keeping a model

IN ONE LINE

Before seeing a new model's results, write down the tests it must pass to be adopted, then apply them as written, and disclose any exception.

SIMPLE EXPLANATION

Teams agree the rules of a match before kick-off, not after seeing the score. If you decide what counts as "better" after you have seen the numbers, it is very easy to find a reason to keep the version you like; scientists call this the **garden of forking paths**. Writing the checks first is called **pre-registration**. It does not make a model good, but it makes your decision honest. And if you break your own rule, you say so plainly, which is what we did for the French E5 reader.

WORKED EXAMPLE

(ours) The four written checks of the French E5 reader:

Check (plain name)	Data	Result	Outcome
US/India quality within tolerance	509,260 pairs of 437,715 labelled fold-8/9 records	French E5 vs E5: AUC 0.998544 vs 0.998570; top-1 0.993161 vs 0.993004; best F0.5 0.992782 vs 0.992796; allowed drops 0.001 / 0.002 / 0.002	pass
Agreement with held-out French pseudo-labels	25,192 pairs (12,616 positive)	0.999206 (E5 reader 0.981304); bar 0.98	pass
Generated labelled French test set	195,380 pairs (69,910 positive)	best F0.5 0.910516 vs 0.914964 for the E5 reader	fail
Expected gain must be safely positive (label-free, final file vs the same pipeline without the French E5 reader)	French links -2,228 / +3,396 (5,437 business rows changed); US, India and candidates identical	point estimate +0.0000302; pessimistic -0.0001806; optimistic +0.0002315	fail

The last rule needed two things: the pessimistic estimate above 0, and the point estimate $\times 0.75$ at least 0.00001. Here $0.0000302 \times 0.75 = 0.0000226 \geq 0.00001$ (passes), but -0.0001806 is below 0 (fails), so the rule fails.

TECHNICAL EXPLANATION

Good acceptance checks are chosen to catch different failure modes. A **guardrail** or **non-inferiority** check verifies that nothing important got worse beyond a stated tolerance (here, no forgetting on labelled US and Indian data). A **fit** check verifies that the model learned its target (agreement with held-out pseudo-labels). An **external-validity** check tests on data from a different source (the generated French set). An **end-to-end utility** check estimates the effect on the real output, ideally with a **lower confidence bound** above zero so that noise alone cannot pass it. Fixing tolerances and bars before the results removes the freedom to move goalposts. A useful companion practice is a **harness control**: before measuring a change, confirm that an unchanged copy of the pipeline reproduces the previous outputs byte for byte, so any difference is caused by the change alone (our written rule files require this).

HOW WE USED IT

New readers faced written pass-or-fail checks, and our evidence folder also holds rules written in advance for the five-seed decision models and the French readers. The French E5 reader passed two checks and failed two. We kept it because the file containing it had our best public score: a leaderboard-driven decision against our own written rule, and our documentation discloses it as such. It changes French links only ($-2,228 / +3,396$), and the same upload also added the five-seed decision models, so its own effect on the public score is not isolated. Two later variants, a second French self-training round and a French MiniLM reader with different pseudo-label filters, were not adopted.

EXPERT NUANCE

A rule that is overridden loses part of its protective value; the remedy is full disclosure, which turns a hidden choice into a documented one that others can judge. Note what each check can and cannot show: the agreement check measures consistency with the teacher, not truth; the generated set measures performance on a simulated distribution; and the expected-gain rule was label-free and wide (from -0.0001806 to $+0.0002315$), so neither its pass nor its fail is decisive. One precise caveat that our Documentation states itself: the expected-gain rule for the French E5 reader is in our evidence folder with its date, written before any French decision result with the new reader existed, but the original pre-training declaration of its other three checks is not included; their tolerances are restated in a later rule file. If asked about pre-registration in general, say: the rules were written before the results they judge, the files record every deviation, and where an original declaration is missing the Documentation says so.

REMEMBER

- Write the pass/fail tests before the results; apply them as written; disclose exceptions.

- French E5 reader: passed US/India tolerance and pseudo-label agreement; failed the generated French set and the expected-gain rule; kept for the best public score, disclosed.
- Checks target different failures: forgetting, learning, external validity, end-to-end gain.

JUDGE MIGHT ASK

- **Q:** "If you wrote rules in advance, why did you break one?"
A: Because the file containing the French E5 reader had our best public score; that was a leaderboard-driven decision against our own written rule, and we disclose it. It passed two checks (US/India AUC 0.998544 against 0.998570; 0.999206 agreement with held-out French pseudo-labels) and failed two (the generated French test set, 0.910516 against 0.914964, and the expected-gain rule, pessimistic -0.0001806), and it changes French links only.

A3.27 Validation hygiene: recall at every stage, assertions and the validator

IN ONE LINE

Measure what every stage loses, assert the invariants the output must satisfy, and run the official checker, so errors are caught where they happen rather than discovered at the end.

SIMPLE EXPLANATION

In a factory you inspect after each machine, not only at the end; if the final product is faulty, you know which machine broke it. For our pipeline: after each candidate-finding step we measure how often the true business is still among the candidates (**recall**), because a business lost early can never be linked later. We also check rules that must always hold: every link we output is also in our candidate list, no record is linked to two businesses, no pair crosses countries, and each output file has exactly one row per test business. Finally we run the organisers' validator. This is unglamorous, but it is what lets a team trust its own numbers.

WORKED EXAMPLE

(ours) Recall on a fold-9 sample (share of owned incoming records whose true business is still a candidate), with candidates per incoming record from the all-fold samples:

Stage	US recall	India recall	US candidates per record	India candidates per record
Index lookup	99.3658 %	98.388 %	27.541	23.145
After the quick rule-based filter	99.3523 %	98.3745 %	3.111	3.878
Shortlist (at most 4)	98.8935 %	97.985 %	1.114	1.174

Derived losses: the quick filter costs 0.0135 points of recall in both countries while cutting candidates per record from 27.5 to 3.1 (US) and from 23.1 to 3.9 (India); the cap of four costs 0.4588 points (US) and 0.3895 points (India). So the cap of four is the largest loss, which is exactly what our documentation says, and it tells you where a bigger budget would help most.

(ours) End-to-end cost of the quick filter, measured in the blocking study of 25 Sep 2026: the true business survived for 99.974 % of fold-8/9 records, and fold-9 macro F0.5 moved from 0.990117 to 0.990086 (fold 8: -0.00004).

TECHNICAL EXPLANATION

In record-linkage terms, **pair completeness** is the recall of blocking (the share of true pairs among candidates) and the **reduction ratio** is $1 - (\text{candidate pairs} / \text{all possible pairs})$; our reduction ratio is 0.99999817 on 6,724,569,566,212 same-country pairs. Recall at each stage is a ceiling on final recall, so measuring it localises losses and quantifies the trade-off between candidates per record and recall. Recall alone is not enough: the stronger test is the effect on the end metric, as in the quick-filter measurement.

Invariant assertions encode properties that must hold whatever the model does; failing loudly at the point of violation is far cheaper than discovering a silent error after an upload. A **metric implementation** must match the official one: our per-business F0.5 code counts each business's true records from the label table, so owners lost by the search are counted as misses. Code-level guards complete the picture: a check that asserts none of the 21 removed data-specific inputs is used by any final decision model, and fold files recorded with their hashes.

HOW WE USED IT

Recall is measured at every candidate step on labelled fold 9 (full fold: index 99.3694 % US / 98.5068 % India; shortlist between 98.8748 % and 99.3628 % US and between 98.0572 % and 98.4831 % India, given as lower and upper bounds). The export asserts that every matched pair is a candidate pair; no pair joins different countries and no incoming record is linked to more than one business; the last stage runs the organisers' validator with `--check-ids`, result PASS: 1,732,544 rows in each file, 100,087 empty and 1,632,457 non-empty rows in the matching file, 125 empty and 1,732,419 non-empty rows in the candidate file. In our written experiment rules, an unchanged copy of the pipeline had to reproduce the previous outputs byte for byte before a change was measured.

EXPERT NUANCE

Many teams measure only the final score, and then cannot say whether a loss came from retrieval or from the decision. Measuring recall per stage is also what justified our design: the quick filter is almost free, the cap of four is the main loss, and the two second-chance searches target the main remaining miss types (native-script names, and same address with a different name). Recall numbers exclude the second-chance candidates, so they are conservative for the final candidate file.

REMEMBER

- Recall at every step: index 99.37 % US / 98.51 % India on fold 9; the cap of four is the largest loss.
- Assertions: every link is a candidate pair; one business per record; no cross-country pairs; validator PASS.
- End-metric checks beat recall alone: the quick filter moved fold-9 F0.5 only from 0.990117 to 0.990086.

JUDGE MIGHT ASK

- **Q**
"How do you know your search doesn't silently lose true matches?"
A: We measure recall at every candidate step on labelled fold 9: 99.37 % of US and 98.51 % of Indian owners survive the index, at least 98.87 % and 98.06 % the shortlist, and the quick filter keeps the true business for 99.974 % of held-out records. The cap of four is the largest loss, and the two second-chance searches target the remaining miss types.
- **Q**
"What guarantees that your output files are internally consistent?"
A: The export asserts that every link is also a candidate pair, and the output has no record linked to two businesses and no pair that crosses countries. The final stage then runs the organisers' validator with the id check, which passed with 1,732,544 rows in each file.

A3.28 Determinism and byte-exact reproducibility

IN ONE LINE

Running the same code on the same data should give the same output; seeds, pinned versions and stored intermediate results make that possible, but GPU arithmetic can still differ in the last digits.

SIMPLE EXPLANATION

A recipe followed twice should give the same cake. In computing, hidden sources of randomness can change the result: random sampling and initialisation, the order in which parallel processors finish their sums, and even how Python orders text in a set. A **seed** fixes the random choices. But on a GPU, thousands of threads add numbers in whatever order they finish, and because computers round every result, $(a + b) + c$ is not always exactly $a + (b + c)$. Tiny differences in the CatBoost ranker change a few shortlists, and those ripple through every later stage. So we offer two kinds of reproduction: from the trained models and stored scores, exact to the byte; from the raw data, very close.

WORKED EXAMPLE

(toy) Rounding makes order matter. In 32-bit floating point, numbers near 100,000,000 are spaced 8 apart, so:

Order	Result
$(100,000,000 + 1) - 100,000,000$	0 (the + 1 is rounded away)
$(100,000,000 - 100,000,000) + 1$	1

In **bf16** (a 16-bit number format; our readers were trained with bf16 autocast, which runs most of the arithmetic in it), numbers near 1 are spaced $2^{-7} = 0.0078125$ apart, so $1 + 0.001 = 1$. Sums done in a different order, or with different padding, therefore differ in the last digits.

(ours) What reproduction achieved:

Route	Result
From the trained weights and the stored arrays of the submitted run (the stages that install the French E5 scores, score the France decision model and export the files)	both files rewritten byte for byte; a fresh extraction of the full archive on 29 Sep reproduced both files byte for byte with the inference-only command
Clean run from raw data, 27-28 Sep, every model retrained	validator PASS; identical match list for 99.07 % of businesses; 99.82 % of submitted links reproduced (5,846,344 in common); 12,334,519 candidate pairs; France shortlist 2,122,285 pairs
CPU retraining of the India native-script and address second-chance models on the same inputs	byte for byte
Output file hashes (SHA-256)	matching file 9183e542...; candidate file e7b4c679...

TECHNICAL EXPLANATION

Sources of non-determinism: unseeded random number generators (Python, NumPy, PyTorch, LightGBM, CatBoost); Python's **hash randomisation** of strings, which changes the iteration order of sets and dictionaries between runs unless `PYTHONHASHSEED` is fixed; thread scheduling in parallel reductions (sums split across CPU threads, for example with OpenMP); GPU kernels that use atomic floating-point additions or choose algorithms by input shape (in cuBLAS and cuDNN, NVIDIA's GPU maths libraries); **mixed precision** (bf16 keeps only 8 bits of precision, so rounding differences are visible); **batch composition**, because each batch is padded to its longest pair, which changes tensor shapes and therefore kernel choice and summation order even though padded tokens are masked out mathematically; library versions (for example a data library's hash function); and hardware. Remedies: seed everything; force deterministic algorithms where they exist (at a speed cost); train on CPU where feasible; pin library versions and model revisions; store intermediate artefacts with hashes; compare outputs byte for byte. GPU training of CatBoost and bf16 training of transformers are not bit-deterministic in practice, so the realistic standard is "byte-exact from stored artefacts, very close from scratch".

HOW WE USED IT

Seeds: fold hash 20260927; CatBoost ranker 2026; readers torch 0 and numpy 0; decision models 1-5; address second-chance scorer 7, 8, 9; French pseudo-label sampling 11. `PYTHONHASHSEED=0` is required, and the second-chance scripts assert it. The two pretrained encoders are downloaded once at pinned revisions; every later stage runs offline. Input files and outputs are recorded with SHA-256 hashes, and the fold manifest records the data library version and fold-file hashes. In the submitted run, 11,638,504 shortlist pairs carry GPU reader scores reused from earlier scoring runs and 48,813 were scored on the CPU. The French E5 reader's weights were deleted after scoring and its scores on the France shortlist are shipped, so a run from raw data must retrain it, which needs an explicit switch.

EXPERT NUANCE

Two levels of reproducibility answer two different questions: byte-exact regeneration from stored models proves that the shipped code produced the shipped files (an audit question), while a close retrain from raw data proves the method is stable (a science question); our 99.82 % link agreement answers the second. France differs most in the clean run (identical match lists for 96.72 % of French businesses against about 99.5 % for the US and India). We did not isolate the cause; a plausible reason is that France chains more learned steps on its own earlier scores (both French readers are self-trained on pseudo-labels taken from those scores, and the France rules act on score bands), so a small change early on can compound. A common misconception is that setting a seed makes GPU training deterministic; it fixes the random draws, not the order of floating-point arithmetic.

REMEMBER

- Seeds fix random draws; GPU and bf16 arithmetic still vary in the last digits, which ripples downstream.
- From stored weights and scores: both files byte for byte. From raw data: 99.82 % of links, 99.07 % of businesses identical, validator PASS.
- `PYTHONHASHSEED=0`, pinned model revisions, SHA-256 hashes for inputs and outputs.

JUDGE MIGHT ASK

- **Q**
"Can someone reproduce your submission exactly?"
A: Yes, from our trained models and stored reader scores: the inference stages rewrite both files byte for byte, and a fresh extraction of the archive did so too. A full retrain from raw data gives very close files, 99.82 % of links and identical match lists for 99.07 % of businesses with a validator PASS, but not identical ones, because GPU training of the ranker and bf16 training of the readers are not bit-deterministic.
- **Q**
"Where do the differences in a retrain come from?"
A: A retrained ranker gives slightly different scores, so some shortlists change (12,334,519 candidate pairs instead of 12,293,019) and every later stage follows. France differs most, with identical match lists for 96.72 % of French businesses against about 99.5 % for the US and India; we did not isolate why, but a likely reason is that its readers are self-trained on pseudo-labels taken from earlier scores, so small differences can compound.

Quick reference: the numbers of this part

Topic	Number	Source
Folds	10, fold = hash(business id, seed 20260927) mod 10; records inherit their business's fold	Documentation 4; code

Topic	Number	Source
Fold sizes (held-out scoring)	fold 9: US 132,365, India 87,778; fold 8: US 132,407, India 88,103 businesses	evidence
Layer folds	ranker folds 0-5 (140,000-record pilot), stop on fold 6; readers folds 0-5, one epoch; decision models fit 6-7, stop on 8, US/India refit 6-9; native-script second chance folds 0-7 (cross-fitted word lists); address second chance folds 6-7	Documentation 4
Early stopping	ranker up to 1,600 trees, patience 180; decision models patience 100, at most 3,000 (US/India) or 1,500 (France/default) rounds	Documentation 4; code
Refit rule	trees = best iteration × rows(6-9) / rows(6-7)	Documentation 4
Seeds	decision models 1-5 (mean); address scorer 7, 8, 9; ranker 2026; readers 0; pseudo-label sampling 11	Documentation 4; code
Decision rule	best candidate; score ≥ 0.75 (US/India, fold 8) or 0.82 (France/default, label-free); lead ≥ 0.4	Documentation 4
Consistency check	0.75 / 0.68 / 0.72 / 0.77 for 0 / 1 / 2 / 3+ other accepted records; folds 8 and 9; US +277 / -284, India +247 / -248	Documentation 4, B.1
Second-chance thresholds	native script 0.88, lead 0.3; address US 0.86 / 0.0, India 0.66 / 0.1; filter 0.06 / 0.015 (98 % of fold-8 reference links); false positives counted twice	Documentation 4
Held-out F0.5 (fold 9 / fold 8)	US 0.991212 / 0.991545; India 0.990046 / 0.990298; France recipe proxy US 0.990508 / 0.990977, India 0.989467 / 0.989850	Documentation 5
Bootstrap	paired over businesses, 2,000 resamples, 95 % intervals	evidence code
Reader training selection	hard records (top probability in (0.003, 0.9995), non-owner first, or second > 0.003) + random 25 %	Documentation 4
French E5 reader self-training	p ≥ 0.97 with every rival < 0.5; negative = hardest pair ≤ 0.03; ≤ 250,000 per class; 5 % held out; 474,808 + 474,808 pairs; 7,419 steps; learning rate 2e-5	Documentation 4
French MiniLM reader	confident pseudo-labels + 500,000 labelled US/Indian pairs; learning rate 2e-5	Documentation 4
French E5 reader checks	AUC 0.998544 vs 0.998570 (pass); agreement 0.999206 (pass); generated French set 0.910516 vs 0.914964 (fail); expected gain pessimistic -0.0001806 (fail)	Documentation B.4
France blend	0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5, in logits	Documentation 4
France rules	887,630 accepted → 867,559 links; word-swap vetoes -29,021	Documentation B.3
France diagnostic	emptying French rows: 0.970125 → 0.839	Documentation B.6
Leaderboard	18 uploads; final 0.991300 (upload 16); first scored baseline 0.968212; refit kept 0.990621 → 0.990676; France refit 0.990726 vs 0.990748; second French round 0.991277; French MiniLM variant 0.991258	Documentation B.6
Recall (fold 9)	index 99.3694 % US / 98.5068 % India; shortlist 98.8748-99.3628 % / 98.0572-98.4831 %	Documentation B.2
Quick filter cost	owner kept 99.974 %; fold-9 F0.5 0.990117 → 0.990086	Documentation 3
Validator	PASS; 1,732,544 rows per file; 100,087 empty matching rows	evidence log
Reproduction	byte-identical from stored weights and scores; raw-data retrain 99.82 % of links (5,846,344), 99.07 % of businesses identical; PYTHONHASHSEED=0	READMEs

A4 - Our pipeline, step by step, following one record

How to read this part

Parts A0-A3 taught the building blocks one at a time: what a record and a business are, precision, recall and F0.5, text cleaning, the inverted index and IDF, gradient-boosted trees, cross-encoders, stacking, folds and thresholds, self-training. This part puts them together. We take one incoming record and walk it through every stop of our final, submitted pipeline, in the order the code runs it. At each stop you get the same blocks: **Simple** (an everyday picture), **Our record at this stop** (what happens to the running example), **Technical** (the real mechanism, at the level a senior scientist expects), **Numbers** (the real figures from the Documentation), **Why it exists**, **What it costs**, **Concepts used** (pointers back to Parts A0-A3), **Expert nuance**, and **Judge might ask** (short spoken answers). After the tour we retell the three real worked examples from the Documentation (US, India, France) as numbered walk-throughs, then explain France, which had no labels, in the full concept format, then list traps to avoid and give model answers that explain the whole pipeline in 30 seconds and in 2 minutes.

TWO KINDS OF NUMBERS

Read this before anything else.

- **Real numbers** come from the submitted Documentation (tagged "Doc 3", "Doc B.1" and so on), the code README, or the clean-run report. They are the only numbers you may say on stage. A few are marked **derived**: plain arithmetic on Documentation numbers, shown so you can redo it.
- **Illustrative numbers** belong to the running example, which is invented. Its text transformations (cleaning, letter pieces, sound keys) were computed exactly with our shipped normalisation code and the index's definitions, but every count, score and probability attached to it is made up to show the arithmetic. They are always marked "illustrative". **Never quote them to the judges.**

WORDS

We use the plain names of the v3 talk: business, incoming record, shortlist, quick rule-based filter, CatBoost ranker, readers (MiniLM reader, E5 reader, French MiniLM reader, French E5 reader), "what changed" features, LightGBM decision model per country, confident + clear-winner rule, second-chance searches, per-business consistency check, France rules computed by code. Internal code names belong only in the glossary of this study guide.

Words you will meet in this part (one line each; Parts A0-A3 explain them in depth).

Word	Meaning in one line
Business	A record of the reference list (Source 1). The thing we link to.
Incoming record	A record from Source 2 or Source 3 that we must link to one business, or to none.
Candidate / pair	A business we are considering for one incoming record; the two together are a "pair".
Blocking	Any cheap step that decides which pairs are worth comparing at all, so we never compare everything with everything.
Inverted index	A table from each word (or letter piece) to the list of businesses containing it, like the index at the back of a book.
Posting list	The list of businesses stored under one word in the inverted index. Its length is that word's document frequency (df).
IDF (inverse document frequency)	A weight that is high for rare words and low for common ones, computed from df.
Letter pieces (character n-grams)	Overlapping chunks of n letters: the 3-letter pieces of "dental" are den, ent, nta, tal.
Core name	The business name with legal words (inc, llc, pvt, sarl ...) and "and"/"the" removed.

Word	Meaning in one line
Precision / recall	Precision: of the links we make, the share that are right. Recall: of the true links, the share we find.
F0.5	One score combining precision and recall that weighs precision more.
Macro average	Score each business separately, then take the plain average over businesses.
Fold / held-out fold	One of 10 groups of businesses; a held-out fold is kept away from a model's training and used only to test it.
Data leakage	When information from the test side sneaks into training and makes scores look better than they are.
Gradient boosting (CatBoost, LightGBM)	A model built from many small decision trees, each one correcting the errors of the ones before it.
Early stopping	Stop adding trees (or training steps) when the score on a separate validation fold stops improving.
Log loss (binary cross-entropy)	The standard training loss for yes/no predictions: it punishes confident wrong probabilities heavily.
Logit / sigmoid	A logit is an unbounded score; the sigmoid $1 / (1 + e^{(-\text{logit})})$ turns it into a probability between 0 and 1.
Cross-encoder ("reader")	A transformer language model that reads both texts of a pair together, as one input, and outputs one logit; ours are small ones.
Fine-tuning	Further training of a pretrained model on our own labelled examples.
Stacking	Training a second model (here the decision model) on the outputs of earlier models plus extra features.
Threshold / margin (lead)	The minimum score to accept a link / the minimum gap between the best and the second-best candidate.
Pseudo-label / self-training	A model's own very confident guess on unlabelled data, used as if it were a label to train on.
Transliteration	Writing a word in another script, for example विजय → vijay.
Seed	The number that fixes a model's random choices; different seeds give slightly different models.

What is in this part

1. The map: the whole pipeline on one page
2. The running analogy (a post office) and our running record
3. Stop 0 - Before the record arrives: the task, the score, and what was trained where
4. Stop 1 - Normalisation (cleaning the text)
5. Stop 2 - Index lookup (finding candidates)
6. Stop 3 - Quick rule-based filter
7. Stop 4 - CatBoost ranker and the shortlist
8. Stop 5 - Readers (cross-encoders)
9. Stop 6 - "What changed" features and the rest of the feature vector
10. Stop 7 - LightGBM decision model, one per country
11. Stop 8 - Confident + clear-winner rule
12. Stop 9 - Native-script second chance (India)
13. Stop 10 - Per-business consistency check (US and India)
14. Stop 11 - Address second chance (US and India)
15. Stop 12 - France rules computed by code (France)
16. Stop 13 - Output and validation

17. The three real examples, retold: US, India, France
18. Concept: France - how we matched with zero labels
19. Traps to avoid when you describe the pipeline
20. Explain the whole pipeline in one sentence, 30 seconds and 2 minutes

A4.1 The map: the whole pipeline on one page

Everything runs **per country** (US, India, France), and every search stays inside one country. A small configuration file per country, the **country profile**, picks the cleaning rules, readers, decision model, threshold, second-chance searches and rules; the code path is the same for every country (Doc 2.2).

```

incoming record (Source 2 or 3)                                businesses of the same country (Source 1)
|                                                                |
[1] normalise text -----> [1] normalise text, build the index
|                                                                |
[2] index lookup: name words, address words, letter pieces, sounds, exact core name
|                                                                |
|                                about 130 candidates per business (blocking study)
[3] quick rule-based filter (no model) -> 21.71 per business
|                                                                |
[4] CatBoost ranker -> shortlist (at most 4 per record) -> 6.75 per business
|                                                                |
[5] readers (cross-encoders) score each shortlist pair
|                                                                |
[6] "what changed" + other features (217 US / 211 India / 158 France)
|                                                                |
[7] LightGBM decision model of that country -> a probability per pair
|                                                                |
[8] confident + clear winner? yes -> link no -> stay unlinked
|                                                                |
US / India: [9] native-script second chance (India) -> [10] per-business consistency check -> [11] address second chance
France:      [12] France rules computed by code
|                                                                |
[13] two output files, checked by the organisers' validator (7.10 candidates per business in the candidate file)

```

#	Stop	Runs for	In, then out (real numbers)	Measured cost	Main concepts
1	Normalisation	all (France adds French rules)	raw text, then cleaned text for the index and features	not timed separately	text normalisation, transliteration
2	Index lookup	all	6,724,569,566,212 possible same-country pairs, then about 130.1 candidates per business (blocking study of 25 Sep)	stops 2-4 together on all 9.97 million test records: about 1.3 h	inverted index, IDF, character n-grams, phonetic key
3	Quick rule-based filter	all	130.1, then 21.7107 per business (37,614,782 pairs)	no model; held-out F0.5 0.990117 to 0.990086	rule-based pruning
4	CatBoost ranker and shortlist	all	21.71, then 6.75 per business (11,687,317 pairs; at most 4 per record, 1.17 on average) (6.75 and 1.17 derived)	inside the 1.3 h	gradient boosting, top-k with a floor
5	Readers	all; which readers depends on the country	one logit per reader per shortlist pair	about 1.4 h for two readers per pair (clean run)	cross-encoders, fine-tuning
6	Features	all	217 (US), 211 (India), 158 (France, default) features per pair	not timed separately	feature engineering, edit operations
7	Decision model	one per country	a probability per pair	not timed separately	stacking, LightGBM, seed averaging
8	Confident + clear winner	all	accepted: US 2,250,036; India 2,726,453; France 887,630	negligible	threshold, margin, abstention
9	Native-script second chance	India	171,713 records searched, +9,759 links	not timed separately	second-stage retrieval, cross-fitting
10	Consistency check	US, India	US +277 / -284; India +247 / -248	negligible	count-dependent thresholds
11	Address second chance	US, India	US +192; India +2,945 links	about 3.2 GB RAM per worker	retrieval cascade, cost-sensitive thresholds
12	France rules	France	887,630 accepted, then 867,559 links	not timed separately	deterministic post-model rules

#	Stop	Runs for	In, then out (real numbers)	Measured cost	Main concepts
13	Output and validator	all	5,856,936 links; 12,293,019 candidate pairs (7.0954 per business); validator PASS	—	validation, reproducibility

Sources: Doc 2.1, 3, 4, B.1, B.3; times from the clean-run report (CodeREADME 6, clean_run/RESULT.md).

THE IDEA BEHIND THE ORDER

Cheap steps at the start decide what gets read; expensive models at the end decide what gets linked. Each stage does the cheapest job it can do well, and we measured what each stage loses (QA 1, Script slide 4).

JUDGE MIGHT ASK

- *Which stage is the most expensive?* "In our clean run from raw data, the two longest inference stages were the index lookup with the quick filter and ranker, about 1.3 hours, and scoring the shortlist with two readers per pair, about 1.4 hours, for 9.97 million records on one machine. On the training side, fine-tuning the MiniLM and E5 readers took about 2.1 hours, and a full retrain from raw data is about 11.5 hours."

A4.2 The running analogy and our running record

The post office picture

Imagine a huge post office. About 10 million letters arrive (the **incoming records**), each with a scribbled shop name and address. The post office has a directory of 1.7 million registered shops (the **businesses**). Each letter belongs to at most one shop, or to no registered shop at all, and a shop may receive several letters. The post office is judged shop by shop: a shop is happy only if it gets its own letters and nobody else's, and a shop that should get no letters is happy only if it gets none. Delivering a letter to the wrong shop is worse than holding it back. Every stop below is one desk in this post office.

Our record (illustrative, invented for this tour)

An incoming record from Source 2, country US:

Field	Raw text
Name	Sunrize Dental Care & Implants, LLC (note the typo: "Sunrize")
Address	0042 North Main Street, Springfield, Illinois

Businesses in the US list that it will meet (also invented):

Label	Business name	Business address
A	Sunrise Dental Care and Implants LLC	42 N Main St, Springfield, IL (in this story, the true owner)
B	Sunrise Dental Care PC	1180 Oak Avenue, Peoria, IL
C	Sunrise Dental Care	77 Lake Road, Springfield, IL
D	Springfield Smile Clinic	900 N Main St, Springfield, IL
E	Dental Care Implants Studio	5 Lake Shore Dr, Chicago, IL

This record is useful because it has a typo, an ampersand, a legal form, a leading zero, a spelled-out state and street type, and competitors that share only the name, only the address, or both. It does not need a second-chance search; the three real examples in section 17 cover those.

A4.3 Stop 0 - Before the record arrives: the task, the score, and what was trained where

SIMPLE

Before the first letter arrives, the post office must know how it will be judged and must have trained its clerks on old letters whose correct shops are known. It is judged shop by shop, and a wrong delivery hurts more than a held-back letter. The clerks were trained in separate groups on separate piles of old letters, so that the supervisor who checks their work never trusts a clerk on letters that clerk has already memorised.

TECHNICAL

- **The task** is many-to-one entity resolution against a reference list: each incoming record (Source 2 or Source 3) links to at most one business (Source 1) or to none; a business can own many records (Doc 2.1).
- **The metric** is F0.5 computed per business and then averaged over all businesses (macro average). Per business: $F0.5 = 1.25 \cdot P \cdot R / (0.25 \cdot P + R) = 1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$, where TP = correct links, predicted = links we made to that business, true = records it really owns; a business with no true records scores 1 only if we predict nothing for it (Doc 2.1; Code: `stack.py macro`). With beta = 0.5, precision weighs more than recall. Written with error counts, $F0.5 = 1.25 \cdot TP / (1.25 \cdot TP + FP + 0.25 \cdot FN)$ (FP = wrong links, FN = missed links), so inside one business a wrong link weighs four times as much as a missed link in the denominator (derived from the formula).
- **Folds.** All businesses are split into 10 folds by a hash of the business id; every owned incoming record goes with its business, and an unowned record is placed by a hash of its own id (Doc 4; the unowned-record detail is from the code, FACTS C1). This is a group split: records of one business never sit on both sides, which would inflate scores (QA 4).
- **Who trained on what (layer-wise fold separation, a form of stacking without leakage).** "Fit on" = the folds whose labels train the model; "early stopping / tuning" = the fold used to decide when to stop training or to pick a threshold.

Layer	Fit on	Early stopping / tuning
Indian-script word list for cleaning	folds 0-5	—
CatBoost ranker	a 140,000-record pilot, its fold 0-5 part	fold 6
MiniLM and E5 readers	labelled pairs of folds 0-5	none (one epoch)
US / India decision models	folds 6-7, then refit on 6-9 (training rows include extra hard examples; describe them only with the QA 15 wording, and only if asked)	fold 8 (at most 3,000 rounds)
France / default decision models	folds 6-7, no refit	fold 8 (at most 1,500 rounds)
Thresholds	—	0.75 on fold 8; consistency check on folds 8 and 9
Native-script second-chance model	folds 0-7, word lists that leave out the record's own fold	threshold on fold 8, false positives counted twice
Address second-chance models	folds 6-7	thresholds on fold 8, false positives counted twice
French readers	French test pseudo-labels plus labelled US/India pairs	French E5 reader: written checks (Doc B.4)

(Doc 4, Doc 5; FACTS C2.)

OUR RECORD AT THIS STOP (ILLUSTRATIVE)

Nothing happens to it yet. In our story its true owner is business A, and A truly owns 3 incoming records; the score table below uses that.

Worked score example (illustrative), for business A and its 3 true records, plus two cases for a business that owns nothing:

What we link to A	TP	predicted	true	F0.5 for A
exactly its 3 records	3	3	3	$1.25 \cdot 3 / (3 + 0.75) = 1.000$
its 3 records plus 1 wrong one	3	4	3	$3.75 / 4.75 = 0.789$
only 2 of its 3 records	2	2	3	$2.5 / 2.75 = 0.909$
a business with no true records, and we link 1 wrong record	0	1	0	0
a business with no true records, and we link nothing	0	0	0	1 (by the rule)

One wrong link costs more (1.000 to 0.789) than one missed link (1.000 to 0.909), and a wrong link to an empty business turns 1 into 0. This is why every later stop is **precision first**.

NUMBERS (REAL)

1,732,544 test businesses (US 663,106; India 809,986; France 259,452) and 9,969,589 incoming records (US 3,817,031; India 4,717,565; France 1,434,993); about 5.75 incoming records per business; labels for the US and India only; France has none (Doc 2.1).

WHY IT EXISTS

Every threshold later on is tuned for this exact per-business metric, not for AUC (area under the ROC curve, a ranking measure that ignores where the threshold sits) (Doc 4), and the fold layering is what lets the decision model trust the reader scores it sees.

WHAT IT COSTS

No compute at this stop. The price of the design is data: each model layer learns from only part of the labelled folds (Expert nuance).

CONCEPTS USED (PARTS A0-A3)

Entity resolution; precision and recall; F-beta and macro-averaging; group k-fold; stacking; data leakage.

EXPERT NUANCE

The fold design is "stacked generalisation with a layer-wise split" rather than the usual out-of-fold predictions: the ranker and readers are trained once on folds 0-5, so their scores on folds 6-9 are honest out-of-sample scores, and the decision models learn only from those folds (FACTS C2). A second benefit: for the US and India, at test time the decision model receives scores from exactly the same ranker and readers, which are also out-of-sample on test records, so the score distributions it learned from match the ones it is applied to; with out-of-fold predictions, the test-time models would differ from the ones that produced the training inputs. The price is that each layer sees less data than a full cross-fitting scheme would give it.

JUDGE MIGHT ASK

- *About 41 % of incoming records end up unlinked. Isn't that a lot?* "Some incoming records belong to no business in the list, and linking them anyway would be a wrong link. Our final file has 0.5875 links per incoming record, so about 41 % stay unlinked (derived), and 100,087 businesses, 5.8 %, have an empty row; under per-business F0.5 a correct empty row is worth a full point, so we link only a clear winner."

A4.4 Stop 1 - Normalisation (cleaning the text)

SIMPLE

Before looking anything up, a clerk rewrites the scribbled envelope in one standard style: all small letters, "&" written as "and", "Street" always written "st", "Illinois" always "il", no leading zeros on house numbers. Two envelopes that mean the same thing should now look the same, so that the directory lookup at the next desk finds them. For Indian names written in Devanagari or other scripts, the clerk uses a phrase book learned from old letters to write each word in Latin letters. France gets extra house rules, for example "15B" becomes "15 bis" and "S.A.R.L." becomes "sarl". The cleaned copy is only for searching and comparing; the expert readers later read the original envelope.

OUR RECORD AT THIS STOP (COMPUTED WITH OUR SHIPPED CODE)

	Raw	After cleaning
Record name	Sunrize Dental Care & Implants, LLC	sunrize dental care and implants llc
Record address	0042 North Main Street, Springfield, Illinois	42 n main st springfield il
Business A name	Sunrise Dental Care and Implants LLC	sunrise dental care and implants llc
Business A address	42 N Main St, Springfield, IL	42 n main st springfield il

After cleaning, the two addresses are identical, and the names differ only in "sunrize" against "sunrise". The cleaning cannot fix the typo; the letter-piece and sound channels of the next stop exist for that.

More cleaning examples, also computed with our code:

Raw	Cleaned	Rule that fired
B0B'S Pizza & Grill (name, US/India rules)	bob s pizza and grill	a 0 next to a letter becomes o; & becomes and; punctuation becomes a space
2 R SUZANNE LENGLEN (French address)	2 rue suzanne lenglen	French street type "R" canonicalised
15B Av. Jean Jaurès (French address)	15 bis av jean jaurès	glued suffix split, accents removed
N° 7 bd St Michel (French address)	7 bd saint michel	N° prefix removed, st expanded to saint
S.A.R.L. Dupont et Fils (French name)	sarl dupont and fils	dotted legal form collapsed, et becomes and
SARL Martin & Cie (French name)	sarl martin and cie	"leet" legal form repaired

TECHNICAL

- **US and India rules** (Doc 2.2; Code: prepare.py): lower case; & to and; every other non-alphanumeric character to a space; in addresses, state names and street abbreviations mapped to one form (street to st, north to n, illinois to il, and so on) and leading zeros dropped from numbers; in names, a 0 next to a letter becomes o. Non-ASCII words in addresses (accented letters or other scripts) are always transliterated with unidecode (a library that maps each Unicode character to the closest plain Latin letters).
- **Indian scripts.** Each non-Latin name word goes through an Indic-to-Latin word list learned from labelled training matches of folds 0-5; if the word is missing, a generic transliteration (unidecode) is used. The word list is a co-occurrence alignment: each native-script word of an incoming name is counted against the words of its true business's cleaned name; it gets its best Latin word only if they co-occur at least 3 times ($n \geq 3$), the pairing covers at least 0.35 of the native word's occurrences, and its score $n / (\text{count of the Latin word} + 1)^{0.6}$ is at least 1.8 times the runner-up's (Code:

learn_lexicon.py). Dividing by the Latin word's own frequency stops very common words such as "private" or "enterprises", which co-occur with almost every native word, from winning everywhere; the 0.6 power softens that penalty. It is not a neural transliterator.

- **France rules** (Doc 2.2): region, department and city components learned from the unlabelled test files; N°/numéro prefixes removed; 15B to 15 bis; bis/ter/quater unified; French street types and their typos canonicalised (the typo table is learned from the test files); st/ste expanded to saint/sainte; dotted and leet legal forms collapsed (S.A.R.L. and 5ar1 to sarl). The code also maps et to and and unifies a few French abbreviations.
- **The readers read the raw text**, not the cleaned text (Doc 2.2).
- **A country without a profile** gets the US/India rules (Code: profiles/default.json).

NUMBERS (REAL)

Word-list thresholds 3 / 0.35 / 1.8 are fixed values in the code; the word list itself is learned from folds 0-5 (Code: learn_lexicon.py). The French geography and street-type typo table are learned from the unlabelled test files, which the Documentation discloses (Doc C.1). Separately, the pipeline computes an unsupervised native-script-to-Latin word alignment on test records whose first-stage top candidate has probability at least 0.9; it uses no labels and is disclosed in the same item (Doc C.1).

WHY IT EXISTS

The index of the next stop matches exact tokens. Without cleaning, "Street" and "St", "Illinois" and "IL", "0042" and "42" would be different words and the true business would score lower or not be found at all.

WHAT IT COSTS

A few string operations per record, done once per record; the Documentation gives no separate time. The real cost is design effort: every rule is a decision about which differences do not matter.

CONCEPTS USED (PARTS A0-A3)

Text normalisation; Unicode and transliteration; tokens.

EXPERT NUANCE

Normalisation is lossy on purpose, so it is used only where exact token equality matters (the index and the hand-made features). The readers get the raw strings, so information that cleaning throws away, such as the original script, the brackets in "(Inc)" or capitalisation, is still available to the most accurate model. A second subtlety: the cleaning word list for Indian scripts is learned from labels, so it can only translate words that appeared in labelled training matches, and it keeps one spelling per word; other words fall back to a generic transliteration that may not match the Latin spelling a business uses. The native-script second chance (Stop 9) attacks the same weakness by trying several spellings per name.

JUDGE MIGHT ASK

- *Why do the readers read raw text if you built careful cleaning rules?* "The cleaning exists for exact token matching in the index and in our hand-made features. The readers are multilingual pretrained models that handle case, punctuation and scripts themselves, so we give them the original text and keep the information that cleaning throws away."
- *How do Indian-script names get into a Latin-script index?* "Each native-script word goes through a word list learned from labelled training matches of folds 0-5: a native word gets a Latin word only if they co-occur at least 3 times and the pairing clearly beats the runner-up; otherwise we fall back to a generic transliteration. For records the main pass still leaves unlinked, the native-script second chance tries up to 64 spellings per name inside the same state."

A4.5 Stop 2 - Index lookup (finding candidates)

SIMPLE

Comparing every letter with every shop in its country would mean 6.7 trillion comparisons, so the clerk uses the directory's index, like the index at the back of a book: for every word, it lists the shops whose name or address contains that word. The clerk looks up the rare words first, because a rare word like "implants" points to a few shops, while a common word like "street" points to almost everyone and is skipped. There are separate indexes for name words, address words, short letter pieces of the name (so a typo still matches most pieces) and a "sound" version of the name. Each shop collects points for every rare word it shares with the letter, and the clerk keeps the best-scoring shops from several lists, so that a shop matching only by address or only by name is not lost.

Our record at this stop (text keys computed with our code's definitions; counts and scores illustrative).

First the record is split into the five channels (Doc 3; Code: matcher.cpp):

Channel	What is indexed	Our record
1. Name words	words of the core name (legal words such as inc, llc, pvt, sarl, sas, and "and"/"the", removed)	sunrize, dental, care, implants
2. Address words	words of the cleaned address	42, n, main, st, springfield, il
3. 4-letter pieces	character 4-grams of the core name written without spaces, sunrizedentalcareimplants	22 distinct pieces: sunr, unr, nriz, rize, ...
4. 3-letter pieces	character 3-grams of the same string	23 distinct pieces
5. Sounds	4-grams of a consonant skeleton: core-name words sorted alphabetically, vowels (a, e, i, o, u and y) deleted, repeated letters collapsed, spaces removed: care dental implants s unrize becomes cr dntl mplnts snrz, then crdntlmpntssnrz	13 distinct pieces
plus exact key	the sorted set of core-name words	{care, dental, implants, sunrize}

How the typo plays out against business A (sunrise dental care implants), computed exactly:

Channel	Shared with A	Comment
Name words	3 of 4 (dental, care, implants)	"sunrize" matches nothing
4-letter pieces	18 of 22	only the 4 pieces around the z differ (nriz, rize, ized, zede)
3-letter pieces	20 of 23	only riz, ize, zed differ
Sound pieces	12 of 13	only snrz against snrs differs
Exact key	no	one typo breaks the exact-key channel

So one wrong letter costs one name word, but most letter pieces and sound pieces still match: that is exactly why those channels exist.

Now the points (illustrative document frequencies; N = 663,106 US businesses is real). The index uses **IDF** = $\ln(1 + (N + 1) / (df + 1))$, where df is how many businesses contain the term:

Term	df (illustrative)	IDF	Used?
implants (name)	300	7.698	yes
dental (name)	5,000	4.895	yes
care (name)	20,000	3.531	no : longer than the name cap of 14,000
sunrize (name)	0	—	not in the index at all

Term	df (illustrative)	IDF	Used?
springfield (address)	2,500	5.584	yes
42 (address)	6,000	4.714	yes
main (address)	9,000	4.313	yes
il, n, st (address)	25,000 / 120,000 / 250,000	3.315 / 1.876 / 1.295	no : longer than the address cap of 10,000

Rarest first, within the address budget of 18,000 postings: springfield (2,500), then 42 (running total 8,500), then main (17,500), all inside the budget.

Business A's scores (word parts only; the real name score also adds the letter-piece and sound points):

- Name-word part: $(7.698 + 4.895) / (1 + 4 \text{ name words of A})^{0.22} = 12.593 / 1.425 = \mathbf{8.838}$
- Address score: $(5.584 + 4.714 + 4.313) / (1 + 6 \text{ address words of A})^{0.30} = 14.611 / 1.793 = \mathbf{8.150}$
- Total: $ns + as + 1.2 \cdot \min(ns, as) = 8.838 + 8.150 + 1.2 \cdot 8.150 = \mathbf{26.768}$

The last term, $1.2 \cdot \min(ns, as)$, is a bonus that is large only when both the name and the address match. Business B (same-sounding name, Peoria) gets name points but no address points, because its only shared address word, "il", is too common to be used for retrieval, so $as = 0$ and its bonus is zero; business D (same street, different name) gets address points but few name points. Business A gets both.

TECHNICAL

- Written in C++17 with OpenMP (a standard library for running one loop on many CPU cores in parallel), built over the businesses of one country on cleaned text (Doc 3).
- **Scoring is an IDF-weighted set overlap**, not TF-IDF cosine and not BM25: every record is a set of unique terms, there is no term-frequency component (Doc 3; FACTS A2). IDF is smoothed: $\ln(1 + (N + 1) / (df + 1))$. Unlike the textbook $\ln(N / df)$, this never reaches 0: even a term found in every business keeps a weight of about $\ln 2 = 0.69$, while a term found in exactly one US business gets $\ln(1 + 663,107 / 2)$, about 12.71 (derived). (In the 76 features, a record term missing from the index is given IDF 12; Code: matcher.cpp.)
- **Rarest first with caps and budgets.** Query terms are processed from rarest to most common (shortest posting list first); a term is skipped if its posting list is longer than the channel's cap, and a term is also skipped if adding its list would push the channel past its total posting budget. Caps / budgets: name 14,000 / 22,000; address 10,000 / 18,000; 4-grams 2,500 / 15,000; 3-grams 1,000 / 6,000; sounds 800 / 4,000. Channel weights 1 / 1 / 0.45 / 0.22 / 0.18 (Doc 3). So one record touches at most $22,000 + 18,000 + 15,000 + 6,000 + 4,000 = 65,000$ posting entries (derived), plus at most 2,000 from the exact-key channel below.
- **Sound key detail** (Code: matcher.cpp): a consonant equal to the previous kept consonant is dropped even when a vowel sits between them, so "lol" becomes "l"; the collapse happens before the spaces are removed, which is why "mplnts snrz" keeps its double s in `crdnt1mplntssnrz`.
- **Scores.** Name score $ns = \text{name-word IDF sum} / (1 + \text{business core-name words})^{0.22} + \text{letter-piece and sound sum} / (1 + \text{business 4-grams})^{0.38}$; address score $as = \text{address IDF sum} / (1 + \text{business address words})^{0.30}$; total $= ns + as + 1.2 \cdot \min(ns, as)$ (Doc 3). Only the terms actually processed (within caps and budgets) add points. The exponents are length normalisations, in the spirit of BM25's document-length term: a long business name has more chances to contain some of the record's words, and a shared word says less when it is one of many, so the sum is divided by a slowly growing (sub-linear) function of the business's term count. Exponents well below 1 correct for length without punishing long names too hard.
- **Kept per record** (a union of several top lists, which is what makes retrieval recall-first): the top 12 by total, the top 6 by name score, the top 6 by address score (address score > 0), and every business with

the **same sorted core-name word set** if at most 2,000 businesses share it (Doc 3).

- **76 pair features** are computed for every kept candidate: 7 string similarities (normalised longest-common-subsequence ratio, $2 \cdot \text{LCS} / (\text{len } a + \text{len } b)$, computed bit-parallel), set statistics for name words, address words, letter pieces, sound pieces and numbers, length counts, equality flags, IDF overlaps, token alignment, a common-prefix share, exact core-name group sizes, and 8 retrieval scores (Doc 3; FACTS A2).
- No embeddings, no vector search and no learned model in this stage.

NUMBERS (REAL)

- Same-country comparison space: 6,724,569,566,212 pairs (Doc 3).
- About 130.1 candidates per business out of the index (blocking study of 25 Sep 2026, Doc 3). In all-fold train samples: 27.541 candidates per incoming record (US), 23.145 (India) (Doc B.2).
- Recall on labelled train fold 9 (share of owned records whose true business is among the candidates): **99.3694 % US, 98.5068 % India** (Doc B.2).

WHY IT EXISTS

It turns an impossible all-pairs problem into a small candidate list while keeping almost every true business. It is deterministic, fast and easy to audit.

WHAT IT COSTS

The index lookup, the quick filter and the ranker together took about 1.3 hours for all 9.97 million test records in our clean run (stage i1; clean-run report). That stage runs the index and the ranker with 6 threads by default (CodeREADME 2, setting `BLOCK_THREADS`); the 8-11 threads quoted in CodeREADME 1 are for the index runs of the training stages. The work per record is bounded by the caps and budgets, so total cost grows linearly with the number of records.

CONCEPTS USED (PARTS A0-A3)

Blocking; inverted index; IDF; character n-grams; phonetic keys; recall ceiling.

EXPERT NUANCE

The cap acts as a stop-word list learned from the data: a word becomes "too common to retrieve with" once it appears in more than 14,000 business names or 10,000 addresses; apart from the small list of legal words (and "and", "the") removed from core names, there is no hand-written stop-word list for retrieval. Common words still count in the 76 features and in the readers; they just do not drive retrieval. The weak spot is a business whose only shared evidence is a common word, or a brand name with no word overlap at all; that is where a dense vector channel would help, which we did not benchmark (QA 1). Also note that the sound key is our own consonant skeleton, not Soundex or Metaphone (FACTS F2).

JUDGE MIGHT ASK

- *Why an IDF-weighted overlap and not BM25 or TF-IDF cosine?* "Business names and addresses are short and almost never repeat a word, so term frequency carries almost no information; we treat each record as a set of unique terms and sum the smoothed IDF of the shared ones. What we do need is length normalisation so long names do not win by size: we divide by one plus the business's term count, raised to 0.22 for name words, 0.38 for letter pieces and 0.30 for address words."
- *How do you stop common words like "street" or "enterprises" from blowing up the work?* "Rarest terms first, with a cap per list: a term whose posting list is longer than 14,000 businesses for name words or 10,000 for address words is skipped, and each channel has a total budget, so one record touches at most 65,000 posting entries. Common words still count in the later features; they just do not drive retrieval."
- *Why keep separate top lists by name and by address?* "Because a single combined score drops candidates that match on one side only, such as a renamed business at the same address or a moved

business with the same name. We keep the top 12 by total, the top 6 by name, the top 6 by address and every business with exactly the same core-name words, and that union keeps the true business for 99.37 % of US and 98.51 % of Indian held-out records."

A4.6 Stop 3 - Quick rule-based filter

SIMPLE

The index desk hands over a pile of candidate shops for every letter; across the whole post office that is about 130 candidates for every registered shop. Before the senior clerk spends any time, a junior clerk throws out the shops that are clearly weaker than the best one, using a short checklist and no judgement at all. The checklist has safety clauses: never throw out the shop with the best address match, a shop whose address covers almost all of the letter's address, a shop with exactly the same name once words like "LLC" or "Pvt Ltd" are ignored, or a candidate for a very short name such as an acronym. It is almost free and it almost never throws away the right shop.

OUR RECORD AT THIS STOP (ILLUSTRATIVE)

Five candidates came out of the index. A candidate is kept if **any** clause holds (Doc 3):

Candidate	total	total / best	best by address?	address coverage	kept?
A, Sunrise Dental Care and Implants LLC, 42 N Main St, Springfield	31.0	1.00	yes	1.000	kept (best total; also best address; also coverage ≥ 0.95)
C, Sunrise Dental Care, 77 Lake Road, Springfield	20.5	0.66	no	0.422	kept (total $\geq 0.6 \times$ best)
B, Sunrise Dental Care PC, 1180 Oak Avenue, Peoria	12.0	0.39	no	0.157	dropped
D, Springfield Smile Clinic, 900 N Main St, Springfield	9.5	0.31	no	0.777	dropped (coverage below 0.95)
E, Dental Care Implants Studio, 5 Lake Shore Dr, Chicago	8.0	0.26	no	0.157	dropped

The coverage column is computed from the illustrative IDF's of Stop 2: coverage = the IDF share of the record's address words found in the business's address. For D, every word except "42" is found: $(21.097 - 4.714) / 21.097 = 0.777$. The "same core name" clause could not fire for any candidate, because the typo "sunrize" gives our record a core-name set that no business has, and the "short name" clause does not apply because the name is long.

TECHNICAL

No model; every test is a threshold on retrieval statistics the index already computed (Doc 3; Code: prune.py). Keep a candidate if any of:

- (a) it has the record's best total (ties kept);
- (b) $\text{total} \geq 0.6 \cdot \max(1, \text{best total})$;
- (c) it is the record's best candidate by address score (address score > 0);
- (d) address coverage ≥ 0.95 ;
- (e) the record name has at most 3 characters and coverage ≥ 0.4 (initials and acronyms);
- (f) same core-name word set and coverage ≥ 0.3 .

Clauses (c) to (f) protect address-only, short-name or acronym, and same-name candidates that a combined score would drop (Doc 3). The `max(1, ...)` in (b) stops the rule from keeping everything when the best total is tiny.

NUMBERS (REAL)

- 37,614,782 pairs kept, **21.7107 per business**; per country US 15.0352, India 23.6328, France 32.7713 (Doc 3). About 3.77 per incoming record (derived: 37,614,782 / 9,969,589).
- In all-fold train samples: 3.111 (US) and 3.878 (India) candidates per incoming record after the filter (Doc B.2).
- Measured cost (blocking study of 25 Sep, with the decision model of that date): the true business survived for **99.974 %** of fold-8/9 records; fold-9 macro F0.5 moved from **0.990117 to 0.990086**; fold 8 -0.00004 (Doc 3). Fold-9 sample recall: 99.3658 % (US) at the index, 99.3523 % after the filter (Doc B.2).

WHY IT EXISTS

Without it, the CatBoost ranker would score about 130 candidates per business; with it, about 22. A learned pruner would itself need a model pass over all 130.

WHAT IT COSTS

Essentially nothing in compute (comparisons on numbers already in memory) and 0.00003 F0.5 on held-out fold 9 (0.990117 - 0.990086, derived).

CONCEPTS USED (PARTS A0-A3)

Blocking; rule-based pruning; the recall-cost trade-off.

EXPERT NUANCE

Three things to be precise about. First, 130.1 per business comes from the blocking study, not from the final run; the final run reports 21.7107 after the filter (FACTS H2). Second, the decision models were trained on shortlists built from the index output without the filter (Doc 4), so the filter is a test-time cost-saving step. That creates a small train/test difference: at test time the ranker ranks, and the shortlist is cut from, the filtered candidates, so a few context features (ranks, shortlist size) can differ from what the decision model saw in training. The 25 September measurement ran the decision model of that date on filtered candidates, so it captures this effect for that model; the Documentation reports no repeat with the final models, so do not present it as one. Third, the rule's thresholds sit on the index's own scores, so it inherits the index's blind spots: a true business with a low total and a weak address match is dropped here unless clause (e) or (f) protects it.

JUDGE MIGHT ASK

- *Why a hand-written filter rather than a learned pruner?* "A learned pruner would need a model pass over all of the roughly 130 candidates per business, while our rule reuses statistics the index already computed. Its measured cost is tiny: the true business survived for 99.974 % of held-out records, and fold-9 F0.5 moved from 0.990117 to 0.990086."
- *Where do 0.6 and the coverage thresholds come from?* "From our blocking study of 25 September, which measured how many candidates the rule keeps and the held-out F0.5 with it. The special clauses protect address-only matches, very short names and acronyms, and same-name businesses, which a single score would drop."

A4.7 Stop 4 - CatBoost ranker and the shortlist

SIMPLE

A senior clerk now looks at the remaining candidates of each letter and puts them in order, most likely first, using the 76 measurements the index desk already wrote down. Only the top candidate always goes forward, plus up to three more if the clerk thinks they have at least a one-in-a-thousand chance. That short list, at most four shops per letter, is the only thing the expensive expert readers will ever see. Most letters end up with just one candidate, because the senior clerk is usually sure.

OUR RECORD AT THIS STOP (ILLUSTRATIVE)

The ranker gives A a probability of 0.97 and C 0.006. Rank 0 (A) always goes forward; rank 1 (C) goes forward because $0.006 \geq 0.001$. Our shortlist is {A, C}: two pairs.

TECHNICAL

- **Model:** CatBoost on the 76 index features; loss Logloss (log loss); up to 1,600 trees; depth 7; learning rate 0.075 (how much each new tree is allowed to change the prediction); L2 regularisation 5 (a penalty that shrinks leaf values towards zero); early stopping 180 (stop if 180 more trees bring no improvement on the stopping fold); trained on GPU (Doc 4). CatBoost grows symmetric (oblivious) trees by default, and our code does not change that: every node at the same depth uses the same split, so depth 7 means $2^7 = 128$ leaves per tree (Code: train_base_v6.py).
- **It is a pointwise binary classifier**, $P(\text{candidate is the true business})$, whose probability we sort by. It is not trained with a learning-to-rank loss such as LambdaMART (Doc 4; FACTS H3).
- **Training data:** a pilot of 140,000 training incoming records (35,000 per country and source), whose fold 0-5 part is fitted; fold 6 is used for early stopping (Doc 4).
- **Shortlist rule:** per incoming record, the top-ranked candidate always, and ranks 2-4 only if the ranker probability is at least 0.001; so at most 4 per record (Doc 3; in code: $K = 4$, floor $1e-3$).
- The ranker's probability and rank are also inputs of the decision model (Doc 4).

NUMBERS (REAL)

- Shortlist: **11,687,317 pairs**; US 4,098,587; India 5,485,623; France 2,103,107 (Doc 3). That is **6.75 per business** and **1.17 per incoming record** (both derived).
- In all-fold train samples: 1.114 (US) and 1.174 (India) shortlist pairs per incoming record (Doc B.2).
- Recall in the shortlist, train fold 9: US 98.8748 % to 99.3628 % (lower and upper bound), India 98.0572 % to 98.4831 %; without the filter, the top 4 alone keep 98.8814 % (US) and 98.0809 % (India) (Doc B.2). **The largest loss of the whole cascade is the cap of four** (Doc 3).

WHY IT EXISTS

To protect the readers: at most four pairs per record, 1.17 on average, reach the expensive cross-encoders, so their cost stays linear in the number of records.

WHAT IT COSTS

Scoring is inside the 1.3 hours of Stop 2. Training the ranker is part of clean-run stage t1 (folds, word list, index and ranker training), about 1.4 hours. In recall, up to about half a percentage point: US 99.37 % at the index against at least 98.87 % in the shortlist. The one exact full-fold comparison is the cap alone, without the filter: US 99.3694 % to 98.8814 %, India 98.5068 % to 98.0809 %, a loss of 0.49 and 0.43 points (derived, Doc B.2).

CONCEPTS USED (PARTS A0-A3)

Gradient boosting; CatBoost; log loss; pointwise ranking; top-k with a probability floor.

EXPERT NUANCE

The cap of 4 is a ceiling that is rarely reached: about 3.77 candidates per record reach the ranker (derived), yet the shortlist averages only 1.17 per record, and 90.59 % of records end with exactly one candidate in the final candidate file (Doc B.1). So the quick filter and the 0.001 floor do most of the narrowing, while the cap mainly bounds the worst case per record, which is what keeps reader cost linear. The Documentation records no tuning study for the floor (FACTS B.5), so present it as a fixed design value. A pointwise classifier is a reasonable ranker here because the candidate lists are tiny and we also need a usable absolute probability, both for the floor and as an input to the decision model.

JUDGE MIGHT ASK

- *Is your CatBoost ranker a learning-to-rank model?* "No. It is a pointwise binary classifier, CatBoost with log loss, predicting the probability that a candidate is the true business, and we sort by that probability. We need two things from it: a good order for the top four and an absolute probability for the 0.001 floor, and its probability and rank are also inputs to the decision model."
- *The shortlist costs about half a percentage point of recall. Why accept that?* "It is our largest recall loss: on fold 9 the US goes from 99.37 % at the index to at least 98.87 % in the shortlist, India from 98.51 % to at least 98.06 %. In exchange at most four pairs per record, 1.17 on average, reach the expensive readers, the second-chance searches recover part of the misses, and with finer blocking keys we would have room for a larger cap."

A4.8 Stop 5 - Readers (cross-encoders)

SIMPLE

Now an expert reader puts the letter and one candidate shop's registration card side by side and reads them together, word against word, then gives one number: how strongly they believe it is the same business. Because they read both texts at once, they notice small but decisive details, like "Inc" turning into "LLC", or the same name in a different city. The readers are small pretrained language models (they already "know" many languages from general text) that we fine-tuned, meaning trained further, on labelled US and Indian examples. For France, two extra readers were trained on French examples that our own models had labelled with very high confidence (Section 18).

OUR RECORD AT THIS STOP (ILLUSTRATIVE)

The reader sees, as one input, the raw record text and the raw business text:

```
record side:  Sunrize Dental Care & Implants, LLC | 0042 North Main Street, Springfield, Illinois
business side: Sunrise Dental Care and Implants LLC | 42 N Main St, Springfield, IL
```

Illustrative outputs: MiniLM reader logit +6.2 for A and -4.8 for C; E5 reader +5.9 for A and -5.3 for C. A **logit** is a score on an unbounded scale; the sigmoid turns it into a probability: $\text{sigmoid}(6.2) = 1 / (1 + e^{-6.2}) = 0.998$, $\text{sigmoid}(-4.8) = 0.008$.

A real example of why reading both sides matters (France, Doc D.3): for "TQ COMITE SARL, 2 R SUZANNE LENGLEN, NANTES" against the business with the identical name "TQ Comite SARL" in Pessac, another city, the plain MiniLM reader gave +7.125 (a probability of about 0.999, derived), but the French readers gave -1.9219 and -10.8125, and the blend was -3.2008 (about 0.039, derived).

TECHNICAL

- **Architecture (all four readers):** each side is encoded as raw `name | address` text; the two sides are fed together as one sequence pair, tokenised (split into sub-word units) with the MiniLM tokenizer, and truncated longest-first to 128 tokens (tokens are cut from whichever side is longer until the pair fits); the transformer's last hidden states, one vector per token, are averaged over the real tokens and not the padding (masked mean pooling); dropout 0.1 (randomly zeroing 10 % of values during training, a regulariser); a linear head turns the averaged vector into one logit (Doc 4). The head is tiny: 385 parameters for MiniLM (a 384-wide weight vector plus a bias) and 769 for E5 (768 + 1).
- **Why cross-encoder.** All tokens of both texts attend to each other in every layer, so the model can compare "Inc" with "LLC" directly. A bi-encoder would compress each side into one vector first and compare the vectors afterwards, which is cheaper (business vectors can be precomputed) but can miss that exactly one word changed (Explainer slide 7). The cost is a full transformer pass per pair, which is why readers see only the shortlist.

- **Training recipe (all four):** binary cross-entropy (log loss on the match / no-match label); AdamW optimiser (Adam with weight decay applied separately from the gradient step; decay 0.01), with the new head learning 5 times faster than the pretrained body; a 3 % linear warm-up (the learning rate rises from 0 over the first 3 % of steps, which protects the pretrained weights from large early updates), then linear decay; bf16 autocast (most matrix maths in 16-bit "brain float", which roughly halves activation memory and speeds up the GPU, while the weights stay in 32-bit); gradient clipping at norm 1.0 (the gradient is rescaled whenever its overall size exceeds 1.0, which prevents unstable steps); one epoch (each training pair seen once); batch 128; length-bucketed batches (pairs of similar length are batched together, so less padding is computed) (Doc 4; code).
- **Training pairs (MiniLM and E5 readers):** labelled US/India train pairs of folds 0-5: every hard record (top ranker probability between 0.003 and 0.9995, a non-owner ranked first, or a second probability above 0.003) plus a random 25 % of the others; candidates 1-4 enter if ranked first, with probability at least 0.001, or if they are the owner; candidates 5-8 only if they are the owner (Doc 4). Learning rate 5e-5.

Reader	Base model	Parameters	Licence	Used for
MiniLM reader	paraphrase-multilingual-MiniLM-L12-v2, fine-tuned	117,653,760 + 385	Apache-2.0	all countries (US, India, France, default)
E5 reader	multilingual-e5-base, fine-tuned	278,043,648 + 769	MIT	US and India; starting point of the French E5 reader
French MiniLM reader	the MiniLM reader, fine-tuned on French pseudo-labels	as MiniLM	Apache-2.0	France
French E5 reader	the E5 reader, fine-tuned on French pseudo-labels, 7,419 steps	as E5	MIT	France

(Doc 4, Doc B.5.) Which readers a country uses: US and India, the MiniLM and E5 readers; France, MiniLM + French MiniLM + French E5, blended 0.35 / 0.15 / 0.5 into one score; an unseen country, the MiniLM reader only. The plain E5 reader is not used for France (Doc 4).

NUMBERS (REAL)

Readers score only the shortlist; second-chance candidates are scored by the second-chance models (Doc 4). A larger reader, XLM-R large (about 560 million parameters, MIT), was tried in development and discarded (Doc B.5).

WHY IT EXISTS

It is the most accurate pair judge we have, and the only stage that reads the raw text of both sides together.

WHAT IT COSTS

In our clean run, scoring the shortlist with two readers per pair (every reader except the French E5 reader) took about 1.4 hours on one RTX 4070 Ti SUPER 16 GB; retraining and scoring the French E5 reader took about 0.6 hours; fine-tuning the MiniLM and E5 readers took about 2.1 hours (clean-run report). In the submitted run, most reader logits were reused from earlier scoring runs (11,638,504 pairs) and 48,813 pairs were scored on CPU (CodeREADME 6); together these are exactly the 11,687,317 shortlist pairs (derived).

CONCEPTS USED (PARTS A0-A3)

Transformers and attention; cross-encoder versus bi-encoder; fine-tuning; binary cross-entropy; logits and sigmoid; mixed precision; hard-example mining.

EXPERT NUANCE

A "mini" model with 118 million parameters sounds odd; as general background (not stated in our documents), most of the parameters of these multilingual encoders sit in the large shared vocabulary table, and the MiniLM and E5 base models use the same multilingual SentencePiece vocabulary, which is

why one tokenizer serves all four readers. A second point: the training-pair rule deliberately over-samples hard records, so most of the reader's training budget goes to the confusions that reach it at test time (candidates 1-4 mirror the shortlist rule). Owners that the ranker placed fifth to eighth never reach a reader at test time; they are added as hard positives, true matches whose index statistics look weak, so the reader learns to recognise a true match from the text itself. The random quarter of easy records keeps the training data close to the ordinary cases the reader will also see. Third, readers are trained for one epoch only, which limits over-fitting to the training businesses.

JUDGE MIGHT ASK

- *Why two different readers for the US and India?* "Two different pretrained models make partly different mistakes, and the decision model gets both logits, their mean, and each one's rank and lead within the shortlist, so it can use their agreement. Both are small open models, about 118 million and 278 million parameters, far under the 8-billion limit; the Documentation reports no separate number for the E5 reader's contribution."
- *Which pairs did the readers train on?* "Labelled US and Indian pairs from folds 0-5: every hard record, meaning the ranker was unsure, put a wrong business first, or gave the second candidate more than 0.003, plus a random quarter of the easy ones. Owners the ranker placed fifth to eighth were also included as extra true matches, so the reader learns what a true match looks like even when the index statistics undervalue it."

A4.9 Stop 6 - "What changed" features and the rest of the feature vector

SIMPLE

Before the supervisor decides, a clerk fills in a form for each candidate. The usual question is "how similar are they?". Our form also asks "what exactly changed?": did a house number change and by how much, were words added, dropped or replaced, how rare and how close in meaning are those words, and did the legal form change, say from Inc to LLC? The reason is that similar-looking edits can mean opposite things: dropping a suffix is typical of a genuine record, while swapping the legal form often means a different company. The form also copies in the readers' scores and how each candidate compares with its rivals for the same letter.

OUR RECORD AT THIS STOP (ILLUSTRATIVE; THE DEFINITIONS ARE EXACT)

Comparing the record's core-name words [sunrize, dental, care, implants] and house number 42 with the two shortlisted businesses:

"What changed" feature	vs A: sunrise dental care implants, 42	vs C: sunrise dental care, 77
words replaced / added / deleted	1 / 0 / 0	1 / 1 / 0
first new word, first dropped word	sunrize, sunrise	sunrize, sunrise
relative position of the first new / dropped word	0 / 0	0 / 0
replaced in place (same position)?	yes	yes
common words keep their order?	yes (dental, care, implants)	yes (dental, care)

"What changed" feature	vs A: sunrise dental care implants, 42	vs C: sunrise dental care, 77
character similarity of first new vs first dropped word (RapidFuzz ratio, $= 2 \cdot \text{LCS} / \text{total length} \times 100$)	85.7	85.7
IDF-weighted Jaccard of the word sets (illustrative IDFs: sunrise 12.0, sunrise 7.0, dental 5.0, care 3.5, implants 7.5)	$16.0 / (16.0 + 12.0 + 7.0) = 0.457$	$8.5 / (8.5 + 19.5 + 7.0) = 0.243$
legal-form relation	same (LLC and LLC)	business has none
unmatched house numbers	none ($42 = 42$)	record 42, business 77: difference $42 - 77 = -35$, so absolute difference 35, largest negative difference -35 and no positive one; relative difference $-35 / 77 = -0.45$; digit edit distance 2 ("42" to "77" needs two substitutions); common prefix 0; common suffix 0; digit-length difference 0

How the word counts are defined (Code: clean_feats.py): a "new" word is in the record but not in the business, a "dropped" word is in the business but not in the record; replaced = the smaller of the two counts, added = new – replaced, deleted = dropped – replaced. Against C the record has two new words (sunrise, implants) and one dropped word (sunrise), so 1 replaced, 1 added, 0 deleted. The IDF-weighted Jaccard = IDF of shared words / (IDF of shared + new + dropped words).

Read the A column: the IDF-weighted Jaccard alone would punish the typo (a rare "new" word), but the in-place flag and the character similarity of 85.7 say "this looks like a spelling variant", and the readers saw the raw text too. The decision model learns from labels how these signals combine.

TECHNICAL

- **The "what changed" block** = 25 generic pair features + the legal-form relation + the number relation (Doc 2.2, Doc 4; Code: clean_feats.py):
- **Numbers (8 features)** over address digit runs not shared by both records (up to 8 per address): smallest absolute difference between an extra number of the record and a missing number of the business, smallest positive and largest negative difference (record minus business), relative difference of the closest pair ($(\text{record} - \text{business}) / \text{business}$), Damerau-Levenshtein distance of the two digit strings (the number of single-character insertions, deletions, substitutions or swaps of two neighbours), common prefix and suffix length, digit-length difference. When there is no unmatched pair, the features are left missing (or set to -1), which a tree model can route down its own branch.
- **Name words (14 features)** on the raw name (lower-cased, accents stripped, transliterated with the training word list, legal-form words and and/et/& removed; up to 8 words): counts of replaced, added and deleted words; relative positions of the first new and first dropped word; an in-place flag; an order flag; character similarity of the first new versus first dropped word; IDF of the first new and first dropped word; maximum and minimum IDF over new words; maximum IDF over dropped words; IDF-weighted Jaccard. IDF here = $\ln((N + 1) / (df + 1))$ over all records of the country, label-free.
- **Meaning similarity (3 features)** from the pretrained, not fine-tuned, MiniLM encoder applied to single words: cosine similarity (near 1 for words used in similar contexts, near 0 for unrelated words) of the first new versus first dropped word, and the best cosine of each to any word of the other name.
- **Legal-form relation** (classes such as same, changed, record has none, business has none) and **number relation** of the unmatched digit runs, with forensic legal-form and number comparisons (Doc 4).

- **The rest of the vector** (Doc 4): the 76 index features; reader logits (for the US and India also the E5 logit and the mean of the two readers, each with its rank, best other, margin and top value within the record's shortlist); ranker probability and rank; raw index candidate count; source flag; shortlist size; competition features against the record's other candidates; forensic name, legal-form, number and address comparisons; and, for the US and India only, **collective features** over the other incoming records competing for the same business (4 generic counts, for example how many of them have a different number set or a different legal form, plus 6 value features for the US).
- **Counts:** India 211, US 217 (India + 6), France and default 158 features (Doc 4). Spoken: "about two hundred features per country".
- Design detail: the generic word features ignore legal-form words, so a legal-form change shows up only in the legal-form features and in the readers (Doc D.1).

NUMBERS (REAL)

We removed 21 data-specific inputs and replaced them with 25 generic "what changed" features; a code check asserts that none of the removed inputs is used by a final decision model (Doc 4). On the public leaderboard that switch (upload 14) moved the score from 0.990881 to 0.990934 (Doc B.6).

WHY IT EXISTS

A similarity score treats different edits as the same if they cost the same. We studied how true records differ from their business before building features: true records of a business tend to lose information, by dropping a digit or a suffix, abbreviating or transliterating (Script slide 11). Describing the edit itself lets the model learn which edits are harmless. The features are generic operations, not word lists, so they carry over to France.

WHAT IT COSTS

Cheap per pair: string operations, plus one pass of the small pretrained encoder per distinct word of the records involved, computed once and then looked up (Code: `clean_feats.py`); the Documentation gives no separate time.

CONCEPTS USED (PARTS A0-A3)

Feature engineering; edit distance (Levenshtein, Damerau-Levenshtein, LCS ratio); IDF; Jaccard; word embeddings and cosine similarity; competition and collective (cross-record) features.

EXPERT NUANCE

"What changed" features are edit-operation descriptors, which a similarity-only matcher collapses into one number. Their strength is interpretability and transfer; their weakness is that the token alignment is a fast proxy (first new word, first dropped word, counts) rather than an optimal edit alignment, so a name with several edits is described only roughly. We ran no clean ablation that removes all "what changed" features (QA 21), and we did not prepare a feature-importance ranking (Explainer slide 11), so do not claim a measured gain beyond the upload-14 comparison.

JUDGE MIGHT ASK

- *Why describe the differences instead of adding more similarity scores?* "Because two edits with the same similarity can mean opposite things: dropping a legal suffix is typical of a true record, while swapping Inc for LLC often signals a different company. We studied how true records differ from their business before building features, and describing the edit lets the model learn from labels which edits are harmless."
- *A typo looks like a replaced word. Doesn't that punish true matches?* "On its own the word-overlap score would, which is why it is never alone: the same pair also carries 'replaced in the same position' and a character similarity, for a one-letter typo about 86 out of 100, and the readers see the raw text. The decision model learns from labels how these combine; we did not measure the typo case separately."

A4.10 Stop 7 - LightGBM decision model, one per country

SIMPLE

The supervisor now reads every completed form for the letter, including the experts' opinions and how each candidate compares with the others, and writes one probability per candidate: "how likely is this the right shop?". The supervisor is different for each country, because the countries write addresses and legal forms differently, and France has no answer key at all. The supervisor learned on old letters that the expert readers had never trained on, so they learned how far to trust the experts on letters the experts had not memorised. To be steadier, five supervisors trained with different random choices each give a probability, and we take the average.

OUR RECORD AT THIS STOP (ILLUSTRATIVE)

The US decision model gives A 0.93 and C 0.01.

TECHNICAL

- **Model:** LightGBM binary classifiers; learning rate 0.05; 127 leaves; at least 200 rows per leaf; feature fraction 0.9 (each tree sees a random 90 % of the features); bagging fraction 0.8 every iteration (each tree sees a random 80 % of the rows); L2 1.0; label "candidate is the true business" (Doc 4). LightGBM grows trees leaf-wise (it always splits the leaf that reduces the loss most, so trees can be deep and uneven, unlike CatBoost's symmetric trees) and finds splits on binned feature values (histograms), which is fast on millions of rows. Leaf-wise growth over-fits easily, so the 200-row minimum per leaf, the L2 penalty and the row and feature subsampling are the regularisers.
- **Training rows:** the train shortlist of incoming records that have a candidate in business folds 6-9, built from the index output without the quick filter (Doc 4). For the US and India the training rows also include extra hard examples; say this only with the QA 15 wording, and only if asked. The ranker and readers were trained on folds 0-5, so their scores on these rows are out-of-sample: this is stacking with a layer-wise fold split (QA 5).
- **Fit protocol** (Doc 4; code):

Country	Rows	Fit	Seeds	Threshold
US	US rows	fit on folds 6-7, early stopping on fold 8 (at most 3,000 rounds), then refit on folds 6-9 for best iteration $\times$ rows(6-9) / rows(6-7)	mean of seeds 1-5	0.75, then the consistency check
India	US and India rows	same as US	mean of seeds 1-5	0.75, then the consistency check
France	US and India rows of the normal training set (no French rows)	fit on 6-7, early stopping on 8 (at most 1,500 rounds), no refit	mean of seeds 1-5	0.82
Unseen country (default)	as France	single fit	1	0.82

- **The refit rule** scales the number of trees by the row ratio because no fold is left for early stopping after refitting on 6-9 (Doc 4). It is a common heuristic, not a guarantee: with more rows, each tree's fitted values are less noisy, so a model can usually take more boosting rounds before it starts to over-fit. Example (illustrative): best iteration 1,000 on folds 6-7 and twice as many rows in folds 6-9 gives 2,000 rounds for the refit.
- **France's reader slot:** the France decision model was trained with MiniLM-reader logits in its reader slot; at test time that slot receives the blend 0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5 (Doc 4).

NUMBERS (REAL)

Accepted links after the decision rule of Stop 8: US 2,250,036; India 2,726,453; France 887,630 before the France rules (Doc B.1, B.3). Held-out exact per-business macro F0.5 for close versions of the decision models (the India recipe with 3 of its 5 seeds, consistency check included; fold 9 / fold 8): US rows 0.991212 / 0.991545, India rows 0.990046 / 0.990298 (Doc 5). These exclude the second-chance searches and the France rules.

WHY IT EXISTS

A reader scores one pair at a time, but the decision is a competition between the candidates of one record. The decision model sees each candidate's rank, lead and best rival, the "what changed" features and all reader and ranker scores together, and it can reorder the ranker (Explainer slide 8).

WHAT IT COSTS

Training the US/India decision models, including building their training set, took about 1.0 hour in the clean run (stage t7); inference is a handful of small tree-model calls per pair (five seeds), not timed separately.

CONCEPTS USED (PARTS A0-A3)

Gradient boosting; LightGBM; stacking; early stopping; refit with scaled iterations; seed averaging; group folds.

EXPERT NUANCE

Three points. First, the refit on 6-9 for the US and India was chosen on the public leaderboard (0.990621 to 0.990676), and the same refit for France scored lower (0.990726 against 0.990748), so France keeps the 6-7 fit (Doc B.6; QA 5). Second, five seeds reduce the variance created by the row and feature subsampling, but their own effect is not isolated: they entered the final file together with the French E5 reader (QA 6). Third, putting a three-reader blend into a slot trained on one reader's logits creates a mild mismatch between training and test inputs; because the weights $0.35 + 0.15 + 0.5$ sum to 1, the blend is a weighted average of logits and stays on a logit scale, but its exact match to the MiniLM distribution was not separately measured.

JUDGE MIGHT ASK

- *Why put a decision model on top of the readers?* "A reader scores one pair at a time, but the decision is a competition between the candidates of one record. The decision model sees the reader logits together with the ranker probability, the 'what changed' features and each candidate's rank and lead over its rivals, and it reorders the ranker when needed, as in our US example."
- *What data trains the decision models?* "Labelled shortlist pairs of incoming records whose candidates lie in business folds 6-9, so the readers and ranker, trained on folds 0-5, never saw them; the US model uses US rows, the India model US and Indian rows, and the France and default models US and Indian rows with no French rows at all." Only if a judge presses on hard cases (QA 15): "for the US and India the decision models were also trained with extra hard examples: copies of unlinked training records that sit right next to a candidate business, with small name edits; how many to add was set from label-free counts on the test records, never from labels; the France model uses no such examples."

A4.11 Stop 8 - Confident + clear-winner rule

SIMPLE

The supervisor's rule is short. Take the most likely shop for this letter. Deliver only if you are confident (at least 0.75; 0.82 in France) and it is a clear winner (at least 0.4 ahead of the second most likely shop). Otherwise hold the letter back. A letter goes to at most one shop.

OUR RECORD AT THIS STOP (ILLUSTRATIVE)

A has 0.93, C has 0.01.

1. Confident? $0.93 \geq 0.75$: yes.
2. Clear winner? $0.93 - 0.01 = 0.92 \geq 0.4$: yes.
3. Link the record to A.

Counter-example (illustrative): if C had scored 0.60, the lead would be $0.93 - 0.60 = 0.33 < 0.4$, and the record would stay unlinked even though A is at 0.93. Two strong candidates mean ambiguity, and ambiguity is where wrong links happen.

TECHNICAL

Per incoming record: take the candidate with the best decision-model score; accept it only if $\text{score} \geq \text{thr}$ and $\text{score} - \text{second-best score} \geq 0.4$; otherwise accept nothing (Doc 4; Code: `stack.py decide`). This is an `argmax` (pick the highest-scoring candidate) with a reject option (the right to answer "none") on two conditions, an absolute threshold and a margin. If a record has only one candidate, the runner-up counts as minus infinity, so only the threshold matters (Code: `stack.py`; FACTS H9). Thresholds: 0.75 for the US and India, chosen on fold 8 for the exact per-business macro F0.5; 0.82 for France and the default route, set without French labels (Doc 4, Doc C.1). The 0.4 lead is one fixed value for every country; the Documentation does not record how it was picked (FACTS H8), so do not say it was tuned or that it was never tuned.

NUMBERS (REAL)

US 2,250,036 and India 2,726,453 accepted links; France 887,630 (Doc B.1, B.3). 90.59 % of incoming records have exactly one candidate in the final candidate file (Doc B.1).

WHY IT EXISTS

Under per-business F0.5 a wrong link costs more than a missed one, and a wrong link to a business with no true records turns its score from 1 into 0 (Stop 0).

WHAT IT COSTS

Negligible compute. In recall: correct candidates are rejected when a second candidate is too close or the score is below the bar; the Documentation lists this among the common false negatives (Doc 5).

CONCEPTS USED (PARTS A0-A3)

Decision thresholds; reject option (abstention); margin; tuning on the exact metric.

EXPERT NUANCE

A threshold without a margin would link records where two candidates are almost tied, which is precisely where the errors live (Explainer slide 8). Why is the margin not redundant? The decision model is a binary classifier that scores each pair on its own, so the scores of one record's candidates do not have to add up to 1: two candidates can both score 0.9 (illustrative). If the scores were a proper distribution over the candidates, a best score of 0.75 would already force the runner-up to at most 0.25, a lead of at least 0.5; here only the explicit margin guarantees a gap. The margin is an absolute difference of probabilities, so it interacts with calibration (whether a score of 0.8 really means "right 80 % of the time"): we did not calibrate the scores (no Platt scaling, which fits a logistic curve to held-out scores, and no isotonic regression, which fits a monotone step function) and instead tuned the threshold directly on the exact metric (FACTS F2).

JUDGE MIGHT ASK

- *How does the 0.4 lead work when a record has only one candidate?* "If there is no runner-up, the runner-up counts as minus infinity, so only the threshold matters; 90.59 % of records have exactly one candidate in the final candidate file. The lead rule bites exactly where it should, on records with two or more plausible businesses."

- *Why not link a record to two businesses if both look right?* "The task says each incoming record belongs to at most one business, so two strong candidates mean ambiguity, not two answers. Under F0.5 one of those two links would be wrong, so the rule abstains unless one candidate leads by at least 0.4."

A4.12 Stop 9 - Native-script second chance (India only)

SIMPLE

Some letters are held back not because the supervisor was unsure, but because the right shop never reached the desk at all. An Indian name written in Devanagari (the script of Hindi and Marathi) can send the first search to shops with the right name in the wrong towns. So a "lost letters" desk takes each still-unlinked native-script letter, writes the name in Latin letters in several possible spellings, drops words like "Private Limited", and searches only shops in the same state whose name, without such words, is exactly one of those spellings. A small model then picks the best, using the address, and links it only if it is very sure and far ahead.

OUR RECORD AT THIS STOP

Our record is a US record with a Latin-script name, so this stop does not apply. The real India example in section 17 walks through it.

TECHNICAL

(Doc 3, Doc 4; Code: p5_rescue.py)

- **Who enters:** Indian records with native-script names left unlinked by the decision model.
- **Keys:** legal-word-free word tuples of up to 64 transliterations, from a word list of labelled training matches (alternatives with at least 5 % of a native word's count). In plain terms: a native word may have several Latin spellings in the list; every spelling that accounts for at least 5 % of that word's training occurrences is kept, and the combinations across the words of the name, at most 64, form the search keys.
- **Search:** a key must equal the Latin core-name word tuple of a business exactly, and only businesses in the **same state** are searched (all states if the state is unknown or has no match).
- **Fixed score, top 5 kept:** a weighted sum of same state, first house number found, same digit string of an address segment, share of digit runs found, house-segment similarity, share of place words found, and address token-set ratio (a 0-100 string score that compares the two sets of words, ignoring order and repeats). In code: $2 \cdot \text{same state} + 3 \cdot \text{first house number found} + 2 \cdot \text{same digit-string segment} + 1.5 \cdot \text{share of digit runs found} + 0.02 \cdot \text{house-segment similarity (0-100)} + 1.5 \cdot \text{share of place words found} + 0.01 \cdot \text{token-set ratio (0-100)}$.
- **Decision:** a LightGBM second-chance model scores the 5; link the best if score ≥ 0.88 and its lead over the second ≥ 0.3 . The 0.88 was chosen on fold 8 with false positives counted twice.
- **Training:** India second-chance candidates of train folds 0-7, retrieved with word lists that leave out the record's own fold (cross-fitting), so a record never benefits from its own label.

NUMBERS (REAL)

649,965 pairs scored for 171,713 records; 578,317 of them were not in the shortlist; **+9,759 links** (Doc 3, B.1).

WHY IT EXISTS

It targets a retrieval failure found on training data: native-script names whose transliteration the Latin index does not retrieve (Doc 3). Lowering the main threshold would not help, because the right business is not among the candidates at all (Explainer slide 12).

WHAT IT COSTS

Small: it runs only on unlinked native-script records, and it scores at most 5 candidates each with a small tree model. The Documentation gives no separate time; its CPU retraining reproduced byte for byte (CodeREADME 6).

CONCEPTS USED (PARTS A0-A3)

Second-stage retrieval; transliteration; cross-fitting; cost-sensitive thresholds.

EXPERT NUANCE

The main index commits to one spelling per native word, a single-hypothesis transliteration; the second chance is a multi-hypothesis search (up to 64 spellings) that stays cheap because it demands an exact core-name match inside one state and keeps only 5 candidates by a hand-weighted address score before any model runs. It only touches still-unlinked records, so it can never undo a link (Script slide 12). It runs before the consistency check, and the check never adds a link for a record that already has a native-script second-chance link (Doc 4). One disclosure belongs here: the training records of both second-chance models were selected with earlier decision models whose inputs we later removed (Doc C.8); if pressed, say exactly that.

JUDGE MIGHT ASK

- *Why not put every transliteration into the main index?* "The main index already uses one transliteration per word. The second chance tries up to 64 spelling combinations per name, which would multiply the index work for every record; running it only for the 171,713 unlinked native-script records, inside the same state, keeps the cost small and targets the miss type we found on training data."
- *How did you stop the second-chance model from learning from its own labels?* "Cross-fitting: it trains on India candidates of folds 0-7, and each record's candidates are retrieved with a word list that leaves out that record's own fold, so it never benefits from its own label. Its threshold, 0.88, was chosen on fold 8 with false positives counted twice."

A4.13 Stop 10 - Per-business consistency check (US and India)

SIMPLE

After the first round of deliveries, a clerk looks at each shop and counts how many other letters were already delivered to it. If a shop already has one or two letters, it is very probably a real, active shop, so a slightly less certain letter is allowed through (the bar drops a little). If a shop already has three or more, one more letter would add little even if right and would still cost a lot if wrong, so the bar goes up a little. Every letter is re-decided once with the bar that fits its shop.

OUR RECORD AT THIS STOP (ILLUSTRATIVE)

Suppose business A already has 2 other records accepted by the decision model, so $k = 2$ and the bar is 0.72. Our record's 0.93 clears it with a lead of $0.92 \geq 0.4$, so the link stays. Two more illustrative cases to see how links move (both with a lead of at least 0.4 over their runner-up):

- A record with best score 0.70 whose business has $k = 1$ other accepted record: bar 0.68, so it is now **added** (it failed 0.75 in the main decision).
- A record with best score 0.76 whose business has $k = 3$ others: bar 0.77, so it is now **removed** (it passed 0.75 in the main decision).

TECHNICAL

(Doc 4) For each record, k = the number of **other** incoming records the decision model accepted for the same business. Accept if score ≥ 0.75 ($k = 0$), 0.68 ($k = 1$), 0.72 ($k = 2$), 0.77 ($k \geq 3$), with the same 0.4 lead. The four values were chosen on folds 8 and 9 for the exact metric. k is computed once from the main

decision, with no iteration. The check never adds a link for a record that already has a native-script second-chance link.

Why the shape: the break-even of one extra link under per-business F0.5. This is an illustration of the metric's arithmetic, not how the four values were chosen (they were tuned on folds 8 and 9). If an extra link is right with probability p , it is worth adding when $p \cdot \text{gain} > (1 - p) \cdot \text{loss}$, that is when $p > \text{loss} / (\text{gain} + \text{loss})$, the break-even probability. With the formula $F0.5 = 1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$, and the links already made assumed correct:

Business situation	F0.5 now	If the extra link is right	If the extra link is wrong	Break-even p
owns 3 records, 0 links so far	0	$1.25 \cdot 1 / 1.75 = 0.714$ (gain 0.714)	0 (loss 0)	0
owns nothing, 0 links so far	1 (correct empty row)	not possible	0 (loss 1.000)	any wrong link costs a full point
owns 3 records, 1 correct link	$1.25 / 1.75 = 0.714$	$2.5 / 2.75 = 0.909$ (gain 0.195)	$1.25 / 2.75 = 0.455$ (loss 0.260)	$0.260 / 0.455 = 0.57$
owns 3 records, 2 correct links	$2.5 / 2.75 = 0.909$	$3.75 / 3.75 = 1.000$ (gain 0.091)	$2.5 / 3.75 = 0.667$ (loss 0.242)	$0.242 / 0.333 = 0.73$

How to read it. With $k = 0$ the bar must guard against the second row, a business that owns nothing, where one wrong link turns 1 into 0; so it makes sense to keep the strict main threshold, 0.75, there. With $k \geq 1$, the other accepted links are mostly correct, so the business very probably owns records, the empty-row danger largely disappears and the break-even is lower (0.57 here): the bar can drop. As links accumulate, the gain from one more right link shrinks faster than the loss from a wrong one (break-even 0.73 at $k = 2$), so the bar climbs again. The break-even values assume calibrated probabilities and a known number of true records; our scores are not calibrated (Stop 8), so the tuned bars (0.68, 0.72, 0.77) need not equal them. The arithmetic explains the direction of the shape only.

NUMBERS (REAL)

Final run: US **+277 / -284**; India **+247 / -248** (1 addition skipped because of a native-script link) (Doc B.1).

WHY IT EXISTS

The value of one more link depends on how many links the business already has, because the metric is computed per business.

WHAT IT COSTS

Negligible compute; it moves few links.

CONCEPTS USED (PARTS A0-A3)

Per-business F0.5; count-dependent thresholds; tuning on held-out folds.

EXPERT NUANCE

Unlike the second-chance searches, this step can remove links. Its thresholds were chosen on folds 8 and 9, the same folds used for the held-out scores, which is one reason we say there is no untouched end-to-end validation (Doc 5). France does not use it.

JUDGE MIGHT ASK

- *Why is the bar lower with one other link and higher with three or more?* "The four values were tuned on held-out folds 8 and 9 for the exact metric, and the direction follows the metric. With no other link, the danger is a business that truly owns nothing, where one wrong link costs a full point; one or two accepted links make that unlikely, so the bar can drop to 0.68 and 0.72. As links pile up, a right extra link adds little while a wrong one still costs a lot, so the bar rises to 0.77."
- *Can the consistency check remove links?* "Yes, unlike the second-chance searches: in the final run it removed 284 US links and 248 Indian links and added 277 and 247. It never adds a link for a record

that already has a native-script second-chance link; one such addition was skipped in India."

A4.14 Stop 11 - Address second chance (US and India)

SIMPLE

Some letters have a name that has drifted far from the registered name, but an address that still matches. For letters still unlinked after everything else, a clerk searches by the address instead: shops that share one of the letter's rarest house numbers and one of its rarest street words. A first quick model throws out most of these, and a second model links the best survivor only if it is clearly convincing.

OUR RECORD AT THIS STOP

Our record was linked at Stop 8 and kept at Stop 10, so it never reaches this desk: second-chance searches act only on unlinked records. Had it been unlinked, its search keys would have been its rare digit run "42" and its rarest alphabetic address words, and only businesses sharing both kinds of key would come back.

TECHNICAL

(Doc 3, Doc 4; Code: p5b_addr_rescue.py, addr_lib.py)

- **Retrieval:** businesses that share one of the record's 3 rarest digit runs **and** one of its 3 rarest alphabetic address words (at most 300 per record), ranked by IDF containment (the IDF-weighted share of the record's address words found in the business's address) + $0.5 \cdot$ coverage (the IDF-weighted share of the business's address words found in the record's); the top 3 not already in the record's shortlist go on (Code: addr_lib.py).
- **Filter:** LightGBM, 10 features, 15 leaves, 200 rounds; it keeps at most 2 per record whose filter probability is at least the filter threshold, the largest value on a grid that still keeps 98 % of the fold-8 reference links (so the filter is tuned for recall).
- **Scorer:** LightGBM, 41 features, 31 leaves, 300 rounds, mean of seeds 7, 8 and 9; link the best kept pair if its scorer probability is at least the link threshold and leads the second kept pair's by at least the margin; threshold and margin chosen on fold 8 with false positives counted twice (so the scorer is tuned for precision).
- **Parameters** (filter threshold / link threshold / margin): US 0.06 / 0.86 / 0.0; India 0.015 / 0.66 / 0.1.
- **Training:** per country, on address-retrieval pairs of training records that the main decision leaves unlinked, folds 6-7.

NUMBERS (REAL)

US: 3,029,652 retrieved pairs, 1,827 filter survivors, **+192 links**. India: 4,766,484 retrieved pairs, 25,579 survivors (25,558 new candidates), **+2,945 links** (Doc 3, B.1). Together +3,137 (derived).

WHY IT EXISTS

It targets the second miss type found on training data: records whose name differs while the address matches (Doc 3).

WHAT IT COSTS

The heaviest step in memory: about 3.2 GB per worker, 13 GB with 4 workers (CodeREADME 1). Most retrieved pairs are scored only by the cheap filter and never written to the candidate file.

CONCEPTS USED (PARTS A0-A3)

Retrieval cascade (retrieve, filter, score); IDF; cost-sensitive threshold selection; seed averaging.

EXPERT NUANCE

It is a three-step cascade in miniature: wide cheap retrieval, a recall-tuned filter (98 % of reference links kept), then a precision-tuned scorer. Counting false positives twice when choosing thresholds is a simple cost-sensitive proxy for F0.5's preference for precision; it is not an exact translation of the metric (inside

one business F0.5 weighs a wrong link four times as much as a missed one in its denominator, Stop 0, and the macro average adds the empty-row effect). Note also that the US margin is 0.0, so in the US the link threshold alone decides; that is the value the fold-8 search chose. Some choices around it used public feedback: public scores helped calibrate the label-free estimate for transferring the address second chance to the US (Doc C.4).

JUDGE MIGHT ASK

- *Why require both a rare number and a rare word?* "Requiring both keeps the search small and specific: at most 300 businesses per record, and only the top 3 not already in the shortlist go on. A rare house number alone or a rare street word alone would bring in many unrelated businesses; the pair targets the miss type 'same address, different name'."
- *Why count false positives twice when choosing the second-chance thresholds?* "Because under F0.5 a wrong link costs more than a missed one, and these models act on records the main model already declined, where errors are likely. Counting each false positive twice builds that asymmetry into the choice; it gave the US 0.86, India 0.66, and 0.88 for the native-script search."

A4.15 Stop 12 - France rules computed by code (France only)

SIMPLE

France had no answer key, so the French branch of the post office adds a short book of house rules after the supervisor. Some rules hold back a letter the supervisor wanted to deliver, for example when the address is the same but one word of the shop name was swapped for a very common word. Others deliver a letter the supervisor held back, but only when the evidence is very specific, for example the shop's name plus one generic suffix such as "groupe" on the shop's own street, with no other compatible shop there. The rules are written as code that reads only the supervisor's scores, the raw text and the address; no person and no AI judges any letter.

OUR RECORD AT THIS STOP

US record, so not applicable. The real French example in section 17 shows a veto.

TECHNICAL AND NUMBERS (REAL; DOC B.3)

The rules run in this order on France shortlist pairs only and add no candidate. p1 and p2 are the best and second-best decision-model scores of the record.

Step	Rule (plain name)	What it does	Count	France links after
0	decision model	confident + clear winner at 0.82 / 0.4	887,630 accepted	—
1-2	category-swap veto; word-substitution veto and rescue	veto: at the same address the business lost a core word and the record gained a category word, or one business word was replaced by a frequent word; rescue: unlinked records with a true-looking name difference, $0.4 \leq p1 < 0.82$, $lead \geq 0.4$	vetoed together -29,021; rescue +5,272	
3	residual rescue	unlinked, $p2 < 0.05$; record name = business name + one generic suffix (for example groupe, france, services) in the conditions listed in Doc B.3; the top business is the only compatible one at the address	+5,846	869,727
4	street veto	drop a link when none of the business's street-name words appear in the record's address, allowing typos	-826	868,901
5	generic-word swap veto	same word count, one position where the record has "compagnie" or "services" and the business a non-generic word; same address	-538	868,363
6	generic-word rescue	unlinked, not vetoed, $p2 < 0.05$, same address, name differs only by "groupe" or "developpement" in specific positions, p1 in a low band	+332	868,695
7	cell veto	drop accepted links in seven specific combinations of address relation and name change	-1,322	867,373

Step	Rule (plain name)	What it does	Count	France links after
8	acronym rescue	unlinked, not vetoed, $0.4 \leq p1 < 0.82$, same address, the record name holds an initials variant of the business name, unique at that address	+186	867,559 (final)

Check: $887,630 - 29,021 + 5,272 + 5,846 = 869,727$ (derived). Counts are actions in execution order, not per-rule accuracies (Doc B.3). The full story, including how the rules were found and checked without labels, is in Section 18.

WHY IT EXISTS

Without labels we cannot train a French decision model; the rules encode precise, checkable conditions where the borrowed US/India model was unsure or risky (Explainer slide 13).

WHAT IT COSTS

Cheap per pair; the real costs are disclosure and the risk of fitting the test set (Section 18).

CONCEPTS USED (PARTS A0-A3)

Deterministic post-model rules; label-free validation with control groups.

EXPERT NUANCE

Limitation first: the rules were designed after analysing the unlabelled French test records, including AI-assisted review of sampled pairs, and some parameters were chosen after seeing test results, including public-leaderboard scores (Doc C.2, C.4). So they carry a real risk of fitting the test set, and there is no French F0.5 to measure that risk. Mitigation: each rule is code on decision-model scores, raw text and the address; no test label and no human or AI judgement of any pair is an input when the pipeline runs, and the evidence behind the rules uses no French labels (control groups on the French records, such as same-street records without a house number; for the cell veto, the independence of name noise and address noise, measured on US labels) (Doc B.3, C.2). Technically, each rule is a hand-written region of feature space with a forced decision layered on top of the model: a veto overrides a confident score, and a rescue accepts a record the model left unlinked, under narrow conditions on $p1$ and $p2$ (a band for $p1$, or $p2 < 0.05$) and, for most rescues, a condition that the business is the only compatible one at that address. Because the rules run in a fixed order and later rules skip records vetoed earlier, the counts in the table are not independent per-rule effects. The cleaner production design is to turn the rule conditions into features and learn them from a small labelled French sample (QA 10).

JUDGE MIGHT ASK

- *Do the France rules only remove links?* "No. Four of them add links: the word-substitution rescue (+5,272), the residual rescue (+5,846), the generic-word rescue (+332) and the acronym rescue (+186); the others remove links, the largest being the two word-swap vetoes together (29,021 of 887,630 accepted). Net, France goes from 887,630 accepted links to 867,559."

A4.16 Stop 13 - Output and validation

SIMPLE

At the end the post office writes two logs, one line per shop. The first lists which letters were delivered to each shop. The second lists every shop-letter pair that was seriously considered. A checker makes sure every delivered letter was also on the considered list, and then the organisers' own auditor program checks both logs. Ours passes.

OUR RECORD AT THIS STOP (ILLUSTRATIVE)

Business A's row in the matching file lists our record (and its other linked records). The candidate file lists our record under A and C, the two businesses of its shortlist. If A truly owns exactly the 3 records we linked, A scores 1.000 (Stop 0).

TECHNICAL

(Doc 2.2, Doc 3, Doc B.1; CodeREADME 5)

- `matching_results.tsv`: one row per test business with its linked incoming records.
- `candidate_pairs.tsv`: one row per test business with its candidates = the shortlist + native-script second-chance candidates + address second-chance filter survivors not already in it.
- The export asserts that every matched pair is a candidate pair; the last stage runs the organisers' validator `validate_submission.py --check-ids`: PASS.

NUMBERS (REAL)

- **5,856,936 links**: US 2,250,221; India 2,739,156; France 867,559. Check: US 2,250,036 + 277 – 284 + 192 = 2,250,221; India 2,726,453 + 247 – 248 + 9,759 + 2,945 = 2,739,156 (derived).
- 1,632,457 businesses with at least one link; **100,087 empty rows** (5.8 %, derived); links per business mean 3.3805, median 3, maximum 11 (Doc B.1).
- **12,293,019 candidate pairs**, 7.0954 per business, 1.2331 per incoming record. Check: 11,687,317 shortlist + 578,317 native-script + 1,827 US address + 25,558 India address = 12,293,019 (derived).
- Reduction ratio 0.99999817, so fewer than two pairs in every million possible pairs are written (1.83 per million, derived) (Doc 3).
- No pair joins records of different countries, and no incoming record is linked to more than one business (Doc B.1). Every incoming record except 2 has at least one candidate; 125 businesses have none (Doc B.1).
- Both files have 1,732,544 rows; validator PASS (Evidence: `validation_log_package_output.txt`).

WHY IT EXISTS

The organisers judge both the match list and the candidate file, so a small candidate file is a goal in itself (Explainer slide 2), and a file that fails the validator scores nothing.

WHAT IT COSTS

Writing and validating is quick. Reproducibility: from the trained models, the inference stages rewrite both submitted files byte for byte; a full retrain from raw data (about 11.5 hours, one RTX 4070 Ti SUPER 16 GB, 32 GB RAM, about 70 GB disk) reproduced 99.82 % of the links and passed the validator; it is not bit-identical because GPU CatBoost training and bf16 reader training are not bit-deterministic (CodeREADME 1, 6).

CONCEPTS USED (PARTS A0-A3)

Validation hygiene; reproducibility; candidate efficiency.

EXPERT NUANCE

The candidate file is a union (shortlist + second-chance candidates), so its size, 7.0954 per business, is larger than what the readers saw, 6.75 per business (derived); the export assertion guarantees that the matching file is a subset of the candidate file, so a link can never be "uncandidated". On reproducibility, two different claims must not be mixed up. Byte-identical files need the trained weights and the stored reader logits of the submitted run, because bf16 scores also depend slightly on the device and on how pairs are batched (each batch is padded to its longest pair) (CodeREADME 6). A retrain from raw data gives very close but different files (99.82 % of links reproduced; identical match lists for 99.07 % of businesses), because the GPU training of the ranker and the bf16 training of the readers are not bit-deterministic (Doc A; CodeREADME 6). The regenerated file was never uploaded, so its public score is unknown (clean-run report).

JUDGE MIGHT ASK

- *How do you guarantee the two output files agree?* "The export asserts that every linked pair is also a candidate pair, and the last stage runs the organisers' validator with ID checks, which passes. Both files

have one row for each of the 1,732,544 businesses."

- *Could someone else reproduce your file?* "Yes. From our trained models and stored reader scores, the inference stages rewrite both submitted files byte for byte; a full retrain from raw data, about 11.5 hours on one 16 GB GPU, reproduced 99.82 % of the links and passed the validator. It is not bit-identical because GPU training of the ranker and bf16 training of the readers are not bit-deterministic."

A4.17 The three real examples, retold

These are real test records traced through the submitted run (Doc D). Test labels are private, so the examples show **how** the pipeline decides, not whether each link is correct. Say that sentence whenever you present them.

A4.17.1 US: the legal form and one word decide (Doc D.1)

Incoming record **S2-672512509**: name "Legacy Marketing Enterprises (Inc)", **no address**.

1. **Normalisation**. The name becomes "legacy marketing enterprises inc" (computed with our code); with no address, the address channels have nothing to work with.
2. **Index lookup, quick filter, ranker**. The shortlist ends with 4 businesses, ranker ranks 0-3, all with almost the same name:

Ranker rank	Business	Decision-model score	What changed
0	Legacy Marketing Enterprises Inc; 46 Lincoln Way, Windsor, CT	0.965022	nothing in the name; same legal form
1	Legacy Marketing Enterprises LLC; 2495 St Paul Boulevard, Unit Unit 5, Irondequoit, NY	0.000007	legal form Inc to LLC
2	Legacy Marketing Enterprises PC; 63189 La Brisa Drive, Joshua Tree, CA	0.088664	legal form Inc to PC
3	Legacy Marketing Industries Inc; 2360 Airbase Road, Louisville, TN	0.000088	one word replaced in place (IDF-weighted Jaccard 0.52, meaning similarity of the replaced word 0.77)

3. **Readers and "what changed" features**. With no address, only the name separates the four. The generic word features ignore legal-form words, so for them the three "Enterprises" names are identical; the legal-form features and the readers tell them apart, and the word-replacement features mark "Industries" (Doc D.1).
4. **Decision model**. It reorders the ranker: the LLC, ranker rank 1, gets the lowest score of all, 0.000007.
5. **Confident + clear winner**. $0.965022 \geq 0.75$, and the lead is $0.965022 - 0.088664 = 0.876 \geq 0.4$, so the record is linked to the Windsor, CT business only.
6. **Consistency check**. It keeps the link: 0.965 clears every bar of the check (0.68 to 0.77).
7. **Address second chance**. Not used: it looks only at unlinked records.
8. **Output**. The Windsor business's row lists this record with two other incoming records; the candidate file lists the record under all four businesses.

WHAT IT TEACHES

"What changed" in action: when names are almost identical and there is no address, the type of difference (legal form, one replaced word) decides, and the decision model can overrule the ranker.

JUDGE MIGHT ASK

- *The ranker put the LLC second; why is it last after the decision model?* "The ranker only sees index statistics, where Enterprises Inc and Enterprises LLC look almost the same. The decision model also

sees the legal-form change from Inc to LLC and the reader scores, so it drops the LLC to 0.000007 and keeps the Inc at 0.965."

- *Was it the legal-form features or the readers that made the difference?* "We did not isolate that for this record. Both carry the signal: the legal-form features mark Inc to LLC and Inc to PC, the readers see the raw names, and the generic word features, which ignore legal words, see the three Enterprises names as identical."

A4.17.2 India: the native-script second chance (Doc D.2)

Incoming record **S3-309374692**: name "विजय एंटरप्राइजेज प्राइवेट लिमिटेड", address "24/11, Mumbai, महाराष्ट्र".

1. **Main pass.** The shortlist holds three businesses called "Vijay Enterprises Private Limited", in Agra (Uttar Pradesh), Satara (Maharashtra) and Byculla, Mumbai (Maharashtra). The ranker's top-1 probability is only 0.0028, and the decision model scores them $1.9\text{e-}6$, $6.9\text{e-}6$ and $7.2\text{e-}6$: far below 0.75 and below every consistency-check bar (0.68 to 0.77). The record stays unlinked.
2. **Enter the native-script second chance.** The name is in Devanagari and the record is unlinked, so it qualifies.
3. **Transliteration key.** The word list learned from training matches maps विजय to vijay, एंटरप्राइजेज to enterprises, प्राइवेट to private and लिमिटेड to limited, each as the only alternative. Without the legal words, the key is (vijay, enterprises). The address gives the state: Maharashtra.
4. **Second search.** 10 businesses in Maharashtra have the core name "vijay enterprises"; the fixed address score keeps the top 5.
5. **Fixed score of the winner.** "Vijay Enterprises Private Limited", "24/11, Bombay Sarggam, Vallabh Baug Ghatkopar(E), Mumbai, Maharashtra": same state, same house number 24/11 (both digit runs match), address token-set ratio 100 (66.7 for each of the other four; all five are in Mumbai, so the place-word share is 1.0 for each). Fixed score $13.0 = 2$ (state) + 3 (first house number) + 2 (same digit-string segment) + 1.5 (all digit runs found) + 2 (house-segment similarity 100×0.02) + 1.5 (all place words) + 1 (token-set ratio 100×0.01); the next best is 4.78.
6. **Second-chance model.** Score 0.99994; the second best (Bhandup West) 0.0000191; lead 0.99992.
7. **Decision.** $0.99994 \geq 0.88$ and lead ≥ 0.3 , so the link is added.
8. **Consistency check.** It runs after this step and would never add a different link for this record, because the record already has a native-script link.
9. **Output.** The record appears in the Ghatkopar business's row; the candidate file lists it under 7 businesses (3 from the shortlist, 5 from the second chance, 1 in both).

WHAT IT TEACHES

A retrieval miss cannot be fixed by a lower threshold. The Latin index found the right name in the wrong places; the second search narrowed by transliterated name and state, and the house number 24/11 picked the right business.

JUDGE MIGHT ASK

- *Why did the main pass give the three exact-name businesses almost zero?* "The names match, but nothing ties those three businesses to this record's address, 24/11 in Mumbai: they are in Agra, Satara and Byculla, and the ranker's best probability was only 0.0028. So the decision model scored them between about 2 and 7 in a million, far below any bar."
- *What if the house number had been missing?* "We did not test that case. The house number is what separated the right business from the other four Mumbai candidates, an address token-set ratio of 100 against 66.7, so without it the lead rule would decide whether anything is linked."

A4.17.3 France: a confident link vetoed (Doc D.3)

Incoming record **S2-30376547**: "TQ COMITE SARL", "2 R SUZANNE LENGLEN, NANTES".

1. **Normalisation (French rules).** "2 R SUZANNE LENGLEN" becomes "2 rue suzanne lenglen"; the name becomes "tq comite sarl" (computed with our code).
2. **Index lookup.** 16 candidates.
3. **Quick filter.** Keeps 2.
4. **Ranker and shortlist.** Both enter the shortlist (ranker probabilities 0.9991 and 0.0420, both ≥ 0.001).
5. **Readers.** France uses three readers, blended as 0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5:

Ranker rank	Business	Ranker prob.	MiniLM	French MiniLM	French E5	Blend	Decision score
0	TQ Parents SARL; 2 Rue Suzanne Lenglen, Nantes, Pays de la Loire	0.9991	7.5000	8.9375	10.8125	9.3719	0.999837
1	TQ Comite SARL; 2 RUE du Lycee, Pessac, Nouvelle-Aquitaine	0.0420	7.1250	-1.9219	-10.8125	-3.2008	0.000098

Blend check for rank 0: $0.35 \times 7.5 + 0.15 \times 8.9375 + 0.5 \times 10.8125 = 9.371875$ (derived). The blend rejects the identical-name business in another city, which the plain MiniLM reader alone scored +7.1.

6. **Decision model and rule.** It accepts rank 0: $0.999837 \geq 0.82$ and lead $0.999740 \geq 0.4$.
7. **France rules.** The category-swap veto fires: same house number (2), street token-set ratio $100 \geq 80$, the business name lost the core word "parents", and the record gained the category word "comite". The word-substitution veto also lists the pair ("parents" replaced by the frequent word "comite" at the same address). The link is dropped.
8. **No later rule re-links it.** $p1 = 0.999837$ is above the rescue band $[0.4, 0.82]$; no residual-rescue pattern applies; the generic-word rescue and the acronym rescue skip records vetoed by the first two rules.
9. **Output.** The record is in no row of the matching file and is listed in the candidate file under both businesses; TQ Parents SARL keeps its link to another incoming record, S2-732760024.

WHAT IT TEACHES

Two things at once: the French readers fixed a reader error (identical name, wrong city), and the France rules overruled a very confident model on a risky pattern. Under F0.5, abstaining is cheaper than a wrong link (Script slide 18).

JUDGE MIGHT ASK

- *Couldn't TQ Parents be the right business?* "It could; the labels are private. The rule is a precision choice: if the link was right we lose a little recall on that business, if it was wrong it would have cost precision, and TQ Parents still keeps its link to another incoming record."
- *Why not link the exact-name TQ Comite SARL in Pessac?* "It sits in another city, Pessac, not Nantes. The plain MiniLM reader liked the identical name, +7.1, but both French readers rejected it, the blend was -3.2, and the decision model gave it 0.000098."

A4.18 France: how we matched with zero labels

IN ONE LINE

France had no labels, so we borrowed a decision model trained on labelled US and Indian data, let two French readers teach themselves from their own most confident French guesses, used a stricter bar, and added rules computed by code, all disclosed.

SIMPLE EXPLANATION

Imagine opening a new branch of the post office in France, where nobody has ever recorded which letter belongs to which shop. You bring your experienced US and Indian supervisor, who knows in general what a genuine letter looks like, but French addresses and company words (SARL, "rue", "bis") are new to them. So you let your expert readers practise on French letters they are extremely sure about, either almost certainly right or almost certainly wrong, and you keep giving them old US and Indian letters with known answers so they do not forget what they learned. Because you cannot check the French answers, you ask for more confidence before delivering (0.82 instead of 0.75). Finally, you write a small book of house rules for situations where the supervisor tends to be over-confident, such as the same address with one name word swapped for a very common word. The honest catch: you can never measure French accuracy directly, you designed the rules after looking at the French letters, and you say so.

WORKED EXAMPLE

Part 1, choosing pseudo-labels for the French E5 reader (rule real, records and logits illustrative). Confidence p = the average of sigmoid(MiniLM logit) and sigmoid(French MiniLM logit). A positive needs $p \geq 0.97$ on the record's best pair and every other pair of that record below 0.5; a negative is the record's highest- p pair among those with $p \leq 0.03$, the hardest confident non-match (Doc 4).

Record	Pair	MiniLM logit	French MiniLM logit	p	Becomes
R1	best pair	5.0 (sigmoid 0.9933)	4.0 (0.9820)	0.9877	positive (≥ 0.97 , and R1's other pair has $p = 0.10 < 0.5$)
R2	best pair	—	—	0.80	not a positive (below 0.97)
R2	pair 2	—	—	0.02	negative (the highest p among R2's pairs at or below 0.03)
R2	pair 3	—	—	0.001	not used (a less informative, easier negative)
R3	only pair	—	—	0.50	nothing (neither confident match nor confident non-match)

Part 2, blending the readers (real numbers, Doc D.3). For "TQ COMITE SARL" against "TQ Parents SARL": $0.35 \times 7.5 + 0.15 \times 8.9375 + 0.5 \times 10.8125 = 2.625 + 1.3406 + 5.4063 = 9.3719$. Against the identical name in Pessac: $0.35 \times 7.125 + 0.15 \times (-1.9219) + 0.5 \times (-10.8125) = -3.2008$. The plain MiniLM reader liked both; the blend keeps one and rejects the other.

Part 3, the stricter bar. With $p_1 = 0.84$ and $p_2 = 0.30$ (illustrative), a US record would pass ($0.84 \geq 0.75$, lead $0.54 \geq 0.4$); a French record also passes ($0.84 \geq 0.82$). With $p_1 = 0.80$, the US record passes but the French one does not.

TECHNICAL EXPLANATION

The French part combines six techniques.

- Transfer of the decision model.** The France decision model is LightGBM trained only on labelled US and Indian rows of the normal training set (158 features, mean of 5 seeds, fit on folds 6-7 with early stopping on fold 8, no refit), so it encodes generic "what changed" knowledge rather than French specifics (Doc 4).
- Self-training with confidence-thresholded pseudo-labels.** A teacher's confident predictions on unlabelled target data become training labels for a student. The French MiniLM reader was fine-tuned on very confident pseudo-labels from an earlier run of our pipeline (best candidate, score at least 0.97, clear lead) and confident non-matches, plus 500,000 labelled US/India pairs. The French E5 reader used the combined confidence of two readers as the teacher (the average of their two probabilities, the rule above), at most 250,000 pairs per class, 5 % of records held out, 474,808 French pairs plus 474,808 labelled US/India pairs, 7,419 steps at learning rate $2e-5$ (Doc 4); $949,616 \text{ pairs} / \text{batch } 128 = 7,418.9$, so one epoch is 7,419 steps (derived). The risk of self-training is **confirmation bias**: the student can learn the teacher's mistakes, and very confident pseudo-labels are also the easiest cases, so they teach less about the hard ones. The very high confidence bar and the replay of labelled pairs

limit, but do not remove, that risk. The held-out pseudo-labels do not limit it: agreement with them (0.999206) only shows that the student reproduces its teacher, not that the teacher is right. That is why the generated labelled French test set was the more independent check, and it failed (Doc B.4).

3. **Replay against forgetting.** Mixing in labelled US/India pairs (about one to one) keeps the fine-tuned readers from drifting away from what they learned on labelled data, a standard defence against catastrophic forgetting during domain adaptation (Script slide 13).
4. **Logit blending into a fixed slot.** The France decision model was trained with MiniLM logits in its reader slot; at test time the slot receives 0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5. Because the weights sum to 1 it is a weighted average of logits. Averaging logits is not the same as averaging probabilities: it is a weighted geometric mean of the odds, so one very confident reader can dominate. In the Pessac example the French E5 reader's -10.8125 , with weight 0.5, alone contributes -5.41 and pulls the blend to -3.2008 , although the plain MiniLM reader said $+7.125$. The French MiniLM weight was set by label-free counts on the test records (Doc C.1); $0.35 / 0.15 / 0.5$ equals half of the previous blend (0.7 MiniLM + 0.3 French MiniLM) plus half of the French E5 reader.
5. **A stricter operating point.** 0.82 instead of 0.75, set without French labels from the French link volume per business together with a fold-8 threshold sweep on US/India (Doc C.1). The 0.4 lead is unchanged.
6. **Rules computed by code**, after the decision model (Section 15, table). They read only model scores, raw text and the address. Their evidence uses no French labels: control groups on the French records (for example, records on the same street without a house number) and, for the cell veto, the independence of name noise and address noise measured on US labels (Doc B.3). Background from our code's documentation, not the submitted Documentation, use only if asked: the street veto rests on a US fact; among 1,313,608 decidable true US top-1 matches only 48 (0.004 %) have a street name sharing no word with the business's street, while 60,565 of 60,613 (99.92 %) US top-1 pairs with a genuinely different street are wrong.

Checks of the French E5 reader, written before adoption (Doc B.4):

Check	Result	Outcome
US/India quality within tolerance (509,260 labelled fold-8/9 pairs)	AUC 0.998544 vs 0.998570; top-1 accuracy 0.993161 vs 0.993004; best F0.5 0.992782 vs 0.992796	pass
Agreement with held-out French pseudo-labels (25,192 pairs)	0.999206 (the E5 reader 0.981304); threshold 0.98	pass
Generated labelled French test set (195,380 pairs)	best F0.5 0.910516 vs 0.914964 for its base model	fail
"Expected gain safely positive" rule (final file vs the same pipeline without it)	French links $-2,228 / +3,396$; pessimistic gain -0.0001806	fail

HOW WE USED IT

France flows through the same index lookup (32.7713 candidates per business after the quick filter), the same CatBoost ranker trained on US/India, and a shortlist of 2,103,107 pairs (8.1060 per business) (Doc 3). The three readers are blended 0.35 / 0.15 / 0.5; the France decision model accepts 887,630 links at 0.82 / 0.4; the France rules bring that to 867,559 (Doc B.3). France has no consistency check and no second-chance searches. Final France numbers: 244,207 businesses with at least one link, 15,245 empty rows, 3.3438 links per business (Doc B.1). A diagnostic upload showed how much France matters: emptying every French row of an early file dropped the public score from 0.970125 to 0.839 (Doc B.6). What we tried and dropped, as reported: refitting the France decision model on folds 6-9 (public 0.990726 against 0.990748 without it), a second French self-training round (0.991277 against 0.991300), and the French MiniLM reader retrained with different pseudo-label filters (0.991258) (Doc B.6). The French E5 reader was kept although it failed two of its four written checks, because the file containing it had our best public score; that same upload also added the 5-seed decision models, so its own effect is not isolated (QA 8).

EXPERT NUANCE

State the limitations first, then the mitigation. **We have no measured French F0.5.** The closest number is a proxy: the France recipe applied to US and Indian records at the France operating point scores 0.990508 / 0.989467 on fold 9 and 0.990977 / 0.989850 on fold 8, which says the recipe is sound on labelled countries, not that it is right in France (Doc 5). **The rules were designed after analysing the unlabelled French test records**, including AI-assisted review of sampled pairs, and some parameters were chosen after seeing public-leaderboard results (Doc C.2, C.4); the mitigation is that each rule is code on scores and text, with no test label and no human or AI judgement of any pair as an input. **The French E5 reader failed two written checks** and was kept for a leaderboard reason. A cleaner production design would label a small French sample, turn the rule conditions into features of a French decision model, and set the threshold on labels (QA 10, 18). Alternatives a scientist may raise, such as adversarial domain adaptation or translating French records, are not part of our final approach.

REMEMBER

- No French labels: decision model borrowed from US/India rows; readers self-trained on their most confident French guesses with about one-to-one US/India replay; bar 0.82; rules computed by code.
- 887,630 accepted, then 867,559 after the rules; the two word-swap vetoes removed 29,021.
- Say the three limits plainly: no measured French F0.5; rules designed after looking at test records; French E5 reader kept despite failing two of four checks.

JUDGE MIGHT ASK

- *How do you know self-training did not just teach the readers their own mistakes?* "We cannot fully rule it out; that is the known risk of self-training. We limited it with very confident pseudo-labels and an equal number of labelled US and Indian pairs, and US/India quality stayed within tolerance; the 0.999206 agreement with held-out pseudo-labels only shows the student matches its teacher. The more independent check, a generated labelled French test set, failed, 0.910516 against 0.914964, as did our expected-gain rule, and we kept the reader because the file with it scored best on the public board."
- *How did you choose 0.82 without French labels?* "Without labels: we compared the number of French links per business at different thresholds with a threshold sweep on labelled US and Indian fold 8, and chose 0.82, stricter than the 0.75 we use where we have labels. The 0.4 lead stays the same."

A4.19 Traps to avoid when you describe the pipeline

1. "About 130 per business" comes from the **blocking study of 25 September**, not the final run. The final run reports 21.7107 after the quick filter.
2. 6.75 per business and 1.17 per record for the shortlist are **derived** (11,687,317 / 1,732,544 and / 9,969,589). The candidate file is 7.0954 per business, 1.2331 per record.
3. The CatBoost ranker is a **pointwise classifier** with log loss. Never say it uses a ranking loss.
4. The quick filter's cost (0.990117 to 0.990086) was measured **with the decision model of 25 September**.
5. **France has no consistency check and no second-chance searches**, does not use the plain E5 reader, and its decision model has **no French rows**.
6. **The second-chance searches never undo a link; the consistency check can** (US -284, India -248).
7. The real order for the US and India is: native-script second chance (India), then the consistency check, then the address second chance. The talk groups the two searches together for simplicity.
8. The readers read the **raw** text; the index and hand-made features use the cleaned text.

9. "1.4 hours of reader scoring" is from the **clean run** (two readers per pair, all except the French E5 reader); the submitted run reused stored logits.
10. Held-out scores (US 0.991212 / 0.991545; India 0.990046 / 0.990298) are for **close versions** of the decision models (the India recipe with 3 of its 5 seeds) and **exclude** the second-chance searches and France rules.
11. The 0.4 lead is **one fixed value for every country**; the threshold 0.75 was tuned on fold 8. Do not claim the lead was tuned, and do not say it was never tuned.
12. The unseen-country route is **untested** on real data.
13. There is **no measured French F0.5**; the France "proxy" numbers are US and Indian records.
14. Never use internal code names on stage. For hard near-identical cases use the QA 15 wording, and only if asked.
15. The cap of four rarely binds (1.17 pairs per record on average); do not say the cap is what cuts the candidates down. The quick filter and the 0.001 floor do most of the narrowing; the cap bounds the worst case.
16. The break-even arithmetic of the consistency check explains only the **direction** of its shape; the four bars were **tuned on folds 8 and 9**. Check the Stop 10 table for the metric's own numbers.
17. Agreement with held-out French pseudo-labels (0.999206) shows that the student matches its teacher; it is **not** evidence of French accuracy.
18. The index stage on test runs with 6 threads by default; the "8-11 threads" in the code README refer to the training-stage index runs.

A4.20 Explain the whole pipeline in one sentence, 30 seconds and 2 minutes

ONE SENTENCE (ABOUT 20 SECONDS)

"Per country, a cheap index and two pruning steps cut 6.7 trillion possible pairs to about seven candidates per business, small language-model readers and a per-country decision model score them, and we link a record only to a confident, clear winner, with second-chance searches and French rules on top."

30 seconds (about 80 words). "Per country, we clean the text and a C++ inverted index pulls up businesses that share rare words, letter pieces or sounds, about 130 per business. A quick rule with no model cuts that to 21.7, and a CatBoost ranker keeps at most four per record. Cross-encoder readers score each pair, 'what changed' features describe the differences, and a LightGBM decision model links only a confident, clear winner. Second-chance searches, a consistency check and France rules finish it: public F0.5 0.9913."

2 minutes (about 300 words). "We link about ten million incoming records from the US, India and France to 1.7 million businesses, each record to one business or to none. The metric is F0.5 per business, and a business with no true records scores a point only if we link nothing, so we are precision first. The pipeline runs per country and goes from cheap to expensive.

First we clean the text: lower case, abbreviations unified, Indian-script words transliterated with a word list learned from training matches, and French address and legal-form rules. A C++ inverted index then finds businesses that share rare words, letter pieces or sounds, weighted by IDF, with separate lists for name and address so one-sided matches survive. That gives about 130 candidates per business. A quick rule-based filter with no model cuts that to 21.7, at a held-out cost of 0.00003 F0.5, and a CatBoost ranker keeps at most four per record: 6.75 per business.

Cross-encoder readers, fine-tuned multilingual MiniLM and E5, read each record and candidate together. We add 'what changed' features: which numbers changed, which words were added, dropped or replaced,

how rare and how close in meaning, and whether the legal form changed. A LightGBM decision model per country turns about two hundred features into a probability, and we link only a confident, clear winner: at least 0.75, or 0.82 in France, and 0.4 ahead of the runner-up.

Then targeted fixes: two second-chance searches for unlinked records, by transliterated name in India and by rare address keys, adding 9,759 and 3,137 links; a per-business consistency check; and for France, which has no labels, self-trained French readers and rules computed by code.

The final file scores 0.9913 on the public leaderboard, and close versions of our decision models score between 0.990 and 0.992 on held-out folds."

If a judge then asks for limitations, add: "We had no untouched end-to-end validation of the final file, France has no labels and its rules were designed after looking at unlabelled test records, and we kept the French E5 reader although it failed two of its four written checks."

Part B - Q&A bank

258 questions in 21 categories, numbered Q1 to Q258 across the whole bank. Every item has the same parts:

- *Why they ask* (grey italics): what the judge is really testing.
- **Say** (in the box): the spoken answer, 20-35 seconds. Direct answer first, then one number, then stop.
- **If they push**: the expert follow-up, only if they ask again. **If they push further** appears where two near-identical questions from the source banks were merged into one item: it is the follow-up of the merged answer.
- **Numbers**: the figures behind the answer, with their sources (Doc = our submitted Documentation, the primary truth; CodeREADME / Evidence / Code = the code package, true but secondary, use only if asked; derived = simple arithmetic on documented numbers; background = correct general knowledge not in our Documentation, use only if asked).

Badges: ★ basic, ★★ probing, ★★★ hard or adversarial; LIKELY = a question we expect the judges to ask. Where an answer is our own reasoning rather than a measurement, it says so; never turn it into a claimed result. A *See also* line at the end of a category points to questions of that category that are answered elsewhere.

1. Problem and data (15 questions)

Q1. What exactly is the task, in one sentence? ★ LIKELY

Why they ask: They check that you can frame the problem precisely before talking about models.

SAY We link each incoming record, from Source 2 or Source 3, to at most one business in the reference list, or to none, and one business may own several incoming records. All matching happens inside one country, and the test set has about 10 million incoming records against 1.7 million businesses.

IF THEY PUSH It is many-to-one record linkage against a reference list, not clustering: there is no record-to-record graph to close, so the natural decision is "which business, if any, for this record". The score is computed per business, and both output files have one row per business.

NUMBERS

- 9,969,589 incoming records; 1,732,544 businesses; US, India, France (Doc 2.1)

Q2. How large is the data, country by country? ★

Why they ask: Scale drives every design choice; they want to see that you know your own numbers.

SAY India is the largest, then the US, then France, and every country has between 5.5 and 5.8 incoming records per business. In total that is about 10 million incoming records against 1.7 million businesses.

IF THEY PUSH Per business it is 5.756 in the US, 5.824 in India and 5.531 in France, 5.75 overall. Our output links 3.38 records per business on average, so many incoming records end up unlinked.

NUMBERS

- US 663,106 businesses / 3,817,031 incoming records; India 809,986 / 4,717,565; France 259,452 / 1,434,993 (Doc 2.1)
- 3.3805 links per business in our output (Doc B.1)

Q3. What makes this problem hard? ★ LIKELY

Why they ask: They test whether you understood the real difficulty, not just the mechanics.

SAY Three things. Scale: all same-country pairs would be 6.7 trillion. Look-alikes: many businesses share almost the same name, so the decision is between near-identical candidates, and "none of them" must always be an allowed answer. And France came with no labels at all.

IF THEY PUSH On top of that the text is noisy and multilingual: records without an address, abbreviations, names in Devanagari, French conventions such as "15B" or "R" for "Rue". Our India example shows the look-alike problem: three businesses all called "Vijay Enterprises Private Limited", in Agra, Satara and Byculla.

NUMBERS

- 6,724,569,566,212 same-country pairs (Doc 3)
- France has no labels (Doc 2.1); India example (Doc D.2)

Q4. What did your data analysis show about how true records differ from their business? ★★

LIKELY

Why they ask: They want to know whether the design came from evidence or from habit.

SAY True records of a business tend to lose information: they drop a digit or the legal suffix, abbreviate, or transliterate the name into another script, and some use a brand, website or "doing business as" form. So instead of describing a candidate only by how similar it is, we describe exactly what changed (numbers, words added, dropped or replaced, the legal form) and let the decision model learn which changes are harmless.

IF THEY PUSH That is why retrieval is deliberately tolerant (letter pieces, sound keys, legal words left out of the name lookups) while the decision is strict. In our US example the four candidate names differ only in the legal form or one word, and the decision model gives the "Inc" candidate 0.965022 and the "LLC" candidate 0.000007.

NUMBERS

- US example scores 0.965022 (Inc), 0.000007 (LLC), 0.088664 (PC), 0.000088 (Industries Inc) (Doc D.1)

Q5. What are Source 2 and Source 3, and do you treat them differently? ★

Why they ask: They check whether you looked at the sources separately or ignored the distinction.

SAY They are the two sources of incoming records, and we run both through the same per-country pipeline. The models know the source through a source flag feature, and the CatBoost ranker's training sample was balanced at 35,000 records per country and source.

IF THEY PUSH One code path means one set of thresholds to tune and validate; if the sources behave differently, the trees can split on the flag. We did not build source-specific models.

NUMBERS

- Ranker pilot of 140,000 incoming records = 35,000 per country and source (Doc 4)

Q6. What does "France has no labels" change in practice? ★

Why they ask: They want to see you understand the consequences of a domain with zero supervision.

SAY France appears only in the test files, so we can neither train on French labels nor measure a French score. It changed four things: French cleaning rules, two French readers self-trained on confident French pairs, a stricter threshold of 0.82, and France rules computed by code; the France decision model itself learns from US and Indian rows.

IF THEY PUSH A diagnostic upload shows what is at stake: emptying every French row dropped the public score from 0.970125 to 0.839. The France rules were designed by analysing the unlabelled French test records, with some parameters chosen after seeing public scores, and we disclose that. We have no measured French F0.5, and we say so.

NUMBERS

- France: 259,452 businesses, 1,434,993 incoming records, no labels (Doc 2.1)
- Diagnostic upload 0.970125 → 0.839 (Doc B.6, uploads 4-5)
- France rules designed on unlabelled test records, parameters chosen after seeing test and public results (Doc C items 2, 4)

Q7. How rare is a true pair in the full comparison space? ★★

Why they ask: Extreme imbalance is the classic trap in entity resolution; they test whether you see where it is handled.

SAY Extremely rare: each incoming record has at most one owner among hundreds of thousands of businesses in its country, so at most about 1.5 pairs in a million can be true. Blocking is what removes that imbalance: our candidate file has about 1.23 candidates per incoming record.

IF THEY PUSH So the readers and decision models work on short, competitive lists, on average 1.17 shortlisted pairs per record, and are trained as ordinary binary classifiers on those short lists rather than on a one-in-a-million problem. Our candidate file keeps 1.83 pairs per million, the same order as the at most 1.48 per million that can be true at all.

NUMBERS

- At most 9,969,589 true pairs / 6,724,569,566,212 = 1.48 per million (derived)
- 1.2331 candidates per incoming record (Doc B.1); reduction ratio 0.99999817, so 1.83 per million kept (Doc 3, derived)
- 11,687,317 shortlist pairs / 9,969,589 = 1.17 per record (derived)

Q8. The test set has 5.75 incoming records per business, but you link only 3.38. Where did the rest go? ★★

Why they ask: They check that you understand the "or none" part of the task and your own output.

SAY They are records we left unlinked: either they belong to no business, or we were not confident enough to name a clear winner. We make 0.5875 links per incoming record, so about 41 % stay unlinked; with private labels we cannot split that into true non-matches and our abstentions.

IF THEY PUSH The metric scores businesses, not records, so an unlinked record costs us only if it had a true owner, as a missed record; a wrong link would also cost the business that receives it. Of the 1,732,544 business rows, 1,632,457 have at least one link and the busiest has 11.

NUMBERS

- 5,856,936 links; 0.5875 links per incoming record; about 41 % unlinked (Doc B.1, derived)
- Links per business: mean 3.3805, median 3, max 11 (Doc B.1)

Q9. Which country was the hardest? ★★

Why they ask: They test whether you looked at per-country behaviour rather than one global number.

SAY Among the labelled countries, India: the index finds the true business for 98.51 % of Indian held-out records against 99.37 % in the US, and the held-out F0.5 is lower too. France is the hardest to judge, because without labels we cannot measure it at all.

IF THEY PUSH On fold 9 the same India recipe scores 0.990046 on Indian records and 0.991212 on US records. The Indian miss we target is a native-script name that the Latin index does not retrieve, or retrieves only as same-name businesses in other places, as in our Vijay Enterprises example; that is why the native-script second-chance search exists, and it added 9,759 links.

IF THEY PUSH FURTHER One visible difference is India's lower recall ceiling: the index finds the true business for 99.37 % of held-out US records but 98.51 % of Indian ones, and the shortlist keeps at least 98.87 % and 98.06 %. Native-script names without a known transliteration are one of our missed-match types, and the native-script search targets them.

NUMBERS

- Index recall, fold 9: US 99.3694 %, India 98.5068 % (Doc B.2)
- Held-out F0.5, folds 9 / 8: US 0.991212 / 0.991545; India 0.990046 / 0.990298 (Doc 5)
- Native-script second-chance links +9,759 (Doc B.1); miss type (Doc 3, Doc D.2)
- Held-out (Doc 5); recall (Doc B.2)

Q10. Can you show what a raw incoming record looks like next to its candidate business? ★

Why they ask: A quick way to check that you have looked at the data, not just the metrics.

SAY Our French example: the incoming record is "TQ COMITE SARL", "2 R SUZANNE LENGLEN, NANTES", and its top candidate is "TQ Parents SARL", "2 Rue Suzanne Lenglen, Nantes, Pays de la Loire". You see capitals, "R" for "Rue", a missing region and one name word that differs; our US example has no address at all.

IF THEY PUSH After our cleaning both addresses become "2 rue suzanne lenglen nantes", so retrieval finds the pair easily; the hard part is the decision, and here a France rule vetoed the link because "comite" replaced "parents" at the same address. That is the general pattern: cleaning and retrieval make true pairs easy to find, and the decision has to tell near-identical candidates apart.

NUMBERS

- Decision-model score 0.999837, vetoed by the category-swap veto (Doc D.3)
- Cleaned address: output of our French cleaning code on these two strings (Code)

Q11. What do singletons mean for you, and how do you handle them? ★★ LIKELY

Why they ask: Singletons are where precision-first scoring bites hardest; Amazon lists them as an edge case.

SAY There are two kinds: a business with no true records, and an incoming record that belongs to no business. For the first the only perfect answer is an empty row, and 100,087 businesses, 5.8 %, end with no link; for the second we link nothing unless the best candidate is both confident and a clear winner.

IF THEY PUSH 125 of those empty businesses never received a single candidate; the rest had candidates, but none of those incoming records was linked to them. Without test labels we cannot say how many of the 100,087 are truly empty, only that a correct empty row scores a full point.

IF THEY PUSH FURTHER Links per incoming record: 0.5875 overall (US 0.5895, India 0.5806, France 0.6046), so about 41 % are unlinked (derived). We don't know how many test records truly have no owner, because the labels are private. A record that belongs to no business can never raise the score; it can only lower it, by becoming a wrong link on some business.

NUMBERS

- 100,087 empty rows (Doc B.1); 5.8 % (derived)
- 125 businesses without candidates; 2 incoming records without a candidate (Doc B.1)
- Links per incoming record 0.5875 (Doc B.1); about 41 % unlinked (derived)

Q12. What exactly do you submit, and how is it checked? ★

Why they ask: They check that you know the deliverable and the guard rails around it.

SAY Two files, each with one row per test business: the matching file lists the incoming records we link to that business, and the candidate file lists its shortlisted pairs plus the candidates added by the second-chance searches. Our export asserts that every link is also a candidate, and the last stage runs the organisers' validator with ID checks, which returns PASS.

IF THEY PUSH No pair joins two countries and no incoming record is linked to more than one business. From the stored models, our inference stages rewrite both files byte for byte.

NUMBERS

- 1,732,544 rows in each file (Evidence: validator log)
- 5,856,936 links; 12,293,019 candidate pairs (Doc B.1, Doc 3)

Q13. Only the links are scored, so why does the candidate file matter? ★★ LIKELY

Why they ask: Candidate pairs per entity is an explicit judging angle; they test whether you optimised it on purpose.

SAY Because candidate efficiency, the number of pairs per business, shows how much work the expensive stages really do and how well the system would scale, and the organisers list candidate pairs per entity as a judging topic. Ours is about 7.10 pairs per business, out of a space of 6.7 trillion pairs.

IF THEY PUSH The file is not padded or trimmed after the decision: it is exactly the shortlist our readers read plus the candidates of the two second-chance searches, 1.2331 per incoming record, and 90.59 % of records have exactly one candidate. To be precise, the CatBoost ranker also scores the 21.71 pairs per business that pass the quick filter, and the address search's filter scores its retrieved pairs; those are not written, and our documentation says so. The recall price is measured: the shortlist still holds the true business for at least 98.87 % of US and 98.06 % of Indian held-out records.

NUMBERS

- 12,293,019 pairs, 7.0954 per business, reduction ratio 0.99999817 (Doc 3)
- Ranker input 37,614,782 pairs (21.7107 per business) and address-search retrieved pairs are model-scored but not written (Doc 3)
- 1.2331 candidates per record; 9,031,916 records (90.59 %) with exactly one (Doc B.1)
- Shortlist recall lower bounds: US 98.8748 %, India 98.0572 % (Doc B.2)
- "Candidate pairs per entity" is a listed judging topic (DECK_PLAN)

Q14. Why do you never match across countries? ★

Why they ask: They probe whether a hard constraint could silently lose matches.

SAY We treat the country as a hard block: all retrieval runs inside one country, and no pair in our files joins two countries. That removes impossible pairs for free, and it lets each country use its own cleaning rules, readers and threshold through a country profile.

IF THEY PUSH The comparison space is then the sum over countries of businesses times incoming records, 6.7 trillion pairs, not the full cross product. If the country field could be wrong in production, we would add a cross-country search for records left unlinked; we did not need or test that here.

NUMBERS

- 6,724,569,566,212 same-country pairs (Doc 3); no cross-country pair in the output (Doc B.1)

Q15. What information does each record actually carry? ★

Why they ask: They want to see that you know how thin the signal is.

SAY Very little: an id, a business name, an address and a country, and the source is known from the file. So everything is built from two short text fields, which is why we care so much about exactly how names and addresses differ.

IF THEY PUSH Addresses can be missing, as in our US example, and names can be in Devanagari or carry French legal forms. The readers see each side as raw "name | address" text, with the pair truncated to 128 tokens.

NUMBERS

- Record fields: id, name, address, country (Code: data loading)
- Reader input truncated to 128 tokens (Doc 4); US record without address (Doc D.1)

2. Metric and the precision-first choice (13 questions)

Q16. How is the competition metric computed? ★ LIKELY

Why they ask: Everything else follows from the metric; they check you can state it exactly.

SAY For each business we compare the incoming records we linked to it with its true records and compute F0.5, then we average over all businesses, including those with no true records. A business with no true records scores 1 only if its row is empty.

IF THEY PUSH $F0.5 = 1.25 \cdot P \cdot R / (0.25 \cdot P + R)$, which per business simplifies to $1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$, and is 1 when both counts are zero. Illustration with made-up counts: 4 true records, we link 3 of them and nothing wrong, so $1.25 \cdot 3 / (3 + 0.25 \cdot 4) = 0.9375$.

NUMBERS

- Formula (Doc 2.1; Code: metric function); public score 0.991300 (Doc 5)

Q17. Why does F0.5 favour precision? ★

Why they ask: Anyone can recite "F0.5"; an expert knows what beta does.

SAY In F-beta, beta sets how much recall counts relative to precision; with $\beta = 0.5$ precision weighs more, so a wrong link hurts more than a missed one. That is why every decision in our pipeline is allowed to abstain.

IF THEY PUSH Written as a weighted harmonic mean, $1/F0.5 = 0.8/P + 0.2/R$, so precision carries four times the weight of recall. Illustration, a business with 4 true records: missing one gives 0.9375, while finding all four plus one wrong link gives $1.25 \cdot 4 / (5 + 1) \approx 0.833$.

Q18. Why does the empty-row rule matter so much? ★★ LIKELY

Why they ask: This rule is the sharpest edge of the metric; they test whether you designed around it.

SAY Because a business with no true records gets a full point only if we link nothing, so one wrong link turns its 1 into 0, the largest loss possible for one business. A correct empty row is worth as much as a perfect match list, and 100,087 of our rows are empty.

IF THEY PUSH With macro averaging these businesses weigh exactly as much as busy ones, so a liberal threshold would be punished hardest here. It is one reason for the clear-winner rule, the stricter French threshold and France rules that remove more links than they add.

IF THEY PUSH FURTHER Empty rows: US 38,161, India 46,681, France 15,245. Each empty row scores either one (if the business truly has no records) or zero (if it had some), and we cannot tell which without labels.

NUMBERS

- 100,087 empty rows (Doc B.1)
- France rules: 31,707 links vetoed, 11,636 added (derived from Doc B.3)
- 100,087 empty rows (Doc B.1); 5.8 % (derived)

Q19. Why is a wrong link worse than a missed link, beyond the formula? ★★★

Why they ask: They want the mechanism, not the slogan.

SAY A wrong link usually costs twice: it is a false positive for the business that receives it and, if the record has a true owner, a missed record for that owner. Abstaining costs only the missed record. So when two candidates are close, linking nothing beats guessing.

IF THEY PUSH Our threshold-plus-lead rule encodes this asymmetry, and the second-chance thresholds were chosen on fold 8 with false positives counted twice, which builds the same asymmetry into their tuning. In our French example the decision model said 0.999837, yet a France rule vetoed the link because one word was swapped at the same address.

NUMBERS

- Second-chance thresholds: fold 8, false positives counted twice (Doc 4)
- French example 0.999837, vetoed (Doc D.3)

Q20. What does macro-averaging over businesses imply for your design? ★★

Why they ask: They test whether you see that the metric is per entity, not per pair.

SAY Every business counts equally, whether it has one true record or eleven, so we cannot buy score by getting the busy businesses right. It also means a pair-level measure such as AUC or log loss is not the thing to optimise at the end: our models train with log loss, but every operating point is chosen on the exact per-business metric.

IF THEY PUSH For a business with a single true record, one decision swings its score between 0 and 1. And because the records of one business interact in its score, we added a per-business consistency check, and our folds are split by business so that all records of a business sit on the same side.

NUMBERS

- Links per business: median 3, max 11 (Doc B.1)
- Training losses: ranker Logloss, readers binary cross-entropy, decision models binary LightGBM; operating points on exact per-business macro F0.5 (Doc 4); folds per business (Doc 4)

Q21. How did you choose the decision threshold, and why not tune for AUC? ★★ LIKELY

Why they ask: They test whether operating points were set on the right objective and on held-out data.

SAY We tuned it directly on the competition metric, exact per-business macro F0.5, on held-out fold 8, which gave 0.75 for the US and India. AUC measures how well all pairs are ranked and has no threshold in it at all, while our decision is at most one link per record, scored per business with empty rows worth 1, so the best threshold can only be read off that exact metric.

IF THEY PUSH The per-business consistency check was tuned on folds 8 and 9 and the second-chance thresholds on fold 8 with false positives counted twice. A real example of AUC and F0.5 disagreeing: on our generated French test set the French E5 reader had the higher AUC, 0.98038 against 0.979415, but the lower best F0.5, 0.910516 against 0.914964.

IF THEY PUSH FURTHER The exact metric matters because averaging per business gives small and empty businesses the same weight as large ones, so the best pair-level threshold is not the best business-level one. The honest caveat: fold 8 also appears in our held-out table, so that table is not an untouched test of this choice, as our documentation states.

IF THEY PUSH FURTHER Per business, $F0.5 = 1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$. A business with 4 true records scores 1.0 if all are found, 0.9375 if one is missed, and 0.833 if all are found plus one wrong link, so here a wrong link costs about 2.7 times a miss; at a business with no true records a wrong link costs the whole point. The E5 readers' AUCs are about 0.9985, which says nothing about this trade-off.

NUMBERS

- Threshold 0.75 (fold 8); consistency check 0.75 / 0.68 / 0.72 / 0.77 (folds 8 and 9) (Doc 4)
- Generated French test set: AUC 0.98038 vs 0.979415; best F0.5 0.910516 vs 0.914964 (Doc B.4)
- $F0.5 = 1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$, 1 if both are 0 (Doc 2.1; formula as in Explainer)
- Worked values 1.0 / 0.9375 / 0.833 (derived)

Q22. Why require a lead of 0.4 over the runner-up, not just a threshold? ★★ LIKELY

Why they ask: The lead rule is unusual; they want the reasoning behind it.

SAY Because the dangerous case is two candidates that both look right: a threshold alone would link one of them, and a near-tie is where a wrong link is most likely. The lead rule says: link only a clear winner, otherwise link nothing.

IF THEY PUSH In our US example the best scores 0.965022 and the runner-up 0.088664, a lead of 0.876, so it links. For the 90.59 % of incoming records with a single candidate the lead condition passes automatically and only the threshold decides; 0.4 is one fixed lead used in every country, and the US and India threshold next to it was tuned on fold 8 for the exact metric.

NUMBERS

- Lead 0.4 in every country (Doc 4); US example lead 0.876 (Doc D.1)
- 90.59 % of incoming records with exactly one candidate (Doc B.1)

Q23. Where does "precision first" show up in your pipeline? ★ LIKELY

Why they ask: They test whether the metric really shaped the system end to end.

SAY At every decision point: at most one link per record, the best candidate only if it is confident and a clear winner, a stricter 0.82 bar in France, second-chance searches that only touch unlinked records, and France rules that remove more links than they add. Retrieval, by contrast, is recall-first, because a business that is never retrieved can never be linked.

IF THEY PUSH In France the decision model accepted 887,630 links and after the rules 867,559 remain. The second-chance searches never undo a link, while the consistency check and the France vetoes can remove one.

NUMBERS

- France 887,630 accepted → 867,559 links (Doc B.3, B.1)
- Thresholds 0.75 / 0.82, lead 0.4 (Doc 4)

Q24. Why is France's threshold 0.82 instead of 0.75? ★★

Why they ask: They test how you set an operating point without labels.

SAY Because France has no labels, we could not tune its threshold on the metric, so we chose a more conservative point. It was set without labels, from the number of French links per business together with a threshold sweep on US and Indian fold 8.

IF THEY PUSH As a proxy, the France recipe applied to US and Indian rows at 0.82, without the consistency check, scores 0.990508 and 0.989467 on fold 9; that measures the recipe, not French accuracy. We have no measured French F0.5, and the same 0.82 is used for any unseen country.

NUMBERS

- 0.82 for France and the default route (Doc 4, Doc C item 1)
- Proxy, fold 9: US 0.990508, India 0.989467; fold 8: 0.990977, 0.989850 (Doc 5)

Q25. If wrong links are so costly, why not link only when you are 99 % sure? ★★

Why they ask: They test that you see both sides of the trade-off, not only the precision side.

SAY Because missing hurts too: a business with true records and an empty row scores 0, exactly like a business with no true records and one wrong link. The best bar is where per-business F0.5 peaks on held-out data, and for the US and India that was 0.75, not 0.99.

IF THEY PUSH Illustration: a business with 4 true records scores 0.9375 if we miss one of them, but 0 if we refuse all four. The lead rule lets us keep a moderate threshold while still refusing the dangerous near-ties, and the second-chance searches win back some of the records the main decision leaves unlinked.

NUMBERS

- 0.75 chosen on fold 8 for exact per-business macro F0.5 (Doc 4)

Q26. Could you have optimised expected F0.5 directly instead of using a fixed threshold? ★★★

Why they ask: A senior scientist probes whether you understand the metric's structure, not just a tuned knob.

SAY In principle yes: the metric is per business, so the best decision for one record depends on what else is linked to the same business, and a fixed threshold ignores that. Our per-business consistency check is a cheap approximation: it re-decides each record with a bar that depends on how many other records were accepted for that business.

IF THEY PUSH Toy maths: adding a link that is right with probability q to a row with $TP = t$ and $D =$ predicted + 0.25·true raises expected F0.5 only if $q \geq t/D$, so a business with 4 true records and 3 correct links already needs $q \geq 0.75$, and the break-even moves with the row. The check uses 0.75, 0.68, 0.72 and 0.77 for 0, 1, 2 and 3 or more other accepted records, tuned on folds 8 and 9, and it moved few links (US +277 / -284, India +247 / -248).

NUMBERS

- Consistency check bars and effect (Doc 4, Doc B.1); break-even $q \geq t/D$ (derived from the formula)

Q27. What is your precision-recall trade-off? Did you look at precision and recall separately? ★★

★★ **LIKELY**

Why they ask: Amazon lists the precision-recall trade-off under F0.5 as a topic; they want the knobs and the evidence.

SAY The trade-off is set by a few knobs: the threshold, the consistency-check bars and the second-chance bars, chosen on held-out folds for the exact metric, plus one fixed 0.4 lead for every country and a more conservative French threshold. Our evaluation code counts false positives and false negatives, but our documentation reports only the combined F0.5, so I do not have separate precision and recall figures to quote.

IF THEY PUSH Recall has a hard ceiling upstream: the shortlist holds the true business for at least 98.87 % of US and 98.06 % of Indian owned records on fold 9, so the decision cannot recover the rest without the second-chance searches. The next thing we would report is a precision-recall curve per country on held-out folds.

NUMBERS

- Shortlist recall, fold 9: US 98.8748 % to 99.3628 %; India 98.0572 % to 98.4831 % (Doc B.2)
- Knobs and how each was chosen (Doc 4); false-positive and false-negative counts in the evaluation code (Code: metric function)

Q28. If the metric were F1 instead of F0.5, what would you change? ★★★

Why they ask: A counterfactual shows whether you understand the design or only tuned it.

SAY We would lower the bar and loosen the lead, because F1 weighs precision and recall equally, so a link pays off at a lower probability of being right. The second-chance searches would become more valuable and the France vetoes less so, and we would re-tune every threshold on held-out folds for F1.

IF THEY PUSH In toy maths, a business with 4 true records and 3 correct links needs an extra link to be right with probability at least 0.75 under F0.5, but only about 0.43 under F1. The pipeline itself would not change, only the operating points and the double weighting of false positives in the second-chance tuning; we have not run that, so I cannot give a measured number.

NUMBERS

- Break-even $q \geq t / (\text{predicted} + \beta^2 \cdot \text{true})$: $3 / 4 = 0.75$ for F0.5, $3 / 7 \approx 0.43$ for F1 (derived from the formula, illustration)

See also, answered elsewhere: "What does 0.9913 mean in practice?" see [Q180](#).

3. Normalisation and multilingual text (15 questions)

Q29. What does your text cleaning do? ★ **LIKELY**

Why they ask: They check that you know the unglamorous step that sets retrieval recall.

SAY For the US and India: lower case, "&" becomes "and", other punctuation becomes spaces, state names and street words are mapped to one short form, leading zeros are dropped, and Indian-script name words are transliterated to Latin. France adds French rules: "15B" becomes "15 bis", street types and their typos are unified, "St" becomes "Saint", and "S.A.R.L." or "Sarl" becomes "sarl".

IF THEY PUSH With our cleaning code, a made-up "Suite 0012, 007 Main Street, California" becomes "ste 12 7 main st ca". From our French example, "2 R SUZANNE LENGLEN, NANTES" and "2 Rue Suzanne Lenglen, Nantes, Pays de la Loire" both become "2 rue suzanne lenglen nantes".

NUMBERS

- Cleaning rules (Doc 2.2); outputs above produced by our shipped cleaning code (Code; first input is an illustration)

Q30. Why clean the text at all when your readers read the raw text? ★★

Why they ask: It sounds contradictory; they test whether you know which component needs what.

SAY Because the index lookup needs exact token matches: "Street" and "St", or "Maharashtra" and "MH", must become the same token to find each other. The readers read the raw text instead, because cleaning throws away signals they can use, such as punctuation, capitals and the original script.

IF THEY PUSH So cleaning serves retrieval and the 76 index features, while the "what changed" features use the name lower-cased, accent-stripped and transliterated, with legal-form words removed and described by separate legal-form features. Each component gets the version of the text that suits its job.

NUMBERS

- Readers read raw text (Doc 2.2); 76 index features (Doc 3)

Q31. How do you handle names written in Indian scripts? ★ LIKELY

Why they ask: Multilingual input is a stated challenge; they want the mechanism.

SAY In two layers. During cleaning, each native-script name word is mapped to Latin with a word list learned from labelled training matches, with a generic transliteration as fallback; and for records still unlinked, the native-script second-chance search tries Latin spellings of the name within the same state, which added 9,759 links in India.

IF THEY PUSH In our India example the second-chance search's list maps विजय to vijay, एंटरप्राइजेज to enterprises, प्राइवेट to private and लिमिटेड to limited; without the legal words the key is (vijay, enterprises), searched within Maharashtra. The readers also read the Devanagari directly, because their multilingual tokenizer covers it.

IF THEY PUSH FURTHER The cleaning word list is a co-occurrence alignment learned on folds 0-5: a native token gets a Latin word if they co-occur at least 3 times, with purity at least 0.35 and a score at least 1.8 times the runner-up; otherwise unidecode is used. It is not a neural transliterator. The second-chance list keeps every alternative with at least 5 % of a token's count and tries up to 64 transliterations. Separately, a label-free word alignment on test records whose top candidate has probability at least 0.9 feeds some name features; our documentation discloses it.

NUMBERS

- +9,759 links; 649,965 pairs scored for 171,713 records (Doc 3, B.1)
- Transliteration example (Doc D.2)
- Native-script search: 649,965 pairs scored for 171,713 records, +9,759 links (Doc 3, B.1)

Q32. How was the transliteration word list learned? ★★★

Why they ask: They test whether "learned lexicon" hides something fragile or leaky.

SAY From labelled training pairs of folds 0 to 5: for every native-script word in an incoming name, we count which Latin words appear in its true business's cleaned name, and we keep the best Latin word only if the evidence is strong and unambiguous. It is a co-occurrence alignment learned from labels, not a neural transliterator.

IF THEY PUSH A native word gets a Latin word only if they co-occur at least 3 times, that pairing covers at least 35 % of the native word's occurrences, and its score, $\text{count} / (\text{frequency of the Latin word} + 1)^{0.6}$, is at least 1.8 times the runner-up's; the exponent stops very common Latin words from absorbing every native word. The second-chance search uses its own list from training matches, and when its model is trained that list leaves out the record's own fold, so a record never benefits from its own label.

NUMBERS

- At least 3 co-occurrences, purity ≥ 0.35 , score ratio ≥ 1.8 , exponent 0.6 (Code: word-list learner)
- Second-chance list: leave-own-fold-out for training (Doc 4)

Q33. Why not use a neural transliteration model or an LLM for this? ★★

Why they ask: They test whether you considered the obvious modern alternative.

SAY A word list learned from our own labelled matches is in-domain, deterministic, easy to audit and essentially free at inference, and it only has to serve exact-token retrieval. No LLM runs anywhere in our pipeline.

IF THEY PUSH The multilingual readers already read the native script from the raw text, and the remaining retrieval misses are what the native-script second-chance search targets. We did not benchmark a neural transliterator, so we cannot claim it would be worse; it would be a candidate for words the list does not cover.

NUMBERS

- No LLM in the pipeline (Doc C item 6)

Q34. What exactly changes for French text? ★★

Why they ask: They want evidence that you adapted to a new address format rather than reusing US rules.

SAY French addresses and legal names follow their own conventions, so France gets extra cleaning rules on top of the US and India ones. For example, "N°" prefixes are removed, "15B" becomes "15 bis", "R" and "Rue" both become "rue", street-type typos are fixed with a table learned from the files, "St" becomes "Saint", and dotted or leet legal forms such as "S.A.R.L." or "5arl" collapse to "sarl".

IF THEY PUSH With our cleaning code, the made-up inputs "N° 15B BD Saint Michel, Paris" and "15 bis boulevard St Michel, Paris" both become "15 bis bd saint michel paris", and "S.A.R.L. Dupont & Fils" and "5arl Dupont et Fils" both become "sarl dupont and fils". Region and department parts of an address are recognised from the files and left out of the matching text, which is why both addresses of our French example, one with "Pays de la Loire" and one without, clean to the same string.

NUMBERS

- French rules (Doc 2.2); outputs and region handling from our French cleaning code (Code; inputs are illustrations)

Q35. You learned French geography and a typo table from the test files. Isn't that leakage? ★★

Why they ask: Any use of test data raises a flag; they test whether you know the line between labels and text.

SAY It does use the unlabelled test text, never test labels, and our documentation discloses it. It is the same kind of statistic as the word-rarity weights of our index, which have to be computed over the businesses we match against anyway.

IF THEY PUSH The geography comes from counting which comma-separated address parts occur together in the test files (regions, departments, cities), with no labels; the typo table maps a word at the street-type position of incoming records that (almost) never occurs in business records to the nearest known street type within one or two edits, skipping ambiguous cases. In the same spirit, one word list used inside some Indian decision-model features is aligned, without labels, on test records whose top candidate the ranker scores at 0.9 or more, and our documentation lists that and every other statistic we compute on the test files.

NUMBERS

- Disclosure of all statistics computed on unlabelled test files, including the India word list aligned on test records with top ranker probability ≥ 0.9 (Doc C item 1); geography and typo-table learning (Code: French cleaning)

Q36. How do you handle abbreviations and synonyms such as Street/St or state names? ★

Why they ask: Basic, but they check that you used controlled, auditable mappings.

SAY With small hand-written tables and no outside data: street words are mapped to one short form (road to "rd", street to "st", avenue to "ave"), state names to their codes (California to "ca", Maharashtra to "mh"), and in names "&" becomes "and". France has its own table of street types and name abbreviations, for example "Etablissements" and "Ets" become one form.

IF THEY PUSH The mapping only has to be applied the same way on both sides, not be linguistically perfect, because it serves exact-token retrieval. Our documentation states that hand-written lists are limited to small word lists, and they ship with the code.

NUMBERS

- Hand-written lists limited to small word lists (Doc C item 6); tables (Code: cleaning tables)

Q37. What is the "core name", and why remove legal forms for retrieval? ★★

Why they ask: They test whether you see how retrieval and decision split the work on legal-form differences.

SAY The core name is the business name without legal-form words such as inc, llc, ltd, pvt, private, limited or sarl, and without "and" and "the". Our name lookups, letter pieces, sound keys and the exact-name key all use it, so dropping or changing a legal form from that list does not block retrieval.

IF THEY PUSH "Legacy Marketing Enterprises Inc" and "Legacy Marketing Enterprises LLC" have the same core name, so they are retrieved together, which is why our US example's shortlist holds both. Telling them apart is then the job of the readers and the legal-form features, and the decision model gave 0.965022 to the Inc and 0.000007 to the LLC.

NUMBERS

- Core name (Doc 3; Code: index lookup, which also applies it to the name-word lookup; its list is inc, incorporated, llc, llp, ltd, limited, pvt, private, corp, corporation, co, company, plc, sarl, sas, sa, sci, sasus, eurl, and, the, so for example "pc" stays in the core name)
- US example scores (Doc D.1)

Q38. How do you handle typos and spelling variants? ★ LIKELY

Why they ask: Noisy names and addresses are an explicit judging topic.

SAY We handle them in three places. Cleaning fixes known variants such as French street-type typos and leet spellings, the index has letter-piece and sound channels that still match after a typo, and the readers read raw text and tolerate small edits.

IF THEY PUSH Illustrations: "Kreeshna" and "Krishna" have the same consonant skeleton "krshn" in the sound channel, and "Starbucks" and "Star Bucks" share all letter pieces because spaces are removed before the pieces are cut. In names, a zero next to a letter becomes "o", so "C0ffee" becomes "coffee".

NUMBERS

- Channels (Doc 3); zero-to-o repair (Code: US/India cleaning); examples are illustrations

Q39. How do you treat numbers: house numbers, leading zeros, "24/11"? ★★

Why they ask: Numbers are the strongest and the most fragile evidence in an address.

SAY We normalise them carefully because they carry so much evidence: leading zeros are dropped, so "007" becomes "7", punctuation splits a number like 24/11 into 24 and 11, and in France glued suffixes are split and "N°" removed. Then the "what changed" features describe how the numbers differ: absolute and relative difference, digit edit distance, shared prefix or suffix.

IF THEY PUSH The address second-chance search looks up businesses that share one of the record's three rarest digit runs and one of its three rarest address words. In our India example the house number 24/11, with both digit runs matching, is what separated the right "Vijay Enterprises" from four others in Mumbai.

NUMBERS

- Number features (Doc 4); address search keys (Doc 3); India example (Doc D.2)

Q40. Do you handle accents and mixed-script names? ★

Why they ask: A quick check that real-world text quirks were covered.

SAY Yes, word by word: an accented or non-Latin word is transliterated, so "Société" becomes "societe", and in a name that mixes Latin and Devanagari the Latin words stay as they are while the native words go through the learned word list. Addresses are always transliterated generically.

IF THEY PUSH The "what changed" features also strip accents and use the same learned list, and the readers read the original characters.

NUMBERS

- Word-by-word handling (Code: US/India cleaning)

Q41. Which tokenizer do the readers use, and does it cover Devanagari and French? ★★★

Why they ask: An expert checks whether the multilingual claim holds at the tokenizer level.

SAY All four readers use the MiniLM tokenizer, a multilingual SentencePiece vocabulary that covers Latin, accented and Indian scripts, so raw Devanagari and French text is read directly. Each record-business pair is truncated longest-first to 128 tokens.

IF THEY PUSH The E5 readers can share that tokenizer because both base models use the XLM-RoBERTa vocabulary (general background, not stated in our documentation). We do not use E5's "query:" and "passage:" prefixes, because we fine-tune it as a pair reader that reads both texts together, not as an embedding model for retrieval.

NUMBERS

- MiniLM tokenizer for all readers, 128 tokens, longest-first (Doc 4, B.5)

Q42. One Hindi word can be written several ways in Latin letters. How do you cope? ★★★

Why they ask: One-to-many transliteration is the real difficulty with Indic names; they test whether your word list handles it.

SAY Differently for two jobs. The cleaning word list keeps one best Latin spelling per native word, and only when the evidence clearly favours it; the native-script second-chance search keeps every alternative spelling with at least 5 % of the native word's observed spellings and tries up to 64 combinations per name.

IF THEY PUSH So the main index lookup stays precise and cheap, and the second chance covers spelling variety for records that are still unlinked. Where neither list knows a word, the name falls back to a generic transliteration, and the multilingual readers still read the original script.

NUMBERS

- Alternatives with at least 5 % share, up to 64 transliterations (Doc 3)

Q43. Your state table maps both Tennessee and Tamil Nadu to "tn". Isn't that a bug? ★★★

Why they ask: An adversarial check that you know your own lookup tables and their failure modes.

SAY It would be if we matched across countries, but we never do: retrieval runs inside one country, so in the US "tn" can only mean Tennessee and in India only Tamil Nadu. The same holds for Georgia and Goa ("ga") or Minnesota and Manipur ("mn").

IF THEY PUSH The general rule is that a normalisation collision is harmless when a block above it separates the cases, and the readers see the raw text anyway. Within a country a mapping only needs to be applied the same way on both sides: "Washington Street" becoming "wa st" looks odd but still matches itself.

NUMBERS

- State table (Code: cleaning tables); all retrieval within one country (Doc 3)

4. Blocking and candidate generation (13 questions)

Q44. Why do you need blocking at all? ★ LIKELY

Why they ask: The basic framing question for the whole retrieval stage.

SAY Comparing every incoming record with every business in its country would be 6.7 trillion pairs, which no model can score. Blocking cheaply picks a few likely businesses per record; our final candidate file keeps about 7.10 pairs per business, a reduction ratio of 0.99999817.

IF THEY PUSH That keeps 1.83 pairs in every million (derived), or 1.23 per incoming record, against at most 1.48 in a million that can be true at all, since each record has at most one owner. And the shortlist still keeps the true business for roughly 99 % of US and 98 % of Indian held-out records.

NUMBERS

- 6,724,569,566,212 pairs; 12,293,019 kept; 7.0954 per business; ratio 0.99999817 (Doc 3)
- 1.2331 candidates per incoming record (Doc B.1); 1.83 and 1.48 per million (derived); shortlist recall (Doc B.2)

Q45. Walk me through your candidate funnel. ★ LIKELY

Why they ask: They want the count at each step and to see where cost and recall go.

SAY Four steps, from cheap to expensive: the inverted index finds about 130 candidates per business, a quick rule-based filter with no model drops the clearly weaker ones, the CatBoost ranker keeps at most four per incoming record, and the two second-chance searches add a few for records left unlinked. We end at about 7 per business.

IF THEY PUSH In numbers: 130.1 per business in our 25 September blocking study, 21.7107 after the filter (37,614,782 pairs), 6.75 in the shortlist (11,687,317 pairs, derived) and 7.0954 in the final file (12,293,019 pairs). Per incoming record that is 1.2331 candidates, and 90.59 % of records have exactly one.

NUMBERS

- Funnel (Doc 3); per record and single-candidate share (Doc B.1)
- Per country final: US 6.1836, India 7.5180, France 8.1060 per business (Doc 3)

Q46. How does your inverted index work? ★★

Why they ask: They test whether "inverted index" is understood or just a buzzword.

SAY Like the index at the back of a book: for every word, letter piece or sound key it stores the list of businesses that contain it. For an incoming record we walk the lists of its rarest terms first, add each term's rarity weight to every business on the list, and keep the best-scoring businesses. It is written in C++ with OpenMP and built per country on the cleaned text.

IF THEY PUSH The rarity weight is $IDF = \log(1 + (N + 1)/(df + 1))$; with $N = 663,106$ US businesses, a term found in 10 businesses weighs about 11.0 and one found in 10,000 about 4.2 (derived), so one rare shared word outweighs two common ones. There is no term-frequency part, because a short name is a set of words: the score is an IDF-weighted overlap, not TF-IDF cosine or BM25.

NUMBERS

- IDF formula (Doc 3); 11.0 and 4.2 (derived, illustration)

Q47. What are the five channels, and why so many? ★★

Why they ask: They test whether each channel has a reason to exist.

SAY Name words, address words, 4-letter and 3-letter pieces of the core name, and a sound key of the name. Each catches a different miss: words for clean matches, letter pieces for typos, abbreviations and split or joined words, sounds for spelling variants, and the address channel for records whose name changed but whose address did not.

IF THEY PUSH The channel weights are 1, 1, 0.45, 0.22 and 0.18, so letter pieces and sounds mostly matter when the words fail. On top, an exact-name key pulls in every business with the same sorted core-name word set when at most 2,000 businesses share it.

NUMBERS

- Channels, weights, exact key up to 2,000 (Doc 3)

Q48. What exactly is the sound channel? Soundex? ★★

Why they ask: A precise follow-up that separates people who know the code from people who read about it.

SAY Not Soundex or Metaphone: a simple consonant skeleton. We delete the vowels and "y", collapse repeated letters, and index 4-letter pieces of what remains.

IF THEY PUSH Illustrations: "Krishna" and "Kreeshna" both become "krshn" and share the pieces "krsh" and "rshn"; "Smith" and "Smyth" both become "smth". Repeats are collapsed after the vowels are dropped, so consonants that meet across a removed vowel also merge; the key is language-agnostic, which suits transliterated names where vowel spelling varies a lot, and it has the smallest weight, 0.18.

NUMBERS

- Vowels removed, repeated letters collapsed, 4-grams, weight 0.18 (Doc 3; "y" also removed: Code: index lookup)

Q49. How is the retrieval score computed, and why the min term? ★★★

Why they ask: They test whether the scoring function was designed or copied.

SAY Each candidate gets a name score and an address score, each a sum of rarity weights of shared terms divided by a power of the business's term count, and the total is name plus address plus 1.2 times the smaller of the two. The min term rewards candidates that match on both name and address over ones that match only one of them strongly.

IF THEY PUSH Toy example: name 10 and address 0 gives a total of 10, while name 5 and address 4 gives $5 + 4 + 1.2 \times 4 = 13.8$, so the balanced candidate ranks higher although its plain sum, 9, is lower. The length exponents, 0.22 for name words, 0.38 for letter pieces and 0.30 for address words, stop long names or addresses from winning just because they have more terms to match.

NUMBERS

- ns, as and $total = ns + as + 1.2 \cdot \min(ns, as)$ with the exponents above (Doc 3); toy example is an illustration

Q50. What are posting-list caps and budgets, and why use them? ★★★

Why they ask: Bounded work per query is the core of any scaling argument for an inverted index.

SAY A very common term has a huge list of businesses: walking it costs a lot and tells us little. So each channel skips any term whose list is longer than a cap and stops adding terms once a budget of list entries is spent, always starting from the rarest. That bounds the work per record, whatever the record looks like.

IF THEY PUSH Caps and budgets: name 14,000 / 22,000, address 10,000 / 18,000, 4-letter pieces 2,500 / 15,000, 3-letter pieces 1,000 / 6,000, sounds 800 / 4,000, so at most 65,000 list entries per record (derived). In effect it is a stop-word list learned automatically from document frequency.

NUMBERS

- Caps and budgets (Doc 3); 65,000 (derived: sum of the budgets)

Q51. Which candidates does the index keep per record, and why several lists? ★★

Why they ask: They test whether retrieval is genuinely recall-first.

SAY The top 12 by total score, the top 6 by name score, the top 6 by address score, and every business with exactly the same core-name word set if at most 2,000 share it. Taking the union of several ranked lists makes retrieval recall-first: a business that matches only on the address, or only on the name, still gets in.

IF THEY PUSH On held-out fold 9 this keeps the true business for 99.3694 % of US and 98.5068 % of Indian owned records, with about 27.5 and 23.1 candidates per record in our samples. Each pair also gets 76 features (string similarities, overlaps, rarity overlaps and retrieval scores) for the later steps.

NUMBERS

- Kept lists (Doc 3); recall and candidates per record (Doc B.2)

Q52. What does the quick rule-based filter do, and why no model? ★★ **LIKELY**

Why they ask: Pruning before any model is unusual; they want the rule and its measured cost.

SAY It keeps a candidate if any one of six simple conditions holds, such as having the record's best score, scoring at least 60 % of the best, or covering at least 95 % of the record's address. A model would first have to score all of the roughly 130 candidates per business; the rule is nearly free, and its measured cost is tiny.

IF THEY PUSH The six: best total; total at least 0.6 times the best; best address score; address coverage of at least 0.95; a name of 3 characters or fewer with coverage at least 0.4; the same core name with coverage at least 0.3, where coverage is the rarity-weighted share of the record's address words found in the business's address. The last four keep address-only, acronym and same-name candidates that one combined score would drop, and in the blocking study the true business survived for 99.974 % of fold-8/9 records while fold-9 F0.5 moved only from 0.990117 to 0.990086.

NUMBERS

- Six conditions; 130.1 → 21.7107 per business (Doc 3)
- Owner kept 99.974 %; fold 9 0.990117 → 0.990086; fold 8 -0.00004 (Doc 3)

Q53. How do you know blocking does not lose true matches? ★ LIKELY

Why they ask: Recall is the hidden ceiling of every entity-resolution system; they want measured evidence.

SAY We measure recall at every candidate step on labelled fold 9: the index keeps the true business for 99.37 % of US records and 98.51 % of Indian ones, and the shortlist loses about half a percentage point more, mostly to the cap of four. The quick filter is almost free.

IF THEY PUSH In the fold-9 samples the filter moves US recall from 99.3658 % to 99.3523 % and India from 98.388 % to 98.3745 %; the shortlist's full-fold lower bounds are 98.8748 % and 98.0572 %. These numbers use a ranker that never saw fold 9 and exclude the second-chance candidates, which target the remaining miss types: native-script names, and the same address with a different name.

NUMBERS

- Recall table, fold 9 (Doc B.2)

Q54. What is your blocking recall in France? ★★★

Why they ask: An adversarial probe of the unlabelled country.

SAY We cannot measure it: France has no labels, so our recall tables exist only for the US and India. What we can say is that France uses the same index with French cleaning rules, and the system keeps more alternatives there: 32.77 candidates per business after the filter against 15.04 in the US.

IF THEY PUSH In the final file France has 8.106 candidates per business and 1.4656 per incoming record, against 6.18 and 1.0742 in the US. With a small labelled French sample, recall at every step is the first thing we would measure.

NUMBERS

- After the filter: France 32.7713, US 15.0352 per business (Doc 3)
- Final: France 8.1060 / 1.4656, US 6.1836 / 1.0742 per business / per record (Doc 3, B.1)

Q55. Why do some businesses have hundreds or thousands of candidates? ★★★

Why they ask: Skew in the candidate distribution is where blocking systems blow up.

SAY Because our caps are per incoming record, not per business: a business that many records retrieve, for instance one whose name is made of common words, collects many candidates. The median is 7 per business, but 125 businesses have more than 100 and one French business has 3,182.

IF THEY PUSH Cost stays bounded because the shortlist adds at most four pairs per record, whatever the business. Crowded businesses are where many records compete for the same business: the collective features and the per-business consistency check in the US and India look at exactly that, and the clear-winner rule still requires each link to stand out among that record's own candidates.

IF THEY PUSH FURTHER 125 businesses have more than 100 candidates (US 3, India 32, France 90), with a maximum of 3,182 in France; overall median 7, 99th percentile 17. At the other end, 125 businesses have no candidate at all. Links per business never exceed 11. Our documentation has no breakdown of why each crowded business is crowded.

NUMBERS

- Candidates per business, median / p90 / p99 / max: all 7 / 11 / 17 / 3,182 (Doc B.1)
- 125 businesses with more than 100 candidates (US 3, India 32, France 90) (Doc B.1)
- Collective features and consistency check, US and India only (Doc 4); "common words" is the mechanism, not a profiled finding
- Candidates per business, all: median 7, p90 11, p99 17, max 3,182 (Doc B.1)

Q56. Did you consider classic blocking such as sorted neighbourhood, canopy clustering or MinHash LSH? ★★

Why they ask: They check your awareness of the entity-resolution literature and your reasons.

SAY We did not use them. Our word and letter-piece inverted index retrieves by any shared rare word, letter piece or sound rather than by agreement on one key, and the rarest-first caps act like stop-word removal, so a true pair only needs one rare shared term to be found.

IF THEY PUSH Sorted neighbourhood depends on one sort key, so a typo in the first letters can separate a true pair, which our letter-piece and sound channels tolerate; MinHash LSH approximates the Jaccard overlap of letter-piece sets probabilistically, while our lists compute an exact rarity-weighted overlap, deterministically. Canopy clustering needs a cheap similarity plus thresholds, which is close to what our weighted overlap and top lists already do; we did not benchmark these alternatives.

See also, answered elsewhere: "Why CatBoost for the ranker, and how is it trained?" see [Q71](#); "Why at most four candidates per record?" see [Q72](#); "Why not embedding-based (vector) blocking?" see [Q195](#); "What do the second-chance searches add to the candidate file?" see [Q118](#).

5. Scale, efficiency and cost (11 questions)

Q57. What exactly is bounded per record? ★★★

Why they ask: They test whether "linear" is backed by real caps in the code.

SAY Every stage has a per-record cap: a budget on how many index list entries we walk, a fixed number of top candidates per list, and at most four pairs for the readers. So total work is the number of records times a constant, plus building the index, which is linear in the number of businesses.

IF THEY PUSH The budgets add up to at most 65,000 list entries per record (derived), the candidates are the top 12 + 6 + 6 plus at most 2,000 same-name businesses, and on average we are far below that: about 27.5 and 23.1 index candidates per record in our US and Indian samples. After that only 1.17 pairs per record reach the readers.

NUMBERS

- Budgets and kept lists (Doc 3); candidates per record in samples (Doc B.2); 1.17 (derived)

Q58. Where does inference time go after retrieval and reading? ★★

Why they ask: They test whether you know your profile, not just the total.

SAY Very little goes anywhere else: once the readers are done, every remaining stage takes minutes. In our clean run, features took about 17 minutes, the decision models about 4, the address second-chance search about 10, and writing and validating the files about 6.

IF THEY PUSH In that run all inference stages together took about 3.9 hours of wall clock, including about 0.6 hours to retrain and run the French E5 reader, which a run from our stored models replaces by loading the shipped French E5 scores. These figures come from the clean run's stage timeline (derived), not from a dedicated benchmark.

NUMBERS

- Stage start times (Evidence: clean_run/stage_timeline.txt; durations derived)

Q59. How many model evaluations does a single incoming record trigger? ★★

Why they ask: Per-record cost is the unit that matters when you multiply by billions.

SAY On average: about 23 to 28 index candidates with cheap features, about four CatBoost ranker scores, and 1.17 shortlisted pairs, each read by two readers (three in France) and scored by five small LightGBM models. So the expensive readers run only a few times per record.

IF THEY PUSH Derived from the documentation: 37,614,782 ranker pairs / 9,969,589 records = 3.77 per record, and the index gives 27.541 (US) and 23.145 (India) candidates per record in the all-fold samples. Records still unlinked may also get a second-chance search, which is cheap by comparison.

NUMBERS

- 37,614,782 filtered pairs (Doc 3); 3.77 per record (derived); sample counts (Doc B.2)
- Readers per country and five-seed decision models (Doc 4)

Q60. How would you shard the index? ★★

Why they ask: Sharding is the concrete step behind any billion-scale claim.

SAY Today the index is split by country. At billions we would split by country and then by state or postcode, so each shard's lists and score arrays stay small, and run the shards in parallel.

IF THEY PUSH The risk is records with a missing or wrong state; the native-script second-chance search already handles that by falling back to all states when the state is unknown or has no match, and we would do the same for unlinked records. Smaller shards would also leave room for a larger cap than four where the cap loses true businesses.

NUMBERS

- State fallback of the native-script search (Doc 3)

Q61. At a hundred times the data, what breaks first? ★★★

Why they ask: A senior engineer wants you to find your own bottleneck before they do.

SAY The index, not the readers: reader cost just needs more GPUs, but each index thread keeps score arrays the size of its country's business list, and as lists grow, more common terms hit the caps and are skipped, which could cost recall for records made of common words. Both point to the same fix: shard by state or postcode so each index stays small.

IF THEY PUSH The address second-chance search is the other memory-heavy step, about 3.2 GB per worker today. We have not run at that scale, so these are limits we see in the design, not measured failures.

NUMBERS

- Per-thread score arrays (Code: index lookup); 3.2 GB per worker (CodeREADME 1)

Q62. How much memory and disk does it need? ★

Why they ask: Resource footprint is part of cost.

SAY Every stage needs less than 8 GB of RAM except the address second-chance search, which uses about 3.2 GB per worker, 13 GB with the default four workers, so the whole pipeline fits in 32 GB. A full run writes about 70 GB of intermediate files.

IF THEY PUSH The trained weights are about 2.1 GB, and the full archive with code, weights, encoders and stored reader scores is 3,285,958,103 bytes. Most stages use up to 6 CPU threads; the index lookup uses 8 to 11 in the training stages and some decision-model fits 8 to 12.

NUMBERS

- RAM, threads, disk, weights (CodeREADME 1, 6); archive size (GitHub README)

Q63. How much does the cascade save compared with reading every index candidate? ★★

Why they ask: They test whether you can put a number on the efficiency argument.

SAY About 19 times fewer pairs for the readers: they read the shortlist, about 6.75 pairs per business, instead of the roughly 130 index candidates per business. And thanks to the quick filter, the CatBoost ranker scores 37.6 million pairs rather than every index candidate.

IF THEY PUSH By linear extrapolation from the 1.4 hours our readers took on the shortlist, reading all index candidates would be on the order of 27 GPU-hours (derived, not measured). The 130 figure comes from our 25 September blocking study, not from the final run.

NUMBERS

- $130.1 / 6.7458 = 19.3$ (derived from Doc 3); 27 GPU-hours (extrapolation)

Q64. Is the CatBoost ranker worth it, or could the readers read everything the filter keeps? ★★

Why they ask: They test whether each stage pays for itself.

SAY The ranker cuts the readers' input from 21.71 to about 6.75 pairs per business, roughly three times fewer, using only cheap tabular features. The price is the cap of four, our largest recall loss, about half a percentage point of US recall on fold 9.

IF THEY PUSH By linear extrapolation from the 1.4 hours the readers took on the shortlist, reading all filtered pairs would be on the order of 4.5 GPU-hours (derived, not measured). The ranker's probability and rank are also inputs of the decision model, so it is more than a gate.

NUMBERS

- 21.7107 → 6.7458 per business (Doc 3, derived); recall 99.3694 % → 98.8814 % (Doc B.2)

Q65. How expensive is training the readers? ★★

Why they ask: They test whether fine-tuning cost allows fast iteration.

SAY Cheap enough to iterate on one GPU. The reader stage, which trains the MiniLM and E5 readers for one epoch each on labelled US and Indian pairs and then scores the decision models' training pairs with them, took about 2.1 hours, and fine-tuning the French E5 reader on 949,616 pairs took about 16 minutes.

IF THEY PUSH One epoch, bf16 mixed precision, batches of 128 and length-bucketed batches keep it cheap: grouping pairs of similar length means each batch is padded only to its own longest pair. The readers also train only on hard records plus a random 25 % of the easy ones.

NUMBERS

- 2.1 h for the reader stage, training plus scoring of the decision models' training pairs (Evidence: clean_run/RESULT.md; stage content: Code: run_all.sh); about 16 min for 949,616 pairs (Code: French E5 training)
- Training recipe and data selection (Doc 4)

Q66. What did you do to make the index lookup fast? ★★★

Why they ask: An engineering-depth probe on the stage that touches every record.

SAY It is C++17 with OpenMP: records are processed in parallel batches, each thread keeps its own dense score arrays and resets only the entries it touched, and terms are hashed to 64-bit keys. The rarest-first walk with caps means we never traverse the longest lists.

IF THEY PUSH The string similarities among the 76 features use a bit-parallel longest-common-subsequence computation, much faster than a dynamic-programming table for short names. Results are written in shards that the filter and ranker consume as they appear.

NUMBERS

- C++17 with OpenMP (Doc 3); implementation details (Code: index lookup)

Q67. Can it run without a GPU? ★★

Why they ask: Cost and deployment flexibility.

SAY Training needs a GPU, because the CatBoost ranker and the readers are trained on it. Inference could run on CPU at lower throughput: in our submitted run 48,813 shortlist pairs were in fact scored on CPU, but we have not timed a CPU-only run.

IF THEY PUSH At inference, index lookup, quick filter, ranker scoring, LightGBM decision models, second-chance searches and the France rules already run on CPU. Besides the readers, only the word-meaning features use the GPU when one is present, and that step falls back to CPU.

NUMBERS

- 48,813 pairs scored on CPU; 11,638,504 with reused GPU scores (CodeREADME 6)
- GPU needed for training and reader scoring (CodeREADME 1); ranker scoring on CPU threads, word-meaning encoder on GPU if available, else CPU (Code: blocking job, feature stage)

See also, answered elsewhere: "How would this scale to billions of records?" see [Q208](#); "How long does it take to run, and on what hardware?" see [Q221](#); "Could this run online, one record at a time?" see [Q210](#); "Why small readers, and no LLM?" see [Q194](#) and [Q199](#); "How do you make sure the output is reproducible?" see [Q222](#).

6. The CatBoost ranker and the shortlist (12 questions)

Q68. What does the CatBoost ranker do, and why does it sit between the quick filter and the readers? ★ **LIKELY**

Why they ask: They want to see that every stage of the cascade has one clear job and a cost reason for being where it is.

SAY The ranker orders the candidates that survive the quick rule-based filter, so that only the best few reach the expensive readers. It is a gradient-boosted tree model on similarity numbers the index has already computed, so scoring a pair costs almost nothing compared with a transformer pass. We keep at most four candidates per incoming record, 1.17 on average, and the ranker's probability and rank also feed the decision model.

IF THEY PUSH It turns 37,614,782 filtered pairs into an 11,687,317-pair shortlist, so the readers run on about 31 % of the pairs the ranker scores (derived). The cheap model decides what gets read; the expensive models decide what gets linked.

NUMBERS

- 37,614,782 pairs, 21.7107 per business after the quick filter (Doc 3)
- Shortlist 11,687,317 pairs: 6.75 per business, 1.17 per incoming record (Doc 3; per-business and per-record values derived)

Q69. Why not let the ranker do all the pruning and skip the quick rule-based filter? ★★

Why they ask: They test whether each stage is justified by cost, and whether you measured its price.

SAY Without the filter the ranker would have to score everything the index returns, roughly six times more pairs. The filter needs no model and costs almost nothing in accuracy: the true business survived it for 99.974 % of held-out records. So a free rule makes the coarse cut and the learned ranker makes the fine one.

IF THEY PUSH In the blocking study of 25 September the index gave 130.1 candidates per business and the filter left 21.7107 in the final run; with the decision model of that date, fold-9 macro F0.5 moved from 0.990117 to 0.990086 (fold 8: -0.00004). Its special clauses keep what a single combined score would drop: the record's best address match, an almost complete address match, very short names and acronyms, and businesses with exactly the same core name.

NUMBERS

- 130.1 → 21.7107 candidates per business (Doc 3)
- Owner kept for 99.974 % of fold-8/9 records; fold-9 F0.5 0.990117 → 0.990086 (Doc 3)

Q70. What features does the ranker see? ★★

Why they ask: They check that you know what information exists at each stage, and therefore what each stage can and cannot catch.

SAY Only the 76 pair features the C++ index computes: string similarities of names and addresses, overlaps of words, letter pieces, sound pieces and numbers, IDF-weighted overlaps, and the retrieval scores themselves, including how far the candidate is behind the record's best. It sees no reader score and none of the "what changed" features; those arrive later, in the decision model.

IF THEY PUSH In detail: 7 string similarities (a normalised longest-common-subsequence ratio, 5 on name forms and 2 on address forms); intersection, Jaccard, overlap-coefficient and intersection-over-max statistics for name words, address words, 3-grams, 4-grams, sound grams and numbers; length counts, equality and empty flags, IDF overlaps, token-alignment scores, a common-prefix share, exact core-name group sizes, and 8 retrieval scores (name, address, total, total over best, best, second, best minus second, raw candidate count).

NUMBERS

- 76 pair features (Doc 3; composition from Code: matcher.cpp)

Q71. How was the ranker trained, and on which records? ★★

Why they ask: A first-stage model trained on the wrong records leaks into everything downstream; they check the split.

SAY It is CatBoost with a Logloss objective, so strictly a pointwise classifier whose probability we sort by. It was trained on a pilot of 140,000 labelled US and Indian training records, fitted on the part in folds 0 to 5, with early stopping on fold 6. It is never fitted on folds 6 to 9 on purpose, because the decision models train there on its scores; fold 6 only decides when the ranker stops adding trees.

IF THEY PUSH Up to 1,600 trees of depth 7 (CatBoost's default symmetric trees, so 128 leaves each), learning rate 0.075, L2 regularisation 5, early stopping after 180 rounds, on GPU; the pilot holds 35,000 records per country and source, with all their index candidates as training pairs. Our documentation reports no study of a larger training sample, so we make no claim about one.

IF THEY PUSH FURTHER It is a pointwise binary classifier with log loss, not a listwise ranking loss, and a probability on an absolute scale is what lets the shortlist use an absolute floor of 0.001 for ranks 2 to 4. We did not run a head-to-head against other boosted-tree libraries for this step; its settings were up to 1,600 trees, depth 7, learning rate 0.075.

NUMBERS

- 140,000 records, 35,000 per country and source; folds 0-5 fit, fold 6 early stopping (Doc 4)
- 1,600 trees, depth 7, learning rate 0.075, L2 5, early stopping 180 (Doc 4)
- Ranker settings and training data (Doc 4); floor 0.001 (Doc 3)

Q72. Why at most four candidates per incoming record? ★★ LIKELY

Why they ask: Every cap is a trade-off; they want both the cost side and the recall side, with a measurement.

SAY It is a cost-recall trade-off, and it is our largest recall loss, which we measured rather than assumed. On held-out fold 9, the index finds the true business for 99.37 % of US records and the shortlist still holds it for at least 98.87 %. In return, the readers see on average little more than one pair per record instead of the three to four that survive the filter.

IF THEY PUSH For India the same step goes from 98.51 % to at least 98.06 %. The cap bounds the transformer passes per record, which keeps cost linear; at larger scale we would shard the index by state or postcode to afford a larger cap where it loses owners, and the second-chance searches recover part of what the index and the cap miss.

IF THEY PUSH FURTHER Ranks 2 to 4 are kept only if the ranker gives them at least 0.001, so the shortlist averages 1.17 pairs per record (derived), not four. For India the same step goes from 98.5068 % to 98.0809 %, and the second-chance searches exist to win back the miss types the index and the cap leave behind.

NUMBERS

- Fold 9, index → shortlist: US 99.3694 % → 98.8748-99.3628 %; India 98.5068 % → 98.0572-98.4831 % (Doc B.2)
- Candidates per record in the all-fold samples: after filter 3.111 (US) / 3.878 (India); shortlist 1.114 / 1.174 (Doc B.2)
- Index 99.3694 % vs top four without the filter 98.8814 % (US); 98.5068 % vs 98.0809 % (India) (Doc B.2)

Q73. What is the 0.001 floor for ranks two to four, and why always keep the top candidate? ★★

Why they ask: Small design details reveal whether you understand where cost and recall are spent.

SAY The top-ranked candidate always enters the shortlist, whatever its probability; ranks two to four enter only if the ranker gives them at least 0.001. Keeping the top one means every record with any candidate is read by the readers, so a weak ranker score can still be overturned. The floor stops us paying reader cost for candidates the ranker considers hopeless, and together with a confident ranker it leaves most records with a single candidate.

IF THEY PUSH 90.59 % of incoming records have exactly one candidate in the final candidate file. Our documentation does not record a separate sweep for the 0.001 value; what we measured is recall after the shortlist. In the India example the top ranker probability was only 0.0028: the readers and the decision model still examined that record, rejected its candidates, and it went on to the native-script second-chance search.

NUMBERS

- 90.59 % of records with exactly one candidate; 1.2331 candidates per record (Doc B.1)
- Ranker top-1 probability 0.0028 in the India example (Doc D.2)

Q74. How exactly did you measure the recall of the shortlist, and why do you quote a range? ★★

Why they ask: Blocking sets the ceiling of the whole system, so they test the rigour of the measurement.

SAY On labelled train fold 9, with the submitted index, the quick filter and a ranker fitted on folds 0 to 5, we counted the share of owned incoming records whose true business is still among the candidates after each stage. The range exists because, for the full fold, the evidence gives the top four without the filter exactly but the filter-then-top-four result only as a lower and an upper bound. On fold-9 samples of about 7,400 owned records per country we have exact values at every step, for example 98.8935 % for the US shortlist.

IF THEY PUSH The full-fold bounds are 98.8748 % to 99.3628 % (US) and 98.0572 % to 98.4831 % (India); the exact top four without the filter is 98.8814 % and 98.0809 %. The samples show the filter itself costs little (US 99.3658 % at the index, 99.3523 % after the filter) and the cap of four is the larger loss; second-chance candidates are not included in these figures.

NUMBERS

- Owned records in fold 9: US 457,971, India 303,786 (Doc B.2)
- Fold-9 samples: US 99.3658 → 99.3523 → 98.8935 %; India 98.388 → 98.3745 → 97.985 % (Doc B.2)

Q75. In the US example the ranker put the LLC second, but the decision model put it last. Is the ranker wrong? ★★

Why they ask: They probe whether you understand the division of labour in a cascade.

SAY No: the ranker's job is only to keep the owner inside the top four, not to make the final order. It sees index similarities, and on those the Inc, LLC and PC names are nearly identical. The decision model adds the readers and the legal-form features, so it scores the LLC 0.000007 against 0.965022 for the Inc.

IF THEY PUSH The ranker's probability and rank are themselves inputs of the decision model, so it learns from labels how far to trust the ranker and when the readers and "what changed" features should override it. A cascade is judged on whether the owner survives each stage, not on whether each stage agrees with the last.

NUMBERS

- Decision-model scores: Inc 0.965022, LLC 0.000007, PC 0.088664, Industries 0.000088 (Doc D.1)

Q76. The readers are much stronger than the ranker. Why still give the ranker's probability to the decision model? ★★

Why they ask: A stacking question: they test whether you know that a weaker model can still add information.

SAY Because it summarises different evidence: the ranker condenses the 76 index similarities, including address and number overlap and the retrieval scores, into one number learned from labels, while the readers read raw text. When the two agree the decision is easy; when they disagree, the decision model learns from labels which one to trust in that situation. It enters as log-odds, together with its rank, best rival and lead within the record's shortlist.

IF THEY PUSH Trees pick up such interactions cheaply, for example a high reader score combined with a weak address overlap. We have not prepared a feature-importance ranking, so we do not claim how much weight it carries; the US example shows the decision model is free to overrule the ranker's order.

Q77. The ranker was trained only on US and Indian labels. Doesn't that hurt France? ★★★

Why they ask: Domain mismatch at the first learned stage is a silent recall risk; they want to see you know where it bites.

SAY It can, and we cannot measure it, because France has no labels. Two things limit the risk: the ranker's inputs are language-independent similarity numbers computed on French-cleaned text, and the shortlist keeps the top candidate plus any rank up to four above 0.001, so the French readers and decision model re-score whatever survives. The remaining risk is an owner falling below rank four, which only French labels would reveal.

IF THEY PUSH France does pass more candidates through: 32.7713 per business after the filter and 1.4656 per incoming record in the candidate file, against 15.0352 and 1.0742 for the US. With a small labelled French sample, shortlist recall is the first thing we would measure, exactly as in our recall table.

NUMBERS

- After filter: France 8,502,590 pairs (32.7713 per business), US 9,969,935 (15.0352) (Doc 3)
- Candidates per incoming record: France 1.4656, US 1.0742 (Doc B.1)

Q78. Why a learned ranker at all? Why not cut to four by the index score? ★★

Why they ask: They test whether each learned component earns its place over a simpler heuristic.

SAY The index score is one hand-weighted formula; the ranker learns from labels how to combine all 76 signals, for example a weak name match with a perfect address, so the owner is more likely to land in the top four. It also gives an absolute probability we can floor at 0.001 and pass to the decision model. Our documentation does not report top-four recall by the raw index score alone, so we cannot quote the gain as a number.

IF THEY PUSH The comparison we would run: on fold 9, top-four recall by the index total versus by the ranker, with the filter fixed. What we do have is the recall with the ranker: at least 98.87 % (US) and 98.06 % (India) in the shortlist.

Q79. If you had to improve recall at the shortlist, what would you do first? ★★

Why they ask: They want to see you can turn a measured weakness into a concrete plan.

SAY The cap of four is our largest measured recall loss, so we would start there, but only where it bites: shard the index by state or postcode so lists are shorter, then afford a larger cap where the cap loses true businesses. We would also consider giving the ranker a few cheap "what changed" signals, such as the legal-form relation, since today it sees only index similarities. Each change would be judged by the same per-stage recall table on fold 9 and by reader pairs per record.

IF THEY PUSH Today the gap is 99.37 % at the index against at least 98.87 % in the shortlist (US) and 98.51 % against at least 98.06 % (India), so at most about half a percentage point is recoverable at this step. Owners the index never finds need a different retrieval channel, which is the job of the second-chance searches.

NUMBERS

- Index and shortlist recall, fold 9 (Doc B.2)

7. The readers (cross-encoders) (15 questions)

Q80. What exactly is a "reader" in your pipeline? ★ LIKELY

Why they ask: The basic check that you can explain your core model in one breath.

SAY A reader is a cross-encoder: a small multilingual transformer that reads the incoming record and one candidate business together, as one input, and outputs one match score. Because both texts sit in the same sequence, every word of one side can attend to every word of the other, so it notices small decisive differences such as Inc becoming LLC. We feed it the raw text, "name | address" for each side, cut to 128 tokens.

IF THEY PUSH The encoder's last hidden states are averaged over the real tokens (masked mean pooling), then dropout 0.1 and a linear layer give one logit. Reading jointly means nothing can be pre-computed per business, so every pair costs a full transformer pass, which is why readers see only the shortlist, about 1.17 pairs per record. Raw rather than cleaned text lets the reader see what cleaning removes or rewrites: case, punctuation, native script and the exact spelling of the legal form.

NUMBERS

- 128-token limit, longest-first truncation, dropout 0.1, one logit (Doc 4)

Q81. Why put the name and the address into one text instead of reading them separately? ★★

Why they ask: They test whether you understand what joint attention buys you.

SAY Because the decision depends on how name and address agree together, not separately: the same name in another city is usually another business, while a slightly different name at exactly the same address can be the same one. One sequence lets attention connect the record's city to the business's city and the two names in the same pass. Our French example fits this: a business with exactly the same name, but in Pessac instead of Nantes, was rejected by the French readers.

IF THEY PUSH Within each side the "|" marks the boundary between name and address; between the two sides the tokenizer's pair encoding inserts separator tokens, and we pass no segment ids, so the separators and positions tell the model which text is the record and which the business. Separate name and address models would lose the interactions between fields, which the decision model would then have to reconstruct from two coarse scores.

Q82. Why MiniLM and E5? ★★ LIKELY

Why they ask: Model selection under constraints (licence, size, languages, compute) is a core applied-science judgement.

SAY We needed multilingual models for Indian scripts such as Devanagari and for French, under open licences, small enough to score millions of pairs on one GPU. The MiniLM reader starts from a model of about 118 million parameters under Apache-2.0 and the E5 reader from one of about 278 million under MIT, both far below the competition's size limit. Two differently pretrained encoders give the decision model two partly independent opinions; for the US and India it sees both logits and their mean.

IF THEY PUSH paraphrase-multilingual-MiniLM-L12-v2 is a distilled multilingual paraphrase model and multilingual-e5-base a contrastively pretrained retrieval encoder; as readers both are fine-tuned end to end with a fresh one-logit head. A larger xlm-roberta-large reader, about 560 million parameters, was tried during development and discarded; no stage uses it.

NUMBERS

- 117,653,760 (Apache-2.0) and 278,043,648 (MIT) parameters, plus a 385- / 769-parameter head (Doc B.5)

Q83. Your E5 readers use the MiniLM tokenizer. Isn't that a bug? ★★★

Why they ask: An expert who reads the documentation notices this and checks that you understand tokenisation.

SAY No: both base models use the same XLM-RoBERTa SentencePiece vocabulary, so the MiniLM tokenizer produces exactly the token ids the E5 model expects. That lets the same tokenised pairs serve every reader. We also do not add E5's "query:" and "passage:" prefixes, because each reader is fine-tuned end to end with its own head for pair classification.

IF THEY PUSH Our documentation states it directly: all readers tokenise the pair with the MiniLM tokenizer. The E5 prefixes matter for its original retrieval use, where one embedding space must separate queries from passages; a cross-encoder with a new classification head learns the task through fine-tuning without them.

Q84. Why mean pooling and not the [CLS] token? ★★

Why they ask: A detail question to check you understand the architecture you actually used.

SAY Both base models were pretrained as sentence encoders with mean pooling, so averaging the token vectors matches how their representations were trained. We take the attention-masked mean of the last hidden states, so padding does not dilute it, then dropout and a one-logit linear head. We have no comparison with [CLS] pooling to quote.

IF THEY PUSH For a cross-encoder the head must summarise interactions spread over both texts, and a mean over all real positions is a stable summary; since the whole encoder is fine-tuned, it also adapts to the pooling we chose.

Q85. How were the readers trained, and why only one epoch? ★★

Why they ask: Training details show whether you can reproduce and debug the model, and whether defaults were checked.

SAY Standard full fine-tuning: binary cross-entropy on labelled US and Indian pairs from folds 0 to 5, AdamW, the whole encoder trained and the new head at five times the encoder's learning rate, with a short linear warm-up, bf16 mixed precision and gradient clipping. One epoch, because the training set is large and the encoders are already strong, so in our reasoning one pass adapts them while limiting memorisation of the training businesses' strings; we have no epoch sweep to quote, so we do not claim one epoch is optimal.

IF THEY PUSH Learning rate 5e-5 for the encoder and 2.5e-4 for the head, weight decay 0.01, 3 % warm-up then linear decay, batch 128, clip norm 1.0; with bf16 autocast the weights and the optimiser stay in fp32 and only the forward computations run in bf16, which needs no loss scaling because bf16 has fp32's exponent range. Batches come from chunks of 100 batches sorted by length and then shuffled, which cuts padding without fixing the order; on held-out folds 8 and 9 the E5 reader reaches AUC 0.998570 on businesses it never saw.

IF THEY PUSH FURTHER The recipe: binary cross-entropy; AdamW with weight decay 0.01 and a five times higher learning rate on the head; 3 % linear warm-up then linear decay; bf16 autocast; gradient clipping at 1.0; batch 128; learning rate 5e-5 (2e-5 for the French fine-tunes); longest-first truncation to 128 tokens; length-bucketed batches to cut padding. What we would measure next: the per-business F0.5 of the whole decision after one versus two epochs, since the reader's AUC is already near its ceiling.

NUMBERS

- BCE, AdamW (weight decay 0.01, head $\times 5$), 3 % warm-up, bf16, clipping 1.0, 1 epoch, batch 128, learning rate 5e-5 (Doc 4)
- E5 reader AUC 0.998570, top-1 accuracy 0.993004 on held-out fold-8/9 pairs (Doc B.4)
- One epoch, batch 128, 128 tokens (Doc 4)

Q86. Which pairs did you train the readers on? Did you mine hard examples? ★★ LIKELY

Why they ask: Training-data selection usually matters more than architecture; they want to see you focused on the decision boundary.

SAY Yes. We took every "hard" training record, where the ranker was unsure or ranked a wrong business first, plus a random quarter of the easy ones. For those records we used the candidates the shortlist would keep, plus the true business even when the ranker had put it as low as rank eight.

IF THEY PUSH Hard means a top ranker probability between 0.003 and 0.9995, a non-owner ranked first, or a second candidate above 0.003; candidates 1-4 are kept if ranked first, at least 0.001 or the owner, and candidates 5-8 only if the owner. Easy pairs teach little, but keeping 25 % of them stops the reader from training on a sample that is all borderline, which would distort its scores on the many easy test pairs (our reasoning, not a measured ablation).

IF THEY PUSH FURTHER A hard record has a top ranker probability between 0.003 and 0.9995, or a non-owner ranked first, or a second-best probability above 0.003. Candidates 1-4 enter if ranked first, if their probability is at least 0.001 (the shortlist floor) or if they are the owner; candidates 5-8 only if they are the owner. The MiniLM and E5 readers use the same pairs, from labelled US and India folds 0-5. The reasoning: easy records are already settled by the ranker, so the reader's capacity goes where the ranker is uncertain.

NUMBERS

- Hard records: top probability in (0.003, 0.9995), a non-owner first, or second probability > 0.003; plus 25 % of the others (Doc 4)
- Hard record: top probability in (0.003, 0.9995), a non-owner ranked first, or second probability > 0.003; plus a random 25 % of the others (Doc 4)

Q87. How good are the readers on their own? ★★ LIKELY

Why they ask: They want a number for the component, and to see you know the difference between pair-level and business-level metrics.

SAY On held-out pairs of labelled folds 8 and 9, the E5 reader alone has AUC 0.998570 and puts the right candidate first 99.3 % of the time. These are pair-level numbers on businesses it never trained on; the competition score is per business, after the decision model, so they are not directly comparable with our leaderboard score.

IF THEY PUSH Its best pair-level F0.5 is 0.992796, on 509,260 pairs of 437,715 records. These figures come from the French E5 reader's written check, where the plain E5 reader was the reference; our documentation has no equivalent table for the MiniLM reader, so we do not quote one.

IF THEY PUSH FURTHER On 509,260 pairs of 437,715 labelled fold-8/9 records: E5 reader AUC 0.998570, top-1 accuracy 0.993004, best F0.5 0.992796 (measured on these reader pairs at the best cut-off, not the per-business macro metric); French E5 reader 0.998544, 0.993161 and 0.992782. In our French example the MiniLM reader alone scored the wrong-city business +7.1, which is why no reader decides alone.

NUMBERS

- E5 reader, folds 8-9: AUC 0.998570, top-1 accuracy 0.993004, best F0.5 0.992796 (Doc B.4)
- Reader quality (Doc B.4)

Q88. If the readers are that good, why do you need a decision model? ★★ **LIKELY**

Why they ask: This tests whether you see matching as a comparative decision rather than a pair classification.

SAY A reader scores one pair at a time, but the decision is a competition: among a record's candidates and, for the US and India, among records competing for the same business. The decision model sees the reader logits next to the ranker, each candidate's rank and lead within the shortlist, the "what changed" features and the collective features, and gives one probability on which we set an operating point for the exact metric. A reader also cannot see corpus-level facts, such as how rare a dropped word is across the country.

IF THEY PUSH Transformers are unreliable at arithmetic on digit strings, so exact number differences come in as features. Even with pair-level AUC near 0.999, a record can have two plausible candidates, and choosing between them, or abstaining, needs the comparison that the decision model's context features and the lead rule provide.

Q89. How do the reader scores enter the decision model? ★★

Why they ask: They test the stacking interface: raw scores, probabilities, or derived context.

SAY As raw logits plus context within the record's shortlist: for each reader, the candidate's rank, the best other candidate's logit, the margin to it and the top value. For the US and India both the MiniLM and E5 readers go in, plus their mean with the same context; France has a single reader slot filled by a blend of three readers.

IF THEY PUSH For one feature on its own a tree is invariant to monotone transforms, but differences and means (margins, the two-reader mean) behave differently in logit space, where they add up as log-odds evidence. The ranker probability also enters as log-odds.

Q90. What is a logit, and what does a reader score of 7.5 mean? ★

Why they ask: A basic check that you can read your own numbers.

SAY A logit is the raw output before the sigmoid: the log-odds that the pair is a match. A logit of 0 means 50 %, and 7.5 means about 99.94 %. We keep logits rather than probabilities because they spread out the region near certainty, where the important differences sit.

IF THEY PUSH In the French example the MiniLM reader gave the same-name business in Pessac +7.125 (about 99.9 %), while the French E5 reader gave it –10.8125 (about 0.002 %). Averaging probabilities would hide how strongly the French reader disagreed, while averaging logits keeps it, which is one reason a logit blend suits us.

NUMBERS

- Logits 7.125 (MiniLM) and –10.8125 (French E5) for the Pessac business (Doc D.3); percentages derived with the sigmoid

Q91. Why does France not use the plain E5 reader? ★★

Why they ask: They check that you know exactly which models run where.

SAY For France the E5 reader is replaced by its French version, the French E5 reader, which started from the E5 reader and was fine-tuned on French pseudo-labels with labelled US and Indian replay. The France decision model has one reader slot, and it receives a blend of the MiniLM reader, the French MiniLM reader and the French E5 reader. The US and India use the MiniLM and E5 readers, and an unseen country uses only the MiniLM reader.

IF THEY PUSH The French E5 reader kept the E5 reader's US and Indian quality within our written tolerance (AUC 0.998544 against 0.998570) while agreeing with held-out French pseudo-labels at 0.999206, where the plain E5 reader agreed at 0.981304. That agreement is with labels produced by the two MiniLM readers, so it shows consistency with its teachers, not French accuracy.

NUMBERS

- France blend: 0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5 (Doc 4)
- AUC 0.998544 vs 0.998570; agreement 0.999206 vs 0.981304 (Doc B.4)

Q92. In the French example, the original MiniLM reader liked the same-name business in Pessac. Why did the French readers reject it? ★★

Why they ask: They test whether you can explain model behaviour without overclaiming.

SAY We can show it, but not prove why. The record is in Nantes and that business is in Pessac, another city; the MiniLM reader gave it a strongly positive score, both French readers negative ones, and the decision model gave it 0.000098. Our reading is that fine-tuning on French pairs taught the French readers to weigh French address parts, such as the city, against an identical name.

IF THEY PUSH Logits were +7.125 (MiniLM), -1.9219 (French MiniLM) and -10.8125 (French E5), blend -3.2008. It is one example chosen to show the mechanism, and test labels are private; the systematic evidence is the written checks, for example the generated labelled French test set, where the 50/50 blend with the French E5 reader reached a best pair-level F0.5 of 0.90843 against 0.898031 for the blend without it.

NUMBERS

- Pessac business: logits 7.1250 / -1.9219 / -10.8125, blend -3.2008, decision score 0.000098 (Doc D.3)

Q93. Why fine-tune your multilingual readers for France instead of starting from a French model such as CamemBERT? ★★

Why they ask: They test whether you thought about the adaptation strategy, not just the model name.

SAY Our MiniLM and E5 readers had already learned the matching task from labelled US and Indian pairs, and both base models cover French from multilingual pretraining, so fine-tuning them transfers that skill and only adapts it to French text. A French-only encoder would have had to learn the task again, from pseudo-labels or from English and Indian pairs it reads less well, and our replay of US and Indian pairs needs a multilingual vocabulary. We have no benchmark of a French-only encoder, so this is our reasoning, not a measured comparison.

IF THEY PUSH Replay of labelled US and Indian pairs during French fine-tuning protects that transferred skill: the French E5 reader stayed within tolerance of the E5 reader on held-out US and Indian pairs. Starting from our own readers also suits the France decision model, which was trained with the MiniLM reader's logits in its reader slot: readers fine-tuned from them with the same objective and with replay give the same kind of log-odds score (our reasoning, not a measurement).

Q94. How long do the readers take, and how did you make scoring 11.7 million pairs fast? ★★

Why they ask: Applied scientists at Amazon care about cost per record as much as accuracy.

SAY In our clean run on one RTX 4070 Ti SUPER 16 GB, the stage that trains the MiniLM and E5 readers took about 2.1 hours, and scoring the whole test shortlist with two readers per pair about 1.4 hours. Scoring is fast because pairs are sorted by token length before batching, so batches carry almost no padding, and the forward pass runs in bf16. Readers are still the most expensive stage per pair, which is why at most four pairs per record reach them.

IF THEY PUSH As a rough linear extrapolation, not a measurement: a billion incoming records would mean about 1.17 billion reader pairs, on the order of 140 GPU-hours spread across shards. The French E5 reader fine-tune itself was short: 949,616 pairs, 7,419 steps, about 16 minutes on the same GPU.

NUMBERS

- Reader training stage about 2.1 h (it also builds the training pairs and scores the folds 6-9 training shortlist); test-shortlist reader scoring (all readers except the French E5 reader) about 1.4 h (Evidence: `clean_run/RESULT.md`; stage contents from Code: `run_all.sh`)
- French E5 reader: 949,616 pairs, 7,419 steps, about 16 min (Code: `e5fr/train_e5fr.py`)

8. Features and the "what changed" idea (14 questions)

Q95. What are the "what changed" features, in plain words? ★ LIKELY

Why they ask: This is your headline idea; they check you can explain it simply and precisely.

SAY For every candidate we describe the edit that turns the business into the incoming record, not only how similar the two are. Did a number change, and by how much? Which words were added, dropped or replaced, how rare are they and how close in meaning, and did the legal form change? The decision model then learns from labels which edits are harmless.

IF THEY PUSH There are 25 generic pair features: 8 for numbers, 14 for words and 3 for meaning similarity, plus a legal-form relation (same, changed, record has none, business has none, superset, subset, both none) and a number relation (none, same, extra only, dropped only, one side without a number, substituted).

NUMBERS

- 25 generic pair features plus legal-form and number relations (Doc 4; split 8 / 14 / 3 from Code: clean/clean_feats.py)

Q96. Why describe differences instead of just measuring similarity? ★★ LIKELY

Why they ask: They want the reasoning behind the innovation, not only its description.

SAY A similarity score treats two edits of the same size as equal, but they mean different things. Studying the labelled data, we saw that true records of a business tend to lose information: they drop a digit or a suffix, abbreviate, or transliterate. So we describe the edit itself and let the model learn from labels which kinds of change are harmless and which point to a different business.

IF THEY PUSH "Legacy Marketing Enterprises" against "Legacy Marketing Industries" is very similar by characters, yet it is an in-place replacement of one distinctive word, and the decision model gives it 0.000088. Because the features encode operations rather than word lists, the same definitions carry over to another country.

Q97. Walk me through one example where these features decide. ★ LIKELY

Why they ask: Concrete examples prove understanding better than definitions.

SAY The US record "Legacy Marketing Enterprises (Inc)" has no address, and its shortlist holds four look-alike names. The word features ignore legal-form words, so for them the three "Enterprises" names are identical; the legal-form features flag Inc to LLC and Inc to PC, and the word features flag "Industries" replacing "Enterprises" in place. The decision model gives the Inc 0.965 and the best other 0.089, so we link only the Inc in Windsor, Connecticut.

IF THEY PUSH For the "Industries" candidate: one replaced word, in place, IDF-weighted Jaccard 0.52 and meaning similarity of the replaced word 0.77, so related but different; its score is 0.000088. The lead is $0.965022 - 0.088664 = 0.876$, far above 0.4.

IF THEY PUSH FURTHER The ranker had put the LLC second; the decision model gives it 0.000007 because it sees the legal-form relation. The word features ignore legal-form words, so for them the three "Enterprises" names are identical; for Industries they record one word replaced in place, with an IDF-weighted Jaccard of 0.52 and a meaning similarity of 0.77 for the replaced word. The lead is 0.876.

NUMBERS

- $0.965022 / 0.088664 / 0.000088 / 0.000007$; IDF-Jaccard 0.52, meaning similarity 0.77 (Doc D.1)
- Worked example (Doc D.1)

Q98. How do you compute "added", "dropped" and "replaced" words? Is it an edit alignment?

★★★

Why they ask: An expert checks whether the feature is well defined and where it breaks.

SAY It is a cheap set difference, not an optimal alignment. We tokenise both names (lower case, accents stripped, native script transliterated with the word list learned from training matches, legal-form words and and/et/& removed, up to 8 tokens); words only in the record are "new", words only in the business are "dropped". Replaced is the smaller of the two counts, added and deleted are what remains, and we describe the first new and first dropped word in detail: positions, whether they sit in the same place, character similarity, rarity and meaning similarity.

IF THEY PUSH The weakness is multiple edits, since only the first new and first dropped word get the detailed description; counts, the maximum and minimum IDF over all new words and the maximum IDF over dropped words partly compensate, and an order flag records whether shared words keep their order. A token-level alignment would handle several edits better at higher cost; we have no comparison with one to quote.

Q99. Why use word rarity (IDF), and isn't computing it on the test files a form of leakage?

★★★

Why they ask: They check both the modelling intuition and your data hygiene.

SAY Rarity says how much information a word carries: losing or swapping a distinctive word says much more about identity than losing or swapping a very common one, and the model learns from labels in which direction each case points. IDF is $\log((N + 1) / (df + 1))$ over all records of the country, computed without labels. It does use the unlabelled test records, which we disclose; it is an unsupervised corpus statistic, and no test label enters.

IF THEY PUSH The same holds for the index's IDF, the blocking indexes and the competition features, all listed in our disclosures. IDF is computed per country on each split's own records, so a word's rarity is measured the same way at training and at test time.

NUMBERS

- $IDF = \log((N + 1) / (df + 1))$, label-free (Code: clean/clean_feats.py); test-file statistics disclosed (Doc C.1)

Q100. What are the meaning-similarity features, and why use a frozen model for them? ★★

Why they ask: They test whether "semantic" features are substantive or decoration.

SAY We embed single words with the pretrained multilingual MiniLM, not fine-tuned, and take cosines: the first new word against the first dropped word, the best match of the new word to any business word, and the best match of the dropped word to any record word. They tell the model whether a replacement is a synonym, translation or spelling variant, or a different concept. In the US example "Industries" against "Enterprises" scored 0.77, related but not the same, and the decision model still rejected it.

IF THEY PUSH Frozen keeps the features generic and independent of our labels, and avoids duplicating what the fine-tuned readers already learn. Word-level cosines from a sentence model are coarse, which is why the decision model combines them with rarity and position; when there is no replacement the value is coded -2.

Q101. What do the number features capture? ★★

Why they ask: Addresses carry much of the identity signal; they want to see you treated digits as structure, not text.

SAY From the address digit runs the two records do not share, up to eight per address, we pair every extra number in the record with every missing number in the business. We record the smallest absolute difference, the smallest positive and the largest negative difference over those pairs, and for the closest pair its relative difference, the Damerau-Levenshtein distance between the digit strings, their common prefix and suffix length and their digit-length difference. A separate number relation says whether numbers are the same, only added, only dropped, missing on one side, or substituted.

IF THEY PUSH These separate edits that look alike as strings: a transposition such as 4512 and 4152 has edit distance 1 but a large numeric difference, while a dropped digit shows as a length difference with a shared prefix. The decision model learns from labels which patterns are harmless; no rule about specific number changes is hard-coded in these features.

Q102. Did the "what changed" features actually help? Do you have an ablation? ★★★ LIKELY

Why they ask: Claims of novelty need evidence; they test your honesty about what was and was not measured.

SAY We did not run a clean ablation that removes all "what changed" features. The closest evidence: when we replaced data-specific inputs with the final generic "what changed" inputs, the public score moved from 0.990881 to 0.990934, so the generic version cost nothing and helped slightly. A code check asserts that none of the removed inputs reaches a final decision model.

IF THEY PUSH On held-out folds the generic version was within a few hundred-thousandths of the earlier one, so we chose it for generality, not for a measured gain. The ablation we would run: retrain the decision models without the 25 features, score folds 8 and 9 with the exact per-business F0.5, and report a paired bootstrap interval over businesses.

NUMBERS

- US/India: 21 data-specific inputs removed, 25 generic added; France and default: 16 removed, 25 added (Doc 4)
- Public 0.990881 → 0.990934 (Doc B.6, uploads 13-14)

Q103. Why are legal-form words removed from the word features? Isn't that throwing information away? ★★

Why they ask: They probe a design choice that looks odd at first sight.

SAY It is so that each kind of change is described once, by the feature built for it. Legal-form differences are captured by the legal-form relation and the legal-form comparisons, and the readers still see the raw legal form. If legal words stayed in the word features, Inc to LLC would also look like a replaced word and a dropped suffix like a dropped word, mixing two different signals.

IF THEY PUSH Dropping a legal suffix is a common change in true records, while replacing one legal form with another is a different signal, and separate features let the model weigh them differently. In the US example the word features see three identical "Enterprises" names, and only the legal-form features and the readers separate Inc, LLC and PC.

Q104. What are the collective features, and why only in the US and India? ★★★

Why they ask: Cross-record features are powerful but unusual; they want to know you understand what they look at.

SAY They look at the other incoming records competing for the same business, meaning records whose top ranker candidate is that business with probability at least 0.01. They include four generic counts of how those other records differ from the business, for example how many have a different number set or a different legal form, and for the US six more that summarise how their house numbers compare with the business's. France uses the simpler 158-feature recipe built from our first-generation decision model's feature list, and these collective features belong to the US and India recipes only.

IF THEY PUSH The motivation is the metric: a business owns several incoming records and is scored as a whole, so evidence from its other candidate records changes how plausible each link is. They are computed from ranker outputs and raw text only, never from labels, so the same computation runs on the test files.

NUMBERS

- Collective features in the US and India models, including 4 generic counts (US, India) and 6 value features (US only) (Doc 4)

Q105. Besides "what changed", what else does the decision model see? ★★

Why they ask: They want the full picture of the feature space and why each family is there.

SAY Four more families: the index similarities, the reader logits, the ranker probability and rank, and context, meaning each candidate's rank, lead and best rival within the record's shortlist, the shortlist size, the raw candidate count and the source flag. There are also forensic name, legal-form, number and address comparisons, and for the US and India the collective features; about two hundred features per country in total.

IF THEY PUSH The exact counts are 217 for the US, 211 for India, and 158 for France and the unseen-country route. The context features matter because the decision is comparative: the same reader score means something different when a rival is close behind.

NUMBERS

- US 217, India 211, France and default 158 (Doc 4)

Q106. Which features matter most? ★★ **LIKELY**

Why they ask: A standard question; they check that you know the limits of importance measures.

SAY We did not prepare a feature-importance ranking for this talk, so we will not quote one. The worked examples show the mechanism: in the US example the legal-form features and the readers separate the three "Enterprises" names and the word-replacement features mark "Industries"; in the French example the French readers reject a same-name business in another city. To measure it properly we would use grouped permutation importance or SHAP on held-out folds, by feature family.

IF THEY PUSH Split-gain importance in boosted trees is biased towards features with many split points and is unstable among correlated features, and our families are strongly correlated (two readers, many string similarities). Permuting a whole family on a held-out fold, scored by fits that leave that fold out, and re-scoring the exact per-business F0.5 is the honest measure.

Q107. How do these features carry over to France, which has no labels? ★★

Why they ask: Generalisation to an unlabelled domain is the hardest part of your story.

SAY They describe operations, not French words: a number changed, a word added, dropped or replaced, a legal form changed, with word rarity computed from France's own records. So the same definitions apply to French text, and the France decision model, trained on US and Indian rows, reads them with the meaning it learned there. We cannot measure the transfer without French labels; our closest proxy is the France recipe applied to US and Indian rows at the French operating point.

IF THEY PUSH The word features read the raw names (lower-cased, accents stripped, legal-form words and "et" removed), and the IDF is computed over France's own business and incoming records, so rare French words count as rare. The proxy scores 0.990508 (US) and 0.989467 (India) on fold 9; it shows the recipe and the stricter threshold work where labels exist, not that France reaches the same level.

NUMBERS

- France recipe proxy, fold 9: 0.990508 / 0.989467; fold 8: 0.990977 / 0.989850 (Doc 5)

Q108. What can the readers see that the features cannot, and the other way round? ★★★

Why they ask: This tests whether you understand why a hybrid of neural and hand-built signals beats either alone.

SAY The readers see raw text in context: word order, script, spelling variants, multilingual meaning and the address as language. The features give structured facts that a 128-token text model handles poorly: exact numeric differences, word rarity measured over millions of records, and the competition with other candidates and, for the US and India, other records. The decision model combines both.

IF THEY PUSH Sub-word tokenisers split digit strings into arbitrary pieces, so a transformer has no reliable notion of how far apart two house numbers are, and IDF is a corpus statistic no single pair contains. Conversely, the features only see what their definitions name, while the readers can pick up cues nobody wrote a feature for.

9. Decision models and thresholds (9 questions)

Q109. What does the decision model do? ★ LIKELY

Why they ask: Baseline understanding of the final scoring stage.

SAY For every shortlisted candidate it outputs the probability that this business owns the incoming record, using about two hundred features: reader scores, the ranker, the "what changed" features and the context of the competing candidates. It is a LightGBM binary classifier, one per country, averaged over five seeds. A simple rule then turns those probabilities into at most one link per record.

IF THEY PUSH Settings: learning rate 0.05, 127 leaves, at least 200 rows per leaf, feature fraction 0.9, bagging fraction 0.8 every iteration, L2 1.0; the label is "this candidate is the owner".

NUMBERS

- LightGBM: learning rate 0.05, 127 leaves, 200 rows per leaf, feature fraction 0.9, bagging 0.8, L2 1.0 (Doc 4)

Q110. Explain the "confident and clear winner" rule. Why do you need both conditions? ★★ LIKELY

Why they ask: The decision rule is where precision is won or lost; they test the logic.

SAY Per incoming record we take the best-scoring candidate and link it only if its score is at least 0.75 and it leads the runner-up by at least 0.4; otherwise we link nothing. The threshold asks "is it likely enough?", the lead asks "is it clearly better than the alternative?", which protects us against near-ties between two plausible businesses, where picking either one risks a wrong link. If a record has only one candidate, the lead condition passes automatically.

IF THEY PUSH It is a reject option, arg-max with an absolute threshold and a margin, and at most one link per record enforces the many-to-one structure. Why abstaining pays under F0.5, as an illustration with per-business $F0.5 = 1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$: for a business with 3 true records, linking all 3 plus 1 wrong one scores $3.75 / 4.75 = 0.79$, while missing one and linking only 2 correct ones scores $2.5 / 2.75 = 0.91$, so a wrong link costs more than a missed one.

NUMBERS

- Threshold 0.75 (US, India), 0.82 (France, unseen country); lead 0.4 everywhere (Doc 4)
- 90.59 % of records have exactly one candidate (Doc B.1)

Q111. How did you choose the 0.4 lead? ★★★ LIKELY

Why they ask: They noticed one parameter shared by every country and want to see whether it was justified.

SAY 0.4 is one fixed lead used in every country's rule, and the US and India threshold next to it was tuned on fold 8 for the exact metric. Our documentation records the threshold choice but not a separate study of the 0.4, so we do not claim one. Its role is clear: with scores between 0 and 1, a lead of 0.4 means we link only when one candidate clearly dominates.

IF THEY PUSH Most records have a single candidate, where the lead passes automatically, so 0.4 acts on the minority with real competition. The evidence that would defend it with a number is a joint threshold-by-lead sweep on fold 8, reported on fold 9; our documentation does not include one.

Q112. How often does the system abstain? Isn't linking only about 59 % of incoming records too cautious? ★★

Why they ask: They test whether you can tell caution from under-linking, and what evidence you have either way.

SAY About 41 % of incoming records end unlinked, and 100,087 businesses have no link. That is not automatically too cautious: some incoming records belong to no business, and the threshold was tuned on held-out labels for the exact metric, where a missed link costs less than a wrong one. We cannot measure the true unowned share on the test files, because their labels are private.

IF THEY PUSH Links per incoming record are 0.5895 (US), 0.5806 (India) and 0.6046 (France), and 3.3805 links per business against 5.7543 incoming records per business in the test files. Where labels exist the same rule produced the held-out scores we quote; France, with no labels, uses the more conservative 0.82.

NUMBERS

- 0.5875 links per incoming record overall, so about 41 % unlinked (Doc B.1; percentage derived)
- 100,087 empty rows (Doc B.1)

Q113. Why are the decision models trained on folds 6-9, and how do fit, early stopping and refit work? ★★ **LIKELY**

Why they ask: They test the training protocol for leakage and how the number of trees was set.

SAY The ranker and readers learn on folds 0 to 5, so the decision models train on folds 6 to 9, where those scores are out of sample, just as on test. For the US and India each seed is fitted on folds 6 and 7 with early stopping on fold 8, then refitted on folds 6 to 9 with the best iteration scaled by the row count, since no fold is left for stopping. On the public board that refit moved 0.990621 to 0.990676; the same refit for France scored lower, so France keeps the folds 6-7 fit.

IF THEY PUSH The fit uses early stopping with patience 100 and at most 3,000 rounds (1,500 for France); the refit uses best iteration $\times$ rows(6-9) / rows(6-7) rounds, a heuristic based on the idea that more data supports more trees at the same learning rate. The France refit scored 0.990726 against 0.990748 without it.

NUMBERS

- US/India refit 0.990621 → 0.990676 kept; France refit 0.990726 vs 0.990748 dropped (Doc B.6)

Q114. What data does each country's decision model learn from? ★★

Why they ask: They want to see how you shared data across countries and handled the unlabelled one.

SAY The rows are the training shortlists of incoming records with a candidate in business folds 6 to 9, labelled "candidate is the owner". The US model learns from US rows, the India model from US and Indian rows, and the France model also from US and Indian rows, since France has no labels. The code path is identical; a country profile only switches the model, the threshold and the refinements.

IF THEY PUSH The training shortlists are built from the unpruned index output. Sharing happens where it helps, in the India and France models; the price for France is that its model never sees a French label, which is why its operating point is more conservative. For the US and India the rows also include extra hard examples (Q115); the France model uses none.

Q115. How do the decision models learn to reject near-identical look-alike businesses? ★★★

Why they ask: Look-alike businesses are a main source of wrong links, so they test your defence. Answer only if asked; never volunteer it.

SAY Mainly through the "what changed" features and the lead rule. In addition, for the US and India only, the decision models were trained with extra hard examples: copies of unlinked training records that sit right next to a candidate business, with small name edits. How many to add was set from label-free counts on the test records, never from labels, and our documentation discloses it; the France decision model uses no such extra examples.

IF THEY PUSH It is standard hard-example augmentation: more training examples of the confusable case near the decision boundary, where an ordinary sample has few. The safeguard is that the amount was set without any labels, so no test answer enters training.

IF THEY PUSH FURTHER Synthetic training pairs were allowed by the competition rules. Every held-out decision-model F0.5 we quote is computed on real labelled records only, never on the added examples, so the added rows can change what the model learns but cannot flatter that evaluation.

Q116. Why are the second-chance searches and the consistency check separate steps rather than part of one decision model? ★★★

Why they ask: Post-model steps can look like patches; they test whether you see the cleaner design.

SAY Because they act on different candidates at different moments: the second-chance searches retrieve businesses that were never in the shortlist, and the consistency check needs the main decision's accepted links as its input. Keeping them as separate steps let us choose each threshold on held-out folds and count exactly which links each one changes. Folding them into one decision model is the cleaner design and one of the first things we would do next.

IF THEY PUSH Each step is limited by construction: the second chances act only on unlinked records, with thresholds chosen with false positives counted twice, and the consistency check moves a few hundred links per country. The cost of separation is that our held-out table excludes the second-chance searches, which is one of our disclosed limitations.

Q117. What happens when the true business is not in the shortlist at all? ★★★

Why they ask: They check that you understand how blocking errors and the decision rule interact.

SAY The decision model can only choose among the shortlist, so the best outcome is to link nothing, and the threshold and lead make that likely when every candidate is poor. In the India example all three shortlisted businesses scored below 0.00001, so the record stayed unlinked and went to the native-script second-chance search, which linked it to a same-name business with the same house number. When we tuned 0.75, true records that blocking never retrieved counted as misses, so that loss was priced in.

IF THEY PUSH This is why the second-chance searches add new candidates instead of lowering the threshold: a lower bar cannot help when the owner is absent, and it would add wrong links elsewhere.

NUMBERS

- India example: 1.9e-6, 6.9e-6, 7.2e-6 (Doc D.2)

See also, answered elsewhere: "How did you choose the 0.75 threshold?" see [Q21](#); "Why five seeds?" see [Q146](#); "What are the held-out scores of the decision models?" see [Q149](#); "What happens in a country the system has never seen?" see [Q162](#).

10. Second-chance searches and the consistency check (10 questions)

Q118. What are the second-chance searches, and why do you need them? ★ LIKELY

Why they ask: They test whether you diagnosed the type of error before adding a fix.

SAY Some misses are retrieval failures: the right business never entered the shortlist, so no threshold can fix them. Two targeted searches revisit records still unlinked after the main decision: a native-script search for Indian names, which added 9,759 links, and an address search for the US and India, which added 3,137. Both touch only unlinked records, so they never undo a link.

IF THEY PUSH Each targets a miss type we found on training data: native-script names whose transliteration the Latin index does not retrieve, and records whose name differs while the address matches. Their candidates join the candidate file, which grows from the 11,687,317-pair shortlist to 12,293,019 pairs: 578,317 native-script pairs plus 27,385 address pairs (derived).

IF THEY PUSH FURTHER The native-script search scored 649,965 pairs for 171,713 records, 578,317 of them not already shortlisted; the address search retrieves businesses sharing one of the record's 3 rarest digit runs and one of its 3 rarest alphabetic address words (3,029,652 US and 4,766,484 Indian pairs, scored by a filter), and only 1,827 US and 25,558 Indian new candidates survive. The file is then $11,687,317 + 578,317 + 25,558 + 1,827 = 12,293,019$ pairs.

NUMBERS

- +9,759 native-script (India); +192 (US) and +2,945 (India) address links (Doc B.1)
- 578,317 native-script and $1,827 + 25,558$ address candidates not already in the shortlist (Doc 3)
- Second-chance counts (Doc 3); sum check (derived)

Q119. How does the native-script second-chance search work? ★★ LIKELY

Why they ask: They want the mechanism end to end, on a concrete example.

SAY For an Indian record with a native-script name left unlinked, a word list learned from labelled training matches turns the name into Latin spellings, without legal words, and we look for businesses with that core name in the same state. A fixed address score keeps the top five, and a LightGBM model links the best one only if it is confident and clearly ahead. In our example the Devanagari name became "vijay enterprises", ten such businesses in Maharashtra were found, and the one with house number 24/11 scored 0.99994.

IF THEY PUSH Keys are legal-word-free token tuples of up to 64 transliterations, from alternatives with at least 5 % of a native token's count; if the state is unknown or has no match, all states are searched. The fixed score weighs same state, first house number found, same digit string of an address segment, share of digit runs found, house-segment similarity, share of place words and address token-set ratio (13.0 for the winner, 4.78 for the next); the model must score at least 0.88, chosen on fold 8 with false positives counted twice, and lead by 0.3.

NUMBERS

- 649,965 pairs scored for 171,713 records; 578,317 not in the shortlist; +9,759 links (Doc 3, B.1)
- Example: 0.99994 against 0.0000191 for the second best (Doc D.2)

Q120. Why did the main pass miss that Devanagari record in the first place? ★★

Why they ask: They check that you understand failure modes, not only fixes.

SAY The main pass did find businesses with the right name, three "Vijay Enterprises Private Limited", but in Agra, Satara and Byculla, and nothing tied them to this record's address, so all three scored far below the bar. The business the second search finally linked, in Ghatkopar, Mumbai, was not among the shortlisted candidates. The second search narrows by transliterated name and state, and the house number 24/11, which none of the other rescue candidates has, picks out that business.

IF THEY PUSH Common names have many same-name businesses across a country, and with only a house number and a city in the address, the main ranking had little to separate them. Among the five rescue candidates, all in Mumbai, the address token-set ratio was 100 for the winner against 66.7 for each of the other four. Test labels are private, so the example shows how the search decides, not that the link is correct.

NUMBERS

- Main-pass scores 1.9e-6, 6.9e-6, 7.2e-6; token-set ratio 100 vs 66.7 (Doc D.2)

Q121. Why do you cross-fit the transliteration word list when training the rescue model? ★★★

Why they ask: A subtle leakage question that only someone who built it carefully can answer.

SAY The rescue model is trained on India rescue candidates from train folds 0 to 7, and those candidates are retrieved with word lists that leave out the record's own fold. Otherwise a training record's own name would have taught the word list its transliteration, so retrieval on training data would look easier than on test, where names are new. Cross-fitting makes training candidates look like test candidates.

IF THEY PUSH It is the same principle as the layer-wise folds for the decision model: any component learned from labels must be applied out of sample when it generates training data for a downstream model. Without it, the rescue model would learn from too-easy candidate lists and be over-confident at test time.

Q122. How does the address second-chance search work? ★★

Why they ask: They test the design of a small cascade inside the big one.

SAY For records still unlinked after everything else, we look for businesses that share one of the record's three rarest numbers and one of its three rarest address words, keep the top three not already in the shortlist, and pass them through two LightGBM models. A filter keeps at most two per record, and a scorer links the best one only if it clears that country's threshold and lead over the second. It added 192 links in the US and 2,945 in India.

IF THEY PUSH Retrieval is capped at 300 businesses per record, ranked by IDF containment + 0.5 × coverage; the filter has 10 features and its cut-off keeps 98 % of the fold-8 reference links, and the scorer has 41 features averaged over three seeds. US: 3,029,652 retrieved pairs → 1,827 filter survivors → +192 links; India: 4,766,484 → 25,579 → +2,945.

NUMBERS

- Filter cut-off / threshold / lead: US 0.06 / 0.86 / 0.0; India 0.015 / 0.66 / 0.1 (Doc 4)

Q123. Why count false positives twice when choosing the second-chance thresholds? ★★

Why they ask: They test cost-sensitive thinking that is tied to the metric.

SAY Under F0.5 a wrong link costs more than a missed one, and second-chance links are made on records the main model already rejected, a riskier population. So on fold 8 we chose their thresholds with each false positive counted as two, which pushes them up: 0.88 for the native-script search and 0.86 for the US address search.

IF THEY PUSH In the form $1.25 \cdot TP / (1.25 \cdot TP + 0.25 \cdot FN + FP)$, a false positive weighs four times a false negative in the denominator, but its exact cost per business depends on how many true records the business has; counting false positives twice is a simple, conservative proxy for that.

Q124. Why are the US and India address-search parameters so different? ★★★

Why they ask: Different numbers for the same mechanism invite a "did you overfit?" challenge.

SAY Each country's filter and scorer were trained separately, on that country's unlinked training records from folds 6 and 7, and each threshold was chosen on that country's fold 8. Scores of two separately trained models are not on a common scale, so India's lower bar, 0.66 against 0.86, does not by itself mean a looser rule. The populations also differ a lot, as the filter survivors show.

IF THEY PUSH India's filter keeps 25,579 pairs against 1,827 in the US, and each operating point is what that country's fold 8 preferred with false positives doubled. Like everything in the second chances, it is outside our held-out table, which is one of our disclosed limitations.

Q125. What is the per-business consistency check? ★★ LIKELY

Why they ask: It is an unusual step; they want to know what it does and whether it is principled.

SAY After the main decision in the US and India, we re-decide each record with a bar that depends on how many other incoming records the decision model already accepted for the same business: the usual 0.75 with none, a slightly lower bar with one or two, and a slightly higher one with three or more, always with the same lead. The bars were chosen on folds 8 and 9, and the step moves only a few hundred links in each direction per country.

IF THEY PUSH The bars are 0.75, 0.68, 0.72 and 0.77 for zero, one, two and three or more other accepted records, with the 0.4 lead. Final run: US +277 / -284, India +247 / -248, with one addition skipped because that record already had a native-script link, since the check never adds a link to such a record. The count of other accepted records is computed once from the main decision, with no iteration.

NUMBERS

- Bars 0.75 / 0.68 / 0.72 / 0.77 for 0 / 1 / 2 / 3+ others (Doc 4)
- US +277 / -284; India +247 / -248, 1 addition skipped (Doc B.1)

Q126. Why does the bar go down and then up again? ★★★

Why they ask: They test whether you understand the metric well enough to derive the shape.

SAY It follows the per-business F0.5 value of one extra link. With no other link, a wrong link could land on a business with no true records and turn its perfect empty score into zero, so the bar stays high. Next to one or two accepted links the business very likely does have true records, so an extra link pays off at lower confidence and the bar drops; next to many accepted links the business is likely complete, an extra link adds little and risks over-linking, so the bar rises.

IF THEY PUSH Illustration for a business with 3 true records and 1 correct link already: a correct second link raises its F0.5 from 0.71 to 0.91 and a wrong one lowers it to 0.45, so the extra link pays off above about 57 % confidence, below the standard 0.75. With all 3 already linked, any extra link is wrong and drops the score from 1.0 to 0.79; the actual bars were chosen empirically on folds 8 and 9.

Q127. Isn't the consistency check a hack? Does it create order effects or feedback loops? ★★★

Why they ask: Post-model rules invite scepticism; they test whether it is principled, stable and honestly evaluated.

SAY It is a deliberate use of the metric's per-business structure, and it is built to be stable: the count of other accepted records comes from the main decision once, with no iteration, so there are no feedback loops or order effects, and it uses no labels at inference. Its effect is small, a few hundred links per country. The honest caveat is that folds 8 and 9 chose its bars and also feed our held-out table, so that table is not an untouched test of it.

IF THEY PUSH It also respects the second chances: it never adds a link for a record that already has a native-script link. An alternative design is to give this count to the decision model as a feature and learn the effect, instead of a separate re-thresholding step.

11. France with zero labels (11 questions)

Q128. France has no labels. How did you build it at all? ★ LIKELY

Why they ask: This is the hardest part of the problem; they want the overall strategy in one breath.

SAY Four pieces: French cleaning rules for addresses and legal forms; the decision model trained on labelled US and Indian rows, used at a stricter 0.82; two French readers self-trained on our own most confident French predictions, mixed with labelled US and Indian pairs; and rules computed by code that veto risky links and rescue safe ones. The index, quick filter, ranker and shortlist are the same as in the other countries.

IF THEY PUSH The France decision model has 158 features, is the mean of five seeds and is trained on labelled US and Indian rows; it accepts 887,630 French links and the rules leave 867,559. The plain E5 reader is not used for France; its slot is a blend of the MiniLM, French MiniLM and French E5 readers.

NUMBERS

- 887,630 accepted → 867,559 final French links (Doc B.3, B.1)

Q129. How do you know self-training actually helped France? ★★★ LIKELY

Why they ask: Pseudo-labelling can reinforce its own mistakes; they want your evidence and its limits.

SAY Honestly, we have partial evidence, not proof, because there are no French labels. The French E5 reader agreed with held-out French pseudo-labels at 0.999206, but that measures consistency with its teachers, not truth; on a generated labelled French test set it was slightly below its base model on its own, while the blend containing it scored higher than the blend without it. The file with it had our best public score, but that upload also added the five-seed models, so its effect is not isolated.

IF THEY PUSH Generated set: best F0.5 0.910516 for the French E5 reader against 0.914964 for the E5 reader; the 50/50 blend 0.90843 against 0.898031. We limited confirmation bias with strict confidence cut-offs, a different student from the teachers, and replay; a second self-training round scored lower on the public board (0.991277 against 0.991300) and was not adopted.

NUMBERS

- Agreement 0.999206 (E5 reader 0.981304) on 25,192 held-out pairs (Doc B.4)
- Generated set, 195,380 pairs: 0.910516 vs 0.914964; blends 0.90843 vs 0.898031 (Doc B.4)

Q130. How did you choose the pseudo-label negatives? Why "the hardest confident non-match"? ★★

Why they ask: Negative selection decides what a self-trained model learns; they test whether you thought about it.

SAY For the French E5 reader we took, per record, the highest-confidence pair that was still at or below 0.03: the candidate the readers found most plausible among those they were sure were wrong. Easy negatives teach little; the hardest confident one sits closest to the decision boundary while still being a confident label. We took at most one positive and one negative per record and balanced the classes.

IF THEY PUSH One pair per record per class stops records with many candidates from dominating, and 5 % of French records were held out entirely to check agreement. The French MiniLM reader was trained differently: on very confident pseudo-labels from an earlier run (best candidate, score at least 0.97, clear lead) plus 500,000 labelled US and Indian pairs.

Q131. How did you set the French threshold of 0.82 without labels? ★★ LIKELY

Why they ask: An operating point without labels is a hard problem; they want the method and its weakness.

SAY Without French labels we cannot tune on French F0.5, so we set a more conservative point without labels: from the number of French links per business, together with a threshold sweep on US and Indian fold 8. It is stricter than the 0.75 we use where labels exist, because the France decision model is applied outside the data it was trained on.

IF THEY PUSH For context, France ends with 3.3438 links per business against 3.3935 (US) and 3.3817 (India), from 5.531 incoming records per business in the test files. The held-out proxy of the France recipe at this operating point is 0.990508 (US) and 0.989467 (India) on fold 9; with a small labelled French sample, this threshold is the first thing we would re-tune.

IF THEY PUSH FURTHER Mean links per business: France 3.3438, US 3.3935, India 3.3817. The France recipe at 0.82 on labelled US and Indian rows scores 0.990508 and 0.989467 on fold 9. Per incoming record, France links slightly more often (0.6046 against 0.5895 and 0.5806) because it has fewer incoming records per business (5.531 against 5.756 and 5.824); without labels we cannot tell whether that means more owned records or extra wrong links. Link volume is only a sanity check: it says nothing about whether the links are right.

NUMBERS

- Threshold 0.82, set label-free (Doc 4, C.1)
- Links per business: France 3.3438, US 3.3935, India 3.3817 (Doc B.1)
- Links per business, mean: US 3.3935, India 3.3817, France 3.3438; links per incoming record 0.5895 / 0.5806 / 0.6046 (Doc B.1)
- Incoming records per business: US 5.756, India 5.824, France 5.531 (Doc 2.1)

Q132. The France decision model was trained with the MiniLM reader's scores, but at test time it gets a blend of three readers. Isn't that a mismatch? ★★★ LIKELY

Why they ask: A sharp reviewer spots train-test inconsistencies; they test whether you know it and own it.

SAY Yes, it is a deliberate plug-in substitution, and the input distribution does change. All three readers are trained with the same binary cross-entropy objective on the same kind of input, so each outputs log-odds of a match, and our assumption is that the slot keeps roughly its meaning; for the accepted pair in our French example the three gave 7.5, 8.9 and 10.8. But the decision model never saw the blend in training, and without French labels we cannot measure the effect directly.

IF THEY PUSH Adding the French E5 reader to the blend was compared without labels against the same pipeline without it: only French links changed (−2,228 / +3,396), with the US, India and the candidates identical; that measures how much the decisions move, not whether they are right. Retraining the France decision model on French-reader logits computed for US and Indian rows is the alternative we would test next.

NUMBERS

- Accepted pair logits 7.5000 / 8.9375 / 10.8125 (Doc D.3)
- French links −2,228 / +3,396, 5,437 business rows changed (Doc B.4)

Q133. Where do the blend weights 0.35, 0.15 and 0.5 come from? ★★★

Why they ask: Magic numbers invite scrutiny; they test whether the weights were principled or tuned on the test set.

SAY The earlier French blend was 0.7 MiniLM plus 0.3 French MiniLM, with the French MiniLM weight set by label-free counts on the test records. When we added the French E5 reader, we gave it half the slot and kept the old blend for the other half: half of 0.7 is 0.35, half of 0.3 is 0.15, plus 0.5 for the French E5 reader. No weight was fitted on French labels, because there are none; the half-and-half split is the blend our generated labelled French test set evaluated.

IF THEY PUSH On the generated labelled French test set, the 50/50 blend scored best pair-level F0.5 0.90843 against 0.898031 for the blend without the French E5 reader, although that reader alone was slightly below its base model, a typical ensemble effect. The blend is in logit space, so it averages log-odds evidence.

NUMBERS

- Blend $0.35 / 0.15 / 0.5 = 0.5 \times (0.7 / 0.3) + 0.5$ French E5 (Doc 4, B.4)
- Check: $0.35 \times 7.5 + 0.15 \times 8.9375 + 0.5 \times 10.8125 = 9.371875$ (Doc D.3, derived)

Q134. What do the France rules do, in plain words? ★ LIKELY

Why they ask: They want to know what was hand-built and how big its effect is.

SAY They are small, checkable conditions computed by code on decision-model scores, raw text and the address, applied after the decision model to France shortlist pairs only; they add no candidates. Vetoes drop risky accepted links, for example the same address but one name word swapped for a common or category word, or the business's street name missing from the record's address. Rescues add links the decision model left below the bar when the name difference looks like those of true records, such as the business name plus "groupe", or an acronym of it, at the same address.

IF THEY PUSH In order: the category-swap and word-substitution vetoes together remove 29,021 links; the word-substitution rescue adds 5,272 and the residual rescue 5,846; the street veto removes 826 and the generic-word swap veto 538; the generic-word rescue adds 332; the cell veto removes 1,322; the acronym rescue adds 186.

NUMBERS

- 887,630 → 869,727 → 868,901 → 868,363 → 868,695 → 867,373 → 867,559 French links (Doc B.3)

Q135. Without labels, how did you check that a rule helps rather than hurts? ★★★

Why they ask: They want to know whether there was evidence beyond "it looked right".

SAY With label-free evidence. We used control groups on the French records, for example comparing a name pattern at the same house number with a control group of same-street records without a house number; for the cell veto we relied on the independence of name noise and address noise, measured on US labels. None of this is a French F0.5, and we say so.

IF THEY PUSH The street veto rests on a fact measured on US labels: among 1,313,608 decidable true US top-1 matches only 48 have a street name sharing no word with the business's street, while 60,565 of 60,613 US top-1 pairs with a genuinely different street are wrong. The French street veto allows typos and leaves undecidable pairs alone; it removed 826 links.

NUMBERS

- 48 of 1,313,608; 60,565 of 60,613 (Code: france_rules/street_veto.py)
- Street veto -826 (Doc B.3)

Q136. Walk us through the French example. Why veto a link the model was 99.98 % sure of?

★★ LIKELY

Why they ask: They test whether a veto against a confident model is principled.

SAY The record is "TQ COMITE SARL" at 2 R Suzanne Lenglen, Nantes. One candidate has exactly that name but sits in Pessac, and the French readers rejected it; the other, TQ Parents SARL at exactly the same address, got 0.999837 from the decision model. But at the same address the common word "comite" has replaced "parents", and the category-swap veto drops exactly that pattern, because under F0.5 abstaining is cheaper than a wrong link.

IF THEY PUSH No later rule re-links the record: its best score is above the rescue band of 0.4 to 0.82, and the later rescues skip records vetoed by the word-swap vetoes; TQ Parents SARL keeps its link to another incoming record. Test labels are private, so this shows how we decide, not that the call was right.

NUMBERS

- 16 candidates from the index, 2 after the filter; decision scores 0.999837 and 0.000098; lead 0.999740 (Doc D.3)

Q137. How much does France matter to your score, and how confident are you in it? ★★

Why they ask: They want to see that you can size a risk.

SAY It matters a lot: a diagnostic upload that emptied every French row of an early baseline dropped the public score from 0.970125 to 0.839. It is also our least certain part: no labels, no measured French F0.5, and rule parameters chosen partly after seeing public scores. If the private leaderboard moves, France is where we would expect it.

IF THEY PUSH France has 259,452 of 1,732,544 businesses, about 15 % (derived), and 1,434,993 incoming records. The closest proxy is the France recipe applied to US and Indian rows at the French operating point, 0.990508 and 0.989467 on fold 9: a proxy for the recipe, not a French score.

NUMBERS

- 259,452 businesses, 1,434,993 incoming records (Doc 2.1); about 15 % (derived)
- Upload 4 0.970125 → upload 5 (French rows emptied) 0.839 (Doc B.6)

Q138. Who chose the France rules: the team or an AI assistant? ★★★

Why they ask: Your documentation's disclosures mention AI assistance; they check that you answer plainly.

SAY Both, and our documentation says so. AI coding assistants wrote and ran code and analysis scripts during development; for the three later rule changes, the cell-veto cells, the acronym rescue and the generic-word rescue's lower bound, they also chose the cells and thresholds and wrote the rule code, which the team adopted. At inference the rules are deterministic code on scores and text, and no human or AI judgement of any pair is an input.

IF THEY PUSH The review behind the rules is disclosed too: about 1,000 pairs in five review samples plus targeted checks, and for the three later changes about 470 pairs printed by analysis scripts and about 100 more by review scripts. No LLM runs in the pipeline.

NUMBERS

- Review sizes: about 1,000; about 470 + about 100 pairs (Doc C.2, C.5)

See also, answered elsewhere: "What is self-training with pseudo-labels, and how exactly did you use it?" see [Q143](#); "Why mix in labelled US and Indian pairs?" see [Q144](#); "Why did you keep the French E5 reader when it failed two of its written checks?" see [Q183](#); "Weren't the France rules fitted to the test set?" see [Q182](#); "What did not work in France?" see [Q189](#).

12. Training strategy (10 questions)

Q139. Which data trained which model? ★ LIKELY

Why they ask: Before trusting any score, a scientist checks exactly what each model learned from; it is the first test for leakage.

SAY We split the businesses into ten folds and gave each layer of the pipeline its own folds. The CatBoost ranker and the readers learn on folds zero to five. The LightGBM decision models use folds six to nine: they fit on six and seven, stop early on eight, and the US and India models are then refitted on all four. The two second-chance models have their own training sets, and the French readers learn by self-training on unlabelled French records, mixed with labelled US and Indian pairs.

IF THEY PUSH The full map: the Indic-to-Latin cleaning word list is learned on folds 0-5; the ranker on a 140,000-record pilot, of which the fold 0-5 part is fitted, with early stopping on fold 6; the MiniLM and E5 readers on folds 0-5 for one epoch; the US and India decision models are fitted on 6-7, early-stopped on 8, then refitted on 6-9; the France and default decision models are fitted on 6-7 and early-stopped on 8, with no refit; the native-script model on folds 0-7; the address second-chance models on folds 6-7. The 0.75 threshold was chosen on fold 8, the consistency-check thresholds on folds 8 and 9.

NUMBERS

- 10 folds, assigned per business: $\text{hash}(\text{business id, seed 20260927}) \bmod 10$ (Doc 4)
- Ranker pilot: 140,000 incoming records, 35,000 per country and source (Doc 4)
- Decision models: at most 3,000 rounds (US, India), at most 1,500 (France, default) (Doc 4)

Q140. Why do the ranker and readers train on different folds from the decision models? ★★

LIKELY

Why they ask: The classic stacking trap: a model that sits on top of other models must never see their scores on records those models trained on.

SAY Because the decision model uses the ranker probability and the reader scores as inputs. If those models had trained on the same records, their scores there would be over-confident, and the decision model would learn to trust them more than it should on new data. So the ranker and readers learn on folds zero to five, and the decision models only ever see their scores on folds six to nine, which those models never trained on.

IF THEY PUSH This is stacked generalisation with a layer-wise split rather than out-of-fold predictions. Out-of-fold scoring would let the decision model use all ten folds, but it needs several copies of every reader and of the ranker; the layer split costs labelled data instead of compute. One precise caveat: the ranker used fold 6 for early stopping, so fold 6 is not perfectly untouched for it, although that choice sets only the number of trees. A useful side effect: the readers' quality on folds 8 and 9 is an honest held-out number, for example an AUC of 0.998570 for the E5 reader.

NUMBERS

- E5 reader on 509,260 pairs of 437,715 labelled fold-8/9 records: AUC 0.998570, top-1 accuracy 0.993004 (Doc B.4)

Q141. Why assign folds by business rather than by record? ★ LIKELY

Why they ask: Grouped splitting is basic hygiene for entity resolution; a random record split is the most common way teams inflate their scores.

SAY Because the business is the unit of the metric and of several features. A business owns several incoming records, so a record-level split would put records of the same business on both sides, and we would be testing on businesses the model had partly seen. We hash each business id into one of ten folds, and every record travels with its business.

IF THEY PUSH An incoming record that belongs to no business hashes its own id. The collective features and the per-business consistency check look at all records competing for one business, so a record-level split would leak through them as well. The held-out protocol is stricter still: when we score a fold, the fits also drop every incoming record that has any candidate in that fold.

NUMBERS

- Fold seed 20260927 (Doc 4)

Q142. Why is your CatBoost ranker a classifier and not a learning-to-rank model? ★★

Why they ask: "Ranker" invites the question of which loss you used; they check you know the difference between pointwise and pairwise or listwise ranking.

SAY It is a pointwise binary classifier with log loss: it predicts the probability that a candidate is the owner, and we sort by that probability. We need the probability on an absolute scale, not only an order, because the shortlist keeps ranks two to four only above a probability floor, and the decision model uses the probability as an input. We have no comparison with a ranking loss such as LambdaMART to report, so we make no claim either way.

IF THEY PUSH Pairwise and listwise losses optimise the order inside each record, and their scores are not comparable across records, so a fixed floor such as 0.001 would lose its meaning. Settings: up to 1,600 trees, depth 7, learning rate 0.075, L2 5, early stopping with a patience of 180 rounds on fold 6, on GPU, using the 76 index-lookup features, trained on the 140,000-record pilot.

IF THEY PUSH FURTHER The ranker's job is recall at a cut of four, not fine ordering at the top; the decision model reorders anyway, and in the US example it moved the ranker's second candidate, the LLC, to last. We did not benchmark a ranking loss.

NUMBERS

- Shortlist: top candidate always, ranks 2-4 if probability ≥ 0.001 (Doc 3)
- CatBoost: Logloss, $\leq 1,600$ trees, depth 7, learning rate 0.075, L2 5, early stopping 180 (Doc 4)

Q143. How did you train French readers with no French labels? ★★ LIKELY

Why they ask: France is the unsupervised part of the problem; they want to hear a concrete self-training recipe, not "we fine-tuned on French data".

SAY By self-training. We took French pairs on which our earlier models were very confident, matches at least 97 % sure with a clear lead and clear non-matches, treated them as labels, and fine-tuned a French MiniLM reader and a French E5 reader on them. We mixed in about as many labelled US and Indian pairs, so the readers would not forget what they had learned from real labels.

IF THEY PUSH French E5 reader: confidence p = the mean of the sigmoids of the MiniLM and French MiniLM logits on the France shortlist; positive = the record's best pair with p at least 0.97 and every other pair below 0.5; negative = the record's highest- p pair that is still at most 0.03, so it is the hardest pair the teachers still call a confident non-match; at most 250,000 per class; 5 % of records held out for checking; 474,808 French pairs plus 474,808 labelled pairs; 7,419 steps at learning rate $2e-5$. French MiniLM reader: very confident pseudo-labels from an earlier run (best candidate, score at least 0.97, clear lead) plus 500,000 labelled US and Indian pairs.

NUMBERS

- French E5 reader: 474,808 French + 474,808 US/India pairs (Doc 4)
- French MiniLM reader: pseudo-labels + 500,000 US/India pairs (Doc 4)

Q144. What is replay, and why mix labelled US and Indian pairs into French training? ★★

Why they ask: Tests whether you know catastrophic forgetting and how to guard against it during domain adaptation.

SAY Replay means continuing to train on labelled examples from the original task while adapting to a new one. Fine-tuning only on French pseudo-labels could make a reader drift and forget what it learned from real labels, which is called catastrophic forgetting, and the pseudo-label noise would have nothing to anchor against. With replay, the French E5 reader's quality on held-out US and Indian pairs stayed within our written tolerance.

IF THEY PUSH French E5 reader against the E5 reader on held-out US/India pairs: AUC 0.998544 vs 0.998570, top-1 accuracy 0.993161 vs 0.993004, best F0.5 0.992782 vs 0.992796; tolerance 0.001 on AUC and 0.002 on the other two. We have no run without replay to compare, so this shows that the reader did not forget, not how much of that is due to replay alone. The mix is exactly one to one only for the French E5 reader; the French MiniLM reader used its French pseudo-labels plus 500,000 labelled pairs, so "about one to one".

NUMBERS

- US/India quality check: AUC 0.998544 vs 0.998570; tolerance AUC 0.001, top-1 and best F0.5 0.002; pass (Doc B.4)

Q145. Why is the France decision model trained only on US and Indian rows? ★★

Why they ask: They test whether you understand transfer to an unlabelled domain and the risk of feeding a model inputs it never saw in training.

SAY Because there are no French rows with a known answer. The France decision model learns the general question, is this candidate the owner given these differences and scores, from labelled US and Indian rows, using 158 features that exist in every country. At test time its reader input receives the French reader blend, and we use a stricter threshold of 0.82 because nothing French confirms it.

IF THEY PUSH It was trained with the MiniLM reader's logit in its reader slot; at test that slot receives 0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5 logits. The weights sum to one, so the input is still a logit-scale blend, but its distribution on French pairs is not the one the model was trained on, and we have no labels to recalibrate it; the conservative, label-free threshold reflects that uncertainty. It is the mean of 5 seeds, fitted on folds 6-7 with early stopping on 8, with no refit.

NUMBERS

- France: 158 features, threshold 0.82, lead 0.4 (Doc 4)
- Reader blend 0.35 / 0.15 / 0.5 (Doc 4)

Q146. Why average five seeds? ★★

Why they ask: Seed averaging is cheap and common; they test whether you know what it does (variance, not bias) and whether you measured it.

SAY Our LightGBM settings sample 80 % of rows every iteration and 90 % of features, so single fits vary; averaging five seeds reduces that variance at the price of five small tree-model calls per pair. Honestly, its separate effect is not isolated: the held-out table uses three of the five seeds, and the five-seed models entered the final file in the same upload as the French E5 reader.

IF THEY PUSH Averaging the probabilities of fits that differ only in their random seed, and so in their row and feature samples and their early-stopping point, is a bagging-style variance reduction; it does not reduce the bias. That upload moved the public score from 0.991114 to 0.991300, but we cannot split the gain between the seeds and the reader. The address second-chance scorer also averages three seeds.

IF THEY PUSH FURTHER The cost is a handful of small tree-model calls per pair, negligible next to the readers. Averaging probabilities keeps the score on the same 0-to-1 scale as the rule's threshold and lead.

NUMBERS

- Bagging fraction 0.8 every iteration, feature fraction 0.9 (Doc 4)
- Upload 15 → 16: 0.991114 → 0.991300 (Doc B.6)
- Public 0.991114 → 0.991300 for the upload that added both (Doc B.6)

Q147. How were the second-chance models trained so that they match how they are used?

★★★

Why they ask: A model trained on a different population from the one it scores is a classic silent bug; they want to see train-test consistency.

SAY Each one is trained on the kind of records it will see at test time. The address model is trained on address-retrieved pairs of training records that the main decision left unlinked, because at test it only ever sees unlinked records. The native-script model is trained with transliteration word lists rebuilt without the record's own fold, so a record never benefits from its own label when its candidates are retrieved.

IF THEY PUSH Without that cross-fitting, retrieval in training would be easier than at test and the model would over-trust it. Native-script model: India candidates of folds 0-7; link if the score is at least 0.88 and the lead at least 0.3, with 0.88 chosen on fold 8 counting false positives twice. Address models, folds 6-7: a filter with 10 features, then a scorer with 41 features averaged over 3 seeds; US threshold 0.86 with lead 0.0, India 0.66 with lead 0.1. We disclose that the training records of both were selected with earlier decision models that had inputs we later removed.

NUMBERS

- Native-script: ≥ 0.88 , lead ≥ 0.3 (Doc 4)
- Address: US 0.86 / 0.0; India 0.66 / 0.1 (Doc 4)

Q148. The decision models were trained on shortlists built without the quick filter, but at test the filter runs first. Isn't that a mismatch? ★★★

Why they ask: A sharp reviewer checks that the training rows come from the same pipeline as the test rows.

SAY Yes, a small one, and we measured its cost instead of assuming it away. The quick filter rarely removes the true business, and in our blocking study, with the decision model of that date, switching the filter on moved held-out fold-nine F0.5 only from 0.990117 to 0.990086.

IF THEY PUSH The owner survived the filter for 99.974 % of fold-8/9 records, and fold 8 changed by -0.00004 . The mismatch mainly touches the competition features (rank, lead, best other score), because the filter can remove a rival candidate; the measured F0.5 change is the net of that effect and the few owners the filter loses. Caveat: the measurement used the decision model of 25 September, not the final one.

NUMBERS

- Owner kept 99.974 %; fold 9 0.990117 $\rightarrow$ 0.990086; fold 8 -0.00004 (Doc 3)

See also, answered elsewhere: "How did you choose the number of trees when you refitted on folds six to nine?" see [Q113](#) and [Q243](#); "How did you choose the training pairs for the readers?" see [Q86](#); "Why only one epoch for the readers? Did they overfit or underfit?" see [Q85](#); "How did you handle class imbalance?" see [Q238](#); "Doesn't self-training just teach the reader its teachers' mistakes?" see [Q247](#); "How did the decision models learn to reject near-identical candidates? Did you add synthetic examples?" see [Q115](#).

13. Validation, model selection and the leaderboard (12 questions)

Q149. How did you validate your models? ★ LIKELY

Why they ask: The single most important credibility question: is your offline evidence the real metric, on data the model never saw?

SAY With the exact competition metric, per-business macro F0.5, on labelled folds the models did not train on. To score fold nine we refit the decision models on folds six to eight, leaving out every record that touches fold nine, and the same for fold eight. Close versions of our decision models score between 0.990 and 0.992 there, the same range as our public 0.9913.

IF THEY PUSH US rows 0.991212 (fold 9) and 0.991545 (fold 8); Indian rows 0.990046 and 0.990298. Both come from the India recipe with 3 of its 5 seeds (the final US model trains on US rows only and adds 6 features), with the 0.75 threshold and the consistency check. They exclude the second-chance searches and the France rules, and the number of trees in each fit follows the same row-scaling rule as the final refit.

NUMBERS

- Fold 9 / fold 8: US 0.991212 / 0.991545; India 0.990046 / 0.990298 (Doc 5)
- Scored businesses, fold 9: US 132,365, India 87,778 (Evidence)

Q150. What does "leaving out every record that touches the fold" mean, and why bother? ★★★

Why they ask: They probe whether your held-out protocol has a hidden leak through records whose candidates span several folds.

SAY Folds are assigned to businesses, but an incoming record's shortlist can contain businesses from several folds. If a record with one candidate in fold nine stayed in training, the model would have learned from that record's text and label while we score fold nine. So for each held-out fold we drop, from fitting, every incoming record with any candidate in that fold.

IF THEY PUSH This also closes leakage through the competition and collective features, which look across all candidates of a record and all records of a business. It is stricter than a grouped split by business alone.

Q151. Fold eight chose the number of trees and the threshold, and you also report scores on it. Isn't it contaminated? ★★★

Why they ask: An adversarial check of whether your "held-out" fold was really untouched.

SAY Partly, yes, and we disclose it. Fold eight set the early-stopping point and the 0.75 threshold, and folds eight and nine together set the consistency-check thresholds, so neither is a perfectly untouched test. Fold nine is the cleaner of the two for the main decision, and the two folds give scores in the same range.

IF THEY PUSH Our documentation says it directly: there is no untouched end-to-end validation of the final file. The parameters chosen on these folds are a handful of numbers against roughly 130,000 US and 88,000 Indian businesses per fold, which limits the optimism but does not remove it. Fold 8 scores slightly higher than fold 9 for both countries (US +0.000333, India +0.000252, derived), which fits a small tuning optimism, although ordinary fold-to-fold variation could explain it too. Next time we would hold out one fold that nothing touches until the very end.

NUMBERS

- Scored businesses: fold 9 US 132,365, India 87,778; fold 8 US 132,407, India 88,103 (Evidence)

Q152. Which decisions used held-out folds, and which used the public leaderboard? ★★ LIKELY

Why they ask: They want an honest map of where offline evidence ended and leaderboard feedback began.

SAY Held-out folds decided the quick filter, the 0.75 threshold, the consistency check and the second-chance thresholds, and the French E5 reader was measured against written checks. Public scores decided the US and India refit, dropping the France refit, a second French self-training round and a retrained French MiniLM reader, some France rule parameters, and the final file among our uploads.

IF THEY PUSH Label-free counts on the test records set the France threshold 0.82 and the French MiniLM reader's blend weight. Public scores also calibrated the label-free estimates for transferring the address search to the US and for the generic-word rescue, which then selected two further families of that rescue. Keeping the French E5 reader went against its own written rule.

NUMBERS

- Quick filter: fold 9 0.990117 → 0.990086 (Doc 3)
- US/India refit 0.990621 → 0.990676; France refit 0.990726 vs 0.990748; second French round 0.991277 vs 0.991300; French MiniLM retrain 0.991258 vs 0.991300 (Doc B.6)

Q153. How did you avoid overfitting the public leaderboard? ★★★ LIKELY

Why they ask: Many uploads plus selection by public score is a known way to overfit; they test your honesty first, then your mitigation.

SAY Not completely, and we say so. Our US and Indian thresholds came from held-out folds, not from the leaderboard, but we made 18 uploads, picked the final file among them by public score, and let public feedback shape some France rule parameters. What limits the damage is that held-out scores of close versions of our models sit in the same range, and the late public gains were small.

IF THEY PUSH The last step was 0.991114 → 0.991300, a gain of 0.000186 (derived). Held-out: US 0.991212 / 0.991545, India 0.990046 / 0.990298. We also dropped three changes on public scores (the France refit, a second French self-training round and a retrained French MiniLM reader); to be clear, that is more leaderboard selection, not a safeguard against it. What limits it is size: those three differences were 0.000022, 0.000023 and 0.000042 (derived), so little could be gained or lost by each choice.

NUMBERS

- 18 uploads; final = upload 16 (Doc B.6)

Q154. Will 0.9913 hold on the private leaderboard? ★★★ LIKELY

Why they ask: Tests whether you understand selection bias and can say where your risk sits.

SAY We can't promise it. There is some selection bias, because we chose the final file among our uploads using public scores, which pushes the public number up a little. Held-out scores of close versions of our models are in the same range and the late gains were small, which limits that; the part with the most risk is France, because it has no labels and its rules were tuned with public feedback.

IF THEY PUSH Held-out fold 9 / fold 8: US 0.991212 / 0.991545, India 0.990046 / 0.990298; they cover the main decision only. The French E5 reader, which failed two written checks, also sits in the French part.

NUMBERS

- Public 0.991300 (Doc 5)

Q155. You dropped the France refit because it scored 0.990726 against 0.990748. Isn't that just noise? ★★★

Why they ask: They test whether you can tell a real difference from leaderboard noise, and admit when you can't.

SAY It may well be; a difference of about two hundred-thousandths on one public number is weak evidence either way. With no French labels we had nothing better, so we kept the version that had scored higher and needs one step fewer, and we disclose that the choice used public scores.

IF THEY PUSH The same applies to the second French self-training round (0.991277 against 0.991300) and to the retrained French MiniLM reader (0.991258). Comparisons between uploads are paired on the same public businesses, so they are less noisy than absolute levels, but across 18 uploads small selections add up. The real fix is labelled French data, which we did not have.

NUMBERS

- $0.990748 - 0.990726 = 0.000022$ (derived)

Q156. What were your written acceptance checks? Were they fixed in advance? ★★

Why they ask: Pre-registered acceptance rules are a sign of scientific discipline; they test whether yours were real and whether you followed them.

SAY Before adopting a new reader we wrote down pass-or-fail checks, then looked at the results. The French E5 reader had four: quality on held-out US and Indian pairs within a tolerance, agreement with held-out French pseudo-labels, a generated labelled French test set, and a rule that its estimated gain on the final file must be safely positive. It passed the first two and failed the last two.

IF THEY PUSH The rules were written before the results, and the rule files in our evidence folder record every deviation, including this one. To be precise: for the three checks made at training time, the original declaration is not in our package (the tolerances of the first two are restated in a later rule file), which our documentation states; the gain rule's file is included.

NUMBERS

- Pass: AUC 0.998544 vs 0.998570; agreement 0.999206 (Doc B.4)
- Fail: 0.910516 vs 0.914964; pessimistic gain -0.0001806 (Doc B.4)

Q157. The second-chance searches aren't in your held-out table. How did you validate them? ★★★

Why they ask: They noticed a gap between what was measured and what was shipped and want to see if you noticed it too.

SAY Their thresholds were set on held-out fold eight, with every false positive counted twice, because under F0.5 a wrong link hurts more than a miss. Their effect on the final metric was only seen on the public board: adding the India address search moved it from 0.990676 to 0.990748. That is one reason we say we have no untouched end-to-end validation.

IF THEY PUSH Native-script: link if the score is at least 0.88 and the lead at least 0.3. Address: a filter whose cut-off is the largest grid value that keeps 98 % of the fold-8 reference links, then a scorer with US 0.86 / lead 0.0 and India 0.66 / lead 0.1. Both act only on unlinked records, so they cannot remove a correct link; only the precision of the links they add matters, and counting false positives twice is a simple cost-sensitive stand-in for F0.5's asymmetry.

NUMBERS

- Upload 9 → 10: 0.990676 → 0.990748 (Doc B.6)
- Links added: native-script +9,759; address +192 (US), +2,945 (India) (Doc B.1)

Q158. How did you validate France without labels? ★★ **LIKELY**

Why they ask: Every unlabelled-domain claim needs some evidence; they want to see what you checked and what you admit you could not.

SAY Only indirectly, and we have no measured French F0.5. The French E5 reader was checked against held-out French pseudo-labels and a generated labelled French test set, the France rules rest on label-free control groups on the French records, and as a proxy we ran the France recipe on labelled US and Indian rows at the French threshold. A diagnostic upload that emptied every French row dropped the public score from 0.970125 to 0.839, which shows how much France weighs.

IF THEY PUSH Pseudo-label agreement 0.999206 on 25,192 held-out pairs; generated French test set 0.910516 against 0.914964 (fail). Proxy: fold 9 0.990508 (US rows) and 0.989467 (Indian rows), fold 8 0.990977 and 0.989850; it is a proxy, not a French score. France holds 259,452 of the 1,732,544 test businesses, about 15 % (derived), which is why emptying its rows costs so much.

NUMBERS

- Diagnostic upload 5: 0.839 (from 0.970125, upload 4) (Doc B.6)
- France proxy (Doc 5)

Q159. How do you compare two models when they differ in the fifth decimal? ★★★

Why they ask: Statistical maturity: do you know that absolute scores on different samples vary more than paired differences?

SAY Paired, on the same held-out businesses, never by comparing absolute scores from different samples. Absolute held-out scores move by two to three ten-thousandths between fold eight and fold nine, so only a paired comparison can resolve smaller differences, and our evaluation code computes paired bootstrap intervals over businesses for that.

IF THEY PUSH Fold 8 minus fold 9: US +0.000333, India +0.000252 (derived). The bootstrap resamples businesses 2,000 times. The public board gives one number per upload, so we cannot resample it, which is another reason not to over-read small public differences.

NUMBERS

- US $0.991545 - 0.991212 = 0.000333$; India $0.990298 - 0.990046 = 0.000252$ (derived from Doc 5)

Q160. Why only two held-out folds instead of full ten-fold cross-validation? ★★

Why they ask: They test whether your evaluation design follows from your training design.

SAY Because of the layer split: folds zero to five are reserved for the ranker and readers, so the decision models can only be trained and tested within folds six to nine. Within those, six and seven are always in the fit, and we rotate eight and nine as the held-out fold, which gives two separate checks of every setting.

IF THEY PUSH Rotating six and seven into the held-out role as well would add two more checks, and we would do that with more time. Folds 0-5 cannot serve as held-out folds for the decision models without out-of-fold reader scores, which would need several copies of each reader.

See also, answered elsewhere: "Why tune thresholds on the exact metric instead of AUC or log loss?" see [Q21](#); "How did you choose the 0.75 threshold and the 0.4 lead?" see [Q21](#) and [Q111](#); "How did you set the French threshold of 0.82 without labels?" see [Q131](#).

14. Edge cases (11 questions)

Q161. How do you handle a business that owns many incoming records? ★★

Why they ask: Many-to-one linking is what makes this different from one-to-one matching; they test whether your decisions are coordinated per business.

SAY Each record is decided on its own, and then the per-business consistency check adjusts the bar using how many other records were already accepted for the same business. For the US and India, records competing for the same business also feed collective features. In our file a business has about 3.4 links on average and at most 11.

IF THEY PUSH The bar is 0.75, 0.68, 0.72 or 0.77 for 0, 1, 2 or 3+ other accepted records, with the same 0.4 lead, chosen on folds 8 and 9. The shape matches the metric: if a business already has k correct links and one more true record, one extra link pays off only if its chance of being right exceeds $k / (k + 0.25 \cdot (k + 1))$, which is 0.67, 0.73 and 0.75 for $k = 1, 2, 3$ and approaches 0.8 for large k (derived from the F0.5 formula, treating the score as a calibrated probability; the bars themselves were tuned on folds 8 and 9). With no other links the bar stays at 0.75, because there a wrong link can ruin an empty row. Net effect: US +277 / -284, India +247 / -248. No record is ever linked to two businesses.

NUMBERS

- Consistency check 0.75 / 0.68 / 0.72 / 0.77 (Doc 4); effect (Doc B.1)
- Links per business: mean 3.3805, median 3, max 11 (Doc B.1)

Q162. What do you do with a country you have never seen? ★ LIKELY

Why they ask: Generalisation beyond the test countries is part of the brief; they want a concrete fallback and honesty about testing.

SAY It takes a default route: the US and India cleaning rules, the multilingual MiniLM reader only, a decision model with 158 features that exist in every country, trained on labelled US and Indian rows, and the stricter 0.82 threshold, with no second-chance searches or rules. Honestly, it is untested on real data, because the test set had only the US, India and France.

IF THEY PUSH The index and its rarity weights are built per country from that country's own businesses, so retrieval needs no labels. The proxy row in our held-out table is this same decision-model recipe as a single fit, run on labelled US and Indian rows at 0.82: 0.990508 and 0.989467 on fold 9; but those are countries it was trained on, not a new one. For a real new market we would add a country normaliser, self-train readers as for France, and label a small sample to set the threshold.

NUMBERS

- Default route: 158 features, single fit, 0.82 / 0.4 (Doc 4)

Q163. How do you handle chains: the same brand at several addresses? ★★ LIKELY

Why they ask: Chains are the classic hard case for business matching; names agree, only the location differs.

SAY The address decides. Branches share the name, so the readers and the address and number features carry the difference, and the clear-winner rule refuses to link when two branches score close. Branches of a chain at nearby addresses are still one of our known wrong-link types, so we reduce the problem rather than remove it.

IF THEY PUSH Retrieval keeps every business with the same core name (if at most 2,000 share it) plus the best candidates by address score, so every branch can be retrieved; the quick filter keeps same-name branches with some address overlap, and the ranker passes at most four on. Two index features count how many businesses share that exact core name, which tells the model the name is shared. Our India example shows the mechanism: three same-name businesses in Agra, Satara and Byculla score below one in a hundred thousand, and the second-chance search picks the one with the same house number, 24/11.

NUMBERS

- Same-core-name channel: up to 2,000 businesses (Doc 3)

Q164. What if the reference list itself contains duplicates? ★★★

Why they ask: Reference data is rarely clean; they check that duplicates cannot make you split or double-count links.

SAY We don't merge businesses; each one keeps its own row. If a record matches two near-identical businesses equally well, neither can be 0.4 ahead of the other, so we link nothing, which costs a little recall but puts no wrong link on either. Our documentation doesn't report a count of duplicate businesses, so I can't say how often this happens.

IF THEY PUSH The candidate file still lists the record under both. The same-core-name channel retrieves such businesses together (unless more than 2,000 share the name), and identical text gives them near-identical ranker scores, so they normally reach the shortlist together. What we would measure: the number of same-country business pairs with identical cleaned name and address, and how often the lead rule abstains between them; a production system might add a deduplication step for the reference list, which the competition format did not ask for.

Q165. How do you handle name variations: abbreviations, legal forms, typos? ★★

Why they ask: Noisy names are the core of the task; they want the mechanism at each stage, not just "we normalise".

SAY In three layers. Cleaning maps spellings to one form, for example "&" to "and" and, in France, "S.A.R.L." to "sarl"; the index has letter-piece and sound lists, so a typo still retrieves the business; and the "what changed" features describe which words were added, dropped or replaced and whether the legal form changed, so the model learns which changes are harmless.

IF THEY PUSH Letter pieces are character 3- and 4-grams of the core name (legal words removed); the sound key removes vowels, including y, and collapses repeated letters, then indexes its 4-grams. France also collapses leet legal forms ("5arl" becomes "sarl"), and in US and Indian names a zero next to a letter is read as the letter o. The word features deliberately ignore legal-form words, so among the "what changed" features a legal-form change shows up only in the legal-form relation, and the readers see it in the raw text, as in our US example (Inc against LLC against PC).

Q166. And address variations? ★★

Why they ask: Addresses carry most of the disambiguating signal; they check you handle formats and numbers, not just strings.

SAY Cleaning brings addresses to one form: state names and street abbreviations are mapped and leading zeros dropped, and in France "15B" becomes "15 bis", street types and their typos are canonicalised, and "st" becomes "saint". Then the number features compare house numbers directly, how far apart they are and how many digits differ, instead of treating the address as plain text.

IF THEY PUSH France also removes "N°" and "numéro" prefixes, unifies bis, ter and quater, and learns region, department and city components from the unlabelled test files. The quick filter keeps any candidate whose address covers at least 95 % of the record's address, weighted by rarity. The France street veto tolerates typos: Damerau distance at most 1, or 2 for words of 8 or more characters.

NUMBERS

- Address coverage ≥ 0.95 keeps a candidate (Doc 3)

Q167. What if a record has no address? ★★

Why they ask: Missing fields are common; they check you neither crash nor silently lower the bar.

SAY Then only the name can decide, and the bar does not drop. Our US example has no address: the shortlist holds four businesses with nearly the same name, and the decision model, using the legal-form and word-change features and the readers, gives 0.965 to Enterprises Inc and at most 0.089 to the others, so it is linked to that one.

IF THEY PUSH The number features become missing values; LightGBM learns at each split which side missing values go, so "no address" is information, not an error. The number relation also has explicit classes for "record has no number" and "business has no number". Retrieval then relies on the name lists, and the address second-chance search cannot run without digits.

NUMBERS

- 0.965022 vs 0.088664, lead 0.876 (Doc D.1)

Q168. What if only the house number or the state is missing? ★★★

Why they ask: Partial addresses are the realistic case; they want to know which parts of the pipeline degrade and how.

SAY Without a house number, a record on a street with several businesses is genuinely ambiguous, and that is one of our known missed-match types, because the clear-winner rule often refuses to link. Without a state, the native-script search falls back to searching all states. In France, the street veto skips pairs it cannot decide, so missing information does not by itself remove a link.

IF THEY PUSH In France the word-substitution rescue accepts the same, an empty or an unnumbered address, and the residual rescue covers "business name plus one generic suffix" with no house number on the same street, provided that business is the only compatible one at the address. The address search needs digits: it takes businesses that share one of the record's three rarest numbers and one of its three rarest address words.

Q169. How do you handle very short names and acronyms? ★★★

Why they ask: Short names break most similarity scores in both directions; they test whether you treat them specially.

SAY Carefully, because they fail in both directions. The quick filter keeps a candidate for a record name of three characters or fewer if enough of its address matches, and in France an acronym rescue links a record whose name is an initials version of the business name at the same address, when that business is the only such one there. The opposite case, an acronym that differs by one initial at another address, is a known wrong-link type, and one part of the France cell veto removes it.

IF THEY PUSH Quick filter: name of at most 3 characters with address coverage at least 0.4. Acronym rescue: one upper-case token of 2-5 letters, best score in [0.4, 0.82), +186 links. Cell veto: an acronym with one different initial away from the business's address, 74 links vetoed; a letter edit of an initialism at the same address, 65.

NUMBERS

- Acronym rescue +186; cell veto parts 74 and 65 (Doc B.3)

Q170. What about brand names, websites or "doing business as" names that share few words with the business? ★★★

Why they ask: A lexical retriever's blind spot; they test whether you know where your recall goes.

SAY That is a known weak spot. Our retrieval is built on words, letter pieces and sounds, so a record that shares almost no words with its business may never be retrieved, and we list it among our main missed-match types. The address second-chance search recovers some of these when the address matches, and a dense vector retrieval channel would be the natural addition.

IF THEY PUSH The address search takes businesses sharing one of the record's three rarest numbers and one of its three rarest address words, then two LightGBM models filter and score them: +192 links in the US and +2,945 in India. We did not benchmark a dense retriever, so we don't claim it would be better; it would be an extra list in the index lookup, not a replacement.

Q171. If a record has only one candidate, does the lead rule mean anything? ★★★

Why they ask: A precise reviewer notices that a margin is undefined without a runner-up and checks what your code does.

SAY No, for those records only the threshold applies: with no runner-up, the lead condition passes automatically. About nine in ten incoming records have exactly one candidate in our final candidate file, so for most records the confidence threshold does the work, and the lead matters where there is real competition.

IF THEY PUSH 9,031,916 records, 90.59 %. The decision model still sees context, such as how many candidates the index originally returned and the shortlist size, so a lone survivor of a crowded list is not treated like a truly unique match. In the code the runner-up of a single-candidate record is set to minus infinity.

NUMBERS

- 90.59 % with exactly one candidate (Doc B.1)

See also, answered elsewhere: "What happens to an incoming record that has no matching business?" see [Q11](#); "And a business that owns no incoming records?" see [Q18](#); "How do you handle transliteration, names written in Indian scripts?" see [Q31](#); "Some businesses have thousands of candidates. Why, and is that a problem?" see [Q55](#).

15. Results and error analysis (9 questions)

Q172. What is your final result? ★ LIKELY

Why they ask: The opener; they want the number and whether it is backed by offline evidence.

SAY Our final file scores 0.9913 F0.5 on the public leaderboard, up from 0.968 for our first scored baseline. On held-out folds, close versions of our US and Indian decision models score between 0.990 and 0.992, so the public number is in line with what we measured offline.

IF THEY PUSH Exactly 0.991300, upload 16 of 18. The file has 5,856,936 links and 12,293,019 candidate pairs (7.10 per business), and it passes the organisers' validator.

NUMBERS

- Public 0.991300; first scored baseline 0.968212 (Doc 5, B.6)

Q173. Where did the improvement from 0.968 to 0.9913 come from? ★★ LIKELY

Why they ask: They want to know which ideas actually moved the score, and whether you can tell a bundle of changes from an ablation.

SAY Two steps did most of it: our first LightGBM decision-model submission and the France adaptation, which together took the public score from about 0.970 to 0.9906. After that each kept upload added less than two ten-thousandths: the consistency check, the US and India refit, the address second-chance searches, more France rules, generic "what changed" inputs, and finally the five-seed models with the French E5 reader.

IF THEY PUSH Upload log: 0.970125 (upload 4) → 0.982561 (6, first decision model) → 0.990556 (7, France adaptation with the first four France rules) → 0.990621 → 0.990676 → 0.990748 → 0.990788 → 0.990881 → 0.990934 → 0.991114 → 0.991300 (the not-adopted France refit, 0.990726, is left out). The steps after upload 7 range from +0.000040 to +0.000186 (derived). Most uploads bundle several changes, so this log is not an ablation.

NUMBERS

- Full upload table (Doc B.6)

Q174. What are your most common wrong links? ★★ LIKELY

Why they ask: Error analysis shows whether you understand your model's failures, which matters more than the score.

SAY Three kinds: names at the business's own address with one generic word replaced or added or the legal form changed; acronyms that differ from the business name by one initial at another address; and branches of a chain at nearby addresses. The "what changed" features, the clear-winner rule and, in France, the vetoes each target one of these.

IF THEY PUSH In France the two word-swap vetoes removed 29,021 of 887,630 accepted links (3.3 %, derived), the street veto 826, the generic-word swap veto 538 and the cell veto 1,322. In our US example the legal-form change from Inc to LLC is scored down to 0.000007, although the ranker had put it second.

NUMBERS

- France rule counts (Doc B.3)

Q175. And your most common missed matches? ★★ LIKELY

Why they ask: The other half of error analysis; they check where recall goes and what you did about it.

SAY True businesses that retrieval never found or that the cap of four cut off; native-script names without a known transliteration; brand or website names with little word overlap; records without a house number on a street with several businesses; and correct candidates rejected because a second one was too close. The second-chance searches, the consistency check and the France rescues each recover part of these.

IF THEY PUSH In France, correct candidates below the conservative 0.82 are a further source. Recall on held-out fold 9: index 99.3694 % (US) and 98.5068 % (India); shortlist at least 98.8748 % and 98.0572 %. Links recovered: native-script +9,759; address +192 and +2,945; France word-substitution rescue +5,272, residual rescue +5,846, generic-word rescue +332, acronym rescue +186.

NUMBERS

- Recall (Doc B.2); links (Doc B.1, B.3)

Q176. Where in the pipeline do you lose the most? ★★★

Why they ask: Tests whether you measured loss per stage instead of only at the end.

SAY Before the decision model, the largest measured loss is the cap of four candidates per record: on held-out fold nine the index finds the true business for about 99.4 % of US records, and the shortlist keeps about 98.9 %. After that we do not have a stage-by-stage breakdown of the final metric, because our held-out scores cover only the main decision.

IF THEY PUSH The fold-9 samples give exact values: US 99.3658 % at the index, 99.3523 % after the quick filter, 98.8935 % in the shortlist; India 98.388 %, 98.3745 % and 97.985 %. So the filter costs about 0.01 percentage points and the cap about 0.4-0.5 (derived). Top 4 without the filter on the full fold gives 98.8814 % and 98.0809 %, almost the same: the cap, not the filter, is the loss.

NUMBERS

- Fold-9 samples (Doc B.2); per-stage differences (derived)

Q177. Is your remaining error mostly precision or recall? ★★★

Why they ask: A good scientist knows which side of the trade-off their errors sit on, or says they do not.

SAY We did not report precision and recall separately, so I can't give a measured split. What we know is the recall ceiling: roughly one percent of US and two percent of Indian owned records are out of reach before the decision model runs. Since our operating points deliberately give up recall for precision, we would expect missed links to be the larger part, but that is a hypothesis we would check by computing per-business precision and recall on a held-out fold.

IF THEY PUSH The ceiling is one minus the shortlist recall: about 1.1 % (US) and 2.0 % (India) of owned fold-9 records, before any threshold (derived from the fold-9 samples). The second-chance searches win back part of it, because their candidates come from outside the shortlist. Under F0.5 the best operating point usually sits where precision exceeds recall, because a wrong link costs more than a miss.

Q178. How much does each refinement contribute? ★★

Why they ask: They want to know if the extra machinery earns its complexity.

SAY In links, the two second-chance searches add about thirteen thousand and the France rules remove a net of about twenty thousand, while the consistency check is close to neutral. We don't have a separate held-out F0.5 for each refinement; the metric evidence is the public step when each was added, and most uploads bundled several changes.

IF THEY PUSH Native-script +9,759; address +192 and +2,945; consistency check US +277 / -284, India +247 / -248, so it changes about 500 links in each direction even though the net is small; France 887,630 accepted → 867,559 after the rules (31,707 vetoed and 11,636 rescued, derived). The cleanest public step is the India address search alone: 0.990676 → 0.990748.

NUMBERS

- 12,896 second-chance links; France net -20,071 (derived from Doc B.1, B.3)

Q179. Walk me through a case where you deliberately chose not to link. ★★

Why they ask: A concrete example tests whether you understand your own precision logic end to end.

SAY The French record TQ Comite SARL in Nantes. The decision model was very confident about TQ Parents SARL at exactly the same address, but the only difference is that the core word "parents" was replaced by a common category word, "comite", and our France rule vetoes exactly that pattern at the same address. So we link nothing; test labels are private, so this shows how we decide, not that the call was right.

IF THEY PUSH Decision score 0.999837 and lead 0.999740, so it passed 0.82 and 0.4. The other candidate, TQ Comite SARL in Pessac, has the identical name but another city: the MiniLM reader liked it (+7.125), the French readers rejected it (-1.92 and -10.81), blend -3.20, decision score 0.000098. The category-swap veto fires and the word-substitution veto also lists the pair; the score is above the word-substitution rescue band [0.4, 0.82), and the later rescues skip records vetoed by those two rules, so nothing re-links it. TQ Parents keeps its link to another incoming record.

NUMBERS

- Worked example (Doc D.3)

Q180. What does 0.9913 mean in practical terms? ★

Why they ask: They check you can translate a metric into what it says about the output.

SAY It is the average, over the businesses the leaderboard scores, of a per-business score that is one when that business's list of linked records is exactly right, so the average shortfall per business is under one percent. Because the score weighs precision more, wrong links cost more than misses, and a business with no true records scores zero after a single wrong link.

IF THEY PUSH The public score alone does not show how that shortfall is spread: it could be a few businesses scoring zero or many scoring slightly below one. Per business, $F0.5 = 1.25 \cdot P \cdot R / (0.25 \cdot P + R)$; the test files hold 1,732,544 businesses.

IF THEY PUSH FURTHER We cannot turn it into an error count, because labels are private and the metric mixes precision and recall per business. The held-out scores exclude the second-chance searches and the France rules, so there is no untouched end-to-end validation of the final file.

NUMBERS

- 1,732,544 test businesses (Doc 2.1); average shortfall $1 - 0.991300 = 0.0087$ (derived)
- Public 0.991300, upload 16 (Doc 5, B.6); first scored baseline 0.968212 (Doc B.6)
- Held-out folds 9 / 8: US 0.991212 / 0.991545; India 0.990046 / 0.990298 (Doc 5)

See also, answered elsewhere: "How does performance differ between countries?" see [Q9](#); "Show me the "what changed" features deciding a real case." see [Q97](#); "Do you have an ablation showing the "what changed" features help?" see [Q102](#); "How good are the readers on their own?" see [Q87](#); "Can your results be reproduced?" see [Q222](#).

16. Limitations and what we would do differently (13 questions)

Q181. What is the biggest limitation of your work? ★★ LIKELY

Why they ask: Senior scientists trust teams that name their own weak points before anyone else does.

SAY We never had a fully untouched end-to-end test of the final file. Our held-out scores cover the main decision models, but they exclude the second-chance searches and the France rules, the same folds also helped choose thresholds, and France has no labels at all. The mitigation is that held-out scores of close versions of our models sit in the same range as the public score.

IF THEY PUSH One more detail from our documentation: the final US and India decision models are refitted on folds 6-9, so the shipped models are not the exact ones scored. Held-out fold 9 / fold 8: US 0.991212 / 0.991545, India 0.990046 / 0.990298.

NUMBERS

- Section 5 disclosure (Doc 5)

Q182. The France rules were designed by looking at the test set. Isn't that overfitting? ★★★ LIKELY

Why they ask: The most adversarial question you can get; they want the admission first, then the safeguards.

SAY Partly, yes, and we disclose it. The rules were designed after analysing the unlabelled French test records, and some parameters were chosen after seeing public scores. What limits the damage is that each rule is code on model scores, raw text and the address, with no test label and no human or AI judgement of any pair as an input; the largest action, the two word-swap vetoes, removes 29,021 of 887,630 accepted French links.

IF THEY PUSH The evidence for the rules is label-free: control groups on the French records, and for the cell veto a check on US labels that name noise and address noise are independent. The analysis included AI-assisted review of sampled pairs, about 1,000 pairs in five samples plus smaller checks. In production we would turn the rule conditions into features and learn them from a small labelled French sample.

NUMBERS

- 29,021 of 887,630 (3.3 %, derived) (Doc B.3)

Q183. Why did you keep the French E5 reader when it failed two of its checks? ★★★ **LIKELY**

Why they ask: They test whether you follow your own rules, and whether you are honest when you did not.

SAY Because the file containing it had our best public score: a leaderboard-driven decision against our own written rule, and we disclose it as such. It passed the US and Indian quality check and agreed 99.92 % with held-out French pseudo-labels, but it was slightly worse than its base model on our generated French test set and failed our rule that the expected gain must be safely positive. It changes French links only, and the same upload also added the five-seed models, so its own effect is not isolated.

IF THEY PUSH Generated French test set: best F0.5 0.910516 against 0.914964, although the blend that includes it beat the blend without it, 0.90843 against 0.898031. Gain rule: point estimate +0.0000302, pessimistic -0.0001806, optimistic +0.0002315, and the rule required the pessimistic estimate to be above zero. It changed French links by -2,228 / +3,396 (5,437 business rows); US, India and candidates were identical.

IF THEY PUSH FURTHER It passed the US/Indian quality tolerance (AUC 0.998544 against 0.998570) and agreement with held-out French pseudo-labels (0.999206 against a 0.98 bar). The gain rule's pessimistic estimate was -0.0001806; it changes French links only (-2,228 / +3,396), and the same upload also added the five-seed models, so its own public effect is not isolated.

NUMBERS

- Written checks (Doc B.4); disclosure (Doc C.3)
- Gain estimate: point +0.0000302, pessimistic -0.0001806, optimistic +0.0002315 (Doc B.4)

Q184. Your pipeline computes statistics on the test files. Isn't that a form of leakage? ★★★

Why they ask: They probe the line between unsupervised use of test inputs, which is common, and using test outcomes, which is not.

SAY It uses the unlabelled test records, never their labels, and our documentation lists every such use. Rarity statistics, the blocking indexes, competition features, a native-script word alignment on very confident test matches and the French geography are computed from test text, and the French readers self-train on French test pairs. That is normal in production, where the reference list and the incoming stream are available at run time, but it makes our evaluation transductive, and label-free counts on the test records also set the French threshold and the French reader blend weight.

IF THEY PUSH Transductive means the system is fitted with access to the unlabelled records it is scored on: it uses no labels, but it can adapt to quirks of this one snapshot. The native-script alignment uses test records whose first-stage top candidate has probability at least 0.9. The generated French test set was built from the French test businesses and used only for evaluation. No external data, lookup or API is used anywhere.

NUMBERS

- Test-data disclosure (Doc C.1); external resources (Doc C.6)

Q185. If a strict reviewer looked at your evidence, what would they say is missing? ★★★

Why they ask: They test whether you know the difference between what you showed and what you assumed.

SAY Four things: an end-to-end held-out score of the final file, a measured French F0.5, clean ablations such as removing all "what changed" features or isolating the five seeds, and a single-record latency measurement. We also did not prepare a feature-importance ranking. Each of these is something we would measure rather than claim.

IF THEY PUSH What we do have: recall at every candidate stage on fold 9, the measured cost of the quick filter, held-out per-business F0.5 of the main decision, written checks for new readers, the public upload log, and byte-for-byte reproduction from the trained models and stored reader scores.

Q186. How much does the cap of four candidates cost you? ★★

Why they ask: A fixed cap is an obvious engineering compromise; they check you measured it and know how you would relax it.

SAY It is our largest measured recall loss: on held-out fold nine about 0.4 to 0.5 percentage points of owners are found by the index but fall outside the shortlist of four. We kept four to protect the expensive readers; with finer blocking, such as state or postcode shards, we would have room to raise the cap where it loses true businesses.

IF THEY PUSH Fold-9 samples: US 99.3523 % after the filter → 98.8935 % in the shortlist; India 98.3745 % → 97.985 %. The second-chance searches add candidates from outside the shortlist: 578,317 native-script pairs not in it, and 1,827 (US) and 25,558 (India) new address candidates. Our documentation reports no results for other cap values, so I won't quote a gain for five or six.

NUMBERS

- Recall (Doc B.2); second-chance candidates (Doc 3)

Q187. Don't your hand-written word lists make the system brittle in a new language? ★★★

Why they ask: They test how much of the system is general and how much is tuned to these three countries.

SAY Partly, yes. The France rules and part of the cleaning depend on small hand-written word lists, for example French street types and generic words like "groupe" or "services", so a new language needs its own lists. The core, the index lookup, the readers and the "what changed" features, is mostly language-independent, which is why the default route keeps it and drops the France rules; learning the rule conditions from a small labelled sample would reduce that dependence further.

IF THEY PUSH Our documentation limits hand-written lists to small word lists for cleaning, decision-model features, the India native-script search and the France rules; there is no external data, geocoder, postal or business table. The street veto, for example, depends on a list of French street types and stop words, and the legal-form relation needs each country's legal-form words.

NUMBERS

- External resources (Doc C.6)

Q188. You say the data-specific features were removed. Are they completely gone? ★★★

Why they ask: A probing follow-up on your feature clean-up; they check for indirect dependencies.

SAY As inputs of the final decision models, yes: we replaced 21 data-specific inputs with 25 generic "what changed" features, and a code check asserts that none of the removed inputs is used. An indirect dependence on those inputs remains, and our documentation discloses it: the training records of the two second-chance models were selected with earlier decision models that still had them.

IF THEY PUSH Two training features of those second-chance models, counts of links already accepted at the business, also come from those earlier models' decisions. The 21 apply to the US and India models; the France and default models exclude 16 from their base list. To remove the dependence we would re-select those training records with the final decision models and retrain the two second-chance models; we have no measurement of whether that changes anything.

NUMBERS

- 21 removed (US/India), 16 (France/default), 25 generic added (Doc 4); disclosure (Doc C.8)

Q189. What did you try that did not work? ★★ LIKELY

Why they ask: Negative results show judgement; they also check that you can let go of ideas.

SAY Four things we tried and dropped: a larger XLM-R reader, refitting the France decision model on more folds, a second French self-training round, and retraining the French MiniLM reader with different pseudo-label filters. The last three each scored slightly lower on the public board than the version we kept.

IF THEY PUSH France refit 0.990726 against 0.990748; second French round 0.991277 against 0.991300; French MiniLM retrain 0.991258 against 0.991300. XLM-R large (MIT, about 560 million parameters) was discarded during development; our documentation records no number for it, so we don't quote one. All three public differences are small enough that they may be noise.

NUMBERS

- Dropped items (Doc B.5, B.6)

Q190. Why didn't the second French self-training round help? ★★★

Why they ask: They test whether you understand iterative self-training and can say "we don't know" cleanly.

SAY We don't know for certain; we did not diagnose it. It scored 0.991277 against 0.991300 for one round, a small difference that may be noise. A plausible reason is that a second round mostly re-learns the first round's own confident guesses, so errors can compound while little new information is added.

IF THEY PUSH This is the confirmation-bias risk of self-training: the more the teacher is the student's own earlier version, the less independent the pseudo-labels are. What we would measure: how many second-round pseudo-labels differ from the first round's, and whether the second-round reader does better or worse on the generated labelled French test set; neither number is in our documentation, so I won't guess.

NUMBERS

- Upload 17: 0.991277 vs 0.991300, not adopted (Doc B.6)

Q191. Your pipeline has many moving parts. Isn't that a maintenance burden? ★★

Why they ask: Production scientists care about operating cost, not only accuracy.

SAY Yes, it is a long pipeline, and that is a real cost. The upside is that each stage has its own measurement, recall at each candidate step, a held-out threshold or a written check, so a regression can be traced to one stage. In a production version we would fold the second-chance searches into one decision model and turn the France rules into learned features, which removes the most bespoke parts.

IF THEY PUSH There is one code path for every country; a country profile only selects the cleaning rules, readers, decision model, threshold, second-chance searches and rules. Training and inference run from one entry script, and the organisers' validator is the last stage. A full retrain takes about 11.5 hours on one GPU.

NUMBERS

- About 11 h 30 min full retrain (CodeREADME 1, 6)

Q192. What would you do differently? ★★ LIKELY

Why they ask: The reflection question; they want specific, prioritised lessons tied to your own limitations.

SAY First, keep an untouched end-to-end validation fold, including the second-chance searches, from day one. Second, add finer blocking keys such as state or postcode, which gives room for a larger cap than four where the cap loses true businesses. Third, label a small French sample, to replace the hand-written rules with learned features and set the French threshold on real labels; and fourth, fold the second-chance searches into one decision model.

IF THEY PUSH Also: a dense retrieval channel for brand and "doing business as" names, clean ablations of the "what changed" features and the seeds, and a single-record latency measurement.

Q193. If this went to production at Amazon, what would you do first? ★★

Why they ask: They check whether you can move from a competition mindset to an operational one.

SAY First, a small labelled sample for every country we serve, starting with France, to measure F0.5 directly and set thresholds on real labels. Then we would time single-record latency, since the work per record is bounded but we have not measured it, and add monitoring of links per business and score distributions per country, to catch drift before it costs precision.

IF THEY PUSH For scale, we would split the index by state or postcode as well as by country and run the shards in parallel; the cost per record is bounded at every stage, so it grows linearly with volume. As a rough linear extrapolation, not a measurement, a billion incoming records would mean about 1.17 billion reader pairs, on the order of 140 GPU-hours spread across shards.

NUMBERS

- Scaling estimate (QA 2; extrapolation, not a measurement)

See also, answered elsewhere: "How confident are you in the French part?" see [Q137](#); "Your documentation says AI coding assistants were used. What did they do?" see [Q229](#).

17. Why not X? Alternatives we did not use (14 questions)

Q194. Why not simply ask a large language model whether two records match? ★★ **LIKELY**

Why they ask: They want to see that small models were a deliberate choice (cost, control, competition rules), not a default.

SAY No LLM runs anywhere in our pipeline. The rules cap every model at 8 billion parameters, and even after blocking we score 11.7 million shortlist pairs, so a generative call per pair would cost far more than our readers, which have about 118 and 278 million parameters. Small fine-tuned readers plus a decision model that compares all candidates of a record were accurate enough: close versions of our decision models score between 0.990 and 0.992 on hidden folds.

IF THEY PUSH Our decision is a competition between up to four candidates with a tuned threshold and a 0.4 lead, so it needs a numeric score for every candidate; an LLM judge could give one through its token probabilities, but at the price of a forward pass through a model tens of times larger than our readers for each of 11.7 million pairs. During development, AI assistants helped write code and review sampled pairs when we designed the France rules, which we disclose, but no human or LLM judgement of any pair is an input to the pipeline. In production, an LLM could earn a place offline, for example pre-screening the uncertain band before human review, judged against a labelled audit sample like any other model.

NUMBERS

- 11,687,317 shortlist pairs (Doc 3)
- MiniLM 117,653,760 and E5 278,043,648 parameters (Doc B.5)
- Held-out US 0.991212 / 0.991545, India 0.990046 / 0.990298, fold 9 / fold 8 (Doc 5)
- "No LLM runs in the pipeline" (Doc C.6)

Q195. Why an inverted index for blocking, and not embedding (vector) search with an approximate-nearest-neighbour index? ★★ **LIKELY**

Why they ask: Dense retrieval is the modern default; they test whether we considered it and know how we would judge it.

SAY We did not benchmark a dense retriever, so we cannot claim it would be worse. Our index keeps the true business for 99.37 percent of US and 98.51 percent of Indian held-out records, it is deterministic, cheap and easy to audit, and its separate lists for name words, address words, letter pieces and sounds already catch typos and abbreviations. A vector index would be a natural extra channel for brand or "doing business as" names that share few words with the business.

IF THEY PUSH In our view, embeddings are weakest exactly where our hard cases are: two names that differ only in the legal form or a single word get almost the same vector, so a dense channel would add recall for paraphrase-like names, not precision. We would add it as an extra candidate list unioned with the index output, train it contrastively with hard negatives mined from our own index, and judge it by the same per-stage recall table on fold 9, by candidates per business and by held-out F0.5, because extra candidates compete for the same four shortlist places.

IF THEY PUSH FURTHER Rare exact tokens such as house numbers and unusual name words are what sparse matching captures best, and its misses can be explained term by term. We would add dense retrieval as a sixth channel and measure the recall gain at each step before paying for the embeddings and an approximate-neighbour index.

NUMBERS

- Index recall, train fold 9: US 99.3694 %, India 98.5068 % (Doc B.2)
- About 130 candidates per business from the index, blocking study of 25 Sep (Doc 3)
- Index recall, fold 9 (Doc B.2)

Q196. Why a cascade of five zones instead of one end-to-end model? ★★

Why they ask: Checks that we understand the cost-accuracy trade-off of a cascade and can justify its extra parts.

SAY One end-to-end model would have to score every pair, 6.7 trillion within countries, or trust a single retrieval score. A cascade lets each stage do the cheapest job it can do well, and we can measure what each stage loses: the index, the quick rule-based filter and the shortlist each have a recall number on a held-out fold. The decision model exists because the final decision is a competition between candidates, which a model that sees one pair at a time cannot see.

IF THEY PUSH The price of a cascade is that errors compound: a true business cut by the cap of four can never be linked by the main decision, and only the two second-chance searches can bring back their two miss types, which is why we report recall per stage. A retriever and reader trained jointly is possible, but it needs negatives drawn from the retriever during training and much more compute; we did not try it.

NUMBERS

- 6,724,569,566,212 same-country pairs (Doc 3)
- Per business: about 130 (index) → 21.7107 (filter) → 6.75 (shortlist, derived) → 7.0954 (candidate file) (Doc 3)

Q197. Why cross-encoders as readers, and not a bi-encoder (two embeddings and a cosine)?

★★

Why they ask: Tests whether we understand the architectural difference, not just the model names.

SAY A bi-encoder squeezes each record into one vector before comparing, so it can miss that exactly one word or the legal form changed; a cross-encoder reads both texts in one sequence, so every token can look at the other side. The cost is that nothing can be precomputed per business, so a cross-encoder runs once per pair; that is affordable only because the shortlist sends on average 1.17 pairs per incoming record to the readers.

IF THEY PUSH Bi-encoders win at retrieval because business vectors can be indexed once; cross-encoders win at the final judgement. In our US example, "Legacy Marketing Enterprises Inc" and "... LLC" differ by one token; the readers and the legal-form features separated them, and the decision model gave the LLC 0.000007 against 0.965022 for the Inc.

NUMBERS

- 1.17 shortlist pairs per incoming record (derived: 11,687,317 / 9,969,589, Doc 3)
- US example: 0.965022 vs 0.000007 (Doc D.1)

Q198. Did you consider Ditto, DeepMatcher or other published entity-matching models? ★★

Why they ask: Tests awareness of the entity-resolution literature and where our design sits in it.

SAY Our readers are in the same family as Ditto: a pretrained transformer fine-tuned as a cross-encoder on serialised record pairs, here "name | address" on each side. We did not benchmark Ditto or DeepMatcher themselves. What we added around the reader goes beyond a pairwise matcher: a per-country decision model that compares all candidates of one record, the "what changed" features, and a tuned threshold with a clear-winner lead.

IF THEY PUSH DeepMatcher predates pretrained transformers and summarises each attribute with recurrent or attention layers; Ditto adds tricks such as marking spans with domain knowledge, summarising long attributes and data augmentation. Our attributes are short, so summarising matters little, and we put domain knowledge into explicit features instead of the input text; we chose multilingual base models because India has native-script names and France has French text.

Q199. Why not bigger readers, such as XLM-R large or a 7-billion-parameter model? ★★

Why they ask: Tests whether model size was a deliberate cost-accuracy choice and whether we over-claim.

SAY We tried a larger reader, XLM-R large with about 560 million parameters, during development and dropped it; our documentation records no comparison number, so I will not claim it was worse, only that it did not enter the final pipeline. Reader cost grows with model size and is paid on every shortlist pair, and the hardest part of our problem is choosing between near-tied candidates, which the decision model handles with features a reader does not see.

IF THEY PUSH Our readers have 117.7 million and 278.0 million parameters, far below the 8-billion cap, under Apache-2.0 and MIT licences. With more compute we would rerun the comparison properly: same shortlist, the same decision-model recipe refitted on the larger reader's scores, per-business F0.5 on held-out fold 9, and GPU-hours per million pairs.

NUMBERS

- XLM-R large, MIT, about 560 M parameters: tried and discarded (Doc B.5)
- 117,653,760 and 278,043,648 parameters (Doc B.5)

Q200. Why separate models per country rather than one global model? ★ LIKELY

Why they ask: A global model is simpler to run; they check that per-country models are justified, not accidental.

SAY The countries differ in format, French addresses and legal words and Indian scripts, and in labels: France has none. The code path is the same for every country; a country profile only selects the cleaning rules, readers, decision model, threshold, second-chance searches and rules. We share where it helps: the India decision model and the France decision model both train on US and Indian rows.

IF THEY PUSH One model with a country feature could share more, but the operating points would still be per country: France, with no labels to tune on, uses a more conservative 0.82, against 0.75 tuned on fold 8 for the US and India. Per-country models also let us retrain or roll back one country without touching the others; the closest thing we have to a global model is the unseen-country route, trained on US and Indian rows with the MiniLM reader only, which is untested on real data.

Q201. Why no graph clustering or transitive closure, as in classic deduplication? ★★★

Why they ask: Entity resolution is often framed as clustering; they test whether we understood the structure of this task.

SAY In our view this task does not need it: each incoming record links to at most one business in a reference list, so there is no record-to-record graph to close. Transitive closure also chains errors: one wrong edge would merge two whole groups. The cross-record logic we do need is per business: the collective features look at all records competing for the same business, and the per-business consistency check adjusts the bar by how many other records that business already accepted.

IF THEY PUSH If we had to deduplicate incoming records among themselves, correlation clustering would be the principled choice, but this task does not score that. If the business list itself contained duplicates, deduplicating it would be a separate step before matching. This is our reasoning; our documentation does not discuss clustering.

Q202. Why not a one-to-one assignment, for example the Hungarian algorithm? ★★

Why they ask: Assignment methods are a standard tool for linking two lists; they check we know when it does not fit.

SAY Because the structure is many-to-one: one business owns several incoming records, on average 3.38 links per business in our final file and up to 11. A one-to-one assignment would force us to drop true records. Taking each record's best candidate, only if it is confident and a clear winner, already guarantees at most one business per record.

IF THEY PUSH A capacitated version, b-matching or min-cost flow, fits the many-to-one shape, but it needs a capacity per business, and how many records a business owns is exactly what we do not know. The business-side coupling we handle more softly, for the US and India: the consistency check lowers the bar to 0.68 when one other record of the business is already accepted and raises it to 0.77 with three or more, with the thresholds tuned on folds 8 and 9.

NUMBERS

- Links per business: mean 3.3805, max 11 (Doc B.1)
- Consistency check: 0.75 / 0.68 / 0.72 / 0.77 for 0 / 1 / 2 / 3+ other accepted records (Doc 4)

Q203. Why not active learning, or simply labelling some French pairs? ★★

Why they ask: Labelling a few hundred pairs is often the cheapest fix; they test whether we thought about it.

SAY We had no French labels, and hand-labelling French test pairs would have meant labelling the test set, so we did not: no human or LLM judgement of any pair is an input to our pipeline. Instead, for France we used self-training on very confident pseudo-labels. In production, labelling a small French sample is the first thing we would do.

IF THEY PUSH We did review sampled unlabelled French pairs, with AI assistance, when designing the France rules, and we disclose that, but those reviews never became labels or pipeline inputs. In production we would choose pairs by uncertainty, for example records whose best candidate lies between 0.4 and the 0.82 threshold or that fail the 0.4 lead, and use the labels to set the French threshold and to replace the hand-written France rules with learned features. We would also label a random slice, because uncertainty sampling alone gives a biased estimate of precision.

Q204. Why IDF-weighted word overlap in the index, and not BM25 or TF-IDF cosine? ★★

Why they ask: Probes whether we know classic retrieval scoring and chose ours for a reason.

SAY Names and addresses are short, so a word almost never repeats inside one record; we treat each record as a set, and the score is the sum of the IDF weights of shared terms with our own length normalisation. BM25 adds term-frequency saturation, which has little to work on in such short text, and a length normalisation, where we use our own. The design choice we leaned on instead is scoring name and address separately and rewarding candidates that match on both.

IF THEY PUSH $IDF = \log(1 + (N + 1) / (df + 1))$; the name score divides the name-word IDF sum by $(1 + \text{tokens})^{0.22}$ and adds a letter-piece term, and the total adds $1.2 \times \min(\text{name score}, \text{address score})$. Terms are taken rarest first with posting-list caps, which works like stop-word removal learned from frequency and bounds the work per record; we did not benchmark BM25, so this is a design argument, not a measured comparison.

NUMBERS

- Channel weights 1 / 1 / 0.45 / 0.22 / 0.18; caps and budgets per channel, e.g. name 14,000 / 22,000 (Doc 3)

Q205. Why not classic Fellegi-Sunter probabilistic record linkage? ★★★

Why they ask: It is the textbook method; they test whether we can relate our model to it.

SAY Our decision model is a supervised version of the same idea: turn a vector of field comparisons into a match probability. In its usual form, Fellegi-Sunter estimates a weight per comparison assuming the fields are independent given the match status; with labels for the US and India we can learn interactions directly, such as "the legal form changed and the address is the same". For unlabelled France we chose self-training plus a decision model trained on US and Indian labels.

IF THEY PUSH An unsupervised Fellegi-Sunter model fitted by EM is the textbook option when labels are missing, as in France; we preferred to transfer a model learned with US and Indian labels because, in our view, the signals that matter interact, which the independence assumption ignores. Boosted trees also handle missing comparisons natively: in our US example the record has no address, so the "what changed" number features are missing and the name and legal form decide.

Q206. Why add second-chance searches instead of just lowering the threshold? ★★

Why they ask: Tests whether we diagnosed the type of error before choosing the fix.

SAY Because these misses are mostly retrieval failures: the right business is not among the candidates at all, so no threshold can link it. A targeted second search fixes the miss types we found on training data, native-script names and records whose name differs while the address matches. It only looks at records that are still unlinked, so it never undoes a link.

IF THEY PUSH Lowering the main threshold would add wrong links among the candidates we already have, and under F0.5 a wrong link costs more than a missed one. The second-chance thresholds were chosen on fold 8 with false positives counted twice: 0.88 for the native-script search, which also needs a 0.3 lead, 0.86 for the US address search, and 0.66 with a 0.1 lead for the Indian address search.

NUMBERS

- +9,759 native-script links (India); +192 (US) and +2,945 (India) address links (Doc B.1)

Q207. Why not machine-translate the Indian and French records into English first? ★★

Why they ask: Translation is an obvious idea for multilingual data; they test whether we know why it fits names badly.

SAY Business names are mostly proper nouns, so they need transliteration, not translation: विजय has to become "vijay", not a translated word. We learned an Indic-to-Latin word list from labelled training matches, and the readers are multilingual and read the raw text, including Devanagari and French accents. In our view a translation model would add cost and a new source of errors at the very step where one changed word decides the match; we did not test one.

IF THEY PUSH The word list keeps a native word's Latin form only if it appears at least 3 times with that word in true matches and clearly beats the runner-up, otherwise we fall back to unidecode; it is a simple co-occurrence alignment learned from labels, not a neural transliterator. In the India example, the second-chance search's word list mapped all four words of the name, each with a single alternative.

See also, answered elsewhere: "Why train the CatBoost ranker with a plain classification loss and not a learning-to-rank loss?" see [Q142](#).

18. Taking it to production at Amazon scale (13 questions)

Q208. Amazon has billions of records. What would the deployed system look like? ★★ LIKELY

Why they ask: The core production question: does the design scale, and where would it break?

SAY The work per record is bounded at every stage, posting-list caps in the index, a fixed number of top candidates per list and at most four pairs per record for the readers, so cost grows linearly and shards run independently. Today we split by country; at billions we would split by country and state or postcode, keep each shard's index in memory and run the readers on a pool of GPUs. As a rough linear extrapolation, not a measurement, a billion incoming records means about 1.17 billion reader pairs, on the order of 140 GPU-hours spread across shards.

IF THEY PUSH Finer shards lose recall when the state is missing or wrong, so we would keep a fallback to the country-wide index, as our native-script search already does: it searches the record's state and falls back to all states when the state is unknown or has no match. The posting-list caps are absolute numbers: a word whose list is longer than the cap is skipped, and in much larger shards more words cross that line, so we would re-measure recall per stage on a labelled sample before trusting the extrapolation.

NUMBERS

- Reader scoring of the shortlist, two readers per pair: about 1.4 h on one 16 GB GPU (Evidence: clean_run/RESULT.md)
- About 140 GPU-hours per billion records: extrapolation (QA 2)

Q209. New records and new businesses arrive every day. Can this run incrementally? ★★★

Why they ask: Tests whether we know which parts of the pipeline depend on global statistics.

SAY Most of the per-record path can: clean the text, look up the shard index, rank, read at most four pairs, score and decide. What needs care are the parts computed over the whole data: the IDF weights and the index, the per-business collective features and consistency check, and the learned word lists. We would run the per-record path as a stream and recompute the global parts in periodic batches, re-deciding only the businesses they affect.

IF THEY PUSH A new business must go into the posting lists, and it can change the answer for earlier records that had no good candidate, so we would re-query recently unlinked records against new businesses; it can also create a near tie for a record that is already linked, so linked records that retrieve the new business would be re-decided too. In our view IDF drifts slowly, so a periodic rebuild is enough, and because folds are a hash of the business id, new businesses fall into training folds automatically; we have not built or measured an incremental version.

Q210. What is the latency for a single incoming record? ★★ LIKELY

Why they ask: Online matching needs a latency budget; they check whether we know ours and are honest if we do not.

SAY We have not measured single-record latency; our runs were batch jobs. The path per record is short: one index lookup in its shard, the ranker over a few filtered candidates, on average 1.17 reader pairs, and a handful of small tree-model calls per pair, so streaming is plausible in principle. The second-chance searches only run for records left unlinked and could run asynchronously.

IF THEY PUSH In batch, the index lookup with the filter and ranker took about 1.3 hours and reader scoring about 1.4 hours for 9.97 million records on one machine. For an online service we would measure p50 and p99 per stage; the reader is the stage to watch, because a cross-encoder pass cannot be precomputed per business.

IF THEY PUSH FURTHER For streaming we would keep the index and the business-side statistics in memory and refresh the rarity weights periodically rather than per record; the collective features and the per-business consistency check would need each business's current set of competing and accepted records. Those are design ideas, not things we built or timed.

NUMBERS

- After the filter: 3.111 (US) / 3.878 (India) candidates per record in the all-fold samples (Doc B.2)
- Index + filter + ranker about 1.3 h; reader scoring about 1.4 h (Evidence: clean_run/RESULT.md)
- 1.17 shortlisted pairs per record (derived from Doc 3)

Q211. How would you monitor this in production and detect drift without labels? ★★ LIKELY

Why they ask: Models decay silently; they want concrete signals, not "we would monitor it".

SAY We would track label-free health numbers per country against the baselines of our final run: 0.5875 links per incoming record, 5.8 percent of businesses empty, 1.23 candidates per record, the share of records whose best candidate fails the threshold or the lead, and how often each France rule fires. A sudden move in these signals that the input or a model has changed. A small labelled audit sample each period then gives the real precision.

IF THEY PUSH We already used this kind of statistic to set the French threshold without labels: French link volume per business, compared with a fold-8 threshold sweep on US and Indian data. For drift in the text itself we would watch the share of native-script names, of records without an address, and of words missing from the learned word list.

NUMBERS

- 0.5875 links per incoming record; 100,087 empty rows (5.8 %, derived); 1.2331 candidates per record; 90.59 % with exactly one candidate (Doc B.1)
- France: the two word-swap vetoes removed 29,021 of 887,630 accepted links (Doc B.3)

Q212. Where would human reviewers fit in? ★★

Why they ask: Real entity-resolution systems mix automation with review; they test whether we know where review adds most value.

SAY Between "link" and "no link". Records whose best candidate is fairly likely but not confident, or confident but not a clear winner, would go to a review queue instead of being dropped. Two of our France rescues already work in such a band: they consider only records whose best score is between 0.4 and 0.82, so that band is a natural first queue.

IF THEY PUSH We would size the queue by counting records per band per day and order it by risk: a link that would be the first one for its business matters most, because if that business truly has no records, one wrong link takes its score from 1 to 0. Reviewer answers become labels, exactly the small French sample we lack, and they feed threshold tuning and retraining.

Q213. How would you add a new country or language? ★★ LIKELY

Why they ask: Amazon operates in many marketplaces; they test whether the design generalises beyond three countries.

SAY On day one it takes our default route: the decision model trained on US and Indian rows, the multilingual MiniLM reader and the conservative 0.82 threshold; that route is untested on real data because the test set had no such country. Then we would repeat the France recipe: a country cleaner for its address format and legal words, self-trained readers on very confident pseudo-labels mixed with labelled pairs, and a small labelled sample to set the threshold.

IF THEY PUSH Everything is driven by a country profile, a small JSON file that names the cleaner, readers, decision model, threshold, second-chance searches and rules, so the pipeline itself does not change; the new work is the country's cleaner, its word lists and the training. If the country uses another script, we would learn a transliteration word list from the first labelled matches, as we did for India.

Q214. What does it cost to run, and where would you cut cost first? ★★

Why they ask: At Amazon scale cost decides whether a design ships.

SAY On one consumer GPU, an RTX 4070 Ti SUPER with 16 GB, a full retrain takes about 11.5 hours; at inference the two longest stages are the index with the ranker, about 1.3 hours, and the GPU readers, about 1.4 hours. The cheapest saving is caching: a reader score belongs to a pair, so an incremental run only scores new pairs. Our own submitted run did exactly that: 11,638,504 shortlist pairs reused stored scores and only 48,813 were scored fresh.

IF THEY PUSH Next we would try distilling the two readers into one, lower-precision inference, and skipping the second reader when the first is decisive, each judged by held-out per-business F0.5 against GPU-hours; we have not measured any of these. The tree models are cheap: in the clean run the decision-model scoring stage took about four minutes (derived from the stage timeline).

NUMBERS

- Full retrain about 11 h 30 min; about 70 GB disk (CodeREADME 1)
- 11,638,504 reused vs 48,813 CPU-scored shortlist pairs (CodeREADME 6)
- Decision-model stage about 4 min (derived from Evidence: clean_run/stage_timeline.txt)

Q215. How would you measure precision and recall in production when labels are scarce? ★★★

Why they ask: Tests whether we know the difference between label-free proxies and real accuracy.

SAY With a small labelled audit sample that is never used for tuning: whole businesses sampled by country and score band, with every candidate record of each labelled, so we can compute the same per-business metric. Label-free proxies help between audits, but they measure consistency, not correctness. We learned that directly: our French E5 reader agreed 99.92 percent with held-out French pseudo-labels, yet on a generated labelled French test set it scored slightly below its base model.

IF THEY PUSH Stratifying by score band, then reweighting by band size, estimates precision where it is uncertain without labelling thousands of easy cases; recall also needs a random sample of incoming records, including unlinked ones, because a true record that never became a candidate is invisible in the output. For differences between versions we would use a paired bootstrap over businesses, as our evaluation code does.

NUMBERS

- Agreement 0.999206; best F0.5 0.910516 vs 0.914964 on the generated French set (Doc B.4)

Q216. How often would you retrain, and how would you keep the training layers separate? ★★★

Why they ask: Stacked systems break quietly when one layer is retrained without the others.

SAY We would keep the same layered split: the ranker and readers learn on one part of the businesses and the decision models on another, assigned by a stable hash of the business id, so new businesses fall into a part automatically. The tree models are cheap and could be retrained often; the readers are expensive and would change less often. Any reader update forces a refit of the decision model, because its inputs change.

IF THEY PUSH Our France decision model is an exception we would not repeat in production: it was trained with the MiniLM reader's score in its reader slot but receives a blend of three readers at test time without a refit; the blend keeps the input on the same logit scale, and France's threshold was set separately, without labels. Every new model would pass written acceptance checks before release, like the ones our French E5 reader faced, for example AUC within 0.001 and top-1 accuracy within 0.002 of its base model on labelled US and Indian pairs; and, unlike that reader, which we kept although it failed two of its four checks, a failed check would block the release.

Q217. Different teams want different precision-recall trade-offs. Can you move the operating point? ★

Why they ask: Production users differ; they check the design is configurable and that we know how to choose a point.

SAY Yes, without retraining: the threshold and the lead are settings in each country's profile. We tuned 0.75 for the competition metric, F0.5, on fold 8; a team that wants suggestions for human review would use a lower bar, and fully automatic merging a higher one. The choice should come from a held-out curve of precision and recall per threshold, not from intuition.

IF THEY PUSH The threshold and the lead interact: the lead guards against near ties between candidates, which we expect to be the riskiest links, so for a review queue we would lower the threshold before touching the lead, and check both on a held-out curve. Under F0.5 a wrong link weighs four times as much as a missed link in the formula; a recall-oriented metric such as F2 reverses that weighting and would move the best operating point lower.

Q218. How would you explain a particular link to an auditor or a seller? ★★

Why they ask: Explainability matters when links affect customers or sellers.

SAY Every decision can be traced from stored values: which candidates were found, their scores, the lead over the runner-up, the "what changed" features and any rule that fired. In our US example, "Legacy Marketing Enterprises (Inc)" was linked to the Inc in Windsor at 0.965, while the LLC, whose name differs only in the legal form, scored 0.000007. In France, a link the model scored 0.9998 was vetoed by a readable rule: same address, but the word "parents" was replaced by "comite".

IF THEY PUSH The index and the quick filter are deterministic, so "why was this business not even considered" also has an exact answer, and the candidate file lists every business that the decision model or a second-chance model scored for the record. For the tree model we would add per-decision feature attributions, for example SHAP values, which LightGBM can compute for trees directly; we did not prepare them for this submission.

Q219. Would you keep the hand-written France rules in production? ★★★

Why they ask: Hand rules are a maintenance and overfitting risk; they test whether we see that.

SAY Not as they are. They were designed after analysing the unlabelled French test records, their parameters were chosen after seeing test results, including public scores, and they depend on French word lists, so they are brittle. In production we would turn each rule's condition into a feature, learn it from a small labelled French sample, and keep a rule only if it still helps on held-out data.

IF THEY PUSH Two things make them manageable meanwhile: each rule is code on model scores and raw text, with no test label and no human or LLM judgement as an input, and each rule's firing count is recorded in the run report, for example the two word-swap vetoes removed 29,021 of 887,630 accepted French links, so a change in firing rate is an alert. The largest France risk remains that we have no measured French F0.5.

Q220. What if bad actors create records that copy a real business's name at a nearby address? ★★★

★★★

Why they ask: Production systems face adversarial inputs; they test whether our precision safeguards carry over.

SAY Near-identical candidates are exactly what our decision rule is built for: the "what changed" features describe the one word, number or legal form that differs, and we only link a clear winner, at least 0.4 ahead of the runner-up. When two businesses are too close, the record stays unlinked, which is the cheaper mistake under F0.5.

IF THEY PUSH For the US and India the decision models were also trained with extra hard examples: copies of unlinked training records that sit right next to a candidate business, with small name edits; how many to add was set from label-free counts on the test records, never from labels, and the France model uses no such examples. In production we would add signals a competition does not have, such as account history, and send high-impact near ties to human review.

See also, answered elsewhere: "What if the business list itself contains duplicates?" see [Q164](#).

19. Reproducibility, engineering and rules (12 questions)

Q221. What hardware did you use, and how long does it take? ★ LIKELY

Why they ask: A sanity check on cost and feasibility, and on whether we know our own numbers.

SAY One desktop machine: an NVIDIA RTX 4070 Ti SUPER with 16 GB, a Ryzen 7 9800X3D and 32 GB of RAM, on Windows 11. A full retrain from raw data takes about 11.5 hours of compute and about 70 GB of disk. At inference, the index lookup with the filter and ranker took about 1.3 hours and reader scoring about 1.4 hours for almost 10 million records.

IF THEY PUSH Every stage needs less than 8 GB of RAM except the address second-chance search, about 3.2 GB per worker or 13 GB with four workers, and most stages use at most 6 CPU threads. The only network access is the first stage, which downloads the two pretrained encoders at pinned revisions; everything after it runs offline.

NUMBERS

- 11 h 30 min retrain; 70 GB disk; RAM < 8 GB except address search 13 GB with 4 workers (CodeREADME 1)
- Longest clean-run stages: folds, word list, index and ranker training 1.4 h; MiniLM and E5 reader training 2.1 h; first French scoring + French MiniLM reader fine-tune 2.0 h; test index + filter + ranker 1.3 h; reader scoring 1.4 h (Evidence: clean_run/RESULT.md)

Q222. Can someone reproduce your submitted file exactly? ★★ LIKELY

Why they ask: Reproducibility is a submission requirement and a sign of engineering discipline.

SAY Yes, at two levels. From our trained models and stored reader scores, the inference stages rewrite both submitted files byte for byte; a fresh extraction of the full archive on 29 September reproduced both, with matching SHA-256 hashes. From raw data, a clean run of every stage passed the organisers' validator and reproduced 99.82 percent of the submitted links.

IF THEY PUSH In that clean run 99.07 percent of businesses had an identical list of linked records; France agreed least, 96.72 percent: its shortlist changed most (2,122,285 pairs instead of 2,103,107), and both self-trained French readers were retrained in that run as well. The regenerated file was never uploaded, so its leaderboard score is unknown.

IF THEY PUSH FURTHER Clean run of 27-28 September: identical match list for 99.07 % of businesses, 5,846,344 links in common, about 11.5 hours of compute on one RTX 4070 Ti SUPER 16 GB. A fresh extraction of the full archive on 29 September reproduced both files byte for byte. The stored reader scores are needed because bf16 scores depend slightly on the device and on how pairs are batched and padded. The French E5 reader's weights were deleted after scoring; its logits are shipped, and a flag retrains it for a run from raw data. The regenerated file was never uploaded, so its public score is unknown.

NUMBERS

- 99.82 % of links reproduced (5,846,344 in common); 99.07 % identical match lists (CodeREADME 6; Evidence: clean_run/RESULT.md)
- SHA-256: matching 9183e542..., candidates e7b4c679... (CodeREADME 5)
- Reproduction (CodeREADME 6; Evidence)

Q223. Why is a run from raw data not bit-identical? ★★

Why they ask: Tests whether we understand sources of non-determinism in GPU training.

SAY Two GPU training steps are not bit-deterministic: the CatBoost ranker trained on the GPU, and the readers trained in bf16 mixed precision. A slightly different ranker changes some shortlists, and every later stage follows from them. Even bf16 reader scores depend slightly on the device and on which pairs share a batch, because each batch is padded to its longest pair, which changes the shapes of the computation and so the order of the floating-point sums.

IF THEY PUSH On a GPU, parallel reductions and atomic additions sum in a varying order, and floating-point addition is not associative, so two training runs drift apart; that is why byte-exact reproduction needs the stored reader scores as well as the weights. The CPU-trained parts we re-checked are deterministic: retraining the India native-script and India address second-chance models on the same inputs reproduced their files byte for byte, and the pipeline requires a fixed Python hash seed.

Q224. Which pretrained models did you use, under which licences, and how big are they? ★

LIKELY

Why they ask: Direct check of the competition rules: open licence and at most 8 billion parameters.

SAY Two, both from Hugging Face at pinned revisions: multilingual MiniLM-L12, Apache-2.0, 117.7 million parameters, and multilingual E5-base, MIT, 278.0 million. All four readers are fine-tuned from these two; the ranker, decision models and second-chance models are trained from scratch on the provided data. Our largest model is under 4 percent of the 8-billion-parameter limit.

IF THEY PUSH The MiniLM encoder is also used without fine-tuning to embed single words for the meaning-similarity features. A larger XLM-R reader, MIT and about 560 million parameters, was tried and discarded and is not downloaded or used. The libraries are open source, mostly BSD, MIT and Apache-2.0, plus text-unidecode used under the Artistic License; their licences are listed in our documentation and their pinned versions in the requirements file.

NUMBERS

- 117,653,760 (Apache-2.0) and 278,043,648 (MIT) parameters (Doc B.5)
- $278,043,648 / 8,000,000,000 = 3.5 \%$ (derived)

Q225. Did you use any external data, lookup table or API? ★ LIKELY

Why they ask: External lookup was banned; they want a clear, confident no.

SAY No. No geocoder, postal table, business registry, API or other external data; the only network access downloads the two pretrained encoders. Hand-written lists are limited to small word lists for cleaning, features, the India second-chance search and the France rules, and the Indic-to-Latin word list is learned from labelled training matches.

IF THEY PUSH No LLM runs in the pipeline, and every model, including any model used to produce training data, has at most 8 billion parameters. Our documentation states all of this in its disclosures.

Q226. Did you use the test data in any way? ★★★ LIKELY

Why they ask: The key integrity question; they want a complete disclosure without having to drag it out.

SAY Yes, without labels, and it is all disclosed. The pipeline computes unsupervised statistics on the test files, such as word rarity, the search indexes and competition features and, for France, the geography and a street-type typo table, and it self-trains the two French readers on confident pseudo-labels of French test pairs. The France rules were designed after analysing the unlabelled French test records, including AI-assisted review of sampled pairs, and their parameters were chosen after seeing test results, including public scores.

IF THEY PUSH Label-free counts on the test records also set the French threshold, the French MiniLM reader's blend weight and, for the US and India, how many extra hard training examples to add; for India, a native-script word alignment is computed on test records whose first-stage top candidate has probability at least 0.9. During development, a generated labelled French test set made from the French test businesses was used for evaluation only. We never had test labels, and no human or LLM judgement of a pair is an input.

Q227. Pretrained encoders already contain world knowledge. Isn't that external information? ★★★

Why they ask: An adversarial probe of the "no external data" claim.

SAY They are general-purpose multilingual language models under open licences, and the rules allow such models up to 8 billion parameters. We never query them for facts: after fine-tuning on the provided data they only read a pair and output a score, and the frozen MiniLM encoder only embeds single words for one feature group. And nothing leaves the machine at inference: after the first stage downloads the encoders, everything runs offline.

IF THEY PUSH What a pretrained encoder brings is language, for example that "Enterprises" and "Industries" are related but different words, and a vocabulary that splits Devanagari and French text into word pieces. Like any language model it may have seen some business names during pretraining, but we never ask it about a business: it only scores the two texts in front of it, and we used no lookup of any kind.

Q228. Who did what in your team? ★

Why they ask: They want to know the presenter's own role and that the team understands the work.

SAY We are a team of three, Monika, Hemachand and I, and we built and checked the pipeline together; I am presenting it today. The documentation, code and evidence files are our shared work, and all three of us stand behind them. **TO FILL IN** (before 7 October: one accurate sentence, agreed with Monika and Hemachand, on the split of work. Our documents do not record roles, so do not improvise one on stage.)

IF THEY PUSH We are happy to share a breakdown afterwards; the full method, code and disclosures are in our documentation and repository.

Q229. Did you use AI tools to build this? ★★

Why they ask: They want a straight, factual answer; our documentation already discloses it.

SAY Yes, and our documentation discloses it: AI coding assistants wrote and ran code and analysis scripts during development. For three later France rule changes, the cells of the cell veto, the acronym rescue and the lower bound of the generic-word rescue, they also chose the cells and thresholds and wrote the rule code, which we adopted. No LLM runs in the submitted pipeline.

IF THEY PUSH Their output went through the same checks as the rest of our work: held-out folds where labels exist, label-free tests and public scores for the France rules, written acceptance checks for new readers, and the organisers' validator. We can walk through any part of the code and the reason for each choice.

IF THEY PUSH FURTHER The three changes were the cell-veto cells, the acronym rescue and the lower bound of the generic-word rescue. Every rule is code on model scores and raw text; no human or AI judgement of a pair is an input.

NUMBERS

- Disclosure (Doc C.5); models (Doc B.5)

Q230. Why is the index written in C++, and how is the code organised? ★★

Why they ask: Tests engineering judgement: where performance mattered and how the system is structured.

SAY The index lookup is the only stage that compares every incoming record against large posting lists, so we wrote it in C++17 with OpenMP threads; everything else is Python with LightGBM, CatBoost and PyTorch. One shell entry script runs the training stages and then the inference stages in a fixed order, and a JSON profile per country selects the parts.

IF THEY PUSH The index takes query terms rarest first, skips any term whose posting list is longer than the cap and stops a channel when its budget is used up, which keeps the work per record bounded. It also computes the 76 features per candidate in the same pass; its string similarity is a longest-common-subsequence ratio computed with a bit-parallel algorithm. After a failure the script can be re-run from the failed stage onwards.

Q231. With 18 leaderboard uploads, how did you keep track of what each change did? ★★

Why they ask: Tests experiment discipline and honesty about leaderboard use.

SAY Every upload is logged in our documentation with its time, what changed and its public score, from the first scored baseline at 0.968212 to the final file at 0.991300, including the three changes we tried and did not adopt. Thresholds that could be tuned with labels were set on held-out folds, and new readers had written pass-or-fail checks; the evidence files ship with the code.

IF THEY PUSH The log is also how we disclose leaderboard influence honestly: the US and Indian refit was kept on public evidence, 0.990621 to 0.990676, the France refit was dropped, 0.990726 against 0.990748, and the final file was chosen among the uploads by public score.

Q232. What safety checks protect the output files? ★★

Why they ask: Tests whether the pipeline guards itself against silent errors.

SAY The export asserts that every linked pair is also in the candidate file, and the last stage runs the organisers' validator with ID checks, which passes. The final file has no pair across countries and no incoming record linked to more than one business. Stored French reader scores are installed only if the France shortlist matches the one they were scored on, checked by its SHA-256 hash.

IF THEY PUSH A clean end-to-end run from a fresh folder found four issues, a Windows runtime-library clash, an over-strict assertion, a full disk, and that French-score check stopping as designed, and all fixes are in the package. A code check also asserts that none of the 21 data-specific inputs we removed is used by a final decision model, and the validator log shows 1,732,544 rows in each file.

20. Rapid-fire ML fundamentals a scientist may probe (15 questions)

Q233. Explain gradient boosting in two sentences. Why boosted trees, and not a neural network, for the decision model? ★ **LIKELY**

Why they ask: A basic competence check, and whether we can justify the model family for our data.

SAY Gradient boosting adds decision trees one at a time: each new tree is fitted to the negative gradient of the loss with respect to the current log-odds, which for log loss is simply the label minus the predicted probability, and it is added with a small learning rate. We chose it because our decision model has about two hundred tabular features of mixed scales, often missing; a record with no address, for example, has no "what changed" number features. And the key signals are interactions such as "legal form changed and same address", which trees capture directly.

IF THEY PUSH LightGBM and CatBoost take a second-order step, each leaf value is minus the sum of gradients over the sum of Hessians plus the L2 term, and LightGBM learns a default direction for missing values at each split. Our settings: learning rate 0.05, 127 leaves, at least 200 rows per leaf, feature fraction 0.9, bagging 0.8, L2 1.0, mean of five seeds; a neural network could use the same features, but on tabular data like this boosted trees are usually as accurate with far less tuning, and we did not benchmark one.

Q234. You use both CatBoost and LightGBM. What is the difference, and why both? ★★

Why they ask: Checks that we know the libraries beyond their names.

SAY CatBoost by default grows symmetric trees, the same split across a whole level, so depth 7 gives 128 leaves, fast scoring and some built-in regularisation; we train it on the GPU as the ranker over the 76 index features. LightGBM grows trees leaf by leaf, always splitting the leaf that reduces the loss most, on binned histograms, which suits our larger feature sets on CPU. Both are gradient-boosted trees, and we did not compare swapping them.

IF THEY PUSH The ranker uses at most 1,600 trees, depth 7, learning rate 0.075, L2 5 and early stopping after 180 rounds without improvement; the decision models use 127 leaves and stop after 100 rounds without improvement (the patience is set in the code). Leaf-wise growth can overfit, and the minimum of 200 rows per leaf guards against that.

Q235. What does attention do inside your readers, and why is a cross-encoder expensive? ★★

Why they ask: Tests transformer basics and whether we connect them to cost.

SAY Self-attention lets every token build its representation as a weighted mix of all tokens in the sequence, with weights from query-key similarity, softmax of $Q \cdot K^T$ divided by $\sqrt{d}$, applied to the values. Because a cross-encoder puts the record and the business in one sequence, "Inc" on one side can attend directly to "LLC" on the other, in each of the 12 layers. The price is that nothing can be precomputed per business, so every pair needs a full forward pass, and attention's cost grows with the square of the sequence length; that is why we cap pairs at 128 tokens and read only the shortlist.

IF THEY PUSH At 128 tokens the linear layers still cost more than the attention itself, so the real saving comes from reading fewer pairs. We pool by the masked mean of the last hidden states, ignoring padding, then apply dropout 0.1 and a linear head to one logit: 385 parameters for MiniLM, 384 weights plus a bias, and 769 for E5. Mean pooling matches how both base models were pretrained as sentence encoders (background knowledge, not in our documentation).

Q236. Did you use contrastive learning anywhere? ★★

Why they ask: Contrastive training is central to modern matching; they check we know what it is and where it would fit.

SAY Not in our own training: all four readers are fine-tuned with binary cross-entropy on labelled or pseudo-labelled pairs. Contrastive learning pulls matching pairs together and pushes non-matching ones apart in an embedding space, usually with in-batch negatives and an InfoNCE loss; it is how bi-encoders such as E5 are pretrained (background knowledge), and we start from those pretrained weights. Our negatives come from the ranker instead: the non-owner candidates it ranked highest, taken from every hard record, where the ranker was unsure or wrong, plus a random 25 percent of the other records.

IF THEY PUSH If we added a dense retrieval channel, we would train it contrastively with hard negatives mined from our own index, because random in-batch negatives are far too easy for business names.

Q237. Are your probabilities calibrated? Does it matter? ★★★ LIKELY

Why they ask: A threshold is only meaningful if we understand what the score means; this separates practitioners from tutorial users.

SAY We did not calibrate them, for example with Platt or isotonic scaling, and we did not measure calibration; instead we tuned the threshold directly on the exact per-business F0.5 on a held-out fold. For a threshold alone that is enough: any order-preserving recalibration just moves the best threshold with it and selects the same links. Calibration matters when a threshold has to transfer to data whose scores are distributed differently, which is France's situation.

IF THEY PUSH The 0.4 lead is a difference of probabilities, so unlike the threshold it does change under recalibration; that is one reason our operating points stay tied to a specific model. The France decision model was trained with the MiniLM reader's score in its reader slot but receives a three-reader blend at test time, and France has no labels to tune on, so its threshold, a more conservative 0.82, was set from French link volume per business, compared with a fold-8 threshold sweep on US and Indian data.

Q238. How did you deal with class imbalance? ★★ LIKELY

Why they ask: Matching is extremely imbalanced; they test whether we handled it consciously.

SAY Mostly by blocking: across the 6.7 trillion possible pairs a true pair is extremely rare, but the shortlist keeps at most four pairs per record, 1.17 on average on the test files, with at most one owner, so positives are no longer rare where the models are trained. We use plain, unweighted log loss and handle the cost asymmetry at decision time, with thresholds tuned for F0.5 and false positives counted twice when setting the second-chance thresholds.

IF THEY PUSH For the readers we chose which pairs to train on instead of reweighting: every hard record, where the ranker was unsure or wrong, plus a random 25 percent of the other records, so the training effort goes to informative pairs. Reweighting classes would mostly shift the log-odds by a constant, which a threshold tuned on the metric largely absorbs anyway.

IF THEY PUSH FURTHER Shortlist size in the labelled all-fold samples: 1.114 candidates per record for the US and 1.174 for India. The LightGBM objective is plain binary log loss with no class weights. Reweighting classes would mainly rescale the predicted probabilities and so move the best threshold, which we tune on the exact metric anyway. The French E5 reader's pseudo-labels are capped at 250,000 per class.

NUMBERS

- Shortlist candidates per incoming record, all-fold samples: US 1.114, India 1.174 (Doc B.2)

Q239. Walk me through the F0.5 maths. Why does one wrong link hurt so much? ★ LIKELY

Why they ask: The metric drives every design choice; they check we can do the arithmetic, not just name it.

SAY Per business, $F0.5 = 1.25 \times \text{correct links} / (\text{links we made} + 0.25 \times \text{true links})$, then we average over all businesses. A business with no true records scores 1 only if we link nothing, so one wrong link takes it from 1 to 0. Example: a business with three true records; linking two correct ones scores $1.25 \times 2 / (2 + 0.75) = 0.91$, while linking all three plus one wrong scores $1.25 \times 3 / (4 + 0.75) = 0.79$.

IF THEY PUSH In counts, $F_\beta = (1 + \beta^2) \cdot TP / ((1 + \beta^2) \cdot TP + \beta^2 \cdot FN + FP)$, so with $\beta = 0.5$ a missed link weighs 0.25 against 1 for a wrong link. Macro averaging gives every business equal weight, so a business with no true records, which scores either 1 or 0, counts as much as the busiest one; that is why we link only a confident, clear winner, and we leave 100,087 rows empty.

Q240. What kind of cross-validation did you use, and how does it prevent stacking leakage? ★★

Why they ask: Leakage is the most common reason for over-optimistic results in stacked models.

SAY Grouped by business: ten folds from a hash of the business id, and every record goes with its business, so no business sits on both sides. The layers use different folds: the ranker and readers learn on folds 0 to 5 and the decision models on 6 to 9, so the decision model never sees reader scores on records the readers trained on, where they would be over-confident. To test a change we score fold 9 with fits on folds 6 to 8 that also drop every record touching fold 9, and fold 8 the same way.

IF THEY PUSH The textbook alternative is out-of-fold stacking: train k readers, each scoring the fold it did not see, so every record gets an honest score and both layers use all the data. It costs k reader trainings on the GPU; our layer split gives the same guarantee more cheaply, at the price of each layer seeing less data.

Q241. How did you control overfitting in the decision model? ★★

Why they ask: Tests regularisation knowledge and honesty about where overfitting can still hide.

SAY Through regularisation, early stopping and honest evaluation: at least 200 rows per leaf, 90 percent of features and 80 percent of rows per iteration, L2 of 1.0, learning rate 0.05, and early stopping on fold 8. We also removed 21 data-specific inputs and replaced them with 25 generic "what changed" features, which should transfer better to new data. On held-out folds 8 and 9, close versions of our decision models score between 0.990 and 0.992.

IF THEY PUSH In our view the larger remaining risk is in our selection, not inside the model: thresholds were chosen on folds 8 and 9, which are also our evaluation folds, the final file was chosen among 18 uploads by public score, and the France rules were designed on the unlabelled test records. Held-out and public scores in the same range limit that risk, but do not remove it.

NUMBERS

- Held-out US 0.991212 / 0.991545, India 0.990046 / 0.990298 (Doc 5)

Q242. 127 leaves is a lot. How do you think about bias and variance here? ★★★

Why they ask: Checks whether we can reason about capacity rather than copy default settings.

SAY Many leaves lower bias: the model can represent interactions like "number changed, same street, legal form unchanged". We pay for that in variance and control it with at least 200 rows per leaf, row and feature subsampling, L2, and by averaging five seeds. Seed averaging reduces the variance that comes from the random subsampling, not the variance from having a finite training set.

IF THEY PUSH Honestly, the seeds' own effect is not isolated: the held-out table uses three of the five seeds, and the five-seed models entered the final file together with the French E5 reader (public 0.991114 to 0.991300), so we claim no separate number for them.

Q243. Early stopping, then a refit with more rounds: is scaling the iterations by the row count justified? ★★★

Why they ask: Probes whether we understand the refit heuristic and its limits.

SAY It is a heuristic, not a law. We fit on folds 6 and 7, find the best number of trees by early stopping on fold 8, then refit on folds 6 to 9 for that number times $\text{rows}(6-9) / \text{rows}(6-7)$, because no fold is left to stop on and, at a fixed learning rate, more data usually supports more trees. For the US and India it helped on the public board, 0.990621 to 0.990676; for France it did not, 0.990726 against 0.990748, so France keeps the 6-7 fit.

IF THEY PUSH The refitted model can no longer be evaluated on folds 8 or 9, which is one reason we have no untouched end-to-end validation; our held-out numbers come from fits that leave the scored fold out, using the same iteration rule.

Q244. Why blend the French readers in logit space rather than averaging probabilities? ★★

Why they ask: Tests understanding of how to combine model outputs.

SAY A logit is log-odds, so a weighted sum of logits adds evidence: a reader that is very sure carries more weight than one that is only fairly sure, whereas in probability space every confident "no" is squashed into the narrow range near zero. In our French example, the business with the identical name in another city got +7.1 from the MiniLM reader, -1.9 from the French MiniLM reader and -10.8 from the French E5 reader; the $0.35 / 0.15 / 0.5$ blend gives -3.2, and the decision model scored it 0.000098.

IF THEY PUSH With weights that sum to one, the blend is a weighted geometric mean of the odds. Averaging probabilities with the same weights would give about 0.37, a much weaker "no" (our own arithmetic on those three logits). The blend also keeps the input on the scale the France decision model was trained on, a single reader logit. The French MiniLM reader's weight was set from label-free counts on the test records, and $0.35 / 0.15 / 0.5$ equals half of the earlier blend, 0.7 MiniLM and 0.3 French MiniLM, plus half the French E5 reader.

NUMBERS

- $0.35 \times 7.1250 + 0.15 \times (-1.9219) + 0.5 \times (-10.8125) = -3.2008$ (Doc D.3)
- Probability average with the same weights ≈ 0.369 , logit about -0.54 (derived from Doc D.3)

Q245. How do the readers handle Devanagari, French accents and long records? ★★

Why they ask: Tests tokenisation basics in a multilingual setting.

SAY They read the raw text, not our cleaned text, through a multilingual subword tokenizer, so Devanagari, accents and capital letters all reach the model as word pieces. Each pair is "name | address" on both sides, truncated to 128 tokens longest-first, which trims the longer side before the shorter one. Business names and addresses are short, so truncation should rarely bite, though we did not measure how often it does.

IF THEY PUSH The E5 readers also use the MiniLM tokenizer; as background knowledge, both base models share the XLM-RoBERTa SentencePiece vocabulary, which is why one tokenizer serves both. The cleaned text, with transliteration and unified abbreviations, is used by the index and the hand-made features instead.

Q246. Your reader recipe: why AdamW, warm-up, bf16 and only one epoch? ★★

Why they ask: Checks that the fine-tuning settings were understood, not copied.

SAY AdamW decouples weight decay from the adaptive step, so the decay is not distorted by Adam's per-parameter scaling, and the new linear head gets five times the encoder's learning rate because it starts from random weights. A 3 percent linear warm-up avoids large early updates that would damage the pretrained weights while Adam's statistics are still noisy. bf16 has the same exponent range as fp32, so it cuts memory and time on a 16 GB GPU without loss scaling, and one epoch over a large labelled set limits both overfitting and cost.

IF THEY PUSH After the warm-up the learning rate decays linearly to zero. Gradient clipping at 1.0 guards against rare large gradients, and length-bucketed batches, pairs of similar length together, cut padding waste. For scale: the French E5 reader trained 7,419 steps of batch 128, about 16 minutes on our GPU (noted in its training script).

Q247. Self-training can reinforce its own mistakes. How did you limit confirmation bias? ★★★

Why they ask: Pseudo-labelling is risky; they test whether we understand its failure mode.

SAY We cannot rule it out, because France has no labels, but for the French E5 reader we limited it in four ways. Only very confident pseudo-labels: positives at least 0.97 with every other candidate of the record below 0.5, so ambiguous records were skipped, and negatives at most 0.03. The teacher was the averaged confidence of two other readers, not the student itself, and we mixed in as many labelled US and Indian pairs, 474,808 of each, to anchor it. We also held out 5 percent of records to check agreement.

IF THEY PUSH The teacher is not fully independent, since one of its two readers, the French MiniLM reader, was itself self-trained on French pseudo-labels. And agreement with pseudo-labels measures consistency, not correctness: the reader agreed 0.999206 but scored slightly below its base model on the generated labelled French test set, 0.910516 against 0.914964. A second self-training round did not help either: it scored 0.991277 on the public board against 0.991300, so we did not adopt it.

21. Curveballs and reflection (11 questions)

Q248. What was the hardest bug you hit? ★★

Why they ask: Reveals real hands-on experience and how we debug.

SAY The nastiest was silent: in a clean run from a fresh folder, the freshly compiled C++ index exited with no output at all. On Windows it had loaded an incompatible runtime library from Git for Windows, found earlier on the system path; the fix was to ship the compiler's own runtime libraries next to the executable. It taught us to test the whole package from scratch, not just our working folder.

IF THEY PUSH The subtler issue was numerical: bf16 reader scores change slightly with batch composition, because each batch is padded to its longest pair, so byte-exact reproduction needs the stored scores, not just the weights. That clean run found four issues in total, and all fixes are in the package.

Q249. What surprised you most? ★★

Why they ask: Shows whether we learned from the data rather than just executing a plan.

SAY How cheap the quick rule-based filter was. With no model at all it cuts the index output from about 130 to 21.7 candidates per business, and on held-out records the true business survived 99.974 percent of the time while F0.5 moved only from 0.990117 to 0.990086. A simple rule, measured carefully, removed most of the work for almost no loss.

IF THEY PUSH A second surprise was the French example: the business with exactly the same name but in another city was liked by our original reader, +7.1, and rejected by both French readers, firmly by the French E5 reader at -10.8; it is one example, not a measurement, but it suggests self-training taught them to weigh the address. And a diagnostic upload showed how much France weighs: emptying every French row dropped the public score from 0.970125 to 0.839.

Q250. With ten times the compute, what would you do? ★★ **LIKELY**

Why they ask: Tests whether we know where our system is limited and can prioritise.

SAY Spend it where we measured the largest loss, the cap of four candidates: on fold 9 the index finds the true business for 99.37 percent of US records, but the shortlist keeps at least 98.87 percent. We would let the readers see more candidates where the ranker is unsure and measure the gain. Second, train the readers out-of-fold, so the decision models can learn on all ten folds instead of four.

IF THEY PUSH Third, rerun the larger-reader comparison properly on the same shortlist and held-out fold, and try a dense retrieval channel judged by the per-stage recall table. Each of these is a plan with a measurement attached, not a result we have.

Q251. What is the most important thing you learned? ★ **LIKELY**

Why they ask: An open question to see whether we can name a real, transferable insight.

SAY Study how true records differ from their business before building features. True records tend to lose information, they drop a digit or a suffix, abbreviate or transliterate, and describing exactly what changed gave the decision model the right language. The second lesson is validation first: measure what each stage loses, tune on the exact metric, and write the acceptance checks before seeing the results.

IF THEY PUSH Describing the change, rather than the word, also made the features generic: when we replaced data-specific inputs with the final generic "what changed" inputs, the public score moved from 0.990881 to 0.990934, so on the public board generality cost nothing. We did not run a clean ablation that removes all "what changed" features.

Q252. Explain this to a business stakeholder in thirty seconds. ★ **LIKELY**

Why they ask: Senior scientists must explain work to non-experts; they test clarity.

SAY Every day, records about businesses arrive from many sources, with typos, abbreviations and different scripts, and each must be attached to the right business in our catalogue, or to none. We first pull up about seven possible matches per business out of trillions of combinations, then a careful model attaches a record only when one match is clearly right. When it is not sure, it leaves the record alone, because a wrong attachment does more damage than a missing one; on the competition's strict score we reached 0.9913.

IF THEY PUSH The real business decision is where to set the bar: lowering it attaches more records, but under this score each extra wrong attachment costs more than a missed one. The bar is a setting per country, so it can follow what the business can tolerate.

Q253. Give me your approach in one sentence. ★ **LIKELY**

Why they ask: Tests whether we can compress the whole system without losing the key idea.

SAY We keep about seven candidates per business, make one careful decision per record, and link nothing whenever we are not sure. That is how 6.7 trillion possible pairs became a score of 0.9913.

IF THEY PUSH The technical version: a recall-first inverted index and a quick rule cut the space, a CatBoost ranker keeps up to four candidates, multilingual cross-encoders read each pair, and a LightGBM decision model per country links only a confident, clear winner, with second-chance searches for missed candidates and self-training for France.

Q254. Which part of your solution are you least confident about? ★★

Why they ask: Tests honesty and whether we know our own weak spots.

SAY France. It has no labels, so we have no measured French F0.5; its readers are self-trained, its rules were designed after analysing the unlabelled French test records, some rule parameters were tuned with public-leaderboard feedback, and we kept the French E5 reader although it failed two of its four written checks. What we can say is that every rule is code on model scores and raw text with no labels as inputs, and that we disclose the rest.

IF THEY PUSH Second on the list is the unseen-country route: it is built like France, with the MiniLM reader only and the 0.82 threshold, but the test set had no such country, so it is untested on real data.

Q255. If you had to remove one component, which would it be? ★★★

Why they ask: Tests whether we know the contribution of each part, and whether we over-claim.

SAY We have no full ablation of every component, so I would decide with a held-out test, not by instinct. By link counts the smallest pieces are the French acronym rescue, 186 links, the US address second-chance search, 192, and the per-business consistency check, which changes about a thousand decisions out of about 8.5 million US and Indian records. But small counts do not mean small value: under per-business F0.5 one wrong link can zero a business.

IF THEY PUSH The two US and Indian pieces were selected on held-out evidence: the consistency check's thresholds on folds 8 and 9, the address search's thresholds on fold 8 with false positives counted twice; the acronym rescue, like every France rule, has no labelled evidence. The part we would least like to remove is the clear-winner rule, because it is what stops us choosing between near-identical candidates.

NUMBERS

- Consistency check US +277 / -284, India +247 / -248, 1,056 changed decisions in total (Doc B.1; total derived)
- US address-rescue links +192 (Doc B.1); France acronym rescue +186 (Doc B.3)
- US + India incoming records 8,534,596 (derived from Doc 2.1)

Q256. Are the differences between your leaderboard uploads statistically meaningful? ★★★

Why they ask: A scientist's reflex: is the signal bigger than the noise?

SAY Some are probably not, and we treat them carefully. The late steps are tiny, the second French round was 0.000023 below our final file and the last gains from 0.991114 to 0.991300 add up to 0.000186, and without labels or a known public subset we cannot put an error bar on them. That is why we present leaderboard-driven choices as a disclosed limitation, not as evidence.

IF THEY PUSH On held-out folds we compare versions with a paired bootstrap over businesses, 2,000 resamples, because both versions are scored on the same businesses and pairing removes the variance from which businesses are easy. The differences come from relatively few businesses: for example, adding the French E5 reader changed 5,437 business rows.

Q257. What limits your score? Where are the remaining errors? ★★

Why they ask: Tests error analysis: do we know what is left to fix?

SAY Two ceilings. Retrieval: the shortlist holds the true business for roughly 99 percent of US and 98 percent of Indian held-out records, and the rest can only come back through the second-chance searches. Decision: the wrong-link types we found are names at the business's address with one generic word replaced or added or the legal form changed, acronyms that differ by one initial at another address, and branches of a chain at nearby addresses.

IF THEY PUSH Known misses are native-script names without a known transliteration, brand or website names with little word overlap, records without a house number on a street with several businesses, and correct candidates rejected because a second one is too close. Each of our components targets one of these, and none removes it completely.

Q258. Looking back, which decision would you reverse? ★★★

Why they ask: Tests mature self-criticism without undermining the work.

SAY Not keeping an untouched end-to-end validation fold from the start. Because folds 8 and 9 also chose our thresholds and the final file was picked by public score, we cannot give one clean number for the whole pipeline. The model decision I would revisit is keeping the French E5 reader against our own written rule: a rule only helps if you follow it, or change it before you see the result.

IF THEY PUSH For balance: it passed the checks on US and Indian quality and on agreement with held-out French pseudo-labels, AUC 0.998544 against 0.998570 on US and Indian pairs and 0.999206 agreement, and it changes French links only, 2,228 removed and 3,396 added. It entered the final file together with the five-seed decision models, so its own effect on the public score is not isolated.

See also, answered elsewhere: "With one more month, what would you change?" see [Q192](#).

Part C - Cheat sheet

Everything to look at on the morning of 7 October: (a) the numbers, (b) an A-Z glossary of every technical term with a link to where it is taught, (c) the internal code names and the plain names to say instead, (d) the honest-limitations answers in the order to say them, and (e) what to say when you do not know.

(a) Numbers table

Every number below is exact and comes from the submitted Documentation (Doc), the code READMEs or the clean-run report, as collected in the shared fact sheet and the talk's prep sheet. "Derived" means plain arithmetic on documented numbers. Say numbers exactly as written; round only where the "say" column rounds.

The headline

Item	Number	Say	Source
Public F0.5 of the submitted file	0.991300, upload 16 of 18	"0.9913"	Doc 5, B.6
First scored baseline	0.968212 (upload 2)	"0.968"	Doc B.6
Held-out F0.5, fold 9 / fold 8 (close versions of our decision models: the India recipe with 3 of its 5 seeds; excludes second-chance searches and France rules)	US 0.991212 / 0.991545; India 0.990046 / 0.990298	"between 0.990 and 0.992 on hidden folds"	Doc 5
France recipe proxy (one fit, applied to US / Indian rows at 0.82, no consistency check; not a French score)	fold 9 0.990508 / 0.989467; fold 8 0.990977 / 0.989850	"a proxy, not a French score"	Doc 5
Measured French F0.5	none (France has no labels)	"we have no measured French F0.5"	Doc 2.1, 5

The data and the metric

Item	US	India	France	All	Source
Businesses (reference list)	663,106	809,986	259,452	1,732,544	Doc 2.1
Incoming records (Source 2 + 3)	3,817,031	4,717,565	1,434,993	9,969,589	Doc 2.1
Incoming records per business	5.756	5.824	5.531	5.7543	Doc 2.1
Labels	train	train	none		Doc 2.1

Metric: F0.5 per business, macro-averaged over all businesses; per business $F0.5 = 1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$, and a business with no true records scores 1 only if its row is empty (Doc 2.1).

The candidate funnel

Step	Pairs	Per business	Per incoming record	Source
Same-country comparison space	6,724,569,566,212			Doc 3
Index lookup (blocking study of 25 Sep 2026, not the final run)		about 130.1	27.541 US / 23.145 India (all-fold samples)	Doc 3, B.2
After the quick rule-based filter	37,614,782	21.7107 (US 15.0352, India 23.6328, France 32.7713)	3.111 US / 3.878 India (samples)	Doc 3, B.2
Shortlist (at most 4 per record)	11,687,317 (US 4,098,587; India 5,485,623; France 2,103,107)	6.75 (derived)	1.17 (derived)	Doc 3
Candidate file (shortlist + second-chance candidates)	12,293,019	7.0954, say 7.10 (US 6.1836, India 7.5180, France 8.1060)	1.2331	Doc 3, B.1
Reduction ratio	0.99999817, so 1.83 pairs per million kept (derived)			Doc 3
Records with exactly one candidate	9,031,916 (90.59 %)			Doc B.1
Candidates per business, median / p90 / p99 / max	7 / 11 / 17 / 3,182			Doc B.1
Businesses with no candidate / more than 100	125 / 125			Doc B.1

Index lookup details (Doc 3): $IDF = \log(1 + (N + 1) / (df + 1))$; five channels (name words, address words, letter 4-grams and 3-grams of the core name, sound key) with weights 1 / 1 / 0.45 / 0.22 / 0.18; caps / budgets name 14,000 / 22,000, address 10,000 / 18,000, 4-grams 2,500 / 15,000, 3-grams 1,000 / 6,000, sounds 800 / 4,000; kept: top 12 by total, top 6 by name, top 6 by address, plus every business with the same core-name word set if at most 2,000 share it; 76 pair features. Quick filter (Doc 3): keep a candidate if any of six conditions holds (best total; total at least 0.6 times the best; best address score; address coverage at least 0.95; name of at most 3 characters with coverage at least 0.4; same core name with coverage at least 0.3).

Recall on held-out fold 9 (Doc B.2)

Stage	US	India
Index lookup	99.3694 %	98.5068 %
Top 4 without the quick filter	98.8814 %	98.0809 %
Shortlist (lower to upper bound)	98.8748 % to 99.3628 %	98.0572 % to 98.4831 %
Fold-9 sample: index → filter → shortlist	99.3658 → 99.3523 → 98.8935 %	98.388 → 98.3745 → 97.985 %
Cost of the quick filter (blocking study, decision model of 25 Sep)	owner kept for 99.974 % of fold-8/9 records; fold-9 F0.5 0.990117 → 0.990086; fold 8 -0.00004	(Doc 3)

The models

Model	Key settings	Source
CatBoost ranker	pointwise binary classifier, Logloss; up to 1,600 trees, depth 7, learning rate 0.075, L2 5, early stopping 180, GPU; 140,000-record pilot (35,000 per country and source), fitted on its fold 0-5 part, early stopping on fold 6; shortlist = top candidate always + ranks 2-4 if probability ≥ 0.001	Doc 3, 4
MiniLM reader	paraphrase-multilingual-MiniLM-L12-v2; 117,653,760 parameters + 385-parameter head; Apache-2.0; learning rate 5e-5; all countries	Doc 4, B.5
E5 reader	multilingual-e5-base; 278,043,648 parameters + 769-parameter head; MIT; learning rate 5e-5; US and India	Doc 4, B.5
French MiniLM reader	the MiniLM reader fine-tuned on very confident French pseudo-labels + 500,000 labelled US/Indian pairs; learning rate 2e-5	Doc 4
French E5 reader	the E5 reader fine-tuned on 474,808 French pseudo-labelled + 474,808 labelled US/Indian pairs; 7,419 steps; learning rate 2e-5	Doc 4
Reader recipe (all four)	raw "name address" for each side, 128 tokens, longest-first truncation, masked mean pooling, dropout 0.1, one logit; binary cross-entropy; AdamW (weight decay 0.01, head learning rate $\times 5$); 3 % warm-up; bf16; gradient clipping 1.0; one epoch; batch 128	Doc 4
LightGBM decision models	learning rate 0.05, 127 leaves, at least 200 rows per leaf, feature fraction 0.9, bagging fraction 0.8, L2 1.0; mean of 5 seeds (US, India, France); features US 217, India 211, France and default 158	Doc 4
"What changed" features	25 generic pair features (8 number, 14 word, 3 meaning) + legal-form relation + number relation; 21 data-specific inputs removed (16 for France / default)	Doc 4
Larger reader tried	XLNet-R large, about 560 million parameters, MIT: tried in development, discarded, no number reported	Doc B.5

Decisions and thresholds (Doc 4)

Decision	Value	How chosen
Threshold US / India	0.75	fold 8, exact per-business macro F0.5

Decision	Value	How chosen
Threshold France / unseen country	0.82	without labels: French link volume per business with a fold-8 sweep on US / India
Lead over the runner-up	0.4, every country	one fixed value (the Documentation does not record how it was picked)
Per-business consistency check (US, India)	0.75 / 0.68 / 0.72 / 0.77 for 0 / 1 / 2 / 3+ other accepted records, same 0.4 lead	folds 8 and 9
Native-script second chance (India)	score $\geq$ 0.88, lead $\geq$ 0.3	fold 8, false positives counted twice
Address second chance US / India	filter cut-off 0.06 / 0.015; threshold 0.86 / 0.66; lead 0.0 / 0.1	fold 8, false positives counted twice; filter keeps 98 % of fold-8 reference links
France reader blend	0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5 (logits)	half the earlier 0.7 / 0.3 blend + half the French E5 reader

Links in the final file (Doc B.1, B.3)

Item	US	India	France	All
Accepted by the decision model	2,250,036	2,726,453	887,630 (before the France rules)	
Consistency check added / removed	+277 / -284	+247 / -248		
Native-script second chance		+9,759 (649,965 pairs for 171,713 records)		
Address second chance	+192 (3,029,652 $\rightarrow$ 1,827 $\rightarrow$ 192)	+2,945 (4,766,484 $\rightarrow$ 25,579 $\rightarrow$ 2,945)		+3,137 (derived)
Links in the matching file	2,250,221	2,739,156	867,559	5,856,936
Empty rows	38,161	46,681	15,245	100,087 (5.8 %, derived)
Links per business (mean)	3.3935	3.3817	3.3438	3.3805 (median 3, max 11)
Links per incoming record	0.5895	0.5806	0.6046	0.5875 (about 41 % unlinked, derived)

France rules computed by code, in order (Doc B.3)

Step	Rule	Effect	French links after
0	decision model, 0.82 / 0.4	887,630 accepted	
1-2	category-swap veto + word-substitution veto	-29,021 together (3.3 % of accepted, derived)	
2	word-substitution rescue	+5,272	
3	residual rescue	+5,846	869,727
4	street veto	-826	868,901
5	generic-word swap veto	-538	868,363
6	generic-word rescue	+332	868,695
7	cell veto (seven address $\times$ name-change combinations)	-1,322	867,373
8	acronym rescue	+186	867,559 (final)

French E5 reader written checks (Doc B.4)

Check	Result	Outcome
US / Indian quality within tolerance (509,260 held-out pairs)	AUC 0.998544 vs 0.998570; top-1 0.993161 vs 0.993004; best F0.5 0.992782 vs 0.992796	pass
Agreement with held-out French pseudo-labels (25,192 pairs)	0.999206 (E5 reader 0.981304), bar 0.98	pass

Check	Result	Outcome
Generated labelled French test set (195,380 pairs)	best F0.5 0.910516 vs 0.914964; blend with it 0.90843 vs 0.898031 without	fail
Expected-gain rule on the final file	French links -2,228 / +3,396 (5,437 rows); pessimistic -0.0001806, point +0.0000302	fail

The upload log in plain words (Doc B.6)

#	Upload	Public score
1-2	early baseline (upload 1 failed on the portal)	0.968212
3-4	early baselines	0.969966; 0.970125
5	diagnostic: upload 4 with every French row emptied	0.839
6	first decision-model submission	0.982561
7	France adaptation with the first four France rules	0.990556
8	+ consistency check, generic-word swap veto	0.990621
9	+ US / India decision-model refit on folds 6-9	0.990676
10	+ India address second chance	0.990748
11	France decision-model refit on folds 6-9 (not adopted)	0.990726
12	+ generic-word rescue; India second-chance model retrained without a city list	0.990788
13	+ cell veto, more generic-word rescue families, US address second chance	0.990881
14	generic decision-model inputs (data-specific inputs removed)	0.990934
15	+ more cell-veto cells, acronym rescue, generic-word rescue lower bound	0.991114
16	five-seed decision models + French E5 reader: final file	0.991300
17	second French self-training round (not adopted)	0.991277
18	French MiniLM reader retrained with different pseudo-label filters (not adopted)	0.991258

Folds, compute and reproduction

Item	Value	Source
Folds	10, per business: hash(business id, seed 20260927) mod 10	Doc 4
Who trains where	ranker and readers folds 0-5 (ranker early stopping on 6); decision models fit 6-7, early stopping 8 (at most 3,000 rounds US / India, 1,500 France), US / India refit on 6-9 for best iteration $\times$ rows(6-9) / rows(6-7); native-script model folds 0-7; address models 6-7	Doc 4
Machine	1 $\times$ RTX 4070 Ti SUPER 16 GB; Ryzen 7 9800X3D; 32 GB RAM; Windows 11	CodeREADME 1
Full retrain from raw data	about 11 h 30 min (say 11.5 hours); about 70 GB disk; every stage under 8 GB RAM except the address second chance (3.2 GB per worker, 13 GB with 4)	CodeREADME 1
Longest inference stages (clean run)	index + filter + ranker about 1.3 h; reader scoring, two readers per pair (all but the French E5 reader), about 1.4 h	clean-run report
Training stages (clean run)	reader training stage about 2.1 h; French E5 reader fine-tune about 16 min (code)	clean-run report
Reproduction	from trained models + stored reader scores: both files byte for byte; clean retrain: 99.82 % of links reproduced (5,846,344), identical match lists for 99.07 % of businesses, validator PASS	CodeREADME 6
Reader scores in the submitted run	11,638,504 reused GPU scores; 48,813 scored on CPU	CodeREADME 6
Weights / full archive	about 2.1 GB / 3,285,958,103 bytes	CodeREADME 6, GitHub README

Item	Value	Source
Scaling estimate (extrapolation, not a measurement)	a billion incoming records $\approx$ 1.17 billion reader pairs $\approx$ 140 GPU-hours across shards	QA 2

More detail, only if asked (FACTS section B)

Item	Value	Source
After the quick filter, per country	US 9,969,935 pairs (15.0352 per business); India 19,142,257 (23.6328); France 8,502,590 (32.7713)	Doc 3
Shortlist per business, unrounded	6.7458 (11,687,317 / 1,732,544)	derived
Candidate file, per country	US 4,100,414 pairs (6.1836 per business); India 6,089,498 (7.5180); France = shortlist, 2,103,107 (8.1060)	Doc 3
Candidates per business, median / p90 / p99 / max, per country	US 6 / 9 / 12 / 296; India 7 / 12 / 17 / 1,125; France 7 / 13 / 24 / 3,182	Doc B.1
Businesses without a candidate / with more than 100	US 17 / 3; India 71 / 32; France 37 / 90; all 125 / 125	Doc B.1
Candidates per incoming record	US 1.0742; India 1.2908; France 1.4656; all 1.2331	Doc B.1
Incoming records with at least one candidate	all but 2 (9,969,587)	Doc B.1
Native-script second chance, candidates	649,965 pairs scored for 171,713 records; 578,317 not already in the shortlist	Doc 3
Address second chance, filter survivors / new candidates	US 1,827 / 1,827; India 25,579 / 25,558	Doc 3
Owned incoming records in the fold-9 recall table	US 457,971; India 303,786 (fold-9 samples 7,411 / 7,444)	Doc B.2
Shortlist size in the all-fold samples	US 1.114, India 1.174 candidates per incoming record	Doc B.2
Businesses with at least one link	US 624,945; India 763,305; France 244,207; all 1,632,457	Doc B.1
French E5 reader checks, details	US / India check: 509,260 pairs of 437,715 records, tolerances AUC 0.001, top-1 and best F0.5 0.002; pseudo-label check: 25,192 pairs (12,616 positive), bar 0.98; generated French test set: 195,380 pairs (69,910 positive), AUC 0.98038 vs 0.979415; gain estimate: point +0.0000302 ($\times$ 0.75: +0.0000226), pessimistic -0.0001806, optimistic +0.0002315	Doc B.4
Clean run from raw data (27-28 September)	12,334,519 candidate pairs (7.12 per business); France shortlist 2,122,285 instead of 2,103,107	CodeREADME 6; clean-run report
Output file hashes (SHA-256)	matching 9183e542...; candidates e7b4c679...	CodeREADME 5

(b) A-Z glossary

279 technical terms, each with a one-line simple definition and links to where it is taught (the first link is the main place). Terms marked "not used by us" are ones a judge may ask about.

A

Term	Simple definition	Taught in
Abstention (reject option)	Choosing to link nothing when no candidate is good enough or clearly best; under F0.5 this is often the cheaper mistake.	A1.7 , A3.15
Abugida (matra, virama)	A script such as Devanagari where each consonant carries a vowel "a"; vowel signs (matras) replace it and the virama removes it.	A1.10
Accent stripping	Removing accents (é becomes e) so that differently typed spellings match.	A1.9
Accumulator	The running score of each business while the index is being looked up for one record.	A1.20
Acronym rescue	France rule that links a record whose name is an initials version of the business name at the same address, if that business is the only such one there.	A4.15
Activation function (ReLU, GELU)	The small non-linear function each neural-network unit applies after its weighted sum.	A0.14
AdamW	The optimiser used for the readers: Adam's per-weight step sizes plus weight decay applied separately from the gradient step.	A0.9 , A2.17
Adaptive overfitting	Overfitting to a test set by making many choices based on its score, such as repeated leaderboard uploads.	A3.25 , A0.10
Address normalisation and parsing	Bringing addresses to one form: street types, state names, house numbers, French "bis" and "ter".	A1.11
Address second-chance search	For unlinked US and Indian records, a second search by rare numbers and rare address words, followed by a filter model and a scorer model.	A4.14 , A2.27
Adversarial validation	Training a classifier to tell training rows from evaluation rows; if it can, the two differ.	A3.3
Ambiguity rejection	Abstaining when the top two candidates are close; our clear-winner lead.	A3.15
Approximate nearest neighbours (ANN)	Fast search for the closest vectors that accepts a small chance of missing the exact best; used in dense retrieval, not by us.	A1.25
Argmax	The candidate with the highest score among a record's candidates.	A0.3
Attention	The transformer mechanism that lets each token build its meaning from a weighted mix of all other tokens.	A2.11
AUC	The chance that a random true pair scores higher than a random false pair; it measures ranking and has no threshold.	A1.6 , A0.4
Autocast (mixed precision)	Running most maths in a cheaper 16-bit format while keeping the weights in 32-bit.	A2.18

B

Term	Simple definition	Taught in
Backpropagation	The method that computes, layer by layer, how much each weight of a network should change to reduce the loss.	A0.9
Bagging	Training several models on random resamples of the data and averaging them, to reduce variance.	A0.13
Base rate	The share of examples that are positive, for example the share of candidate pairs that are true matches.	A0.6

Term	Simple definition	Taught in
Batch (mini-batch)	A group of training examples processed together in one optimisation step; ours are 128 pairs.	A0.9 , A2.17
bf16	A 16-bit number format with the same exponent range as 32-bit floats: faster and smaller, slightly less precise.	A2.18
Bias (in the bias-variance sense)	Error from a model that is too simple to capture the pattern.	A0.10 , A2.3
Bi-encoder	A model that turns each text into a vector separately and compares the vectors; good for retrieval, weaker for fine decisions.	A2.13 , A1.25
Binary cross-entropy (log loss)	The standard training loss for yes/no predictions; it punishes confident wrong probabilities very heavily.	A0.8 , A2.2
Blending (logit space)	A weighted average of several models' logits; for France 0.35 MiniLM + 0.15 French MiniLM + 0.5 French E5.	A2.22 , A3.22
Blocking	Any cheap step that decides which pairs are worth comparing at all, so that we never compare everything with everything.	A1.19
BM25	A classic search-ranking formula with term-frequency saturation and length normalisation; we use a simpler IDF-weighted overlap.	A1.16
Boosting	Building models one after another, each new one correcting the errors of those before it.	A0.13 , A2.5
Bootstrap (paired)	Resampling businesses with replacement many times to get an uncertainty interval for the difference between two models scored on the same businesses.	A3.17
Business	A record of the reference list (Source 1): the thing we link to. The test files hold 1,732,544.	A1.1

C

Term	Simple definition	Taught in
Calibration	Whether a score of 0.8 really means "right 80 % of the time"; we did not calibrate, we tuned thresholds on the exact metric.	A2.26 , A3.14
Candidate	A business that our search proposes for one incoming record; record plus candidate is a pair.	A1.1
Candidate file	The output file listing, for each business, the pairs we considered: the shortlist plus second-chance candidates (7.0954 per business).	A4.16
Canopy clustering	A blocking method that groups records with a cheap distance and two thresholds; not used by us.	A1.24
Capacity	How complex a pattern a model can fit (depth, leaves, trees, epochs); it moves the model along the bias-variance curve.	A0.10
Cascade	A cheap model applied to many items, then a more expensive one to the few survivors; our whole pipeline is one.	A2.27 , A4.1
Catastrophic forgetting	A model losing what it learned before when it is fine-tuned on new data; replay prevents it.	A2.12 , A3.21
CatBoost	A gradient-boosted tree library with symmetric trees and ordered boosting; our ranker uses it.	A2.7
CatBoost ranker	Our first learned model: it orders each record's candidates by the probability that each is the owner, and the shortlist keeps at most four.	A4.7 , A2.9
Category-swap veto	France rule that drops a link when, at the same address, the business name lost a core word and the record gained a category word.	A4.15
Cell veto	France rule that drops accepted links falling in seven specific address × name-change combinations.	A4.15

Term	Simple definition	Taught in
Character n-grams (letter pieces)	Overlapping chunks of n letters; the 3-letter pieces of "dental" are den, ent, nta, tal.	A1.12
Classification	Predicting a category, for example "match" or "not a match".	A0.3
Class imbalance	Far more negative examples than positive ones; blocking removes most of it before our models run.	A0.6
Clear winner (lead, margin)	The best candidate must be at least 0.4 ahead of the runner-up before we link it.	A3.15 , A1.7
Code point	The number of one Unicode character, written U+00C9 and so on.	A1.9
Collective features	Features computed over the other incoming records that compete for the same business (US and India only).	A4.9
Combining mark	A character that attaches to the previous one, such as an accent or an Indic vowel sign.	A1.9
Comparison space	All possible pairs; 6,724,569,566,212 within countries for our test files.	A1.19
Concept shift	What counts as a match for a given input changes between training and use.	A3.24
Confident + clear-winner rule	Link the best candidate only if its score is at least the threshold (0.75 or 0.82) and it leads the runner-up by at least 0.4.	A4.11 , A3.15
Confirmation bias (self-training)	A self-trained model reinforcing its own or its teacher's mistakes.	A2.21 , A3.20
Confusion matrix	The table of true positives, false positives, false negatives and true negatives.	A0.4 , A1.3
Consistency check (per business)	A re-decision for US and Indian records with a bar that depends on how many other records the business already accepted.	A3.16 , A4.13
Contrastive learning	Training embeddings so that matching texts end up close together and non-matching ones far apart; how E5 was pretrained.	A2.16
Core name	The business name without legal-form words (inc, llc, pvt, sarl ...) and without "and" and "the".	A1.8
Cosine similarity	How similar two vectors are, by the angle between them: 1 means the same direction.	A0.16
Cost-sensitive threshold selection	Choosing a threshold where some errors count more; our second-chance thresholds count each false positive twice.	A3.13
Country profile	A small configuration file per country that picks the cleaning rules, readers, decision model, threshold, second-chance searches and rules.	A4.1 , A4.4
Covariate shift	The inputs change between training and use while the task stays the same, as when France's blend replaces one reader.	A3.24 , A2.22
Cross-encoder (reader)	A transformer that reads both texts of a pair together, as one input, and outputs one score.	A2.13 , A2.14
Cross-fitting	Producing a learned component's output for a record with a version that never saw that record, so training looks like test.	A3.7
Cross-validation (k-fold)	Splitting the data into k folds and rotating which fold is held out.	A3.2

D

Term	Simple definition	Taught in
Damerau-Levenshtein distance	Edit distance that also counts swapping two neighbouring characters as one edit.	A1.13
Data augmentation	Creating extra training examples by transforming existing ones.	A3.19
Data leakage	Information from the test side or the label sneaking into training, which makes scores look better than they are.	A3.3 , A0.2

Term	Simple definition	Taught in
Data skew	One key with a huge group (a very common word) overloading one worker; posting-list caps prevent it in our index.	A1.26
Decision model (LightGBM, per country)	The final model: it turns about two hundred features into a probability that the candidate is the owner.	A4.10, A2.8
Decision rule	The rule that turns scores into actions (link or not).	A0.5
Decision tree	A model that asks a series of yes/no questions about the features and ends in a leaf with a prediction.	A0.12, A2.4
Deduplication	Finding duplicates inside one list; not our task, which links to a reference list.	A1.1, A1.2
Default route (unseen country)	What a country without a profile gets: US/India cleaning, the MiniLM reader, a decision model trained on US and Indian rows, threshold 0.82.	A3.24, A4.4
Dense retrieval	Finding candidates by embedding similarity instead of shared words; not used by us.	A1.25
Determinism	Getting exactly the same output when a run is repeated; GPU and bf16 training are not bit-deterministic.	A3.28
Dice coefficient	Twice the shared items divided by the two list lengths added together.	A1.17
Document frequency (df)	The number of records that contain a term; IDF is computed from it.	A1.16
Domain adaptation (unsupervised)	Adapting a model from a labelled domain (US, India) to an unlabelled one (France).	A2.21, A2.12
Domain shift	The data a model meets in use differ from the data it learned from; France is our case.	A3.24
Dropout	Randomly switching off units during training so that a network does not rely on any single one.	A0.11, A2.14
Dynamic programming	Solving a big problem by filling a table of small sub-problems, each computed from neighbours already filled in.	A1.13, A0.15

E

Term	Simple definition	Taught in
E5 reader	Our reader fine-tuned from multilingual-e5-base (278 million parameters, MIT); used for the US and India.	A2.16, A4.8
Early stopping	Stop adding trees or training steps when the score on a separate validation fold stops improving.	A3.9, A0.10
Edit distance (Levenshtein)	The number of single-character insertions, deletions or substitutions that turn one string into another.	A1.13
Embarrassingly parallel	Work that splits into independent pieces that need no communication, such as our per-country runs.	A1.26
Embedding	A list of numbers (a vector) that represents a word or text, so that similar meanings get similar vectors.	A0.16, A2.10
Empirical risk minimisation	Training as minimising the average loss over the training examples plus a regulariser.	A2.1
Empty row	A business with no linked records; it scores 1 if the business truly has no records and 0 otherwise.	A1.5, A1.7
Ensemble	Several models whose predictions are combined.	A0.13
Entity resolution (record linkage)	Deciding which records refer to the same real-world thing.	A1.1
Epoch	One full pass over the training data; our readers train for one epoch.	A0.9
Exact-key blocking	Retrieving every business with the same sorted core-name word set (if at most 2,000 share it).	A1.21, A4.5

Term	Simple definition	Taught in
Expected loss	The average loss of a model on new data from the deployment distribution: what validation tries to estimate.	A3.1
Exponent and mantissa	The parts of a floating-point number: the exponent sets the range, the mantissa sets the precision.	A2.18

F

Term	Simple definition	Taught in
False negative / false positive	A true link we missed / a wrong link we made.	A0.4, A1.3
F-beta, F0.5	A single score combining precision and recall; beta = 0.5 weighs precision more, so a wrong link hurts more than a missed one.	A1.4, A0.4
Feature, feature vector	A clue written as a number; the list of all clues for one pair.	A0.2
Fine-tuning	Further training of a pretrained model on our own task and data.	A0.17, A2.12
Focal loss	A variant of log loss that down-weights examples the model already gets right; not used by us.	A0.6
Fold	One of 10 groups of businesses used to separate training from checking.	A0.7, A3.2
fp16 / fp32	16-bit and 32-bit floating-point formats; fp16 has a small range, fp32 is the precise default.	A2.18
France rules computed by code	Fixed vetoes and rescues on French shortlist pairs, computed from scores, raw text and addresses after the decision model.	A4.15, A3.23
French E5 reader	The E5 reader fine-tuned on French pseudo-labels plus labelled US/Indian pairs; kept although it failed two of four written checks.	A4.18, A3.20
French MiniLM reader	The MiniLM reader fine-tuned on confident French pseudo-labels plus 500,000 labelled US/Indian pairs.	A4.18, A2.21
Funnel (candidate funnel)	The candidate counts at each step: about 130 → 21.71 → 6.75 → 7.10 per business.	A1.23

G

Term	Simple definition	Taught in
Garden of forking paths	Many small analysis choices made after seeing data, which together make results look better than they are.	A3.26, A3.25
Generalisation	Doing well on new data, not just on the training data.	A0.1, A2.3
Generic-word rescue	France rule that links an unlinked record at the business's address whose name differs from the business name only by "groupe" or "developpement" in specific positions, when the model's score is in a low band.	A4.15
Generic-word swap veto	France rule that drops a link when one name word was replaced by "compagnie" or "services" at the same address.	A4.15
Gini impurity	A measure of how mixed the labels in a tree node are; splits try to reduce it.	A0.12, A2.4
GOSS	LightGBM's sampling of rows by gradient size (keep large gradients, sample small ones).	A2.8
Gradient	The direction in which a small change of each parameter increases the loss fastest.	A0.9, A2.5
Gradient boosting (GBDT)	Adding small decision trees one at a time, each fitted to the remaining errors (the negative gradient of the loss).	A2.5, A0.13
Gradient clipping	Capping the size of a gradient step (ours at 1.0) to avoid rare huge updates.	A0.9, A2.17
Gradient descent	Repeatedly stepping parameters against the gradient to reduce the loss.	A0.9

Term	Simple definition	Taught in
Group k-fold	Cross-validation where whole groups (for us, businesses with their records) stay in one fold.	A3.4
Guardrail (non-inferiority) check	A check that nothing important got worse beyond a stated tolerance, such as no forgetting on US and Indian data.	A3.26

H

Term	Simple definition	Taught in
Hard example, hard negative, hard positive	A case the model finds difficult; a non-match that looks like a match; a match that looks different.	A3.18, A2.19
Harmonic mean	An average that is pulled towards the smaller value; F-scores use a weighted harmonic mean of precision and recall.	A0.4, A1.4
Hashing ids into folds	Assigning each business to a fold by a hash of its id, so the split is fixed and reproducible.	A3.5
Hash randomisation	Python changes string hashing between runs unless PYTHONHASHSEED is fixed, which changes set and dictionary order.	A3.28
Head (classification head)	The small final layer that turns the pooled text representation into one logit.	A2.14
Held-out data	Data kept away from a model's training and used only to check it.	A3.1, A0.7
Held-out evaluation protocol	To score fold 9, refit without fold 9 and without any record that touches it; the same for fold 8.	A3.8
Hessian	The second derivative of the loss; boosted trees use it to size each leaf's step.	A2.5
Histogram binning	Grouping feature values into bins so that tree splits are found quickly (LightGBM).	A2.8, A0.12
Histogram subtraction	A LightGBM trick: a child node's histogram is its parent's minus its sibling's, which halves the work.	A2.8
HNSW, FAISS	Popular approximate-nearest-neighbour index and library for dense retrieval; not used by us.	A1.25
Homoglyphs	Characters that look alike but are different code points, such as a Cyrillic and a Latin "a".	A1.9
Hungarian assignment	A one-to-one matching algorithm; it does not fit our many-to-one task.	A1.2
Hyper-parameter	A setting chosen by people rather than learned, such as learning rate or number of leaves.	A0.18
Hypothesis class	All the functions a model type can represent, for example all sums of trees of a given size.	A0.1

I

Term	Simple definition	Taught in
IDF (inverse document frequency)	A weight that is high for rare words and low for common ones; the index uses $\log(1 + (N + 1) / (df + 1))$.	A1.16
IDF-weighted Jaccard	Jaccard overlap where each word counts by its rarity.	A1.17
In-batch negatives, InfoNCE	The contrastive training trick and loss where the other pairs in a batch serve as negatives.	A2.16
Incoming record	A record from Source 2 or Source 3 that must be linked to one business or to none. The test files hold 9,969,589.	A1.1
Indel distance	Edit distance with insertions and deletions only; it equals $/a/ + /b/ - 2 \cdot \text{LCS}$.	A1.13, A1.14
Index lookup (inverted index)	A table from each word, letter piece or sound key to the businesses that contain it, like the index at the back of a book.	A1.20, A4.5
Inference	Using a trained model on new data.	A0.1

Term	Simple definition	Taught in
Intra-class correlation	Rows of one group (a business and its records) resemble each other, which is why folds must be split by group.	A3.4
Invariant assertions	Checks in code of properties that must hold whatever the model does, such as: every link is also a candidate pair, and no record is linked to two businesses.	A3.27
Isotonic regression, Platt scaling	Two standard calibration methods (a monotone step function, a logistic curve); not used by us.	A2.26 , A3.14

J

Term	Simple definition	Taught in
Jaccard similarity	Shared items divided by all distinct items of two sets.	A1.17
Jaro-Winkler similarity	A string similarity that rewards matching prefixes; not used by us.	A1.15

K

Term	Simple definition	Taught in
k-fold cross-validation	See Cross-validation.	A3.2
KL divergence	A measure of how far one probability distribution is from another; the loss of knowledge distillation.	A2.15
Knowledge distillation	Training a small student model to imitate a larger teacher; how MiniLM was made.	A2.15

L

Term	Simple definition	Taught in
L2 regularisation	A penalty that shrinks parameters or leaf values towards zero.	A0.11 , A2.6
Label	The known right answer for a training example; France has none.	A0.2
Label-free control groups	Checking a France rule by comparing a pattern with similar records where the pattern should not matter, without labels.	A3.23
Layer normalisation, residual connection	Transformer building blocks that keep training stable: rescaling each token's vector, and adding a layer's input to its output.	A2.11
Layer-wise fold separation	The ranker and readers train on folds 0-5 and the decision models on 6-9, so stacked inputs are never over-confident.	A3.6 , A2.23
Leaf, leaf-wise growth	The end node of a tree; LightGBM always splits the leaf that reduces the loss most.	A0.12 , A2.8
Learning rate	How big a step each update or each new tree may take.	A0.9
Learning-rate schedule	How the learning rate changes during training; ours is a linear warm-up over the first 3 % of steps, then linear decay.	A0.9 , A2.17
Learning to rank (pointwise, pairwise, listwise)	Ways to train an ordering: score each item alone, compare pairs, or optimise a whole list.	A2.9 , A0.3
Legal form	Words such as Inc, LLC, Pvt Ltd or SARL that state a company's legal type.	A1.8
Legal-form relation, number relation	"What changed" features: how the legal form changed (same, changed, missing on one side) and how unmatched digit runs relate.	A4.9
Length-bucketed batches (dynamic padding)	Putting pairs of similar length in the same batch so little padding is needed.	A2.17
Levenshtein distance	See Edit distance.	A1.13
LightGBM	A fast gradient-boosted tree library with histograms and leaf-wise growth; our decision models and second-chance models use it.	A2.8

Term	Simple definition	Taught in
Logit	A model's raw score before the sigmoid: the log-odds of a match; 0 means 50 %.	A2.2 , A0.5
Log-linear pooling	Averaging models' logits; in probability terms a weighted geometric mean of the odds, which keeps strong evidence strong.	A2.22
Log loss	See Binary cross-entropy.	A0.8 , A2.2
Longest common subsequence (LCS) ratio	$2 \times \text{LCS length} / (\text{length a} + \text{length b})$; the string similarity of our index features.	A1.14
Longest-first truncation	When a pair is longer than 128 tokens, remove tokens from the end of whichever side is currently longer.	A0.15 , A2.14
Loss function	The number training tries to make small; it measures how wrong the predictions are.	A0.8
Loss scaling	Multiplying the loss before back-propagation and unscaling afterwards so that small fp16 gradients do not underflow; bf16 does not need it.	A2.18
LSH with MinHash	Probabilistic blocking that groups records with similar sets of pieces; not used by us.	A1.24

M

Term	Simple definition	Taught in
Macro average	Score each business separately, then take the plain average over businesses.	A1.5
Many-to-one linking	Each incoming record links to at most one business, but a business can own many records.	A1.2
MapReduce	A classic framework for large batch jobs: map emits key-value pairs, a shuffle groups them by key, reduce processes each group.	A1.26
Masked mean pooling	Averaging the encoder's token vectors over the real tokens only (ignoring padding) to get one vector per pair.	A2.14
Maximum-likelihood estimation	Choosing the parameters under which the observed labels are most probable; minimising log loss does exactly this.	A0.8
Meaning-similarity features	Cosines between single words, embedded by the frozen pretrained MiniLM, for words that were replaced, added or dropped.	A2.20 , A4.9
Micro average	Pooling the right and wrong links of all businesses before computing one score; the competition uses the macro average instead.	A0.4
MiniLM reader	Our reader fine-tuned from paraphrase-multilingual-MiniLM-L12-v2 (about 118 million parameters, Apache-2.0); used in every country.	A2.15 , A4.8
Model	A function, learned from examples, that turns an input into an output such as a match score.	A0.1
Monge-Elkan similarity	A word-by-word similarity: each word of one name is matched to its best word in the other, and the scores are averaged.	A1.14
Multi-channel retrieval	Running several lookups (name words, address words, letter pieces, sounds) and taking the union of their top lists.	A1.21
Multi-head attention	Several attention patterns computed in parallel, each looking for a different kind of relation.	A2.11

N

Term	Simple definition	Taught in
Native-script second-chance search	For unlinked Indian records with native-script names: search again by transliterated core name in the same state, then a small model decides.	A4.12
NDCG, MRR, recall@k	Ranking metrics: discounted gain, reciprocal rank of the first right answer, share of owners in the top k.	A2.9

Term	Simple definition	Taught in
Neural network	Layers of weighted sums and simple non-linear functions, trained by gradient descent.	A0.14
Newton step	Setting a boosted tree's leaf value from the sums of gradients and second derivatives: $w = -G / H$.	A2.5
Non-decomposable metric	A metric where the value of one link depends on the other links of the same business, so pair-level summaries can disagree with it.	A3.12, A1.5
Normalisation (text cleaning)	Making text uniform: lower case, punctuation removed, abbreviations unified, numbers cleaned.	A1.8, A4.4

O

Term	Simple definition	Taught in
Oblivious (symmetric) tree	A tree that uses the same split for every node at one depth; CatBoost's default.	A2.7
One-to-one assignment	Matching each item of one list to at most one of the other and vice versa; not our task.	A1.2
Operating point	The chosen threshold (and lead) at which a model is used.	A1.6
Ordered boosting	CatBoost's method of computing each example's gradient from models that have not seen that example.	A2.7
OSA distance (optimal string alignment)	A cheaper, restricted version of the Damerau-Levenshtein distance.	A1.13
Out-of-fold predictions	Scores for each record from a model trained without that record's fold; the textbook way to stack.	A2.23, A3.2
Overfitting / underfitting	Learning the training data's noise / failing to learn the real pattern.	A0.10, A2.3
Overlap coefficient	Items on both lists divided by the length of the shorter list; a short list inside a long one scores 1.	A1.17
Owner	The business an incoming record truly belongs to.	A1.1

P

Term	Simple definition	Taught in
Pair	One incoming record together with one candidate business.	A1.1
Pair completeness, pairs quality, reduction ratio	Blocking measures: share of true pairs kept, share of kept pairs that are true, share of all pairs removed.	A1.19
Parameter	A number inside a model that training learns.	A0.18
Patience (early stopping)	How many rounds without improvement early stopping waits before it stops; 100 rounds in the worked example.	A2.3, A3.9
Per-business F0.5	The competition metric per business: $1.25 \cdot TP / (\text{predicted} + 0.25 \cdot \text{true})$, then averaged over businesses.	A1.5
Percentile interval, Monte Carlo error	Bootstrap interval from the empirical quantiles / the extra randomness from using a finite number of resamples.	A3.17
Phonetic key (Soundex, Metaphone)	Codes that make similar-sounding names equal; we use our own consonant skeleton instead.	A1.18
Posting list, cap, budget	The list of businesses stored under one term; terms with too long a list are skipped, and each channel stops after a total budget.	A1.20
Precision / recall	Of the links we made, the share that are right / of the true links, the share we found.	A0.4, A1.3
Precision-recall curve, ROC curve	Plots of how precision and recall (or true- and false-positive rates) change with the threshold.	A1.6

Term	Simple definition	Taught in
Prediction shift, ordered target statistics	Prediction shift: in plain boosting a training row's gradient comes from a model fitted on that row. CatBoost's ordered boosting and ordered target statistics use only earlier rows to avoid it.	A2.7
Pre-registration (written acceptance checks)	Writing down pass/fail checks before seeing the results; our French E5 reader had four.	A3.26
Pretrained model	A model already trained on large public data, which we then fine-tune.	A0.17, A2.12
Prior shift (label shift)	The share of true matches changes between training and use.	A3.24
Probability score	A number between 0 and 1 that a model gives; a decision still needs a threshold and a rule.	A0.5
Proper scoring rule	A loss, such as log loss, that is minimised only by honest probabilities.	A0.8, A2.2
Pseudo-label	A model's own very confident guess on unlabelled data, used as if it were a label.	A3.20, A2.21
Public leaderboard	The score the competition portal shows after each upload (0.9913 for our final file); choosing among uploads by it can overfit.	A3.25

Q

Term	Simple definition	Taught in
q-gram lemma	One edit changes at most n of a string's n-grams, so strings with few edits still share many n-grams: why letter pieces catch typos.	A1.12
Quick rule-based filter	Fixed rules with no model that keep a candidate if any of six conditions holds; it cuts about 130 to 21.71 per business.	A1.22, A4.6

R

Term	Simple definition	Taught in
Ranking	Predicting an order of items rather than a yes/no answer.	A0.3
RapidFuzz ratio, partial ratio, token-set ratio	A family of 0-100 string similarities; token-set ratio compares word sets and ignores order and repeats.	A1.14
Reader	Our name for a cross-encoder: MiniLM reader, E5 reader, French MiniLM reader, French E5 reader.	A2.14, A4.8
Recall ceiling	The share of true businesses still reachable after blocking; nothing later can link the rest.	A1.23
Reference list	The clean list of businesses (Source 1) that incoming records are linked to.	A1.1
Refit with scaled iterations	Retraining on more folds for the early-stopping tree count $\times$ (new rows / old rows).	A3.10, A2.24
Regression	Predicting a number on a scale.	A0.3
Regularisation	Anything that stops a model from fitting noise: penalties, minimum leaf sizes, subsampling, dropout, early stopping.	A0.11
Reliability diagram, Brier score	Calibration checks: predicted probability against the observed share of true matches / the mean of $(p - y)^2$.	A3.14
Replay	Mixing labelled examples of the original task into fine-tuning on a new domain, to prevent forgetting.	A3.21, A2.12
Residual (in boosting)	What the current model still gets wrong; each new tree fits it.	A2.5
Residual rescue	France rule that links an unlinked record whose name is the business name plus one generic suffix, when that business is the only compatible one.	A4.15
Retrieve-then-rerank	A cheap retriever proposes candidates and an accurate model re-scores them; the pattern of our cascade.	A2.13

S

Term	Simple definition	Taught in
Second-chance searches	Two extra searches for records left unlinked: the native-script search (India) and the address search (US and India).	A4.12 , A4.14
Seed, seed averaging	The number that fixes a model's random choices; averaging models trained with different seeds reduces variance.	A3.11 , A2.25
Selection optimism	The extra score a setting shows on the data used to pick it, because part of its advantage was luck there.	A3.1 , A3.25
Self-supervised learning	Pre-training on tasks whose answers come from the text itself, such as predicting masked words, so no human labels are needed.	A0.17
Self-training	Training a model on its own (or a teacher's) confident predictions on unlabelled data.	A2.21 , A3.20
SentencePiece, sub-word tokens	A tokenizer that splits text into frequent word pieces, so any script and any word can be represented.	A2.10 , A0.15
Sentinel value	A special value (such as -1 or -2) that means "not applicable" so a tree can route it separately.	A0.2
Sharding	Splitting the data and index into parts that run independently; today by country.	A1.26
Shortcut learning	A model keying on a quirk that only some examples have instead of the real signal.	A3.19
Shortlist	The at most 4 candidates per incoming record kept by the CatBoost ranker: the only pairs the readers read.	A4.7
Shrinkage	The boosting learning rate: each tree's contribution is scaled down, so many small steps generalise better.	A0.11 , A2.6
Sigmoid	The function $1 / (1 + e^{(-x)})$ that turns a logit into a probability.	A2.2 , A0.5
Singleton	A business with no true records, or an incoming record with no true business.	A1.1
Softmax	Turning several scores into probabilities that sum to 1.	A0.3
Sorted neighbourhood	A blocking method that sorts records by a key and compares neighbours; not used by us.	A1.24
Sound key (consonant skeleton)	Our phonetic key: vowels and "y" removed, repeated letters collapsed, then 4-letter pieces indexed.	A1.18
Stacking (stacked generalisation)	A second model trained on the outputs of earlier models plus extra features; our decision model.	A2.23 , A0.13
Stratified k-fold	Cross-validation that keeps the class ratio equal in every fold.	A3.2
Street veto	France rule that drops a link when none of the business's street-name words appear in the record's address (typos allowed).	A4.15
Supervised learning	Learning from examples whose right answers are known.	A0.1
Synthetic pairs	Labelled examples created rather than collected; allowed in this competition.	A3.19

T

Term	Simple definition	Taught in
Target encoding	Replacing a category by the mean label of its rows; it leaks badly unless computed out-of-fold.	A3.7
Target leakage, train-test contamination	A feature that depends on the label / the same or near-same items on both sides of a split: two forms of leakage.	A3.3
Teacher / student	In self-training, the model that labels unlabelled data / the model trained on those labels.	A3.20 , A2.21

Term	Simple definition	Taught in
Temperature	A constant that divides scores before a softmax; it sharpens or softens the comparison (E5's contrastive loss, temperature scaling).	A2.16 , A3.14
Temperature scaling	A calibration method that divides neural logits by one fitted constant; not used by us.	A3.14
Tensor cores	GPU units specialised in fast low-precision matrix arithmetic.	A2.18
TF-IDF	A word weight combining term frequency and IDF; we use IDF only, because names are sets of words.	A1.16
Threshold	The minimum score needed to act; 0.75 for the US and India, 0.82 for France.	A0.5 , A1.6
Token, tokenizer	A piece of text (word or word piece) / the program that cuts text into tokens.	A0.15 , A2.10
Top-1 accuracy	The share of incoming records whose highest-scored candidate is the right business.	A2.27
Training, validation and test data	Data to learn from / to choose settings on / to measure the final result on.	A0.7 , A3.1
Transductive	Using the unlabelled test inputs (never their labels) while building the system.	A3.3 , A1.11
Transfer learning	Reusing what a model learned on one task or domain for another.	A2.12
Transformer	The neural network architecture built from attention layers, used by our readers.	A2.11 , A0.14
Transitive closure	Linking A to C because A-B and B-C are linked; not needed for our reference-list task.	A1.2
Transliteration	Writing a word in another script, for example विजय becomes vijay.	A1.10
Triangle inequality	The rule a true distance obeys: going directly between two strings never costs more than going via a third.	A1.13
True positive / true negative	A right link we made / a non-match we correctly left unlinked.	A0.4 , A1.3

U

Term	Simple definition	Taught in
Unicode normalisation (NFC, NFKD)	Standard ways of writing the same character with the same code points.	A1.9
unidecode	A library that maps each Unicode character to the closest plain Latin letters.	A1.8

V

Term	Simple definition	Taught in
Validation set	Data used to choose settings such as the number of trees or a threshold.	A0.7 , A3.1
Variance	Error from a model that is too sensitive to the particular training sample.	A0.10
Viterbi search	Dynamic programming that finds a tokenizer's most probable split of a string without listing every split.	A0.15

W

Term	Simple definition	Taught in
Warm-up (learning rate)	Starting with a tiny learning rate and raising it over the first steps (our first 3 %).	A0.9 , A2.17
Weak supervision	Using rules or other noisy sources instead of hand labels to supervise or correct a model.	A3.23
Weight decay	A regulariser that shrinks weights a little at every step.	A0.11 , A0.9

Term	Simple definition	Taught in
"What changed" features	Features describing exactly how a candidate differs from the record: numbers, words added, dropped or replaced, legal form.	A4.9
Winner's curse	The best of many noisy scores is usually an overestimate.	A3.25
Word-substitution veto and rescue	France rule: drop a link when a business word was replaced by a frequent word at the same address; rescue true-looking name differences in a score band.	A4.15

(c) Internal code names and their plain names

These names appear in our code, logs and Documentation tables. **They never go into a spoken answer.** If a judge uses one (for example after reading the Documentation), answer with the plain name: "Yes, that is our French E5 reader ...". This is the only place in the guide where the code names are printed.

Code name (Documentation or code)	Plain name to say
S1, Source 1 record	business
target, S2 / S3 record	incoming record
matcher, stage 0, inverted-index matcher, v6 matcher	index lookup (inverted index)
channels: name tokens, address tokens, 4-grams, 3-grams, phonetic	name words, address words, letter pieces, sounds
stage 1, deterministic pruning, <code>prune.py</code> , clauses (a)-(f), ALPHA	quick rule-based filter
v6 base model, first-stage ranker, base CatBoost, <code>model.cbm</code> , stage 2	CatBoost ranker
CE-set, <code>ceset_*.npy</code> , top-4, stage-2 pruning	shortlist (up to 4 candidates per record)
<code>candidate_pairs.tsv</code>	candidate file
<code>matching_results.tsv</code>	matching file
CE, cross-encoder	reader
<code>ce_v1</code> , <code>mminilm</code>	MiniLM reader
<code>ce_e5</code> , <code>me5base</code> , <code>e5</code>	E5 reader
<code>ce_v1fr</code>	French MiniLM reader
<code>ce_e5fr</code>	French E5 reader
<code>ce_e5fr2</code> , GXE2	second French self-training round (not adopted)
<code>ce_v1fr_clean</code> , GXE1c	French MiniLM reader retrained with different pseudo-label filters (not adopted)
<code>xlmr1</code>	larger XLM-R reader (discarded)
France blend 0.35 / 0.15 / 0.5	French reader blend
stacker	LightGBM decision model
<code>robust2_S5</code> , <code>S_us_6789_s1..5</code> , <code>S_us</code>	US decision model
<code>robust2_G5</code> , <code>r2g6789_s1..5</code> , <code>robust2_G</code> , R2G	India decision model
<code>fr_G5</code> , <code>frg5_s1..5</code>	France decision model
<code>fr_G</code> , <code>frg_es</code>	default (unseen-country) decision model
<code>robust2_C</code> , COMBO, <code>ce1_mf_ff_nosup</code> , <code>stack_C*</code>	earlier decision models (training dependencies only)
GXE	the final submitted file (upload 16)
stress universe	harder training set with extra hard examples (answer only with the wording of Q115)
normal universe	normal training set
<code>thr</code> , operating point	threshold (confident)
margin	lead over the runner-up (clear winner)
per-S1 ladder, <code>s1_ladder</code> , <code>thr_by_k</code>	per-business consistency check
rescue, <code>india_rescue</code> , <code>p5_rescue.py</code> , <code>rescue_model_A_nc</code>	native-script second-chance search
<code>addr_cascade</code> , <code>us_addr_rescue</code> , <code>india_addr_rescue</code> , <code>p5b_addr_rescue.py</code> , <code>tau1</code> , <code>k0</code> , <code>top1</code>	address second-chance search (retrieval, filter, scorer)

Code name (Documentation or code)	Plain name to say
lexicon.json, lexcounts.parquet	word list learned from training matches
g_num_*	"what changed": number features
g_tok_* (g_tok_n_repl, g_tok_n_add, g_tok_n_del, g_tok_inplace, g_tok_idfjac)	"what changed": words added, dropped or replaced
g_sem_* (g_sem_repl, g_sem_add, g_sem_drop)	"what changed": meaning similarity
legal_rel, f_l_*	"what changed": legal-form features
num_kind, num_delta, num_nt, num_nr, f_n_*	number-relation features
f_m_*, f_a_*, f_c_*	name, address and competition comparisons
nv_*, rb_*, gcoll (4), gsubv (6)	collective features (other records competing for the same business)
base_*, ce_*, e5_*, cem_*, grp_n, ncand, is_s3	ranker and reader scores with their rank, lead and group context; raw candidate count; source flag
RECIPE (15) + GREY (6) = 21 excluded; BAN; check()	data-specific inputs removed from the final decision models, and the code check that asserts it
rules A-H, fr_patterns	France rules computed by code
rule A, pattern_catswap.py	category-swap veto
rule B, pattern_sub.py	word-substitution veto and rescue
rule C, pattern_residual.py	residual (generic-suffix) rescue
rule D, street_veto.py	street veto
rule E, pattern_genswap.py, genswap_veto	generic-word swap veto
rule F, pattern_genrescue.py, genword_rescue	generic-word rescue
rule G, pattern_cellveto.py, cell_veto	cell veto (seven address × name-change combinations)
rule H, pattern_acrrescue.py, acronym_rescue	acronym rescue
G1, G2, H guards; integration rule; e5fr_guards.json	the French E5 reader's written checks
guard H synthetic proxy	generated labelled French test set
sc / es protocols, eval_honest.py	held-out evaluation protocol
t0-t9, i1-i8, run_all.sh	pipeline stages; entry script
validate_submission.py --check-ids	organisers' validator
profile, US.json, India.json, France.json, default.json	country profile
v6 normaliser, france_v2	cleaning rules (US / India; France)

(d) The honest-limitations answers in one place

The pattern is always the same: **say the limitation plainly first, then the mitigation, then stop.** These are disclosed in our Documentation (Section 5 and Appendix C), so saying them is a strength, not a risk. The full answers are in the linked questions.

#	Limitation (say it first)	Mitigation (then this)	Full answer
1	No untouched end-to-end validation of the final file. The held-out scores exclude the second-chance searches and the France rules, folds 8 and 9 also chose thresholds and the consistency check, and the final US / India models were refitted on folds 6-9.	Held-out per-business F0.5 from fits that leave the scored fold out: US 0.991212 / 0.991545, India 0.990046 / 0.990298, the same range as the public 0.9913.	Q181
2	France has no labels, so we have no measured French F0.5.	French readers self-trained with labelled US / Indian replay; a stricter 0.82 threshold; the France recipe on labelled US / Indian rows at 0.82 scores 0.990508 / 0.989467 on fold 9 (a proxy, not a French score).	Q158
3	The France rules were designed after analysing the unlabelled French test records, including AI-assisted review of sampled pairs, and some parameters were chosen after seeing test results, including public scores.	Each rule is code on model scores, raw text and the address; no test label and no human or AI judgement of any pair is an input; the evidence is label-free control groups; the largest action removes 29,021 of 887,630 accepted French links.	Q182
4	We kept the French E5 reader although it failed two of its four written checks (the generated French test set and the expected-gain rule).	It passed US / Indian quality (AUC 0.998544 vs 0.998570) and pseudo-label agreement (0.999206); it changes French links only (-2,228 / +3,396); it was a leaderboard-driven decision and we disclose it.	Q183
5	Public-leaderboard feedback influenced selections: 18 uploads, the final file chosen by public score, the refit kept and the France refit dropped on public scores.	US / India thresholds came from held-out folds; held-out scores sit in the same range; the late public gains were small (0.991114 → 0.991300).	Q153
6	The pipeline computes statistics on the unlabelled test files (word rarity, indexes, French geography, self-training, the French threshold from link volume), so our evaluation is transductive.	No test labels are ever used, every such use is listed in our Documentation, and in production the reference list and the incoming stream are available at run time anyway.	Q184
7	The unseen-country route is untested on real data.	It is conservative by design: the MiniLM reader, a decision model trained on labelled US / Indian rows, threshold 0.82; a new market would get its own cleaner, self-trained readers and a small labelled sample.	Q162
8	No clean ablation of the "what changed" features and no feature-importance ranking.	Closest evidence: generic inputs replacing data-specific ones moved the public score 0.990881 → 0.990934; we would run grouped permutation importance on held-out folds.	Q102
9	Single-record latency was not measured; our runs were batch jobs.	Work per record is bounded at every stage (on average 1.17 reader pairs per record); batch: about 1.3 h index + ranker and 1.4 h readers for 9.97 million records.	Q210
10	(Only if pressed.) The two second-chance models' training records were selected with earlier decision models whose inputs included data-specific features we later removed.	Disclosed in our Documentation; the fix would be to re-select those records with the final decision models and retrain.	Q188

One sentence that covers the first four, for when time is short: "We had no untouched end-to-end validation of the final file, France has no labels and its rules were designed after looking at unlabelled test records, and we kept the French E5 reader although it failed two of its four written checks; all of this is disclosed, and close versions of our models score in the same range on held-out folds."

(e) If you don't know: how to answer gracefully

Senior scientists respect "we did not measure that" far more than a guessed number. A confident, honest non-answer followed by what you *would* measure is a good answer.

Phrases that work.

Situation	Say
The number was never measured	"We did not measure that, so I won't guess. What we would measure is ... on a held-out fold, with the exact per-business F0.5."
The number exists but you do not remember it	"I don't want to misquote it; it is in Section ... of our Documentation, and I'm happy to send it right after."
A good idea you did not try	"That's a good idea, and we didn't try it. We'd judge it the same way as everything else: same shortlist, held-out fold 9, per-business F0.5, and a paired bootstrap over businesses."
The question is unclear	"Just to make sure I answer the right question: do you mean ... or ...?"
It touches a weakness	"Honestly, that is a limitation: ... What limits the damage is ..."
You are asked for a comparison we lack (for example dense retrieval, BM25, XLM-R large, a ranking loss)	"We did not benchmark it, so I can't claim ours is better. Our reason for the choice was ..., and here is how we'd compare them."
Something was decided on public scores	"That one was a leaderboard-driven decision, and our Documentation discloses it."
A question on near-identical look-alike businesses or hard negatives	Use the wording of Q115 exactly, and add nothing.
Team roles	Use the one sentence agreed with Monika and Hemachand before 7 October (Q228).
AI assistance	Answer briefly and factually, as in Q229 .
You are running long	"The short answer is ...; I'm happy to go deeper if useful." Then stop.
You realise you misspoke	"Let me correct that: the exact number is ..." Correcting yourself is fine.

What never to say.

- Anything about near-identical look-alike businesses beyond the agreed wording of [Q115](#), and never raise the topic yourself.
- Internal code names or rule letters (see the [code-name table](#)), or a record id such as "S1-...".
- A number that is not in the [numbers table](#) or in a question's **Numbers** list; never quote toy or illustrative numbers.
- "About 130 per business in the final run" (it is from the blocking study of 25 September; the final run reports 21.7107 after the quick filter).
- "The ranker uses a ranking loss" (it is a pointwise classifier with log loss).
- "The 0.4 lead was never tuned" or "the 0.4 lead was tuned" (say: one fixed value for every country; the threshold was tuned on fold 8).
- "Our held-out scores are an end-to-end test" (they exclude the second-chance searches and the France rules).
- "The French readers are 99.9 % accurate" (0.999206 is agreement with their own teachers' pseudo-labels, not accuracy).
- Any break-even story for the consistency check's bars (say: the four bars, 0.75 / 0.68 / 0.72 / 0.77, were tuned on folds 8 and 9).
- "One LightGBM call per record" (say: a handful of small tree-model calls per pair, five seeds).

Three breaths before any hard question. (1) What is the judge really testing? (2) Which one number proves my answer? (3) Is there a limitation I should say first?